iPod nano 3G FMSS engine (FMSS_M1)

Contents

nano 3G FMSS ("M1"): sequencer, NAND controller and ECC engine 16

Chapters . . . 16

Sources . . . 16

Chapter 1 — Overview and terms 17

1.1 Evidence classes . . . 17

1.2 Numbering and labels . . . 17

1.3 Terms . . . 17

1.4 The three blocks . . . 18

1.5 Named parts of the data path . . . 18

1.5.1 System memory . . . 19

1.5.2 Data buffer 0 and data buffer 1 . . . 19

1.5.3 Parity buffer 0 and parity buffer 1 . . . 19

1.5.4 Slot . . . 20

1.5.5 Chunk . . . 20

1.5.6 ECC area . . . 20

1.5.7 Metadata registers . . . 20

1.5.8 FIFO port (FMC_FIFO) . . . 20

1.5.9 Status register . . . 20

1.5.10 Chip page . . . 21

1.5.11 What the ECC engine touches . . . 21

Chapter 2 — Architecture: execution model, start, stop and fault handling 22

2.1 How the sequencer is started and stopped . . . 22

2.2 Execution model . . . 23

2.3 END and fault . . . 23

2.4 Start word behaviour . . . 24

Chapter 3 — The register window as the ARM CPU sees it 25

3.1 Offsets and names . . . 25

3.2 Memory map: the register window . . . 25

3.3 CPU-only rules . . . 26

3.4 CPU-visible interrupt lines . . . 26

3.5 Starting and finishing a run . . . 27

Chapter 4 — The register window as the sequencer sees it 28

4.1 Window offsets and target widths . . . 28

4.2 Segments and mirrors, sequencer view . . . 28

4.3 Register file r0..r7, the PC and register numbers above 7 . . . 28

4.4 Parameter RAM . . . 29

4.5 System memory and the program . . . 29

4.6 What the sequencer sees differently from the CPU . . . 29

Chapter 5 — Instruction encoding **31**

5.1 Encoding . . . 31

5.2 Operand map . . . 31

5.3 Opcodes . . . 32

5.4 Illegal opcodes . . . 33

5.5 ALU operand rule . . . 34

5.6 Byte order . . . 34

Chapter 6 — Opcode reference **35**

6.1 Conventions and measurement notes . . . 35

6.2 Opcodes . . . 35

00 end . . . 35

01 wr . . . 35

02 st . . . 35

03 ldi . . . 36

04 ld . . . 36

05 movi . . . 36

06 mov . . . 36

07 wait . . . 36

08 nop . . . 37

09 wait2 . . . 37

0a and . . . 38

0b or . . . 38

0c add . . . 38

0d sub . . . 38

0e bnz . . . 39

0f jr . . . 39

10 sig . . . 39

11 sti . . . 39

12 pause . . . 40

13 shl . . . 40

14 shr . . . 40

15 tmwait . . . 40

16 evjmp . . . 41

17 bz . . . 41

18 pld . . . 41

19 pst . . . 41

ff nop2 . . . 42

1a..fe and other values without a meaning . . . 42

Chapter 7 — Events, waits and the timeout **43**

7.1 Wait events . . . 43

7.1.1 Rules for wait event 5 . . . 43

7.1.2 Timeouts . . . 44

7.2 The conditions wait2 waits for . . . 44
7.3 tmwait and evjmp . . . 45
7.4 Timer C14 . . . 45
7.5 Timeout length . . . 45
Chapter 8 — Sequencer control registers 47
8.1 Sequencer control block . . . 47
8.1.1 Start word and timeout (FMSS_CTRL, device) . . . 47
8.1.2 FMSS_PROG (0xc04) . . . 48
8.1.3 CS_STATUS (0xc08) . . . 48
8.1.4 CS_IRQ (0xc0c) . . . 48
8.1.5 FMSS_C10 (0xc10) . . . 48
8.1.6 FMSS_C14 (0xc14): the timeout timer . . . 49
8.1.7 r0 to r7 (0xc18 to 0xc34) . . . 49
8.1.8 FMSS_PC (0xc38): the program counter . . . 49
8.1.9 FMSS_C3C (0xc3c) . . . 49
8.1.10 0xc40 and 0xc44: the held instruction . . . 49
8.1.11 0xc48 . . . 49
8.1.12 FMSS_C4C (0xc4c): the fault vector . . . 49
8.1.13 FMSS_C50 (0xc50): pending events . . . 49
8.1.14 0xc54 . . . 50
8.1.15 FMSS_C58 (0xc58): fault routing . . . 50
8.1.16 FMSS_C5C (0xc5c) . . . 50
Chapter 9 — NAND controller (FMC) 51
9.1 FMC_CTRL0 (0x000): NAND controller configuration (not a sequencer control) . . . 51
9.2 FMCTRL1 (0x004): NAND controller operation trigger (not a sequencer control) . . . 51
9.3 Other NAND controller registers . . . 52
9.4 FMC_STATUS bits . . . 53
9.5 The second NAND controller register set (0x400) . . . 54
9.6 Status latch needs FMC_DATA_SEL = 1 . . . 55
Chapter 10 — ECC engine 56
10.1 How error correction works . . . 56
10.2 The methods at a glance . . . 56
10.2.1 Running one operation . . . 57
10.3 BCH: 8 bits per sector . . . 57
10.3.1 Strength . . . 57
10.3.2 How the code works . . . 57
10.3.3 What an encode and a decode leave . . . 58
10.4 Reed-Solomon: common to all three strengths . . . 58
10.5 Reed-Solomon, 8 parity symbols: 4 bytes per sector . . . 59
10.6 Reed-Solomon, 12 parity symbols: 6 bytes per sector . . . 59
10.7 Reed-Solomon, 16 parity symbols: 8 bytes per sector . . . 59
10.8 What each method corrects . . . 59

10.9 What is stored on the chip . . . 60
10.9.1 BCH (the original layout, device) . . . 60
10.9.2 Reed-Solomon . . . 60
10.10 ECC_MODE bit map . . . 60
10.11 ECC_DONE and ECC_RESULT . . . 61
10.12 Modes with bit 8 clear: no code . . . 61
10.13 Registers . . . 63
10.14 Byte order . . . 64
Chapter 11 — NAND operations 65
11.1 Common behaviour . . . 65
11.1.1 C1: Placement . . . 65
11.1.2 C2: Prologue . . . 65
11.1.3 C3: Control word . . . 65
11.1.4 C4: CE index . . . 65
11.1.5 C5: One command cycle . . . 65
11.1.6 C6: Address cycles . . . 65
11.1.7 C7: Ready procedure (status poll) . . . 65
11.1.8 C8: Data transfer . . . 66
11.1.9 C9: Acknowledge hygiene . . . 66
11.1.10 C10: End state . . . 66
11.1.11 C11: Absent chips . . . 66
11.1.12 C12: One FMCTRL1 operation at a time . . . 66
11.1.13 C13: Prime after a reset (CPU side) . . . 66
11.1.14 How a chip and a page are selected . . . 67
11.1.15 Worked example: chip CE1, block 3000, page 5 . . . 67
11.2 ECC and data path (shared by READ and PROGRAM) . . . 69
11.2.1 R1: Page stream . . . 69
11.2.2 R2: Buffers . . . 69
11.2.3 R3: Decode of one sector (READ) . . . 69
11.2.4 R4: Per-sector verdict . . . 69
11.2.5 R5: Encode of one sector (PROGRAM) . . . 69
11.2.6 R6: Meta layout word . . . 69
11.2.7 R7: DMA addressing . . . 70
11.3 RESET and READ ID (program READID) . . . 70
11.3.1 ID1: Control word . . . 70
11.3.2 ID2: Reset . . . 70
11.3.3 ID3: Settle delay . . . 70
11.3.4 ID4: Per CE . . . 70
11.3.5 ID5: End . . . 70
11.4 Page read (program READ) . . . 70
11.4.1 RD1: Pipeline across chips . . . 70
11.4.2 RD2: Before the data . . . 71

11.4.3 RD3: Order inside a sector . . . 71
11.4.4 RD4: Decode and DMA . . . 71
11.4.5 RD5: Unit boundary (4 KiB pages) . . . 71
11.4.6 RD6: Switching chips . . . 71
11.4.7 RD7: Chunk-ECC flag . . . 71
11.4.8 RD8: Page result . . . 71
11.4.9 RD9: Meta . . . 71
11.4.10 RD10: Erased and uncorrectable pages . . . 72
11.4.11 RD11: End . . . 72
11.5 Page program (program PROGRAM) . . . 72
11.5.1 WR1: First sector staging . . . 72
11.5.2 WR2: Chip free . . . 72
11.5.3 WR3: Meta encode . . . 72
11.5.4 WR4: Sector order . . . 72
11.5.5 WR5: DMA list . . . 72
11.5.6 WR6: Confirm . . . 73
11.5.7 WR7: Status and failure . . . 73
11.5.8 WR8: Result . . . 73
11.5.9 WR9: Variants . . . 73
11.5.10 Device checks of the write sequence . . . 73
11.6 Block erase (program ERASE) . . . 74
11.6.1 ER1: Entries . . . 74
11.6.2 ER2: Final sweep . . . 74
11.6.3 ER3: Result . . . 74
11.6.4 ER4: End . . . 74
11.7 Fault handler and timeouts . . . 74
11.7.1 FL1: Handler . . . 74
11.7.2 FL2: Hung hardware . . . 74
11.7.3 FL3: What the CPU does after a fault . . . 74
Chapter 12 — CPU interface: parameter words, result words and timing word 75
12.1 Parameter words per program . . . 75
12.2 Result words . . . 75
12.3 Timing word . . . 76
Chapter 13 — Experiments: what was done and why 77
13.1 How an experiment runs on the device . . . 77
13.1.1 Soft reset . . . 77
13.1.2 NAND scratch blocks . . . 77
13.1.3 Loading and reading the buffers . . . 77
13.1.4 Steps around each program run . . . 78
13.2 Device runs . . . 78
13.3 Experiments . . . 79
13.3.1 regmap . . . 79

13.3.2 clock . . . 79
13.3.3 alu . . . 79
13.3.4 memory . . . 79
13.3.5 indirect . . . 79
13.3.6 control . . . 79
13.3.7 wait . . . 80
13.3.8 unknown . . . 80
13.3.9 pram . . . 80
13.3.10 c58 . . . 80
13.3.11 resume . . . 80
13.3.12 op15 . . . 80
13.3.13 op12 . . . 80
13.3.14 c58ill . . . 80
13.3.15 pending . . . 80
13.3.16 c14 . . . 80
13.3.17 startseq . . . 81
13.3.18 op12ack . . . 81
13.3.19 op10 . . . 81
13.3.20 startseq2 . . . 81
13.3.21 startseq3 . . . 81
13.3.22 oneshot . . . 81
13.3.23 c50 . . . 81
13.3.24 holdout . . . 81
13.3.25 op12wait . . . 81
13.3.26 ev11 . . . 81
13.3.27 ev5 . . . 82
13.3.28 opsweep . . . 82
13.3.29 optime . . . 82
13.3.30 ctrlbits . . . 82
13.3.31 fmcmap . . . 82
13.3.32 evsweep . . . 82
13.3.33 rwmask . . . 82
13.3.34 c50x . . . 82
13.3.35 op0f . . . 82
13.3.36 ctrlhold . . . 83
13.3.37 evclear . . . 83
13.3.38 timeregs . . . 83
13.3.39 ev5rule . . . 83
13.3.40 c08 . . . 83
13.3.41 chan1 . . . 83
13.3.42 readev . . . 83
13.3.43 regnum . . . 83
13.3.44 branch . . . 83

13.3.45 waitf . . . 84
13.3.46 opcost . . . 84
13.3.47 op12rel . . . 84
13.3.48 pause . . . 84
13.3.49 fmc28 . . . 84
13.3.50 blk100 . . . 84
13.3.51 irqline . . . 84
13.3.52 restore . . . 84
13.3.53 irqen . . . 84
13.3.54 ev5t . . . 85
13.3.55 r8 . . . 85
13.3.56 state . . . 85
13.3.57 ev8 . . . 85
13.3.58 evcons . . . 85
13.3.59 dmaloop . . . 85
13.3.60 op0916 . . . 85
13.3.61 op16 . . . 85
13.3.62 opstate . . . 86
13.3.63 op16b . . . 86
13.3.64 op09 . . . 86
13.3.65 op09b . . . 86
13.3.66 op09c . . . 86
13.3.67 chan1pins . . . 86
13.3.68 pcwrite . . . 86
13.3.69 op12cont . . . 86
13.3.70 ctrlfx . . . 86
13.3.71 op09d . . . 87
13.3.72 fwidth . . . 87
13.3.73 irqsrc . . . 87
13.3.74 eccpar . . . 87
13.3.75 bufsize . . . 87
13.3.76 eccstr . . . 87
13.3.77 access . . . 87
13.3.78 fields0 . . . 87
13.3.79 tmofield . . . 88
13.3.80 eccstr2 . . . 88
13.3.81 ev5en . . . 88
13.3.82 tmobits . . . 88
13.3.83 eccmode . . . 88
13.3.84 eccchunk . . . 88
13.3.85 rawpage . . . 88
13.3.86 rawwrite . . . 88
13.3.87 writeseq . . . 89

13.3.88 op09nand . . . 89
13.3.89 eraseseq . . . 89
13.3.90 eccchunk2 . . . 89
13.3.91 eccmode2 . . . 89
13.3.92 tmobits2 . . . 89
13.3.93 ev5ready . . . 89
13.3.94 pageorder . . . 90
13.3.95 op09w33 . . . 90
13.3.96 ev5arm . . . 90
13.3.97 writeseq2 . . . 90
13.3.98 eccmode3 . . . 90
13.3.99 ev5two . . . 90
13.3.100 b15ecc . . . 90
13.3.101 eccmode4 . . . 91
13.3.102 finale . . . 91
13.3.103 startdiff . . . 91
13.3.104 startbits . . . 91
13.3.105 rsdecode . . . 91
13.3.106 vic20 . . . 91
13.3.107 fmc28b . . . 92
13.3.108 c58b1 . . . 92
13.3.109 evsweep2 . . . 92
13.3.110 c50fault . . . 92
13.3.111 ctrlwords . . . 92
13.3.112 c08self . . . 92
13.3.113 latchrep . . . 92
13.3.114 irq044 . . . 92
13.3.115 fmcbits . . . 93
13.3.116 fmcstat . . . 93
13.3.117 eccregs . . . 93
13.3.118 seqrestart . . . 93
13.3.119 done5 . . . 93
13.3.120 progtime . . . 93
13.3.121 regsweep . . . 93
13.3.122 seqfirst . . . 93
13.3.123 w2state . . . 94
13.3.124 c50b8 . . . 94
13.3.125 startregion . . . 94
13.3.126 bufport . . . 94
13.3.127 c58b2 . . . 94
13.3.128 stprev . . . 94
13.3.129 tmwait2 . . . 95
13.3.130 irqop . . . 95

13.3.131 done5b . . . 95
13.3.132 rsresult . . . 95
13.3.133 rschip . . . 95
13.3.134 miscorr . . . 95
13.3.135 fullpage . . . 95
13.3.136 dmactl . . . 95
13.3.137 fmcstat2 . . . 96
13.3.138 ch1buf . . . 96
13.3.139 evsweep3 . . . 96
13.3.140 blk100b . . . 96
13.3.141 straps . . . 96
13.3.142 w2vic . . . 96
Chapter 14 — Evidence: outcomes 97
14.1 Scope . . . 97
14.2 Device outcomes . . . 97
14.2.1 regmap (run 1) . . . 97
14.2.2 clock (run 1) . . . 97
14.2.3 alu (run 1) . . . 97
14.2.4 memory (run 1) . . . 97
14.2.5 indirect (run 1) . . . 97
14.2.6 control (run 1) . . . 97
14.2.7 wait (run 1) . . . 97
14.2.8 unknown (run 1) . . . 98
14.2.9 pram (run 2) . . . 98
14.2.10 alu (run 2) . . . 98
14.2.11 indirect (run 2) . . . 99
14.2.12 control (run 2) . . . 99
14.2.13 c58 (run 2) . . . 99
14.2.14 unknown (run 2) . . . 99
14.2.15 pram (run 3) . . . 100
14.2.16 alu (run 3) . . . 100
14.2.17 indirect (run 3) . . . 100
14.2.18 control (run 3) . . . 100
14.2.19 c58 (run 3) . . . 100
14.2.20 unknown (run 3) . . . 101
14.2.21 resume (run 4) . . . 102
14.2.22 wait (run 4) . . . 102
14.2.23 op15 (run 4) . . . 103
14.2.24 op12 (run 5) . . . 103
14.2.25 c58ill (run 5) . . . 103
14.2.26 pending (run 5) . . . 103
14.2.27 c14 (run 5) . . . 104
14.2.28 regmap (run 6) . . . 104

14.2.29 startseq (run 6) . . . 104
14.2.30 pending (run 6) . . . 104
14.2.31 op12ack (run 6) . . . 104
14.2.32 op10 (run 6) . . . 105
14.2.33 regmap (run 7) . . . 105
14.2.34 startseq2 (run 7) . . . 105
14.2.35 startseq3 (run 8) . . . 105
14.2.36 oneshot (run 8) . . . 105
14.2.37 c50 (run 9) . . . 105
14.2.38 holdout (run 9) . . . 105
14.2.39 holdout (run 10) . . . 106
14.2.40 op12wait (run 11) . . . 106
14.2.41 ev11 (run 11) . . . 106
14.2.42 ev5 (run 11) . . . 106
14.2.43 opsweep (run 11) . . . 106
14.2.44 optime (run 11) . . . 117
14.2.45 ctrlbits (run 11) . . . 117
14.2.46 fmcmap (run 11) . . . 118
14.2.47 evsweep (run 11) . . . 119
14.2.48 rwmask (run 12) . . . 119
14.2.49 c50x (run 13) . . . 120
14.2.50 evsweep (run 14) . . . 120
14.2.51 op0f (run 14) . . . 120
14.2.52 ctrlhold (run 14) . . . 120
14.2.53 evclear (run 14) . . . 121
14.2.54 timeregs (run 14) . . . 121
14.2.55 ev5rule (run 15) . . . 121
14.2.56 c08 (run 15) . . . 122
14.2.57 chan1 (run 15) . . . 122
14.2.58 readev (run 15) . . . 122
14.2.59 regnum (run 15) . . . 122
14.2.60 branch (run 15) . . . 122
14.2.61 waitf (run 15) . . . 123
14.2.62 opcost (run 15) . . . 123
14.2.63 op12rel (run 15) . . . 123
14.2.64 pause (run 15) . . . 124
14.2.65 fmc28 (run 15) . . . 124
14.2.66 blk100 (run 15) . . . 124
14.2.67 irqline (run 15) . . . 124
14.2.68 restore (run 16) . . . 124
14.2.69 readev (run 16) . . . 125
14.2.70 irqline (run 16) . . . 125
14.2.71 irqen (run 16) . . . 125

14.2.72 ev5t (run 16) . . . 126
14.2.73 r8 (run 16) . . . 126
14.2.74 state (run 17) . . . 126
14.2.75 readev (run 17) . . . 126
14.2.76 irqline (run 17) . . . 127
14.2.77 irqen (run 17) . . . 127
14.2.78 ev5t (run 17) . . . 127
14.2.79 r8 (run 17) . . . 127
14.2.80 state (run 18) . . . 128
14.2.81 ev8 (run 18) . . . 128
14.2.82 evcons (run 18) . . . 128
14.2.83 restore (run 19) . . . 128
14.2.84 dmaloop (run 20) . . . 129
14.2.85 restore (run 21) . . . 129
14.2.86 readev (run 21) . . . 129
14.2.87 dmaloop (run 21) . . . 129
14.2.88 op0916 (run 22) . . . 130
14.2.89 op16 (run 23) . . . 130
14.2.90 opstate (run 24) . . . 130
14.2.91 op16b (run 24) . . . 130
14.2.92 op09 (run 24) . . . 130
14.2.93 op09b (run 25) . . . 131
14.2.94 op09c (run 26) . . . 131
14.2.95 chan1pins (run 27) . . . 131
14.2.96 pcwrite (run 27) . . . 132
14.2.97 op12cont (run 27) . . . 132
14.2.98 ctrlfx (run 27) . . . 132
14.2.99 op09d (run 27) . . . 132
14.2.100 fwidth (run 27) . . . 132
14.2.101 irqsrc (run 27) . . . 133
14.2.102 eccpar (run 27) . . . 133
14.2.103 bufsize (run 27) . . . 133
14.2.104 eccstr (run 27) . . . 134
14.2.105 access (run 27) . . . 134
14.2.106 fields0 (run 28) . . . 136
14.2.107 tmofield (run 28) . . . 136
14.2.108 eccstr2 (run 28) . . . 136
14.2.109 ev5en (run 28) . . . 137
14.2.110 tmobits (run 29) . . . 137
14.2.111 eccmode (run 29) . . . 137
14.2.112 eccchunk (run 29) . . . 138
14.2.113 rawpage (run 29) . . . 138
14.2.114 rawwrite (run 29) . . . 139

14.2.115 writeseq (run 29) . . . 139
14.2.116 op09nand (run 29) . . . 139
14.2.117 eraseseq (run 29) . . . 139
14.2.118 eccchunk2 (run 30) . . . 140
14.2.119 eccmode2 (run 30) . . . 140
14.2.120 tmobits2 (run 30) . . . 143
14.2.121 ev5ready (run 30) . . . 143
14.2.122 pageorder (run 30) . . . 143
14.2.123 op09w33 (run 30) . . . 144
14.2.124 ev5arm (run 31) . . . 144
14.2.125 writeseq2 (run 31) . . . 144
14.2.126 eccmode3 (run 31) . . . 144
14.2.127 ev5two (run 32) . . . 144
14.2.128 b15ecc (run 32) . . . 145
14.2.129 eccmode4 (run 33) . . . 145
14.2.130 finale (run 34) . . . 145
14.2.131 startdiff (run 35) . . . 146
14.2.132 startbits (run 35) . . . 147
14.2.133 rsdecode (run 35) . . . 149
14.2.134 vic20 (run 35) . . . 149
14.2.135 fmc28b (run 35) . . . 150
14.2.136 c58b1 (run 35) . . . 150
14.2.137 evsweep2 (run 35) . . . 150
14.2.138 c50fault (run 35) . . . 150
14.2.139 ctrlwords (run 35) . . . 150
14.2.140 c08self (run 35) . . . 151
14.2.141 latchrep (run 35) . . . 151
14.2.142 irq044 (run 35) . . . 151
14.2.143 fmcbits (run 35) . . . 151
14.2.144 fmcstat (run 35) . . . 152
14.2.145 eccregs (run 35) . . . 152
14.2.146 seqrestart (run 35) . . . 152
14.2.147 done5 (run 35) . . . 153
14.2.148 progtime (run 35) . . . 153
14.2.149 regsweep (run 35) . . . 153
14.2.150 seqfirst (run 36) . . . 159
14.2.151 w2state (run 36) . . . 159
14.2.152 c50b8 (run 36) . . . 160
14.2.153 startregion (run 36) . . . 160
14.2.154 bufport (run 36) . . . 161
14.2.155 c58b2 (run 36) . . . 161
14.2.156 stprev (run 36) . . . 161
14.2.157 tmwait2 (run 36) . . . 162

14.2.158 irqop (run 36) . . . 162
14.2.159 done5b (run 37) . . . 162
14.2.160 rsresult (run 37) . . . 162
14.2.161 rschip (run 37) . . . 162
14.2.162 miscorr (run 37) . . . 163
14.2.163 fullpage (run 37) . . . 163
14.2.164 dmactl (run 37) . . . 163
14.2.165 fmcstat2 (run 37) . . . 163
14.2.166 ch1buf (run 37) . . . 164
14.2.167 evsweep3 (run 37) . . . 164
14.2.168 blk100b (run 37) . . . 164
14.2.169 straps (run 37) . . . 164
14.2.170 irqop (run 37) . . . 164
14.2.171 w2vic (run 38) . . . 164
14.3 Reverse-engineering outcomes . . . 165
14.3.1 RE-1: Nine sequencer programs in the iPod OS image . . . 165
14.3.2 RE-2: Program table . . . 165
14.3.3 RE-3: Program locations as memory addresses . . . 166
14.3.4 RE-4: Instruction encoding and opcode set used by the programs . . . 166
14.3.5 RE-5: Prologue values . . . 166
14.3.6 RE-6: Opcodes 18 and 19 . . . 166
14.3.7 RE-7: Wait event 5 shape . . . 166
14.3.8 RE-8: ERASE ready waits . . . 166
14.3.9 RE-9: Control word and chip-enable changes . . . 167
14.3.10 RE-10: READID control word . . . 167
14.3.11 RE-11: FMCTRL1 codes and FMC_DATA_SEL fields . . . 167
14.3.12 RE-12: ECC protocol in READ . . . 167
14.3.13 RE-13: Page verdict in READ . . . 167
14.3.14 RE-14: Decode order . . . 168
14.3.15 RE-15: Page stream . . . 168
14.3.16 RE-16: Meta engine use . . . 168
14.3.17 RE-17: READ batches . . . 168
14.3.18 RE-18: Page word and FMC_STATUS b27 . . . 168
14.3.19 RE-19: DMA address and DMA control . . . 168
14.3.20 RE-20: WRITE DMA list . . . 169
14.3.21 RE-21: Two-plane status polls . . . 169
14.3.22 RE-22: Program FAIL path . . . 169
14.3.23 RE-23: ERASE sequence . . . 169
14.3.24 RE-24: READID sequence . . . 169
14.3.25 RE-25: Fault handlers . . . 170
14.3.26 RE-26: RAWREAD . . . 170
14.3.27 RE-27: Write variants . . . 170
14.3.28 RE-28: Releases 1.0.1 to 1.1.3 . . . 170

14.3.29 RE-29: Chip addressing in the programs . . . 171
14.3.30 RE-30: Absent programs . . . 171
14.3.31 RE-31: CPU driver: start . . . 171
14.3.32 RE-32: CPU driver: wait and classify . . . 171
14.3.33 RE-33: CPU driver: reset . . . 171
14.3.34 RE-34: CPU driver: clear . . . 171
14.3.35 RE-35: CPU driver: timing word . . . 172
14.3.36 RE-36: CPU driver: chip reset . . . 172
14.3.37 RE-37: CPU driver: initialisation . . . 172
14.3.38 RE-38: CPU driver: modes . . . 173
14.3.39 RE-39: CPU driver: result classification . . . 173
14.3.40 RE-40: CPU driver: other sequencer registers . . . 173
14.3.41 RE-41: Second microcode copy . . . 174
14.3.42 RE-42: Driver generation . . . 174
14.3.43 RE-43: Boot ROM NAND access . . . 174
14.3.44 RE-44: No cache maintenance in the CPU-side driver . . . 175
14.3.45 RE-45: Chip table row of this unit's chip . . . 175
14.3.46 RE-46: Bank count in WRITECACHE . . . 176
14.3.47 RE-47: Boot ROM boot-device selection . . . 176
Chapter 15 — Evidence interpretation 177
15.1 Instruction set and registers . . . 177
15.2 Opcodes . . . 178
15.3 Events, waits and the timeout . . . 178
15.4 Control registers . . . 179
15.5 Memory maps . . . 180
15.6 NAND controller . . . 180
15.7 ECC codes and modes . . . 181
15.8 NAND operations . . . 182
15.9 Interrupts . . . 182
15.10 Original firmware (reverse engineering) . . . 183
15.11 Open questions . . . 185
Chapter 16 — Device findings: risks and what only a device can settle 186
16.1 Risks and their status . . . 186
16.2 Settled on a device . . . 186
16.2.1 Registers, instructions and fields . . . 186
16.2.2 Opcodes . . . 187
16.2.3 Events, waits and the timeout . . . 188
16.2.4 Start word, states and the window . . . 188
16.2.5 NAND controller and DMA . . . 189
16.2.6 ECC engine . . . 190
16.2.7 NAND operations . . . 190
16.2.8 Soft reset and power-on values . . . 190

Chapter 17 — Open questions **192**

17.1 Instruction set and sequencer behaviour . . . 192

17.2 Events and the status latch . . . 192

17.3 Interrupts . . . 193

17.4 NAND controller registers . . . 193

17.5 ECC engine . . . 193

17.6 Original programs and the CPU interface . . . 194

17.7 Soft reset versus power cycle . . . 194

Chapter 18 — Finding list **195**

18.1 Findings . . . 195

nano 3G FMSS ("M1"): sequencer, NAND controller and ECC engine

These documents describe how the FMSS engine of the iPod nano 3G (S5L8702) works: the sequencer coprocessor inside the NAND controller, the NAND controller (FMC) it drives, and the ECC engine. They cover the engine's behaviour, registers, instruction set, memory maps, ECC, events, timing and interfaces, and how the original firmware uses it. Each statement about the engine's behaviour comes with its evidence.

The documents form a book of numbered chapters. Chapter 1 defines the evidence classes, the numbering and the terms. Chapters 2 to 12 describe the engine. Chapters 13 to 15 hold the experiments, their outcomes and the interpretation of the outcomes. Chapters 16 to 18 collect the device findings, the open questions and the finding list.

Chapters

chapter	title	content
1	Overview and terms	evidence classes, numbering, terms, the three blocks
2	Architecture: execution model, start, stop and fault handling	execution model, start, pause, hold, stop, END and fault handling, diagrams of the CPU and sequencer interface
3	The register window as the ARM CPU sees it	segments, mirrors, access matrix, CPU-only rules, interrupt lines, how the original CPU driver runs a program
4	The register window as the sequencer sees it	12-bit offsets, register file, parameter RAM, system memory, program fetch
5	Instruction encoding	instruction word layout, operand map, opcode summary table, ALU operand rule, byte order
6	Opcode reference	one entry per valid opcode, and one for the opcode values without a meaning
7	Events, waits and the timeout	`wait` events, the rules for `wait` event 5, `wait2` conditions, `tmwait` and `evjmp`, the timer `C14`, the timeout length table
8	Sequencer control registers	the register table of the control block `0xc00..0xc5c` and the start-word bits
9	NAND controller (FMC)	`FMC_CTRL0`, the `FMCTRL1` codes, the other registers, `FMC_STATUS`, `FMC_DATA_SEL`, the second register set
10	ECC engine	on-flash layout, BCH and Reed-Solomon codes, the `ECC_MODE` map, modes without bit 8, strength, registers, byte order
11	NAND operations	how the original firmware performs reset, ID read, page read, page program, block erase and fault handling, and the device checks
12	CPU interface: parameter words, result words and timing word	parameter words, result words and the timing word
13	Experiments: what was done and why	every experiment run on the device
14	Evidence: outcomes	the outcomes of the device experiments and of the reverse engineering of the original firmware
15	Evidence interpretation	what the outcomes are taken to show, by topic
16	Device findings: risks and what only a device can settle	the risks and their status, and the results settled on a device
17	Open questions	every point marked open, not measured or unexplained
18	Finding list	findings 1 to 175 with evidence and class

Sources

- lemonjesus, "S5L8702-FMISS-Tools", https://github.com/lemonjesus/S5L8702-FMISS-Tools/tree/main

Chapter 1 — Overview and terms

Summary: What the FMSS engine is, the evidence classes and numbering used in these chapters, the terms, and the three hardware blocks (sequencer, NAND controller (FMC), ECC engine).

These documents describe how the FMSS engine of the iPod nano 3G (S5L8702) works: the sequencer coprocessor inside the NAND controller, the NAND controller (FMC) it drives, and the ECC engine. They cover the engine's behaviour, registers, instruction set, memory maps, ECC, events, timing and interfaces, and how the original firmware uses it. Each statement about the engine's behaviour comes with its evidence.

1.1 Evidence classes

The statements in the chapters rest on two sources:

- static analysis of the original programs and of the original CPU driver;
- device tests, which are small programs run on the sequencer of one unit (chapter 13).

Each statement carries one of the following evidence classes, in the text and in the finding list (chapter 18):

- **observed**: seen in the original programs or in the original CPU driver; the address is given in the finding list.
- **device**: measured on a unit.
- **inferred**: an interpretation of observed use or of device results.
- **3p**: a driver written by a third party, reported by its authors as tested on a device.
- **open**: not settled.

The outcomes themselves are in chapter 14, under the IDs `D-<experiment>-<device run>` (device) and `RE-<n>` (reverse engineering of the original firmware). Chapter 15 states what each outcome is taken to show and which evidence class the statement has.

1.2 Numbering and labels

Chapters are numbered 1 to 18. Headings inside chapter N are numbered N.1, N.2, N.2.1 and so on. A cross-reference names the chapter and the section, for example chapter 16, section 16.2. The entries of chapter 6 (opcode reference) are the exception: each is headed by its opcode and mnemonic and carries no number.

Chapter 11 labels the behaviours of the original programs `C1`..`C13` (common), `R1`..`R7` (ECC and data path), `ID1`..`ID5` (reset and ID), `RD1`..`RD11` (page read), `WR1`..`WR9` (page program), `ER1`..`ER4` (erase) and `FL1`..`FL3` (fault handling). Each label is defined in chapter 11 and is used as a cross-reference in other chapters. Finding numbers (`finding 55`) refer to the entries of chapter 18. Names such as `C58`, `C4C` and `C14` are the sequencer control words at offsets `0xc58`, `0xc4c` and `0xc14` (chapter 8).

Conventions used throughout:

- Bits are written "bit N" or "bits H:L" (high bit first); register offsets and values are hexadecimal in code style (`0x38a00000`).
- Times are given in us and ms. Approximate values are marked with "about".
- "`wait` event N" is the status flag with number N in the list of flags that the `wait` instruction waits for (chapter 7, section 7.1). "`wait2` condition N" is the entry with number N in the separate list of conditions that the `wait2` instruction waits for (chapter 7, section 7.2). The two lists are different, so `wait` event N and `wait2` condition N with the same N are unrelated.
- "Original firmware", "original programs" and "original CPU driver" name Apple's firmware, the nine sequencer programs in it, and its NAND driver that runs on the ARM CPU. "Third-party driver" names the third-party driver of the evidence class 3p.

1.3 Terms

term	meaning	source of the name
NAND controller (FMC)	flash memory controller: the SoC's NAND controller at `0x38a00000`. It produces command, address and data cycles on the NAND bus and contains the data buffers and the DMA. It has two channels, whose register sets are at offsets `0x000` and `0x400` (chapter 9, section 9.5)	register names of the SoC family (`FMCTRL0`, `FMCTRL1`, ...) as used by third-party headers

term	meaning	source of the name
FMSS engine	the whole NAND hardware described here: the sequencer, the NAND controller and the ECC engine (chapter 1, section 1.4)	the original firmware's own name (its log reads "FMSS sequence timed out", its driver functions are named `...fmss...`); the firmware never spells the name out (an expansion such as "flash memory sequencer subsystem" is a guess)
sequencer	the small coprocessor inside the NAND controller that runs programs of 8-byte instructions (chapter 5)	name used in these documents
M1 engine	the engine of the nano 3G described in these documents. Whether it is the same hardware version as the M2 engine is pending: the 3G's original programs use more opcodes than the 4G / 5G programs (`pld` and `pst`, opcodes `18` and `19`, chapter 6)	name used in these documents, after the 4G/5G name M2
M2 engine	the same kind of engine in the nano 4G / 5G	the 4G/5G firmware
ECC engine	error-correcting code (ECC) hardware at offset `0x800` that generates parity for 512-byte sectors and corrects them (how it works and where the parity is kept: chapter 10, section 10.1)	name used in these documents
CE	chip enable: the signal that selects one NAND chip (`FMC_CTRL0` bits 8:1)	NAND standard
CS_IRQ	the sequencer's flag word at `0xc0c` (bits for END, fault and others)	the original firmware calls it `FC_CS_IRQ`; what "CS" stands for is not stated
event	one of the status flags that `wait` can wait for, selected by a number: "`wait` event N" (chapter 7, section 7.1)	name used in these documents
condition	the number N of `wait2`: `wait2` condition N is interrupt source N of the CPU's interrupt controllers (VIC0 lines 0..31, VIC1 lines 32..63), and `wait2 N` waits until that line is raised (chapter 7, section 7.2)	name used in these documents
armed	a state of the NAND controller that `wait` event 5 depends on: a status latch sets the event's flag only while the event is armed, and each latch uses up the armed state (chapter 7, section 7.1)	name used in these documents
VIC0, VIC1	the CPU's two vectored interrupt controllers; "VIC1 raw bit 22" is bit 22 of VIC1's raw interrupt status (interrupt source 54), the line the original CPU driver watches for the end of a run (chapter 3, section 3.4)	SoC register names
parameter RAM	32 words at `0xd00` that the CPU and the program both access; they hold a program's inputs and outputs (chapter 4, section 4.4)	name used in these documents

1.4 The three blocks

The NAND hardware at `0x38a00000` has three parts that share one register window (chapter 3, section 3.2):

block	registers	what it does	who drives it
sequencer	control block `0xc00..0xc5c` (chapter 8, section 8.1)	runs a program of 8-byte instructions fetched from system memory	the CPU starts and stops it through `FMSS_CTRL` (`0xc00`)
NAND controller (FMC)	`0x000..0x0fc` (channel 0), `0x400..0x4fc` (channel 1) (chapter 9)	puts commands, addresses and data on the NAND bus and moves data by DMA	whoever writes its registers: the sequencer program, or the CPU directly
ECC engine	`0x800..0x8fc` (chapter 10)	generates parity for 512-byte sectors, checks them and corrects them	the same: the sequencer program or the CPU

The sequencer does no NAND work itself: its program writes the NAND controller's and the ECC engine's registers, exactly as the original CPU driver does, and waits for their completion flags (`wait` events, chapter 7, section 7.1). How the sequencer is started and stopped, and how the CPU and the sequencer interact, is in chapter 2.

The original firmware carries nine sequencer programs (ERASE, READ, READID, WRITE, RAWREAD, WRITE2P, WRITE2P_78, WRITE_78, WRITECACHE; RE-2) and an original CPU driver that starts them (RE-31); both are described in the chapters that follow, together with the behaviour of the hardware they use.

1.5 Named parts of the data path

Data passes through the following parts on its way between system memory and a NAND chip. Each part is defined below by where it is, how big it is, how it is selected, what fills and empties it, and what is known about its contents over time. The other chapters use exactly these names. "Device" marks what was measured and "inferred" marks what follows from it.

Where the buffers are. The data buffers and the parity buffers are storage inside the NAND controller block of the SoC (register window at CPU address 0x38a00000). They have **no address**: neither the CPU memory map nor the register window maps their bytes to any address range.

- No word of the register window is known to give access to a parity buffer: the read / write matrix of all 1024 window words, from the CPU and from the sequencer, found none that does (device, access).
- The word FMC_FIFO (0x080, the **FIFO port**) is the only window word through which data from the chip has been read. The original READID program uses it in this order (observed, RE-24; the third-party driver does the same):
  1. It reads 8 bytes from the chip (FMC_DATA_SEL = 1, FMCTRL1 = 0x1c2).
  2. It sets FMC_DATA_SEL = 0x100 and FMCTRL1 = 0x340.
  3. It reads 0x080 twice and obtains the 8 ID bytes in order.
- The FIFO port does not read a data buffer that a DMA filled (device, bufport): after data buffer 0 or 1 (and parity buffer 0) was filled by DMA with a known pattern, 130 reads of 0x080 returned 0, with and without FMCTRL1 = 0x340. Each read advanced FMC_28 by 4 (0, 128, 256, 384 after every 32 reads, wrapping at 512), so FMC_28 is the FIFO port's read position. 128 writes to 0x080 after FMCTRL1 = 0x340 changed 26 of the 128 words of data buffer 0. Whether the bytes of a READ ID are held in a data buffer or in separate storage is open.
- All other ways into and out of a buffer are transfers. They are the chip transfers and DMAs of the NAND controller (FMCTRL1), the ECC operations (ECC_KICK) and the metadata path (FMC_METACTL).

1.5.1 System memory

- **What:** the SoC's SDRAM and internal SRAM on the CPU bus, addressed with 32-bit bus addresses.
- **Used by the FMSS engine for:** the program (fetched starting at the address in FMSS_PROG), the per-request lists (read with ldi), the results (written with sti), and the data of every DMA (bus address in FMC_DMA_ADDR).
- **Details:** chapter 3, chapter 4, section 4.2.

1.5.2 Data buffer 0 and data buffer 1

- **Where:** inside the NAND controller, belonging to channel 0 (0x000 register set); no address (see "Where the buffers are" above). The CPU and a program reach their bytes only through a DMA, a chip transfer or an ECC operation; reads of the FIFO port returned 0 for them (%s). Channel 1 (0x400 register set) has its own data buffers 0 and 1: a pattern DMA-ed into each through the 0x400 set came back unchanged (ch1buf).
- **Size:** 512 bytes each: a DMA always moves exactly 512 bytes into or out of one, whatever FMC_DNUM holds (device, bufsize).
- **Selected by** one-hot fields, one for each direction of transfer (device):

operation	field	data buffer 0	data buffer 1
filled by a chip read (FMCTRL1 = 0xe2) or a DMA from system memory (0x2e0)	FMC_DATA_SEL bits 1:0	0x1	0x2
emptied by a chip write (0xe4) or a DMA to system memory (0x100, 0x1e0)	FMC_DATA_SEL bits 9:8	0x100	0x200
read and corrected in place by an ECC operation	ECC_KICK bits 9:8	0x100	0x200

- **Contents over time:** a data buffer keeps its bytes between operations within one program (device: a DMA into the buffer, an encode and a decode in one program work on the same bytes). The set-up word at the start of a program (FMCTRL1 = 0x0ff3f8e0) clears the buffers (inferred: a decode run in a later program than its encode found nothing to check). The buffers are not tied to any one chip enable (observed / 3p).
- **Use:** each holds the 512 data bytes of one sector. The original programs alternate between the two (FMC_DATA_SEL 0x102 / 0x201): one is filled while the other is emptied.

1.5.3 Parity buffer 0 and parity buffer 1

- **Where:** inside the NAND controller; no address and no known window port (see "Where the buffers are" above). DMAs through the 0x400 register set (channel 1) into parity buffer 0 and 1 came back unchanged, and after the one into parity buffer 0, channel 0's parity buffer 0 read back channel 1's pattern; so parity buffer 0 is one buffer shared by both channels (inferred, ch1buf). A set-up word written to channel 1's FMCTRL1 (0x404) left channel 0's parity buffer 0 reading all 0xff (same evidence).
- **Size:** 64 bytes each, divided into four slots (1.5.4): a 512-byte DMA into a parity buffer wraps around within the 64 bytes, so the buffer ends up holding the last 64 bytes of the source (device, eccchunk).

- **Selected by** (device):

operation	field	parity buffer 0	parity buffer 1	which slot
filled by a chip read of a chunk (FMCTRL1 = 0x32)	FMC_DATA_SEL bits 5:4	0x10	0x20	byte offset in FMC_24 (0, 0x10, 0x20, 0x30)
filled by a DMA from system memory (0x2e0)	FMC_DATA_SEL bits 5:4	0x10	0x20	the whole buffer, wrapping
emptied by a chip write of a chunk (0x34)	FMC_DATA_SEL bits 13:12	0x1000	0x2000	byte offset in FMC_28
emptied by a DMA to system memory (0x100)	FMC_DATA_SEL bits 13:12	0x1000	0x2000	from slot 0
written by an encode, read by a decode	ECC_KICK bits 13:12	0x1000	0x2000	ECC_KICK bits 17:16 = k (BCH); always slot 0 (Reed-Solomon)
written by a meta encode, read by a meta decode	FMC_METACFG nibbles (0..3 = buffer 0, 4..7 = buffer 1; inferred)			the nibble value modulo 4 (inferred)

- **Contents over time:** after the set-up word every byte reads 0xff (device: unwritten slot bytes read 0xff in every DMA out). Bytes not written by an operation keep their value until the next set-up word (device: a BCH encode leaves slot bytes 0..2 as the meta encode wrote them).
- **Use:** it holds the four chunks of one 2 KiB unit, one chunk per slot. The original programs use parity buffer 0 for all four sectors.

1.5.4 Slot

- **What:** slot k (k = 0..3) is bytes 16k to 16k + 15 of a parity buffer.
- **Holds:** the chunk (1.5.5) of one sector while it is moved between the chip and the ECC engine. Its contents per ECC method (which bytes hold metadata, which parity) are in chapter 10, section 10.2.
- **Boundaries:** a BCH operation uses the slot that ECC_KICK bits 17:16 name. A Reed-Solomon operation ignores that field and always uses slot 0; its 20-byte parity with 16 symbols fills slot 0 and bytes 0..3 of slot 1 (device, chapter 10, section 10.4).

1.5.5 Chunk

- **What:** the bytes that are stored on the chip in front of each 512-byte sector, in the page's byte stream. In the original layout a chunk is 16 bytes: 3 metadata bytes and 13 BCH parity bytes (device).
- **Path:** a chunk is written to the chip from a slot (0x34) and read from the chip into a slot (0x32); the transfer length is FMC_DNUM + 1 bytes (FMC_DNUM = 0xf, giving 16 bytes, in the original programs).
- **Details:** chapter 10, section 10.9.

1.5.6 ECC area

- **What:** the ECC engine's window words 0x900..0x913 (20 bytes), which the CPU and the program can read (the program with ld or pld).
- **Contents:** after an encode, the parity just computed in reverse byte order (BCH 13 bytes, Reed-Solomon 10, 15 or 20) and, for some modes, further bytes. When no code is used, it holds the next value of a fixed sequence. A decode clears 0x900..0x90c.
- **Details:** chapter 10, section 10.13.

1.5.7 Metadata registers

- **What:** FMC_META0..2 (0x060..0x068), three 32-bit words, 12 bytes in all, holding the metadata of one 2 KiB unit.
- **Path:** a meta encode (FMC_METACTL = 1) copies bytes 3k..3k + 2 of the metadata registers into bytes 0..2 of slot k (k = 0..3; the bytes are numbered in the order of the little-endian words); a meta decode (FMC_METACTL = 2) copies them back. Which slots are used is set by FMC_METACFG (chapter 11, R6).

1.5.8 FIFO port (FMC_FIFO)

- **What:** the word FMC_FIFO (0x080). After a READ ID transfer (FMCTRL1 = 0x1c2, FMC_DNUM + 1 bytes, 8 in the original READID), each read of 0x080 with FMCTRL1 = 0x340 and FMC_DATA_SEL = 0x100 returns the next 4 ID bytes. Each read advances FMC_28 by 4 (its read position, 9 bits). It returned 0 for data buffers filled by DMA, so where the ID bytes are held is open ("Where the buffers are" above). chapter 9, section 9.3.

1.5.9 Status register

- **What:** `FMC_NSTAT` (`0x04c`), the most recent NAND status byte, delivered by a status latch (`FMCTRL1` = `0xca`, `wait` event 5). chapter 7, section 7.1.

1.5.10 Chip page

- **What:** one page of a NAND chip: on this unit 4096 data bytes and 128 spare bytes, 4224 bytes in one byte stream (chapter 10, section 10.9).
- **Path:** the chip loads a page into its own page register with `00h`, address cycles, `30h`; the controller then reads the page out byte by byte into a slot and a data buffer. A write sends the bytes into the chip's page register and `10h` programs them (chapter 11).

1.5.11 What the ECC engine touches

The ECC engine reads and writes only a data buffer and a slot of a parity buffer, and it writes the ECC area. It never reads or writes system memory or a chip. A sector reaches the chip only when the program sends the sector's slot (the chunk) and its data buffer with chip writes.

References: chapter 13 (what was done and why), chapter 14 (outcomes), chapter 15 (what the outcomes mean).

Related chapters: chapter 2, chapter 5, chapter 9, chapter 10, chapter 14.

Chapter 2 — Architecture: execution model, start, stop and fault handling

Summary: How the sequencer runs a program, how the CPU starts, pauses, holds, resets and ends a run, and what happens when a program ends and when a fault occurs. The three blocks (sequencer, NAND controller (FMC), ECC engine) are described in chapter 1, section 1.4.

2.1 How the sequencer is started and stopped

Two NAND controller registers look like start and stop switches, but they are not part of the sequencer:

- `FMC_CTRL0` (`0x000`) is the NAND controller's configuration word: controller enable (bit 0), chip selection (bits 8:1), bus timings, and DMA enable (bit 24) (chapter 9, section 9.1). Programs write it at their start. To switch chips they modify it in three steps: `ld` with a mask that keeps everything except the chip bits, `or` in the new chip, and `st`. It does not start or stop anything. The CPU writes 0 to it at start-up and at power-off, so that no chip is selected.
- `FMCTRL1` (`0x004`) is the NAND controller's "go" register. Each write starts one operation (sending the address bytes, reading or writing data, latching the status byte, or a DMA), and a flag in `FMC_STATUS` shows when the operation is done (chapter 9, section 9.2, chapter 9, section 9.4).

Starting the sequencer (done by the CPU; chapter 3, section 3.5, chapter 8, section 8.1):

1. The parameters are already in parameter RAM (`0xd00..`) and the program is already in system memory.
2. `FMSS_PROG` (`0xc04`) = the program's bus address; `CS_IRQ` (`0xc0c`) = `0xff` (this clears the flags).
3. `FMSS_CTRL` (`0xc00`) = `0xfff5`. Bit 0 starts the program at its first instruction, and r0..r7 keep the values the CPU wrote. After a sequencer reset, the first start only primes the sequencer (`startseq3`; C13).

Stopping:

- normally: the program executes `end`. `CS_IRQ` bit 0 is set, the value of r6 appears in `SEQ_RESULT` (`0xc30`), and the sequencer stays idle until the next start (`FMSS_CTRL` bit 18 is clear).
- by a fault: a `wait` (or `wait2`, `evjmp`) that times out, or an illegal opcode, sets `CS_IRQ` bit 2. The sequencer then continues at the fault handler (address in `C4C`) or at the next instruction, as `C58` selects (chapter 2, section 2.3). The program must still reach an `end`.
- by the CPU: writing `FMSS_CTRL` = `0x30000` resets the sequencer, whatever it is doing. The reset clears r0..r7, `C08`, `C40` and `C44`, and keeps `C58`, `C4C` and `C5C`. The original CPU driver does this after a fault or after 2 s without an END (RE-33, RE-32).
- handed to the CPU by the program: `pause` (opcode 12) stops the program and sets `CS_IRQ` bit 3. The CPU then writes the PC (`0xc38`; normally the offset of the `pause` instruction + 8) and a start word, and the program continues at that PC (chapter 6, opcode 12).
- paused, not stopped: while the program runs, a start word written with bit 1 set freezes it where it is, and a start word written without bit 1 continues it. A start word with bit 3 or bit 16 set, written while the program runs, aborts the program and returns the sequencer to the hold state that follows a reset (chapter 8, section 8.1.1).

The CPU can observe a run through four registers:

- `CS_IRQ`: bit 0 reports END and bit 2 reports a fault.
- `FMSS_CTRL` bit 18: set when the program has started and not yet finished.
- `C08`: a state code (running, waiting, paused, stopped at `pause`).
- the PC (`0xc38`).

One run, as seen at the interface between the CPU and the sequencer (what happens inside the program is described from chapter 2, section 2.2 onwards):

diagram (mermaid source):

```
sequenceDiagram
    autonumber
    participant CPU
    participant SEQ as sequencer
    participant HW as NAND controller and ECC
    CPU->>SEQ: write inputs to parameter RAM (0xd00)
    CPU->>SEQ: FMSS_PROG = program address
    CPU->>SEQ: CS_IRQ = 0xff (clear flags)
    CPU->>SEQ: FMSS_CTRL = 0xfff5 (start)
    loop until the program ends
```

```
        SEQ->>HW: write controller registers
        HW-->>SEQ: completion flag (event)
    end
    alt normal end
        SEQ-->>CPU: CS_IRQ bit 0, result in SEQ_RESULT
        CPU->>SEQ: read SEQ_RESULT and parameter RAM
    else fault, or no end within 2 s
        SEQ-->>CPU: CS_IRQ bit 2, or nothing
        CPU->>SEQ: FMSS_CTRL = 0x30000 (reset)
        CPU->>SEQ: start an END-only program
    end
    CPU->>SEQ: CS_IRQ = 5 (acknowledge)
```

The sequencer's states as the CPU controls them (details in "Stopping" above and chapter 8, section 8.1; after a reset the first start only primes the sequencer; "WithCPU" is the state of a program stopped at `pause`, with `CS_IRQ` bit 3 set):

diagram (mermaid source):

```
stateDiagram-v2
    direction LR
    [*] --> Idle
    Idle --> Running : start (FMSS_CTRL = 0xfff5)
    Running --> Idle : program executes end
    Running --> Paused : start word with bit 1
    Paused --> Running : start word without bit 1
    Running --> WithCPU : program executes pause
    WithCPU --> Running : CPU writes PC, then start
    Running --> Reset : FMSS_CTRL = 0x30000
    Paused --> Reset : FMSS_CTRL = 0x30000
    WithCPU --> Reset : FMSS_CTRL = 0x30000
    Reset --> Idle : END-only program
```

2.2 Execution model

- The sequencer is a small processor inside the NAND controller. It fetches 8-byte instructions from system memory, starting at the bus address in `FMSS_PROG` (8-byte aligned). The original CPU driver performs no cache maintenance around a run (RE-44).
- The sequencer has eight 32-bit registers, r0..r7. The CPU can read them at controller offsets `0xc18 + 4*n`. After END, r6 (`0xc30`) serves as the program's result word (`SEQ_RESULT`). The sequencer has no flags or condition codes; a branch tests the value of a register.
- The program counter (`FMSS_PC`, `0xc38`) is a byte offset from the start of the program. Branch targets and the fault vector are also byte offsets; the original programs keep them multiples of 8. There are no delay slots.
- Register-access instructions address the controller's register window with a 12-bit byte offset from the controller base address: `0x000..` NAND controller, `0x800..` ECC engine, `0xc00..` sequencer control, `0xd00..` parameter RAM (chapter 3, section 3.2).
- The memory-access instructions (`ldi`, `sti`) use a 32-bit bus address held in a register, rounded down to a multiple of 4. Their accesses are not kept coherent with the CPU data cache.
- No cycle count per instruction is specified. The measured time per instruction is determined by the instruction fetch: 217 ns when the program is in SDRAM and 138 ns when it is in internal SRAM (chapter 5, section 5.3).
- A program ends only with `end` (the all-zero instruction) or through a `wait` timeout that jumps to the fault handler. A program without an `end` runs on into whatever follows it in memory; zeroed memory reads as `end`.

2.3 END and fault

- Normal end: the program executes `end`. The sequencer sets `CS_IRQ` bit 0 and presents the value of r6 in `SEQ_RESULT` (`0xc30`). The original CPU driver polls VIC1 raw status bit 22 (interrupt source 54) to detect the end (RE-32). In the device tests, that VIC1 bit stayed clear after END, after a fault and with any start word (`irqline`, device run 15, `irqline`, device run 16, `irqline`, device run 17, `irqen`, device run 16, `irqen`, device run 17); when a NAND controller or ECC interrupt enable was set, the only effect was that `CS_IRQ` bits 4 and 6 were set. Whether a further enable outside the sequencer block, which the original firmware sets, is needed for the VIC1 bit is open (chapter 17, section 17.3). `CS_IRQ` bits 0 and 2 report the end and the fault.
- Fault: a `wait` timeout, or an opcode without a defined meaning (which first stalls for the same time, about 4.87 ms), sets `CS_IRQ` bit 2 and, with `C58` bit 2 set, jumps to the fault vector (device). The handler ends with `end`, so `CS_IRQ` bit 0 is also set. The original handlers test `FMSS_C50` bit 0 and write 1 to acknowledge

it. On the device, FMSS_C50 bit 0 is set while a command flag is pending and clear otherwise; a timeout or an illegal opcode does not change it, and the handler's write of 1 does not clear it (c50fault). A PROGRAM or ERASE handler entered by a timeout with no command flag pending therefore returns the failure bit 28 without bit 27 (inferred from the handler code, RE-25, and this device result). The READ, READID and RAWREAD handlers set bit 27 unconditionally. The original CPU driver treats CS_IRQ bit 2 set as a failed run and then resets the sequencer (RE-32).

- After a sequencer reset, the next start only primes the sequencer: it fetches instruction 0 and executes it only if that instruction is end (device, startseq3). The original CPU driver follows its reset with a start of an idle program of zero length (RE-33). A second start also runs a program that is not END-only (startseq3).
- A program can also jump to its own handler by choice (for example, when the NAND status reports FAIL). That path does not set CS_IRQ bit 2. The CPU sees a normal END and reads the failure from r6 (FL1).

2.4 Start word behaviour

The bits of the start word (FMSS_CTRL, 0xc00), and what each bit does while the sequencer is idle and while a program runs (including the pause, hold and abort behaviour), are described in chapter 8, section 8.1.1. The device results on pause and abort are in chapter 16, section 16.2 (rows "start word bits 1, 3, 16", "pause and abort while running", "pause (opcode 12)", "reset and start").

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 3, chapter 4, chapter 5, chapter 8, chapter 16.

Chapter 3 — The register window as the ARM CPU sees it

Summary: The NAND hardware occupies a 4 KB register window at CPU address `0x38a00000`. This chapter describes it from the CPU side: the segments of the window, the mirrors (offsets that reach the same register), the words that always read 0, the access rules, the interrupt lines and the way the original CPU driver runs a program. The sequencer's own view is in chapter 4.

3.1 Offsets and names

All offsets are from the base of the NAND controller (FMC) register window, `0x38a00000`. Register names are descriptive labels for window offsets. Registers that are not listed are not touched by the original programs.

3.2 Memory map: the register window

The sequencer works with three address spaces; the first is the register window described here, the other two are system memory and the program itself (chapter 4, section 4.5).

1. The register window is 4 KB at CPU address `0x38a00000`. Instructions name a word in it by its offset:

- `wr`, `st` and `ld` carry the offset in `arg`.
- `pld` and `pst` take the offset from a register and wrap it to 12 bits, so they reach any word of the window, r0..r7 included.

The CPU sees the same words at `0x38a00000 + offset`. Below `0xc00`, offset bit 9 is not decoded. Some ranges are therefore **mirrors**: the same registers under a second address, not separate ones (device, chapter 16, section 16.2):

offsets	what is there	kind
`0x000..0x0fc`	first register set: the NAND controller the programs use (chapter 9)	registers
`0x100..0x1fc`	a block whose purpose is unknown; the words at `0x1b8..0x1d8` are writable (at power-on `0x1cc` = `0x8060`, `0x1d0` = `0x3010d0`, and the other writable words are all ones); the rest of the block is read-only	registers
`0x200..0x3fc`	`0x000..0x1fc` again: every word reads the same and every write mask is the same (CPU and sequencer writes, chapter 16, section 16.2)	**mirror** of `0x000..0x1fc`
`0x400..0x4fc`	**second register set**: a second, independent NAND controller with the same register layout (chapter 9, section 9.5). Not a mirror: it holds its own values	registers
`0x500..0x5fc`	read from the sequencer, every word equals the word `0x400` lower in the `0x100` block (`0x100..0x1fc`)	mirror of the `0x100` block (inferred from the equal contents)
`0x600..0x7fc`	`0x400..0x5fc` again (same reads, same write masks)	**mirror** of `0x400..0x5fc`
`0x800..0x8fc`	ECC engine (chapter 10, section 10.1)	registers
`0x900..0x9fc`	further ECC registers: `0x900..0x90c` hold the 13 parity bytes after an encode (read-only), `0x940..0x950` are five 32-bit read/write words (no effect known), `0x9a0` changes during a decode, `0x9a4` is a decode counter, and `0x9c0` has a read/write bit 0 (1 at power-on)	registers
`0xa00..0xbfc`	`0x800..0x9fc` again (same reads, same write masks)	**mirror** of `0x800..0x9fc`
`0xc00..0xc5c`	sequencer control block (chapter 8, section 8.1): start word, program address, `CS_IRQ`, r0..r7 at `0xc18..0xc34`, PC, fault vector and others	registers
`0xc60..0xcfc`	nothing: reads 0, writes are lost	empty
`0xd00..0xd7c`	parameter RAM, 32 words shared by the CPU and the program (chapter 4, section 4.4)	RAM
`0xd80..0xdfc`, `0xe00..0xfffc`	nothing: reads 0, writes are lost	empty

Who can read and write what (device, access): every word was written with all ones and with zero, once from the CPU and once from the sequencer. Almost every register behaves the same for both. The differences are:

- The PC (`0xc38`): the sequencer can write it (a jump); the CPU can write it only while a program is stopped at pause.
- `C3C` (`0xc3c`), `C48` (`0xc48`) and `C54` (`0xc54`): a value written by the CPU before a start is not what the program reads, because the start clears these registers. A value that the program writes to `C48` or `C54` stays visible to the CPU after the run. `C3C` is cleared at the end of the run.

- After the CPU wrote the words 0x3a8..0x3b4, the sequencer read back the written values, but the CPU itself read 0 (unexplained).
- The status words (0x050, 0x250, 0x450, 0x650, FMC_28 and their copies) changed by themselves during the test, so the write masks measured on them carry no information.

3.3 CPU-only rules

Rules that hold for CPU accesses to the window, in order of offset (device unless stated). The full register table is in chapter 8; see also findings 55 and 103.

- 0xc08 (CS_STATUS): CPU writes are ignored.
- 0xc14 (FMSS_C14): neither CPU nor program writes change it.
- 0xc18..0xc34 (r0..r7): a value that the CPU writes before the start becomes the register's starting value; a start leaves the registers unchanged; the sequencer reset clears them.
- 0xc38 (FMSS_PC): the CPU can write it only while the program is stopped at pause (chapter 8, section 8.1, chapter 6, opcode 12); otherwise CPU writes are ignored.
- 0xc3c (FMSS_C3C): reads 0 after every run, whatever the CPU wrote.
- 0xc40, 0xc44: CPU writes are ignored; the sequencer reset clears them.
- 0xc48: 32 bits, writable by the CPU and by the program, not used by the original programs. A program start clears it, so the program does not see a value written by the CPU. A value written by the program stays after the end. No effect found.
- 0xc50 (FMSS_C50): CPU writes are ignored.
- 0xc54: 12 bits, writable by the CPU and by the program. A program start clears it; a value written by the program stays after the end. No effect found; it is not a timeout setting.
- 0xc60..0xcfc and 0xd80..0xdfc: read 0 and drop writes, from the CPU and from the program.

CPU write masks (row "CPU register masks" of chapter 16, section 16.2, device). Each mask is the set of bits that kept their value, found by reading each word back after a CPU write of all ones, of 0 and of 0xa5a5a5a5. The table gives the number of writable low bits, or an explicit mask.

block	writable bits per word	notes
sequencer	0xc04, r0..r7, 0xc3c, 0xc48, 0xc4c: 32 bits; 0xc10, 0xc58: 8 bits; 0xc54: 12 bits; 0xc5c: 1 bit	read-only: 0xc08, 0xc14, 0xc38, 0xc40, 0xc44, 0xc50; the rest of 0xc00..0xcfc reads 0
NAND controller (0x000..0x0fc)	0x000, 0x00c, 0x010, 0x018, 0x034, 0x060..0x078: 32 bits; 0x014: 16 bits; 0x020: 24 bits; 0x024, 0x028: 9 bits; 0x02c: 3 bits; 0x030: 13 bits; 0x044: 16 bits	
ECC engine	0x804, 0x808: 32 bits; 0x814: 25 bits; 0x844: mask 0x2f	0x800 reads 0x20060808 and ignores writes
second register set (0x400)	the layout of the first set again: 0x400..0x478 match 0x000..0x078 bit for bit; 0x438: 13 bits; 0x47c: mask 0xffc	0x404 has write mask 0x19; its bits 1, 2, 8 and 9 behave like status bits

3.4 CPU-visible interrupt lines

The original CPU driver polls the VIC1 raw status bit 22 (source 54) for the end of a run (RE-32). In the device tests that bit stayed clear after END, after a fault and with any start word. With a NAND controller or ECC interrupt enable set, only CS_IRQ bits 4 and 6 responded. Whether a further enable outside the sequencer block is needed is open (chapter 2, section 2.3).

Device results (row "interrupt line" of chapter 16, section 16.2):

- 0x044 **bit per source** (device, one operation per program with its flag left pending, irqop, device run 37):

0x044 **bit**	**source (**FMC_STATUS **flag)**	CS_IRQ **bit 4**	**VIC0 line 20**
0	command sent (bit 1)	set	set
1	address sent (bit 2)	set	set
2	data transfer done (bit 3)	set	set
11	DMA done (bit 20)	set	set
12	status latched (bit 23)	set	set

- Each bit routes exactly one flag; with no bit set nothing was raised. All NAND controller sources share VIC0 line 20 (interrupt source 20). The ECC engine's sources, enabled in 0x844, raise VIC1 line 11 (interrupt source 43): with 0x844 bit 0 set and an ECC_DONE flag pending, the raw status showed line 43 and no other new line (w2vic). In the status-latch program the 70h command flag, acknowledged before the latch, still left CS_IRQ bit 4 set with 0x044 bit 0, while VIC0 line 20 stayed clear: CS_IRQ bit 4 keeps a routed flag after the flag is acknowledged, and VIC0 line 20 follows a routed flag only while it is pending

(inferred from this program).

- After a power cycle and after a soft reset alike, VIC1 raw bit 22 stayed clear for an END, a fault, opcode `10` and a `12` stop, with and without bit 6 in the start word.
- With every VIC0 and VIC1 raw line recorded, no line followed END, faults or start-word bits.
- With `0x044` = `0xffff`, VIC0 line 20 was seen in the command and status cases. It was not seen in the address, transfer and DMA cases, although `CS_IRQ` bit 4 was set in those cases too. VIC1 line 22 was never seen.
- Single `0x044` bits that set a pending flag in `CS_IRQ` bit 4, by the kind of operation:
  - command: bit 0;
  - address: bits 0 and 1;
  - transfer: bits 0, 1, 2 and 11;
  - DMA: bit 11;
  - status latch: bits 0 and 13.
- These programs raised several flags each (for example, the address case also sent a command), which accounts for the extra bits (inferred from the one-flag table above).
- `0x044` bit 0 routes `FMC_STATUS` bit 1 into `CS_IRQ` bit 4; with a command byte sent, the other fifteen bits routed nothing.
- `0x844` bits 0, 2 and 5 route `ECC_DONE` into `CS_IRQ` bit 6; bits 1 and 3 route nothing, and bit 4 is not writable.
- Even with these enables set, the line stayed clear. Whether a further enable outside the sequencer block is needed is open.
- One source at a time after a soft reset (`finale`):
  - VIC0 line 20 appeared with a command and `0x044` bit 0, and with address cycles and bit 1.
  - It did not appear with a command and bit 1 or 2, with an address and bit 0, with a status latch and bit 0 or 13, or with a DMA and bit 11.
  - `CS_IRQ` bit 4 was set for a command with bit 0, an address with bit 1, a status latch with bit 0 and a DMA with bit 11.
- In the `irqsrc` tests, line 20 appeared with a command and bit 0 and with a status latch and bit 13, in place of an address and bit 1. The `CS_IRQ` bit 4 results also differed for an address with bit 0 and a status latch with bit 13. Line 20 is therefore not the same as `CS_IRQ` bit 4.
- Each case three times after a soft reset (`vic20`, `irq044`):
  - VIC0 line 20 appeared with a command and `0x044` bit 0 and with an address and bit 1, the same in all three repetitions of each case. It did not appear with a status latch (bit 0 or 13) or a DMA (bit 11).
  - In a program with a command, an address, a data read, a status latch and a DMA, with the flags acknowledged, `0x044` bits 0, 1, 2, 11 and 12 each set `CS_IRQ` bit 4. None of them raised VIC0 line 20.
  - VIC1 line 22 was never seen.

Related findings: 70, 82, 108, 140, 146, 150, 167. The `CS_IRQ` flags themselves are in chapter 8.

3.5 Starting and finishing a run

How the original CPU driver runs a program (RE-31, RE-32, RE-33, RE-34):

1. It writes the parameter words (chapter 12, section 12.1) and fills the lists in memory. The sequencer fetches the program and reads the lists from system memory. Its accesses are not coherent with the CPU data cache.
2. It writes the program's bus address to `FMSS_PROG` and `0xff` to `CS_IRQ`, then writes `0xfff5` to `FMSS_CTRL`.
3. It waits for completion by polling the raw VIC1 status bit 22 and reading `CS_IRQ`: bit 0 is END, bit 2 is a fault. It allows a run at most 2 s. On a fault or a timeout it resets the sequencer in this order: it writes `0x30000` to `FMSS_CTRL`, waits 1 ms, runs an idle program, clears `CS_IRQ` and writes 0 to `FMC_CTRL0`.
4. After a clean END the result is `SEQ_RESULT`. After a fault the result is the value that the fault handler left in r6.

A run that does not end within 2 s is reported as a timeout and followed by a sequencer reset.

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 2, chapter 4, chapter 8, chapter 9, chapter 12.

Chapter 4 — The register window as the sequencer sees it

Summary: The sequencer addresses the same register window as the CPU (segments and mirrors: chapter 3, section 3.2), and in addition reaches system memory through `ldi` and `sti` and fetches its own program from memory. This chapter collects what is specific to the sequencer's side: field widths, address decode, the register file, parameter RAM and system memory.

4.1 Window offsets and target widths

Field widths (device):

- The register field `R` is decoded in all 8 bits: register numbers 24 and 26 are not treated as 8 and 10 (chapter 4, section 4.3).
- Window offsets are 12 bits. `wr`, `st` and `ld` with `arg` = `0x1d70`, `0x8d70` or `0xfd70` all reached the word at `0xd70`, so bits 15:12 of `arg` are ignored. `pld` and `pst` wrap their register value at 12 bits in the same way.
- Branch targets and values written to r8 use more bits than window offsets.
  - A branch target is a byte offset from the program start. A target with bit 12, 16 or 24 set did not reach the label it was built from: the bit is not cut off, so the offset points beyond the program. With bit 12 or 16 the run then ended without reaching the label; with bit 24 the sequencer fetched from an address that held no valid instruction and timed out (`fwidth`).
  - A target with bit 31 set reached the label, so bit 31 is ignored.
- `jr` with any of bits 12, 16, 24 and 31 set in the register did not jump at all; execution continued with the next instruction.

4.2 Segments and mirrors, sequencer view

The segment table is in chapter 3, section 3.2. Device result (row "address decode (sequencer view)" of chapter 16, section 16.2), found by reading every word `0x000..0xffc` with `pld`:

- Offset bit 9 is not decoded: `0x200..0x3fc` repeats `0x000..0x1fc`, `0x600` repeats `0x400`, `0xa00` repeats `0x800` and `0xb00` repeats `0x900`.
- `0x400..0x5fc` is a second NAND controller (FMC) register set. The boot ROM also addresses its channels at a spacing of `0x400`.
- `0x100..0x1fc` is a separate block. The same block is visible again at `0x300`, `0x500` and `0x700`. The words `0x1bc..0x1fc` read all ones, except `0x1cc` = `0x8060` and `0x1d0` = `0x3010d0`.
- The word `0x9c0` reads 1. The words `0xe00..0xffc` read 0.

4.3 Register file r0..r7, the PC and register numbers above 7

Register file and PC (rows of the control-block table; the primary description is in chapter 8, section 8.1):

offset	name	meaning
`0xc18..0xc34`	r0..r7	the registers; r6 is `SEQ_RESULT`. A value that the CPU writes before the start becomes the register's starting value; a start leaves the registers unchanged; the sequencer reset clears them (device)
`0xc38`	`FMSS_PC`	byte offset of the current instruction while a program runs or waits; 0 after the end. It is register r8 (`0xc18 + 4 * 8`). The program reads it as r8 or with `ld`. Writing it (`movi r8`, `wr` or `st` to `[0xc38]`) is a jump. The CPU can write it only while the program is stopped at `pause` (chapter 6, opcode `12`); otherwise CPU writes are ignored (device)

Register numbers (device, every number from 8 to 255 in every role, `regsweep`; further samples `regnum`): the register fields hold 8 bits and are decoded in full. Before testing each role, the program loaded distinct values into r0..r7 and into the writable control words `C10`, `C48`, `C54` and `C3C`. This made it possible to tell apart three outcomes: the operand reads r0, the operand reads 0, and the operand reads the control word at `0xc18 + 4 × N`.

role	8	10	9 and 11..255
destination (`movi`, `mov`, `add`)	the PC: a write is a jump	written into all of r0..r7	no register and no window word changes
argument-field source (`mov`, ALU, immediate form)	the offset of the instruction itself	the destination register's own value	r0
argument-field source (ALU, register form)	the offset of the instruction itself	r0	r0

role	8	10	9 and 11..255
register-field source of `st`	see the first bullet below	see the first bullet below	see the first bullet below
tested register of `bnz` / `bz`	never branches	never branches	never branches (also `bz` with r0 = 0)
target register of `jr`	jumps to itself (the program never ends)	r0	r0
address register of `ldi`	reads a code-like word (`e90d1c00`)	r0	r0
address register of `pld`	reads 0	r0	r0

- `st [REG], rN` with N from 8 to 255 does not store the value of a fixed register. It stores the last value that passed through the register file: the value the instruction before it wrote into a register, or, if that instruction wrote no register, the register value it read (device, `stprev`; also `regnum` and `regsweep`). With N = 9, 14 and 200 alike, after:
  - `movi r0` / `r3` / `r7`, `add r2, r2, #1`, `mov r4, r1`, `ld r5, [...]`, `pld r6, [r0]`, `ldi r1, [r0]`: the value written to that register;
  - `st [0xd7c], r5`: r5's value (the register `st` read);
  - `wr [0xd7c], #...` and `nop`: r0's value (instructions whose register field is 0);
  - `wait 11`: the value of the instruction before the `wait` (`wait` leaves it unchanged).
- None of the numbers reads or writes a control word: the values preloaded into `C10`, `C48`, `C54` and `C3C` never appeared.
- r8 is the PC: `movi r8, #label` and `mov r8, r1` jump to the label, `mov r1, r8` returns the offset of that instruction (`r8`, device run 16, `r8`, device run 17).

No original program uses any register number above 7.

4.4 Parameter RAM

The parameter RAM consists of plain read/write words shared by the CPU and the program. It ends at `0xd7c`; `0xd80..0xdfc` read 0 and drop writes, from the CPU and from the program (device).

- The original CPU driver writes only the words of chapter 12, section 12.1 for the job being started, and rewrites every word it needs before each run.
- The words `0xd2c..0xd3c` and `0xd50..0xd7c` are never written by the original CPU driver. The original programs use them as scratch words, and the value they hold when a program starts is not defined.
- The word `0xd00` is read by the programs of release 1.1.3 that send the command `78h` (WRITE2P_78 and WRITE_78) as a row address (RE-28); what writes it is open (chapter 17).
- The original READ program uses `pld` and `pst` only on the words `0xd60..0xd7c`, a queue of chip masks (RE-6).
- The original programs keep working state in registers or in scratch words. They advance the pointer words in place, writing the updated value back to the same word.

4.5 System memory and the program

2. System memory (the CPU's bus addresses): `ldi` and `sti` take a full 32-bit bus address from a register and access a little-endian 32-bit word. An address that is not a multiple of 4 is rounded down to the word. The DMA of the NAND controller (chapter 9, section 9.2) also moves data between its buffers and bus addresses (`FMC_DMA_ADDR`). These are bus (physical) addresses, as the original CPU driver passes them (inferred: every address used in the tests is the same in the CPU's view, and a load from address 0 returned code-like words).

3. The program: fetched from bus address `FMSS_PROG` (`0xc04`) onwards, 8 bytes per instruction (chapter 5, section 5.1). The PC and the branch and jump targets are byte offsets from `FMSS_PROG`, not bus addresses. A program can sit in SDRAM (217 ns per instruction) or in internal SRAM (138 ns) (device, chapter 16, section 16.2).

4.6 What the sequencer sees differently from the CPU

The comparison of CPU and sequencer access (chapter 3, section 3.2) shows these differences:

- The sequencer can write the PC (a jump); the CPU can write it only while a program is stopped at `pause`.
- A value written by the CPU to `C3C`, `C48` or `C54` before a start is not what the program reads, because the start clears them. A value that the program writes to `C48` or `C54` stays visible to the CPU after the run. `C3C`

is cleared at the end of the run.

- After the CPU wrote the words `0x3a8..0x3b4`, the sequencer read back the written values, but the CPU itself read 0 (unexplained).
- The status words (`0x050`, `0x250`, `0x450`, `0x650`, `FMC_28` and their copies) changed by themselves during the test, so the write masks measured on them carry no information.

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 3, chapter 5, chapter 12.

Chapter 5 — Instruction encoding

Summary: The 8-byte instruction word, what each field means for each opcode, the opcode summary table, the ALU operand rule and the byte order. Details per opcode are in chapter 6; field widths are in chapter 4, section 4.1.

5.1 Encoding

```
word0 = opcode << 24 | R << 16 | arg        (arg: u16)
word1 = imm                                  (u32)
```

The instruction is stored little-endian; `word0` is bytes +0..+3 and `word1` is bytes +4..+7. The fields mean:

- `R` names a register (0..7) or an event number.
- `arg` is a window offset (in register-access instructions) or a register number (in register-operand instructions).
- `imm` is an immediate value, a mask or a branch target.

Every instruction is 8 bytes, whatever the opcode. A program is an array of instructions, and branch and jump targets are byte offsets into it. The bytes lie in memory as follows (bus order); the sequencer fetches from `FMSS_PROG + pc`:

```
byte:   +0      +1      +2      +3      +4      +5      +6      +7
        arg[7:0] arg[15:8] R    opcode  imm[7:0] imm[15:8] imm[23:16] imm[31:24]
```

For example `movi r6, #0x600d600d` is the bytes `00 00 06 05 0d 60 0d 60`, and `wr [0xc58], #4` is `58 0c 00 01 04 00 00 00`.

5.2 Operand map

What each field means for each opcode. **used**: the field carries the operand named. **ignored**: the field was set to different values on a device (such as `0xff` and offset-like values, for fields the original programs leave 0) and nothing changed. **0**: the field must be 0, because other values change the behaviour. Every field of every valid opcode has been tested; none is left unknown. Register numbers 8..255 in a register field are described in chapter 4, section 4.3.

op	mnemonic	`R` (byte +2)	`arg` (bytes +0..1)	`imm` (bytes +4..7)
`00`	`end`	0 (a non-zero value faults, as an illegal opcode)	0 (a non-zero value faults)	ignored
`01`	`wr`	ignored	used: window offset	used: value
`02`	`st`	used: source register	used: window offset	ignored
`03`	`ldi`	used: destination register	used: register holding the bus address	ignored
`04`	`ld`	used: destination register	used: window offset	used: AND mask
`05`	`movi`	used: destination register	ignored	used: value
`06`	`mov`	used: destination register	used: source register	ignored
`07`	`wait`	used: event number = which hardware condition to wait for (chapter 7, section 7.1; 8 bits)	ignored	ignored
`08`	`nop`	ignored	ignored	ignored
`09`	`wait2`	used: the number N of the interrupt source to wait for (VIC0 line N for N = 0..31, VIC1 line N - 32 for N = 32..63; chapter 7, section 7.2)	ignored	ignored
`0a..0d`, `13`, `14`	ALU	used: destination (and first source in the register form)	used: source register	used: immediate; 0 selects the register form
`0e`	`bnz`	used: tested register	ignored	used: branch target (byte offset)
`0f`	`jr`	ignored	used: register holding the target	ignored
`10`	`sig`	ignored	ignored	ignored
`11`	`sti`	used: source register	used: register holding the bus address	ignored
`12`	`pause`	ignored	ignored	ignored
`15`	`tmwait`	ignored	ignored	ignored
`16`	`evjmp`	ignored	ignored	ignored
`17`	`bz`	used: tested register	ignored	used: branch target (byte offset)

op	mnemonic	R (byte +2)	arg (bytes +0..1)	imm (bytes +4..7)
18	pld	used: destination register	used: register holding the window offset	ignored
19	pst	used: source register	used: register holding the window offset	ignored
1a..fe	(illegal)	ignored	ignored	ignored
ff	nop2	ignored	ignored	ignored

Field widths (device): R is decoded in all 8 bits, window offsets are 12 bits, and branch targets use more bits; the details are in chapter 4, section 4.1.

5.3 Opcodes

The mnemonics of the opcodes that no original program uses (nop, wait2, jr, sig, pause, tmwait, evjmp, nop2) are names given in these documents after their measured behaviour; the original programs give no names for them.

wait2 is named for behaving exactly like wait (same waiting state, timer and fault). Its register field selects an interrupt source of the CPU instead of one of wait's events: wait2 N continues once interrupt source N (VIC0 line N for N = 0..31, VIC1 line N - 32 for N = 32..63) is raised, and numbers 64..255 never pass. See chapter 7, section 7.2.

All 256 opcode values were run on a device, each in three operand forms (chapter 16, section 16.2). The **source** column says where the description of an opcode comes from:

- **original + device**: the original programs use it (encoding and use observed there), and device tests confirm its behaviour.
- **device only**: no original program uses it; everything known comes from device tests.
- **device (illegal)**: it faults in every form tried.

An opcode that stalls for the full timeout (about 4.87 ms) and then faults looks the same as a wait whose condition is never met. "Illegal" therefore means "no condition was found that makes the opcode do anything else".

op	mnemonic	semantics	source	notes
00	end	stop, raise CS_IRQ bit 0; r6 is the result	original + device	R and arg 0 (a non-zero value faults), imm ignored; SEQ_RESULT = r6
01	wr [REG], #imm	REG = imm	original + device	write to the register window (adds 37 ns to the instruction time)
02	st [REG], rR	REG = rR	original + device	
03	ldi rR, [rA]	rR = mem32[rA]	original + device	system memory, bus address in rA; little-endian; an unaligned address is rounded down to the word
04	ld rR, [REG], mask	rR = REG & mask	original + device	mask in imm; full word = 0xffffffff
05	movi rR, #imm	rR = imm	original + device	
06	mov rR, rA	rR = rA	original + device	
07	wait N	block until wait event N is set (chapter 7, section 7.1), else time out after about 4.87 ms	original + device	R = event number; all 8 bits count (16, 27, 139 and 255 do not act as events 0, 11 or 15); argument and immediate ignored; events are levels (a flag stays set while its condition holds) and a wait does not clear them
08	nop	no operation	device only	no visible effect in any operand form; costs one instruction time
09	wait2 N	like wait, but N selects an interrupt source of the CPU (chapter 7, section 7.2): it continues once source N is raised in the raw status of VIC0 (N = 0..31) or VIC1 (N = 32..63); otherwise it waits for the timeout and faults like an unmet wait	device only	arg and imm ignored; not a wait for chip ready and not a mask test. Details: chapter 6, opcode 09

op	mnemonic	semantics	source	notes
0a	and	ALU, see chapter 5, section 5.5	original + device	
0b	or	ALU	original + device	
0c	add	ALU	original + device	wraps mod 2^32
0d	sub	ALU	original + device	wraps mod 2^32
0e	bnz rR, target	if rR != 0: pc = imm	original + device	tests all 32 bits; argument ignored; the target is used as given (not rounded to a multiple of 8)
0f	jr rA	jump through a register: pc = r(arg)	device only	byte offset from the program start, not rounded; immediate ignored; no return address is kept, so together with movi rX, #return it provides calls and returns, and it also provides computed jumps
10	sig	signal the CPU: set CS_IRQ bit 1, continue	device only	a flag the CPU can see while the program runs
11	sti [rA], rR	mem32[rA] = rR	original + device	little-endian; unaligned rounded down
12	pause	stop and hand control to the CPU (CS_IRQ bit 3); the CPU continues the program by writing the PC and a start word	device only	fields ignored; details: chapter 6, opcode 12
13	shl	ALU, logical left	original + device	a count of 32 or more gives 0
14	shr	ALU, logical right	original + device	a count of 32 or more gives 0
15	tmwait	wait for the timeout timer C14	device only	if the timer is non-zero: reload it with 0xff and wait about 4.85 ms until it reaches 0; if the timer is 0: reload it with 0xff at once (chapter 8, section 8.1.6)
16	evjmp	event jump: wait until any event flag is set (in either NAND controller (FMC) register set, in the ECC engine or in the status flags), then jump to the fault vector in C4C; if no flag is set within the timeout, a fault is raised	device only	fields ignored; no fault is raised when it jumps; the flag stays set. Details: chapter 6, opcode 16
17	bz rR, target	if rR == 0: pc = imm	original + device	as bnz
18	pld rR, [rA]	rR = window word at byte offset rA	original + device	3G only; the originals use parameter RAM offsets only; the offset wraps at 12 bits and reaches every word of the window, r0..r7 included
19	pst [rA], rR	window word at byte offset rA = rR	original + device	3G only; as pld
1a..fe		illegal	device (illegal)	every value in the range stalls for the timeout and then faults; the opcode byte is decoded in all 8 bits (see "Opcode decoding" below)
ff	nop2	no operation	device only	as 08: an all-ones word (erased or 0xff-filled memory) is a no-op, not an end

Opcode decoding. The opcode byte counts in all 8 bits. 20, 40 and 80 are 00 (end) with one high bit set. A decoder that looked only at the low bits would treat them as end. On the device they stall and fault like every other illegal value, and all three behave alike. 21 (01 with bit 5 set) likewise does not act as wr. No instruction has a second encoding (chapter 6, the opcode values without a meaning).

Instruction time (device): the time is set by fetching the 8-byte instruction, 217 ns from SDRAM and 138 ns from internal SRAM. A register-window access adds about 35 ns, and ldi and sti add 120 to 220 ns (chapter 16, section 16.2).

5.4 Illegal opcodes

The values 1a..fe are the illegal opcodes: they fault in every form tried (table above and chapter 6). Opcodes 09 (wait2) and 16 (evjmp) fault in most tests and so resemble illegal opcodes, but each has a

condition under which it passes. They are valid opcodes (chapter 6, chapter 7). Register numbers above 7 in a register field are described in chapter 4, section 4.3.

5.5 ALU operand rule

- Immediate form (`imm != 0`): `rR = r(arg) OP imm`. The source register is in `arg`, the destination in `R`.
- Register form (`imm == 0`): `rR = rR OP r(arg)`.
- An immediate of 0 is indistinguishable from the register form, so no instruction encodes "`rD = rA op #0`"; `mov` performs that copy.
- The original programs have no unconditional jump. They branch on a register known to be non-zero, or they pair `bz` and `bnz` on one register. The hardware does have a jump through a register, opcode `0f` (chapter 5, section 5.3), which also gives computed jumps and subroutine returns.
- There is no compare instruction. The original programs test equality by subtracting and branching on the difference.

5.6 Byte order

The FMSS engine is little-endian throughout, with one exception in the ECC engine's parity:

- Instruction fields and words: the instruction is stored little-endian (chapter 5, section 5.1); `ldi` and `sti` access little-endian 32-bit words (chapter 4, section 4.5); the parameter and result words are little-endian words (chapter 12).
- ECC area: the words `0x900..0x90c` hold the parity bytes in little-endian word order, which is the reverse of the order in the parity slot (chapter 10, section 10.14).
- Parity in the slot and on the flash: the parity bytes are stored most significant byte first (chapter 10, section 10.14).

The reading of why the parity order differs (inferred) is in chapter 15, section 15.7.

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 4, chapter 6, chapter 7, chapter 10.

Chapter 6 — Opcode reference

Summary: One entry per valid opcode, in opcode order, followed by one entry for the opcode values that have no meaning. The entries are headed by the opcode and its mnemonic and are not numbered. The summary table is in chapter 5, section 5.3.

6.1 Conventions and measurement notes

The fields are those of chapter 5, section 5.1: `R` (8 bits), `arg` (16 bits) and `imm` (32 bits). Statements marked "device" give behaviour measured on a unit; times are per instruction with the program in SDRAM unless stated. Each entry says whether the original programs use the opcode (RE-4 lists the opcodes that the nine original programs contain). The mnemonics of the opcodes that no original program uses (`08`, `09`, `0f`, `10`, `12`, `15`, `16`, `ff`) are names given in these documents after the measured behaviour. The source of a description is **original + device** (used by the original programs and confirmed on a device) or **device only** (no original program uses it).

6.2 Opcodes

`00` end

- Stop the program.
- Encoding: `R` and `arg` are 0 in the original programs. The original programs encode `end` as two zero words.
- If `R` = 1, if `arg` = 1, or if both are set, the instruction does not end the program. It faults like an illegal opcode instead (it runs the fault vector and sets `CS_IRQ` bit 2) (device).
- `imm` is ignored: `0x12345678` ended the program normally. The two words of the `end` instruction, including `imm`, are left in `0xc40` and `0xc44` (device).
- Effect: the program stops. `CS_IRQ` bit 0 is set, and `SEQ_RESULT` (`0xc30`) shows the value of r6. Afterwards `FMSS_PC` reads 0, `C08` returns to 0 and the timer `C14` stops counting.
- If `C58` bit 0 is set, an `end` also sets `C5C` and leaves a latch. With the latch set, every later `end` runs the fault vector once (chapter 8, section 8.1.15).
- If the one-shot is armed by `C58` bit 1, one `end` is diverted to the fault vector (device).
- Original programs: used by every original program.
- Evidence: `fields0`, `oneshot`, `pending`, device run 5, `pending`, device run 6, `regmap`, device run 1, `regmap`, device run 6, `regmap`, device run 7, RE-4.

`01` wr

- Write an immediate value to a register of the register window.
- Syntax: `wr [REG], #imm`.
- Encoding: `arg` = window offset, `imm` = value; `R` is ignored. The window offset selects a register of the NAND controller (FMC), of the ECC engine, of the sequencer block or in the parameter RAM. The offset is 12 bits: bits 15:12 of `arg` are ignored (device).
- Effect: writes `imm` to the register at that offset. For offsets below `0xc00`, bit 9 of the offset is ignored (chapter 3, section 3.2).
- The write has the register's own side effects. A command byte written to `FMC_CMD` is sent to the chip. A code written to `FMCTRL1` starts an operation.
- Timing: about 37 ns more than `movi` (device).
- Original programs: used for every fixed register value.
- Evidence: `opcost`, `fwidth`, `fields0`, RE-4.

`02` st

- Store a sequencer register to a register of the register window.
- Syntax: `st [REG], rR`.
- Encoding: `arg` = window offset, `R` = source register; `imm` is ignored.
- Effect: writes the value of register rR to the window register.
- With `R` from 8 to 255, `st` stores the last value that passed through the register file: the value the previous instruction wrote into a register, or the register value the previous instruction read (r5 after `st [..], r5`; r0 after `nop` or `wr`, whose register field is 0); `wait` leaves it unchanged (device, `stprev`; details in chapter 4, section 4.3).

- Timing: as `wr`.
- Original programs: used (control words built in a register, list pointers into parameter RAM).
- Evidence: `regnum`, `regsweep`, `fwidth`, `fields0`, `opcost`.

03 `ldi`

- Load a word from system memory.
- Syntax: `ldi rR, [rA]`.
- Encoding: `arg` = register holding a bus address, `R` = destination; `imm` is ignored.
- Effect: a 32-bit little-endian load from system memory. An unaligned address is rounded down to the word boundary (device).
- If `arg` is a register number from 9 to 255, r0 supplies the address. If `arg` is 8, the load returns the word `e90d1c00`, which looks like code, as a load from a low address does (device, `regsweep`).
- Timing: about 170 ns more than `movi` from SDRAM, 218 ns more from internal SRAM (device).
- Original programs: used to read the lists of values supplied for each request (chip-enable lists, DMA address lists).
- Evidence: `memory`, `regnum`, `regsweep`, `opcost`, `fields0`.

04 `ld`

- Read a window register and keep only the bits selected by a mask.
- Syntax: `ld rR, [REG], mask`.
- Encoding: `arg` = window offset, `imm` = mask (`0xffffffff` keeps the whole word), `R` = destination register.
- Effect: rR = (window register) AND mask. Reading a status register does not clear any flag (device).
- Timing: as `wr`.
- Original programs: used for status tests and read-modify-write of `FMC_CTRL0`.
- Evidence: `alu`, device run 1, `alu`, device run 2, `alu`, device run 3, `evclear`, `fwidth`, RE-9.

05 `movi`

- Load an immediate.
- Syntax: `movi rR, #imm`.
- Encoding: `R` = destination, `imm` = value; `arg` is ignored.
- Effect: rR = imm. Register numbers above 7 behave differently (device, `regsweep`):
  - `R` = 8 jumps to `imm`, because r8 is the program counter.
  - `R` = 10 loads all eight registers r0..r7.
  - `R` = 9 and `R` = 11..255 change no register and no window word.
- Timing: the base instruction time, 217 ns (SDRAM) / 138 ns (SRAM) (device).
- Original programs: used (constants, markers, loop counters).
- Evidence: `r8`, device run 16, `r8`, device run 17, `regnum`, `opcost`, `fields0`.

06 `mov`

- Copy a register.
- Syntax: `mov rR, rA`.
- Encoding: `R` = destination, `arg` = source register; `imm` is ignored.
- Effect: rR = r(arg). Register numbers above 7 behave differently (device, `regsweep`):
  - As a source, `arg` = 8 reads the byte offset of the current instruction.
  - As a source, `arg` = 10 reads the destination register.
  - As a source, `arg` = 9 and `arg` = 11..255 read r0.
  - As a destination, `R` = 8 jumps and `R` = 10 loads all eight registers, as for `movi`.
- Original programs: used.
- Evidence: `regnum`, `r8`, device run 16, `r8`, device run 17, `fields0`.

07 `wait`

- Wait for an event.
- Syntax: `wait N`.

- Encoding: `R` = event number (8 bits); `arg` and `imm` are ignored (device; the 4G/5G form with a register and mask does not exist here).
- Effect: if `wait` event N is set, the program continues at once. Otherwise the sequencer waits.
- The wait ends with a timeout fault if the event is still not set after the timeout. The timeout is set by the start word: about 4.87 ms with the usual start word `0xfff5`, and no timeout at all without start-word bit 2 (chapter 8, section 8.1.1).
- The timeout fault sets `CS_IRQ` bit 2. The program then continues at the fault vector if `C58` bit 2 is set, and at the next instruction otherwise.
- While waiting, `C08` = 4 and the timer `C14` counts down from `0xff`.
- Events are levels: an event stays set while its condition holds, and a wait that passes does not clear it (chapter 7, section 7.1). A `wait` for an event number above 15 never passes.
- Timing: a wait that passes costs the base instruction time.
- Original programs: used after every command, address, transfer, DMA, ECC operation and status poll.
- Evidence: `wait`, device run 1, `wait`, device run 4, `waitf`, `c58`, device run 2, `c58`, device run 3, `ctrlfx`, `evcons`, RE-7.

08 nop

- No operation.
- Encoding: all fields ignored.
- Effect: no visible effect in any operand form. A pending flag of either NAND controller register set or of the ECC engine does not change this (device). An all-ones word (erased or `0xff`-filled memory) is `ff`, not `08`; see `ff`.
- Timing: the same as `movi`.
- Original programs: not used.
- Evidence: `unknown`, device run 1, `unknown`, device run 2, `unknown`, device run 3, `optime`, `opstate`, `opsweep`.
- Note: two nop opcodes exist, `08 nop` and `ff nop2`; in every test they behave the same way (no visible effect, same time). Why there are two is open.

09 wait2

- Wait for an interrupt line of the CPU: `wait2 N` continues once interrupt source N is raised in the raw status of the CPU's interrupt controllers (VIC0 line N for N = 0..31, VIC1 line N - 32 for N = 32..63); numbers 64..255 never pass (device, `w2vic`; chapter 7, section 7.2).
- Syntax: `wait2 N`.
- Encoding: `R` = the number N of the `wait2` condition to wait for (chapter 7, section 7.2). `arg` and `imm` are ignored: ten operand forms, some matching the status byte and some not, made no difference.
- Effect (device): which conditions were true:
  - `wait2` condition 5 (VIC0 line 5, a source outside the FMSS engine) was true in every state tried, so the program continues at once.
  - `wait2` condition 20 (VIC0 line 20, the NAND controller's line) is true while a NAND controller flag is pending and enabled in `0x044` (`w2state`, `w2vic`).
  - `wait2` condition 43 (VIC1 line 11, the ECC engine's line) is true while an `ECC_DONE` flag is pending and enabled in `0x844` (`w2vic`).
  - `wait2` condition 33 passed while the chip was idle in `op09w33` and during an ECC operation and a DMA in `finale`; it did not pass in `op09nand` (idle and during an erase), `evsweep2`, `ctrlwords`, `w2state`, `blk100b` or `w2vic`. Condition 33 is VIC1 line 1, a source outside the FMSS engine.
  - For every other N (0..255), the sequencer waits about 4.87 ms in the waiting state (`C08` = 4) and then raises the timeout fault, as for a `wait` whose event is never set. The timer `C14` is handled as for `wait`.
- `wait2` condition 5 continued at once in every state tried: status flag pending, status flag acknowledged, and during a block erase while the chip reported busy (status `0x81`). In the same block-erase state, `wait` event 5 waited 180 us for ready.
- From a clean state with nothing pending, N = 0..31 with no chip, CE0 or CE1 selected: only condition 5 passed. In the same state `wait` passed only for `wait` event 11.
- Against chip activity (`op09nand`), N = 0..31 was tried during each of these states, and only condition 5 passed in all of them:
  - while a block erase runs, and right after it;

    - while a page program runs, and right after it;
    - during the busy time of a page read.
- N = 32..255 while idle and during an erase: none passed (`op09nand`). N = 0..63 while an ECC operation and a DMA run: only conditions 5 and 33 passed (`finale`).
- Pending flags of the second register set made no difference.
- In `evsweep2`, N = 0..255 was tried with a short timeout, in forward and in reverse order:
    - In each of these states only condition 5 passed: idle, erase busy, program busy, read busy, ECC operation in flight, DMA in flight, and after a command to the second register set.
    - With every enable on and a command flag pending, conditions 5 and 20 passed.
    - In the same series, `wait2` condition 33 also timed out in three other programs (`ctrlwords`, `c08self`).
- Original programs: not used.
- Evidence: `op09`, `op09b`, `op09c`, `op09d`, `op09nand`, `op09w33`, `op0916`, `finale`, `evsweep2`.

0a and

- Bitwise AND.
- Encoding: `R` = destination register (and first source register in the register form), `arg` = source register, `imm` = immediate. `imm` = 0 selects the register form, in which the second operand comes from a register instead of from `imm`.
- Effect: bitwise AND. With a non-zero immediate, `rR = r(arg) AND imm` (source register in `arg`, destination in `R`); with an immediate of 0 the register form `rR = rR AND r(arg)` applies (chapter 5, section 5.5). Register numbers of 8 and above behave as for `mov` (chapter 4, section 4.3).
- Timing: the base instruction time, as `movi` (217 ns from SDRAM, 138 ns from internal SRAM).
- Original programs: used (counts in RE-4).
- Evidence: `alu`, device run 1, `alu`, device run 2, `alu`, device run 3, `regnum`.

0b or

- Bitwise OR.
- Encoding: `R` = destination register (and first source register in the register form), `arg` = source register, `imm` = immediate. `imm` = 0 selects the register form, in which the second operand comes from a register instead of from `imm`.
- Effect: bitwise OR. With a non-zero immediate, `rR = r(arg) OR imm` (source register in `arg`, destination in `R`); with an immediate of 0 the register form `rR = rR OR r(arg)` applies (chapter 5, section 5.5). Register numbers of 8 and above behave as for `mov` (chapter 4, section 4.3).
- Timing: the base instruction time, as `movi` (217 ns from SDRAM, 138 ns from internal SRAM).
- Original programs: used (counts in RE-4).
- Evidence: `alu`, device run 1, `alu`, device run 2, `alu`, device run 3, `regnum`.

0c add

- Addition.
- Encoding: `R` = destination register (and first source register in the register form), `arg` = source register, `imm` = immediate. `imm` = 0 selects the register form, in which the second operand comes from a register instead of from `imm`.
- Effect: addition, wrapping at 32 bits (device). With a non-zero immediate, `rR = r(arg) + imm` (source register in `arg`, destination in `R`); with an immediate of 0 the register form `rR = rR + r(arg)` applies (chapter 5, section 5.5). Register numbers of 8 and above behave as for `mov` (chapter 4, section 4.3).
- Timing: the base instruction time, as `movi` (217 ns from SDRAM, 138 ns from internal SRAM).
- Original programs: used (counts in RE-4).
- Evidence: `alu`, device run 1, `alu`, device run 2, `alu`, device run 3, `regnum`, `opcost`.

0d sub

- Subtraction.
- Encoding: `R` = destination register (and first source register in the register form), `arg` = source register, `imm` = immediate. `imm` = 0 selects the register form, in which the second operand comes from a register instead of from `imm`.

- Effect: subtraction, wrapping at 32 bits (device). With a non-zero immediate, `rR = r(arg) - imm` (source register in `arg`, destination in `R`); with an immediate of 0 the register form `rR = rR - r(arg)` applies (chapter 5, section 5.5). Register numbers of 8 and above behave as for `mov` (chapter 4, section 4.3).
- Timing: the base instruction time, as `movi` (217 ns from SDRAM, 138 ns from internal SRAM).
- Original programs: used (counts in RE-4).
- Evidence: `alu`, device run 1, `alu`, device run 2, `alu`, device run 3, `regnum`.

0e bnz

- Branch if non-zero.
- Syntax: `bnz rR, target`.
- Encoding: `R` = tested register, `imm` = target (byte offset from the program start); `arg` is ignored (device).
- Effect: branch if rR is non-zero. All 32 bits are tested, so `0x80000000` counts as non-zero.
- The target is used as given. An offset that is not a multiple of 8 starts decoding in the middle of an instruction: a target 4 bytes past an instruction boundary lands mid-instruction, and a target 8 bytes before the intended label runs the instruction before the label.
- With any `R` from 8 to 255 (8 included), the branch was never taken, whether r0 was zero or non-zero (device, `regsweep`).
- The target is not cut to 12 bits: with bit 12, 16 or 24 set it points beyond the program, and the branch does not reach the label the low bits name. Target bit 31 is ignored (chapter 4, section 4.1).
- Timing: not taken, as `movi`.
- Original programs: used for every loop and condition; they have no unconditional branch.
- Evidence: `branch`, `regnum`, `regsweep`, `fwidth`.

0f jr

- Jump through a register.
- Syntax: `jr rA`.
- Encoding: `arg` = register holding the target (byte offset from the program start); `R` and `imm` are ignored.
- Effect (device): the program counter is set to the value of r(arg), without rounding. No return address is saved.
- `arg` = 9..255 jumped through r0, and so did `arg` = `0xd70` (a value above 255 in the 16-bit field). `arg` = 8 jumps to the `jr` instruction itself, so the program never ends (`regsweep`).
- If a `movi rX, #return` runs before the jump, a `jr` through rX continues at `return`. The opcode therefore provides computed jumps and subroutine returns.
- If the register value has bit 12, 16, 24 or 31 set, no jump happens.
- Original programs: not used.
- Evidence: `op0f`, `indirect`, device run 1, `indirect`, device run 2, `indirect`, device run 3, `fwidth`, `regnum`.

10 sig

- Signal the CPU (`CS_IRQ` bit 1).
- Encoding: all fields ignored.
- Effect (device): sets `CS_IRQ` bit 1 and continues with the next instruction, also when a flag is pending. The CPU can read the flag while the program runs. The CPU interrupt line was not raised.
- Original programs: not used.
- Evidence: `op10`, `opstate`, `irqline`, device run 15, `irqline`, device run 16, `irqline`, device run 17.

11 sti

- Store a word to system memory.
- Syntax: `sti [rA], rR`.
- Encoding: `arg` = register holding a bus address, `R` = source; `imm` is ignored.
- Effect: a 32-bit little-endian store of rR to system memory. An unaligned address is rounded down to the word boundary (device).
- Timing: about 186 ns more than `movi` from SDRAM, 120 ns more from internal SRAM (device).
- Original programs: used to store the result and the metadata of each request.
- Evidence: `memory`, `opcost`, `fields0`.

12 pause

- Hand control to the CPU.
- Encoding: all fields ignored; pending flags make no difference.
- Effect (device): `pause` stops the program at this instruction and raises `CS_IRQ` bit 3. Its fields are ignored. It acts as a breakpoint: the CPU continues the program by writing the PC (`0xc38`) and a start word.
- While stopped: `CS_IRQ` bit 3 is set, `C08` = `0xa`, the instruction is latched in `0xc40` and `0xc44`, and `FMSS_CTRL` bit 18 stays set. There is no timeout.
- Continuing: the CPU writes the PC (`0xc38`) with the byte offset at which to continue (normally the offset of the `pause` itself + 8) and then writes a start word (`0xfff5`). The program runs on from that offset. In the measurement the pause lasted 2 ms, and the `end` ran 25 us after the CPU's writes.
- The CPU does not need to acknowledge `CS_IRQ` bit 3 to continue. If it does not, bit 3 stays set.
- If the CPU writes a start word without writing the PC, the sequencer continues at the `pause` instruction itself, which pauses again.
- The CPU can write the PC only while the sequencer is stopped at a `pause`. The write is ignored when the sequencer is idle.
- Resetting the sequencer also ends the pause. Eighteen other kinds of CPU write were tried, and none of them released it.
- The 4G/5G driver uses the same sequence for its "pause" (`FMSS_PC += 8; CS_IRQ = 8; FMSS_CTRL |= 1`).
- Original programs: the 3G original programs do not use it.
- Evidence: `op12`, `op12wait`, `op12ack`, `op12rel`, `op12cont`, `pause`, `ctrlhold`, `c08`.

13 shl

- Logical shift left.
- Encoding: `R` = destination register (and first source register in the register form), `arg` = source register, `imm` = immediate. `imm` = 0 selects the register form, in which the second operand comes from a register instead of from `imm`.
- Effect: logical shift left. A shift count of 32 or more gives 0, whether the count is the immediate or comes from a register (device). With a non-zero immediate, `rR = r(arg) << imm` (source register in `arg`, destination in `R`); with an immediate of 0 the register form `rR = rR << r(arg)` applies (chapter 5, section 5.5). Register numbers of 8 and above behave as for `mov` (chapter 4, section 4.3).
- Timing: the base instruction time, as `movi` (217 ns from SDRAM, 138 ns from internal SRAM).
- Original programs: used (counts in RE-4).
- Evidence: `alu`, device run 1, `alu`, device run 2, `alu`, device run 3, `holdout`, device run 9, `holdout`, device run 10.

14 shr

- Logical shift right.
- Encoding: `R` = destination register (and first source register in the register form), `arg` = source register, `imm` = immediate. `imm` = 0 selects the register form, in which the second operand comes from a register instead of from `imm`.
- Effect: logical shift right (not arithmetic, so the top bits are filled with 0). A shift count of 32 or more gives 0, whether the count is the immediate or comes from a register (device). With a non-zero immediate, `rR = r(arg) >> imm` (source register in `arg`, destination in `R`); with an immediate of 0 the register form `rR = rR >> r(arg)` applies (chapter 5, section 5.5). Register numbers of 8 and above behave as for `mov` (chapter 4, section 4.3).
- Timing: the base instruction time, as `movi` (217 ns from SDRAM, 138 ns from internal SRAM).
- Original programs: used (counts in RE-4).
- Evidence: `alu`, device run 1, `alu`, device run 2, `alu`, device run 3.

15 tmwait

- Wait until the timeout timer reaches 0.
- Encoding: all fields ignored; pending flags make no difference.
- Effect (device): if the timer `C14` is non-zero, `tmwait` reloads the timer from bits 15:8 of the start word (`0xff` with `0xfff5`). It then waits until the timer reaches 0 (about 4.85 ms) and leaves it at 0. If the timer is 0,

`tmwait` reloads it and continues at once. Two `tmwait` in one program made the whole program run 4841, 18 and 3250 us in three runs with different values written to `C10`, `C48` and `C54` before it (`ctrlwords`; not explained). Four `tmwait` in one program alternated regularly in five repetitions: `C14` read 0, `0xff`, 0, `0xff` after them, and each repetition took about 9.7 ms (`tmwait2`).

- Original programs: not used.
- Evidence: `op15`, `c14`, `opstate`.

16 `evjmp`

- Event jump: wait for any event flag, then jump to the fault vector.
- Encoding: no field is used.
- Effect (device): `evjmp` waits until any event flag of either NAND controller register set or of the ECC engine is set. The events are 0..9: command sent, address sent, DMA done, transfer done, ECC done, status latched, and the second register set's flags. It then continues at the fault vector `C4C`.
- No fault is raised: after the `end` at the vector, `CS_IRQ` holds only bit 0.
- If no flag is pending, `evjmp` waits about 4.87 ms in the waiting state and raises the timeout fault like an unmet `wait`. It then continues as for any fault: at the fault vector if `C58` bit 2 is set, at the next instruction if it is clear.
- Each of the following sent the program to the vector within about 20 us: a command flag, a DMA flag, an `ECC_DONE` bit 0, a status flag, `ECC_DONE` bits left by a page read, and the command and DMA flags of the second register set (events 6 and 8). With every flag acknowledged, `evjmp` timed out.
- `evjmp` also reacts to a flag that is set during the wait: a command written just before took 19 us, and a DMA started just before took 22 us.
- The target is whatever `C4C` holds. `C58` makes no difference to the target. The flag is still set at the target, and `C3C` is not written.
- In the opcode sweep it always timed out because no flag was pending there.
- Original programs: not used.
- Evidence: `op16`, `op16b`, `opstate`, `op0916`, `opsweep`.

17 `bz`

- Branch if zero.
- Syntax: `bz rR, target`.
- Encoding: `R` = tested register, `imm` = target (byte offset from the program start); `arg` is ignored (device).
- Effect: branch if rR is zero; all 32 bits are tested. Target behaviour as for `bnz` (`0e`); with any `R` from 8 to 255 it never branched, also with r0 = 0 (device, `regsweep`).
- Timing: not taken, as `movi`.
- Original programs: used (51 uses, RE-4).
- Evidence: `branch`, `regnum`.

18 `pld`

- Load a window word at a computed offset.
- Syntax: `pld rR, [rA]`.
- Encoding: `arg` = register holding a window byte offset, `R` = destination; `imm` is ignored.
- Effect (device): rR = the word of the register window at byte offset rA, where rA is the value of the register named by `arg`.
- The offset wraps at 12 bits (`0x1d64` reads `0xd64`). Every word of the window can be read this way, including the registers r0..r7: a `pld` of offset `0xc18` returns r0.
- If `arg` is a register number from 9 to 255, r0 supplies the offset (`regsweep`).
- This opcode is present on the 3G only.
- Timing: as `wr`.
- Original programs: used twice, in READ, on parameter RAM `0xd60..0xd7c` (a queue of chip masks) (RE-6).
- Evidence: `indirect`, device run 1, `indirect`, device run 2, `indirect`, device run 3, `fmcmap`, `fields0`, `opcost`, RE-6.

19 `pst`

- Store a window word at a computed offset.
- Syntax: `pst [rA], rR`.
- Encoding: `arg` = register holding a window byte offset, `R` = source; `imm` is ignored.
- Effect: stores rR into the word of the register window at byte offset rA, where rA is the value of the register named by `arg` (device). The offset is as for `pld`: it wraps at 12 bits, and every word of the window can be reached.
- This opcode is present on the 3G only.
- Timing: as `wr`.
- Original programs: used once, in READ, on parameter RAM `0xd60..0xd7c` (RE-6).
- Evidence: `indirect`, device run 1, `indirect`, device run 2, `indirect`, device run 3, `fmcmap`, `fields0`, `opcost`, RE-6.

`ff` nop2

- No operation.
- Encoding: all fields ignored.
- Effect: as `08`: no visible effect in any operand form, the same time as `movi`, no reaction to a pending flag (device). An all-ones word (erased or `0xff`-filled memory) therefore runs as a no-operation, not as an end.
- Original programs: not used.
- Evidence: `unknown`, device run 1, `unknown`, device run 2, `unknown`, device run 3, `optime`, `opstate`, `opsweep`.

`1a..fe` and other values without a meaning

- Illegal opcodes.
- Effect (device): all 256 opcode values were run, each in three operand forms. The values `1a..fe` fault in every form tried: the instruction stalls for the timeout (about 4.87 ms in the waiting state, `C08` = 4) and then raises the timeout fault. None of these values changed a register, RAM, parameter RAM or any window word.
- The opcode byte is decoded in all 8 bits, so no opcode has a second encoding. `20`, `40` and `80` are `00` (`end`) with one extra high bit set (`0x20` = `0010 0000`, `0x40` = `0100 0000`, `0x80` = `1000 0000`). If the hardware decoded only the low bits of the opcode byte, these three would act as further copies of `end`. They do not: each behaves like every other value in this range and like each other, stalling for the timeout (about 4.87 ms) and then faulting.
- The same holds for `21`, which is `01` (`wr`) with bit 5 set: it does not act as `wr`. The event number of `wait` is decoded in all 8 bits in the same way: `wait` events 16, 27, 139 and 255 do not act as `wait` events 0, 11 or 15.
- A fault that comes after the full timeout of about 4.87 ms in the waiting state cannot by itself distinguish an illegal opcode from a wait whose condition was never met. "Illegal" here means that no condition was found that makes the value do anything else.
- The valid opcodes are the 19 used by the original programs plus `08`, `09`, `0f`, `10`, `12`, `15`, `16` and `ff`.
- Original programs: not used.
- Evidence: `opsweep`, `unknown`, device run 1, `unknown`, device run 2, `unknown`, device run 3, `op0916`, `c58ill`.

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 5, chapter 7, chapter 8.

Chapter 7 — Events, waits and the timeout

Summary: The hardware flags that `wait` waits for, the conditions that `wait2` waits for, the rules of `wait` event 5, the length of the timeout and the timeout timer `C14`. Each event was identified from the status bit that the original programs pair with the corresponding `wait`, and was then mapped on a device. No data sheet was used.

Two separate number lists. `wait` and `wait2` both take a number, but the numbers name different things.

- `wait` event 5 is the status-latch event (chapter 7, section 7.1). It passes when a status latch completes.
- `wait2` condition 5 is an entry of the separate list of `wait2` (chapter 7, section 7.2). It is always true and has nothing to do with the status latch.

These documents always write "`wait` event N" and "`wait2` condition N" in full, so that the two lists are not confused.

7.1 Wait events

A `wait` blocks the program until a flag of the NAND controller (FMC) or of the ECC engine is set. The flags are sticky: a flag stays set until the program clears it by writing 1 to its bit (write-1-to-clear). A `wait` on a flag that is already set returns immediately.

The original programs clear each flag after the `wait` that used it, and begin each run with all flags clear (C9). A flag left set by an earlier operation lets the next `wait` return before the data has moved (observed by the third-party driver).

event	flag	set by the hardware when	clear with
0	`FMC_STATUS` bit 1 (`0x2`)	a command byte written to `FMC_CMD` has been sent to the NAND	`FMC_STATUS = 0x2`
1	`FMC_STATUS` bit 2 (`0x4`)	the address cycles started with `FMCTRL1 = 1` have been sent	`FMC_STATUS = 0x4`
2	`FMC_STATUS` bit 20 (`0x100000`)	a DMA transfer between a controller data buffer and memory has finished	`FMC_STATUS = 0x100000`
3	`FMC_STATUS` bit 3 (`0x8`)	a NAND data transfer (read into a buffer, write from a buffer, parity slice, ID) has finished	`FMC_STATUS = 0x8`
4	`ECC_DONE` bit 0	the ECC engine has finished the operation started with `ECC_KICK`	`ECC_DONE = 1`
5	`FMC_STATUS` bit 23 (`0x800000`)	the NAND status poll started with `FMCTRL1 = 0xca` has delivered a status byte into `FMC_NSTAT`	`FMC_STATUS = 0x800000`

Events are levels, not one-shot signals.

- A `wait` does not consume its event: `wait` events 0 and 5 each passed twice in a row.
- A `wait` event whose flag a previous program left set passes at once.
- `wait` event 4 passes on any `ECC_DONE` bit other than bit 4. It still passed after bit 0 was acknowledged and the value `0x74` was left over.

Further event numbers (device, chapter 16, section 16.2):

- `wait` events 6, 7, 8 and 9 are the command, address, DMA and transfer flags of the second register set (chapter 9, section 9.5). They are bits 1, 2, 20 and 3 of word `0x448`, the same bit numbers as `wait` events 0, 1, 2 and 3 use in the first set. `wait` event 8 is the DMA-done flag of the second set: a loopback from system memory into a data buffer and back to system memory through the second set passed `wait` event 8 and did not pass `wait` event 2. `wait` event 8 also passed while a second-set DMA was running (`evsweep3`), and after a page read of CE0 through the first register set, whose DMA flags had been acknowledged (`w2state`; why is open).
- `wait` event 11 passes in every state tried, including while a chip erases, so it is not the chips' ready line.
- `wait` event 15 passed only in early tests where word `0x044` had been left non-zero. It did not pass with every `0x044` / `0x844` enable written on and a command flag pending (`evsweep2`), nor with each `0x044` bit 0, 1, 2, 11 and 12 set and its own flag pending (`irqop`, device run 37). So it is not "an enabled source is pending"; what it waits for is open.
- `wait` events 10 and 12 to 14 never passed, and neither did event numbers above 15. This held also during an erase, a page program, a page read, an ECC encode and decode, a DMA, a parity-buffer DMA, a second-set DMA, a meta encode and decode and a chip data write (`evsweep2`, `evsweep3`).

7.1.1 Rules for `wait` event 5

`wait` event 5 is the flag `FMC_STATUS` bit 23, read as a level. A status latch (`FMCTRL1 = 0xca`) sets it under these rules (device: `ev5ready`, `ev5arm`, `ev5two`):

- **Arm.** A status latch sets bit 23 only while `wait` event 5 is armed, and each latch uses up the arm.
  - With an idle chip, the latch sets the bit at once.
  - With a busy chip, the bit is set when the chip becomes ready, and the `wait` lasts until then. The busy status byte in `FMC_NSTAT` is `0x81` during an erase or a program and `0x80` during the busy time of a page read.
  - A latch made with the wrong select (`FMC_DATA_SEL = 0x100`) also uses up the arm.
  - Every latch updates `FMC_NSTAT`.
- **What arms it.** These re-arm it: an `FMCTRL1` operation (the set-up word, `0x100e0`, a transfer from the chip with `0x32`, a DMA with `0x2e0`), address cycles (`FMCTRL1 = 1`), and the polled chip becoming ready. These do not re-arm it: `FMCTRL1 = 0x20`, the status-clear word (`FMC_STATUS` = `0x0ff00ffe`), a CE deselect followed by a reselect, and an ECC operation.
- **Consequence for repeated latches.** Suppose two latches have only `FMCTRL1 = 0x20` and a fresh `70h` command (or a chip reset `FFh`) between them. Then one `wait` event 5 passes and the other times out. With only a fresh `70h` between two latches, the second `wait` event 5 timed out 40 of 40 times (`latchrep`).
- **Two chips.** Two chips were programmed and then polled.
  - Polling the first chip (which waits for ready) and then the second passed both `wait` event 5.
  - Polling the second chip first (the one that finishes later) and then the first timed out on the first chip. The first chip's ready transition had already happened before the earlier latch.
- **Level.** If bit 23 is cleared after a `wait` event 5, a second `wait` event 5 times out. If bit 23 is left set, the second `wait` event 5 passes on the old flag. A second latch with no controller operation in between does not set bit 23 again.
- **After a latch.** The controller is in status mode and refuses the next command byte until `FMCTRL1 = 0x20`: after the command write, `FMC_STATUS` bit 1 (command sent) stays clear and bit 26 is set instead; the original programs write `0x20` before `00h`. A latch after `90h` instead of `70h` sets no bit 23.
- **Timing.** A status poll during an erase passed `wait` event 5 once the chip was ready (about 155 us in), with the status byte going from `0x81` to `0xc0`; an idle chip passes at once.
- **Reads.** A sequencer read of `FMC_STATUS` or `FMC_NSTAT` does not consume an event.
- **Repeatability.** 160 single latches (with and without a fresh arm before them, with and without `0x044` / `0x844` written) all passed, with bit 23 set and bit 24 clear (`latchrep`). A `wait` event 5 placed straight after the pair "`70h`, latch" timed out once, although bit 23 read set afterwards. Any instruction or wait between the latch and the `wait` event 5 made it pass (open).

The status latch needs `FMC_DATA_SEL = 1` **(device).** After a write, with `FMC_DATA_SEL = 0x100` left from the data transfer, the latch returned `0x70` (the last command byte) instead of the chip's status. Bit 6 of `0x70` reads as "ready". The poll therefore ended while the chip was still programming, and the program command for the next page was ignored (those pages read back erased). With `FMC_DATA_SEL = 1` (C7) the status byte is the chip's. The status bytes are `0xc0` when ready and `0x81` when busy.

How the original programs use `wait` event 5:

- They test `FMC_NSTAT` bit 6 first and wait on `wait` event 5 only while the chip is busy. Of the 19 uses, 13 (all outside ERASE) have this five-instruction shape; the 6 uses in ERASE are staged differently (RE-7, RE-8).
- The READ and WRITE programs do not test `FMC_NSTAT` again after the wait. The ready procedure with a re-test and a bounded re-latch is described in C7.
- The third-party driver measured bit 23 set after every one of its status polls, whether the chip was ready or not (128 of 128, 3p).

7.1.2 Timeouts

Every original program writes `FMSS_C4C` (fault vector) and `FMSS_C58` (value 4) in its prologue. A `wait` whose flag is not set times out after about 4.87 ms with the start word `0xfff5` that the original CPU drivers use, whatever `C58` holds, and raises `CS_IRQ` bit 2 (device, chapter 16, section 16.2). Without start-word bit 2 there is no timeout (chapter 8, section 8.1.1).

`C58` bit 2 decides where the program goes on: if set (the prologue's 4), at the fault vector; if clear, at the instruction after the `wait`. The original erase program relies on this when it lowers `C58` to 0 around its waits, acknowledges the timeout itself (`CS_IRQ = 4`) and tests again. The other bits of `C58` are in chapter 8, section 8.1.15. `C10`, `C3C`, `C48` and `C54` do not change the timeout.

7.2 The conditions `wait2` waits for

`wait` and `wait2` (opcode `09`) both take a number N in their register field. Each waits until the hardware condition with that number is true, or faults after about 4.87 ms.

- For `wait`, the numbers are the events of chapter 7, section 7.1: `wait` event 0 is "command sent", `wait` event 1 is "address sent", and so on.
- For `wait2`, the numbers are the CPU's interrupt sources. `wait2` **condition N is true while interrupt source N is raised in the raw status of the CPU's interrupt controllers**: VIC0 line N for N = 0..31 and VIC1 line N - 32 for N = 32..63 (device, `w2vic`). Numbers 64..255 never pass.

How this was established: just before each `wait2 N` (N = 0..63) the program read both raw status words (`0x38e00008` for VIC0, `0x38e01008` for VIC1). In four states, idle and with three different interrupt sources raised, the pass or timeout of all 64 conditions matched the raw line, 64 of 64 each time (`w2vic`).

Conditions seen true:

`wait2` **condition N**	**interrupt line**	**when it was raised**	**seen**
5	VIC0 line 5	in every state tried (a source outside the FMSS engine)	device
20	VIC0 line 20, the NAND controller's line	while a NAND controller flag is pending and enabled in `0x044` (command flag with bit 0, DMA flag with bit 11, chapter 3, section 3.4)	device
33	VIC1 line 1	raised while the chip was idle in `op09w33` and during an ECC operation and a DMA in `finale`; not raised in `op09nand`, `evsweep2`, `w2state` or `w2vic` (a source outside the FMSS engine)	device
43	VIC1 line 11, the ECC engine's line	while an `ECC_DONE` flag is pending and enabled in `0x844` (bit 0)	device

All other numbers never passed in any state tried, also while a chip erased, programmed or read, during ECC operations and DMAs, and with the second register set's flags pending (`w2state`, `evsweep2`).

So `wait2` condition 5 is unrelated to `wait` event 5 (the status latch), despite the same number. Lines 5 and 33 belong to devices outside the FMSS engine. This explains why `wait2` condition 33 passed while the chip was idle in `op09w33` and during an ECC operation and a DMA in `finale`; it did not pass in `op09nand` (idle and during an erase), `evsweep2`, `ctrlwords`, `w2state`, `blk100b` or `w2vic`. Which devices raise lines 5 and 33 is not determined here.

7.3 tmwait and evjmp

The descriptions of both opcodes are in chapter 6, opcode `15` (`tmwait`) and chapter 6, opcode `16` (`evjmp`). `tmwait` waits for the timer `C14` to reach 0 (about 4.85 ms). `evjmp` waits for any event flag and then continues at the fault vector `C4C`.

7.4 Timer `C14`

`FMSS_C14` (`0xc14`) is the timeout timer, an 8-bit down-counter (chapter 8, section 8.1.6). The timeout lasts (start-word bits 15:8) + 1 steps, and the length of one step is set by start-word bits 7:4 (chapter 8, section 8.1.1). Device behaviour:

- A `wait` that has to wait loads the start word's bits 15:8 (`0xff` with `0xfff5`) and counts down (`0xf6` was read in the middle of a wait). With one of bits 8..15 cleared in the start word, `C14` read `0xfe`, `0xfd`, `0xfb`, `0xf7`, `0xef`, `0xdf`, `0xbf`, `0x7f` after a run with waits (`startbits`).
- After a timeout the timer reads its reload value; the end of a program freezes it; CPU and program writes do not change it.
- Opcode `15` waits for it: from a non-zero value it reloads and waits until 0 (about 4.85 ms, leaving 0); from 0 it reloads at once. Two `tmwait` in one program made the whole program run 4841, 18 and 3250 us in three runs with different values written to `C10`, `C48` and `C54` before it (`ctrlwords`; not explained).
- The write of `0xff` that the erase program makes after a timeout therefore has no visible effect.

Findings 48 and 51 (chapter 18). The device rows "timer `C14`" and "`C08`" are in chapter 16, section 16.2.

7.5 Timeout length

The timeout length is (bits 15:8 + 1) x 2^s bus cycles, where s depends on n = bits 7:4 of the start word. The table was measured with 24 start words, all within the measurement step (`tmofield`, `tmobits`, `tmobits2`):

n (bits 7:4)	**s**	**example (start word: time to the fault)**
0	at once	`0xff05`: 0 to 2 us
1	3	`0xff15`: 18 us
2	5	`0xff25`: 72 us

n (bits 7:4)	s	example (start word: time to the fault)
3	7	`0xff35`: 296 us
4	9	`0xff45`: 1218 us
5	10	`0xff55`: 2432 us
6 to 15	11	`0xfff5`: 4868 us

More examples:

- n = 0: `0x0005`, `0x0105` to `0xff05` and `0x0f05` all fault after 0 to 2 us.
- n = 1: `0x0f15` after 2 us. n = 2: `0x0f25` after 0 us. n = 4: `0x0f45` after 72 us. n = 5: `0x0f55` after 152 us.
- n = 6 to 15: `0xff65`, `0xff75`, `0xff85`, `0xffc5` and `0xfff5` after 4868 us; `0xffb5` after 4.85 ms.
- n = 7 and 8 with bits 15:8 = `0x0f`: `0x0f75` after 310 us and `0x0f85` after 308 us.
- n = 15 with other bits 15:8: `0x00f5` after 16 us, `0x01f5` 46 us, `0x03f5` 72 us, `0x07f5` 152 us, `0x0ff5` 310 us, `0x3ff5` 1218 us and `0x7ff5` 2430 us.

The steps are measured, not derived from a rule: n = 5 breaks the factor-4 pattern of n = 1 to 4. The bus clock of about 108 MHz is inferred.

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 6, chapter 9, chapter 10, chapter 16, chapter 17.

Chapter 8 — Sequencer control registers

Summary: The registers of the sequencer's control block (register-window offsets `0xc00` to `0xc5c`): what each register holds, who writes it and how it behaves, including the meaning of each bit of the start word. How the sequencer runs a program is in chapter 2; the timeout length table is in chapter 7, section 7.5.

8.1 Sequencer control block

The control block sits at register-window offsets `0xc00` to `0xc5c`. The table gives one line per register. The sections after it give the details in the same order. "Device" marks behaviour measured on the device, and "inferred" marks an interpretation.

offset	name	what it is	written by
`0xc00`	`FMSS_CTRL`	start word, reset, and a "program not finished" status bit	CPU
`0xc04`	`FMSS_PROG`	bus address of the program	CPU
`0xc08`	`CS_STATUS`	state code: running, waiting, paused, stopped at `pause`, idle	read only
`0xc0c`	`CS_IRQ`	flags: end, signal, fault, pause, interrupt sources	set by hardware, cleared by CPU and program
`0xc10`	`FMSS_C10`	8-bit word that the program prologue writes with 1; no effect found	program
`0xc14`	`FMSS_C14`	timeout timer, 8-bit down-counter	hardware only
`0xc18` to `0xc34`	r0 to r7	the program's registers; r6 is the result	CPU and program
`0xc38`	`FMSS_PC`	program counter (register r8)	program; CPU only at a pause
`0xc3c`	`FMSS_C3C`	read by fault handlers; inside a fault handler it holds the offset of the faulting instruction + 8	hardware
`0xc40`		instruction word of the instruction the sequencer is holding	hardware only
`0xc44`		immediate of the instruction the sequencer is holding	hardware only
`0xc48`		32-bit word; no effect found	CPU and program
`0xc4c`	`FMSS_C4C`	fault vector (byte offset in the program)	program prologue
`0xc50`	`FMSS_C50`	which of `wait` events 0 to 5 are pending	hardware
`0xc54`		12-bit word; no effect found	CPU and program
`0xc58`	`FMSS_C58`	fault routing	program prologue
`0xc5c`	`FMSS_C5C`	set to 1 by the fault routing	hardware, CPU

8.1.1 Start word and timeout (`FMSS_CTRL`, device)

`FMSS_CTRL` (`0xc00`) is the register that the CPU writes to reset the sequencer and to start, pause, hold and continue a program. The value written is called the start word.

The original firmware writes these two start words:

- `0x30000` resets the sequencer.
- `0xfff5` starts a program. The 3G and the 4G/5G operating systems use it.

The 4G/5G bootloader starts a program with `0xffb5`. This differs from `0xfff5` only in bit 6. The bootloader also polls for the end of the program instead of taking the interrupt.

What bit 6 does is open. No effect of bit 6 has been found in any program tried. These programs included program fetch from SDRAM and SRAM, 2000 `ldi` / `sti` / `pld`, `sig`, a DMA with an ECC encode, a vectored fault, and reads and writes of the reserved window words (`startbits`, `startdiff`). The reading "bit 6 = interrupt enable" is not confirmed.

bit	effect when written	read back
0	start	0
1	pause: the program stops before its first instruction, or, when bit 1 is written during a run, exactly where it is. A start word without bit 1 continues the program	as written
2	timeout enable: without it, a `wait` whose event never comes, or an illegal opcode, waits for ever (no fault; `C14` stays `0xff`)	as written
3	hold: the sequencer stays in the hold state that follows a reset until a start word without bit 3 (and without bit 16) is written. If bit 3 is written during a run, the program is aborted, and releasing the hold runs it again from instruction 0	0
7:4	timeout step size n (chapter 7, section 7.5). With n = 0 the timeout comes at once (0 to 2 us, and `C14` then reads `0xfe`). What bit 6 does is open (chapter 17, item 8)	as written

bit	effect when written	read back
15:8	timeout length in steps, minus 1	as written
16	hold, same as bit 3	0
17	no effect seen (`startbits`)	0
18	not taken from a write	set while a started program has not finished (held, paused or blocked)
19 to 31	no effect seen in the same set of programs (`startbits`)	0

- Bits 15:4 written alone (together with bit 0, without bit 2) changed nothing in a loop, a `wait`, an illegal opcode, `evjmp` or the interrupt line.
- `0xfff5` and `0xffb5` gave the same timeout length, the same CPU interrupt line and the same behaviour in every test tried.
- A start word without bits 1, 3 and 16, written while a program runs, is ignored.
- `0xfff5` sets every bit from 0 to 15 except bits 1 and 3.
- There is no single-step mode: no combination of bits made the sequencer run one instruction per start.

8.1.2 FMSS_PROG (0xc04)

The bus address of the program in system memory, written by the CPU before the start.

8.1.3 CS_STATUS (0xc08)

A state code of the sequencer (device; the CPU read it while a program was in each state). Neither the original programs nor the original CPU driver use it.

value	state
0	idle, or held by start-word bit 3 or 16
1	running
3	paused by start-word bit 1
4	waiting for a `wait` event, or stalled on an illegal opcode (one state: the timer `C14` counts down in both)
5	the value that a program reads from `CS_STATUS` itself (with `ld` or `pld`) while it runs. It was 5 in every place tried: right after starting a DMA, an ECC operation or a chip command, after `sig`, and inside a fault handler (`c08self`). The CPU reads 1 in the same state
0xa	stopped at `pause` (opcode `12`)

- CPU writes are ignored. The reset clears it.
- A start word with bit 1 or bit 3, written while the program is stopped at `pause`, changes the code to 3 or 0 respectively. The program stays stopped at the `pause`.

8.1.4 CS_IRQ (0xc0c)

Flags, cleared by writing 1 to the bit (write-1-to-clear).

bit	set when (device)
0	the program ended (`end`)
1	the program executed `sig` (opcode `10`)
2	a fault (a timeout or an illegal opcode)
3	the program is stopped at `pause` (opcode `12`); the 4G/5G CPU driver calls this bit "pause"
4	an interrupt source of the NAND controller (FMC) that is enabled in its word `0x044`. A program with a command, an address, a data read, a status latch and a DMA was run. In it, bits 0, 1, 2, 11 and 12 of `0x044` each raised `CS_IRQ` bit 4, and the other 27 bits did not (three of three runs, `irq044`). Which operation each of those bits belongs to is not identified
5	bit 2 of the NAND controller words `0x054` to `0x05c`
6	an `ECC_DONE` bit that is enabled in ECC word `0x844`

- Bits 4 to 6 follow their sources only once those sources are enabled in `0x044` and `0x844`. Both words are 0 after power-on. The bits were seen after an experiment had left `0x044` = `0xa5a5` and `0x844` = `0x25`.
- The program prologue writes `0xff` to `CS_IRQ`. The CPU writes `0xff` before the start, and writes `5` to acknowledge a finished run.

8.1.5 FMSS_C10 (0xc10)

The program prologue writes 1 to it (interpreted as "enable", inferred). The register holds eight bits. No value written to it had a visible effect on a run, the timeout, a `wait`, a DMA or an ECC encode (device, `ctrlwords`).

8.1.6 FMSS_C14 (0xc14): the timeout timer

An 8-bit down-counter (device). The timeout lasts (start-word bits 15:8) + 1 steps, and the length of one step is set by start-word bits 7:4 (chapter 8, section 8.1.1).

- A `wait` that has to wait loads the start word's bits 15:8 into the counter (`0xff` with `0xfff5`) and counts down; `0xf6` was read in the middle of a wait.
- With one of bits 8..15 cleared in the start word, the counter read `0xfe`, `0xfd`, `0xfb`, `0xf7`, `0xef`, `0xdf`, `0xbf` or `0x7f` (one value per cleared bit) after a run (`startbits`).
- After a timeout it reads its reload value. The end of a program freezes it at its current value.
- Writes by the CPU or the program do not change it.
- `tmwait` (opcode 15) waits for it: from a non-zero value it reloads and waits until 0 (about 4.85 ms, leaving 0); from 0 it reloads at once (chapter 6, opcode 15).
- The original erase program writes `0xff` to it after a timeout; that write has no visible effect.

8.1.7 r0 to r7 (0xc18 to 0xc34)

The program's eight registers; r6 is the result (SEQ_RESULT, `0xc30`).

- A CPU write before the start sets the register's starting value.
- A start keeps the values. A reset clears them (device).

8.1.8 FMSS_PC (0xc38): the program counter

The byte offset of the current instruction while a program runs or is stopped; 0 after the end.

- It is register r8 (`0xc18 + 4 * 8`). A program reads it as r8 or with `ld`. A program that writes it (`movi r8`, or `wr` / `st` to `0xc38`) jumps to the written offset.
- The CPU can write it only while the program is stopped at `pause` (chapter 6, opcode 12); at any other time CPU writes are ignored (device).

8.1.9 FMSS_C3C (0xc3c)

The original fault handlers read this register.

- It reads 0 after every run, whatever the CPU wrote (device).
- Inside a fault handler it holds the offset of the faulting instruction + 8 (device). Faults at `0x28`, `0x40` and `0x70` gave `0x30`, `0x48` and `0x78`. A second fault inside the handler gave the offset of the faulting instruction in the handler + 8 (`ctrlwords`, `c50fault`).

8.1.10 0xc40 and 0xc44: the held instruction

The instruction word (`0xc40`) and the immediate (`0xc44`) of the instruction the sequencer is holding (device):

- after the priming start that follows a reset;
- while a program is stopped at `pause`;
- while an instruction waits (`wait`, `wait2`, an illegal opcode).

CPU writes are ignored. The reset clears both.

8.1.11 0xc48

32 bits, writable by the CPU and by the program; the original programs do not use it.

- A program start clears it, so the program never sees a value that the CPU wrote.
- A value written by the program stays after the end of the program.
- No effect was found on a run, the timeout, a wait, a DMA or an ECC encode (device, `ctrlwords`).

8.1.12 FMSS_C4C (0xc4c): the fault vector

The byte offset of the fault handler in the program, written by the program prologue.

8.1.13 FMSS_C50 (0xc50): pending events

Bits 0 to 6 show which of `wait` events 0 to 6 are pending (device; bits 0..5 were each seen with only that flag pending, bit 6 after a command sent through the second register set, `c50b8`):

bit	`wait` **event**
0	event 0, command sent
1	event 1, address sent
2	event 2, DMA done
3	event 3, data transfer done
4	event 4, an `ECC_DONE` bit
5	event 5, status latch
6	event 6, command sent by the second register set

- Each bit follows its flag and clears when that flag is acknowledged.
- The original fault handlers test bit 0 and write 1 to it. Bit 0 is set while a command flag is pending and clear otherwise. A timeout or an illegal opcode does not change it, and the handler's write of 1 does not clear it (device, `c50fault`).
- The value at power-on is `0xffff1800`. Bit 8 was seen set once after a soft reset (`0xffff1900`). It stayed clear in `c50fault` and in `c50b8`, which read it 17 times around soft resets, a sequencer reset, a fault, a second-set command, an ECC decode and ten `end`-only runs (`c50b8`). Its meaning is open.

8.1.14 `0xc54`

12 bits, writable by the CPU and by the program.

- A program start clears it; the value stays after the end.
- No effect was found; it is not a timeout setting (device).

8.1.15 `FMSS_C58` (`0xc58`): fault routing

The register holds eight bits (writing `0xffff` reads back `0xff`, writing `0x100` reads back 0). The program prologue writes 4. The register keeps its value across a reset. It is not a timeout scale: every value 0..31, and also `0x40`, `0x80`, `0xff` and `0x100`, gave the same timeout time (device).

bit	effect (device)
0	at an `end`: sets `C5C`, and every later `end` then runs the fault vector once, until the CPU rewrites `C58`, `C4C` or `C5C`. At a fault: sets `C5C`
1	with bit 1 alone (`C58` = 2) no `end` ran the fault vector in three programs in a row, whether the CPU wrote `C4C` before or after `C58` or the program wrote it (in `c58b2`, with the clean-up of the device job switched off so that `C58` kept its value over three programs, `c58b2`; also in `c58b1`, `c58b1`). With bits 1 and 2 (`C58` = 6) the first `end` after the CPU wrote `C58` and `C4C` ran the fault vector once, and the next two programs, with `C58` still 6, did not (`c58b2` and `c58b1`). In `oneshot`, `C58` = 2 with `C4C` written by the program diverted one `end`. The condition is open
2	a timeout or an illegal-opcode fault continues at the fault vector (with the bit clear, it continues at the next instruction), and sets `C5C`
3 to 7	no visible effect

8.1.16 `FMSS_C5C` (`0xc5c`)

The fault routing of `C58` (section 8.1.15) sets it to 1. The CPU can write it. It keeps its value across a reset (device).

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 2, chapter 3, chapter 6, chapter 7.

Chapter 9 — NAND controller (FMC)

Summary: The registers of the NAND controller (FMC): the configuration word, the operation codes, the other registers, the status flags, the data-buffer routing and the second register set (channel 1).

9.1 FMC_CTRL0 (0x000): NAND controller configuration (not a sequencer control)

bits	meaning	evidence
0	controller enable; always 1	observed
8:1	chip enables: which NAND chips the controller talks to (details below the table)	observed / 3p / device
9	set in the control word of the original READID program (0x33201) and clear in the other original programs. Setting it during a whole page read or a READ ID changed neither the data, the decode results nor the ID (fmcbits)	observed / device; meaning unknown
10	named "enable DMA" in third-party headers. It is clear in every original program: there, DMA is started through bit 24 and FMC_DMA_ADDR. The third-party driver clears it for single-chunk reads	observed / 3p
11	set in every original program except READID (the control word 0x801 consists of bit 11 and bit 0). Clearing it during a whole page read or a READ ID changed nothing (fmcbits)	observed / device; meaning unknown
14:12	timing: hold ("write pulse") clocks	observed formula (chapter 12, section 12.3)
18:16	timing: setup clocks	observed formula
24	DMA enable: set while the data phase of a run uses DMA, cleared at the end (details below the table)	observed / 3p / device
29:28	EDO (read-capture delay)	observed formula

Chip enables (bits 8:1):

- Each bit selects one chip-enable line: bit 1 is chip 0 (CE0), bit 2 is CE1, and so on up to bit 8 for CE7.
- Normally exactly one of these bits is set, so commands, addresses and data go to that chip only. With several bits set, the same cycles go to all those chips at once; the original programs do that only for the RESET command (FFh).
- Examples with the test programs' control word: 0x43801 selects no chip, 0x43803 CE0, 0x43805 CE1 and 0x43809 CE2.
- On the unit tested, CE0 to CE3 each answered READ ID (chapter 16, section 16.2).
- How a run chooses the chip and the page is described in chapter 11, section 11.1.14.

DMA enable (bit 24, device):

- With bit 24 clear, no DMA runs at all. This holds in both directions (0x2e0 system memory to data buffer, 0x100 or 0x1e0 data buffer to system memory) and in both register sets. FMC_STATUS bit 20 never sets, so the wait times out.
- With bit 24 set, the same transfers complete (512 bytes, data intact).
- Leaving bit 24 set through a whole page read (chip transfers included) changed nothing (fmcbits).

The programs load 0x801 | timing once and then change the CE bits with a read-modify-write that preserves every other bit. A control word with bit 0 clear is not usable.

9.2 FMCTRL1 (0x004): NAND controller operation trigger (not a sequencer control)

Writing a value (a "code" in the table below) to FMCTRL1 starts the operation that the code names. Bit names (third-party driver): bit 0 address cycles, bit 1 NAND-to-controller data, bit 2 controller-to-NAND data, bit 6 clear write FIFO, bit 7 clear read FIFO. The rest of each code is known from observed use only.

code	operation	data buffer routing from FMC_DATA_SEL	evidence
0x1	send FMC_ANUM + 1 address cycles from FMC_ADDR0 and FMC_ADDR1	none	observed / 3p
0x20	end of a status phase	none	observed / 3p
0x32	read FMC_DNUM + 1 bytes from the NAND into a 16-byte slot of a parity buffer	parity buffer: FMC_DATA_SEL bits 5:4; slot offset: FMC_24	observed / 3p
0x34	write a 16-byte slot of a parity buffer to the NAND	parity buffer: FMC_DATA_SEL bits 13:12; slot offset: FMC_28	observed / 3p

code	operation	data buffer routing from FMC_DATA_SEL	evidence
0xca	start the NAND status poll; the status byte appears in FMC_NSTAT and FMC_STATUS bit 23 is set	none	observed / 3p
0xe2	read FMC_DNUM + 1 bytes from the NAND into a data buffer	data buffer: FMC_DATA_SEL bits 1:0	observed / 3p
0xe4	write a data buffer to the NAND	data buffer: FMC_DATA_SEL bits 9:8	observed / 3p
0x100	DMA a data buffer to system memory. A second FMCTRL1 write before this DMA's wait event 2 has returned replaces the DMA (C12)	drain: FMC_DATA_SEL bits 9:8	observed / 3p / device
0x1c2	read FMC_DNUM + 1 bytes from the chip (READ ID), to be read through the FIFO port FMC_FIFO	FMC_DATA_SEL = 1	observed / 3p
0x1e0	as 0x100, used for the last DMA of a run, which is preceded by a write of FMC_DMA_CTRL = 7	drain: FMC_DATA_SEL bits 9:8	observed / 3p
0x2e0	DMA system memory to a data buffer, no NAND transfer	data buffer: FMC_DATA_SEL bits 1:0	observed / 3p
0x2e4	write one data buffer to the NAND while a DMA from system memory fills the other	write: FMC_DATA_SEL bits 9:8; fill: FMC_DATA_SEL bits 1:0	observed / 3p
0x340	FIFO read mode, written before reading FMC_FIFO	FMC_DATA_SEL = 0x100	observed / 3p
0x100e0	prefix written before the 00h command that returns a chip from status output to data output	none	observed / 3p
0x0ff3f8e0	set-up word, written once at the start of every run, followed by the all-flags clear (FMC_STATUS = 0x0ff00ffe)	none	observed / 3p

Codes that are not in this table (for example 0x12, 0x42, 0x180 and 0x1a0) occur in the 4G / 5G firmware or in the third-party driver's single-chunk path. The original 3G programs do not use them.

9.3 Other NAND controller registers

offset	name	meaning
0x008	FMC_CMD	NAND command byte; writing sends one command cycle; flag bit 1
0x00c / 0x010	FMC_ADDR0 / FMC_ADDR1	address bytes, least significant first: ADDR0 bytes 0..3, ADDR1 byte 4
0x014	FMC_DATA_SEL	selects the data buffer and parity buffer of an operation (table below)
0x024	FMC_24	byte offset of the slot in the parity buffer (0, 0x10, 0x20, 0x30) that a parity read (0x32) fills
0x028	FMC_28	byte offset of the slot in the parity buffer that a parity write (0x34) sends; it also acts as a counter (notes below the table)
0x02c	FMC_ANUM	address cycles minus 1
0x030	FMC_DNUM	transfer length minus 1 of NAND transfers (0xf = 16 bytes, 0x1ff = 512, 7 = 8); 13 bits writable (device). It does not set the DMA length (below the table)
0x034	FMC_DMA_ADDR	bus address of the next DMA transfer. The original programs write it once per 2 KiB unit, so the controller must advance the address by 512 internally after each data buffer DMA. After a single 512-byte DMA the register still reads the value written (device)
0x038	FMC_DMA_CTRL	the original programs and the third-party driver write 7 before the last DMA of a run (observed). In a page read test (device, fmcbits), the values 0 and 7 deliver correct data, and 7 adds about 100 us. The value 0xff hangs the DMA: the wait times out and that sector is wrong. Each bit alone, written before every data DMA of a page read (device, dmactl): bit 0 and bit 3 deliver correct data (bit 0 adds about 60 us, bit 3 about 120 us per page); bit 1 and bit 2 deliver the first sector correctly, then every later DMA times out, and the register then reads 0x42 and 0x84 (bit 6 or bit 7 set as well); each of bits 4..7 makes every DMA time out. The value persists between runs. Meaning unknown
0x048	FMC_STATUS	event flags, write-1-to-clear (chapter 9, section 9.4)
0x04c	FMC_NSTAT	last NAND status byte latched by FMCTRL1 = 0xca (NAND status bits: bit 0 and bit 1 are fail flags, bit 5 is array ready, bit 6 is ready)
0x060..0x068	FMC_META0..2	12-byte spare metadata: output of a meta decode, input of a meta encode
0x078	FMC_METACFG	metadata layout: four 4-bit nibbles, low nibble first. Nibble j names the slot that meta fragment j (3 bytes) is copied from or to. The nibble value is parity buffer number x 4 + slot number (0..7) (inferred, R6)
0x07c	FMC_METACTL	write 1 = meta encode (poll until bit 0 reads 0); write 2 = meta decode (poll until bit 1 reads 0)
0x080	FMC_FIFO	FIFO port: with FMCTRL1 = 0x340 and FMC_DATA_SEL = 0x100 each read returns the next 4 bytes of a READ ID transfer; reads returned 0 for data buffers filled by DMA, and each read advances FMC_28 by 4 (bufport; chapter 1, section 1.5)

Notes on the registers (device unless stated; chapter 16, section 16.2):

- FMC_28 is also the read position of the FIFO port: each read of FMC_FIFO (0x080) advances it by 4, and it wraps at 512 (it read 0, 128, 256, 384 and 0 after each 32 reads, bufport). While a program runs without reading 0x080 it does not change, except that a DMA and the set-up word set it to 0.
- Between the CPU's dump of the register window before a run and the start of the run, it advanced by 248. Between the end of the run and the CPU's dump after the run, it advanced by 124 (fmc28b).

- It advanced by exactly 128 over 128 iterations of a `pld`/`sti` loop, and by `0x7c` per probe run.
- In programs that write the set-up word first, it stayed 0 over 32 repetitions each of plain instructions, `sti`, `ldi`, parameter-RAM writes, DMAs, ECC encodes and 16-byte chip reads.
- The CPU's dumps of the whole window read `0x080` and its mirrors, which fits the advances between runs (inferred).
- `FMC_DNUM` does not set the DMA length: a DMA always moves 512 bytes (256 to 2048 tried). The 512-byte sector size is known from two sources. In the original programs, every data transfer uses `0x1ff` with one ECC operation and one 512-byte DMA per sector. The page format agrees: 4 x (512 + 16) = 2112 bytes, a 2 KiB page plus its 64-byte spare.
- For offsets below `0xc00`, the register window ignores address bit 9, so `0x200` reads as `0x000`, `0x600` as `0x400` and `0xa00` as `0x800`.
- `0x400..0x5fc` is a second register set (chapter 9, section 9.5) with the same layout and write masks. The boot ROM addresses the NAND controller of each channel at a stride of `0x400`. The CPU set-up writes the word `0x400` as 0.
- `0x100..0x1fc` is a further block of words that the sequencer can only read. The same block appears again at `0x300`, `0x500` and `0x700`. Its meaning is open.

`FMC_DATA_SEL` (`0x014`) holds one-hot fields:

bits	meaning
1:0	data buffer being filled (`1` = data buffer 0, `2` = data buffer 1)
5:4	parity buffer being filled by a read (`0x10` or `0x20`)
9:8	data buffer being drained (`0x100` or `0x200`)
13:12	parity buffer feeding a NAND write (`0x1000` or `0x2000`)

Two combined values occur, which alternate the two data buffers: `0x102` (fill data buffer 1, drain data buffer 0) and `0x201` (fill data buffer 0, drain data buffer 1).

Drain selects on a page read (device):

- With `FMC_DMA_CTRL` = 0, 1 or 8, the drain select `0x1e0` delivered correct data in every sector, and so did the combined values used as the original READ uses them: each sector read with `0x201` / `0x102` into data buffer 0 / 1, decoded there and copied out with `0x102` / `0x201` (`dmactl`).
- In `fmcbits`, `0x102` and `0x1e0` delivered wrong data in every sector and left `FMC_DMA_CTRL` reading `0xf` (`fmcbits`). That read followed the case that wrote `FMC_DMA_CTRL` = `0xff`, a value that persists and makes every DMA fail (inferred cause).

The controller has two 512-byte data buffers and two parity buffers of four 16-byte slots each. The buffers belong to no chip enable (observed / 3p).

9.4 `FMC_STATUS` bits

bit	mask	meaning	acknowledged by writing
1	`0x2`	command sent (`wait` event 0)	the same mask
2	`0x4`	address sent (`wait` event 1)	the same mask
3	`0x8`	data transfer done (`wait` event 3)	the same mask
20	`0x100000`	DMA done (`wait` event 2)	the same mask
23	`0x800000`	status byte latched (`wait` event 5)	the same mask
27	`0x8000000`	set after a sector is read from the chip if that sector had been programmed; clear if it was erased, RD7 (not a `wait` event)	the same mask
all	`0x0ff00ffe`	bits 1..11 and 20..27: the flags above and the other bits in those two ranges, cleared at the start of every run	the same mask

All flags are levels: a `wait` passes while its flag is set and does not clear it (device).

- After power-on, `FMC_STATUS` reads `0x000c3000` (bits 12, 13, 18 and 19; meaning open). These four bits are also set after a soft reset, and writing 1 to them does not clear them.
- A 16-byte data read from the chip (`FMCTRL1` = `0x32`) clears bits 13 and 19; a 512-byte data read (`0xe2`) clears all four. After the 16-byte read, a DMA into a data buffer or a parity buffer, a status latch and the set-up word set bits 13 and 19 again; a DMA out of a data buffer, an ECC encode, a command and a second-set data read do not (`fmcstat2`).
- With all four bits set, a command, an address, a status latch, a DMA, an ECC encode, a meta encode and a command in the second register set leave them set (`fmcstat`).

- The second register set (chapter 9, section 9.5) has the same flags in its own status word `0x448`. These flags drive `wait` events 6 (bit 1), 7 (bit 2), 8 (bit 20) and 9 (bit 3) (device, chapter 7, section 7.1).
- In some runs a status latch failed to raise `wait` event 5, and `FMC_STATUS` showed bit 24 (`0x01000000`) instead of bit 23.
  - In `irqsrc` it happened in both status-latch programs that had an interrupt enable written (`0x044` = `0xffff` or `0x844` = `0x2f`), and not in the one without.
  - In `op16` it occurred in a status-latch program with no enable written.
  - In the fifteen programs of `ev5en` (five enable settings, three times each) it did not occur.
  - The cause is open (chapter 17, item 12). One possible explanation, not tested, is a status latch made without a fresh arm of `wait` event 5 (chapter 7, section 7.1).
  - In `latchrep`, 160 latches (with and without `0x044` / `0x844` written), all set bit 23 and none set bit 24 (`latchrep`).

9.5 The second NAND controller register set (`0x400`)

Where it sits in the window: chapter 3, section 3.2.

The second register set (`0x400..0x4fc`, CPU address `0x38a00400`) is a second NAND channel controller, laid out word for word like the first set, with the same write masks:

- `0x400` is its `FMC_CTRL0`, `0x404` its `FMCTRL1`, `0x408` its command byte, `0x40c` and `0x410` its address words, `0x414` its data select, `0x42c` its address count, `0x430` its length and `0x434` its DMA address;
- `0x448` is its status, `0x44c` its status byte, and `0x424`, `0x438`, `0x444`, `0x460..0x478` and `0x47c` are the remaining words.

Unlike `0x200..0x3fc`, it is not a mirror of the first set. Device results show this:

- The two sets held different values at the same time (command byte `0x008` = `0x70` while `0x408` = `0x90`; `0x048` = `0x008c3002` while `0x448` = `0x000c3002`).
- A READ ID through the second set changed only its own words.
- A DMA in the second set raised `wait` event 8 and not `wait` event 2.
- The two DMA address registers held different values.

Further device results:

- Its state machine runs: a READ ID through it sends the command (`0x448` bit 1) and the address cycles (bit 2) and moves the 8 bytes (bit 3; `0x424` counts them).
- Its flags drive `wait` events 6 (command), 7 (address), 8 (DMA) and 9 (transfer), as `wait` events 0, 1, 2 and 3 do for the first set. The `evjmp` instruction reacts to them.
- It has its own DMA engine and its own data buffers 0 and 1: system memory to buffer to system memory works through it with its own `FMC_CTRL0` bit 24 set. Its parity buffers 0 and 1 return a pattern DMA-ed into them; parity buffer 0 is shared with the first register set (inferred: after the second set's DMA into its parity buffer 0, the first set's parity buffer 0 read the second set's pattern, `ch1buf`).
- No chip answered on its bus on any chip enable when only the channel-0 pins were configured; its FIFO port returned junk.

The second register set is channel 1 of the NAND controller, on its own pins. The boot ROM's NAND boot routine takes a channel number, 0 or 1 (`bootrom mem:0x20002e0c`: `channel < 2`). The routine reads pages from CE0 of that channel into SRAM with a page reader that addresses the registers at `channel * 0x400` (RE-43). The routine's pin set-up (`bootrom mem:0x20001830`; GPIO configuration words at `0x3cf00000 + port * 0x20`, function 2 per 4-bit field) differs per channel:

Which device the boot ROM boots from is set by two strap pins, GPIO port 3 pins 6 and 7 (RE-47): 0 / 0 SPI flash on SPI port 0, 0 / 1 SPI flash on SPI port 2, 1 / 0 NAND channel 0, 1 / 1 NAND channel 1. On this unit both pins read 0, so it boots from the SPI flash (`straps`).

channel	GPIO ports and pins switched to function 2
0 (`0x000` set)	port 8 all eight pins; port 9 two pins (fields 0 and 4); port 10 pins 0..3
1 (`0x400` set)	port 10 pins 4..7; port 11 pins 0..4; port 12 pins 0..3; port 7 pin 4

In the first device tests only the channel-0 pins were configured (ports 8, 9 and the lower half of port 10), so the channel-1 controller drove no pins and the FIFO port returned junk (`ev8`, `chan1`). With the channel-1 pins switched as the boot ROM does, a READ ID through the `0x400` set read `0xff` bytes on CE1 to CE7 (an undriven bus) and nothing on CE0 (device). No chip answers on channel 1 of this board, so the nano 3G uses only channel 0 (inferred from the one board).

The original programs and the original CPU driver never use the second register set, apart from the original CPU driver writing its `FMC_CTRL0` (`0x400`) as 0 at start-up and power-off (RE-37).

9.6 Status latch needs `FMC_DATA_SEL = 1`

After a write, `FMC_DATA_SEL` was still `0x100` from the data transfer. In that state the status latch returned `0x70` (the last command byte) instead of the chip's status. Bit 6 of `0x70` reads as "ready", so a poll ended while the chip was still programming. The next page's program command was then ignored, and those pages read back erased. With `FMC_DATA_SEL = 1` (C7) the status byte is the chip's: `0xc0` when ready and `0x81` when busy.

The arming rule of `wait` event 5 and the other status-latch rules are in chapter 7, section 7.1.

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 3, chapter 7, chapter 10, chapter 11.

Chapter 10 — ECC engine

Summary: The ECC engine at 0x800. How error correction works, the four correction methods (one BCH code and three strengths of a Reed-Solomon code) with how each is selected, how many bits or bytes each corrects and how it computes its parity, what is stored on the chip for each, the full ECC_MODE map, the status words, the modes without a code, the registers and the byte order.

10.1 How error correction works

The correction is done by a hardware engine, the ECC engine at 0x800: it computes parity and corrects errors in its own logic. There are no ECC tables in memory, and neither the CPU nor the program supplies any. The generator of each code and its field arithmetic are built into the engine. They appear in no firmware image and in no memory table.

The engine works on one 512-byte sector at a time. For each sector it uses two parts of the NAND controller (FMC), named in chapter 1, section 1.5:

- **data buffer 0** or **data buffer 1** (512 bytes each) holds the sector's data;
- **parity buffer 0** or **parity buffer 1** (64 bytes each) holds four 16-byte **slots**, one per sector of a 2 KiB unit; slot k holds the parity of sector k (and, for the BCH code, three metadata bytes).

Metadata here means the 12 bytes per 2 KiB unit that the CPU passes with a page besides its data (FMC_META0..2, chapter 12). The metadata path stores 3 of them in each of the four slots (bytes 0..2), so they are written to the chip in each sector's chunk (R6). Whether a method also protects these 3 bytes against bit errors is a property of the method (table below).

The program moves data between system memory, the data and parity buffers and the chip; the engine only reads and writes a data buffer and a slot (FMC_DATA_SEL, chapter 9, section 9.3; the buffer rules are in chapter 11, section 11.2).

- **Encode** (before writing): the engine reads the data buffer and writes the parity into the slot named in the ECC_KICK word. The program then sends the slot (as the chunk) and the data buffer to the chip.
- **Decode** (after reading): the program has put the stored data into a data buffer and the stored parity into the slot; the engine checks the data against the parity, corrects the data buffer in place if it can, and reports the result in ECC_RESULT.

The parity is stored on the chip next to the data, in the chunk in front of each sector (chapter 10, section 10.9). The engine never touches the chip.

10.2 The methods at a glance

ECC_MODE (0x814) selects the method. It is written before every kick (a write to ECC_KICK that starts an operation). All values below were measured on the device. Column meanings:

- "corrects": the largest number of errors per 512-byte sector that a decode corrected in the tests. With one more error, the decode reported the sector uncorrectable.
- "protects metadata": whether the code also covers the 3 metadata bytes of the sector's slot.
- "slot after encode": what the encode leaves in the slot.

method	ECC_MODE	code	parity bytes	corrects	protects metadata	slot after encode	details
BCH	0x180	binary BCH over GF(2^13)	13	**8 bits** anywhere in data, metadata and parity	yes, 3 bytes	bytes 0..2 metadata (left as they were), bytes 3..15 parity	10.3
RS-8	0x8100	Reed-Solomon over GF(2^10), 8 parity symbols	10	**4 bytes** (any number of bits inside each)	no	slot 0 bytes 0..9 parity, bytes 10..15 not written; the slot field is ignored	10.5
RS-12	0x8104	the same, 12 parity symbols	15	**6 bytes**	no	slot 0 bytes 0..14 parity, byte 15 not written; the slot field is ignored	10.6
RS-16	0x8108	the same, 16 parity symbols	20	**8 bytes**	no	parity-buffer bytes 0..19 (slot 0 and bytes 0..3 of slot 1); the slot field is ignored	10.7
none	0, 0x80, 0xe00000 (bit 8 and bit 15 clear)	no code	–	nothing (every decode reports uncorrectable)	–	slot not written	10.12

The original programs use only the BCH mode `0x180` (chapter 11). Other values that select the same BCH code and strength: `0x100`, `0x1c0`, `0x181`, `0x380`, `0x780`; the same RS-8 code: `0x8180`. Mode bits that act on every method (the full map is in chapter 10, section 10.10):

- bit 1 with RS-8: the encode writes two more slot bytes, a copy of slot bytes 8 and 9 (one sample);
- bit 4 = 32-bit byte swap: the engine reverses the bytes of each 32-bit word of the data it reads and of the slot it writes (BCH and RS);
- bits 20:16 = data length L (1..31; 0 = 512): only the first L bytes of the data buffer are encoded and checked, and the rest of the sector counts as `0xff` (BCH and RS);
- bits 24:21: no effect on the code; they change `ECC_DONE` bit 5 (chapter 10, section 10.11).

10.2.1 Running one operation

The same steps for every method (device; the original programs use them for the BCH code, chapter 11, section 11.2):

step	write / wait	value
1	`ECC_DONE` (`0x840`)	`0x1ff`: clear all flags
2	`ECC_MODE` (`0x814`)	the method's value (table above)
3	`ECC_KICK` (`0x80c`)	bit 0 = decode or bit 1 = encode; bits 9:8 select the data buffer (`0x100` data buffer 0, `0x200` data buffer 1); bits 13:12 select the parity buffer (`0x1000` parity buffer 0, `0x2000` parity buffer 1); bits 17:16 select slot k (BCH only; a Reed-Solomon operation always uses slot 0)
4	`wait` event 4	the operation is done (`ECC_DONE` bit 0)
5	read `ECC_RESULT` (`0x810`), decode only	bits 19:16 = number corrected (BCH: bits; RS: bytes); `0x11` = uncorrectable, data left as it was
6	`ECC_DONE`	write 1 to acknowledge

Examples of `ECC_KICK` values: an encode of sector k from data buffer 0 into slot k of parity buffer 0 is `0x1102 | k << 16`; the matching decode is `0x1101 | k << 16`. The kick words used by the original programs are listed in chapter 10, section 10.13.

The encode and the decode of a sector must use the same `ECC_MODE` value.

A decode run in a later program than its encode found nothing to check. The set-up word (`FMCTRL1` = `0x0ff3f8e0`) that starts each program clears the data buffers and parity buffers (inferred). When data is read from the chip, the program first reloads the slot from the stored chunk (chapter 10, section 10.9).

10.3 BCH: 8 bits per sector

Selected by `ECC_MODE` = `0x180` (bit 8 set, bit 15 clear). This is the code of every page the original firmware writes.

10.3.1 Strength

- **8 bit errors per sector, in any positions** of the 4224-bit code word (512 data bytes, 3 metadata bytes, 13 parity bytes). With 1 to 8 flipped bits a decode corrects the data and reports `ECC_RESULT` = `n << 16`; with 9 or more it reports `0x11` and leaves the data as it was (`eccstr2`: 1..8 corrected, 9..12 not).
- The count covers the whole code word: errors in metadata and parity bytes are corrected and counted like data errors (4 data + 4 parity bits = 8 corrected; 5 data + 3 metadata bits = 8 corrected; 9 parity bits uncorrectable, `eccchunk2`).
- The same holds for errors stored on the chip: they decode the same way (8 data bits corrected, 9 not, 2 metadata bits corrected and counted, an all-zero chunk uncorrectable, `rawwrite`).
- Eight bits in one byte count as 8 errors: the BCH code counts bits, not bytes.

10.3.2 How the code works

The code is a binary BCH code over GF(2^13) that corrects 8 bits. Every parity measured on the device matches this definition:

- field polynomial x^13 + x^4 + x^3 + x + 1 (`0x201b`);
- generator g(x) of degree 104 (13 bytes), roots alpha^1 .. alpha^16: `g = 0x115f914e07b0c138741c5c4fb23` (bit k = coefficient of x^k);
- message: the 512 data bytes followed by the slot's 3 metadata bytes (slot bytes 0..2), most significant bit first (data byte 0 bit 7 is the highest power); so a bit error in a metadata byte on the chip is corrected like one in the data;
- parity = message × x^104 mod g(x), 13 bytes, most significant byte first.

An encode takes the metadata bytes from the slot itself, not from FMC_META0..2. The metadata path (R6) places them in slot bytes 0..2 before the encode, and the encode writes slot bytes 3..15 only.

The metadata is part of the message (an unwritten slot holds 0xff). All-zero data therefore gives non-zero parity (zeros: 22 24 98 95 03 a2 a2 f3 10 b3 a3 77 3e in 0x900..0x90c).

All twelve encodes measured match this definition (zeros, ones, a ramp, eight single-bit sectors at bits 0, 1, 2, 7, 8, 9, 4088 and 4095, and the strength test's sector; eccstr2, eccpar). Twenty sectors of pages written by the original firmware are code words of this code with their real metadata in slot bytes 0..2 (rawpage).

10.3.3 What an encode and a decode leave

- After an encode: the 13 parity bytes in the slot (bytes 3..15) and, in reverse order, in the ECC-area words 0x900..0x90c (the last word holds one byte). 0x990 reads 0 (eccregs).
- After a decode: the data buffer corrected in place; 0x900..0x90c cleared; 0x9a4 counts the decodes. ECC_RESULT = n << 16 (n = corrected bits, 0..8) or 0x11.
- ECC_DONE = 0x33 after an encode and 0x35 after a decode (chapter 10, section 10.11).

10.4 Reed-Solomon: common to all three strengths

ECC_MODE bit 15 (with bit 8) selects a Reed-Solomon code instead of the BCH code; bits 3:2 select its strength (0 = 8, 1 = 12, 2 = 16 parity symbols; 3 = as 0). The original programs never use it. The following description matches the device data:

- **Field:** GF(2^10), primitive polynomial x^10 + x^3 + 1 (0x409).
- **Symbols:** each data byte is one symbol (values 0..255 in a 10-bit field), and data byte 0 is the coefficient of the highest power. A sector is 512 data symbols; the code word is 520, 524 or 528 symbols, well inside the field's limit of 1023.
- **Generator:** g(y) = (y - 1)(y - a)...(y - a^(2t-1)) with 2t = 8, 12 or 16 parity symbols, where a is a root of x^10 + x^3 + 1 (the field's primitive element). For 8 symbols the coefficients (y^0..y^7, then y^8 = 1) are 0x190 0x299 0x2a5 0x01d 0x3ee 0x1ab 0x30a 0x0ff.
- **Parity:** m(y) × y^(2t) mod g(y), where m(y) is the polynomial of the 512 data symbols. The result is 2t symbols of 10 bits.
  - The ECC area 0x900.. holds them little-endian, 10 bits each, packed into 10, 15 or 20 bytes.
  - The slot receives the same bytes in reverse order (chapter 10, section 10.14).
- **The slot field is ignored.** A Reed-Solomon encode writes its parity at byte 0 of the parity buffer that ECC_KICK names (slot 0), whatever slot bits 17:16 name, and a decode reads it from there (rschip):
  - an RS-16 encode with slot field 3 wrote its 20 bytes at bytes 0..19 of parity buffer 0, and a decode with slot field 3 then corrected 2 bad bytes;
  - in pages written with one encode per sector (slot fields 0..3, each slot sent to the chip as that sector's chunk) and read back the same way, sector 0 decoded and sectors 1..3 were reported uncorrectable.
- So the parity of only one sector at a time can be in a parity buffer. BCH uses the slot field (chapter 10, section 10.3).
- **Only the 512 data bytes are protected.** The Reed-Solomon code covers the data alone; the 3 metadata bytes of the slot are not part of it (all-zero data gives all-zero parity, whatever the slot holds).
- **What it corrects:** up to t bad bytes (t = 4, 6, 8), whatever the number of flipped bits inside each byte. A decode reports ECC_RESULT = n << 16 with n = the number of corrected bytes, or 0x11 (data left as it was) when more bytes are bad. A burst that crosses a byte boundary counts as two bytes (a 10-bit burst across two bytes: 0x20000, b15ecc).
- **Trace:** after an encode with 8 symbols, 0x90a..0x913 hold a trace of the encoder (the encoder computes the parity in a shift register of 8 symbols; the trace is the value of its highest symbol just before each of the last 8 data symbols was fed in; 38 of 38 sectors); with 12 symbols the 5 bytes after the parity hold further bytes.
- 0x990**:** a 16-bit value, repeated in both halves of the word, that depends on the data and not on the strength (zeros 0000, all ones fc36, the ramp c68b with 8 and with 16 symbols, eccregs, eccmode4). Its meaning is open.

How the code was identified:

- Fits of a cyclic code over GF(2), tried for every bit and byte order, failed.
- Within every sampled byte, the parity of bit k+1 is exactly 2 × the parity of bit k in a degree-10 field. This singled out 0x409 among all 60 primitive polynomials of degree 10.

- With bytes as symbols, the parity of every sample then follows from repeated multiplication by y modulo one polynomial of degree 2t.
- Matched: 43 of 43 sampled sectors for 8 symbols (zeros; one bit at bytes 0, 1, 2, 3, 4, 8, 16, 32, 64, 128, 256, 384, 448, 496, 504, 508, 510, 511; bits 15:0 and 4080..4095; two bits together; the ramp `i × 37 + 11`), and 10 of 10 each for 12 and 16 symbols (`finale`).

The length field (bits 20:16) applies to this code too: with L = 31 the parity is that of the first 31 bytes with the rest `0xff` (`eccmode4`).

10.5 Reed-Solomon, 8 parity symbols: 4 bytes per sector

- **Select:** `ECC_MODE` = `0x8100` (`0x8180` selects the same code).
- **Strength:** 4 bad bytes. 1, 2 and 4 bytes in error were corrected (`ECC_RESULT` `0x10000`, `0x20000`, `0x40000`); 8, 9, 10, 12, 16 and 17 bad bytes, and bursts of 40, 80 and 90 bits, were reported uncorrectable (`b15ecc`). The same on the chip: a sector stored with 4 bad bytes was corrected (`0x40000`), one with 5 was reported uncorrectable (`rschip`).
- **Parity:** 8 symbols = 80 bits = 10 bytes. Bytes 0..9 of slot 0 hold them; bytes 10..15 are not written (they keep `0xff` from the set-up word). For the ramp data, the slot holds `522ee44c7cc113acebcd ffffffffffff` (`finale`).
- **With** `ECC_MODE` **bit 1** (`0x8102`): slot bytes 10 and 11 receive a copy of bytes 8 and 9 (`…ebcd ebcd ffffffff`, one sample).

10.6 Reed-Solomon, 12 parity symbols: 6 bytes per sector

- **Select:** `ECC_MODE` = `0x8104`.
- **Strength:** 6 bad bytes. 4 and 6 bad bytes were corrected (`ECC_RESULT` `0x40000`, `0x60000`); 7 bad bytes were reported uncorrectable (`0x11`, `ECC_DONE` `0x3d`, data left as it was) (`rsdecode`). The same on the chip: 6 stored bad bytes corrected (`0x60000`), 7 uncorrectable (`rschip`).
- **Parity:** 12 symbols = 120 bits = 15 bytes. Bytes 0..14 of slot 0 hold them; byte 15 is not written. For the ramp data, the slot holds `186d249847a87441d927d4965ed6e4 ff`. The ECC area holds the same 15 bytes reversed, followed by 5 more bytes (`a8012da801`) (`rsdecode`, `finale`).

10.7 Reed-Solomon, 16 parity symbols: 8 bytes per sector

- **Select:** `ECC_MODE` = `0x8108`.
- **Strength:** 8 bad bytes. 4 and 8 bad bytes were corrected (`0x40000`, `0x80000`); 9 and 12 were reported uncorrectable (`rsdecode`).
- **Parity:** 16 symbols = 160 bits = 20 bytes, more than a 16-byte slot. The encode writes all 20 bytes at bytes 0..19 of the parity buffer: slot 0 and **bytes 0..3 of slot 1**, also when the kick names slot 3. For the ramp data the parity buffer then holds `4e2923086f4cfc4b7b77718291507950 ffa89d05 ff…`, and the ECC area `0x900..0x913` holds the same 20 bytes reversed (`rsdecode`, `rschip`).
- A decode reads the 20 bytes from the same place: with the slot field 3 it corrected 2 bad bytes from the parity at bytes 0..19 (`rschip`).

10.8 What each method corrects

method	parity bytes	corrects	as bits, scattered (one per byte)	as bits, clustered	evidence
BCH	13	8 bits	8	8	`eccstr2`
RS-8	10	4 bytes	4	up to 32 (4 whole bytes)	`b15ecc`
RS-12	15	6 bytes	6	up to 48	`rsdecode`
RS-16	20	8 bytes	8	up to 64	`rsdecode`

- No method corrects more than 8 scattered bits per 512-byte sector.
- For errors spread one bit per byte, BCH and RS-16 correct the same number (8); for errors clustered in a few bytes, RS-16 corrects more.
- How the "x bits per y bytes" of a chip datasheet maps onto these numbers depends on the error model of the chip, not on the engine.
- Every case beyond the limits above was reported uncorrectable with the data unchanged. This includes 240 BCH sectors with 9 to 64 random bad bits and 240 RS-8 sectors with 5 to 40 random bad bytes: none was reported corrected or error-free (`miscorr`).

10.9 What is stored on the chip

The engine does not write the chip; the program sends each slot and each data buffer to the chip in the order and with the lengths it chooses (`FMC_DNUM` + 1 bytes per transfer, chapter 9, section 9.3). What ends up on the chip is therefore the layout the program uses.

This unit's chip. Four chip enables answer READ ID `ec d5 14 b6 74` (`chan1`). The restore OS has a chip table row for this ID. The row holds 4096, 128, 8 and 128 in the fields read as blocks per chip enable, pages per block, 512-byte sectors per page and spare bytes per page, and 3872 user blocks (RE-45; field meanings inferred). A page therefore holds 8 × 512 = 4096 data bytes plus 128 spare bytes, 4224 bytes in all, the length the original programs transfer per page (chapter 11, section 11.2).

10.9.1 BCH (the original layout, device)

Every 512-byte sector is stored as its 16-byte chunk followed by the 512 data bytes, all in the page's normal byte stream starting at column 0. A 2 KiB unit is four such sectors (2112 bytes), and a 4 KiB page is two units:

```
| chunk 0 (16) | data 0 (512) | chunk 1 (16) | data 1 (512) | chunk 2 (16) | data 2 (512) | chunk 3 (16) | data 3 (512
  ) |   = 2112 bytes, twice per page
chunk k = [3 metadata bytes][13 BCH parity bytes]
```

- 8 × (16 + 512) = 4224 bytes: the layout fills the page, spare area included, exactly.
- The 3 metadata bytes of chunk k are bytes 3k..3k+2 of the unit's 12 metadata bytes (R6); they are protected by the code.
- Read back: the program puts the chunk into slot k and the data into a data buffer, then runs a decode (chapter 11, section 11.2).
- Pages written by the original firmware confirm this layout: all twenty sectors read raw are code words with their metadata (`rawpage`); pages built by the CPU and stored errors behave as the code predicts (`rawwrite`).
- An erased (never written) page reads all `0xff`, which is not a code word; `FMC_STATUS` bit 27 tells whether a read carried ECC data at all (RD7).

10.9.2 Reed-Solomon

Pages were written with RS-8 and RS-12 by the engine, one encode per sector followed by sending the slot and the data, and read back (`rschip`):

- The chunk stored on the chip is the slot as it was: RS-8 `2a5c0a397c8a9f585350` followed by six `0xff`, RS-12 15 parity bytes followed by one `0xff`.
- Because a Reed-Solomon encode always writes slot 0 (chapter 10, section 10.4), only sector 0, whose chunk was sent from slot 0, carried its parity; sectors 1..3, sent from slots 1..3, were reported uncorrectable. A page with Reed-Solomon parity in every sector needs each sector's chunk sent from slot 0 right after its encode, and loaded into slot 0 before its decode (inferred).
- Stored errors in sector 0 decoded as in the buffers: RS-8 corrected 4 bad bytes and not 5, RS-12 corrected 6 and not 7.

The table is computed from the parity sizes and the page size (inferred). "Bytes per page" is 8 × (512 + parity bytes), plus 3 metadata bytes per sector for BCH. "Free slot bytes" are the slot bytes that the encode leaves available for metadata:

method	parity bytes	bytes per page	fits 4224 bytes	free slot bytes
BCH	13 (+ 3 metadata)	4224	yes, exactly	3, protected
RS-8	10	4176	yes (48 bytes left)	6 (bytes 10..15), not protected
RS-12	15	4216	yes (8 bytes left)	1 (byte 15), not protected
RS-16	20	4256	**no** (32 bytes too many)	none; the parity also overwrites bytes 0..3 of slot 1

- The Reed-Solomon parity starts at slot byte 0, the same place where the metadata path puts the metadata. The metadata layout of the BCH chunks therefore does not carry over.
- A read with a Reed-Solomon mode needs the stored parity back at bytes 0.. of the parity buffer (for RS-16 bytes 0..19) before the decode.
- Zero data gives zero Reed-Solomon parity, so an erased page (all `0xff`) is not a code word in these modes either (not tested).

10.10 `ECC_MODE` bit map

ECC_MODE has 25 writable bits (mask 0x01ffffff) and reads 0x00e00000 at power-on. The map was measured by encoding a known sector with 0x180 plus one more bit at a time, and then with other bit combinations (eccmode, eccmode2, eccmode3, finale, done5):

bits	effect
bit 0	no difference seen in parity or slot
bit 1	with RS-8: the slot gets two more bytes, a copy of slot bytes 8 and 9 (one sample); with BCH no effect
bits 3:2	with bit 15: Reed-Solomon strength, 0 = 8 parity symbols (4 bytes), 1 = 12 (6 bytes), 2 = 16 (8 bytes), 3 = as 0; with BCH no effect
bit 4	32-bit byte swap: the data buffer is read and the slot written with each 32-bit word byte-reversed (both codes; the code itself is unchanged)
bits 7:5	no difference seen in parity or slot (bit 7 tested with bit 8 set)
bit 8	slot path: the encode writes its parity into the named slot and a decode reads it from there; with bits 8 and 15 clear no code is computed (chapter 10, section 10.12)
bits 14:9	no difference seen in parity, slot or ECC_DONE
bit 15	Reed-Solomon code (chapter 10, section 10.4) instead of BCH
bits 20:16	data length L (5 bits, 1..31; 0 = 512): only the first L bytes of the data buffer are encoded and checked, the rest counts as 0xff (BCH: L = 1, 2, 3, 4, 5, 8, 12, 16 and 31 matched exactly; a decode with L = 16 ignored a change at byte 100 and corrected 8 bits in byte 5; RS: L = 31 matched)
bits 24:21	no difference seen in parity or slot; they clear ECC_DONE bit 5 (chapter 10, section 10.11)
bits 31:25	not writable

ECC_MODE = 0x1ffffff (every writable bit set) acts as RS-8 with the byte swap and L = 31, because bits 3:2 both set select 8 symbols (eccmode4).

10.11 ECC_DONE and ECC_RESULT

ECC_RESULT (0x810) after a decode:

value	meaning
0	no error
n << 16 (bits 19:16)	n errors corrected: bits (BCH, 1..8) or bytes (Reed-Solomon, 1..8)
0x11 (bits 0 and 4)	uncorrectable; the data buffer is left as it was

What an encode does to ECC_RESULT (device):

- In the modes without a code it becomes 0x11.
- An encode in the same code as the last decode leaves it unchanged: a BCH encode after a BCH decode (0x30000, 0x40000 stayed), a Reed-Solomon encode after a Reed-Solomon decode (0x20000 stayed) (rsresult, eccmode2).
- An encode in the other code changed it: a Reed-Solomon encode after a BCH decode with 3 corrected bits set it to 0 (from 0x30000), and a BCH encode after a Reed-Solomon decode set it to 0x30000 (from 0 and from 0x20000) (rsresult). In eccmode2 eccmode2, Reed-Solomon encodes after a BCH decode set it to 0x20000. Where these values come from is open.

ECC_DONE (0x840) after an operation: bit 0 is the done flag that wait event 4 waits for. The register reads 0x33 after an encode and 0x35 after a decode. Two more bits vary:

- **Bit 3 = the last decode was uncorrectable.** A failed decode sets it. It stays set through later encodes, other programs and the ECC_DONE = 0x1ff clear. The next decode that succeeds clears it. An uncorrectable decode reads 0x3d; an encode after it reads 0x3b.
- **Bit 5 is clear** when any one of ECC_MODE bits 21..24 is set (in done5, encode followed by a decode, done5; in done5b, an encode alone, an encode followed by a decode, and an encode after a decode, each after a soft reset, done5b).
  - In done5 this held on all three base modes (0x180, 0x8100, 0), after the encode and after the decode (ECC_DONE = 0x13, 0x15, 0x1b, 0x1d).
  - Bits 9..14 and 16..20 alone leave it set.
  - In eccmode2, encodes with bit 21, 22 or 23 alone on 0x180 read 0x33 (bit 5 set); in that series bit 24 alone cleared it. Why eccmode2 differs from done5 and done5b is open.

10.12 Modes with bit 8 clear: no code

The modes 0, 0x80 and 0xe00000 (device). With bits 8 and 15 clear the engine computes no parity code. An encode writes the next value of a fixed sequence into the ECC area instead:

- The slot is not written; it keeps the `0xff` that the set-up word put there.
- 13 bytes are written into `0x900..0x90c` (the last word holds one byte), and a 16-bit value, repeated in both halves of the word, into `0x990`.
- The written values do not depend on the data in the data buffer or on which of the three mode values is used. The first such encode writes value 1 of the sequence, the second value 2, and so on.
- A decode in these modes reports every sector uncorrectable (`ECC_RESULT` = `0x11`), including a sector whose data and slot are unchanged, because the slot holds no parity. A decode does not advance the sequence and leaves `0x990` unchanged.
- In `seqrestart`, none of the actions tried between encodes restarts the sequence (`seqrestart`).
  - The values continued in order (values 7 to 18, `seqrestart`) across a soft reset, the sequencer reset (`FMSS_CTRL` = `0x30000`), writes of `ECC_MODE`, `ECC_KICK` (`0`), `ECC_DONE`, `0x940..0x950` and `FMC_CTRL0`, and BCH encodes and decodes.
  - The first such operation gave value 1 in `eccregs`, value 1 in `eccmode4` and value 14 in `finale` (`eccregs`, `eccmode4`, `finale`); in `seqfirst`, started right after a power cycle of the unit, it gave value 2 (`seqfirst`).
  - Which event restarts it is open.
- `0x9a0` read `4f888131` in every encode-only run, and `f80fffff`, `ffffffff` or `ffffff80` after a decode in these modes. `0x9a4` (decode count) and `0x9c0` = 1 behave as in the other modes.

The 61 measured values of the sequence, numbered n (the 13 bytes in ECC-area byte order, and the 16-bit `0x990` value):

n	`0x900..0x90c`	`0x990`
1	`233c853f32c8ce55a1c321a1fc`	`f0f4`
2	`2e1c79573386228532df6ca2da`	`5bb8`
3	`7b55880c275b93c33a7204f807`	`4734`
4	`611570dd25c74b621d4b9efe4b`	`f6ad`
5	`e87c56af4cfa3be376f15bb3e4`	`3d6d`
6	`dcfca60c49c28aa039836fbe7c`	`7353`
7	`edd42e2dda3f79ae9517f0dc37`	`aa32`
8	`a62f0baf90c8082570138c3f12`	`83d6`
9	`c47f1becb633ef38283ab3fa84`	`b5b6`
10	`528950e823dd0c2ee3334b3ccf`	`854a`
11	`b5d2b4ab2eacd0192881214ff7`	`ce7f`
12	`993f23a304711734be92d1c260`	`ca68`
13	`5788eb241e93af5b28f7042410`	`ca68`
14	`2ca900f00bae330c7f30f0c62a`	`52fa`
15	`b0c691ff3e6a42d353fb5a0bcb`	`73a8`
16	`4684475615107a7cfd75b3cebe`	`cb7d`
17	`5da0a18c3e1f8acedf03f2ac68`	`e61b`
18	`b1250ddf69ebfa90821e212783`	`7881`
19	`a49605af7f7209f9bc12b71a3a`	`df16`
20	`5f6698cd901dfb497dc805f4f8`	`6e53`
21	`56fb4de8fda80f967a303d769f`	`828f`
22	`83e1b2e862f0f8fb82654c520f`	`fdda`
23	`91db19a3b89a11448d953d56c0`	`6aed`
24	`18152367c7acec9306decbe7f5`	`1f8a`
25	`1f9ab13532fe2de062de3c167e`	`dc9a`
26	`0d07c4bdcd92d74f7549d07515`	`1a4b`
27	`20e225dd66b046a4c6a92a6f17`	`3f63`
28	`04d8cecd9969b2fbe987f3a8c1`	`a51b`
29	`5e120d0ccf2c902c8e46069dc5`	`e889`
30	`359d1fe870186a9fabfaa0eb7d`	`0007`
31	`8109986bdd922e3164784b8075`	`2039`
32	`74ec7966e37cc95a54e0129410`	`8af2`
33	`1cc57661b8694006cbe5c54300`	`38f9`
34	`f60eb57ac4b58fd1abd5766bca`	`69f6`
35	`265cab74729f9d6895ded8c4eb`	`f860`
36	`d130e886cba011cb2f5eaa6c6a`	`de0f`

n	0x900..0x90c	0x990
37	7195d49aa7f535b95248f63329	7adb
38	bcb796bb950d3ef25ca9079a3f	ed88
39	fcfcef834da77616a685bf24b9	46c8
40	66b96bc129576180ba475c7794	1ca1
41	c5d45d74d885e34434fe38f38c	a8f7
42	f15f55f11065cc680d7aff54d6	0281
43	b784399bf3c0c9e11009365ce7	3552
44	df922891620196b96201b91352	dcc1
45	5324f145a44a9dab59e72b0230	0727
46	a0f31794c74e3117c61721644f	3cd8
47	b89ea43d4ad92733b0db04478b	7865
48	5e31699e8dd17f4a8f3a118b75	db4a
49	4d10cb08d779410e18424e34e8	027b
50	a2b4948a19efe2f01d60715500	89e6
51	a70d1462ed388c744871dbd22e	0def
52	5abf6fa33192d88538d5b1e9eb	f613
53	50cd6e72d83d058d93f7e5e6b6	2136
54	89535d3520efbf63095f246929	8b06
55	9db75f97f3b004725f1a8c7793	68c7
56	0c71fcdc429262a311ab1b91b9	3f12
57	07423d5da4aa078cc6c15f55d8	a2dd
58	25cf7acac6efcb2e5ba370988d	b98a
59	33a9f8c90b9e0170f576f8104e	6466
60	77b377e7ce149935ceb3a68ae5	6440
61	7884b72515701e84e9f8a36277	3c0f

Value 13's `0x990` word (ca68) was read after a decode, which does not change it; value 12 has the same word. Evidence: `eccmode2`, `eccmode4` (values 1 to 13), `finale` (values 14 to 61), `seqrestart` (values 7 to 18 again, in order).

What generates the sequence is not known. Checked and ruled out:

- All 61 values are distinct.
- Read as one bit stream (either byte and bit order), the sequence has a linear complexity of about half its length (3171 to 3173 of 6344 bits). Each bit position over the 61 values has a linear complexity of 27 to 34. So no short linear feedback shift register produces the sequence.
- Consecutive values are not one register multiplied by a fixed power of x modulo the BCH generator (checked at the roots a and a^3, both byte orders).
- The `0x990` values are not a 16-bit Galois LFSR stepped 1, 8, 13, 16 or 104 times per operation (all 65535 polynomials).

10.13 Registers

offset	name	meaning
0x800		reads `0x20060808` and ignores writes (device; an identification word, perhaps a date)
0x80c	ECC_KICK	start an operation: bit 0 decode, bit 1 encode, bits 9:8 select the data buffer (one-hot `0x100` or `0x200`), bits 13:12 select the parity buffer (`0x1000` or `0x2000`), bits 17:16 select the slot 0..3 (BCH; ignored by Reed-Solomon, which uses slot 0)
0x810	ECC_RESULT	after a decode: `n << 16` corrected (BCH bits, RS bytes), `0x11` uncorrectable (chapter 10, section 10.11)
0x814	ECC_MODE	the method and its options (chapter 10, section 10.2, full map 10.10); the original programs write `0x180` (BCH) before every kick
0x840	ECC_DONE	bit 0 = done (`wait` event 4); write `0x1ff` to clear all before a kick, 1 to acknowledge; bit 3 = a decode was uncorrectable (stays set), bit 5 (chapter 10, section 10.11)
0x844		interrupt enable of the ECC sources (6 bits, mask `0x2f`; 0 after power-on): with it non-zero, pending `ECC_DONE` bits raise `CS_IRQ` bit 6 and the CPU's interrupt source 43 (VIC1 line 11; device, `w2vic`)
0x900..0x913	ECC area	the parity of the last encode (BCH: 13 bytes, RS: 10 / 15 / 20 bytes) or the sequence value of the modes without a code; cleared by a decode
0x940..0x950		five read/write words; values 0, all ones and `0x55555555` changed neither the parity nor a decode (`eccregs`)

offset	name	meaning
0x990		16-bit value repeated in both halves: 0 after a BCH encode, data-dependent after an RS encode, the sequence value without a code
0x9a4		decode count

A decode corrects the data buffer in place; the DMA of the corrected data follows the decode (observed order in the original programs and in the third-party driver). A BCH encode writes slot bytes 3..15 and leaves bytes 0..2 as the metadata path put them (`eccchunk`, `rawwrite`).

Kick words used by the original programs. Slot k is in bits 17:16, and data buffer 1 is selected where bit 9 is set:

- decode: `0x1101`, `0x11201`, `0x21101`, `0x31201`;
- encode: `0x1102`, `0x11202`, `0x21102`, `0x31202`.

The third-party driver also uses decode kicks with parity buffer 1 selected (`0x2000`). This shows that any combination of buffers and slot is accepted.

Parity buffer by DMA (device). A parity buffer is 64 bytes (four 16-byte slots). A DMA into it (`FMC_DATA_SEL` = `0x10` or `0x20`, `FMCTRL1` = `0x2e0`) still moves 512 bytes. The data wraps over the 64-byte buffer, so the buffer ends up holding the last 64 bytes of the source (slot 0 = source bytes 448 to 463). A DMA out of it (`FMC_DATA_SEL` = `0x1000` or `0x2000`, `FMCTRL1` = `0x100`) starts at slot 0 (`eccchunk`).

Raw writes (device). A chunk can also be sent from a data buffer (`FMC_DNUM` = `0xf`, `FMC_DATA_SEL` = `0x100`, `FMCTRL1` = `0xe4`): only `FMC_DNUM` + 1 bytes go to the chip. A page with chunks built by the CPU is therefore written without the ECC engine at all, and reads back as written (`rawwrite`).

Why 512 bytes. A DMA always moves 512 bytes, whatever `FMC_DNUM` says. With a length of 256, the drain (data buffer to system memory) still wrote 512 bytes. With lengths of 513 to 2048, only 512 bytes arrived (`bufsize`). The DMA unit of the controller is therefore 512 bytes, and the ECC methods all work on 512-byte sectors.

10.14 Byte order

The FMSS engine is little-endian: instruction fields, memory words and the words of the ECC area are little-endian. The parity bytes in the slot and on the chip are stored most significant byte first. Details:

- BCH parity: 13 bytes, most significant byte first in the slot (bytes 3..15) and on the chip; the ECC-area words `0x900..0x90c` hold the same 13 bytes in reverse order.
- Reed-Solomon parity: the ECC area holds the symbols little-endian, 10 bits each; the slot receives the same 10, 15 or 20 bytes in reverse order, starting at slot byte 0.
- `ECC_MODE` bit 4 swaps the bytes of each 32-bit word of the data buffer read and of the slot written, for both codes; the code itself is unchanged.
- The metadata path takes `FMC_META0..2` little-endian and places bytes 3k to 3k+2 in slot k.

The interpretation of why the parity order differs from the rest, and of what `ECC_MODE` bit 4 is for, is given in chapter 15, section 15.7 (inferred).

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 5, chapter 7, chapter 9, chapter 11.

Chapter 11 — NAND operations

Summary: How the original firmware performs each NAND operation with the engine, with the device measurements that confirm or complement it. The chapter covers the common behaviour (including how a chip and a page are selected, section 11.1.14, with a worked example of a read, an erase and a write of chip CE1, block 3000, page 5), the ECC and data path shared by READ and PROGRAM, RESET and READ ID, page read, page program, block erase, and the fault handler. Each behaviour carries a label (`C1`..`C13`, `R1`..`R7`, `ID1`..`ID5`, `RD1`..`RD11`, `WR1`..`WR9`, `ER1`..`ER4`, `FL1`..`FL3`) defined in this chapter; other chapters use the labels as cross-references. The behaviours are those of the nine original programs (RE-2) and of the original CPU driver.

11.1 Common behaviour

11.1.1 C1: Placement

Each original program is a flat array of instructions starting at offset 0, 8-byte aligned (RE-1).

11.1.2 C2: Prologue

Before it accesses a chip, every original program writes the following registers (observed in all nine programs, RE-5):

- `CS_IRQ = 0xff`.
- `FMSS_C10 = 1`.
- `FMSS_C58 = 4`.
- `FMSS_C4C` = the offset of the program's fault handler (FL1).
- `FMCTRL1 = 0x0ff3f8e0`.
- `FMC_STATUS = 0x0ff00ffe`.

The third-party driver writes the last two (`FMCTRL1` and `FMC_STATUS`) before every operation.

11.1.3 C3: Control word

After the prologue, the programs write `FMC_CTRL0 = 0x801 | timing` (the timing word is in chapter 12, section 12.3). Later chip selection changes only bits 8:1 of `FMC_CTRL0`: the program reads the register, modifies those bits and writes it back (RE-9). The chip-enable bit is `1 << (CE index + 1)`. At any time exactly one chip-enable bit is set, except when the reset command is sent to all chips at once (ID2).

11.1.4 C4: CE index

The chip-enable index CE (0..7) of list entry `i` is `CE = cemap[cel[i]]`: `cel[i]` is the logical chip of entry `i`, and `cemap` is a table that maps logical chips to CE indices (RE-29, chapter 12, section 12.1).

11.1.5 C5: One command cycle

A command cycle has three steps: write the command byte to `FMC_CMD`, `wait` event 0, write `FMC_STATUS = 0x2` (acknowledging the event).

11.1.6 C6: Address cycles

The address cycles of one command are sent as follows:

1. Set `FMC_ANUM`, `FMC_ADDR0`, and `FMC_ADDR1` (only for five cycles).
2. Write `FMCTRL1 = 1`.
3. `wait` event 1.
4. Write `FMC_STATUS = 0x4`.

The formats:

use	ANUM	ADDR0	ADDR1
page read / program, five cycles: column 0 (two bytes), row (three bytes)	4	`row << 16`	`row >> 16`
block erase, three row cycles	2	`row`	not written
enhanced status `78h`, three row cycles	2	`row`	not written
READ ID, one cycle	0	0	not written

11.1.7 C7: Ready procedure (status poll)

To learn whether a chip has finished its array operation and to fetch its FAIL bit, the original programs do the following:

1. Select its CE and write FMC_DATA_SEL = 1.
2. Send command 70h (C5) and write FMCTRL1 = 0xca.
3. Read FMC_NSTAT and test bit 6 (0x40). If the bit is clear, wait event 5.
4. Acknowledge FMC_STATUS = 0x800000.
5. Read the FAIL bit as FMC_NSTAT & 1 (the 78h variant tests FMC_NSTAT & 3), then write FMCTRL1 = 0x20.

The chip is left in status-output mode. The original programs use one wait event 5 and do not read FMC_NSTAT again after it. This holds for the 13 uses of wait event 5 outside ERASE, which share one five-instruction shape. The 6 uses in ERASE arrange the wait differently, with C58 lowered (RE-7, RE-8).

The third-party driver does not rely on wait event 5. It polls FMC_NSTAT repeatedly, up to 10 000 reads per latch and up to 200 ms (3p). How wait event 5 is armed and consumed is described in chapter 7, section 7.1.

The ready wait after a page read or program takes 25 us to 2 ms. A block erase takes about 1.1 ms for a block holding data (device, eraseseq), the longest wait. In the device checks of the write sequence, the poll ran with FMC_DATA_SEL = 1 and latched the status byte repeatedly without wait events (writeseq2).

11.1.8 C8: Data transfer

A data transfer has these steps:

1. Set FMC_DNUM and FMC_DATA_SEL. For a chunk transfer also set FMC_24 (chunk read) or FMC_28 (chunk write), the byte offset of the slot in the parity buffer.
2. Write the FMCTRL1 code.
3. wait event 3.
4. Write FMC_STATUS = 0x8.

When a transfer is meant to overlap another operation, the program starts the other operation first and only then waits for the transfer's event (chapter 11, section 11.2).

11.1.9 C9: Acknowledge hygiene

In the original programs, every event flag in the table of chapter 7, section 7.1 is acknowledged (cleared) after the program's wait for it. Each program begins with all flags clear (the all-flags clear of C2). A stale set flag makes the next wait return before the data has moved and corrupts the transfer (observed by the third-party driver: "stale bits let the stage waits pass before the data has moved"). For the ECC engine, the program writes ECC_DONE = 0x1ff before every ECC_KICK and ECC_DONE = 1 after every wait event 4 (RE-12).

11.1.10 C10: End state

At every normal END of the original programs all CE bits in FMC_CTRL0 are cleared, bit 24 is cleared, and every flag is acknowledged.

11.1.11 C11: Absent chips

READID is the only program that may be given CE indices of chips that are not fitted. It waits only for wait events 0, 1 and 3 and never for wait event 5 (RE-24). In the other programs, a chip that does not answer gives no status byte. wait event 5 then never occurs, and the sequencer faults when the wait times out.

11.1.12 C12: One FMCTRL1 operation at a time

FMCTRL1 holds one pending operation: a second write replaces the first before it has run. Therefore, in the original programs, each FMCTRL1 code is followed by a wait for that operation's own completion event, and by its acknowledgement, before the next FMCTRL1 code is written.

Device: a read program wrote FMCTRL1 = 0x100 (DMA of a data buffer to system memory) and then at once FMCTRL1 = 0xe2 (NAND read into the other data buffer). The NAND read completed (wait event 3), but the DMA-done event (wait event 2, FMC_STATUS bit 20) never occurred. The sequencer timed out at that wait event 2 after the first sector of the page had reached system memory (finding 36).

Two data transfers overlap in time only inside a single FMCTRL1 code that names both data buffers through FMC_DATA_SEL (for example 0x2e4). ECC operations (ECC_KICK) are the exception: the ECC engine is a separate unit, and its operations run while one FMCTRL1 operation is in flight.

11.1.13 C13: Prime after a reset (CPU side)

After FMSS_CTRL = 0x30000, the sequencer executes instruction 0 of the first started program only if that instruction is end (startseq3). The original CPU driver follows its reset immediately with a start of an idle program of zero bytes (RE-33). The registers C58, C4C and C5C keep their values across the reset. If a previous program left C58 bit 0 set, every later END runs the fault handler once (device, pending, device run 5, pending, device run 6).

11.1.14 How a chip and a page are selected

This section collects how a run chooses the NAND chip and the page it accesses. The chip is chosen by FMC_CTRL0, the page by the row number in the address cycles.

Which chip.

- FMC_CTRL0 bits 8:1 hold one chip-enable bit per chip: bit 1 is CE0, bit 2 is CE1, and so on up to bit 8 for CE7 (chapter 9, section 9.1).
- Before it talks to a chip, a program writes FMC_CTRL0 with exactly one of these bits set (C3). Example control words: 0x43803 selects CE0, 0x43805 CE1 and 0x43809 CE2.
- With several bits set, the same cycles go to all those chips at once. The original programs do this only for the reset command FFh (ID2).
- On the unit tested, CE0 to CE3 each answered READ ID.

Who decides.

- The original CPU driver builds lists in memory and writes their addresses into parameter RAM (chapter 12, section 12.1): P_ROWS is the row of each page, P_CELIST is the logical chip of each page, and P_CEMAP points to the table cemap[8].
- The program steps through the lists. For entry i it loads the row and the logical chip from system memory with ldi, maps the logical chip through the table (CE = cemap[cel[i]]; the table is the identity 0..7 in the original CPU driver), and writes the matching FMC_CTRL0 word by a read-modify-write of bits 8:1 (C4, RE-29, RE-9).
- READID takes the entries of P_CELIST directly as CE indices, with no map, and uses P_ROWS as the count of chips (ID4).
- The erase list holds the first row of each block.

Which page.

- The row number is the block number times the pages per block, plus the page within the block. In the erase list the row is that of the first page of the block.
- The device experiments used 128 pages per block on the unit tested, with scratch blocks from block 3000 on (chapter 13, section 13.1.2).

How the row reaches the chip.

- The address cycles follow the table of C6. FMC_ANUM is the number of cycles minus 1. A page read or program sends 5 cycles: column 0 in two bytes, then the row in three bytes, with FMC_ADDR0 = row << 16 and FMC_ADDR1 = row >> 16. Erase and the 78h status send 3 row cycles with FMC_ADDR0 = row. READ ID sends one cycle with address 0.
- The program then writes FMCTRL1 = 1, waits for wait event 1 and acknowledges FMC_STATUS = 0x4.
- The controller sends the bytes of FMC_ADDR0 from the least significant byte upward, then the byte of FMC_ADDR1 (inferred from the formats that work on the device).
- An erase sent with a fourth address cycle (FMC_ANUM = 3) still erased the block: the chip ignores the extra cycle (device, pageorder).

Within the page. The column starts at 0, and the page is read or written as one stream in order: for each sector its 16-byte chunk, then its 512 data bytes (R1, chapter 10, section 10.9). The programs do not send a new column between sectors.

Two-plane programs. The first page of a pair is in block B and the second page in the next block, B+1. Both are addressed in the same way, and the second is sent with 81h (WR9, chapter 11, section 11.5.10).

11.1.15 Worked example: chip CE1, block 3000, page 5

One page, followed through a read, an erase of its block and a write, with every register write and the bytes that go over the NAND bus. The sequences are the ones the device experiments used on the unit tested (rawpage, rawwrite, eraseseq, writeseq2).

Choosing the chip and the page.

- Chip: CE1 is bit 2 of FMC_CTRL0, so the control word is 0x43805 (0x43801, the word with all chip-enable bits clear, with bit 2 set). Setting bit 24 as well gives 0x1043805, which allows DMA.
- Page: the row number is block x 128 + page = 3000 x 128 + 5 = 384 005 = 0x5DC05.
- For a read or a write the five address cycles carry column 0 (two bytes) and the row (three bytes): FMC_ANUM = 4, FMC_ADDR0 = row << 16 = 0xDC050000, FMC_ADDR1 = row >> 16 = 0x5. The bytes on the NAND bus are 00 00 05 DC 05 (lowest byte first; this order is inferred from the formats that work on the device).
- For an erase the three address cycles carry the row of the block's first page, 3000 x 128 = 384 000 = 0x5DC00: FMC_ANUM = 2, FMC_ADDR0 = 0x5DC00; bus bytes 00 DC 05.

Every command byte is sent the same way: FMC_CMD = the byte, wait event 0, FMC_STATUS = 0x2. Every set of address cycles is sent the same way: write FMC_ANUM, FMC_ADDR0, FMC_ADDR1, then FMCTRL1 = 1, wait event 1, FMC_STATUS = 0x4.

Waiting for the chip (status poll). The program writes FMC_DATA_SEL = 1 and sends command 70h. Then FMCTRL1 = 0xca latches the chip's status byte into FMC_NSTAT. While FMC_NSTAT bit 6 is clear, the program waits for wait event 5, acknowledges with FMC_STATUS = 0x800000 and latches again. Finally FMCTRL1 = 0x20 leaves status mode. The status byte read 0x81 while the chip was busy and 0xc0 when it was ready; bit 0 set would mean the operation failed.

Reading the page.

step	register writes	NAND bus
select the chip	FMC_CTRL0 = 0x1043805	CE1 active
start the page load	command 00h; address cycles (5); command 30h	00, 00 00 05 DC 05, 30; the chip copies the page into its page register (busy)
wait for the chip	status poll	70, status bytes until 0xc0
back to data	FMCTRL1 = 0x100e0; command 00h	00
per sector k = 0..3: the chunk	FMC_DNUM = 0xf, FMC_DATA_SEL = 0x10, FMC_24 = 16 x k, FMCTRL1 = 0x32, wait event 3, FMC_STATUS = 0x8	16 bytes from the chip into slot k of parity buffer 0
per sector: the data	FMC_DNUM = 0x1ff, FMC_DATA_SEL = 1, FMCTRL1 = 0xe2, wait event 3, FMC_STATUS = 0x8	512 bytes into data buffer 0
per sector: decode and correct the data in data buffer 0	ECC_DONE = 0x1ff, ECC_MODE = 0x180, ECC_KICK = 0x1101 + (k << 16), wait event 4, read ECC_RESULT	none
per sector: copy to system memory	FMC_DMA_ADDR = the system memory address, FMC_DATA_SEL = 0x100, FMCTRL1 = 0x100, wait event 2, FMC_STATUS = 0x100000	none
end	FMC_CTRL0 = 0x43801	CE1 inactive

The chip sends the page as one byte stream. The steps above read its first 2 KiB unit: four times a 16-byte chunk followed by 512 data bytes, 2112 bytes. The second unit of a 4 KiB page (bytes 2112..4223 of the 4224-byte page) follows in the same stream and is read the same way. The controller only splits the stream into the transfers above. No new address or command is needed between sectors.

Erasing the block.

step	register writes	NAND bus
select the chip	FMC_CTRL0 = 0x43805	CE1 active
erase	command 60h; address cycles (3); command D0h	60, 00 DC 05, D0; the chip erases all 128 pages of block 3000 (about 1.1 ms with data in it)
wait for the chip	status poll	70, 0x81 until 0xc0

Writing the page.

step	register writes	NAND bus
per sector: data into data buffer 0	FMC_CTRL0 = 0x1043805; FMC_DNUM = 0x1ff, FMC_DATA_SEL = 1, FMC_DMA_ADDR = the system memory address, FMCTRL1 = 0x2e0, wait event 2, FMC_STATUS = 0x100000	none (system memory to data buffer 0)
metadata (once per page)	FMC_META0..2 = the 12 metadata bytes, FMC_METACFG = 0x3210, FMC_METACTL = 1, poll until bit 0 clears	none
start the program	command 80h; address cycles (5)	80, 00 00 05 DC 05
per sector k: parity	ECC_DONE = 0x1ff, ECC_MODE = 0x180, ECC_KICK = 0x1102 + (k << 16), wait event 4	none (the 13 parity bytes go into slot k of parity buffer 0)
per sector: send the chunk	FMC_DNUM = 0xf, FMC_DATA_SEL = 0x1000, FMC_28 = 16 x k, FMCTRL1 = 0x34, wait event 3, FMC_STATUS = 0x8	16 bytes to the chip

step	register writes	NAND bus
per sector: send the data	FMC_DNUM = 0x1ff, FMC_DATA_SEL = 0x100, FMCTRL1 = 0xe4, wait event 3, FMC_STATUS = 0x8	512 bytes to the chip
confirm	command 10h	10; the chip programs the page (about 0.2 to 0.8 ms)
wait for the chip	status poll	70, 0x81 until 0xc0; bit 0 clear = written

A page can be written only at an erased position of a block. Programming changes bits from 1 to 0 only, so a page written twice without an erase holds the AND of both contents.

11.2 ECC and data path (shared by READ and PROGRAM)

11.2.1 R1: Page stream

A 2 KiB unit is four sectors, each stored as [16-byte chunk][512 data bytes]: 2112 bytes per unit. A 4 KiB page is two consecutive units: 4224 bytes, read or written as one continuous stream starting at column 0 (RE-15). The 16-byte chunk holds two things:

- the sector's ECC parity (13 bytes; device, chapter 10, section 10.9);
- one 3-byte fragment of the page's 12-byte spare metadata. The chunk of sector k carries fragment k (inferred from the original programs, device-confirmed on the flash).

On write, the chunk of a sector precedes its data.

11.2.2 R2: Buffers

The controller has two data buffers and two parity buffers. Each parity buffer is divided into four slots. A sector's chunk is read into slot k of a parity buffer (k = sector index within the unit), and its 512 data bytes are read into a data buffer. The decode kick names the data buffer, the parity buffer and the slot that hold the sector (R3). The original programs use parity buffer 0 for all four slots and alternate between data buffer 0 and data buffer 1. The third-party driver alternates between parity buffer 0 and parity buffer 1 as well.

11.2.3 R3: Decode of one sector (READ)

After a sector's chunk (in its slot) and its 512 data bytes have been fully transferred and the waits acknowledged, the original programs do the following (RE-12):

1. Write ECC_DONE = 0x1ff and ECC_MODE = 0x180.
2. Write ECC_KICK = the decode word (data buffer, parity buffer, slot).
3. wait event 4.
4. Write ECC_DONE = 1 and read ECC_RESULT.

The decode corrects the data in the data buffer in place. The DMA of that data buffer to system memory starts after the decode has completed. A decode overlaps the NAND read of the next sector into the other data buffer (the ECC engine is a separate unit). The DMA of a corrected data buffer and the NAND read into the other data buffer are two FMCTRL1 operations, so they run one after the other (C12): the program starts one, waits for its event, then starts the other. A data buffer is refilled only after its DMA has completed (its wait event 2 has returned). It is not drained or refilled while a decode or encode on it is running.

11.2.4 R4: Per-sector verdict

The page word (the result word res[i] of the page, chapter 12, section 12.2) is built from the verdicts of the page's sectors, both units of a 4 KiB page included, combined by OR (RE-13). After each decode:

- if ECC_RESULT bit 0 is set, the page word gets bit 30;
- otherwise, with n = (ECC_RESULT >> 16) & 0xf, if n > 0 the page word gets bit n - 1.

11.2.5 R5: Encode of one sector (PROGRAM)

After the sector's data is in a data buffer (DMA done, acknowledged) and after the page's meta encode (WR3), the original programs write ECC_DONE = 0x1ff, ECC_MODE = 0x180 and ECC_KICK = the encode word, then wait event 4 and ECC_DONE = 1. The chunk in the sector's slot is sent to the chip after the encode has completed. The encode of sector k+1 overlaps the NAND write of sector k's data.

11.2.6 R6: Meta layout word

FMC_METACFG selects which slots the 12 metadata bytes are spread over (three bytes per slot). The layout of the original programs uses slots 0..3 of parity buffer 0, one slot per sector of a unit; its word is 0x3210 (RE-16). The third-party driver places the four chunks of a unit in slot 0 of parity buffer 0, slot 0 of parity

buffer 1, slot 1 of parity buffer 0 and slot 1 of parity buffer 1, and uses `0x5140`. The nibbles of `0x5140`, taken from the lowest, are 0, 4, 1, 5: the slot index `buffer * 4 + slot` for fragments 0..3. Both words are inferred to describe the layout that the respective driver uses. The first word is observed in the original programs, the second in the CPU-driven read path.

11.2.7 R7: DMA addressing

`FMC_DMA_ADDR` is written with a unit's bus address before the first DMA of that unit. It is written again for the next unit once the previous unit's last DMA has been acknowledged. The address advances by 512 for each sector transfer. `FMC_DMA_CTRL = 7` is written before the last DMA of a run, in the original programs and in the third-party driver (RE-19). `FMC_CTRL0` bit 24 is set before the first DMA of the run. It is cleared before the final transfer without DMA (PROGRAM) or at the end of the run (READ). The reason for this placement is unknown.

11.3 RESET and READ ID (program READID)

11.3.1 ID1: Control word

READID writes `FMC_CTRL0 = 0x33201` with all CE bits clear (RE-10). The third-party driver uses `0x43803`, which has the CE0 bit set. Both words contain slow timing values. The ID read does not depend on the timing (3p: every legal combination reads the ID).

11.3.2 ID2: Reset

READID sets all eight CE bits at once and sends command `FFh` (C5). It then waits a settle delay (a loop that counts down the value of the parameter word `0xd14`) and only after the delay deselects all CEs. The original CPU driver does not reset the chips itself; READID is the only place the reset happens (RE-24, RE-36).

11.3.3 ID3: Settle delay

`P_CEMAP` holds a loop count for READID. After the reset command and before the first ID command, the program runs a two-instruction loop for that many iterations (the count must be at least 1). The original CPU driver passes `0x8000`. The third-party driver waits 5 ms, and a caller retries once after a further 5 ms if the first ID is blank. The delay takes the place of a status poll, because the listed CEs may be absent (C11).

11.3.4 ID4: Per CE

READID repeats the following for each of the `P_ROWS` list entries. The CE index comes directly from `P_CELIST`, with no `cemap`.

1. Select the CE.
2. Send command `90h`, then one address cycle with address 0.
3. Set `FMC_DNUM` = 7 and `FMC_DATA_SEL` = 1, write `FMCTRL1 = 0x1c2`, `wait` event 3, acknowledge. The 8 ID bytes are now in data buffer 0.
4. Set `FMC_DATA_SEL = 0x100` and write `FMCTRL1 = 0x340`.
5. Read `FMC_FIFO` twice.

The first read is the word stored into `idbuf[i]` (ID bytes 0..3, byte 0 in bits 7:0; for example `0xa514d3ad` for the ID bytes `AD D3 14 A5`). The second word (ID bytes 4..7) is read and discarded. The third-party driver reads once or twice. An absent CE yields blank data (0 or `0xffffffff`; the original CPU driver rejects both) and does not fault, because only `wait` events 0, 1 and 3 are used.

11.3.5 ID5: End

READID deselects all CEs and ends with no failure.

11.4 Page read (program READ)

Per page, the protocol as seen by the chip is:

1. Command `00h`.
2. Five address cycles.
3. Command `30h`. The chip is then busy for the page read time tR.
4. Status poll (`70h`, C7).
5. Command `00h`, after writing `FMCTRL1 = 0x100e0`.
6. The stream of R1: `4 * P_CHUNKS` sectors, with no further commands.

11.4.1 RD1: Pipeline across chips

READ first sends the page-load command 30h for several pages, each on a different CE, before it transfers the data of any of them. It then transfers the pages one after another, in request order, while the chips of the other pages are still busy. A chip does not receive a second read command before all units of its previous page have been transferred. The original program starts every page of a batch. A batch ends at the first page whose CE is already used in the batch (RE-17).

11.4.2 RD2: Before the data

For the chip whose page is to be transferred, READ selects it, runs C7, writes FMCTRL1 = 0x100e0, and sends command 00h. Without the status poll the read can start before tR has elapsed; without 00h the status byte is read instead of data.

11.4.3 RD3: Order inside a sector

READ first reads the 16-byte chunk into slot k of a parity buffer (FMC_DNUM = 0xf, FMC_DATA_SEL bits 5:4 selecting the parity buffer, FMC_24 = 16 * k, FMCTRL1 = 0x32, wait event 3). It then reads the 512 data bytes into a data buffer (FMC_DNUM = 0x1ff, FMC_DATA_SEL bits 1:0 selecting the data buffer, FMCTRL1 = 0xe2). The chunk of sector k is read before the sector's data and before the decode of sector k. The chunk of the next sector can be read while the previous sector decodes.

11.4.4 RD4: Decode and DMA

Decode and verdict are as in R3 and R4. The DMA of each sector to system memory (FMCTRL1 = 0x100; 0x1e0 for the last of the run) writes to the address in FMC_DMA_ADDR, which advances by 512 per sector, that is by 2 KiB per unit.

11.4.5 RD5: Unit boundary (4 KiB pages)

The next unit's first sector is read straight after the previous unit's last sector (one stream, no new command). The DMA address of the next unit comes from the next entry of the DMA list.

11.4.6 RD6: Switching chips

READ changes the CE selection only between pages, after the last NAND read of the previous page has completed (its data wait event 3 acknowledged). The decode, DMA and meta decode of the previous page's last sector overlap the status poll and the first sector read of the next page (the data and parity buffers are not tied to a CE). The meta decode (RD9) of a page finishes before the chunk read of the next page's sector 0 overwrites slot 0, if both pages use the same parity buffer.

11.4.7 RD7: Chunk-ECC flag

FMC_STATUS bit 27 is set by the controller while a chunk is read from the chip. It indicates that the sector being read had been written (programmed). The original firmware treats a page whose bit 27 stayed clear from the page start to the end of its last data read as "no ECC data" (an erased page). A third-party driver measured over a whole 4 GB unit that every programmed page sets the bit and every erased page does not (3p).

READ tests bit 27 once per page, after the last data read of the page has completed. It then acknowledges the bit (writes 0x8000000) so that the next page starts clear (RE-18).

Device: bit 27 was set after reading every written sector (valid code words, sectors with stored errors, and a written sector whose chunk is all 0xff) and clear after reading every erased sector (all 0xff). FMC_STATUS read 0x0803c000 after a written sector and 0x0003c000 after an erased one (rawpage, rawwrite).

11.4.8 RD8: Page result

The page word depends on bit 27:

- If bit 27 stayed clear, the page word is 0x20000000 OR-ed with the verdict of the page's last sector only. Anything accumulated from earlier sectors is discarded.
- Otherwise the page word is the OR of all sector verdicts (R4).

This is the exact behaviour of the original firmware: the test is made before the last sector's decode result is added (RE-18). Why the earlier verdicts are discarded is not known; decoding an erased sector gives ECC_RESULT = 0 (device, chapter 16, section 16.1, item 4). The word is stored in res[i] and ORed into r6.

11.4.9 RD9: Meta

After the last sector of the page (the last unit) has been decoded, READ writes FMC_METACFG (R6) and FMC_METACTL = 2, polls until bit 1 reads 0, and copies FMC_META0..2 into meta[3 * i .. 3 * i + 2]. The

original performs one meta decode per page, using the slots of the last unit (RE-16). On the chip, both units of a 4 KiB page carry the same 12 metadata bytes: in all 42 pages written by the original firmware that were read whole (pages 0, 1 and 64 of 7 blocks on CE0 and CE1), the 3 metadata bytes of each chunk of the second unit equalled those of the first (device, `fullpage`; WR3).

11.4.10 RD10: Erased and uncorrectable pages

The data is still DMA-ed to system memory (an uncorrectable page delivers whatever the data buffers hold), the metadata is still decoded and stored, and the run continues with the next page. Failure is reported only through the page word.

11.4.11 RD11: End

After the last page, READ clears `FMC_CTRL0` bit 24, deselects all CEs, and ends with `r6` = the OR of the page words.

11.5 Page program (program PROGRAM)

Protocol as seen by the chip, per page:

- Single plane: `80h`; five address cycles; `4 * P_CHUNKS` times `[16-byte chunk][512 data bytes]`; `10h`; then the chip is busy for the program time tPROG.
- Two plane (`P_FLAGS` bit 0): the two pages of a pair are on one CE, plane 0 first. First page: `80h`, address, data, `11h`. Second page: `81h`, address, data, `10h`.

11.5.1 WR1: First sector staging

Before the first command of the run, the program DMAs the first sector's 512 bytes from system memory into data buffer 0 (`FMC_DMA_ADDR` = first list entry, bit 24 set, `FMC_DNUM = 0x1ff`, `FMC_DATA_SEL = 1`, `FMCTRL1 = 0x2e0`, `wait` event 2, acknowledge).

11.5.2 WR2: Chip free

Before sending `80h` or `81h` to a CE that already got a program command in this run, the original programs check that the CE is ready and test its FAIL bit (C7). In a two-plane pair this includes the second page's `81h`: after `11h` the chip is busy for a short time, and it is polled before `81h` (RE-21). A FAIL ends the run (WR7).

11.5.3 WR3: Meta encode

For each page, after the previous page's last chunk has been sent and before the encode of the page's first sector, the program does the following (RE-16):

1. Write the page's 12 metadata bytes to `FMC_META0..2`.
2. Write `FMC_METACFG` (R6).
3. Write `FMC_METACTL` = 1.
4. Poll bit 0 until it reads 0.

The meta encode runs once per page. The same fragments are carried by every unit of a 4 KiB page (inferred).

11.5.4 WR4: Sector order

For each sector, the program does the following:

1. Encode (R5).
2. Send the chunk from its slot: `FMC_DNUM = 0xf`, `FMC_DATA_SEL = 0x1000` (parity buffer 0), `FMC_28 = 16 * k`, `FMCTRL1 = 0x34`, `wait` event 3.
3. Send the data: `FMC_DNUM = 0x1ff`, `FMC_DATA_SEL` = the data buffer to empty (and the data buffer to fill, for `0x2e4`), `FMCTRL1 = 0x2e4` (or `0xe4` without DMA), `wait` event 3.

With `0x2e4`, the next sector's data is DMA-ed into the other data buffer during the transfer. The next sector is the next unit's first sector, or the next page's first sector. The `wait` event 2 for that DMA comes before the next encode. Using `0x2e0` and then `0xe4` one after the other is equivalent.

11.5.5 WR5: DMA list

The DMA list has one entry per unit, consumed in order. The final page reads no entry beyond the last (RE-20). `FMC_DMA_CTRL = 7` is written before the last DMA. The last sector of the run is written with `FMCTRL1 = 0xe4` (no DMA) after `FMC_CTRL0` bit 24 is cleared.

11.5.6 WR6: Confirm

After the last data of a page, the program sends `10h` (or `11h` for the first plane of a pair), then waits for `wait` event 0 and acknowledges. Then it selects the CE of the next page. The next page's first sector is already staged in a data buffer and encoded, because the data and parity buffers are not tied to a CE.

11.5.7 WR7: Status and failure

The status poll with FAIL test is done (a) before a CE is reused in the run (WR2) and (b) for every CE used, after the last confirm. The FAIL test is `FMC_NSTAT & 1` for `70h`. On a FAIL, the original program does the following (RE-22):

1. Put the failing CE (the CE currently selected, one-hot) into r6 bits 7:0.
2. Set bit 28.
3. Run the common end (C10) and end.

The remaining pages are not programmed.

11.5.8 WR8: Result

Normal end: `r6` = the mask of the CEs used, bit 28 clear, and the CE bits in `FMC_CTRL0` cleared.

11.5.9 WR9: Variants

`P_FLAGS` bit 0 = 1 selects the `80h`/`11h`, `81h`/`10h` form (the original firmware's chip mode 8). The original firmware has further forms:

- two-plane with `80h` for both planes;
- status `78h` with three row cycles (the row of the page about to be programmed on that CE) and fail mask 3;
- cache program (WRITECACHE): it confirms a page with `15h` (cache program) instead of `10h` as long as the remaining page count AND (0 minus the bank count) is non-zero, and ends with a `71h` status sweep. The bank count comes from the parameter word `0xd04`; the program stores 0 minus it in `0xd44` (RE-46). With a bank count that is a power of two, the AND is non-zero while at least one full group of pages, one page per bank, remains (inferred).

These forms are used for chip modes 2, 3, 4, 9, 12, 13 and are not exercised by the validated chips of the original CPU driver (RE-27, RE-38). The `78h` forms differ in release 1.1.3. There, the final status sweep per CE reads the status with `78h` at four row addresses (a row given in parameter word `0xd00`, that row + `0x80`, rows `0` and `0x80`) with fail mask 3. The other releases use one `70h` with mask 1 (RE-28).

11.5.10 Device checks of the write sequence

The checks used scratch blocks. Every page was read back raw and compared byte for byte (data, metadata and parity) (`writeseq`, `writeseq2`, `pageorder`, `eraseseq`):

- One page per run, WR3 + WR4 with `0xe4` for the data: correct on CE0 and on CE1, also both CEs programmed back to back in one run (`80h` .. `10h` on CE0, then on CE1, then a status poll of each).
- Two-plane pair (`80h` .. `11h`, status poll, `81h` .. `10h`, blocks B and B+1): both pages correct, about 470 us.
- Several pages per run, with the status poll on `FMC_DATA_SEL = 1` (C7) and the status latch repeated without waits: eight pages in order with `0x2e4` overlap and `FMC_DMA_CTRL = 7` before the last DMA, pages 9 and 11, and pages 1 and 3. Every page was correct.
- A run that polled with `FMC_DATA_SEL = 0x100` (left over from the data transfer): the latch returned `0x70` instead of the status (chapter 7, section 7.1). The poll ended while the chip was still programming, and the chip ignored the next `80h` (that page read back erased). There is no page-order rule: with a correct poll, multi-page writes are correct.
- Program time (device, one run per page over every page of three blocks, CE0 and CE1, `progtime`): the run times fall into two groups in a fixed pattern.
  - Pages 0 to 3 and then every pair 6-7, 10-11, ..., 122-123 take about 478 us per run.
  - Pages 4-5, 8-9, ..., 124-125 and 126-127 take 728 to 873 us (CE1 50 to 90 us slower than CE0).
  - The pattern is the same in all three blocks (inferred: the chip's lower and upper pages of a cell pair).
  - The run time includes the transfer of the page (2112 bytes, about 0.21 ms) and the ready poll.
- A page programmed a second time without an erase: the flash holds old AND new (bits only go from 1 to 0), so the sectors decode uncorrectable; the status byte reports no FAIL (`0xc0`).
- Status byte: `0xc0` when ready; `0x81` right after `10h` and during an erase, `0x80` during a page read's tR.
- Erase time: about 1.1 ms for a block holding data, about 0.18 ms for a blank block.

- Erase (60h, three row cycles, D0h): about 1.1 ms; written pages read back erased afterwards; a two-plane erase (60h B, 60h B+1, D0h) erased both blocks; an erase on CE1 worked. An erase sent with four address cycles (FMC_ANUM = 3) erased the block (about 1.1 ms): the chip ignores the extra cycle.

11.6 Block erase (program ERASE)

NAND-visible protocol per entry: 60h; three row cycles; D0h; (chip busy tBERS); later status poll (70h, C7).

11.6.1 ER1: Entries

For each of the P_COUNT entries, ERASE selects cemap[cel[i]], sends command 60h, sends the address (FMC_ANUM = 2, FMC_ADDR0 = rows[i]), and sends command D0h. Several CEs erase in parallel: the program does not wait for tBERS between entries (RE-23). The original CPU driver lists each CE at most once per run.

11.6.2 ER2: Final sweep

After the last D0h, the program handles each CE used in turn: it selects the CE, runs the ready procedure C7 (the ready wait here is the long one), and takes FAIL = FMC_NSTAT & 1.

11.6.3 ER3: Result

Normal end: r6 = the mask of the CEs used, bit 28 clear. On a FAIL, the failing CE is set as a one-hot mask in r6 bits 7:0, bit 28 is set, and the program ends. The original CPU driver maps the mask back to the entry.

11.6.4 ER4: End

ERASE deselects all CEs and ends.

11.7 Fault handler and timeouts

11.7.1 FL1: Handler

Every original program has one handler at the offset written to FMSS_C4C (RE-25). The handler works as follows (RE-25):

1. It reads FMSS_C3C (the handler leaves the value in r2 for the CPU's diagnostic print).
2. It puts the selected CE bits ((FMC_CTRL0 >> 1) & 0xff) into r6 bits 7:0, clearing the old byte.
3. In READ, READID and RAWREAD it writes FMSS_C50 = 1 and sets r6 bit 27 unconditionally.
4. In ERASE and in the five WRITE programs it does the same only if FMSS_C50 bit 0 is set.
5. It deselects all CEs. In ERASE and the five WRITE programs it then also sets bit 28, the failure bit. A status FAIL jumps here with C50 clear, and therefore yields bit 28 without bit 27.
6. It ends.

The handlers differ in length. Counted from the offset in FMSS_C4C to the last instruction, a handler has 13 instructions in ERASE and in the five WRITE programs, and 9 in READ, READID and RAWREAD (RE-1).

11.7.2 FL2: Hung hardware

The original programs rely on the wait timeout of the hardware and on the 2 s bound of the CPU. A hung run is reported as a timeout and followed by a sequencer reset (chapter 3, section 3.5).

11.7.3 FL3: What the CPU does after a fault

The original CPU driver resets the sequencer, discards the run, and reports the bank from r6 bits 7:0 (for a program) or the entry (for an erase). A READ fault has no per-page result: the original CPU driver treats the whole run as failed and ignores res[].

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 9, chapter 10, chapter 12, chapter 7.

Chapter 12 — CPU interface: parameter words, result words and timing word

Summary: How the original CPU driver and the original programs exchange data through parameter RAM: the parameter words that carry a program's inputs and outputs, the result words that report each run, and the timing word that sets the NAND bus timing. The sequence the driver follows to start a run is in chapter 3, section 3.5.

12.1 Parameter words per program

All pointers are bus addresses of arrays of 32-bit little-endian words. A program derives the CE index as `cemap[cel[i]]`, where `cel[i]` is the logical chip of list entry `i`. The original CPU driver fills `cemap` with the identity map 0..7, and the original programs read the table (RE-29). The words are listed in order of offset. How the lists select the chip and the page of each access is described in chapter 11, section 11.1.14.

word	name	READID	READ	PROGRAM	ERASE
`0xd04`	banks	unused	in: the bank count; READ does not read it	in: the bank count; only WRITECACHE reads it (chapter 11, WR9)	in: the bank count; ERASE does not read it
`0xd08`	`P_IDBUF`	out: ID words			
`0xd0c`	`P_ROWS`	**count of CEs**	in: row (page number) per page	in: row per page	in: first row of the block per entry (`block * pages per block`)
`0xd10`	`P_CELIST`	in: CE indices to probe	in: logical chip per page	in: logical chip per page	in: logical chip per entry
`0xd14`	`P_CEMAP`	**reset settle loop count**	in: pointer to `cemap[8]`	in: pointer to `cemap[8]`	in: pointer to `cemap[8]`
`0xd18`	`P_COUNT`		in: pages (1..32)	in: pages (1..32; even for two-plane)	in: entries (1..8; each CE at most once)
`0xd1c`	`P_META`		out: 3 words per page	in: 3 words per page	
`0xd20`	`P_DMALIST`		in: one bus address per 2 KiB unit	in: one bus address per 2 KiB unit	
`0xd24`	`P_RESULTS`		out: one result word per page		
`0xd28`	`P_CHUNKS`		in: 2 KiB units per page (1 or 2)	in: 1 or 2	
`0xd44`	`P_FLAGS`			in: bit 0 = two-plane pairs	
`0xd48`	`P_WTIMING`			in: timing word	
`0xd4c`	`P_TIMING`		in: timing word		in: timing word

Layout of the lists (the original CPU driver builds them; `i` is the page position and `u` the unit index within a page):

- `rows[i]`, `cel[i]`: one word per page (the row and the logical chip of page `i`).
- `dma[i * P_CHUNKS + u]` = the bus address of unit `u` of page `i`. The units of a page lie one after the other in system memory, in order. The program reads exactly `P_COUNT * P_CHUNKS` entries, in this order. It reads no further entries, so the array need not be longer.
- `res[i]` (READ out): the result word of page `i`. The words are in request order, whatever order the pages are processed in.
- `meta[i * 3 .. i * 3 + 2]` (READ out / PROGRAM in): the 12 spare metadata bytes of page `i` as three words, `FMC_META0..2` in that order.
- `idbuf[i]` (READID out): the ID word read for list position `i`.

Limits per run: at most 32 pages, at most 2 units per page and at most 8 CEs. The program size is at most `0x1000` bytes for READ and PROGRAM, `0x800` bytes for ERASE and `0x400` bytes for READID.

12.2 Result words

READ: each page has a result word `res[i]`, and the OR of all page words is returned in `r6` (`SEQ_RESULT`). The bits of a page word:

bit	meaning
30	at least one sector of the page was uncorrectable (`ECC_RESULT` bit 0)

bit	meaning
29	marker: the chunk-ECC flag (FMC_STATUS bit 27) stayed clear for the whole page (RD8)
n - 1 (n = 1..15)	at least one sector needed n corrected bits (n = ECC_RESULT[19:16])

The original CPU driver classifies a page as failed if bit 30 is set, as corrected if any of bits 7:0 is set, and as clean otherwise. It does not interpret bit 29. After a fault, r6 holds the mask of the selected CEs in bits 7:0 together with bit 27 (0x08000000).

PROGRAM and ERASE, r6:

- Normal end: the bit mask of the CEs used (1 << CE), with bit 28 clear.
- Failure: bit 28 set and bits 7:0 = the failing CE or CEs as a one-hot mask. The original CPU driver takes the lowest set bit for a program failure, and the first list entry whose CE is in the mask for an erase failure.
- Fault (FL1): bit 27 is set in addition.

The original CPU driver tests only bit 28 and the CE bits.

READID: the original CPU driver does not read r6 after a clean END. After a fault, r6 is as for READ above.

12.3 Timing word

The program ORs a timing word into FMC_CTRL0. The word has the fields setup << 16 | hold << 12 | edo << 28. P_TIMING is used by READ, RAWREAD and ERASE, and P_WTIMING by the five WRITE programs. READID uses a fixed word (ID1).

The original CPU driver builds the word from three times in nanoseconds taken from the chip table record of the chip found: the cycle time, the low time and the high time of the bus signals (RE-35). T is the bus clock period in nanoseconds: 1000 divided by the value that the driver's clock function returns for argument 2, read as the bus clock frequency in MHz (inferred from that function). c1, c2 and c3 are the three times in whole clock periods, rounded up:

1. c1 = ceil(cycle / T), c2 = ceil(low / T), c3 = ceil(high / T).
2. If c3 + c2 < c1, then c3 = c1 - c2.
3. hold = max(c3 - 1, 1) and setup = max(c2 - 1, 1).
4. edo = min(hold + 1, 3); edo is the read-capture delay field (FMC_CTRL0 bits 29:28, chapter 9, section 9.1).
5. The word is (setup & 7) << 16 | (hold & 7) << 12 | (edo & 3) << 28.

The driver keeps the read word and the write word in a structure, at offsets 0x1c and 0x20. Before it starts a program, it copies them to the parameter words 0xd4c (P_TIMING) and 0xd48 (P_WTIMING). The figures come from a table of 18 records of 44 bytes, one record per chip type. The bus clock frequency is known only as an inferred value of about 108 MHz (chapter 7, section 7.5). The third-party driver runs the default word (4 << 16) | (3 << 12) (no EDO) on hardware.

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 11, chapter 3.

Chapter 13 — Experiments: what was done and why

Summary: This chapter lists every experiment run on the device, in the order of its first run, with the procedure and the question it was run to answer. It records no outcomes and no interpretation; outcomes are in chapter 14, their reading is in chapter 15.

13.1 How an experiment runs on the device

An experiment is one or more small sequencer programs (8-byte instructions, the format of chapter 5) that the CPU loads into memory and starts. For each run of a program the CPU first presets what the experiment needs (registers r0..r7, parameter words, other register words), writes the program address to `FMSS_PROG`, clears `CS_IRQ` and writes a start word to `FMSS_CTRL` (`0xfff5` unless the experiment sets another). During the run the CPU can write further words at set times (for example a second start word 1 ms or 2 ms after the start) and samples chosen registers (the PC, r0, `FMSS_CTRL`, `C08`, `C14` and others) at intervals. After the run it records the time to END, to a fault or to the time bound it gave the run, the register file, the PC, the control, NAND controller (FMC), ECC and parameter windows before and after, and the samples. Chip enables are deselected during the experiments unless the experiment says otherwise. Experiments that need a clean state write `C58`, `C4C` and `C5C` as 0 before each start.

The experiments ran in 38 device runs; a device run is one batch of experiments executed in sequence on one unit. The outcome of an experiment is the summary text its code printed; chapter 14 reproduces it word for word.

13.1.1 Soft reset

A soft reset, as used by the `restore` experiment and by experiments that start from a defined state, is: every writable NAND controller, ECC and sequencer word is written to its power-on value, the status words are acknowledged, the `0x100` block is rewritten by the sequencer, and parameter RAM is zeroed. `restore` then compares every word with the power-on values. A power cycle is the other way to reach the power-on state; `state` records the state as it is after a power cycle.

13.1.2 NAND scratch blocks

Experiments that write or erase NAND do so only in scratch blocks starting at block 3000 (128 pages per block), on CE0 and, in four experiments, on CE1, and in `startregion` also blocks above the user area. Blocks used (erased, programmed or, for `readev`, read): 3000 (`ev11`, `ev5`, `evsweep`, `op09b`, `rawwrite`, `writeseq`, `eraseseq`, `readev`), 3001 and 3002 (`writeseq`, `eraseseq`), 3004 to 3006 (`op09nand`), 3007 (`ev5ready`), 3008 to 3013 (`pageorder`), 3014 and 3015 (`writeseq2`), 3016 on CE0 and CE1 (`ev5two`). `rawpage` and `readev` only read. All other experiments do not access the NAND array; the experiments `chan1` and `chan1pins` read the chip ID. In device run 35: block 3000 (`evsweep2`, `irq044`), 3050 (`fmcbits`), 3060 and 3061 of CE0 and 3060 of CE1 (`progtime`). In device runs 36 and 37: blocks 3002 and 4000 of CE0 and block 3900 of CE1 (`startregion`), 3051 (`dmactl`), 3070 (`rschip`), 3071 (`evsweep3`, a program cancelled before its confirm).

13.1.3 Loading and reading the buffers

The buffers have no address (chapter 1, section 1.5), so every experiment reached them through the transfers of the NAND controller and the ECC engine. Known data was placed in a **scratch area** of system memory: the CPU wrote it there before the start, and read the area back after the run.

step	program writes	effect
data into data buffer 0 (1)	`FMC_CTRL0` = `0x1043801` (bit 24, DMA enable), `FMC_DNUM` = `0x1ff`, `FMC_DATA_SEL` = `0x1` (`0x2`), `FMC_DMA_ADDR` = bus address in the scratch area, `FMCTRL1` = `0x2e0`, `wait` event 2, `FMC_STATUS` = `0x100000`	512 bytes from system memory into the data buffer
data buffer 0 (1) back to memory	`FMC_DATA_SEL` = `0x100` (`0x200`), `FMC_DMA_ADDR`, `FMCTRL1` = `0x100`, `wait` event 2	512 bytes from the data buffer into the scratch area
chunks into parity buffer 0 (1)	as the first row with `FMC_DATA_SEL` = `0x10` (`0x20`)	the DMA moves 512 bytes and wraps over the 64-byte buffer: slot k receives source bytes 448 + 16k to 463 + 16k, so host-built chunks were placed in the last 64 bytes of the 512-byte source
parity buffer 0 (1) back to memory	`FMC_DATA_SEL` = `0x1000` (`0x2000`), `FMCTRL1` = `0x100`, `wait` event 2	the buffer from slot 0 onwards into the scratch area
parity computed into slot k	data buffer filled as above, then `ECC_DONE` = `0x1ff`, `ECC_MODE`, `ECC_KICK` = `0x1102` | k << 16, `wait` event 4	the engine writes the parity into slot k of parity buffer 0 (and a copy, reversed, into the ECC area `0x900..0x913`, read with `ld` or by the CPU)

step	program writes	effect
decode	stored data in a data buffer and the chunk in slot k (by DMA as above, or from the chip), then ECC_KICK = 0x1101 | k << 16, wait event 4, ld of ECC_RESULT	the data buffer corrected in place; the result word read by the program
from or to the chip	chip reads 0x32 (chunk into a slot, FMC_24 = 16k) and 0xe2 (data); chip writes 0x34 (slot, FMC_28 = 16k) and 0xe4 (data)	as in the original programs (chapter 11, section 11.2)

Encode and decode of one sector were run in the same program, because the set-up word at the start of each program clears the buffers (inferred, chapter 10, section 10.2.1). No experiment read or wrote a buffer through the FIFO port FMC_FIFO (0x080), the only window word known to reach a data buffer (chapter 1, section 1.5).

13.1.4 Steps around each program run

Each program of an experiment runs inside a fixed sequence of CPU steps. An experiment can skip single steps; its description then names them ("no preflight", "no clean-up", "no reset").

Before the start, in this order:

1. **Reset:** FMSS_CTRL = 0x30000, then 1 ms.
2. **Clean-up:** the CPU writes 0 to C58, C4C and C5C.
3. CS_IRQ is cleared.
4. **+0x400 = 0:** the CPU writes 0 to FMC_CTRL0 of the second register set (0x400).
5. **Preflight:** the CPU starts a program that consists of end only, with 0xfff5, and waits up to 10 ms for it to end. Its result is recorded ("preflight ok"); a failed preflight means the sequencer does not run at all.
6. FMC_CTRL0 (0x000) = 0, unless the experiment keeps it.
7. The experiment's own register writes (registers r0..r7, parameter words, other words such as C58 or C4C).
8. **Before-snapshot:** the CPU reads the windows 0x000, 0x400, 0x800, 0xc00 and 0xd00 (64 words each; 0x080 is not read). If the experiment asks for it, the CPU also reads all 1024 words of the window, 0x080 and its mirrors included.
9. **Start:** FMSS_PROG = the program address, CS_IRQ cleared, FMSS_CTRL = the start word.

After the end (or the time bound): the after-snapshot (and, if asked for, the full read of the window), then the **park**: FMSS_CTRL = 0x30000, 1 ms, the clean-up of step 2, the end-only program run once, CS_IRQ cleared, FMC_CTRL0 = 0. In the first device runs the park had no clean-up step; experiments that recreate those steps say so in their description.

13.2 Device runs

A device run is one session on the unit: emCORE is started on the iPod, and one command runs a list of experiments in order. Device runs are numbered in the order they were made. An evidence entry names the experiment and the device run it comes from (for example D-done5b-37: experiment done5b, device run 37); when an experiment ran in more than one device run, the run number tells the entries apart. Device run 37 repeated the experiments that device run 36 could not complete after the unit stopped answering.

device run	experiments run in it
1	regmap, clock, alu, memory, indirect, control, wait, unknown
2	pram, alu, indirect, control, c58, unknown
3	pram, alu, indirect, control, c58, unknown
4	resume, wait, op15
5	op12, c58ill, pending, c14
6	regmap, startseq, pending, op12ack, op10
7	regmap, startseq2
8	startseq3, oneshot
9	c50, holdout
10	holdout
11	op12wait, ev11, ev5, opsweep, optime, ctrlbits, fmcmap, evsweep
12	rwmask
13	c50x
14	evsweep, op0f, ctrlhold, evclear, timeregs
15	ev5rule, c08, chan1, readev, regnum, branch, waitf, opcost, op12rel, pause, fmc28, blk100, irqline
16	restore, readev, irqline, irqen, ev5t, r8

device run	experiments run in it
17	state, readev, irqline, irqen, ev5t, r8
18	state, ev8, evcons
19	restore
20	dmaloop
21	restore, readev, dmaloop
22	op0916
23	op16
24	opstate, op16b, op09
25	op09b
26	op09c
27	chan1pins, pcwrite, op12cont, ctrlfx, op09d, fwidth, irqsrc, eccpar, bufsize, eccstr, access
28	fields0, tmofield, eccstr2, ev5en
29	tmobits, eccmode, eccchunk, rawpage, rawwrite, writeseq, op09nand, eraseseq
30	eccchunk2, eccmode2, tmobits2, ev5ready, pageorder, op09w33
31	ev5arm, writeseq2, eccmode3
32	ev5two, b15ecc
33	eccmode4
34	finale
35	startdiff, startbits, rsdecode, vic20, fmc28b, c58b1, evsweep2, c50fault, ctrlwords, c08self, latchrep, irq044, fmcbits, fmcstat, eccregs, seqrestart, done5, progtime, regsweep
36	seqfirst, w2state, c50b8, startregion, bufport, c58b2, stprev, tmwait2, irqop
37	done5b, rsresult, rschip, miscorr, fullpage, dmactl, fmcstat2, ch1buf, evsweep3, blk100b, straps, irqop
38	w2vic

13.3 Experiments

13.3.1 regmap

Device runs: 1, 6, 7.

Procedure and question: The experiment checks whether the registers r0..r7 are the CPU-visible words at 0xc18 + 4n, and whether SEQ_RESULT (0xc30) is r6.

13.3.2 clock

Device run: 1.

Procedure and question: The experiment measures how fast the sequencer executes instructions, using a loop of two instructions run at two different lengths.

13.3.3 alu

Device runs: 1, 2, 3.

Procedure and question: The experiment tests the edge cases of the ALU instructions: the kind of shift and the range of shift counts, wrap-around, the form selected by an immediate (imm) of 0, and the mask of ld.

13.3.4 memory

Device run: 1.

Procedure and question: The experiment tests ldi and sti on system memory (the scratch area): word access, byte order, and an unaligned address.

13.3.5 indirect

Device runs: 1, 2, 3.

Procedure and question: The experiment checks which offsets pld and pst reach: the register file, parameter RAM, and offsets beyond 0xfff.

13.3.6 control

Device runs: 1, 2, 3.

Procedure and question: The experiment records what the sequencer's control words (0xc00..0xc5c) read back after a run, and checks whether the CPU can preload r0..r7.

13.3.7 wait

Device runs: 1, 4.

Procedure and question: The experiment runs `wait` on event N while nothing is pending. It finds which events are already set, which time out, and how long the timeout is.

13.3.8 unknown

Device runs: 1, 2, 3.

Procedure and question: The experiment runs opcodes without a known meaning and records what they change. The registers are seeded with values beforehand and the fault vector is set.

13.3.9 pram

Device runs: 2, 3.

Procedure and question: The experiment checks which words of parameter RAM (0xd00..0xdfc) keep a value written by the CPU, and which words the sequencer can read and write.

13.3.10 c58

Device runs: 2, 3.

Procedure and question: The experiment checks what each value of FMSS_C58 does to a `wait` that times out: the time taken, the fault flag, and whether the program continues at the fault vector or at the next instruction.

13.3.11 resume

Device run: 4.

Procedure and question: The experiment checks where a start begins execution. The CPU sets FMSS_C58 and FMSS_C5C before the start, and a marker is placed at the fault vector.

13.3.12 op15

Device run: 4.

Procedure and question: Opcode 0x15 had changed FMSS_C14: to 0 with imm 0, and to 0xff with imm 0xd88. The experiment checks what value the opcode writes and from which instruction field the value comes.

13.3.13 op12

Device run: 5.

Procedure and question: Opcode 0x12 stops the program without an event timeout and latches itself in 0xc40/0xc44. The experiment checks whether it is a conditional wait on a register. The CPU presets 0xd60 to 0x00001234. In each case the instruction either passes (the program continues) or stops (the time bound of the test expires).

13.3.14 c58ill

Device run: 5.

Procedure and question: The experiment checks whether the bits of FMSS_C58 also govern the fault raised by an illegal opcode (0x09), as they govern the timeout of a `wait`.

13.3.15 pending

Device runs: 5, 6.

The experiment changed between runs. Procedure and question per version:

- device run 5: The state found in the second device run. Does a program that leaves C58 bit 0 set (after a fault, or not) make the next start begin at the fault vector? The programs run back to back with no cleanup in between, so the state is kept.
- device run 6: The state found in the second device run. A program leaves C58 = 1 (after a fault, or not); where do the next starts begin? This version recreates the steps used then (chapter 13, section 13.1.4): a park without the clean-up after the run, and reset, the `0x400` write and the preflight before it.

13.3.16 c14

Device run: 5.

Procedure and question: The experiment checks whether a write by the sequencer sets FMSS_C14, and whether opcode 0x15 inverts it (applied twice, the value is unchanged).

13.3.17 startseq

Device run: 6.

Procedure and question: After a sequencer reset, a START is accepted but nothing executes unless a further step is taken. The experiment checks which step lets a program run.

13.3.18 op12ack

Device run: 6.

Procedure and question: Opcode 0x12 stops the program with CS_IRQ bit 3 set. The experiment checks which CPU write, made 2 ms after the start, lets the program continue.

13.3.19 op10

Device run: 6.

Procedure and question: A run of opcode 0x10 had ended with CS_IRQ = 3. The experiment checks whether the opcode sets CS_IRQ bit 1 and lets the program carry on.

13.3.20 startseq2

Device run: 7.

Procedure and question: A START right after a reset runs the program, but with register accesses in between it does not. The experiment looks for the access responsible. Each case is a reset with no preflight and no cleanup; an END flag in CS_IRQ means that the program ran.

13.3.21 startseq3

Device run: 8.

Procedure and question: After a reset, a START fetches instruction 0 (into C40/C44) but does not execute it unless it is END. The experiment confirms this and looks for what releases the instruction, trying a CPU write of FMSS_CTRL 2 ms after the start. Each case is a reset with no preflight and nothing else in between.

13.3.22 oneshot

Device run: 8.

Procedure and question: FMSS_C58 bit 1 makes the next END run the fault vector once. The experiment checks whether the bit is also armed by a write from the sequencer, and whether a fault disarms it.

13.3.23 c50

Device run: 9.

Procedure and question: The original fault handlers test FMSS_C50 bit 0 while they run and then write 1 to it. The experiment checks whether a fault sets the bit, and whether the acknowledge or the END clears it.

13.3.24 holdout

Device runs: 9, 10.

Procedure and question: The experiment tests rules found in earlier experiments on combinations of conditions that none of those experiments used. For each combination the outcome that the rules predict was written down before the run, and the run shows whether the device agrees.

13.3.25 op12wait

Device run: 11.

Procedure and question: The experiment checks whether opcode 0x12 is a `wait` without a timeout, with the event number in the register field as in `wait`. It makes no NAND access.

13.3.26 ev11

Device run: 11.

Procedure and question: The experiment checks whether `wait` event 11 is the chips' ready line. It runs `wait` event 11 right after starting a block erase on CE0 (destructive, in a scratch block).

NAND access: Erases block 3000 of CE0 (the erase is started and left running while the event is tested).

13.3.27 ev5

Device run: 11.

Procedure and question: The experiment checks whether `wait` event 5 depends on a refreshed status byte or on the chip being ready. It takes a status poll (70h, FMCTRL1 = 0xca) during an erase and when the chip is idle.

NAND access: Erases block 3000 of CE0 (status polls during the erase).

13.3.28 opsweep

Device run: 11.

Procedure and question: The experiment runs every opcode 0x00..0xff without a known meaning, in three operand forms. The registers are seeded with harmless pointers, C58 is 0 (a fault sets its flag and the program continues) and the timeout is 20 ms. It records which opcodes end the program, fault, block, stall or change something.

13.3.29 optime

Device run: 11.

Procedure and question: The experiment checks whether opcodes with no visible effect take time. Each is executed 1000 times in a loop and compared with `movi`, in several operand forms. The same battery includes each opcode that opsweep found to continue fast with no change.

13.3.30 ctrlbits

Device run: 11.

Procedure and question: The driver writes 0xfff5 as the start word. The experiment sets each other bit alone (together with bit 0) on a loop of 1000 iterations, and checks whether any bit changes the speed, single-stepping, interrupts or the start itself.

13.3.31 fmcmap

Device run: 11.

Procedure and question: The experiment reads the whole 4 KB register window as the sequencer sees it (pld of every word into RAM). This includes registers outside the windows that the CPU snapshot covers.

13.3.32 evsweep

Device runs: 11, 14.

The experiment changed between runs. Procedure and question per version:

- device run 11: All 16 `wait` events are tried around NAND activity (destructive: erases scratch block 3000 of CE0). The experiment records which pass at once, which pass later and which time out, and reads FMC_STATUS just before and after each wait.
- device run 14: All 16 `wait` events are tried around NAND activity (destructive: erases scratch block 3000 of CE0). The experiment records which pass at once, which pass later and which time out, and reads FMC_STATUS after each wait. A sequencer read of FMC_STATUS before the wait had consumed events on the device, so this version does not read it first.

NAND access: Erases block 3000 of CE0.

13.3.33 rwmask

Device run: 12.

Procedure and question: For every word of the 0x000, 0x800, 0xc00 and 0x400 windows (registers that trigger an action are left out), the experiment writes all ones and then zero from the CPU before the start and reads the word back. This shows which bits are writable, read-only or write-1-to-clear.

13.3.34 c50x

Device run: 13.

Procedure and question: The experiment checks which fault sets FMSS_C50 bit 0. It runs on its own and last, because the bus cases may hang the SoC.

13.3.35 op0f

Device run: 14.

Procedure and question: A device run had shown that opcode 0x0f acts as an indirect jump (the PC ended at the seeded register's value + 8). The experiment checks which register field names the target, whether the target is a byte offset from the program start, and whether a return address is kept anywhere.

13.3.36 ctrlhold

Device run: 14.

Procedure and question: FMSS_CTRL bits 1, 3 and 16 kept a program from running (device). The experiment checks whether they pause it (a later write without them releases it), whether they single-step it (each later write runs one instruction), and whether bit 17 clears r0..r7.

13.3.37 evclear

Device run: 14.

Procedure and question: The experiment checks whether a sequencer read of FMC_STATUS (or FMC_NSTAT) consumes a pending event before a `wait` sees it.

13.3.38 timeregs

Device run: 14.

Procedure and question: The experiment checks whether any of the writable sequencer words with no known use (C10, C3C, C48, C54) changes the `wait` timeout.

13.3.39 ev5rule

Device run: 15.

Procedure and question: The experiment checks what arms `wait` event 5. A status latch (FMCTRL1 = 0xca) after 70h had passed `wait` event 5 when the command-done flag (FMC_STATUS bit 1) had been cleared first, never when the flag was still set, and never for a second latch.

13.3.40 c08

Device run: 15.

Procedure and question: The experiment checks whether FMSS C08 is a state code. The CPU reads it while a program runs, waits, stalls, blocks, is paused or is held.

13.3.41 chan1

Device run: 15.

Procedure and question: The experiment checks whether the register set at +0x400 is a second NAND channel. It sends READ ID through that set for each chip enable and compares the result with the same READ ID sent through the first set. Waits that time out continue (C58 = 0).

NAND access: Reads the chip ID through the second register set and through the first; no array access.

13.3.42 readev

Device runs: 15, 16, 17, 21.

Procedure and question: The experiment tries all 16 `wait` events at each step of a page read (CE0, scratch block 3000, page 0). The steps are 00h, address, 30h, status poll, parity and data transfer, ECC decode, and DMA. It records which event belongs to which step and which events pass together.

NAND access: Reads page 0 of block 3000 on CE0; no erase or program.

13.3.43 regnum

Device run: 15.

Procedure and question: The experiment checks which register is read and written when a register number from 8 to 255 is used in the register field and in the argument field.

13.3.44 branch

Device run: 15.

Procedure and question: The experiment tests the edge cases of bz and bnz: a misaligned target, the argument field, and a sign bit.

13.3.45 waitf

Device run: 15.

Procedure and question: The experiment checks the other fields of `wait`. The 4G/5G engine waits on a register under a mask. The experiment checks whether the 3G uses arg and imm, and whether event numbers above 15 act as their low bits.

13.3.46 opcost

Device run: 15.

Procedure and question: The experiment measures the time per instruction type, with each type executed 100 x 10 times in a loop and compared against movi. The program runs from SDRAM and from internal SRAM.

13.3.47 op12rel

Device run: 15.

Procedure and question: Opcode 0x12 stops the program. The experiment checks whether any CPU write other than those tried so far releases it. The write is made 2 ms after the start.

13.3.48 pause

Device run: 15.

Procedure and question: The experiment checks whether start-word bit 1, written while a program runs, pauses it midway, and whether a later start without that bit resumes it.

13.3.49 fmc28

Device run: 15.

Procedure and question: The experiment checks what FMC_28 counts, by running 64 instructions of one kind and comparing the result with an empty program.

13.3.50 blk100

Device run: 15.

Procedure and question: The experiment reads the block at 0x100..0x1fc (repeated at 0x300, 0x500 and 0x700). It then writes all ones and zero from the sequencer and reads back what is left, at 0x100 and at the 0x300 copy. Finally it writes the old value back.

13.3.51 irqline

Device runs: 15, 16, 17.

Procedure and question: The experiment checks which sequencer states raise the CPU interrupt line (VIC1 raw bit 22), and whether start-word bit 6 is its enable.

13.3.52 restore

Device runs: 16, 19, 21.

The experiment changed between runs. Procedure and question per version:

- device run 16: A soft reset is tried instead of a power cycle. The state is recorded before; then the writable NAND controller, ECC and sequencer words are written back to the values they had before any experiment touched them, and pending ECC_DONE bits are acknowledged; then the state is recorded after, for comparison with `state` taken right after a power cycle.
- device runs 19, 21: A soft reset is tried instead of a power cycle. Every writable NAND controller, ECC and sequencer word is written to its power-on value, status words are acknowledged, the 0x100 block is rewritten by the sequencer, and parameter RAM is zeroed. Then every word is compared with the power-on values. What differs afterwards can be cleared only by a power cycle or by a reset not found yet.

13.3.53 irqen

Device runs: 16, 17.

Procedure and question: The experiment uses NAND controller word 0x044 and ECC word 0x844 as interrupt enables. One enable bit is set at a time, and then a command byte (FMC_STATUS bit 1) or an ECC decode

(ECC_DONE) is triggered. It records which CS_IRQ bit is set and whether the CPU interrupt line follows.

13.3.54 ev5t

Device runs: 16, 17.

Procedure and question: In evsweep, `wait` event 5 straight after 70h and a status latch timed out, but in ev5rule it passed when a `wait` event 0 was placed in between. The experiment checks whether the cause is timing, the pending command flag, or an ordering rule.

13.3.55 r8

Device runs: 16, 17.

Procedure and question: Register numbers 8 and 10 had effects: `movi r8` ended the program early and `movi r10` wrote every register. The experiment checks whether r8 is the PC and what r10 is.

13.3.56 state

Device runs: 17, 18.

Procedure and question: The experiment records the state as it is. It runs first after a power cycle, so that it records the power-on values.

13.3.57 ev8

Device run: 18.

Procedure and question: `wait` event 8 passed at every step after a soft reset (the register set at +0x400 still held a READ ID sent through it) and never after a power cycle. The experiment checks whether events 6..15 belong to the second register set. It runs the READ ID of the second set step by step and tries every event after each step.

13.3.58 evcons

Device run: 18.

Procedure and question: The experiment checks whether a `wait` consumes its event. The sequence '70h, latch, `wait` event 15, `wait` event 5' had timed out once while every other order passed.

13.3.59 dmaloop

Device runs: 20, 21.

The experiment changed between runs. Procedure and question per version:

- device run 20: DMA without a chip, through each register set: from RAM into the controller's data buffer (FMCTRL1 = 0x2e0) and back to RAM (FMCTRL1 = 0x100, or 0x1e0 after FMC_DMA_CTRL = 7). Does the 0x400 set have its own buffers and DMA, which event says DMA done, and why did the first set's DMA in the page read never finish?
- device run 21: DMA without a chip, through each register set: from RAM into the controller's data buffer (FMCTRL1 = 0x2e0) and back to RAM (FMCTRL1 = 0x100, or 0x1e0 after FMC_DMA_CTRL = 7). Does the 0x400 set have its own buffers and DMA, which event says DMA done, and does a DMA need FMC_CTRL0 bit 24 (the read programs set it around their DMA; the probes did not)?

13.3.60 op0916

Device run: 22.

Procedure and question: Opcodes 09 and 16 had been classed as illegal. The experiment checks whether they are really illegal or are waits whose condition the opcode sweep never met. Each is run with operands that would satisfy a register-and-mask wait (the 4G/5G `wait` has that form; the 3G `wait` ignores arg and imm), with a register-field condition, or with a short delay. Opcode 07 (`wait`) and opcodes 1a and 20 (illegal) serve as references.

13.3.61 op16

Device run: 23.

Procedure and question: Opcode 16 timed out in the opcode sweep but passed at once in every form after DMA, ECC and page-read runs. The experiment checks which condition lets it pass. Each case is a soft reset, one action, and then opcode 16 (with opcode 09 as a reference).

13.3.62 opstate

Device run: 24.

Procedure and question: The experiment covers the opcodes before 1a that showed nothing or only timed out (08, 09, 10, 12, 15, 16, ff). It checks whether any of them reacts to pending completion flags of either register set. Each case is a soft reset, one state, and then the opcode. C4C points to a handler, so a jump to the fault vector shows up as 'vector'.

13.3.63 op16b

Device run: 24.

Procedure and question: Opcode 16 jumped to the fault vector at once while a completion flag was pending, and otherwise timed out. The experiment checks whether it goes to C4C whatever C4C holds, whether it waits for a flag that arrives later, whether C58 matters, and whether the flag is cleared.

13.3.64 op09

Device run: 24.

Procedure and question: The experiment checks whether opcode 09 means 'wait until the timeout' (a real instruction) or is just illegal. It is compared with opcodes 1a and 20 and with an unmet `wait`. It is started once with the timer `C14` at 0 (a `tmwait` instruction, opcode `15`, run before it leaves `C14` at 0) and once with `C14` at `0xff`. Timing and state while it waits are recorded, and cases with a pending flag named in its condition number or its mask are included.

13.3.65 op09b

Device run: 25.

Procedure and question: Opcode 09 (`wait2`) with condition number 5 continued at once while the status flag (`wait` event 5) was pending, and timed out for the other flags. The experiment checks whether it is a status wait with a mask test on FMC_NSTAT (the 4G/5G 'wait for ready' form), which event numbers it takes, whether it needs the flag, and whether it waits for a busy chip to become ready.

NAND access: Erases block 3000 of CE0 in some cases.

13.3.66 op09c

Device run: 26.

Procedure and question: Opcode 09 (`wait2`) passed with condition number 5 in every state tried and timed out with condition numbers 0..15 otherwise. From a clean state with nothing pending, the experiment checks which condition numbers 0..31 let it pass, with no chip selected and with CE0 / CE1 selected. As a reference, `wait` events 0..15 are tried in the same states.

13.3.67 chan1pins

Device run: 27.

Procedure and question: Channel 1 of the NAND controller (the 0x400 register set) has its own pins, which the test setup leaves unswitched (the boot ROM switches them for a channel-1 NAND boot). The experiment sends READ ID through channel 1 on CE0..7 with those pins switched for the run. As a control, READ ID through the first set is run with and without the pin switch.

NAND access: Reads the chip ID through the second register set and through the first; no array access.

13.3.68 pcwrite

Device run: 27.

Procedure and question: The CPU cannot write the PC (0xc38). The experiment checks whether the sequencer can, and whether such a write makes a jump (as a write to r8 does).

13.3.69 op12cont

Device run: 27.

Procedure and question: Opcode 12 stops the program with CS_IRQ bit 3. The 4G/5G driver treats CS_IRQ bit 3 as 'pause' and continues the program with three steps: FMSS_PC += 8, CS_IRQ = 8, FMSS_CTRL |= 1. The experiment checks whether this sequence, or a part of it, continues the program on the 3G.

13.3.70 ctrlfx

Device run: 27.

Procedure and question: The experiment checks what start-word bits 2 and 4..15 enable. Each start word is run on four behaviours that an enable could gate: a `wait` that times out, an illegal opcode, evjmp with a flag pending, and the CPU interrupt line with the NAND controller command-flag source enabled (0x044 bit 0).

13.3.71 op09d

Device run: 27.

Procedure and question: The experiment checks whether `wait2` (opcode 09) waits on the second register set's flags. Each flag of the second set (0x400) is made pending on its own (command, address, transfer, DMA, status latch). In that state `wait2` conditions 0..15 and `wait` events 0..15 are tried.

13.3.72 fwidth

Device run: 27.

Procedure and question: The experiment checks field widths: whether wr, st and ld use the upper bits of their 16-bit offset, and whether branches, jr and writes to r8 use the upper bits of their 32-bit target. (pld and pst wrap at 12 bits, which is already known.)

13.3.73 irqsrc

Device run: 27.

Procedure and question: The experiment tests interrupt sources. Each flag of the first register set (command, address, transfer, DMA, ECC, status) is made pending on its own, with no enables, with all NAND controller enables (0x044 = 0xffff), or with all ECC enables (0x844 = 0x2f). CS_IRQ, C50 and every VIC0 / VIC1 raw line are recorded. It then finds which single 0x044 bit routes each flag that reached CS_IRQ.

13.3.74 eccpar

Device run: 27.

Procedure and question: The experiment checks where the ECC parity bytes and the decoder's results become visible. It uses no flash. A known 512-byte sector is moved by DMA into data buffer 0 and encoded into parity buffer 0 slot 0, then decoded unchanged and with one and two bits flipped. The ECC engine window words 0x800..0x9fc are dumped after each step.

13.3.75 bufsize

Device run: 27.

Procedure and question: The experiment checks how big a data buffer is. N bytes of a known pattern are moved by DMA into data buffer 0 and N bytes are moved back out (FMC_DNUM = N - 1), for N = 256, 512, 513, 1024 and 2048. It checks whether a longer transfer wraps, stops or times out at 512.

13.3.76 eccstr

Device run: 27.

Procedure and question: The experiment measures the ECC correction strength. A known sector is encoded and then decoded with 1..12 bits flipped, spread over the sector, to find from how many flipped bits it becomes uncorrectable. It also checks whether ECC_MODE (always 0x180 in the original programs) changes that, with other modes tried at 8, 9 and 12 flipped bits.

13.3.77 access

Device run: 27.

Procedure and question: For every word 0x000..0xffc, the experiment checks whether a CPU write reads back to the CPU and to the sequencer, and whether a sequencer write reads back to the sequencer and to the CPU. It writes all ones and zero and notes which bits follow the write. The original values are put back after each group of 16 words, and a soft reset is done at the end.

13.3.78 fields0

Device run: 28.

Procedure and question: The experiment covers the fields that the original programs always leave 0: end's R / arg / imm, wr's R, movi's arg, and the imm of st, ldi, mov, sti, pld and pst. Each is tried with a mask-like value (0xff) and an offset-like value (4) to see whether anything changes.

13.3.79 tmofield

Device run: 28.

Procedure and question: The experiment checks the timeout length. Start-word bit 2 enables the timeout; the experiment checks which bits set its length, and in what unit.

13.3.80 eccstr2

Device run: 28.

Procedure and question: The experiment measures ECC strength and identifies the code, with encode and decode in one program (the set-up word at the start of a program clears the buffers). 0..12 bits are flipped before the decode. The parity of single-bit sectors is recorded so that the code can be identified by comparing it with computed parity afterwards.

13.3.81 ev5en

Device run: 28.

Procedure and question: In every run with an interrupt enable written, the status latch failed (FMC_STATUS bit 24 was set instead of bit 23, and there was no `wait` event 5). The experiment repeats the runs three times each with no enable, with single enable bits, and with the ECC enable. It also runs the case in which the program, instead of the CPU, writes the enable.

13.3.82 tmobits

Device run: 29.

Procedure and question: This is a second sweep of the timeout length. It checks which of start-word bits 7:4 enable or scale the count (bits 7:4 = 0 gave an immediate timeout, 0xf the long one), and how the length depends on bits 15:8 with bits 7:4 = 0xf.

13.3.83 eccmode

Device run: 29.

Procedure and question: The experiment tries ECC_MODE values other than 0x180 and checks whether the parity (in the slot and in the 0x900 area, with the whole ECC engine dumped) or the strength changes. For each mode a sector is encoded and then decoded in the same program with 0, 8, 9, 12 and 16 flipped data bits.

13.3.84 eccchunk

Device run: 29.

Procedure and question: (1) Can a parity slot be filled by DMA from memory? This is needed for raw writes and for chunk tests. (2) The meta encode: FMC_META0..2, FMC_METACFG = 0x3210 and FMC_METACTL = 1 are written, then an ECC encode is run. Where do the 12 metadata bytes land, and are they part of the parity? (3) Decodes of chunks built by the CPU (to the identified code) with errors in the metadata or parity bytes.

13.3.85 rawpage

Device run: 29.

Procedure and question: Pages written by the original firmware are read raw (chunk and data as stored, before the correction). They are checked against the identified code by a separate computation, and the engine's own decode of the same pages is also run.

NAND access: Reads six rows of CE0 (rows 0, 128, 12801, 128000, 256005, 448000) raw; no erase or program.

13.3.86 rawwrite

Device run: 29.

Procedure and question: DESTRUCTIVE (scratch block of CE0): erase, then program pages by every path and read them back raw. Case p0 is the engine's own path (meta encode and encode, parity slot taken from the parity buffer) and shows the on-flash layout and the metadata order. Case p1 uses chunks built by the CPU and moved by DMA into the parity buffer. Case p2 uses chunks built by the CPU and sent from a data buffer (no ECC engine at all: a raw write). Case p3 stores errors on the flash (8 or 9 data bits, a zero chunk, 2 metadata bits). Case p4 is left erased.

NAND access: Erases block 3000 of CE0, programs pages in it by four paths, leaves one page erased, and reads the pages back raw.

13.3.87 writeseq

Device run: 29.

Procedure and question: DESTRUCTIVE (scratch blocks): the write sequences that the original firmware uses, checked by raw read-back. (1) Four pages in one run with the overlapped transfer 0x2e4 (one sector is sent while the next is moved by DMA into the other data buffer), FMC_DMA_CTRL = 7 before the last page, the last sector sent with 0xe4, a meta encode per page, and a ready poll before the next 80h. (2) One run that programs a page on CE0 and a page on CE1 back to back and then polls both. (3) A two-plane pair (80h .. 11h, poll, 81h .. 10h) on blocks B and B+1. (4) A page programmed a second time without an erase: does the status register (FMC_NSTAT) report FAIL, and what is stored?

NAND access: Erases blocks 3000 and 3001 of CE0 and block 3000 of CE1; programs pages in them (including page 30 of block 3001 and a second program of one page without an erase) and reads them back raw.

13.3.88 op09nand

Device run: 29.

Procedure and question: DESTRUCTIVE (scratch blocks): does `wait2` (opcode 09) track write, erase or read activity of the chip? `wait2` condition N is tried for N = 0..31 while a block erase runs, right after it finished, while a page program runs, right after it finished, and during a page read's tR (the time the chip needs to load the page). For N = 32..255 it is tried idle and during an erase (not tried before). Per case the experiment records whether the wait passed or timed out, how long it waited, and the chip's status right after it (bit 6 = ready).

NAND access: Erases block 3006 of CE0 and programs and reads pages in blocks 3004 and 3005 of CE0.

13.3.89 eraseseq

Device run: 29.

Procedure and question: DESTRUCTIVE (scratch blocks): the erase. The experiment measures the time of an erase (of a block with written pages and of an already erased block) and the status register (FMC_NSTAT) right after D0h (busy?) and at ready. It checks whether written pages read back erased. It also runs a two-plane erase (60h B, 60h B+1, D0h, with B and B+1 being adjacent blocks), the erase on CE1, and an erase sent with 4 address cycles instead of 3 (FMC_ANUM 3: accepted or refused?).

NAND access: Reads pages 10, 20 and 30 of CE0 block 3000, page 30 of CE0 block 3001 and page 20 of CE1 block 3000 before and after erases of CE0 block 3000 (twice), a two-plane erase of CE0 blocks 3000 and 3001, an erase of CE1 block 3000, and an erase of CE0 block 3002 sent with FMC_ANUM = 3.

13.3.90 eccchunk2

Device run: 30.

Procedure and question: The experiment decodes chunks built by the CPU (to the identified code) and loaded into parity slot 0 by DMA (the chunk is in the last 64 bytes of the 512-byte DMA, which is what the 64-byte buffer keeps). The chunks carry errors in the metadata or parity bytes, errors mixed with data errors, and the metadata byte order is checked.

13.3.91 eccmode2

Device run: 30.

Procedure and question: ECC_MODE map, encode only (no decode afterwards, so that the ECC area shows what the encode left): 0x180 plus each single bit 9..24, and the values 0, 0x80, 0x100, 0xe00000 and 0x1ffffff. For the modes that differ, the parity of zeros and of single-bit sectors is recorded, to fit their code in the way the code of 0x180 was fitted.

13.3.92 tmobits2

Device run: 30.

Procedure and question: Timeout: where does the growth with start-word bits 7:4 stop? The values n = 5, 6 and 7 of bits 7:4 were untested; n of 8 and above = 2**11.

13.3.93 ev5ready

Device run: 30.

Procedure and question: DESTRUCTIVE (erase of a scratch block). An erase program's second status latch timed out every time, while the first (taken while the erase ran) waited for the end of the erase. The experiment checks whether `wait` event 5 (status latch done) means the chip becoming ready rather than 'status byte latched'. The cases are: latches when idle (after a soft reset and not), twice in one program, after a chip reset (FFh), and after an erase. FMC_STATUS and FMC_NSTAT are read before each wait. The original firmware's order (read FMC_NSTAT first, wait only while the chip is busy) is also run.

NAND access: Erases block 3007 of CE0.

13.3.94 pageorder

Device run: 30.

Procedure and question: DESTRUCTIVE (scratch blocks). In writeseq, pages 11 and 13 (programmed after pages 10 and 12, block otherwise empty) read back erased although each program took its busy time. Is this an MLC page-order rule? The cases are: (a) pages 0..7 in order in one run (the original firmware's overlapped transfers); (b) page 5 alone in an erased block; (c) page 4 alone; (d) pages 4 and 5; (e) pages 1 and 3 (odd only); (f) an erase sent with 4 address cycles (FMC_ANUM 3) of a block holding a page: is it erased?

NAND access: Programs and reads pages in blocks 3008 to 3013 of CE0 (one scenario per block) and erases a block that holds a page.

13.3.95 op09w33

Device run: 30.

Procedure and question: `wait2` condition 33 passed when idle and during an erase (condition 5 is the other true one). The experiment tries condition 33 in the other chip states. It also tries the condition numbers that share bits with 5 or 33 (1, 4, 32, 37, 65, 69, 97, 133, 161, 165) when idle, to look for a pattern.

13.3.96 ev5arm

Device run: 31.

Procedure and question: `wait` event 5 is set by the first status latch of a program only. What re-arms it inside a program? Each case is: latch, X, latch, with X being nothing, the set-up word, the status-clear word, a CE deselect and reselect, FMCTRL1 = 0x100e0, a 16-byte read from the chip (00h and a 0x32 transfer), an ECC kick, or a DMA. A further case is a latch with FMC_DATA_SEL = 0x100 (what does it set?).

13.3.97 writeseq2

Device run: 31.

Procedure and question: DESTRUCTIVE (scratch blocks). Multi-page runs with the status poll done right (FMC_DATA_SEL = 1): eight pages in order with overlapped transfers and FMC_DMA_CTRL = 7 before the last DMA; pages 1 and 3; a page programmed again without an erase (FAIL bit?); and the status byte while a program runs.

NAND access: Programs pages in blocks 3014 and 3015 of CE0 and reads them back.

13.3.98 eccmode3

Device run: 31.

Procedure and question: ECC_MODE: (1) the length field, bits 20:16, with the values 3, 5, 12 and 31, and with bits 24:21 added (is the field wider?); (2) a decode with the length field set (does it check only those bytes?); (3) parity samples for the codes not yet identified (bit 15 and the modes without bit 8): sectors with a single bit set, at bit positions 0..15 and 4080..4095.

13.3.99 ev5two

Device run: 32.

Procedure and question: DESTRUCTIVE (scratch block on CE0 and CE1). `wait` event 5 is re-armed by FMCTRL1 operations. Does a chip becoming ready re-arm it too? A page is programmed on CE0 and on CE1; CE0 is polled (its latch waits for ready), and then CE1 is polled with no FMCTRL1 operation in between (the original WRITE does this). A further question: do address cycles (FMCTRL1 = 1) re-arm it?

NAND access: Programs a page in block 3016 on CE0 and on CE1 (after erasing it).

13.3.100 b15ecc

Device run: 32.

Procedure and question: ECC_MODE bit 15 (0x8180) selects the GF(2^10) code with 20 parity bytes. The experiment measures its correction strength (encode and decode in one program, with k flipped bits spread one per 10-bit symbol, and with bursts inside a few symbols). It also takes parity samples across the sector to identify the generator.

13.3.101 eccmode4

Device run: 33.

Procedure and question: ECC_MODE leftovers. For the modes without bit 8: is the result reproducible (same sector three times, after a soft reset or not), is it linear over GF(2) (data with bit 0, bit 1 and both set), and does an earlier operation change it (after a BCH decode or encode)? For 0x1ffffff: which bit beyond bit 8, bit 15 and length 31 changes the code? For bit 15: the upper 10 bytes of the ECC area after an encode only and after a decode.

13.3.102 finale

Device run: 34.

Procedure and question: The last open points. (1) ECC_MODE bits 5:1 with the BCH (0x180) and the Reed-Solomon (0x8100) code, with single-byte samples for the bits that change something. (2) The sequence of the modes without bit 8: 48 values in one program. (3) What FMC_28 counts (sti, ldi, DMA, chip transfers, ECC operations, plain instructions). (4) Do the words 0x1cc and 0x1d0 of the 0x100 block change the time of a NAND transfer? (5) Which NAND controller interrupt sources raise VIC0 line 20? (6) `wait2` conditions 0..63 while an ECC operation and a DMA are still running.

13.3.103 startdiff

Device run: 35.

Procedure and question: Start-word bit 6: the original operating systems start with 0xfff5 and the 4G/5G bootloader with 0xffb5, and no difference between them has been found. Bit 6 is part of the timeout step n (bits 7:4). The values n = 15 and 11 (and 13 and 9) give the same step, so each such pair differs in bit 6 alone. The same set of programs is run under 0xfff5 / 0xffb5 and under 0xffd5 / 0xff95, each after a soft reset, and every observable is compared. The observables are the end reason, the time, CS_IRQ, r0..r7, the interrupt lines, the whole window after the run, and the sequencer's reads and writes of the reserved parts of the window (0xc60..0xcfc, 0xd80..0xffc, 0x100..0x1fc, 0x3a8..0x3b4).

13.3.104 startbits

Device run: 35.

Procedure and question: The experiment checks what the start-word bits do beyond the known ones (bit 0 start, bit 1 pause, bit 2 timeout enable, bits 3 and 16 hold, bits 7:4 timeout step, bits 15:8 timeout length, bit 18 read-back status). Each bit 4..31 is flipped alone in 0xfff5 (bit 16 is left out because it holds the sequencer). For each variant a set of programs is run that looks at things not measured before: instruction fetch from SDRAM and from internal SRAM, system-memory reads (ldi) and writes (sti), window reads (pld), sig and the interrupt lines, a DMA plus an ECC encode, and a fault with vectoring. Every variant is compared with the 0xfff5 baseline, which is run three times to measure the run-to-run spread. Only differences beyond that spread are reported. Timeout lengths change with bits 4..15 by design and are shown for the fault program only.

13.3.105 rsdecode

Device run: 35.

Procedure and question: The experiment covers Reed-Solomon with 12 and 16 parity symbols (ECC_MODE bit 15 together with bit 2 or bit 3). It checks the decode strength (encode and decode in one program, with k bytes changed) and where the 20 parity bytes of the 16-symbol code go. After the encode, the whole parity buffer (all four slots) is copied out; a slot holds 16 bytes.

13.3.106 vic20

Device run: 35.

Procedure and question: VIC0 line 20 was seen for different FMC interrupt sources in two runs. Each source is tried with its enable bit, three times, each time after a soft reset. Is the line stable per source, and which CS_IRQ bits come with it?

13.3.107 fmc28b

Device run: 35.

Procedure and question: FMC_28 stays 0 through every operation after the set-up word, but rose by about 124 per probe run without it. A series of runs reads FMC_28 at its start and end, with and without the set-up word, and with nothing, a long loop, a chip read, a DMA or an ECC encode in between. The CPU's window dumps give the value between runs.

13.3.108 c58b1

Device run: 35.

Procedure and question: C58 bit 1: an END ran the fault vector once after the CPU wrote C58 = 2 and then C4C, and never when the program wrote C4C. The CPU writes in different orders and repeats them (no clean-up writes before them). Which write arms the one-shot, and is it used up?

13.3.109 evsweep2

Device run: 35.

Procedure and question: `wait` events 0..15 and `wait2` conditions 0..255 are tried in states not tried before: during an erase, a page program and a page read (chip busy); with an ECC encode in flight; with a DMA in flight; with every 0x044 / 0x844 interrupt source enabled and a command flag pending; after a second-set command; and idle. The timeout is short (start word 0x3f15: 64 x 8 cycles) and C58 is 0, so a timeout continues; a pass means no fault. Each state is swept in forward and in reverse order, because the chip state changes while the sweep runs.

13.3.110 c50fault

Device run: 35.

Procedure and question: C50 bit 0 (the original fault handlers test it and write 1 to it) and bit 8. A fault (the timeout of a `wait` on event 10, and an illegal opcode) is raised while a command flag is pending, with C58 = 0 and with C58 = 4. C50 is read before the fault, right after it (C58 = 0) or in the handler (C58 = 4), and after the handler's write of 1. For bit 8, C50 is read after a soft reset, after an END-only run, and after each kind of operation.

13.3.111 ctrlwords

Device run: 35.

Procedure and question: C10, C48 and C54 hold values but have no known effect, and the original fault handlers read C3C. The same operations (a timeout fault with C58 = 4, an illegal opcode, `wait2` condition 33, a DMA plus ECC encode, and two `tmwait` instructions (opcode `15`)) are run with the three words C10, C48 and C54 written all ones by the program and with them written zero. C3C is also read in handlers entered from three different faulting offsets, and in a second fault inside the handler.

13.3.112 c08self

Device run: 35.

Procedure and question: CS_STATUS as the program itself reads it (ld 0xc08 and pld), in places where the CPU cannot sample it: right after a DMA, an ECC kick and a chip command are started, inside a fault handler, after sig, after a `wait2` condition 33, and after a CPU pause and continue (C08 read after the pause point).

13.3.113 latchrep

Device run: 35.

Procedure and question: The experiment covers the intermittent results: `wait` event 5 straight after the 70h latch, the status latch's `wait` event 5 (once timed out in two identical runs), FMC_STATUS bit 24 instead of bit 23, and `wait` event 3 on CE0. Each is run 40 times, with and without a fresh arm (FMCTRL1 = 0x20 before the latch), and with and without 0x044 / 0x844 written. The tallies are of passes and timeouts and of the FMC_STATUS bits after the latch.

13.3.114 irq044

Device run: 35.

Procedure and question: Word 0x044 is set bit by bit (bits 0..31), with three runs per bit, each after a soft reset, using one program that does a command, an address, a 16-byte data read, a status latch and a DMA. Which bits raise CS_IRQ bit 4 or bit 5, VIC0 line 20 and VIC1 line 22, and is the result stable?

13.3.115 fmcbits

Device run: 35.

Procedure and question: DESTRUCTIVE (block 3050 of CE0). A page written by the engine is read back with FMC_CTRL0 bit 9 set, bit 11 clear and bit 24 (DMA enable) set for the whole read; with FMC_DMA_CTRL = 0, 7 or 0xff before the last DMA; and with the drain select 0x102 or 0x1e0 instead of 0x100. READID is also run with bit 9 set and bit 11 clear. The data, the decode results and the time are recorded.

13.3.116 fmcstat

Device run: 35.

Procedure and question: FMC_STATUS has power-on bits 12, 13, 18 and 19. Each is acknowledged alone (write 1), and the experiment then finds which operation sets it again (a command, an address, a data read, a status latch, a DMA, an ECC encode, a meta encode, a second-set command). The value right after a soft reset is also recorded.

13.3.117 eccregs

Device run: 35.

Procedure and question: ECC engine words without a known effect. Words 0x940..0x950 are written with 0, all ones or 0x55555555 before an encode and a decode with 8 bit errors (parity and decode result are compared). Words 0x990..0x99c are read after encodes of zeros, ones and a ramp in the BCH, Reed-Solomon and no-bit-8 modes.

13.3.118 seqrestart

Device run: 35.

Procedure and question: The sequence of the modes without bit 8 restarted once and continued once between runs. Each run does three encodes of the same sector in mode 0, with one action between the runs: nothing, a soft reset, an ECC_MODE write, an write of 0 to ECC_KICK, a BCH encode, a BCH decode, an ECC_DONE write, a write of 0x940..0x950, an FMC_CTRL0 write, or the sequencer reset (0x30000). The area words 0x900..0x913 of every encode are recorded.

13.3.119 done5

Device run: 35.

Procedure and question: ECC_DONE after an encode and after a decode, for ECC_MODE bits 9..14 and 16..24 set one at a time, on the BCH mode 0x180, the Reed-Solomon mode 0x8100 and the no-bit-8 mode 0x0. The rule for ECC_DONE bit 5 has so far been fitted to only a few modes.

13.3.120 progtime

Device run: 35.

Procedure and question: DESTRUCTIVE (blocks 3060 and 3061 of CE0, block 3060 of CE1). Every page of a block is programmed in order with four sectors of a pattern (engine encode), one run per page. The run time (ready poll included) is recorded per page. Are pages 4..11 slower, and what is the pattern over pages 0..127?

13.3.121 regsweep

Device run: 35.

Procedure and question: Register numbers 8..255 in every operand role. Only samples were tested before: 9, 11..16, 24 and 255 as a movi destination, 9 / 15 / 255 as an argument-field source, 9 / 14 as a register-field source, and 9 as an ldi address; r10 was never tried as a source. Before each role, r0..r7 are seeded with distinct values and the program first writes distinct values to the writable control words C10, C48, C54 and C3C. This makes 'reads r0', 'reads 0' and 'reads a control word (0xc18 + 4 N)' distinguishable. Program A tests destinations (number 8 is left out because a write to it is a jump): movi, mov and add to N, with r0..r7 stored to parameter RAM after each and the whole window compared. Program B tests sources: mov, add and or with N in the argument field, st, bnz and bz with N in the register field, and jr N. Program C tests addresses and is run for all numbers after A and B: ldi with N as the address register (r0..r7 and C48 / C3C hold scratch addresses with markers), and pld with N as the address register (r0..r7 hold parameter-RAM offsets with markers). Numbers are grouped by identical behaviour.

13.3.122 seqfirst

Device run: 36.

Procedure and question: The fixed sequence of the ECC modes without a code started at value 1 in some device runs and continued in others. One encode in mode 0 is the first ECC operation of the run, after a power cycle of the unit: which value of the sequence comes out?

13.3.123 w2state

Device run: 36.

Procedure and question: `wait2` condition 33 was true in every state of device runs 29 to 34 and in none of device run 35; condition 20 was true once, with every interrupt enable on and a command flag pending. All `wait` events 0..15 and `wait2` conditions 0..255 are tried with a short timeout (start word `0x3f15`), first in the first program of the run, then after one change of state at a time: a soft reset; `0x044` = `0xa5a5` and `0x844` = `0x25` left set; each `0x044` bit 0..15 alone with a command flag pending; `0x844` = `0x2f` with a command flag pending; each `FMC_STATUS` / `ECC_DONE` flag left pending; a READ ID through the second register set; the channel-1 pins switched; a page read of CE0; `C10` = 0 and `0xff`; no chip, CE0 or CE1 selected; the start word `0x3ff5`.

13.3.124 c50b8

Device run: 36.

Procedure and question: `C50` bit 8 was seen once after a soft reset (`0xffff1900`). The CPU reads `C50` before and after each of these runs: the first two runs, a run after a soft reset, a run after a sequencer reset alone (`FMSS_CTRL` = `0x30000`), a run that faults, a run with a second-set command, a run with an ECC decode, and ten `end`-only runs.

13.3.125 startregion

Device run: 36.

Procedure and question: On the 4G / 5G the bootloader starts its programs with `0xffb5` and reads only its own blocks, while the operating system starts with `0xfff5` and works the user area; every earlier comparison of the two start words ran in the user area. Page 0 of blocks in the user area (blocks 0..3871 of this chip) and of blocks above it, on CE0 and CE1, is read under `0xfff5`, `0xffb5`, `0xffd5` and `0xff95` (the second pair also differs in bit 6 alone); per read the end of the run, its time, the four decode results, the `FMC_STATUS` words and a hash of the data and chunks are compared. Then (destructive) a user block (3002) and the first block above 3871 whose page 0 read erased are erased and programmed under each of the two start words in turn, and each is read back under both.

13.3.126 bufport

Device run: 36.

Procedure and question: The original READID reads the 8 ID bytes of data buffer 0 through `FMC_FIFO` (`0x080`). Here a buffer is filled by DMA with a known pattern (`i * 29 + 7`) and `0x080` is read 130 times (one word more than 512 bytes), with `FMC_28` read every 32 reads: data buffer 0 (drain select `0x100`) and data buffer 1 (`0x200`) after `FMCTRL1` = `0x340`, data buffer 0 without `0x340`, parity buffer 0 (`0x1000`), each once and twice in a row. Then `0x080` is written 128 times with `FMC_DATA_SEL` = 1 (without and after `FMCTRL1` = `0x340`) and data buffer 0 is copied out by DMA: can a program write a data buffer through the port?

13.3.127 c58b2

Device run: 36.

Procedure and question: `C58` bit 1: an `end` ran the fault vector once in device run 8, and never in device run 35, where the job's park wrote 0 to `C58`, `C4C` and `C5C` after every run. Here the job's clean-up is switched off before and after the run, so `C58` keeps its value for three runs in a row. Cases: the CPU writes `C58` = 2 and then `C4C`; `C4C` and then `C58` = 2; `C58` = 2 with the program writing `C4C` (the control case of device run 8, also with the job's normal steps); the program writing `C58` = 2 and `C4C`; the CPU writing `C58` = 3 or 6 and `C4C`.

13.3.128 stprev

Device run: 36.

Procedure and question: `st` with a register number above 7 stored 0 after `movi r6, #0` (device run 15) and r7's value after `movi r7` (device run 35). The instruction just before `st [0xd70], rN` (N = 9, 14, 200) is varied, with r0..r7 seeded with distinct values first: `movi` to r0, r3 or r7; `add`; `mov`; `ld` of a window word; `pld`; `ldi`; `wr` to the window; `st` of r5; a `wait` that passes; `nop`; and nothing after the seeding.

13.3.129 tmwait2

Device run: 36.

Procedure and question: Two `tmwait` in one program made the program run 4841, 18 and 3250 us in three runs. `C14` is read at the start and after each of four `tmwait`, and the program's run time is recorded: the same program three times in a row, then after a run that timed out, then after a soft reset.

13.3.130 irqop

Device runs: 36, 37.

Procedure and question: Which `FMC_STATUS` flag each `0x044` bit routes into `CS_IRQ` bit 4, and what `wait` event 15 waits for. For `0x044` bits 0, 1, 2, 11 and 12 (and no bit), one operation per run whose flag is left pending (a command, an address, a 16-byte data read, a status latch, a DMA): `CS_IRQ` bit 4, VIC0 line 20, and whether `wait` event 15 passes (`C58` = 0; the fault flag is read after it). In device run 37 each program also acknowledges every flag and writes 0 to `0x044` and `0x844` before its `end`, and the experiment runs last.

13.3.131 done5b

Device run: 37.

Procedure and question: `ECC_DONE` bit 5 with `ECC_MODE` bit 21, 22 or 23 alone on `0x180` read set in device run 30 (encode only) and clear in device run 35 (encode, then decode, in one program). For each of bits 21..24: an encode only; an encode and then a decode; a decode first and then the encode; each case after a soft reset.

13.3.132 rsresult

Device run: 37.

Procedure and question: A Reed-Solomon encode (`0x8180`) left `ECC_RESULT` = `0x20000` where BCH encodes kept the earlier value. `ECC_RESULT` is read before and after an encode with 8, 12 and 16 Reed-Solomon symbols and with BCH, that follows: nothing (after a soft reset), a BCH decode with 0 and with 3 corrected bits, a Reed-Solomon decode with 0 and with 2 corrected bytes.

13.3.133 rschip

Device run: 37.

Procedure and question: Destructive (block 3070 of CE0). Reed-Solomon parity on the chip: pages written by the engine with RS-8 (`0x8100`), RS-12 (`0x8104`) and BCH, read back as stored and decoded in the same mode. Stored byte errors: a page is programmed a second time without an erase, with `0x00` at k data bytes of each sector and `0xff` elsewhere (programming only clears bits), and decoded: RS-8 with 4 and 5 bad bytes, RS-12 with 6 and 7. In the buffers: an RS-16 encode with the kick's slot field set to 3, both parity buffers copied out, and a decode with the slot field 3 of data with 2 bad bytes.

13.3.134 miscorr

Device run: 37.

Procedure and question: Can a decode report success with wrong data? BCH decodes with 9..64 random flipped bits and RS-8 decodes with 5..40 random bad bytes, 240 sectors each (12 per program, each encoded and decoded in the same program): how many are reported uncorrectable, how many corrected with the original data back, how many reported corrected with other data, and how many reported without error with the data unchanged.

13.3.135 fullpage

Device run: 37.

Procedure and question: Read-only. Do both 2 KiB units of a 4 KiB page written by the original firmware carry the same metadata? Pages 0, 1 and 64 of blocks 1, 2, 10, 100, 500, 1000 and 2000 of CE0 and CE1 are read whole (8 sectors, as stored): per page whether it is written, and the 3 metadata bytes of each of the 8 chunks.

13.3.136 dmactl

Device run: 37.

Procedure and question: Destructive (block 3051 of CE0). `FMC_DMA_CTRL` = 1 << b (b = 0..7) written before every data DMA of a page read, with the drain selects `0x100` and `0x1e0`; and the combined selects of the original READ (filling one data buffer while selecting the other for draining: `0x201` / `0x102`), alternating the

data buffers per sector.

13.3.137 fmcstat2

Device run: 37.

Procedure and question: `FMC_STATUS` bits 12, 13, 18 and 19 are set after a soft reset, are not cleared by writing 1, and a 16-byte data read clears bits 13 and 19. After that data read, which operation sets bits 13 and 19 again: a DMA into a data buffer, a DMA out of one, a DMA into a parity buffer, a 512-byte chip data read, an ECC encode, a status latch, a command, a second-set data read, the set-up word?

13.3.138 ch1buf

Device run: 37.

Procedure and question: Does the second register set (channel 1) have parity buffers and two data buffers? Through the `0x400` set a pattern is DMA-ed into parity buffer 0 or 1 (`0x414` = `0x10` / `0x20`) and back (`0x1000` / `0x2000`), and into data buffer 0 or 1 and back; before that the first set's parity buffer 0 is filled with another pattern, and it is copied out afterwards.

13.3.139 evsweep3

Device run: 37.

Procedure and question: All `wait` events 0..15 and `wait2` conditions 0..255 (short timeout) during operations not swept before: a second-set DMA, a meta encode, a meta decode, an ECC decode, a chip data write (`80h`, address, 512 bytes; cancelled with `FFh` afterwards) and a parity-buffer DMA, each still running.

13.3.140 blk100b

Device run: 37.

Procedure and question: The `0x100` block and the words `0x3a8..0x3b4`. Each writable word of the `0x100` block is set by the sequencer (to 0; `0x1b8` to 1; `0x1cc` and `0x1d0` also to `0xffffff`), and the same program then runs 8 DMAs, 8 ECC encodes and decodes, a 512-byte chip read, a `wait` that times out, and `wait2` conditions 33 and 5; the word is restored at the end. `0x3a8..0x3b4` are written by the CPU before the start and by the program (`pst`), and read by both.

13.3.141 straps

Device run: 37.

Procedure and question: Read-only, CPU only. The boot ROM chooses its boot device from GPIO port 3 pins 6 and 7, and checks port 1 pins 0 and 1 together with bit 0 of the USB PHY word `0x3c400028` for a USB boot. The CPU reads the configuration and data words of ports 1 and 3 and that PHY word.

13.3.142 w2vic

Device run: 38.

Procedure and question: Are the `wait2` conditions the CPU's interrupt lines? `wait2` condition 20 is true exactly when VIC0 line 20, the NAND controller's interrupt line, is raised. For N = 0..63, just before each `wait2 N` (short timeout, start word `0x3f15`), the program reads the raw interrupt status words of VIC0 (`0x38e00008`) and VIC1 (`0x38e01008`) with `ldi`. For each N the pass or timeout is compared with raw line N (VIC0 bit N for N < 32, VIC1 bit N - 32 for N = 32..63). States: idle; a command flag pending with `0x044` bit 0; a DMA flag pending with `0x044` bit 11; an `ECC_DONE` flag pending with `0x844` bit 0. Then N = 64..255 once, idle. Each program acknowledges every flag and writes 0 to `0x044` and `0x844` before its `end`.

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 14, chapter 15.

Chapter 14 — Evidence: outcomes

Summary: The outcomes on which this documentation rests, without interpretation: the printed result of every experiment run on the device, and the observations made in the original firmware by reverse engineering.

14.1 Scope

This chapter holds outcomes only, in two groups: the printed outcome of every experiment run on the device (chapter 14, section 14.2), and the observations made in the original firmware by reverse engineering (chapter 14, section 14.3). It contains no interpretation; readings are in chapter 15. What was done in each experiment, and why, is in chapter 13. Device runs are numbered 1 to 38 in the order they were made. Entry IDs are `D-<experiment>-<device run>` for device outcomes and `RE-<n>` for reverse-engineering observations. The printed text of each outcome is reproduced word for word, except that names of the test setup were left out.

14.2 Device outcomes

171 entries: 142 experiments in 38 device runs. Entries are in the order the runs were made.

14.2.1 regmap (run 1)

ID: `D-regmap-1`. Device run 1. Experiment: regmap.

```
END: r0..r7 at 0xc18..0xc34 match; SEQ_RESULT 11116666 == r6
```

14.2.2 clock (run 1)

ID: `D-clock-1`. Device run 1. Experiment: clock.

```
loop of 2 instructions: 873 us for 2000, 8657 us for 20000 iterations -> 216.2 ns per instruction (~5 MHz if 1 per clo
  ck)
```

14.2.3 alu (run 1)

ID: `D-alu-1`. Device run 1. Experiment: alu.

```
ALU:
  shr 0x80000000 by 1                -> 40000000  logical
  shl 1 by 32 (imm)                  -> 00000000  shift >= 32 gives 0
  shl 1 by 33 (imm)                  -> 00000000  gives 0
  shr 0x80000000 by 32               -> 00000000  gives 0
  add wraps                          -> 00000001  32-bit wrap
  sub below 0                        -> ffffffff  32-bit wrap
  add imm 0 is the 2-operand form    -> 0000000c  r2 = r2 + r1 (2-operand)
  shl by a register >= 32            -> 00000000  gives 0
  ld mask                            -> 00000000  UNEXPECTED
```

14.2.4 memory (run 1)

ID: `D-memory-1`. Device run 1. Experiment: memory.

```
memory: END; ldi [base] = 03020100 (expect 03020100 little endian); ldi [base+2] = 03020100 (03020100 if the address i
  s word-aligned down, 05040302 if unaligned access works); sti [base+0x40] -> bytes d4c3b2a1 (expect d4c3b2a1); sti [
  base+0x81] -> bytes 0x80..0x87 d4c3b2a184858687 (aligned down: d4c3b2a1 at 0x80)
```

14.2.5 indirect (run 1)

ID: `D-indirect-1`. Device run 1. Experiment: indirect.

```
indirect access:
  pld 0xc18 (r0 through the window)    -> r2 = 5a5a0000
  pld 0xd94 (parameter RAM)            -> r2 = 00000000
  pld 0x1d94 (above the window)        -> r2 = 00000000
  pst 0xd98 then ld                    -> r2 = 00000000
```

14.2.6 control (run 1)

ID: `D-control-1`. Device run 1. Experiment: control.

```
control window after a run: c00:0000fff4 c04:0801af40 c08:00000000 c0c:00000001 c10:00000001 c14:0000007f c18:00000000
   c1c:00000000 c20:00000000 c24:0badcafe c28:00000000 c2c:00000000 c30:00000000 c34:00000000 c38:00000000 c3c:0000000
  0 c40:00000000 c44:00000000 c48:00000000 c4c:00000030 c50:ffff1800 c54:00000000 c58:00000000 c5c:00000000
  CPU wrote r3 (0xc24) = 0badcafe before start; the program stored r3 = 00000000 (registers reset at start)
```

14.2.7 wait (run 1)

ID: `D-wait-1`. Device run 1. Experiment: wait.

```
WAIT:
  wait 0  -> FAULT   fault vector after   4881 us, CS_IRQ 00000005, C50 ffff1800
  wait 1  -> FAULT   fault vector after   4881 us, CS_IRQ 00000005, C50 ffff1800
  wait 2  -> FAULT   fault vector after   4881 us, CS_IRQ 00000005, C50 ffff1800
  wait 3  -> FAULT   fault vector after   4869 us, CS_IRQ 00000005, C50 ffff1800
  wait 4  -> FAULT   fault vector after   4870 us, CS_IRQ 00000005, C50 ffff1800
  wait 5  -> FAULT   fault vector after   4870 us, CS_IRQ 00000005, C50 ffff1800
  wait 6  -> FAULT   fault vector after   4881 us, CS_IRQ 00000005, C50 ffff1800
  wait 7  -> FAULT   fault vector after   4883 us, CS_IRQ 00000005, C50 ffff1800
  wait 8  -> FAULT   fault vector after   4880 us, CS_IRQ 00000005, C50 ffff1800
  wait 9  -> FAULT   fault vector after   4870 us, CS_IRQ 00000005, C50 ffff1800
  wait 10 -> FAULT   fault vector after   4879 us, CS_IRQ 00000005, C50 ffff1800
  wait 11 -> END     passed       after     15 us, CS_IRQ 00000001, C50 ffff1800
  wait 12 -> FAULT   fault vector after   4863 us, CS_IRQ 00000005, C50 ffff1800
  wait 13 -> FAULT   fault vector after   4880 us, CS_IRQ 00000005, C50 ffff1800
  wait 14 -> FAULT   fault vector after   4881 us, CS_IRQ 00000005, C50 ffff1800
  wait 15 -> FAULT   fault vector after   4872 us, CS_IRQ 00000005, C50 ffff1800
  wait 1 with FMSS_C58 = 0  -> FAULT   after    4882 us, r6 600d600d
  wait 1 with FMSS_C58 = 1  -> FAULT   after    4881 us, r6 600d600d
  wait 1 with FMSS_C58 = 2  -> END     after      15 us, r6 fa17fa17
  wait 1 with FMSS_C58 = 8  -> END     after      15 us, r6 fa17fa17
  wait 1 with FMSS_C58 = 16 -> END     after      15 us, r6 fa17fa17
```

14.2.8 unknown (run 1)

ID: `D-unknown-1`. Device run 1. Experiment: unknown.

```
unknown opcodes (.insn op, r3, arg 1, imm; r1 = 0xd84, r3 = 0x30000003):
  op 08 imm 0      -> END     continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000000c0->0000013c
  op 08 imm d88    -> END     continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000001b8->00000034 0
  x050:00001800->00002000
  op 09 imm 0      -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000000b0->0000012c
  op 09 imm d88    -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000001a8->00000024 0
  x050:00002000->00002800
  op 0f imm 0      -> END     continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000000a0->0000011c
  op 0f imm d88    -> END     continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000198->00000014 0
  x050:00002800->00003000
  op 10 imm 0      -> END     continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000090->0000010c
  op 10 imm d88    -> END     continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000188->00000004 0
  x050:00003000->00003800
  op 12 imm 0      -> TIMEOUT r6=60000006  r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000080->000000fc
  op 12 imm d88    -> TIMEOUT r6=60000006  r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000178->000001f4
  op 15 imm 0      -> END     continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000070->000000ec
  op 15 imm d88    -> END     continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000168->000001e4
  op 16 imm 0      -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000060->000000dc
  op 16 imm d88    -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000158->000001d4
  op 1a imm 0      -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000050->000000cc
  op 1a imm d88    -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000148->000001c4
  op 1b imm 0      -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000040->000000bc
  op 1b imm d88    -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000138->000001b4
  op 1c imm 0      -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000030->000000ac
  op 1c imm d88    -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000128->000001a4
  op 1d imm 0      -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000020->0000009c
  op 1d imm d88    -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000118->00000194
  op 1e imm 0      -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000010->0000008c
  op 1e imm d88    -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000108->00000184
  op 1f imm 0      -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:00000000->0000007c
  op 1f imm d88    -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000000f8->00000174
  op 20 imm 0      -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000001f0->0000006c 0
  x050:00003800->00000000
  op 20 imm d88    -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000000e8->00000164
  op 40 imm 0      -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000001e0->0000005c 0
  x050:00000000->00000800
  op 40 imm d88    -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000000d8->00000154
  op 80 imm 0      -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000001d0->0000004c 0
  x050:00000800->00001000
  op 80 imm d88    -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000000c8->00000144
  op ff imm 0      -> END     continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000001c0->0000003c 0
  x050:00001000->00001800
  op ff imm d88    -> END     continued    r0=00000000 r1=00000000 r2=00000000 r3=00000000; 0x028:000000b8->00000134
```

14.2.9 pram (run 2)

ID: `D-pram-2`. Device run 2. Experiment: pram.

```
CPU write then CPU read keeps: 0xd00..0xd7c
  sequencer ld of 0xd04, 0xd20, 0xd40, 0xd5c, 0xd60, 0xd7c, 0xd80, 0xdfc -> 00000000 00000000 00000000 a5000d5c a5000d
  60 a5000d7c 00000000 00000000
  sequencer wr then CPU read keeps (0xd80..0xdfc): none
```

14.2.10 alu (run 2)

ID: `D-alu-2`. Device run 2. Experiment: alu.

```
ALU:
  shr 0x80000000 by 1               -> 00000000  UNEXPECTED
```

```
  shl 1 by 32 (imm)                 -> 00000000  shift >= 32 gives 0
  shl 1 by 33 (imm)                 -> 00000000  gives 0
  shr 0x80000000 by 32              -> 00000000  gives 0
  add wraps                         -> 00000000  UNEXPECTED
  sub below 0                       -> 00000000  UNEXPECTED
  add imm 0 is the 2-operand form   -> 00000000  UNEXPECTED
  shl by a register >= 32           -> 00000000  gives 0
  ld mask                           -> 00000000  UNEXPECTED
```

14.2.11 indirect (run 2)

ID: `D-indirect-2`. Device run 2. Experiment: indirect.

```
indirect access:
  pld 0xc18 (r0 through the window)    -> r2 = 00000000
  pld 0xd64 (parameter RAM)            -> r2 = 00000000
  pld 0x1d64 (above the window)        -> r2 = 00000000
  pst 0xd68 then ld                    -> r2 = 00000000
```

14.2.12 control (run 2)

ID: `D-control-2`. Device run 2. Experiment: control.

```
control window after a run: c00:0000fff4 c04:0801af40 c08:00000000 c0c:00000001 c10:00000001 c14:000000ff c18:00000000
   c1c:00000000 c20:00000000 c24:0badcafe c28:00000000 c2c:00000000 c30:00000000 c34:00000000 c38:00000000 c3c:0000000
  0 c40:00000000 c44:00000000 c48:00000000 c4c:00000030 c50:ffff1800 c54:00000000 c58:00000001 c5c:00000001
  CPU wrote r3 (0xc24) = 0badcafe before start; the program stored r3 = 00000000 (not kept (0))
```

14.2.13 c58 (run 2)

ID: `D-c58-2`. Device run 2. Experiment: c58.

```
C58 sweep (wait 1, nothing pending):
  C58 = 0x0     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x1     -> END     vector    after      16 us, CS_IRQ 00000001
  C58 = 0x2     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x3     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x4     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x5     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x6     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x7     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x8     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x9     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0xa     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0xb     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0xc     -> END     vector    after      16 us, CS_IRQ 00000001
  C58 = 0xd     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0xe     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0xf     -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x10    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x11    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x12    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x13    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x14    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x15    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x16    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x17    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x18    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x19    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x1a    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x1b    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x1c    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x1d    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x1e    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x1f    -> END     vector    after      16 us, CS_IRQ 00000001
  C58 = 0x40    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x80    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0xff    -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0x100   -> END     vector    after      15 us, CS_IRQ 00000001
  C58 = 0xffff  -> END     vector    after      15 us, CS_IRQ 00000001
```

14.2.14 unknown (run 2)

ID: `D-unknown-2`. Device run 2. Experiment: unknown.

```
unknown opcodes (.insn op, r3, arg 1, imm; r1 = 0xd70, r3 = 0x30000003; C58 = 0, so a fault flags and continues; op 0b
   is the control: or r3, r1 [, #imm]):
  op 0b imm 0       -> END     continued    r0=00000000 r1=00000000 r2=00000000; 0x028:00000010->0000008c
  op 0b imm d74     -> END     continued    r0=00000000 r1=00000000 r2=00000000 r3=00000d74; 0x028:00000108->00000184
  op 08 imm 0       -> END     continued    r0=00000000 r1=00000000 r2=00000000; 0x028:00000000->0000007c
  op 08 imm d74     -> END     continued    r0=00000000 r1=00000000 r2=00000000; 0x028:000000f8->00000174
  op 09 imm 0       -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000; 0x028:000001f0->0000006c 0x050:0000300
  0->00003800
  op 09 imm d74     -> FAULT   continued    r0=00000000 r1=00000000 r2=00000000; 0x028:000000e8->00000164
  op 0f imm 0       -> END     r6=60000006  no register changed; 0x028:000001e0->0000005c 0x050:00003800->00000000
  op 0f imm d74     -> END     r6=60000006  no register changed; 0x028:000000d8->00000154
```

```
  op 10 imm 0      -> END     continued    no register changed; 0x028:000001d0->0000004c 0x050:00000000->00000800
  op 10 imm d74    -> END     continued    no register changed; 0x028:000000c8->00000144
  op 12 imm 0      -> TIMEOUT r6=60000006  no register changed; 0x028:000001c0->0000003c 0x050:00000800->00001000
  op 12 imm d74    -> TIMEOUT r6=60000006  no register changed; 0x028:000000b8->00000134
  op 15 imm 0      -> END     continued    no register changed; 0x028:000001b0->0000002c 0x050:00001000->00001800
  op 15 imm d74    -> END     continued    no register changed; 0x028:000000a8->00000124
  op 16 imm 0      -> FAULT   continued    no register changed; 0x028:000001a0->0000001c 0x050:00001800->00002000
  op 16 imm d74    -> FAULT   continued    no register changed; 0x028:00000098->00000114
  op 1a imm 0      -> FAULT   continued    no register changed; 0x028:00000190->0000000c 0x050:00002000->00002800
  op 1a imm d74    -> FAULT   continued    no register changed; 0x028:00000088->00000104
  op 1b imm 0      -> FAULT   continued    no register changed; 0x028:00000180->000001fc
  op 1b imm d74    -> FAULT   continued    no register changed; 0x028:00000078->000000f4
  op 1c imm 0      -> FAULT   continued    no register changed; 0x028:00000170->000001ec
  op 1c imm d74    -> FAULT   continued    no register changed; 0x028:00000068->000000e4
  op 1d imm 0      -> FAULT   continued    no register changed; 0x028:00000160->000001dc
  op 1d imm d74    -> FAULT   continued    no register changed; 0x028:00000058->000000d4
  op 1e imm 0      -> FAULT   continued    no register changed; 0x028:00000150->000001cc
  op 1e imm d74    -> FAULT   continued    no register changed; 0x028:00000048->000000c4
  op 1f imm 0      -> FAULT   continued    no register changed; 0x028:00000140->000001bc
  op 1f imm d74    -> FAULT   continued    no register changed; 0x028:00000038->000000b4
  op 20 imm 0      -> FAULT   continued    no register changed; 0x028:00000130->000001ac
  op 20 imm d74    -> FAULT   continued    no register changed; 0x028:00000028->000000a4
  op 40 imm 0      -> FAULT   continued    no register changed; 0x028:00000120->0000019c
  op 40 imm d74    -> FAULT   continued    no register changed; 0x028:00000018->00000094
  op 80 imm 0      -> FAULT   continued    no register changed; 0x028:00000110->0000018c
  op 80 imm d74    -> FAULT   continued    no register changed; 0x028:00000008->00000084
  op ff imm 0      -> END     continued    no register changed; 0x028:00000100->0000017c
  op ff imm d74    -> END     continued    no register changed; 0x028:000001f8->00000074 0x050:00002800->00003000
```

14.2.15 pram (run 3)

ID: `D-pram-3`. Device run 3. Experiment: pram.

```
CPU write then CPU read keeps: 0xd00..0xd7c
  sequencer ld of 0xd04, 0xd20, 0xd40, 0xd5c, 0xd60, 0xd7c, 0xd80, 0xdfc -> a5000d04 a5000d20 a5000d40 a5000d5c a5000d
  60 a5000d7c 00000000 00000000
  sequencer wr then CPU read keeps (0xd80..0xdfc): none
```

14.2.16 alu (run 3)

ID: `D-alu-3`. Device run 3. Experiment: alu.

```
ALU:
  shr 0x80000000 by 1               -> 40000000  logical
  shl 1 by 32 (imm)                 -> 00000000  shift >= 32 gives 0
  shl 1 by 33 (imm)                 -> 00000000  gives 0
  shr 0x80000000 by 32              -> 00000000  gives 0
  add wraps                         -> 00000001  32-bit wrap
  sub below 0                       -> ffffffff  32-bit wrap
  add imm 0 is the 2-operand form   -> 0000000c  r2 = r2 + r1 (2-operand)
  shl by a register >= 32           -> 00000000  gives 0
  ld mask                           -> 12005600  AND with the mask
```

14.2.17 indirect (run 3)

ID: `D-indirect-3`. Device run 3. Experiment: indirect.

```
indirect access:
  pld 0xc18 (r0 through the window)    -> r2 = 5a5a0000
  pld 0xd64 (parameter RAM)            -> r2 = 77777777
  pld 0x1d64 (above the window)        -> r2 = 77777777
  pst 0xd68 then ld                    -> r2 = 00c0ffee
```

14.2.18 control (run 3)

ID: `D-control-3`. Device run 3. Experiment: control.

```
control window after a run: c00:0000fff4 c04:0801af80 c08:00000000 c0c:00000001 c10:00000001 c14:000000ff c18:00000000
   c1c:00000000 c20:00000000 c24:0badcafe c28:00000000 c2c:00000000 c30:00000000 c34:00000000 c38:00000000 c3c:0000000
  0 c40:00000000 c44:00000000 c48:00000000 c4c:00000038 c50:ffff1800 c54:00000000 c58:00000004 c5c:00000000
  CPU wrote r3 (0xc24) = 0badcafe before start; the program stored r3 = 0badcafe (preload works)
```

14.2.19 c58 (run 3)

ID: `D-c58-3`. Device run 3. Experiment: c58.

```
C58 sweep (wait 1, nothing pending):
  C58 = 0x0     -> FAULT   continued after    4867 us, CS_IRQ 00000005; 0xc4c:00000000->00000030
  C58 = 0x1     -> FAULT   continued after    4881 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  001 0xc5c:00000000->00000001
  C58 = 0x2     -> FAULT   continued after    4868 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  002
  C58 = 0x3     -> FAULT   continued after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  003 0xc5c:00000000->00000001
  C58 = 0x4     -> FAULT   vector    after    4881 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  004 0xc5c:00000000->00000001
```

```
  C58 = 0x5     -> FAULT   vector    after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  005 0xc5c:00000000->00000001
  C58 = 0x6     -> FAULT   vector    after    4866 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  006 0xc5c:00000000->00000001
  C58 = 0x7     -> FAULT   vector    after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  007 0xc5c:00000000->00000001
  C58 = 0x8     -> FAULT   continued after    4869 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  008
  C58 = 0x9     -> FAULT   continued after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  009 0xc5c:00000000->00000001
  C58 = 0xa     -> FAULT   continued after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  00a
  C58 = 0xb     -> FAULT   continued after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  00b 0xc5c:00000000->00000001
  C58 = 0xc     -> FAULT   vector    after    4881 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  00c 0xc5c:00000000->00000001
  C58 = 0xd     -> FAULT   vector    after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  00d 0xc5c:00000000->00000001
  C58 = 0xe     -> FAULT   vector    after    4868 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  00e 0xc5c:00000000->00000001
  C58 = 0xf     -> FAULT   vector    after    4868 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  00f 0xc5c:00000000->00000001
  C58 = 0x10    -> FAULT   continued after    4869 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  010
  C58 = 0x11    -> FAULT   continued after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  011 0xc5c:00000000->00000001
  C58 = 0x12    -> FAULT   continued after    4868 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  012
  C58 = 0x13    -> FAULT   continued after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  013 0xc5c:00000000->00000001
  C58 = 0x14    -> FAULT   vector    after    4868 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  014 0xc5c:00000000->00000001
  C58 = 0x15    -> FAULT   vector    after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  015 0xc5c:00000000->00000001
  C58 = 0x16    -> FAULT   vector    after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  016 0xc5c:00000000->00000001
  C58 = 0x17    -> FAULT   vector    after    4868 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  017 0xc5c:00000000->00000001
  C58 = 0x18    -> FAULT   continued after    4868 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  018
  C58 = 0x19    -> FAULT   continued after    4868 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  019 0xc5c:00000000->00000001
  C58 = 0x1a    -> FAULT   continued after    4869 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  01a
  C58 = 0x1b    -> FAULT   continued after    4869 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  01b 0xc5c:00000000->00000001
  C58 = 0x1c    -> FAULT   vector    after    4869 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  01c 0xc5c:00000000->00000001
  C58 = 0x1d    -> FAULT   vector    after    4882 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  01d 0xc5c:00000000->00000001
  C58 = 0x1e    -> FAULT   vector    after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  01e 0xc5c:00000000->00000001
  C58 = 0x1f    -> FAULT   vector    after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  01f 0xc5c:00000000->00000001
  C58 = 0x40    -> FAULT   continued after    4881 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  040
  C58 = 0x80    -> FAULT   continued after    4882 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  080
  C58 = 0xff    -> FAULT   vector    after    4869 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  0ff 0xc5c:00000000->00000001
  C58 = 0x100   -> FAULT   continued after    4869 us, CS_IRQ 00000005; 0xc4c:00000000->00000030
  C58 = 0xffff  -> FAULT   vector    after    4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000
  0ff 0xc5c:00000000->00000001
```

14.2.20 unknown (run 3)

ID: `D-unknown-3`. Device run 3. Experiment: unknown.

```
unknown opcodes (.insn op, r3, arg 1, imm; r1 = 0xd70, r3 = 0x30000003; C58 = 0, so a fault flags and continues; op 0b
   is the control: or r3, r1 [, #imm]):
  op 0b imm 0      -> END     continued    r3=30000d73; 0x028:00000150->000001cc; control 0xc4c:00000000->00000070
  op 0b imm d74    -> END     continued    r3=00000d74; 0x028:00000048->000000c4; control 0xc4c:00000000->00000070
  op 08 imm 0      -> END     continued    no register changed; 0x028:00000140->000001bc; control 0xc4c:00000000->0000
  0070
  op 08 imm d74    -> END     continued    no register changed; 0x028:00000038->000000b4; control 0xc4c:00000000->0000
  0070
  op 09 imm 0      -> FAULT   continued    no register changed; 0x028:00000130->000001ac; control 0xc4c:00000000->0000
  0070
  op 09 imm d74    -> FAULT   continued    no register changed; 0x028:00000028->000000a4; control 0xc4c:00000000->0000
  0070
  op 0f imm 0      -> END     r6=60000006  no register changed; 0x028:00000120->0000019c; control 0xc4c:00000000->0000
  0070
  op 0f imm d74    -> END     r6=60000006  no register changed; 0x028:00000018->00000094; control 0xc4c:00000000->0000
  0070
  op 10 imm 0      -> END     continued    no register changed; 0x028:00000110->0000018c; control 0xc4c:00000000->0000
  0070
```

```
  op 10 imm d74    -> END     continued    no register changed; 0x028:00000008->00000084; control 0xc4c:00000000->0000
  0070
  op 12 imm 0      -> TIMEOUT r6=60000006  no register changed; 0x028:00000100->0000017c; control 0xc08:00000000->0000
  000a 0xc40:00000000->12030001 0xc4c:00000000->00000070
  op 12 imm d74    -> TIMEOUT r6=60000006  no register changed; 0x028:000001f8->00000074 0x050:00002000->00002800; con
  trol 0xc08:00000000->0000000a 0xc40:00000000->12030001 0xc44:00000000->00000d74 0xc4c:00000000->00000070
  op 15 imm 0      -> END     continued    no register changed; 0x028:000000f0->0000016c; control 0xc14:000000ff->0000
  0000 0xc4c:00000000->00000070
  op 15 imm d74    -> END     continued    no register changed; 0x028:000001e8->00000064 0x050:00002800->00003000; con
  trol 0xc14:00000000->000000ff 0xc4c:00000000->00000070
  op 16 imm 0      -> FAULT   continued    no register changed; 0x028:000000e0->0000015c; control 0xc4c:00000000->0000
  0070
  op 16 imm d74    -> FAULT   continued    no register changed; 0x028:000001d8->00000054 0x050:00003000->00003800; con
  trol 0xc4c:00000000->00000070
  op 1a imm 0      -> FAULT   continued    no register changed; 0x028:000000d0->0000014c; control 0xc4c:00000000->0000
  0070
  op 1a imm d74    -> FAULT   continued    no register changed; 0x028:000001c8->00000044 0x050:00003800->00000000; con
  trol 0xc4c:00000000->00000070
  op 1b imm 0      -> FAULT   continued    no register changed; 0x028:000000c0->0000013c; control 0xc4c:00000000->0000
  0070
  op 1b imm d74    -> FAULT   continued    no register changed; 0x028:000001b8->00000034 0x050:00000000->00000800; con
  trol 0xc4c:00000000->00000070
  op 1c imm 0      -> FAULT   continued    no register changed; 0x028:000000b0->0000012c; control 0xc4c:00000000->0000
  0070
  op 1c imm d74    -> FAULT   continued    no register changed; 0x028:000001a8->00000024 0x050:00000800->00001000; con
  trol 0xc4c:00000000->00000070
  op 1d imm 0      -> FAULT   continued    no register changed; 0x028:000000a0->0000011c; control 0xc4c:00000000->0000
  0070
  op 1d imm d74    -> FAULT   continued    no register changed; 0x028:00000198->00000014 0x050:00001000->00001800; con
  trol 0xc4c:00000000->00000070
  op 1e imm 0      -> FAULT   continued    no register changed; 0x028:00000090->0000010c; control 0xc4c:00000000->0000
  0070
  op 1e imm d74    -> FAULT   continued    no register changed; 0x028:00000188->00000004 0x050:00001800->00002000; con
  trol 0xc4c:00000000->00000070
  op 1f imm 0      -> FAULT   continued    no register changed; 0x028:00000080->000000fc; control 0xc4c:00000000->0000
  0070
  op 1f imm d74    -> FAULT   continued    no register changed; 0x028:00000178->000001f4; control 0xc4c:00000000->0000
  0070
  op 20 imm 0      -> FAULT   continued    no register changed; 0x028:00000070->000000ec; control 0xc4c:00000000->0000
  0070
  op 20 imm d74    -> FAULT   continued    no register changed; 0x028:00000168->000001e4; control 0xc4c:00000000->0000
  0070
  op 40 imm 0      -> FAULT   continued    no register changed; 0x028:00000060->000000dc; control 0xc4c:00000000->0000
  0070
  op 40 imm d74    -> FAULT   continued    no register changed; 0x028:00000158->000001d4; control 0xc4c:00000000->0000
  0070
  op 80 imm 0      -> FAULT   continued    no register changed; 0x028:00000050->000000cc; control 0xc4c:00000000->0000
  0070
  op 80 imm d74    -> FAULT   continued    no register changed; 0x028:00000148->000001c4; control 0xc4c:00000000->0000
  0070
  op ff imm 0      -> END     continued    no register changed; 0x028:00000040->000000bc; control 0xc4c:00000000->0000
  0070
  op ff imm d74    -> END     continued    no register changed; 0x028:00000138->000001b4; control 0xc4c:00000000->0000
  0070
```

14.2.21 resume (run 4)

ID: `D-resume-4`. Device run 4. Experiment: resume.

```
start address:
  C58 0, C4C mark          -> END     started at 0; after: C58 0 C5C 0 C4C 20 CS_IRQ 00000001
  C58 1, C4C mark          -> END     started at 0; after: C58 1 C5C 1 C4C 20 CS_IRQ 00000001
  C58 2, C4C mark          -> END     started r0=57a27000 r1=3a2c3a2c; after: C58 2 C5C 0 C4C 20 CS_IRQ 00000001
  C58 4, C4C mark          -> END     started at 0; after: C58 4 C5C 0 C4C 20 CS_IRQ 00000001
  C5C 1, C58 0, C4C mark   -> END     started at 0; after: C58 0 C5C 1 C4C 20 CS_IRQ 00000001
  C5C 1, C58 1, C4C mark   -> END     started at 0; after: C58 1 C5C 1 C4C 20 CS_IRQ 00000001
```

14.2.22 wait (run 4)

ID: `D-wait-4`. Device run 4. Experiment: wait.

```
WAIT:
  wait 0  -> FAULT   fault vector after   4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 1  -> FAULT   fault vector after   4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 2  -> FAULT   fault vector after   4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 3  -> FAULT   fault vector after   4881 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 4  -> FAULT   fault vector after   4881 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 5  -> FAULT   fault vector after   4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 6  -> FAULT   fault vector after   4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
```

```
  wait 7  -> FAULT   fault vector after   4867 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 8  -> FAULT   fault vector after   4870 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 9  -> FAULT   fault vector after   4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 10 -> FAULT   fault vector after   4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 11 -> END     passed       after     15 us, CS_IRQ 00000001; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  wait 12 -> FAULT   fault vector after   4870 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 13 -> FAULT   fault vector after   4868 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 14 -> FAULT   fault vector after   4881 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
  wait 15 -> FAULT   fault vector after   4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000030 0xc58:00000000->00000004
  0xc5c:00000000->00000001
```

14.2.23 op15 (run 4)

ID: `D-op15-4`. Device run 4. Experiment: op15.

```
op 0x15 (C14 preset 0x55 by the CPU; r5 = a5a5a5a5):
  imm 0          reg 0 arg 0 -> END     C14 00000000  0xc14:000000ff->00000000 0xc4c:00000000->00000030
  imm 0          reg 5 arg 0 -> END     C14 000000ff  0xc14:00000000->000000ff 0xc4c:00000000->00000030
  imm 0          reg 0 arg 5 -> END     C14 00000000  0xc14:000000ff->00000000 0xc4c:00000000->00000030
  imm 1          reg 0 arg 0 -> END     C14 000000ff  0xc14:00000000->000000ff 0xc4c:00000000->00000030
  imm 1          reg 5 arg 0 -> END     C14 00000000  0xc14:000000ff->00000000 0xc4c:00000000->00000030
  imm 1          reg 0 arg 5 -> END     C14 000000ff  0xc14:00000000->000000ff 0xc4c:00000000->00000030
  imm 7f         reg 0 arg 0 -> END     C14 00000000  0xc14:000000ff->00000000 0xc4c:00000000->00000030
  imm 7f         reg 5 arg 0 -> END     C14 000000ff  0xc14:00000000->000000ff 0xc4c:00000000->00000030
  imm 7f         reg 0 arg 5 -> END     C14 00000000  0xc14:000000ff->00000000 0xc4c:00000000->00000030
  imm 80         reg 0 arg 0 -> END     C14 000000ff  0xc14:00000000->000000ff 0xc4c:00000000->00000030
  imm 80         reg 5 arg 0 -> END     C14 00000000  0xc14:000000ff->00000000 0xc4c:00000000->00000030
  imm 80         reg 0 arg 5 -> END     C14 000000ff  0xc14:00000000->000000ff 0xc4c:00000000->00000030
  imm fe         reg 0 arg 0 -> END     C14 00000000  0xc14:000000ff->00000000 0xc4c:00000000->00000030
  imm fe         reg 5 arg 0 -> END     C14 000000ff  0xc14:00000000->000000ff 0xc4c:00000000->00000030
  imm fe         reg 0 arg 5 -> END     C14 00000000  0xc14:000000ff->00000000 0xc4c:00000000->00000030
  imm ff         reg 0 arg 0 -> END     C14 000000ff  0xc14:00000000->000000ff 0xc4c:00000000->00000030
  imm ff         reg 5 arg 0 -> END     C14 00000000  0xc14:000000ff->00000000 0xc4c:00000000->00000030
  imm ff         reg 0 arg 5 -> END     C14 000000ff  0xc14:00000000->000000ff 0xc4c:00000000->00000030
  imm 100        reg 0 arg 0 -> END     C14 00000000  0xc14:000000ff->00000000 0xc4c:00000000->00000030
  imm 100        reg 5 arg 0 -> END     C14 000000ff  0xc14:00000000->000000ff 0xc4c:00000000->00000030
  imm 100        reg 0 arg 5 -> END     C14 00000000  0xc14:000000ff->00000000 0xc4c:00000000->00000030
  imm 12345678   reg 0 arg 0 -> END     C14 000000ff  0xc14:00000000->000000ff 0xc4c:00000000->00000030
  imm 12345678   reg 5 arg 0 -> END     C14 00000000  0xc14:000000ff->00000000 0xc4c:00000000->00000030
  imm 12345678   reg 0 arg 5 -> END     C14 000000ff  0xc14:00000000->000000ff 0xc4c:00000000->00000030
```

14.2.24 op12 (run 5)

ID: `D-op12-5`. Device run 5. Experiment: op12.

```
op 0x12 (r2 = expected value, arg = register offset, imm = mask):
  [0xd60] & ffffffff == r2 (1234)    -> BLOCKED  TIMEOUT after  50030 us; 0xc08:00000000->0000000a 0xc40:00000000->120
  20d60 0xc44:00000000->ffffffff 0xc4c:00000000->00000040
  [0xd60] & ffffffff == r2 (9999)    -> BLOCKED  TIMEOUT after  50026 us; 0xc08:00000000->0000000a 0xc40:00000000->120
  20d60 0xc44:00000000->ffffffff 0xc4c:00000000->00000040
  [0xd60] & 0 == r2 (0)              -> BLOCKED  TIMEOUT after  50041 us; 0xc08:00000000->0000000a 0xc40:00000000->120
  20d60 0xc4c:00000000->00000040
  [0xd60] & ff00 == r2 (1200)        -> BLOCKED  TIMEOUT after  50028 us; 0xc08:00000000->0000000a 0xc40:00000000->120
  20d60 0xc44:00000000->0000ff00 0xc4c:00000000->00000040
  [0xd60] & ff00 == r2 (0034)        -> BLOCKED  TIMEOUT after  50023 us; 0xc08:00000000->0000000a 0xc40:00000000->120
  20d60 0xc44:00000000->0000ff00 0xc4c:00000000->00000040
  [0xd60] & ffffffff != r2 (0)       -> BLOCKED  TIMEOUT after  50028 us; 0xc08:00000000->0000000a 0xc40:00000000->120
  20d60 0xc44:00000000->ffffffff 0xc4c:00000000->00000040
  [0xd64] (0) & ffffffff, r2 0       -> BLOCKED  TIMEOUT after  50036 us; 0xc08:00000000->0000000a 0xc40:00000000->120
  20d64 0xc44:00000000->ffffffff 0xc4c:00000000->00000040
  [0xd64] (0) & ffffffff, r2 1       -> BLOCKED  TIMEOUT after  50026 us; 0xc08:00000000->0000000a 0xc40:00000000->120
  20d64 0xc44:00000000->ffffffff 0xc4c:00000000->00000040
```

14.2.25 c58ill (run 5)

ID: `D-c58ill-5`. Device run 5. Experiment: c58ill.

```
illegal opcode 0x09 under each C58:
  C58 = 0 -> FAULT   continued after  4870 us, CS_IRQ 00000005; 0xc4c:00000000->00000038
  C58 = 1 -> FAULT   continued after  4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000038 0xc58:00000000->00000001 0xc5
  c:00000000->00000001
  C58 = 2 -> FAULT   continued after  4865 us, CS_IRQ 00000005; 0xc4c:00000000->00000038 0xc58:00000000->00000002
  C58 = 4 -> FAULT   vector    after  4880 us, CS_IRQ 00000005; 0xc4c:00000000->00000038 0xc58:00000000->00000004 0xc5
  c:00000000->00000001
  C58 = 5 -> FAULT   vector    after  4869 us, CS_IRQ 00000005; 0xc4c:00000000->00000038 0xc58:00000000->00000005 0xc5
  c:00000000->00000001
```

14.2.26 pending (run 5)

ID: D-pending-5. Device run 5. Experiment: pending.

The experiment changed between runs.

```
start after a program that left state behind:
  first: C58 1, WAIT times out (fault, continue) -> TIMEOUT, C58 0 C5C 0 C4C 0 CS_IRQ 00000000
    next start (no cleanup)  -> neither marker           C58 0 C5C 0 C4C 0 CS_IRQ 00000000
    again (no cleanup)       -> neither marker           C58 0 C5C 0 C4C 0 CS_IRQ 00000000
    after the cleanup        -> started at 0             C58 0 C5C 0 C4C 0 CS_IRQ 00000001
  first: C58 1, no fault (END) -> TIMEOUT, C58 0 C5C 0 C4C 0 CS_IRQ 00000000
    next start (no cleanup)  -> neither marker           C58 0 C5C 0 C4C 0 CS_IRQ 00000000
    again (no cleanup)       -> neither marker           C58 0 C5C 0 C4C 0 CS_IRQ 00000000
    after the cleanup        -> started at 0             C58 0 C5C 0 C4C 0 CS_IRQ 00000001
  first: C58 4, WAIT times out (fault, vector) -> TIMEOUT, C58 0 C5C 0 C4C 0 CS_IRQ 00000000
    next start (no cleanup)  -> neither marker           C58 0 C5C 0 C4C 0 CS_IRQ 00000000
    again (no cleanup)       -> neither marker           C58 0 C5C 0 C4C 0 CS_IRQ 00000000
    after the cleanup        -> started at 0             C58 0 C5C 0 C4C 0 CS_IRQ 00000001
  first: C58 0, WAIT times out (fault, continue) -> TIMEOUT, C58 0 C5C 0 C4C 0 CS_IRQ 00000000
    next start (no cleanup)  -> neither marker           C58 0 C5C 0 C4C 0 CS_IRQ 00000000
    again (no cleanup)       -> neither marker           C58 0 C5C 0 C4C 0 CS_IRQ 00000000
    after the cleanup        -> started at 0             C58 0 C5C 0 C4C 0 CS_IRQ 00000001
```

14.2.27 c14 (run 5)

ID: D-c14-5. Device run 5. Experiment: c14.

```
FMSS_C14:
  wr C14 #0x12         -> C14 before 000000ff, read by the program 000000ff, after 000000ff
  wr C14 #0x00         -> C14 before 000000ff, read by the program 000000ff, after 000000ff
  wr C14 #0xff         -> C14 before 000000ff, read by the program 000000ff, after 000000ff
  wr C14 #0x0f; op15   -> C14 before 000000ff, read by the program 00000000, after 00000000
  op15                 -> C14 before 00000000, read by the program 000000ff, after 000000ff
  op15; op15           -> C14 before 000000ff, read by the program 000000ff, after 000000ff
  op15 x3              -> C14 before 000000ff, read by the program 00000000, after 00000000
```

14.2.28 regmap (run 6)

ID: D-regmap-6. Device run 6. Experiment: regmap.

```
END: r0..r7 at 0xc18..0xc34 match; SEQ_RESULT 11116666 == r6
```

14.2.29 startseq (run 6)

ID: D-startseq-6. Device run 6. Experiment: startseq.

```
start after a sequencer reset (marker program):
  reset only               -> TIMEOUT neither marker         FMSS_CTRL after 0004fff4, C08 0, CS_IRQ 00000000
  reset + clean            -> TIMEOUT neither marker         FMSS_CTRL after 0004fff4, C08 0, CS_IRQ 00000000
  reset + 0x400            -> TIMEOUT neither marker         FMSS_CTRL after 0004fff4, C08 0, CS_IRQ 00000000
  reset + preflight        -> END     started at 0           FMSS_CTRL after 0000fff4, C08 0, CS_IRQ 00000001
  reset + clean + 0x400    -> TIMEOUT neither marker         FMSS_CTRL after 0004fff4, C08 0, CS_IRQ 00000000
  reset + all (default)    -> END     started at 0           FMSS_CTRL after 0000fff4, C08 0, CS_IRQ 00000001
  no reset, nothing else   -> END     started at 0           FMSS_CTRL after 0000fff4, C08 0, CS_IRQ 00000001
```

14.2.30 pending (run 6)

ID: D-pending-6. Device run 6. Experiment: pending.

The experiment changed between runs.

```
start after a program that left state behind:
  first: C58 1, WAIT times out (fault, continue) -> FAULT, C58 1 C5C 1 C4C 20 CS_IRQ 00000005
    next start (old steps)  -> ran 0, then the vector   C58 1 C5C 1 C4C 20 CS_IRQ 00000001
    again (old steps)       -> ran 0, then the vector   C58 1 C5C 1 C4C 20 CS_IRQ 00000001
    current steps (clean)   -> started at 0             C58 0 C5C 0 C4C 0 CS_IRQ 00000001
  first: C58 1, no fault (END) -> END, C58 1 C5C 1 C4C 20 CS_IRQ 00000001
    next start (old steps)  -> ran 0, then the vector   C58 1 C5C 1 C4C 20 CS_IRQ 00000001
    again (old steps)       -> ran 0, then the vector   C58 1 C5C 1 C4C 20 CS_IRQ 00000001
    current steps (clean)   -> started at 0             C58 0 C5C 0 C4C 0 CS_IRQ 00000001
  first: C58 4, WAIT times out (fault, vector) -> FAULT, C58 4 C5C 1 C4C 20 CS_IRQ 00000005
    next start (old steps)  -> started at 0             C58 4 C5C 1 C4C 20 CS_IRQ 00000001
    again (old steps)       -> started at 0             C58 4 C5C 1 C4C 20 CS_IRQ 00000001
    current steps (clean)   -> started at 0             C58 0 C5C 0 C4C 0 CS_IRQ 00000001
  first: C58 0, WAIT times out (fault, continue) -> FAULT, C58 0 C5C 0 C4C 20 CS_IRQ 00000005
    next start (old steps)  -> started at 0             C58 0 C5C 0 C4C 20 CS_IRQ 00000001
    again (old steps)       -> started at 0             C58 0 C5C 0 C4C 20 CS_IRQ 00000001
    current steps (clean)   -> started at 0             C58 0 C5C 0 C4C 0 CS_IRQ 00000001
  first: C58 2, no fault (END) -> END, C58 2 C5C 0 C4C 20 CS_IRQ 00000001
    next start (old steps)  -> started at 0             C58 2 C5C 0 C4C 20 CS_IRQ 00000001
    again (old steps)       -> started at 0             C58 2 C5C 0 C4C 20 CS_IRQ 00000001
    current steps (clean)   -> started at 0             C58 0 C5C 0 C4C 0 CS_IRQ 00000001
```

14.2.31 op12ack (run 6)

ID: D-op12ack-6. Device run 6. Experiment: op12ack.

```
op 0x12 and a CPU write while it blocks:
  none (control)           -> blocked  TIMEOUT after  30015 us (late write at 0 us), CS_IRQ 00000008, C08 a
  CS_IRQ = 0x08 (ack b3)   -> blocked  TIMEOUT after  30016 us (late write at 2010 us), CS_IRQ 00000000, C08 a
  CS_IRQ = 0xff            -> blocked  TIMEOUT after  30013 us (late write at 2010 us), CS_IRQ 00000000, C08 a
  C08 = 0                  -> blocked  TIMEOUT after  30024 us (late write at 2010 us), CS_IRQ 00000008, C08 a
  C08 = 0xb                -> blocked  TIMEOUT after  30018 us (late write at 2010 us), CS_IRQ 00000008, C08 a
  C40 = 0                  -> blocked  TIMEOUT after  30014 us (late write at 2010 us), CS_IRQ 00000008, C08 a
  C14 = 0                  -> blocked  TIMEOUT after  30017 us (late write at 2006 us), CS_IRQ 00000008, C08 a
  C10 = 0                  -> blocked  TIMEOUT after  30024 us (late write at 2010 us), CS_IRQ 00000008, C08 a
```

14.2.32 op10 (run 6)

ID: `D-op10-6`. Device run 6. Experiment: op10.

```
op 0x10: END, r6 600d600d (carried on), CS_IRQ read by the program right after it 00000002, CS_IRQ at the end 00000003
```

14.2.33 regmap (run 7)

ID: `D-regmap-7`. Device run 7. Experiment: regmap.

```
END: r0..r7 at 0xc18..0xc34 match; SEQ_RESULT 11116666 == r6
```

14.2.34 startseq2 (run 7)

ID: `D-startseq2-7`. Device run 7. Experiment: startseq2.

```
reset, then one kind of access, then START:
  nothing in between     -> TIMEOUT no run   FMSS_CTRL after 0004fff4, CS_IRQ 00000000
  FMC_CTRL0 = 0          -> TIMEOUT no run   FMSS_CTRL after 0004fff4, CS_IRQ 00000000
  full snapshot          -> TIMEOUT no run   FMSS_CTRL after 0004fff4, CS_IRQ 00000000
  read window 0x000      -> TIMEOUT no run   FMSS_CTRL after 0004fff4, CS_IRQ 00000000
  read window 0x800      -> TIMEOUT no run   FMSS_CTRL after 0004fff4, CS_IRQ 00000000
  read window 0xc00      -> TIMEOUT no run   FMSS_CTRL after 0004fff4, CS_IRQ 00000000
  read window 0xd00      -> TIMEOUT no run   FMSS_CTRL after 0004fff4, CS_IRQ 00000000
  CPU writes r0          -> TIMEOUT no run   FMSS_CTRL after 0004fff4, CS_IRQ 00000000
  CPU writes 0xd60       -> TIMEOUT no run   FMSS_CTRL after 0004fff4, CS_IRQ 00000000
```

14.2.35 startseq3 (run 8)

ID: `D-startseq3-8`. Device run 8. Experiment: startseq3.

```
reset, START (no preflight), optionally a CPU write of FMSS_CTRL 2 ms later:
  END-only program         -> END     r0 00000000 r6 00000000  FMSS_CTRL after 0000fff4, C40 00000000 C44 00000000, CS
  _IRQ 00000001
  no-op (0x08) first       -> TIMEOUT r0 00000000 r6 00000000  FMSS_CTRL after 0004fff4, C40 08000000 C44 00000000, CS
  _IRQ 00000000
  wr CS_IRQ first          -> TIMEOUT r0 00000000 r6 00000000  FMSS_CTRL after 0004fff4, C40 01000c0c C44 000000ff, CS
  _IRQ 00000000
  marker (control)         -> TIMEOUT r0 00000000 r6 00000000  FMSS_CTRL after 0004fff4, C40 05000000 C44 57a27000, CS
  _IRQ 00000000
  marker, then CTRL fff5   -> END     r0 57a27000 r6 600d600d  FMSS_CTRL after 0000fff4, C40 00000000 C44 00000000, CS
  _IRQ 00000001
  marker, then CTRL 4fff5  -> END     r0 57a27000 r6 600d600d  FMSS_CTRL after 0000fff4, C40 00000000 C44 00000000, CS
  _IRQ 00000001
  marker, then CTRL 40000  -> TIMEOUT r0 00000000 r6 00000000  FMSS_CTRL after 00040000, C40 05000000 C44 57a27000, CS
  _IRQ 00000000
  marker, then CTRL 0      -> TIMEOUT r0 00000000 r6 00000000  FMSS_CTRL after 00040000, C40 05000000 C44 57a27000, CS
  _IRQ 00000000
```

14.2.36 oneshot (run 8)

ID: `D-oneshot-8`. Device run 8. Experiment: oneshot.

```
C58 bit 1 one-shot:
  CPU sets bit 1, END (control)    -> END     started at 0             C58 2 C5C 0 C4C 18 CS_IRQ 00000001
  program sets bit 1, END          -> END     started at 0             C58 2 C5C 0 C4C 20 CS_IRQ 00000001
  CPU sets bit 1, fault, END       -> FAULT   started at 0             C58 2 C5C 0 C4C 20 CS_IRQ 00000005
  program sets bit 1, fault, END   -> FAULT   started at 0             C58 2 C5C 0 C4C 28 CS_IRQ 00000005
```

14.2.37 c50 (run 9)

ID: `D-c50-9`. Device run 9. Experiment: c50.

```
FMSS_C50 seen by the fault handler:
  timeout, handler acks                    -> FAULT   C50 in the handler ffff1800, after the ack ffff1800, after the e
  nd ffff1800
  timeout, no ack                          -> FAULT   C50 in the handler ffff1800, after the ack ffff1800, after the e
  nd ffff1800
  illegal opcode, handler acks             -> FAULT   C50 in the handler ffff1800, after the ack ffff1800, after the e
  nd ffff1800
  program jumps to the handler (no fault)  -> END     C50 in the handler ffff1800, after the ack ffff1800, after the e
  nd ffff1800
```

14.2.38 holdout (run 9)

ID: `D-holdout-9`. Device run 9. Experiment: holdout.

```
held-out combinations:
  three timeouts, C58 0          -> FAULT      4877 us  CS_IRQ 00000004  r2 00000000 r3 00000000 r4 00000000 r6 000000
  00  C14 ff->f6
  shl 1 by 31 / shr by 31        -> END          18 us  CS_IRQ 00000001  r2 80000000 r3 00000001 r4 00000000 r6 600d60
  0d  C14 f0->f0
  pld 0xc30 reads r6             -> END          18 us  CS_IRQ 00000001  r2 6666aaaa r3 00000000 r4 00000000 r6 6666aa
  aa  C14 f0->f0
  op15 x4                        -> END        9710 us  CS_IRQ 00000001  r2 000000ff r3 00000000 r4 00000000 r6 000000
  00  C14 f0->ff
  loop of 100                    -> END          53 us  CS_IRQ 00000001  r2 00000000 r3 00000000 r4 00000000 r6 600d60
  0d  C14 ff->ff
  wait 11, timeout, op10         -> FAULT      4876 us  CS_IRQ 00000007  r2 00000000 r3 00000000 r4 00000000 r6 600d60
  0d  C14 ff->ff
  sti then ldi at scratch+0x200  -> END          19 us  CS_IRQ 00000001  r2 13572468 r3 08052f03 r4 13572468 r6 600d60
  0d  C14 ff->ff
```

14.2.39 holdout (run 10)

ID: `D-holdout-10`. Device run 10. Experiment: holdout.

```
held-out combinations (followed to the END after a fault):
  three timeouts, C58 0          -> FAULT     14618 us  CS_IRQ 00000005  r2 00000000 r3 00000000 r4 00000000 r6 600d60
  0d  C14 ff->ff  first fault at 4864 us
  shl 1 by 31 / shr by 31        -> END          18 us  CS_IRQ 00000001  r2 80000000 r3 00000001 r4 00000000 r6 600d60
  0d  C14 ff->ff
  pld 0xc30 reads r6             -> END          18 us  CS_IRQ 00000001  r2 6666aaaa r3 00000000 r4 00000000 r6 6666aa
  aa  C14 ff->ff
  op15 x4                        -> END        9717 us  CS_IRQ 00000001  r2 000000ff r3 00000000 r4 00000000 r6 000000
  00  C14 ff->ff
  loop of 100                    -> END          55 us  CS_IRQ 00000001  r2 00000000 r3 00000000 r4 00000000 r6 600d60
  0d  C14 ff->ff
  wait 11, timeout, op10         -> FAULT      4866 us  CS_IRQ 00000007  r2 00000000 r3 00000000 r4 00000000 r6 600d60
  0d  C14 ff->ff  first fault at 4848 us
  sti then ldi at scratch+0x200  -> END          18 us  CS_IRQ 00000001  r2 13572468 r3 08052f43 r4 13572468 r6 600d60
  0d  C14 ff->ff
```

14.2.40 op12wait (run 11)

ID: `D-op12wait-11`. Device run 11. Experiment: op12wait.

```
opcode 0x12 with the event in the register field:
  0x12 reg 11 (event 11, set at idle)            -> TIMEOUT BLOCKED after  20015 us, CS_IRQ 00000008, C08 a
  0x12 reg 0 after a command byte (event 0 set)  -> TIMEOUT BLOCKED after  20014 us, CS_IRQ 00000008, C08 a
  wait 0 after a command byte (control)          -> END     passed after     17 us, CS_IRQ 00000001, C08 0
  0x12 reg 3 (event 3, not set)                  -> TIMEOUT BLOCKED after  20015 us, CS_IRQ 00000008, C08 a
  0x12 reg 11, arg 5, imm 0x40                   -> TIMEOUT BLOCKED after  20015 us, CS_IRQ 00000008, C08 a
```

14.2.41 ev11 (run 11)

ID: `D-ev11-11`. Device run 11. Experiment: ev11.

```
event 11 around a block erase (tBERS ~1-2 ms):
  idle CE0, wait 11 (control)        -> END         17 us, CS_IRQ 00000001
  erase CE0, end at once (control)   -> END         17 us, CS_IRQ 00000001
  erase CE0, wait 11                 -> END         17 us, CS_IRQ 00000001
  erase CE0, select CE1, wait 11     -> END         17 us, CS_IRQ 00000001
  erase CE0, deselect, wait 11       -> END         16 us, CS_IRQ 00000001
  erase CE0, wait 11, wait 11        -> END         17 us, CS_IRQ 00000001
```

14.2.42 ev5 (run 11)

ID: `D-ev5-11`. Device run 11. Experiment: ev5.

```
event 5 and the status byte (b6 = ready, b0 = fail):
  idle: poll, wait 5                 -> END         17 us, status before wait 5 c0, after c0, last 00, CS_IRQ 00000001
  erase, poll, wait 5                -> END        172 us, status before wait 5 81, after c0, last 00, CS_IRQ 00000001
  erase, poll, wait 5, re-latch x8   -> FAULT    39160 us, status before wait 5 81, after c0, last c0, CS_IRQ 00000005
  erase, wait 11, poll, wait 5       -> END        172 us, status before wait 5 81, after c0, last 00, CS_IRQ 00000001
```

14.2.43 opsweep (run 11)

ID: `D-opsweep-11`. Device run 11. Experiment: opsweep.

```
opcode classes (first operand form):
  END continued 4.9 ms CS_IRQ 01 window              15
  END continued fast CS_IRQ 01 window                08 ff
  END continued fast CS_IRQ 03 window                10
  FAULT continued 4.9 ms CS_IRQ 05 window            09 16 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d
   2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f 50 51 52 53 5
  4 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 79 7a
  7b 7c 7d 7e 7f 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 a1
   a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 c1 c2 c3 c4 c5 c6 c7 c
  8 c9 ca cb cc cd ce cf d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee
  ef f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe
  FAULT continued 5.0 ms CS_IRQ 05 window
  FAULT continued 5.1 ms CS_IRQ 05 window
```

```
  FAULT continued 5.2 ms CS_IRQ 05 window
  FAULT stopped 4.9 ms CS_IRQ 04 window              0f
  TIMEOUT stopped 20.0 ms CS_IRQ 08 window           12
all runs:
  op 08 reg 3, arg 1, imm 0          -> END continued fast CS_IRQ 01 window; 0x028:0000007c->000000f8 0xc14:000000f8->
  000000ff
  op 08 reg 3, arg 0xd70, imm 0xd74  -> END continued fast CS_IRQ 01 window; 0x028:00000174->000001f0
  op 08 reg 0, arg 0, imm 0xffffffff -> END continued fast CS_IRQ 01 window; 0x028:0000006c->000000e8
  op 09 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000164->000001e0
  op 09 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000005c->000000d8
  op 09 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000154->000001d0
  op 0f reg 3, arg 1, imm 0          -> FAULT stopped 4.9 ms CS_IRQ 04 window; 0x028:0000004c->000000c8 0xc08:00000000
  ->00000004 0xc14:000000ff->000000f6 0xc40:00000000->e7922103 0xc44:00000000->0a000076
  op 0f reg 3, arg 0xd70, imm 0xd74  -> FAULT stopped 4.9 ms CS_IRQ 04 window; 0x028:00000144->000001c0 0xc08:00000000
  ->00000004 0xc14:000000f0->000000f6 0xc40:00000000->22833001 0xc44:00000000->e3500b01
  op 0f reg 0, arg 0, imm 0xffffffff -> FAULT stopped 4.9 ms CS_IRQ 04 window; 0x028:0000003c->000000b8 0xc08:00000000
  ->00000004 0xc14:000000f0->000000f6 0xc40:00000000->eaffff86 0xc44:00000000->05982c0c
  op 10 reg 3, arg 1, imm 0          -> END continued fast CS_IRQ 03 window; 0x028:00000134->000001b0
  op 10 reg 3, arg 0xd70, imm 0xd74  -> END continued fast CS_IRQ 03 window; 0x028:0000002c->000000a8
  op 10 reg 0, arg 0, imm 0xffffffff -> END continued fast CS_IRQ 03 window; 0x028:00000124->000001a0
  op 12 reg 3, arg 1, imm 0          -> TIMEOUT stopped 20.0 ms CS_IRQ 08 window; 0x028:0000001c->00000098 0xc08:00000
  000->0000000a 0xc40:00000000->12030001
  op 12 reg 3, arg 0xd70, imm 0xd74  -> TIMEOUT stopped 20.0 ms CS_IRQ 08 window; 0x028:00000114->00000190 0xc08:00000
  000->0000000a 0xc40:00000000->12030d70 0xc44:00000000->00000d74
  op 12 reg 0, arg 0, imm 0xffffffff -> TIMEOUT stopped 20.0 ms CS_IRQ 08 window; 0x028:0000000c->00000088 0xc08:00000
  000->0000000a 0xc40:00000000->12000000 0xc44:00000000->ffffffff
  op 15 reg 3, arg 1, imm 0          -> END continued 4.9 ms CS_IRQ 01 window; 0x028:00000104->00000180 0xc14:000000f0
  ->00000000
  op 15 reg 3, arg 0xd70, imm 0xd74  -> END continued fast CS_IRQ 01 window; 0x028:000001fc->00000078 0x050:00003800->
  00000000 0xc14:00000000->000000ff
  op 15 reg 0, arg 0, imm 0xffffffff -> END continued 4.9 ms CS_IRQ 01 window; 0x028:000000f4->00000170 0xc14:000000ff
  ->00000000
  op 16 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ec->00000068 0x050:000000
  00->00000800 0xc14:00000000->000000ff
  op 16 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000e4->00000160
  op 16 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001dc->00000058 0x050:000008
  00->00001000
  op 1a reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000d4->00000150
  op 1a reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001cc->00000048 0x050:000010
  00->00001800
  op 1a reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000c4->00000140
  op 1b reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001bc->00000038 0x050:000018
  00->00002000
  op 1b reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000b4->00000130
  op 1b reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ac->00000028 0x050:000020
  00->00002800
  op 1c reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000a4->00000120
  op 1c reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000019c->00000018 0x050:000028
  00->00003000
  op 1c reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000094->00000110
  op 1d reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000018c->00000008 0x050:000030
  00->00003800
  op 1d reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000084->00000100
  op 1d reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000017c->000001f8
  op 1e reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000074->000000f0
  op 1e reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000016c->000001e8
  op 1e reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000064->000000e0
  op 1f reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000015c->000001d8
  op 1f reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000054->000000d0
  op 1f reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000014c->000001c8
  op 20 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000044->000000c0
  op 20 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000013c->000001b8
  op 20 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000034->000000b0
  op 21 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000012c->000001a8
  op 21 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000024->000000a0
  op 21 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000011c->00000198
  op 22 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000014->00000090
  op 22 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000010c->00000188
  op 22 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000004->00000080
  op 23 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000fc->00000178
  op 23 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001f4->00000070 0x050:000038
  00->00000000
  op 23 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ec->00000168
  op 24 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001e4->00000060 0x050:000000
  00->00000800
  op 24 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000dc->00000158
  op 24 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001d4->00000050 0x050:000008
  00->00001000
  op 25 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000cc->00000148
  op 25 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001c4->00000040 0x050:000010
  00->00001800
  op 25 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000bc->00000138
  op 26 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001b4->00000030 0x050:000018
  00->00002000
  op 26 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ac->00000128
```

```
op 26 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001a4->00000020 0x050:000020
00->00002800
op 27 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000009c->00000118
op 27 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000194->00000010 0x050:000028
00->00003000
op 27 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000008c->00000108
op 28 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000184->00000000 0x050:000030
00->00003800
op 28 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000007c->000000f8
op 28 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000174->000001f0
op 29 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000006c->000000e8
op 29 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000164->000001e0
op 29 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000005c->000000d8
op 2a reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000154->000001d0
op 2a reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000004c->000000c8
op 2a reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000144->000001c0
op 2b reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000003c->000000b8
op 2b reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000134->000001b0
op 2b reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000002c->000000a8
op 2c reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000124->000001a0
op 2c reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000001c->00000098
op 2c reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000114->00000190
op 2d reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000000c->00000088
op 2d reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000104->00000180
op 2d reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001fc->00000078 0x050:000030
00->00003800
op 2e reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000f4->00000170
op 2e reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ec->00000068 0x050:000038
00->00000000
op 2e reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000e4->00000160
op 2f reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001dc->00000058 0x050:000000
00->00000800
op 2f reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000d4->00000150
op 2f reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001cc->00000048 0x050:000008
00->00001000
op 30 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000c4->00000140
op 30 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 5.0 ms CS_IRQ 05 window; 0x028:000001bc->00000038 0x050:000010
00->00001800
op 30 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000b4->00000130
op 31 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ac->00000028 0x050:000018
00->00002000
op 31 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000a4->00000120
op 31 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000019c->00000018 0x050:000020
00->00002800
op 32 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000094->00000110
op 32 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000018c->00000008 0x050:000028
00->00003000
op 32 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000084->00000100
op 33 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000017c->000001f8
op 33 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000074->000000f0
op 33 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000016c->000001e8
op 34 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000064->000000e0
op 34 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000015c->000001d8
op 34 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000054->000000d0
op 35 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000014c->000001c8
op 35 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000044->000000c0
op 35 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000013c->000001b8
op 36 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000034->000000b0
op 36 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000012c->000001a8
op 36 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000024->000000a0
op 37 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000011c->00000198
op 37 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000014->00000090
op 37 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000010c->00000188
op 38 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000004->00000080
op 38 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000fc->00000178
op 38 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001f4->00000070 0x050:000030
00->00003800
op 39 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ec->00000168
op 39 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001e4->00000060 0x050:000038
00->00000000
op 39 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000dc->00000158
op 3a reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001d4->00000050 0x050:000000
00->00000800
op 3a reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000cc->00000148
op 3a reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001c4->00000040 0x050:000008
00->00001000
op 3b reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000bc->00000138
op 3b reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001b4->00000030 0x050:000010
00->00001800
op 3b reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ac->00000128
op 3c reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001a4->00000020 0x050:000018
00->00002000
op 3c reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000009c->00000118
op 3c reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000194->00000010 0x050:000020
00->00002800
```

```
op 3d reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000008c->00000108
op 3d reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000184->00000000 0x050:000028
00->00003000
op 3d reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000007c->000000f8
op 3e reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000174->000001f0
op 3e reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000006c->000000e8
op 3e reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000164->000001e0
op 3f reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000005c->000000d8
op 3f reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000154->000001d0
op 3f reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000004c->000000c8
op 40 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000144->000001c0
op 40 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000003c->000000b8
op 40 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000134->000001b0
op 41 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000002c->000000a8
op 41 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000124->000001a0
op 41 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000001c->00000098
op 42 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000114->00000190
op 42 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000000c->00000088
op 42 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000104->00000180
op 43 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001fc->00000078 0x050:000028
00->00003000
op 43 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000f4->00000170
op 43 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ec->00000068 0x050:000030
00->00003800
op 44 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000e4->00000160
op 44 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001dc->00000058 0x050:000038
00->00000000
op 44 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000d4->00000150
op 45 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001cc->00000048 0x050:000000
00->00000800
op 45 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000c4->00000140
op 45 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001bc->00000038 0x050:000008
00->00001000
op 46 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000b4->00000130
op 46 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ac->00000028 0x050:000010
00->00001800
op 46 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000a4->00000120
op 47 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000019c->00000018 0x050:000018
00->00002000
op 47 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000094->00000110
op 47 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000018c->00000008 0x050:000020
00->00002800
op 48 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000084->00000100
op 48 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000017c->000001f8
op 48 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000074->000000f0
op 49 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000016c->000001e8
op 49 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000064->000000e0
op 49 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000015c->000001d8
op 4a reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000054->000000d0
op 4a reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000014c->000001c8
op 4a reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000044->000000c0
op 4b reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000013c->000001b8
op 4b reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000034->000000b0
op 4b reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000012c->000001a8
op 4c reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000024->000000a0
op 4c reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000011c->00000198
op 4c reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000014->00000090
op 4d reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000010c->00000188
op 4d reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000004->00000080
op 4d reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000fc->00000178
op 4e reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001f4->00000070 0x050:000028
00->00003000
op 4e reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ec->00000168
op 4e reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001e4->00000060 0x050:000030
00->00003800
op 4f reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000dc->00000158
op 4f reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001d4->00000050 0x050:000038
00->00000000
op 4f reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000cc->00000148
op 50 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001c4->00000040 0x050:000000
00->00000800
op 50 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000bc->00000138
op 50 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001b4->00000030 0x050:000008
00->00001000
op 51 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ac->00000128
op 51 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001a4->00000020 0x050:000010
00->00001800
op 51 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000009c->00000118
op 52 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000194->00000010 0x050:000018
00->00002000
op 52 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000008c->00000108
op 52 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000184->00000000 0x050:000020
00->00002800
op 53 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000007c->000000f8
op 53 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000174->000001f0
```

```
op 53 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000006c->000000e8
op 54 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000164->000001e0
op 54 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000005c->000000d8
op 54 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000154->000001d0
op 55 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000004c->000000c8
op 55 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 5.1 ms CS_IRQ 05 window; 0x028:00000144->000001c0
op 55 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000003c->000000b8
op 56 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000134->000001b0
op 56 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000002c->000000a8
op 56 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000124->000001a0
op 57 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000001c->00000098
op 57 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000114->00000190
op 57 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000000c->00000088
op 58 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000104->00000180
op 58 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001fc->00000078 0x050:000020
00->00002800
op 58 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000f4->00000170
op 59 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ec->00000068 0x050:000028
00->00003000
op 59 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000e4->00000160
op 59 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001dc->00000058 0x050:000030
00->00003800
op 5a reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000d4->00000150
op 5a reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001cc->00000048 0x050:000038
00->00000000
op 5a reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000c4->00000140
op 5b reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001bc->00000038 0x050:000000
00->00000800
op 5b reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000b4->00000130
op 5b reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ac->00000028 0x050:000008
00->00001000
op 5c reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000a4->00000120
op 5c reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000019c->00000018 0x050:000010
00->00001800
op 5c reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000094->00000110
op 5d reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000018c->00000008 0x050:000018
00->00002000
op 5d reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000084->00000100
op 5d reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000017c->000001f8
op 5e reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000074->000000f0
op 5e reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000016c->000001e8
op 5e reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000064->000000e0
op 5f reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000015c->000001d8
op 5f reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000054->000000d0
op 5f reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000014c->000001c8
op 60 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000044->000000c0
op 60 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000013c->000001b8
op 60 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000034->000000b0
op 61 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000012c->000001a8
op 61 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000024->000000a0
op 61 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000011c->00000198
op 62 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000014->00000090
op 62 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000010c->00000188
op 62 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000004->00000080
op 63 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000fc->00000178
op 63 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001f4->00000070 0x050:000020
00->00002800
op 63 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ec->00000168
op 64 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001e4->00000060 0x050:000028
00->00003000
op 64 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000dc->00000158
op 64 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001d4->00000050 0x050:000030
00->00003800
op 65 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000cc->00000148
op 65 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001c4->00000040 0x050:000038
00->00000000
op 65 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000bc->00000138
op 66 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001b4->00000030 0x050:000000
00->00000800
op 66 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ac->00000128
op 66 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001a4->00000020 0x050:000008
00->00001000
op 67 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000009c->00000118
op 67 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000194->00000010 0x050:000010
00->00001800
op 67 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000008c->00000108
op 68 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000184->00000000 0x050:000018
00->00002000
op 68 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000007c->000000f8
op 68 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000174->000001f0
op 69 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000006c->000000e8
op 69 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000164->000001e0
op 69 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000005c->000000d8
op 6a reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000154->000001d0
op 6a reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000004c->000000c8
```

```
op 6a reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000144->000001c0
op 6b reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000003c->000000b8
op 6b reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000134->000001b0
op 6b reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000002c->000000a8
op 6c reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000124->000001a0
op 6c reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000001c->00000098
op 6c reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000114->00000190
op 6d reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000000c->00000088
op 6d reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000104->00000180
op 6d reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001fc->00000078 0x050:000018
00->00002000
op 6e reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000f4->00000170
op 6e reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ec->00000068 0x050:000020
00->00002800
op 6e reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000e4->00000160
op 6f reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001dc->00000058 0x050:000028
00->00003000
op 6f reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000d4->00000150
op 6f reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001cc->00000048 0x050:000030
00->00003800
op 70 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000c4->00000140
op 70 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001bc->00000038 0x050:000038
00->00000000
op 70 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000b4->00000130
op 71 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ac->00000028 0x050:000000
00->00000800
op 71 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000a4->00000120
op 71 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000019c->00000018 0x050:000008
00->00001000
op 72 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000094->00000110
op 72 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000018c->00000008 0x050:000010
00->00001800
op 72 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000084->00000100
op 73 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000017c->000001f8
op 73 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000074->000000f0
op 73 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000016c->000001e8
op 74 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000064->000000e0
op 74 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000015c->000001d8
op 74 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000054->000000d0
op 75 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000014c->000001c8
op 75 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000044->000000c0
op 75 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000013c->000001b8
op 76 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000034->000000b0
op 76 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000012c->000001a8
op 76 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000024->000000a0
op 77 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000011c->00000198
op 77 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000014->00000090
op 77 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000010c->00000188
op 78 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000004->00000080
op 78 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000fc->00000178
op 78 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001f4->00000070 0x050:000018
00->00002000
op 79 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ec->00000168
op 79 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001e4->00000060 0x050:000020
00->00002800
op 79 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000dc->00000158
op 7a reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001d4->00000050 0x050:000028
00->00003000
op 7a reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000cc->00000148
op 7a reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001c4->00000040 0x050:000030
00->00003800
op 7b reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000bc->00000138
op 7b reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001b4->00000030 0x050:000038
00->00000000
op 7b reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ac->00000128
op 7c reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001a4->00000020 0x050:000000
00->00000800
op 7c reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000009c->00000118
op 7c reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000194->00000010 0x050:000008
00->00001000
op 7d reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000008c->00000108
op 7d reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000184->00000000 0x050:000010
00->00001800
op 7d reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000007c->000000f8
op 7e reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000174->000001f0
op 7e reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000006c->000000e8
op 7e reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000164->000001e0
op 7f reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000005c->000000d8
op 7f reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000154->000001d0
op 7f reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000004c->000000c8
op 80 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000144->000001c0
op 80 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000003c->000000b8
op 80 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000134->000001b0
op 81 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000002c->000000a8
op 81 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000124->000001a0
```

```
op 81 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000001c->00000098
op 82 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000114->00000190
op 82 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000000c->00000088
op 82 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000104->00000180
op 83 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001fc->00000078 0x050:000010
00->00001800
op 83 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000f4->00000170
op 83 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ec->00000068 0x050:000018
00->00002000
op 84 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000e4->00000160
op 84 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001dc->00000058 0x050:000020
00->00002800
op 84 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000d4->00000150
op 85 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001cc->00000048 0x050:000028
00->00003000
op 85 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000c4->00000140
op 85 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001bc->00000038 0x050:000030
00->00003800
op 86 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000b4->00000130
op 86 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ac->00000028 0x050:000038
00->00000000
op 86 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000a4->00000120
op 87 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000019c->00000018 0x050:000000
00->00000800
op 87 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000094->00000110
op 87 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000018c->00000008 0x050:000008
00->00001000
op 88 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000084->00000100
op 88 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000017c->000001f8
op 88 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000074->000000f0
op 89 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000016c->000001e8
op 89 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000064->000000e0
op 89 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000015c->000001d8
op 8a reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000054->000000d0
op 8a reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000014c->000001c8
op 8a reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000044->000000c0
op 8b reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000013c->000001b8
op 8b reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000034->000000b0
op 8b reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000012c->000001a8
op 8c reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000024->000000a0
op 8c reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000011c->00000198
op 8c reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000014->00000090
op 8d reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000010c->00000188
op 8d reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000004->00000080
op 8d reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000fc->00000178
op 8e reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001f4->00000070 0x050:000010
00->00001800
op 8e reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ec->00000168
op 8e reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001e4->00000060 0x050:000018
00->00002000
op 8f reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000dc->00000158
op 8f reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001d4->00000050 0x050:000020
00->00002800
op 8f reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000cc->00000148
op 90 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001c4->00000040 0x050:000028
00->00003000
op 90 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000bc->00000138
op 90 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001b4->00000030 0x050:000030
00->00003800
op 91 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ac->00000128
op 91 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001a4->00000020 0x050:000038
00->00000000
op 91 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000009c->00000118
op 92 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000194->00000010 0x050:000000
00->00000800
op 92 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000008c->00000108
op 92 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000184->00000000 0x050:000008
00->00001000
op 93 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000007c->000000f8
op 93 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000174->000001f0
op 93 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000006c->000000e8
op 94 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000164->000001e0
op 94 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000005c->000000d8
op 94 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000154->000001d0
op 95 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000004c->000000c8
op 95 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000144->000001c0
op 95 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000003c->000000b8
op 96 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000134->000001b0
op 96 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000002c->000000a8
op 96 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000124->000001a0
op 97 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000001c->00000098
op 97 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000114->00000190
op 97 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000000c->00000088
op 98 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000104->00000180
op 98 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001fc->00000078 0x050:000008
```

```
00->00001000
op 98 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000f4->00000170
op 99 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ec->00000068 0x050:000010
00->00001800
op 99 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000e4->00000160
op 99 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001dc->00000058 0x050:000018
00->00002000
op 9a reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000d4->00000150
op 9a reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001cc->00000048 0x050:000020
00->00002800
op 9a reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000c4->00000140
op 9b reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001bc->00000038 0x050:000028
00->00003000
op 9b reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000b4->00000130
op 9b reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ac->00000028 0x050:000030
00->00003800
op 9c reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000a4->00000120
op 9c reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000019c->00000018 0x050:000038
00->00000000
op 9c reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000094->00000110
op 9d reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000018c->00000008 0x050:000000
00->00000800
op 9d reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000084->00000100
op 9d reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000017c->000001f8
op 9e reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000074->000000f0
op 9e reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000016c->000001e8
op 9e reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000064->000000e0
op 9f reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000015c->000001d8
op 9f reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000054->000000d0
op 9f reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000014c->000001c8
op a0 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000044->000000c0
op a0 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000013c->000001b8
op a0 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000034->000000b0
op a1 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000012c->000001a8
op a1 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000024->000000a0
op a1 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000011c->00000198
op a2 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000014->00000090
op a2 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000010c->00000188
op a2 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000004->00000080
op a3 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000fc->00000178
op a3 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001f4->00000070 0x050:000008
00->00001000
op a3 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ec->00000168
op a4 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001e4->00000060 0x050:000010
00->00001800
op a4 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000dc->00000158
op a4 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001d4->00000050 0x050:000018
00->00002000
op a5 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000cc->00000148
op a5 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001c4->00000040 0x050:000020
00->00002800
op a5 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000bc->00000138
op a6 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001b4->00000030 0x050:000028
00->00003000
op a6 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ac->00000128
op a6 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001a4->00000020 0x050:000030
00->00003800
op a7 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000009c->00000118
op a7 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000194->00000010 0x050:000038
00->00000000
op a7 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000008c->00000108
op a8 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000184->00000000 0x050:000000
00->00000800
op a8 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000007c->000000f8
op a8 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000174->000001f0
op a9 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000006c->000000e8
op a9 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000164->000001e0
op a9 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000005c->000000d8
op aa reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000154->000001d0
op aa reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000004c->000000c8
op aa reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000144->000001c0
op ab reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000003c->000000b8
op ab reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000134->000001b0
op ab reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000002c->000000a8
op ac reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000124->000001a0
op ac reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000001c->00000098
op ac reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000114->00000190
op ad reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000000c->00000088
op ad reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000104->00000180
op ad reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001fc->00000078 0x050:000000
00->00000800
op ae reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000f4->00000170
op ae reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ec->00000068 0x050:000008
00->00001000
op ae reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000e4->00000160
```

```
op af reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001dc->00000058 0x050:000010
00->00001800
op af reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000d4->00000150
op af reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001cc->00000048 0x050:000018
00->00002000
op b0 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000c4->00000140
op b0 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001bc->00000038 0x050:000020
00->00002800
op b0 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000b4->00000130
op b1 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ac->00000028 0x050:000028
00->00003000
op b1 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000a4->00000120
op b1 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000019c->00000018 0x050:000030
00->00003800
op b2 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000094->00000110
op b2 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000018c->00000008 0x050:000038
00->00000000
op b2 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000084->00000100
op b3 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000017c->000001f8
op b3 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000074->000000f0
op b3 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000016c->000001e8
op b4 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000064->000000e0
op b4 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000015c->000001d8
op b4 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000054->000000d0
op b5 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000014c->000001c8
op b5 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000044->000000c0
op b5 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000013c->000001b8
op b6 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000034->000000b0
op b6 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000012c->000001a8
op b6 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000024->000000a0
op b7 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000011c->00000198
op b7 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000014->00000090
op b7 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000010c->00000188
op b8 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000004->00000080
op b8 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000fc->00000178
op b8 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001f4->00000070 0x050:000000
00->00000800
op b9 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ec->00000168
op b9 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001e4->00000060 0x050:000008
00->00001000
op b9 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000dc->00000158
op ba reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001d4->00000050 0x050:000010
00->00001800
op ba reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000cc->00000148
op ba reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001c4->00000040 0x050:000018
00->00002000
op bb reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000bc->00000138
op bb reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001b4->00000030 0x050:000020
00->00002800
op bb reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ac->00000128
op bc reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001a4->00000020 0x050:000028
00->00003000
op bc reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000009c->00000118
op bc reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000194->00000010 0x050:000030
00->00003800
op bd reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000008c->00000108
op bd reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000184->00000000 0x050:000038
00->00000000
op bd reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000007c->000000f8
op be reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000174->000001f0
op be reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000006c->000000e8
op be reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000164->000001e0
op bf reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000005c->000000d8
op bf reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000154->000001d0
op bf reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000004c->000000c8
op c0 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000144->000001c0
op c0 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000003c->000000b8
op c0 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000134->000001b0
op c1 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000002c->000000a8
op c1 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000124->000001a0
op c1 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000001c->00000098
op c2 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000114->00000190
op c2 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000000c->00000088
op c2 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000104->00000180
op c3 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001fc->00000078 0x050:000038
00->00000000
op c3 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000f4->00000170
op c3 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ec->00000068 0x050:000000
00->00000800
op c4 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000e4->00000160
op c4 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001dc->00000058 0x050:000008
00->00001000
op c4 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000d4->00000150
op c5 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001cc->00000048 0x050:000010
00->00001800
```

```
op c5 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000c4->00000140
op c5 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001bc->00000038 0x050:000018
00->00002000
op c6 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000b4->00000130
op c6 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ac->00000028 0x050:000020
00->00002800
op c6 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000a4->00000120
op c7 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000019c->00000018 0x050:000028
00->00003000
op c7 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000094->00000110
op c7 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000018c->00000008 0x050:000030
00->00003800
op c8 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000084->00000100
op c8 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000017c->000001f8
op c8 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000074->000000f0
op c9 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000016c->000001e8
op c9 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000064->000000e0
op c9 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000015c->000001d8
op ca reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000054->000000d0
op ca reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000014c->000001c8
op ca reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000044->000000c0
op cb reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000013c->000001b8
op cb reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000034->000000b0
op cb reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000012c->000001a8
op cc reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000024->000000a0
op cc reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000011c->00000198
op cc reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000014->00000090
op cd reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000010c->00000188
op cd reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000004->00000080
op cd reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000fc->00000178
op ce reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001f4->00000070 0x050:000038
00->00000000
op ce reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ec->00000168
op ce reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001e4->00000060 0x050:000000
00->00000800
op cf reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000dc->00000158
op cf reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001d4->00000050 0x050:000008
00->00001000
op cf reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000cc->00000148
op d0 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001c4->00000040 0x050:000010
00->00001800
op d0 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000bc->00000138
op d0 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001b4->00000030 0x050:000018
00->00002000
op d1 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ac->00000128
op d1 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001a4->00000020 0x050:000020
00->00002800
op d1 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000009c->00000118
op d2 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000194->00000010 0x050:000028
00->00003000
op d2 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000008c->00000108
op d2 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000184->00000000 0x050:000030
00->00003800
op d3 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000007c->000000f8
op d3 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000174->000001f0
op d3 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000006c->000000e8
op d4 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000164->000001e0
op d4 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000005c->000000d8
op d4 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000154->000001d0
op d5 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000004c->000000c8
op d5 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000144->000001c0
op d5 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000003c->000000b8
op d6 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000134->000001b0
op d6 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000002c->000000a8
op d6 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000124->000001a0
op d7 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000001c->00000098
op d7 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000114->00000190
op d7 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000000c->00000088
op d8 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000104->00000180
op d8 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001fc->00000078 0x050:000030
00->00003800
op d8 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000f4->00000170
op d9 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ec->00000068 0x050:000038
00->00000000
op d9 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000e4->00000160
op d9 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001dc->00000058 0x050:000000
00->00000800
op da reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000d4->00000150
op da reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001cc->00000048 0x050:000008
00->00001000
op da reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000c4->00000140
op db reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001bc->00000038 0x050:000010
00->00001800
op db reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000b4->00000130
op db reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ac->00000028 0x050:000018
```

```
00->00002000
op dc reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000a4->00000120
op dc reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000019c->00000018 0x050:000020
00->00002800
op dc reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000094->00000110
op dd reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000018c->00000008 0x050:000028
00->00003000
op dd reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000084->00000100
op dd reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000017c->000001f8
op de reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000074->000000f0
op de reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000016c->000001e8
op de reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000064->000000e0
op df reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000015c->000001d8
op df reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000054->000000d0
op df reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000014c->000001c8
op e0 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000044->000000c0
op e0 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000013c->000001b8
op e0 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000034->000000b0
op e1 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000012c->000001a8
op e1 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000024->000000a0
op e1 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000011c->00000198
op e2 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000014->00000090
op e2 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000010c->00000188
op e2 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000004->00000080
op e3 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000fc->00000178
op e3 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001f4->00000070 0x050:000030
00->00003800
op e3 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ec->00000168
op e4 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001e4->00000060 0x050:000038
00->00000000
op e4 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000dc->00000158
op e4 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001d4->00000050 0x050:000000
00->00000800
op e5 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000cc->00000148
op e5 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001c4->00000040 0x050:000008
00->00001000
op e5 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000bc->00000138
op e6 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001b4->00000030 0x050:000010
00->00001800
op e6 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ac->00000128
op e6 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001a4->00000020 0x050:000018
00->00002000
op e7 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000009c->00000118
op e7 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000194->00000010 0x050:000020
00->00002800
op e7 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000008c->00000108
op e8 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000184->00000000 0x050:000028
00->00003000
op e8 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000007c->000000f8
op e8 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000174->000001f0
op e9 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000006c->000000e8
op e9 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000164->000001e0
op e9 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000005c->000000d8
op ea reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000154->000001d0
op ea reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000004c->000000c8
op ea reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000144->000001c0
op eb reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000003c->000000b8
op eb reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 5.2 ms CS_IRQ 05 window; 0x028:00000134->000001b0
op eb reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000002c->000000a8
op ec reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000124->000001a0
op ec reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000001c->00000098
op ec reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000114->00000190
op ed reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000000c->00000088
op ed reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000104->00000180
op ed reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001fc->00000078 0x050:000028
00->00003000
op ee reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000f4->00000170
op ee reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ec->00000068 0x050:000030
00->00003800
op ee reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000e4->00000160
op ef reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001dc->00000058 0x050:000038
00->00000000
op ef reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000d4->00000150
op ef reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001cc->00000048 0x050:000000
00->00000800
op f0 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000c4->00000140
op f0 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001bc->00000038 0x050:000008
00->00001000
op f0 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000b4->00000130
op f1 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001ac->00000028 0x050:000010
00->00001800
op f1 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000a4->00000120
op f1 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000019c->00000018 0x050:000018
00->00002000
op f2 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000094->00000110
```

```
  op f2 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000018c->00000008 0x050:000020
  00->00002800
  op f2 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000084->00000100
  op f3 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000017c->000001f8
  op f3 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000074->000000f0
  op f3 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000016c->000001e8
  op f4 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000064->000000e0
  op f4 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000015c->000001d8
  op f4 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000054->000000d0
  op f5 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000014c->000001c8
  op f5 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000044->000000c0
  op f5 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000013c->000001b8
  op f6 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000034->000000b0
  op f6 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000012c->000001a8
  op f6 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000024->000000a0
  op f7 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000011c->00000198
  op f7 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000014->00000090
  op f7 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000010c->00000188
  op f8 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000004->00000080
  op f8 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000fc->00000178
  op f8 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001f4->00000070 0x050:000028
  00->00003000
  op f9 reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ec->00000168
  op f9 reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001e4->00000060 0x050:000030
  00->00003800
  op f9 reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000dc->00000158
  op fa reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001d4->00000050 0x050:000038
  00->00000000
  op fa reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000cc->00000148
  op fa reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001c4->00000040 0x050:000000
  00->00000800
  op fb reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000bc->00000138
  op fb reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001b4->00000030 0x050:000008
  00->00001000
  op fb reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000000ac->00000128
  op fc reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:000001a4->00000020 0x050:000010
  00->00001800
  op fc reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000009c->00000118
  op fc reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000194->00000010 0x050:000018
  00->00002000
  op fd reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000008c->00000108
  op fd reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000184->00000000 0x050:000020
  00->00002800
  op fd reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000007c->000000f8
  op fe reg 3, arg 1, imm 0          -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000174->000001f0
  op fe reg 3, arg 0xd70, imm 0xd74  -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:0000006c->000000e8
  op fe reg 0, arg 0, imm 0xffffffff -> FAULT continued 4.9 ms CS_IRQ 05 window; 0x028:00000164->000001e0
  op ff reg 3, arg 1, imm 0          -> END continued fast CS_IRQ 01 window; 0x028:0000005c->000000d8
  op ff reg 3, arg 0xd70, imm 0xd74  -> END continued fast CS_IRQ 01 window; 0x028:00000154->000001d0
  op ff reg 0, arg 0, imm 0xffffffff -> END continued fast CS_IRQ 01 window; 0x028:0000004c->000000c8
```

14.2.44 optime (run 11)

ID: `D-optime-11`. Device run 11. Experiment: optime.

```
1000 executions each:
  movi (control): 261 us
  op 08 reg 0 arg 0     imm 0        -> END        262 us (+0.00 us per execution vs movi), CS_IRQ 00000001, r6 600d60
  0d
  op 08 reg 7 arg ffff  imm ffffffff -> END        263 us (+0.00 us per execution vs movi), CS_IRQ 00000001, r6 600d60
  0d
  op 08 reg 3 arg d70   imm 1000     -> END        263 us (+0.00 us per execution vs movi), CS_IRQ 00000001, r6 600d60
  0d
  op ff reg 0 arg 0     imm 0        -> END        261 us (+0.00 us per execution vs movi), CS_IRQ 00000001, r6 600d60
  0d
  op ff reg 7 arg ffff  imm ffffffff -> END        262 us (+0.00 us per execution vs movi), CS_IRQ 00000001, r6 600d60
  0d
  op ff reg 3 arg d70   imm 1000     -> END        262 us (+0.00 us per execution vs movi), CS_IRQ 00000001, r6 600d60
  0d
  op 10 reg 0 arg 0     imm 0        -> END        262 us (+0.00 us per execution vs movi), CS_IRQ 00000003, r6 600d60
  0d
  op 10 reg 7 arg ffff  imm ffffffff -> END        262 us (+0.00 us per execution vs movi), CS_IRQ 00000003, r6 600d60
  0d
  op 10 reg 3 arg d70   imm 1000     -> END        262 us (+0.00 us per execution vs movi), CS_IRQ 00000003, r6 600d60
  0d
```

14.2.45 ctrlbits (run 11)

ID: `D-ctrlbits-11`. Device run 11. Experiment: ctrlbits.

```
start words (loop of 1000 x 2 instructions, ~430 us at 0xfff5):
  start 0x0000fff5 -> END        419 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 0000fff4, 36 samples,
  pcs 0 8 28 30
  start 0x00000001 -> END        408 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 35 samples,
  pcs 8 28 30
  start 0x00000003 -> TIMEOUT  50030 us, r0    0, r6 00000000, CS_IRQ 00000000, FMSS_CTRL after 00040002, 1 samples, p
```

```
  cs 0
  start 0x00000005 -> END        419 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000004, 36 samples,
  pcs 0 8 28 30
  start 0x00000009 -> TIMEOUT  50031 us, r0    0, r6 00000000, CS_IRQ 00000000, FMSS_CTRL after 00040000, 1 samples, p
  cs 0
  start 0x00000011 -> END        419 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000010, 36 samples,
  pcs 0 8 28 30
  start 0x00000021 -> END        419 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000020, 36 samples,
  pcs 0 8 28 30
  start 0x00000041 -> END        418 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000040, 36 samples,
  pcs 0 8 28 30
  start 0x00000081 -> END        418 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000080, 36 samples,
  pcs 8 28 30
  start 0x00000101 -> END        407 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000100, 35 samples,
  pcs 8 28 30
  start 0x00000201 -> END        418 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000200, 36 samples,
  pcs 0 8 28 30
  start 0x00000401 -> END        408 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000400, 35 samples,
  pcs 8 28 30
  start 0x00000801 -> END        418 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000800, 36 samples,
  pcs 0 8 28 30
  start 0x00001001 -> END        407 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00001000, 35 samples,
  pcs 8 28 30
  start 0x00002001 -> END        418 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00002000, 36 samples,
  pcs 0 8 28 30
  start 0x00004001 -> END        419 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00004000, 36 samples,
  pcs 8 28 30 38
  start 0x00008001 -> END        419 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00008000, 36 samples,
  pcs 0 8 28 30
  start 0x00040001 -> END        418 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 8 28 30 40
  start 0x00080001 -> END        419 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 0 8 28 30
  start 0x00100001 -> END        418 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 0 8 28 30
  start 0x00200001 -> END        419 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 0 8 28 30
  start 0x00400001 -> END        419 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 0 8 28 30
  start 0x00800001 -> END        418 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 0 8 28 30
  start 0x01000001 -> END        418 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 0 8 28 30
  start 0x02000001 -> END        418 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 8 28 30 40
  start 0x04000001 -> END        418 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 8 28 30
  start 0x08000001 -> END        408 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 35 samples,
  pcs 8 28 30
  start 0x10000001 -> END        408 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 35 samples,
  pcs 8 28 30
  start 0x20000001 -> END        419 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 0 8 28 30
  start 0x40000001 -> END        419 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 0 8 28 30
  start 0x80000001 -> END        419 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 0 8 28 30
  start 0x0000fff7 -> TIMEOUT  50028 us, r0    0, r6 00000000, CS_IRQ 00000000, FMSS_CTRL after 0004fff6, 1 samples, p
  cs 0
  start 0x0000fffd -> TIMEOUT  50030 us, r0    0, r6 00000000, CS_IRQ 00000000, FMSS_CTRL after 0004fff4, 1 samples, p
  cs 0
  start 0x0000ffff -> TIMEOUT  50031 us, r0    0, r6 00000000, CS_IRQ 00000000, FMSS_CTRL after 0004fff6, 1 samples, p
  cs 0
  start 0x0000fff4 -> TIMEOUT  50031 us, r0    0, r6 00000000, CS_IRQ 00000000, FMSS_CTRL after 0000fff4, 1 samples, p
  cs 0
  start 0x00010001 -> TIMEOUT  50032 us, r0    0, r6 00000000, CS_IRQ 00000000, FMSS_CTRL after 00040000, 1 samples, p
  cs 0
  start 0x00020001 -> END        418 us, r0    0, r6 600d600d, CS_IRQ 00000001, FMSS_CTRL after 00000000, 36 samples,
  pcs 0 8 28 30
```

14.2.46 fmcmap (run 11)

ID: `D-fmcmap-11`. Device run 11. Experiment: fmcmap.

```
END; non-zero words per 256-byte block:
  0x000..0x0fc: 008:00000070 00c:0005dc00 014:00000001 020:007f167f 028:00000148 02c:00000002 038:00000007 048:000c300
  0 04c:000000ff 050:00003800
  0x100..0x1fc: 1bc:ffffffff 1c0:ffffffff 1c4:ffffffff 1c8:ffffffff 1cc:00008060 1d0:003010d0 1d4:ffffffff 1d8:ffffff
  f 1dc:ffffffff 1e0:ffffffff 1e4:ffffffff 1e8:ffffffff 1ec:ffffffff 1f0:ffffffff 1f4:ffffffff 1f8:ffffffff 1fc:ffffff
  ff
  0x200..0x2fc: 208:00000070 20c:0005dc00 214:00000001 220:007f167f 228:000001c8 22c:00000002 238:00000007 248:000c300
  0 24c:000000ff 250:00003800
  0x300..0x3fc: 3bc:ffffffff 3c0:ffffffff 3c4:ffffffff 3c8:ffffffff 3cc:00008060 3d0:003010d0 3d4:ffffffff 3d8:ffffff
  f 3dc:ffffffff 3e0:ffffffff 3e4:ffffffff 3e8:ffffffff 3ec:ffffffff 3f0:ffffffff 3f4:ffffffff 3f8:ffffffff 3fc:ffffff
  ff
  0x400..0x4fc: 420:007f167f 438:00000007 448:000c3000 44c:00000001
```

```
  0x500..0x5fc: 5bc:ffffffff 5c0:ffffffff 5c4:ffffffff 5c8:ffffffff 5cc:00008060 5d0:003010d0 5d4:ffffffff 5d8:ffffffff 5dc:ffffffff 5e0:ffffffff 5e4:ffffffff 5e8:ffffffff 5ec:ffffffff 5f0:ffffffff 5f4:ffffffff 5f8:ffffffff 5fc:ffffffff
  0x600..0x6fc: 620:007f167f 628:00000080 638:00000007 648:000c3000 64c:00000001
  0x700..0x7fc: 7bc:ffffffff 7c0:ffffffff 7c4:ffffffff 7c8:ffffffff 7cc:00008060 7d0:003010d0 7d4:ffffffff 7d8:ffffffff 7dc:ffffffff 7e0:ffffffff 7e4:ffffffff 7e8:ffffffff 7ec:ffffffff 7f0:ffffffff 7f4:ffffffff 7f8:ffffffff 7fc:ffffffff
  0x800..0x8fc: 800:20060808 814:00e00000 840:00000010
  0x900..0x9fc: 9c0:00000001
  0xa00..0xafc: a00:20060808 a14:00e00000 a40:00000010
  0xb00..0xbfc: bc0:00000001
  0xc00..0xcfc: c00:0004fff4 c04:0801b140 c08:00000005 c14:000000ff c18:08053958 c1c:00000c1c c20:000000f8 c24:000000f8 c38:00000030 c40:18030001 c4c:00000070 c50:ffff1800
  0xd00..0xdfc: d00:a5000d00 d04:a5000d04 d08:a5000d08 d0c:a5000d0c d10:a5000d10 d14:a5000d14 d18:a5000d18 d1c:a5000d1c d20:a5000d20 d24:a5000d24 d28:a5000d28 d2c:a5000d2c d30:a5000d30 d34:a5000d34 d38:a5000d38 d3c:a5000d3c d40:c0de0d40 d44:c0de0d44 d48:c0de0d48 d4c:c0de0d4c d50:c0de0d50 d54:c0de0d54 d58:c0de0d58 d5c:c0de0d5c ... 8 more
  0xe00..0xefc: all zero
  0xf00..0xffc: all zero
```

14.2.47 evsweep (run 11)

ID: `D-evsweep-11`. Device run 11. Experiment: evsweep.

The experiment changed between runs.

```
wait N after each context (time beyond the same program without the wait; T = timed out):
  idle, CE0 selected           status before 000c3000 | 0:T 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:0us 12:T 13:T 14:T 15:T
  after a command byte (70h)   status before 000c3002 | 0:0us 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:0us 12:T 13:T 14:T 15:T
  after 70h + status latch     status before 008c3002 | 0:0us 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:0us 12:T 13:T 14:T 15:T
  during an erase              status before 000c3000 | 0:T 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:0us 12:T 13:T 14:T 15:T
  2 ms after an erase start    status before 000c3000 | 0:T 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:0us 12:T 13:T 14:T 15:T
```

14.2.48 rwmask (run 12)

ID: `D-rwmask-12`. Device run 12. Experiment: rwmask.

```
registers that keep a CPU write or change (others read the same idle value):
  0x000 idle 00000000  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x00c idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x010 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x014 idle 0000a5a5  ones 0000ffff zeros 00000000 a5 0000a5a5 -> writable 0000ffff
  0x018 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x020 idle 00a5a5a5  ones 00ffffff zeros 00000000 a5 00a5a5a5 -> writable 00ffffff
  0x024 idle 000001a5  ones 000001ff zeros 00000000 a5 000001a5 -> writable 000001ff
  0x028 idle 0000009d  ones 000001ff zeros 00000000 a5 000001a5 -> writable 000001ff
  0x02c idle 00000005  ones 00000007 zeros 00000000 a5 00000005 -> writable 00000007
  0x030 idle 000005a5  ones 00001fff zeros 00000000 a5 000005a5 -> writable 00001fff
  0x034 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x044 idle 0000a5a5  ones 0000ffff zeros 00000000 a5 0000a5a5 -> writable 0000ffff
  0x060 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x064 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x068 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x06c idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x070 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x074 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x078 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x804 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x808 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x814 idle 01a5a5a5  ones 01ffffff zeros 00000000 a5 01a5a5a5 -> writable 01ffffff
  0x844 idle 00000025  ones 0000002f zeros 00000000 a5 00000025 -> writable 0000002f
  0xc04 idle 09f92e80  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0xc10 idle 000000a5  ones 000000ff zeros 00000000 a5 000000a5 -> writable 000000ff
  0xc18 idle 00000000  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0xc1c idle 00000000  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0xc20 idle 00000000  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0xc24 idle 00000000  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0xc28 idle 00000000  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0xc2c idle 00000000  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0xc30 idle 00000000  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0xc34 idle 00000000  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0xc3c idle 00000000  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0xc48 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0xc4c idle 00000000  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0xc54 idle 000005a5  ones 00000fff zeros 00000000 a5 000005a5 -> writable 00000fff
  0xc58 idle 00000000  ones 000000ff zeros 00000000 a5 000000a5 -> writable 000000ff
  0xc5c idle 00000000  ones 00000001 zeros 00000000 a5 00000001 -> writable 00000001
  0x400 idle 00000000  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x404 idle 00000105  ones 0000031f zeros 00000306 a5 00000105 -> writable 00000019
  0x408 idle 000000a5  ones 000000ff zeros 00000000 a5 000000a5 -> writable 000000ff
  0x40c idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x410 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
```

```
  0x414 idle 0000a5a5  ones 0000ffff zeros 00000000 a5 0000a5a5 -> writable 0000ffff
  0x418 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x420 idle 00a5a5a5  ones 00ffffff zeros 00000000 a5 00a5a5a5 -> writable 00ffffff
  0x424 idle 000001a5  ones 000001ff zeros 00000000 a5 000001a5 -> writable 000001ff
  0x428 idle 000000a5  ones 000001ff zeros 00000000 a5 000001a5 -> writable 000001ff
  0x42c idle 00000005  ones 00000007 zeros 00000000 a5 00000005 -> writable 00000007
  0x430 idle 000005a5  ones 00001fff zeros 00000000 a5 000005a5 -> writable 00001fff
  0x434 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x438 idle 000005a5  ones 00001fff zeros 00000000 a5 000005a5 -> writable 00001fff
  0x444 idle 0000a5a5  ones 0000ffff zeros 00000000 a5 0000a5a5 -> writable 0000ffff
  0x460 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x464 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x468 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x46c idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x470 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x474 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x478 idle a5a5a5a5  ones ffffffff zeros 00000000 a5 a5a5a5a5 -> writable ffffffff
  0x47c idle 000005a4  ones 00000ffc zeros 00000000 a5 000005a4 -> writable 00000ffc
```

14.2.49 c50x (run 13)

ID: `D-c50x-13`. Device run 13. Experiment: c50x.

```
faults and FMSS_C50 b0:
  wait 3, CE0 selected, nothing started    -> END         17 us, C50 in the handler 00000000, C3C 00000000, CS_IRQ 000
  00001
  ldi from 0x50000000                      -> END         17 us, C50 in the handler 00000000, C3C 00000000, CS_IRQ 000
  00001
  DMA to 0x50000000                        -> FAULT     4878 us, C50 in the handler ffff1800, C3C 00000070, CS_IRQ 000
  00015
```

14.2.50 evsweep (run 14)

ID: `D-evsweep-14`. Device run 14. Experiment: evsweep.

The experiment changed between runs.

```
wait N after each context (time beyond the same program without the wait; T = timed out):
  idle, CE0 selected           status 000c3000 | 0:T 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:0us 12:T 13:T 14:T 15
  :T
  after a command byte (70h)   status 000c3002 | 0:1us 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:1us 12:T 13:T 14:T
  15:1us
  after 70h + status latch     status 008c3002 | 0:0us 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:0us 12:T 13:T 14:T
  15:0us
  during an erase              status 000c3000 | 0:T 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:0us 12:T 13:T 14:T 15
  :T
  2 ms after an erase start    status 000c3000 | 0:T 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:11us 12:T 13:T 14:T 1
  5:T
```

14.2.51 op0f (run 14)

ID: `D-op0f-14`. Device run 14. Experiment: op0f.

```
opcode 0x0f (labels are byte offsets from the program start):
  arg 1 -> r1 = lab1               -> END     lab1         r2..r5,r7 0 0 0 0 0; C3C 00000000 C48 00000000; CS_IRQ 0000
  0001
  reg 1, arg 0 -> r0 = lab2        -> END     lab2         r2..r5,r7 0 0 0 0 0; C3C 00000000 C48 00000000; CS_IRQ 0000
  0001
  arg 9 (r1 if 3 bits)             -> END     lab2         r2..r5,r7 0 0 0 0 0; C3C 00000000 C48 00000000; CS_IRQ 0000
  0001
  imm = lab2, r1 = lab1            -> END     lab1         r2..r5,r7 0 0 0 0 0; C3C 00000000 C48 00000000; CS_IRQ 0000
  0001
  r1 = lab1 + 4 (unaligned)        -> END     nothing      r2..r5,r7 0 0 0 0 0; C3C 00000000 C48 00000000; CS_IRQ 0000
  0001
  r1 = lab1 - 8 (the 0x08 no-op)   -> END     lab1         r2..r5,r7 0 0 0 0 0; C3C 00000000 C48 00000000; CS_IRQ 0000
  0001
```

14.2.52 ctrlhold (run 14)

ID: `D-ctrlhold-14`. Device run 14. Experiment: ctrlhold.

```
start-word bits 1, 3, 16 and 17 (loop of 1000, later writes 2 ms after the start):
  0x3, no later write    -> TIMEOUT  20017 us, r0    0, r6 00000000, PC 0, FMSS_CTRL 00040002, CS_IRQ 00000000, PCs se
  en 0
  0x3, later 0x1         -> END       2411 us, r0    0, r6 600d600d, PC 0, FMSS_CTRL 00000000, CS_IRQ 00000001, PCs se
  en 0 0 28 30 30 30 28 30 30 30 30 28
  0x3, later 0xfff5      -> END       2411 us, r0    0, r6 600d600d, PC 0, FMSS_CTRL 0000fff4, CS_IRQ 00000001, PCs se
  en 0 0 28 28 30 30 28 28 28 28 28 30
  0x9, no later write    -> TIMEOUT  20015 us, r0    0, r6 00000000, PC 0, FMSS_CTRL 00040000, CS_IRQ 00000000, PCs se
  en 0
  0x9, later 0x9         -> TIMEOUT  20015 us, r0    0, r6 00000000, PC 0, FMSS_CTRL 00040000, CS_IRQ 00000000, PCs se
  en 0
  0x9, later 0x9 x5      -> TIMEOUT  20015 us, r0    0, r6 00000000, PC 0, FMSS_CTRL 00040000, CS_IRQ 00000000, PCs se
  en 0
  0x9, later 0x8 x5      -> TIMEOUT  20015 us, r0    0, r6 00000000, PC 0, FMSS_CTRL 00040000, CS_IRQ 00000000, PCs se
  en 0
```

```
  0x9, later 0x1         -> END       2417 us, r0    0, r6 600d600d, PC 0, FMSS_CTRL 00000000, CS_IRQ 00000001, PCs se
  en 0 0 30 28 28 30 30 30 28 30 30 30
  0x10001, later 0x1     -> END       2411 us, r0    0, r6 600d600d, PC 0, FMSS_CTRL 00000000, CS_IRQ 00000001, PCs se
  en 0 0 28 28 30 28 28 30 30 30 30 28
  0x3, later 0x2 x5      -> TIMEOUT  20015 us, r0    0, r6 00000000, PC 0, FMSS_CTRL 00040002, CS_IRQ 00000000, PCs se
  en 0
  END-only, r2..r5 preloaded, start 0xfff5  -> END     r2..r5 22222222 33333333 44444444 55555555
  END-only, r2..r5 preloaded, start 0x20001 -> END     r2..r5 22222222 33333333 44444444 55555555
  END-only, r2..r5 preloaded, start 0x20000 -> TIMEOUT r2..r5 22222222 33333333 44444444 55555555
```

14.2.53 evclear (run 14)

ID: `D-evclear-14`. Device run 14. Experiment: evclear.

```
pending events and FMC_STATUS reads:
  status latch, wait 5                     -> END         17 us (passed), FMC_STATUS read before 00000000, after 008c3
  000
  status latch, ld FMC_STATUS, wait 5      -> END         17 us (passed), FMC_STATUS read before 008c3000, after 008c3
  000
  status latch, ld FMC_NSTAT, wait 5       -> END         17 us (passed), FMC_STATUS read before 000000c0, after 008c3
  000
  status latch, ld FMC_STATUS x2, wait 5   -> END         17 us (passed), FMC_STATUS read before 008c3000, after 008c3
  000
  idle, wait 3                             -> FAULT     4891 us (timed out), FMC_STATUS read before 00000000, after 00
  0c3000
  idle, ld FMC_STATUS, wait 3              -> FAULT     4870 us (timed out), FMC_STATUS read before 000c3000, after 00
  0c3000
  70h, ld FMC_STATUS, wait 0               -> END         17 us (passed), FMC_STATUS read before 000c3002, after 000c3
  002
  idle, ld FMC_STATUS, wait 11             -> END         17 us (passed), FMC_STATUS read before 000c3000, after 000c3
  000
```

14.2.54 timeregs (run 14)

ID: `D-timeregs-14`. Device run 14. Experiment: timeregs.

```
WAIT 2 (times out) with each word set by the CPU before the start:
  control          -> FAULT     4878 us, first fault at   4862 us, r6 600d600d, after: C10 a5 C3C 00000000 C48 0000000
  0 C54 5a5
  C54 = 0x0        -> FAULT     4864 us, first fault at   4848 us, r6 600d600d, after: C10 a5 C3C 00000000 C48 0000000
  0 C54 000
  C54 = 0x1        -> FAULT     4877 us, first fault at   4862 us, r6 600d600d, after: C10 a5 C3C 00000000 C48 0000000
  0 C54 001
  C54 = 0x10       -> FAULT     4878 us, first fault at   4862 us, r6 600d600d, after: C10 a5 C3C 00000000 C48 0000000
  0 C54 010
  C54 = 0x100      -> FAULT     4870 us, first fault at   4854 us, r6 600d600d, after: C10 a5 C3C 00000000 C48 0000000
  0 C54 100
  C54 = 0x5a5      -> FAULT     4878 us, first fault at   4862 us, r6 600d600d, after: C10 a5 C3C 00000000 C48 0000000
  0 C54 5a5
  C54 = 0xfff      -> FAULT     4862 us, first fault at   4848 us, r6 600d600d, after: C10 a5 C3C 00000000 C48 0000000
  0 C54 fff
  C10 = 0x0        -> FAULT     4878 us, first fault at   4862 us, r6 600d600d, after: C10 00 C3C 00000000 C48 0000000
  0 C54 fff
  C10 = 0x1        -> FAULT     4870 us, first fault at   4854 us, r6 600d600d, after: C10 01 C3C 00000000 C48 0000000
  0 C54 fff
  C10 = 0x80       -> FAULT     4864 us, first fault at   4848 us, r6 600d600d, after: C10 80 C3C 00000000 C48 0000000
  0 C54 fff
  C10 = 0xff       -> FAULT     4891 us, first fault at   4876 us, r6 600d600d, after: C10 ff C3C 00000000 C48 0000000
  0 C54 fff
  C48 = 0x0        -> FAULT     4891 us, first fault at   4876 us, r6 600d600d, after: C10 ff C3C 00000000 C48 0000000
  0 C54 fff
  C48 = 0xffffffff -> FAULT     4884 us, first fault at   4868 us, r6 600d600d, after: C10 ff C3C 00000000 C48 fffffff
  f C54 fff
  C3C = 0x0        -> FAULT     4877 us, first fault at   4862 us, r6 600d600d, after: C10 ff C3C 00000000 C48 fffffff
  f C54 fff
  C3C = 0xffffffff -> FAULT     4878 us, first fault at   4862 us, r6 600d600d, after: C10 ff C3C 00000000 C48 fffffff
  f C54 fff
```

14.2.55 ev5rule (run 15)

ID: `D-ev5rule-15`. Device run 15. Experiment: ev5rule.

```
what arms event 5 (the last WAIT 5 of each program is the question):
  70h, wait 0, clear b1, latch (control)   -> last WAIT 5 passed        20 us, FMC_STATUS 008c3000, status byte c0, CS
  _IRQ 00000031
  70h, wait 0, latch (b1 still set)        -> last WAIT 5 passed        19 us, FMC_STATUS 008c3002, status byte c0, CS
  _IRQ 00000031
  70h, latch, clear b1                     -> last WAIT 5 passed        19 us, FMC_STATUS 008c3000, status byte c0, CS
  _IRQ 00000031
  70h, wait 0, latch, clear b1             -> last WAIT 5 passed        20 us, FMC_STATUS 008c3000, status byte c0, CS
  _IRQ 00000031
  control, then clear b23, latch again     -> last WAIT 5 TIMED OUT   4873 us, FMC_STATUS 000c3000, status byte c0, CS
  _IRQ 00000035
  control, then latch again (b23 kept)     -> last WAIT 5 passed        20 us, FMC_STATUS 008c3000, status byte c0, CS
  _IRQ 00000031
  control, then a new 70h cycle            -> last WAIT 5 TIMED OUT   9752 us, FMC_STATUS 040c3000, status byte c0, CS
```

```
  _IRQ 00000035
  control, but 0x90 instead of 70h         -> last WAIT 5 TIMED OUT   4873 us, FMC_STATUS 000c3000, status byte 90, CS
  _IRQ 00000035
```

14.2.56 c08 (run 15)

ID: `D-c08-15`. Device run 15. Experiment: c08.

```
C08 1 ms after the start:
  running (loop)           -> TIMEOUT C08 1, PC 28, FMSS_CTRL 0004fff4, CS_IRQ 00000020, C14 ff
  waiting (wait 2)         -> TIMEOUT C08 4, PC 18, FMSS_CTRL 0004fff4, CS_IRQ 00000020, C14 c0
  stalled (opcode 0x09)    -> TIMEOUT C08 4, PC 18, FMSS_CTRL 0004fff4, CS_IRQ 00000020, C14 c0
  timer (0x15 from 0xff)   -> END     C08 0, PC 0, FMSS_CTRL 0000fff4, CS_IRQ 00000021, C14 ff
  blocked (0x12)           -> TIMEOUT C08 a, PC 18, FMSS_CTRL 0004fff4, CS_IRQ 00000028, C14 ff
  paused (start bit 1)     -> TIMEOUT C08 3, PC 0, FMSS_CTRL 00040002, CS_IRQ 00000020, C14 ff
  held (start bit 3)       -> TIMEOUT C08 0, PC 0, FMSS_CTRL 00040000, CS_IRQ 00000020, C14 ff
```

14.2.57 chan1 (run 15)

ID: `D-chan1-15`. Device run 15. Experiment: chan1.

```
READ ID (90h 00h, 8 bytes) through each FMC register set:
  set 0x000, CE0 (control) -> END        20 us, 0 timeouts, ID b614d5ec 14d5ec74, status 0a041000, CS_IRQ 00000031
  set 0x000, CE1       -> END        19 us, 0 timeouts, ID b614d5ec 14d5ec74, status 08041000, CS_IRQ 00000031
  set 0x000, CE2       -> END        20 us, 0 timeouts, ID b614d5ec 14d5ec74, status 08041000, CS_IRQ 00000031
  set 0x000, CE3       -> END        19 us, 0 timeouts, ID b614d5ec 14d5ec74, status 08041000, CS_IRQ 00000031
  set 0x400, CE0       -> FAULT   14633 us, 3 timeouts, ID 4a01faa1 4a01faa1, status 0e041000, CS_IRQ 00000035
  set 0x400, CE1       -> FAULT   14626 us, 3 timeouts, ID 679a704e 679a704e, status 0e041000, CS_IRQ 00000035
  set 0x400, CE2       -> FAULT   14625 us, 3 timeouts, ID 01010101 01010101, status 0c041000, CS_IRQ 00000035
  set 0x400, CE3       -> FAULT   14624 us, 3 timeouts, ID 01010101 01010101, status 0c041000, CS_IRQ 00000035
  set 0x400, CE4       -> FAULT   14632 us, 3 timeouts, ID 01010101 01010101, status 0c041000, CS_IRQ 00000035
  set 0x400, CE5       -> FAULT   14641 us, 3 timeouts, ID 01010101 01010101, status 0c041000, CS_IRQ 00000035
  set 0x400, CE6       -> FAULT   14625 us, 3 timeouts, ID 01010101 01010101, status 0c041000, CS_IRQ 00000035
  set 0x400, CE7       -> FAULT   14630 us, 3 timeouts, ID 01010101 01010101, status 0c041000, CS_IRQ 00000035
```

14.2.58 readev (run 15)

ID: `D-readev-15`. Device run 15. Experiment: readev.

```
page read, wait N after each step (us beyond the program without the wait; T = timed out; the step's own wait is not d
  one yet):
  address sent               20 us, FMC_STATUS 000c3000 ECC_DONE 10 | 0:T 1:0 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:0
  30h sent (tR running)      19 us, FMC_STATUS 000c3002 ECC_DONE 10 | 0:0 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:1
  ready, data mode           55 us, FMC_STATUS 000c3000 ECC_DONE 10 | 0:T 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:0
  parity read started        55 us, FMC_STATUS 00041000 ECC_DONE 10 | 0:T 1:T 2:T 3:0 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:1
  data read started          56 us, FMC_STATUS 00041000 ECC_DONE 10 | 0:T 1:T 2:T 3:49 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11
  :0 12:T 13:T 14:T 15:0
  ECC decode started        106 us, FMC_STATUS 0203c000 ECC_DONE 0 | 0:T 1:T 2:T 3:T 4:0 5:T 6:T 7:T 8:T 9:T 10:T 11:0
   12:T 13:T 14:T 15:0
  DMA started               106 us, FMC_STATUS 0203c000 ECC_DONE 74 | 0:T 1:T 2:T 3:T 4:0 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:0
  all done                 4989 us, FMC_STATUS 0603c000 ECC_DONE 74 | 0:T 1:T 2:T 3:T 4:0 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:0
```

14.2.59 regnum (run 15)

ID: `D-regnum-15`. Device run 15. Experiment: regnum.

```
register numbers above 7 (r0..r7 seeded 0x10101010 * (n + 1)):
  mov r1, r9                     -> END    r6 600d600d; r1=10101010; d70 00000000
  mov r1, r15                    -> END    r6 600d600d; r1=10101010; d70 00000000
  mov r1, r255                   -> END    r6 600d600d; r1=10101010; d70 00000000
  add r1, r9, #1                 -> END    r6 600d600d; r1=10101011; d70 00000000
  st [0xd70], r9                 -> END    r6 600d600d; no register; d70 00000000
  st [0xd70], r14                -> END    r6 600d600d; no register; d70 00000000
  movi r8, #0x5eed0008           -> END    r6 00000000; no register; d70 00000000
  movi r9, #0x5eed0009           -> END    r6 600d600d; no register; d70 00000000
  movi r10, #0x5eed000a          -> END    r6 600d600d; r0=5eed000a r1=5eed000a r2=5eed000a r3=5eed000a r4=5eed000a r5
  =5eed000a r7=5eed000a; d70 00000000
  movi r11, #0x5eed000b          -> END    r6 600d600d; no register; d70 00000000
  movi r12, #0x5eed000c          -> END    r6 600d600d; no register; d70 00000000
  movi r13, #0x5eed000d          -> END    r6 600d600d; no register; d70 00000000
  movi r14, #0x5eed000e          -> END    r6 600d600d; no register; d70 00000000
  movi r15, #0x5eed000f          -> END    r6 600d600d; no register; d70 00000000
  movi r16, #0x5eed0010          -> END    r6 600d600d; no register; d70 00000000
  movi r24, #0x5eed0018          -> END    r6 600d600d; no register; d70 00000000
  movi r255, #0x5eed00ff         -> END    r6 600d600d; no register; d70 00000000
  ldi r1, [r9] (r0 = scratch)    -> END    r6 600d600d; r1=e59d000c; d70 00000000
  bnz r9 taken? (r0 != 0)        -> END    r6 bad0bad0; no register; d70 00000000
```

14.2.60 branch (run 15)

ID: D-branch-15. Device run 15. Experiment: branch.

```
branches:
  bnz r1=1 -> lab (control)    -> END    took the branch, CS_IRQ 00000071
  bnz r1=1 -> lab + 4          -> END    neither (r6 0), CS_IRQ 00000071
  bnz r1=1 -> lab - 8          -> END    took the branch, CS_IRQ 00000071
  bnz r1=1, arg 5 -> lab       -> END    took the branch, CS_IRQ 00000071
  bnz r1=0x80000000 -> lab     -> END    took the branch, CS_IRQ 00000071
  bz r1=0, arg 3 -> lab        -> END    took the branch, CS_IRQ 00000071
  bz r1=1 (not taken)          -> END    fell through, CS_IRQ 00000071
```

14.2.61 waitf (run 15)

ID: D-waitf-15. Device run 15. Experiment: waitf.

```
WAIT with its other fields set:
  wait 11 (control)                          -> END    passed after    19 us
  wait 3, arg 0x048, imm 0x3000 (bits set)   -> FAULT  TIMED OUT after  4869 us
  wait 3, arg 0x048, imm 0x1 (bit clear)     -> FAULT  TIMED OUT after  4872 us
  wait 3, arg 0xd70, imm ~0 (word set)       -> FAULT  TIMED OUT after  4872 us
  wait 3, imm ~0                             -> FAULT  TIMED OUT after  4888 us
  wait 11, arg 0xd74, imm ~0 (word 0)        -> END    passed after    19 us
  wait 27 (11 + 16)                          -> FAULT  TIMED OUT after  4888 us
  wait 16 (0 + 16)                           -> FAULT  TIMED OUT after  4872 us
  wait 0x8b (11 + 128)                       -> FAULT  TIMED OUT after  4888 us
  wait 255                                   -> FAULT  TIMED OUT after  4888 us
```

14.2.62 opcost (run 15)

ID: D-opcost-15. Device run 15. Experiment: opcost.

```
100 iterations of 10 instructions + 2 loop instructions:
  SDRAM movi              -> END      260 us =  217 ns per instruction (  +0 ns vs movi)
  SDRAM mov               -> END      260 us =  217 ns per instruction (  +0 ns vs movi)
  SDRAM add imm           -> END      259 us =  216 ns per instruction (  -1 ns vs movi)
  SDRAM 08 no-op          -> END      264 us =  220 ns per instruction (  +4 ns vs movi)
  SDRAM ldi (RAM)         -> END      430 us =  358 ns per instruction (+170 ns vs movi)
  SDRAM sti (RAM)         -> END      446 us =  372 ns per instruction (+186 ns vs movi)
  SDRAM wr pram           -> END      298 us =  248 ns per instruction ( +38 ns vs movi)
  SDRAM st pram           -> END      299 us =  249 ns per instruction ( +39 ns vs movi)
  SDRAM ld pram           -> END      297 us =  248 ns per instruction ( +37 ns vs movi)
  SDRAM ld FMC_STATUS     -> END      297 us =  248 ns per instruction ( +37 ns vs movi)
  SDRAM wr FMC_ADDR0      -> END      297 us =  248 ns per instruction ( +37 ns vs movi)
  SDRAM pld pram          -> END      297 us =  248 ns per instruction ( +37 ns vs movi)
  SDRAM pst pram          -> END      298 us =  248 ns per instruction ( +38 ns vs movi)
  SDRAM bnz not taken     -> END      259 us =  216 ns per instruction (  -1 ns vs movi)
  SDRAM wait 11 (passes)  -> END      264 us =  220 ns per instruction (  +4 ns vs movi)
  SRAM  movi              -> END      165 us =  138 ns per instruction (  +0 ns vs movi)
  SRAM  mov               -> END      165 us =  138 ns per instruction (  +0 ns vs movi)
  SRAM  add imm           -> END      165 us =  138 ns per instruction (  +0 ns vs movi)
  SRAM  08 no-op          -> END      166 us =  138 ns per instruction (  +1 ns vs movi)
  SRAM  ldi (RAM)         -> END      383 us =  319 ns per instruction (+218 ns vs movi)
  SRAM  sti (RAM)         -> END      285 us =  238 ns per instruction (+120 ns vs movi)
  SRAM  wr pram           -> END      200 us =  167 ns per instruction ( +35 ns vs movi)
  SRAM  st pram           -> END      200 us =  167 ns per instruction ( +35 ns vs movi)
  SRAM  ld pram           -> END      199 us =  166 ns per instruction ( +34 ns vs movi)
  SRAM  ld FMC_STATUS     -> END      200 us =  167 ns per instruction ( +35 ns vs movi)
  SRAM  wr FMC_ADDR0      -> END      199 us =  166 ns per instruction ( +34 ns vs movi)
  SRAM  pld pram          -> END      200 us =  167 ns per instruction ( +35 ns vs movi)
  SRAM  pst pram          -> END      200 us =  167 ns per instruction ( +35 ns vs movi)
  SRAM  bnz not taken     -> END      165 us =  138 ns per instruction (  +0 ns vs movi)
  SRAM  wait 11 (passes)  -> END      166 us =  138 ns per instruction (  +1 ns vs movi)
```

14.2.63 op12rel (run 15)

ID: D-op12rel-15. Device run 15. Experiment: op12rel.

```
op 0x12 and one CPU write while it blocks:
  none (control)             -> blocked   TIMEOUT after  20019 us, CS_IRQ 00000078, C08 a
  C48 = 1                    -> blocked   TIMEOUT after  20026 us, CS_IRQ 00000078, C08 a
  C54 = 1                    -> blocked   TIMEOUT after  20022 us, CS_IRQ 00000078, C08 a
  C3C = 1                    -> blocked   TIMEOUT after  20029 us, CS_IRQ 00000078, C08 a
  C5C = 1                    -> blocked   TIMEOUT after  20029 us, CS_IRQ 00000078, C08 a
  C58 = 0xff                 -> blocked   TIMEOUT after  20027 us, CS_IRQ 00000078, C08 a
  C4C = 0x40                 -> blocked   TIMEOUT after  20021 us, CS_IRQ 00000078, C08 a
  C10 = 0                    -> blocked   TIMEOUT after  20022 us, CS_IRQ 00000078, C08 a
  C10 = 0xff                 -> blocked   TIMEOUT after  20029 us, CS_IRQ 00000078, C08 a
  FMSS_CTRL = 0x1            -> blocked   TIMEOUT after  20028 us, CS_IRQ 00000078, C08 a
  FMSS_CTRL = 0x3            -> blocked   TIMEOUT after  20017 us, CS_IRQ 00000078, C08 3
  FMSS_CTRL = 0x9            -> blocked   TIMEOUT after  20025 us, CS_IRQ 00000078, C08 0
  FMSS_CTRL = 0x4            -> blocked   TIMEOUT after  20029 us, CS_IRQ 00000078, C08 a
  FMSS_CTRL = 0x20001        -> blocked   TIMEOUT after  20030 us, CS_IRQ 00000078, C08 a
  CPU sends 70h (FMC_CMD)    -> blocked   TIMEOUT after  20020 us, CS_IRQ 00000078, C08 a
  FMC_STATUS = ~0            -> blocked   TIMEOUT after  20020 us, CS_IRQ 00000078, C08 a
  pram 0xd00 = 1             -> blocked   TIMEOUT after  20029 us, CS_IRQ 00000068, C08 a
  r0 = 1                     -> blocked   TIMEOUT after  20030 us, CS_IRQ 00000068, C08 a
```

```
  C14 = 0                   -> blocked   TIMEOUT after  20020 us, CS_IRQ 00000068, C08 a
```

14.2.64 pause (run 15)

ID: `D-pause-15`. Device run 15. Experiment: pause.

```
a 20 000-iteration loop with start-word writes while it runs:
  no writes (control, ~8.6 ms)     -> END       8222 us, r0     0, C08 0, FMSS_CTRL 0000fff4, r0 sampled 19996 17516 1
  5023 12542 10063 7579 4996 2513 36
  0xfff7 at 1 ms                   -> TIMEOUT  30016 us, r0 16539, C08 3, FMSS_CTRL 0004fff6, r0 sampled 19995 19702 1
  9408 19115 18820 18525 18234 17945
  0xfff7 at 1 ms, 0xfff5 at 5 ms   -> END      12207 us, r0     0, C08 0, FMSS_CTRL 0000fff4, r0 sampled 19995 17558 1
  4957 12463 9966 7461 4956 2461
  0xfffd at 1 ms, 0xfff5 at 5 ms   -> END      13199 us, r0     0, C08 0, FMSS_CTRL 0000fff4, r0 sampled 19995 19796 1
  6928 14148 11381 8611 5843 3069 144
  0x10001 at 1 ms, 0x1 at 5 ms     -> END      13207 us, r0     0, C08 0, FMSS_CTRL 00000000, r0 sampled 19994 19762 1
  6863 14051 11248 8443 5642 2837 27
  0xfff5 at 1 ms (restart?)        -> END       8235 us, r0     0, C08 0, FMSS_CTRL 0000fff4, r0 sampled 19995 17553 1
  5041 12541 10049 7549 4950 2455
  0xfff4 at 1 ms                   -> END       8220 us, r0     0, C08 0, FMSS_CTRL 0000fff4, r0 sampled 19995 17588 1
  5108 12642 10178 7709 5144 2678 216
```

14.2.65 fmc28 (run 15)

ID: `D-fmc28-15`. Device run 15. Experiment: fmc28.

```
FMC_28 across a run (before = after the CPU set-up):
  empty          -> FMC_28 19c -> 018: +124 (+0 vs empty)
  movi           -> FMC_28 094 -> 110: +124 (+0 vs empty)
  pld pram       -> FMC_28 18c -> 008: +124 (+0 vs empty)
  pst pram       -> FMC_28 084 -> 100: +124 (+0 vs empty)
  sti RAM        -> FMC_28 17c -> 1f8: +124 (+0 vs empty)
  ldi RAM        -> FMC_28 074 -> 0f0: +124 (+0 vs empty)
  wr pram        -> FMC_28 16c -> 1e8: +124 (+0 vs empty)
  ld pram        -> FMC_28 064 -> 0e0: +124 (+0 vs empty)
  ld FMC_STATUS  -> FMC_28 15c -> 1d8: +124 (+0 vs empty)
  wr FMC_ADDR0   -> FMC_28 054 -> 0d0: +124 (+0 vs empty)
  ld FMC_28      -> FMC_28 14c -> 1c8: +124 (+0 vs empty)
```

14.2.66 blk100 (run 15)

ID: `D-blk100-15`. Device run 15. Experiment: blk100.

```
END; words that are not 0 in any read:
  0x1b8: value 00000000, after ~0 00000001 (0x3b8: 00000001), after 0 00000000  writable 00000001
  0x1bc: value ffffffff, after ~0 ffffffff (0x3bc: ffffffff), after 0 00000000  writable ffffffff
  0x1c0: value ffffffff, after ~0 ffffffff (0x3c0: ffffffff), after 0 00000000  writable ffffffff
  0x1c4: value ffffffff, after ~0 ffffffff (0x3c4: ffffffff), after 0 00000000  writable ffffffff
  0x1c8: value ffffffff, after ~0 ffffffff (0x3c8: ffffffff), after 0 00000000  writable ffffffff
  0x1cc: value 00008060, after ~0 00ffffff (0x3cc: 00ffffff), after 0 00000000  writable 00ffffff
  0x1d0: value 003010d0, after ~0 00ffffff (0x3d0: 00ffffff), after 0 00000000  writable 00ffffff
  0x1d4: value ffffffff, after ~0 ffffffff (0x3d4: ffffffff), after 0 00000000  writable ffffffff
  0x1d8: value ffffffff, after ~0 ffffffff (0x3d8: ffffffff), after 0 00000000  writable ffffffff
```

14.2.67 irqline (run 15)

ID: `D-irqline-15`. Device run 15. Experiment: irqline.

```
VIC1 raw bit 22 while polling and at the end:
  END, start 0xfff5                -> END     CS_IRQ 00000061, line seen 1, line at the end 1, preflight line -
  END, start 0xffb5 (no bit 6)     -> END     CS_IRQ 00000061, line seen 1, line at the end 1, preflight line -
  END, start 0x41                  -> END     CS_IRQ 00000061, line seen 1, line at the end 1, preflight line -
  END, start 0x1                   -> END     CS_IRQ 00000061, line seen 1, line at the end 1, preflight line -
  0x10 then block, 0xfff5          -> TIMEOUT CS_IRQ 0000006a, line seen 1, line at the end 1, preflight line -
  0x10 then block, 0xffb5          -> TIMEOUT CS_IRQ 0000006a, line seen 1, line at the end 1, preflight line -
  block only, 0xfff5               -> TIMEOUT CS_IRQ 00000068, line seen 1, line at the end 1, preflight line -
  timeout (C58 0) then block, 0xfff5   -> FAULT   CS_IRQ 0000006c, line seen 1, line at the end 1, preflight line -
  timeout (C58 0) then block, 0xffb5   -> FAULT   CS_IRQ 0000006c, line seen 1, line at the end 1, preflight line -
```

14.2.68 restore (run 16)

ID: `D-restore-16`. Device run 16. Experiment: restore.

The experiment changed between runs.

```
restored 45 words; not as written afterwards: 0x428=00000180
  changed by the restore: 0x00c:00000000->0005dc00 0x010:00000005->00000000 0x014:00000100->00000001 0x018:a5a5a5a5->0
  0000000 0x020:00a5a5a5->007f167f 0x02c:00000004->00000002 0x030:000001ff->00000000 0x034:08053e40->00000000 0x03c:08
  053e40->00000000 0x040:08053e40->00000000 0x044:0000a5a5->00000000 0x050:00003901->00000901 0x054:00000004->00000000
   0x058:00000004->00000000 0x05c:00000004->00000000 0x060:a5a5a5a5->00000000 0x064:a5a5a5a5->00000000 0x068:a5a5a5a5-
  >00000000 0x06c:a5a5a5a5->00000000 0x070:a5a5a5a5->00000000 0x074:a5a5a5a5->00000000 0x078:a5a5a5a5->00000000 0x404:
  00000300->00000200 0x410:a5a5a5a5->00000000 0x414:00000100->00000000 0x418:a5a5a5a5->00000000 0x420:00a5a5a5->000000
  00 0x428:00000108->00000180 0x430:00000007->00000000 0x434:a5a5a5a5->00000000 0x438:00000005->00000000 0x43c:a5a5a5a
  5->00000000 0x440:a5a5a5a5->00000000 0x444:0000a5a5->00000000 0x448:0e041000->0e1c3000 0x450:00000000->00000800 0x45
  4:00000004->00000000 0x458:00000004->00000000 0x45c:00000004->00000000 0x478:a5a5a5a5->00000000 0x480:ffffffff->0000
  0000 0x484:ffffffff->00000000 0x488:ffffffff->00000000 0x48c:ffffffff->00000000 0x490:ffffffff->00000000 0x494:fffff
```

```
  fff->00000000 0x498:ffffffff->00000000 0x49c:ffffffff->00000000 0x4a0:ffffffff->00000000 0x4a4:ffffffff->00000000 0x
  4a8:ffffffff->00000000 0x4ac:ffffffff->00000000 0x4b0:ffffffff->00000000 0x4b4:ffffffff->00000000 0x4b8:ffffffff->00
  000000 0x4bc:ffffffff->00000000 0x4c0:ffffffff->00000000 0x4c4:ffffffff->00000000 0x4c8:ffffffff->00000000 0x4cc:fff
  fffff->00000000 0x4d0:ffffffff->00000000 0x4d4:ffffffff->00000000 0x4d8:ffffffff->00000000 0x4dc:ffffffff->00000000
  0x4e0:ffffffff->00000000 0x4e4:ffffffff->00000000 0x4e8:ffffffff->00000000 0x4ec:ffffffff->00000000 0x4f0:ffffffff->
  00000000 0x4f4:ffffffff->00000000 0x4f8:ffffffff->00000000 0x4fc:ffffffff->00000000 0x804:a5a5a5a5->00000000 0x808:a
  5a5a5a5->00000000 0x814:00000180->00e00000 0x840:00000074->00000010 0x844:00000025->00000000 0xc0c:00000061->0000000
  1 0xc10:000000ff->00000000 0xc48:00000001->00000000 0xc50:ffff1810->ffff1900 0xc54:00000001->00000000
  CS_IRQ after an END-only run 00000061 -> 00000001, interrupt line at the end 1 -> 0
```

14.2.69 readev (run 16)

ID: `D-readev-16`. Device run 16. Experiment: readev.

```
page read, wait N after each step (us beyond the program without the wait; T = timed out; the step's own wait is not d
  one yet):
  address sent               20 us, FMC_STATUS 000c3000 ECC_DONE 10 | 0:T 1:0 2:T 3:T 4:T 5:T 6:T 7:T 8:0 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  30h sent (tR running)      20 us, FMC_STATUS 000c3002 ECC_DONE 10 | 0:0 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:0 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  ready, data mode           56 us, FMC_STATUS 000c3000 ECC_DONE 10 | 0:T 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:0 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  parity read started        55 us, FMC_STATUS 00041000 ECC_DONE 10 | 0:T 1:T 2:T 3:0 4:T 5:T 6:T 7:T 8:0 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  data read started          56 us, FMC_STATUS 00041000 ECC_DONE 10 | 0:T 1:T 2:T 3:49 4:T 5:T 6:T 7:T 8:0 9:T 10:T 11
  :0 12:T 13:T 14:T 15:T
  ECC decode started        104 us, FMC_STATUS 0003c000 ECC_DONE 0 | 0:T 1:T 2:T 3:T 4:1 5:T 6:T 7:T 8:1 9:T 10:T 11:1
   12:T 13:T 14:T 15:T
  DMA started               105 us, FMC_STATUS 0003c000 ECC_DONE 74 | 0:T 1:T 2:T 3:T 4:0 5:T 6:T 7:T 8:0 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  all done                 4963 us, FMC_STATUS 0003c000 ECC_DONE 74 | 0:T 1:T 2:T 3:T 4:15 5:T 6:T 7:T 8:9 9:T 10:T 11
  :0 12:T 13:T 14:T 15:T
```

14.2.70 irqline (run 16)

ID: `D-irqline-16`. Device run 16. Experiment: irqline.

```
VIC1 raw bit 22 while polling and at the end:
  END, start 0xfff5                    -> END     CS_IRQ 00000001, line seen 0, line at the end 0, preflight line -
  END, start 0xffb5 (no bit 6)         -> END     CS_IRQ 00000001, line seen 0, line at the end 0, preflight line -
  END, start 0x41                      -> END     CS_IRQ 00000001, line seen 0, line at the end 0, preflight line -
  END, start 0x1                       -> END     CS_IRQ 00000001, line seen 0, line at the end 0, preflight line -
  0x10 then block, 0xfff5              -> TIMEOUT CS_IRQ 0000000a, line seen 0, line at the end 0, preflight line -
  0x10 then block, 0xffb5              -> TIMEOUT CS_IRQ 0000000a, line seen 0, line at the end 0, preflight line -
  block only, 0xfff5                   -> TIMEOUT CS_IRQ 00000008, line seen 0, line at the end 0, preflight line -
  timeout (C58 0) then block, 0xfff5   -> FAULT   CS_IRQ 0000000c, line seen 0, line at the end 0, preflight line -
  timeout (C58 0) then block, 0xffb5   -> FAULT   CS_IRQ 0000000c, line seen 0, line at the end 0, preflight line -
```

14.2.71 irqen (run 16)

ID: `D-irqen-16`. Device run 16. Experiment: irqen.

```
one enable bit at a time (then 0), after a 70h command (0x044) or an ECC decode (0x844):
  0x044 = 1      -> END    CS_IRQ in program 00000010, after 00000011, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 2      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 4      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 8      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 10     -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 20     -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 40     -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 80     -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 100    -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 200    -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 400    -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 800    -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 1000   -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 2000   -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 4000   -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 8000   -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 0      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
```

```
  54 0
  0x844 = 1      -> END    CS_IRQ in program 00000040, after 00000041, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
  0x844 = 2      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
  0x844 = 4      -> END    CS_IRQ in program 00000040, after 00000041, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
  0x844 = 8      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
  0x844 = 10     -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
  0x844 = 20     -> END    CS_IRQ in program 00000040, after 00000041, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
  0x844 = 0      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
```

14.2.72 ev5t (run 16)

ID: `D-ev5t-16`. Device run 16. Experiment: ev5t.

```
70h + status latch and the last WAIT 5:
  70h, latch, wait 5             -> 0 timeout(s),     19 us, FMC_STATUS after 008c3002
  70h, latch, clear b1, wait 5   -> 0 timeout(s),     20 us, FMC_STATUS after 008c3000
  70h, latch, wait 0, wait 5     -> 0 timeout(s),     20 us, FMC_STATUS after 008c3002
  70h, latch, 20 movi, wait 5    -> 0 timeout(s),     19 us, FMC_STATUS after 008c3002
  70h, latch, wait 15, wait 5    -> 1 timeout(s),   4880 us, FMC_STATUS after 008c3002
  70h, 20 movi, latch, wait 5    -> 0 timeout(s),     19 us, FMC_STATUS after 008c3002
  70h, wait 0, latch, wait 5     -> 0 timeout(s),     19 us, FMC_STATUS after 008c3002
```

14.2.73 r8 (run 16)

ID: `D-r8-16`. Device run 16. Experiment: r8.

```
register numbers 8 / 10 (r0..r7 seeded 0x10101010 * (n + 1)):
  movi r8, #lab                  -> END     LANDED ON lab; r 10101010 20202020 30303030 40404040 50505050 60606060 111
  11111 80808080; PC 0
  movi r8, #0x5eed0008 (repeat)  -> END     ended without either; r 10101010 20202020 30303030 40404040 50505050 60606
  060 00000000 80808080; PC 0
  mov r8, r1 (r1 = lab)          -> END     LANDED ON lab; r 10101010 00000078 30303030 40404040 50505050 60606060 111
  11111 80808080; PC 0
  mov r1, r8 (read r8)           -> END     fell through; r 10101010 00000058 30303030 40404040 50505050 60606060 bad0
  bad0 80808080; PC 0
  movi r10, #0x12345678 (repeat) -> END     fell through; r 12345678 12345678 12345678 12345678 12345678 12345678 bad0
  bad0 12345678; PC 0
  mov r10, r2                    -> END     fell through; r 30303030 30303030 30303030 30303030 30303030 30303030 bad0
  bad0 30303030; PC 0
  add r10, r2, #1                -> END     fell through; r 30303031 30303031 30303031 30303031 30303031 30303031 bad0
  bad0 30303031; PC 0
  movi r26 (10 + 16)             -> END     fell through; r 10101010 20202020 30303030 40404040 50505050 60606060 bad0
  bad0 80808080; PC 0
  movi r24 (8 + 16)              -> END     fell through; r 10101010 20202020 30303030 40404040 50505050 60606060 bad0
  bad0 80808080; PC 0
  ldi r1, [r8]                   -> END     fell through; r 10101010 e321f0d3 30303030 40404040 50505050 60606060 bad0
  bad0 80808080; PC 0
  ldi r1, [r10]                  -> END     fell through; r 10101010 e3520000 30303030 40404040 50505050 60606060 bad0
  bad0 80808080; PC 0
```

14.2.74 state (run 17)

ID: `D-state-17`. Device run 17. Experiment: state.

```
CS_IRQ after an END-only run 00000001, interrupt line 0/0; FMC 0x044 00000000, ECC 0x844 00000000, ECC_DONE 00000010,
  C10 00, 0x404 00000000
```

14.2.75 readev (run 17)

ID: `D-readev-17`. Device run 17. Experiment: readev.

```
page read, wait N after each step (us beyond the program without the wait; T = timed out; the step's own wait is not d
  one yet):
  address sent               20 us, FMC_STATUS 000c3000 ECC_DONE 10 | 0:T 1:0 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  30h sent (tR running)      20 us, FMC_STATUS 000c3002 ECC_DONE 10 | 0:0 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  ready, data mode           55 us, FMC_STATUS 000c3000 ECC_DONE 10 | 0:T 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  parity read started        55 us, FMC_STATUS 00041000 ECC_DONE 10 | 0:T 1:T 2:T 3:0 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  data read started          55 us, FMC_STATUS 00041000 ECC_DONE 10 | 0:T 1:T 2:T 3:49 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11
  :0 12:T 13:T 14:T 15:T
  ECC decode started        105 us, FMC_STATUS 0003c000 ECC_DONE 0 | 0:T 1:T 2:T 3:T 4:0 5:T 6:T 7:T 8:T 9:T 10:T 11:0
   12:T 13:T 14:T 15:T
  DMA started               105 us, FMC_STATUS 0003c000 ECC_DONE 74 | 0:T 1:T 2:T 3:T 4:0 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  all done                 4978 us, FMC_STATUS 0003c000 ECC_DONE 74 | 0:T 1:T 2:T 3:T 4:0 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
```

14.2.76 irqline (run 17)

ID: `D-irqline-17`. Device run 17. Experiment: irqline.

```
VIC1 raw bit 22 while polling and at the end:
  END, start 0xfff5                    -> END     CS_IRQ 00000001, line seen 0, line at the end 0, preflight line -
  END, start 0xffb5 (no bit 6)         -> END     CS_IRQ 00000001, line seen 0, line at the end 0, preflight line -
  END, start 0x41                      -> END     CS_IRQ 00000001, line seen 0, line at the end 0, preflight line -
  END, start 0x1                       -> END     CS_IRQ 00000001, line seen 0, line at the end 0, preflight line -
  0x10 then block, 0xfff5              -> TIMEOUT CS_IRQ 0000000a, line seen 0, line at the end 0, preflight line -
  0x10 then block, 0xffb5              -> TIMEOUT CS_IRQ 0000000a, line seen 0, line at the end 0, preflight line -
  block only, 0xfff5                   -> TIMEOUT CS_IRQ 00000008, line seen 0, line at the end 0, preflight line -
  timeout (C58 0) then block, 0xfff5   -> FAULT   CS_IRQ 0000000c, line seen 0, line at the end 0, preflight line -
  timeout (C58 0) then block, 0xffb5   -> FAULT   CS_IRQ 0000000c, line seen 0, line at the end 0, preflight line -
```

14.2.77 irqen (run 17)

ID: `D-irqen-17`. Device run 17. Experiment: irqen.

```
one enable bit at a time (then 0), after a 70h command (0x044) or an ECC decode (0x844):
  0x044 = 1      -> END    CS_IRQ in program 00000010, after 00000011, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 2      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 4      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 8      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 10     -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 20     -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 40     -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 80     -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 100    -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 200    -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 400    -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 800    -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 1000   -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 2000   -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 4000   -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 8000   -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x044 = 0      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3002, ECC_DONE 74, 0x0
  54 0
  0x844 = 1      -> END    CS_IRQ in program 00000040, after 00000041, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
  0x844 = 2      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
  0x844 = 4      -> END    CS_IRQ in program 00000040, after 00000041, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
  0x844 = 8      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
  0x844 = 10     -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
  0x844 = 20     -> END    CS_IRQ in program 00000040, after 00000041, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
  0x844 = 0      -> END    CS_IRQ in program 00000000, after 00000001, line 0/0, FMC_STATUS 000c3000, ECC_DONE 75, 0x0
  54 0
```

14.2.78 ev5t (run 17)

ID: `D-ev5t-17`. Device run 17. Experiment: ev5t.

```
70h + status latch and the last WAIT 5:
  70h, latch, wait 5             -> 0 timeout(s),     19 us, FMC_STATUS after 008c3002
  70h, latch, clear b1, wait 5   -> 0 timeout(s),     20 us, FMC_STATUS after 008c3000
  70h, latch, wait 0, wait 5     -> 0 timeout(s),     19 us, FMC_STATUS after 008c3002
  70h, latch, 20 movi, wait 5    -> 0 timeout(s),     19 us, FMC_STATUS after 008c3002
  70h, latch, wait 15, wait 5    -> 1 timeout(s),   4880 us, FMC_STATUS after 008c3002
  70h, 20 movi, latch, wait 5    -> 0 timeout(s),     19 us, FMC_STATUS after 008c3002
  70h, wait 0, latch, wait 5     -> 0 timeout(s),     19 us, FMC_STATUS after 008c3002
```

14.2.79 r8 (run 17)

ID: `D-r8-17`. Device run 17. Experiment: r8.

```
register numbers 8 / 10 (r0..r7 seeded 0x10101010 * (n + 1)):
  movi r8, #lab                  -> END     LANDED ON lab; r 10101010 20202020 30303030 40404040 50505050 60606060 111
  11111 80808080; PC 0
  movi r8, #0x5eed0008 (repeat)  -> END     ended without either; r 10101010 20202020 30303030 40404040 50505050 60606
  060 00000000 80808080; PC 0
  mov r8, r1 (r1 = lab)          -> END     LANDED ON lab; r 10101010 00000078 30303030 40404040 50505050 60606060 111
  11111 80808080; PC 0
  mov r1, r8 (read r8)           -> END     fell through; r 10101010 00000058 30303030 40404040 50505050 60606060 bad0
  bad0 80808080; PC 0
  movi r10, #0x12345678 (repeat) -> END     fell through; r 12345678 12345678 12345678 12345678 12345678 12345678 bad0
  bad0 12345678; PC 0
  mov r10, r2                    -> END     fell through; r 30303030 30303030 30303030 30303030 30303030 30303030 bad0
  bad0 30303030; PC 0
  add r10, r2, #1                -> END     fell through; r 30303031 30303031 30303031 30303031 30303031 30303031 bad0
  bad0 30303031; PC 0
  movi r26 (10 + 16)             -> END     fell through; r 10101010 20202020 30303030 40404040 50505050 60606060 bad0
  bad0 80808080; PC 0
  movi r24 (8 + 16)              -> END     fell through; r 10101010 20202020 30303030 40404040 50505050 60606060 bad0
  bad0 80808080; PC 0
  ldi r1, [r8]                   -> END     fell through; r 10101010 e321f0d3 30303030 40404040 50505050 60606060 bad0
  bad0 80808080; PC 0
  ldi r1, [r10]                  -> END     fell through; r 10101010 e59d000c 30303030 40404040 50505050 60606060 bad0
  bad0 80808080; PC 0
```

14.2.80 state (run 18)

ID: `D-state-18`. Device run 18. Experiment: state.

```
events passing in an idle program: 0 4 5 11 (power-on: 11); 17 words differ from the recorded power-on values
  0x008 = 00000070
  0x00c = dc000000
  0x010 = 00000005
  0x014 = 00000001
  0x02c = 00000004
  0x030 = 000001ff
  0x034 = 08053e40
  0x03c = 08053e40
  0x040 = 08053e40
  0x048 = 008c3002
  0x050 = 00002800
  0x450 = 00000800
  0x80c = 00001100
  0x814 = 00000180
  0x840 = 00000075
  0x9a4 = 0000003a
  0xc50 = ffff1831
  CS_IRQ after an END-only run 00000001, interrupt line 0/0; FMC 0x044 00000000, ECC 0x844 00000000, ECC_DONE 00000075
  , C10 00, 0x404 00000000
```

14.2.81 ev8 (run 18)

ID: `D-ev8-18`. Device run 18. Experiment: ev8.

```
events after each step through the second register set (ok = passed, T = timed out):
  set 0x400: 90h             0x448 000c3002 0x450 00000000 0x050 00003000 | 0:ok 1:T 2:T 3:T 4:ok 5:ok 6:ok 7:T 8:T 9:
  T 10:T 11:ok 12:T 13:T 14:T 15:T
  set 0x400: + address       0x448 000c3006 0x450 00000000 0x050 00003800 | 0:ok 1:T 2:T 3:T 4:ok 5:ok 6:ok 7:ok 8:T 9
  :T 10:T 11:ok 12:T 13:T 14:T 15:T
  set 0x400: + ID transfer   0x448 08041006 0x450 00000000 0x050 00003800 | 0:ok 1:T 2:T 3:T 4:ok 5:ok 6:ok 7:ok 8:T 9
  :ok 10:T 11:ok 12:T 13:T 14:T 15:T
  next program, no NAND step 0:ok 1:T 2:T 3:T 4:ok 5:ok 6:T 7:T 8:T 9:T 10:T 11:ok 12:T 13:T 14:T 15:T
```

14.2.82 evcons (run 18)

ID: `D-evcons-18`. Device run 18. Experiment: evcons.

```
which wait of each program passed (ok) or timed out (T):
  70h, latch, wait 15                    -> wait 15:T  (4879 us, FMC_STATUS 008c3002)
  70h, latch, wait 15, wait 5            -> wait 15:T wait 5:ok  (4880 us, FMC_STATUS 008c3002)
  70h, latch, wait 5, wait 15            -> wait 5:ok wait 15:T  (4876 us, FMC_STATUS 008c3002)
  70h, latch, wait 5, wait 5             -> wait 5:ok wait 5:ok  (20 us, FMC_STATUS 008c3002)
  70h, wait 0, wait 0                    -> wait 0:ok wait 0:ok  (19 us, FMC_STATUS 000c3002)
  70h, wait 15, wait 0                   -> wait 15:T wait 0:ok  (4880 us, FMC_STATUS 000c3002)
  70h, wait 0, wait 15                   -> wait 0:ok wait 15:T  (4879 us, FMC_STATUS 000c3002)
  70h, latch, wait 15, wait 15, wait 15  -> wait 15:T wait 15:T wait 15:T  (14625 us, FMC_STATUS 008c3002)
```

14.2.83 restore (run 19)

ID: `D-restore-19`. Device run 19. Experiment: restore.

The experiment changed between runs.

```
differ from power-on before: 58 words, after: 7 words
  0x008 = 00000070 (power-on 00000000)
  0x050 = 00001000 (power-on 00000000)
  0x404 = 00000200 (power-on 00000000)
```

```
  0x408 = 00000090 (power-on 00000000)
  0x450 = 00000800 (power-on 00000000)
  0x80c = 00001100 (power-on 00000000)
  0x9a4 = 0000003a (power-on 00000000)
  CS_IRQ after an END-only run 00000001 -> 00000001, interrupt line 0 -> 0
  events passing in an idle program afterwards: 11 (power-on: 11)
```

14.2.84 dmaloop (run 20)

ID: `D-dmaloop-20`. Device run 20. Experiment: dmaloop.

The experiment changed between runs.

```
RAM -> buffer -> RAM through each register set (ok = the wait passed, T = timed out):
  set 0x000, drain 0x100                    -> fill T, drain T; destination 2/512 bytes match (untouched); status aft
  er fill 000c3000, after drain 000c3000; DMA_ADDR 08055e40 / 00000000
  set 0x000, DMA_CTRL 7, drain 0x1e0        -> fill T, drain T; destination 2/512 bytes match (untouched); status aft
  er fill 000c3000, after drain 000c3000; DMA_ADDR 08055e40 / 00000000
  set 0x400, drain 0x100, wait 8            -> fill T, drain T; destination 2/512 bytes match (untouched); status aft
  er fill 000c3000, after drain 000c3000; DMA_ADDR 08055e40 / 08055e40
  set 0x400, drain 0x100, wait 2            -> fill T, drain T; destination 2/512 bytes match (untouched); status aft
  er fill 000c3000, after drain 000c3000; DMA_ADDR 08055e40 / 08055e40
  set 0x400, DMA_CTRL 7, drain 0x1e0, wait 8 -> fill T, drain T; destination 2/512 bytes match (untouched); status aft
  er fill 000c3000, after drain 000c3000; DMA_ADDR 08055e40 / 08055e40
```

14.2.85 restore (run 21)

ID: `D-restore-21`. Device run 21. Experiment: restore.

The experiment changed between runs.

```
differ from power-on before: 56 words, after: 6 words
  0x008 = 00000070 (power-on 00000000)
  0x050 = 00002000 (power-on 00000000)
  0x408 = 00000090 (power-on 00000000)
  0x450 = 00000800 (power-on 00000000)
  0x80c = 00001100 (power-on 00000000)
  0x9a4 = 0000003a (power-on 00000000)
  CS_IRQ after an END-only run 00000001 -> 00000001, interrupt line 0 -> 0
  events passing in an idle program afterwards: 11 (power-on: 11)
```

14.2.86 readev (run 21)

ID: `D-readev-21`. Device run 21. Experiment: readev.

```
page read, wait N after each step (us beyond the program without the wait; T = timed out; the step's own wait is not d
  one yet):
  address sent               20 us, FMC_STATUS 000c3000 ECC_DONE 10 | 0:T 1:0 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  30h sent (tR running)      19 us, FMC_STATUS 000c3002 ECC_DONE 10 | 0:1 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  ready, data mode           55 us, FMC_STATUS 000c3000 ECC_DONE 10 | 0:T 1:T 2:T 3:T 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
  parity read started        55 us, FMC_STATUS 00041000 ECC_DONE 10 | 0:T 1:T 2:T 3:0 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11:
  1 12:T 13:T 14:T 15:T
  data read started          55 us, FMC_STATUS 00041000 ECC_DONE 10 | 0:T 1:T 2:T 3:49 4:T 5:T 6:T 7:T 8:T 9:T 10:T 11
  :0 12:T 13:T 14:T 15:T
  ECC decode started        104 us, FMC_STATUS 0003c000 ECC_DONE 0 | 0:T 1:T 2:T 3:T 4:0 5:T 6:T 7:T 8:T 9:T 10:T 11:0
   12:T 13:T 14:T 15:T
  DMA started               109 us, FMC_STATUS 0003c000 ECC_DONE 74 | 0:T 1:T 2:14 3:T 4:0 5:T 6:T 7:T 8:T 9:T 10:T 11
  :0 12:T 13:T 14:T 15:T
  all done                  123 us, FMC_STATUS 0003c000 ECC_DONE 74 | 0:T 1:T 2:T 3:T 4:0 5:T 6:T 7:T 8:T 9:T 10:T 11:
  0 12:T 13:T 14:T 15:T
```

14.2.87 dmaloop (run 21)

ID: `D-dmaloop-21`. Device run 21. Experiment: dmaloop.

The experiment changed between runs.

```
RAM -> buffer -> RAM through each register set (ok = the wait passed, T = timed out):
  set 0x000, CTRL0 0x43801, drain 0x100      -> fill T, drain T; destination 2/512 bytes match (untouched); status aft
  er fill 000c3000, after drain 000c3000; DMA_ADDR 08055e40 / 00000000
  set 0x000, CTRL0 0x43801, DMA_CTRL 7, drain 0x1e0 -> fill T, drain T; destination 2/512 bytes match (untouched); sta
  tus after fill 000c3000, after drain 000c3000; DMA_ADDR 08055e40 / 00000000
  set 0x400, CTRL0 0x43801, drain 0x100, wait 8 -> fill T, drain T; destination 2/512 bytes match (untouched); status
  after fill 000c3000, after drain 000c3000; DMA_ADDR 08055e40 / 08055e40
  set 0x400, CTRL0 0x43801, drain 0x100, wait 2 -> fill T, drain T; destination 2/512 bytes match (untouched); status
  after fill 000c3000, after drain 000c3000; DMA_ADDR 08055e40 / 08055e40
  set 0x000, CTRL0 + b24, drain 0x100        -> fill ok, drain ok; destination 512/512 bytes match; status after fill
  021c3000, after drain 021c3000; DMA_ADDR 08055e40 / 08055e40
  set 0x000, CTRL0 + b24, DMA_CTRL 7, drain 0x1e0 -> fill ok, drain ok; destination 512/512 bytes match; status after
  fill 001c3000, after drain 001c3000; DMA_ADDR 08055e40 / 08055e40
  set 0x400, CTRL0 + b24, drain 0x100, wait 8 -> fill ok, drain ok; destination 512/512 bytes match; status after fill
   001c3000, after drain 001c3000; DMA_ADDR 08055e40 / 08055e40
  set 0x400, CTRL0 + b24, drain 0x100, wait 2 -> fill T, drain T; destination 512/512 bytes match; status after fill 0
```

```
  01c3000, after drain 001c3000; DMA_ADDR 08055e40 / 08055e40
```

14.2.88 op0916 (run 22)

ID: `D-op0916-22`. Device run 22. Experiment: op0916.

```
each form: T = timed out and faulted, otherwise the run time (a pass, or a delay shorter than the timeout):
  op 09: [0x048] & 0x1000 (bit set) T | [0x048] == 0xc3000 (all set bits) T | [0x048] & 0x2 (bit clear) T | [0xd70] ==
   0x12345678 T | [0xd70] & 0x8 (bit set) T | [0x800] == 0x20060808 T | r1 = ~0, imm 1 T | r1 = 0, imm 0 (until zero?)
   T | imm 1 (a short delay?) T | imm 0x100 T | event field 11, imm 0 T
  op 16: [0x048] & 0x1000 (bit set) 20us | [0x048] == 0xc3000 (all set bits) 19us | [0x048] & 0x2 (bit clear) 19us | [
  0xd70] == 0x12345678 20us | [0xd70] & 0x8 (bit set) 19us | [0x800] == 0x20060808 19us | r1 = ~0, imm 1 19us | r1 = 0
  , imm 0 (until zero?) 19us | imm 1 (a short delay?) 19us | imm 0x100 19us | event field 11, imm 0 20us
  op 07: [0x048] & 0x1000 (bit set) T | [0x048] == 0xc3000 (all set bits) T | [0x048] & 0x2 (bit clear) T | [0xd70] ==
   0x12345678 T | [0xd70] & 0x8 (bit set) T | [0x800] == 0x20060808 T | r1 = ~0, imm 1 T | r1 = 0, imm 0 (until zero?)
   T | imm 1 (a short delay?) T | imm 0x100 T | event field 11, imm 0 19us
  op 1a: [0x048] & 0x1000 (bit set) T | [0x048] == 0xc3000 (all set bits) T | [0x048] & 0x2 (bit clear) T | [0xd70] ==
   0x12345678 T | [0xd70] & 0x8 (bit set) T | [0x800] == 0x20060808 T | r1 = ~0, imm 1 T | r1 = 0, imm 0 (until zero?)
   T | imm 1 (a short delay?) T | imm 0x100 T | event field 11, imm 0 T
  op 20: [0x048] & 0x1000 (bit set) T | [0x048] == 0xc3000 (all set bits) T | [0x048] & 0x2 (bit clear) T | [0xd70] ==
   0x12345678 T | [0xd70] & 0x8 (bit set) T | [0x800] == 0x20060808 T | r1 = ~0, imm 1 T | r1 = 0, imm 0 (until zero?)
   T | imm 1 (a short delay?) T | imm 0x100 T | event field 11, imm 0 T
```

14.2.89 op16 (run 23)

ID: `D-op16-23`. Device run 23. Experiment: op16.

```
opcode 16 (and 09) after one action from a clean state:
  nothing (clean)                      -> 16 T, 09 T; just before: 0x050 00000000, C50 ffff1800, ECC_DONE 10, FMC_STAT
  US 000c3000
  ECC decode, ECC_DONE left            -> 16 PASS, 09 T; just before: 0x050 00000000, C50 ffff1810, ECC_DONE 3d, FMC_S
  TATUS 000c3000
  ECC decode, ECC_DONE acknowledged    -> 16 T, 09 T; just before: 0x050 00000000, C50 ffff1800, ECC_DONE 10, FMC_STAT
  US 000c3000
  DMA RAM -> buffer, flag left         -> 16 PASS, 09 T; just before: 0x050 00000001, C50 ffff1804, ECC_DONE 10, FMC_S
  TATUS 001c3000
  DMA, flag acknowledged               -> 16 T, 09 T; just before: 0x050 00000001, C50 ffff1800, ECC_DONE 10, FMC_STAT
  US 000c3000
  70h command, flag left               -> 16 PASS, 09 T; just before: 0x050 00000000, C50 ffff1801, ECC_DONE 10, FMC_S
  TATUS 000c3002
  70h + status latch                   -> 16 PASS, 09 T; just before: 0x050 00000000, C50 ffff1800, ECC_DONE 10, FMC_S
  TATUS 010c3000
  READ ID through the first set        -> 16 T, 09 T; just before: 0x050 00000000, C50 ffff1800, ECC_DONE 10, FMC_STAT
  US 08041000
  one sector page read (to RAM)        -> 16 T, 09 T; just before: 0x050 00000000, C50 ffff1810, ECC_DONE 3c, FMC_STAT
  US 050c3000
```

14.2.90 opstate (run 24)

ID: `D-opstate-24`. Device run 24. Experiment: opstate.

```
after each state: on = continued with the next instruction, vector = went to the fault vector (C4C), T = a timeout fau
  lt happened, blocked = no end within 20 ms
  op 08 | nothing:on | command:on | DMA:on | ECC_DONE:on | status:on | all:on | second:on | second:on
  op 09 | nothing:on T | command:on T | DMA:on T | ECC_DONE:on T | status:on T | all:on T | second:on T | second:on T
  op 10 | nothing:on | command:on | DMA:on | ECC_DONE:on | status:on | all:on | second:on | second:on
  op 12 | nothing:blocked | command:blocked | DMA:blocked | ECC_DONE:blocked | status:blocked | all:blocked | second:b
  locked | second:blocked
  op 15 | nothing:on | command:on | DMA:on | ECC_DONE:on | status:on | all:on | second:on | second:on
  op 16 | nothing:on T | command:vector | DMA:vector | ECC_DONE:vector | status:vector | all:vector | second:vector |
  second:vector
  op ff | nothing:on | command:on | DMA:on | ECC_DONE:on | status:on | all:on | second:on | second:on
```

14.2.91 op16b (run 24)

ID: `D-op16b-24`. Device run 24. Experiment: op16b.

```
opcode 16:
  flag pending, C4C = alt                          -> went to alt (C4C)      19 us; at the target FMC_STATUS 000c300
  2 C50 ffff1801; C3C after 00000000
  flag pending, C4C = trap, C58 = 4                -> went to trap           19 us; at the target FMC_STATUS 0000000
  0 C50 00000000; C3C after 00000000
  command written just before (flag arrives later) -> went to alt (C4C)      19 us; at the target FMC_STATUS 000c300
  2 C50 ffff1801; C3C after 00000000
  DMA started just before (~14 us)                 -> went to alt (C4C)      22 us; at the target FMC_STATUS 001c300
  0 C50 ffff1804; C3C after 00000000
  nothing pending, C4C = alt                       -> continued            4861 us (timeout fault); at the target FM
  C_STATUS 00000000 C50 00000000; C3C after 00000000
  flag pending, 16 twice                           -> went to alt (C4C)      19 us; at the target FMC_STATUS 000c300
  2 C50 ffff1801; C3C after 00000000
```

14.2.92 op09 (run 24)

ID: `D-op09-24`. Device run 24. Experiment: op09.

```
opcode 09 against 1a, 20 and an unmet wait:
  C14 = 0 (tmwait just before)   09              -> FAULT    9725 us, fault at  9706 us, C14 before 00 after ff, CS_IR
  Q 04
  C14 = 0 (tmwait just before)   1a              -> FAULT    9726 us, fault at  9708 us, C14 before 00 after ff, CS_IR
  Q 04
  C14 = 0 (tmwait just before)   20              -> FAULT    9741 us, fault at  9724 us, C14 before 00 after ff, CS_IR
  Q 04
  C14 = 0 (tmwait just before)   wait 3 (unmet)  -> FAULT    9741 us, fault at  9722 us, C14 before 00 after ff, CS_IR
  Q 04
  C14 = 0xff (tmwait twice)      09              -> FAULT    9736 us, fault at  9718 us, C14 before ff after ff, CS_IR
  Q 04
  C14 = 0xff (tmwait twice)      1a              -> FAULT    9726 us, fault at  9708 us, C14 before ff after ff, CS_IR
  Q 04
  C14 = 0xff (tmwait twice)      20              -> FAULT    9739 us, fault at  9722 us, C14 before ff after ff, CS_IR
  Q 04
  C14 = 0xff (tmwait twice)      wait 3 (unmet)  -> FAULT    9728 us, fault at  9710 us, C14 before ff after ff, CS_IR
  Q 04
  state 1 ms in                  09              -> C08 4, C14 bf, C3C 00000000, C40 09000000, C44 00000000, C50 ffff1
  801, PC 20, CS_IRQ 00
  state 1 ms in                  1a              -> C08 4, C14 c0, C3C 00000000, C40 1a000000, C44 00000000, C50 ffff1
  801, PC 20, CS_IRQ 00
  state 1 ms in                  20              -> C08 4, C14 c0, C3C 00000000, C40 20000000, C44 00000000, C50 ffff1
  801, PC 20, CS_IRQ 00
  state 1 ms in                  wait 3 (unmet)  -> C08 4, C14 c0, C3C 00000000, C40 07030000, C44 00000000, C50 ffff1
  801, PC 20, CS_IRQ 00
  command flag (b1)              09 event field  -> TIMED OUT,   4895 us
  command flag (b1)              09 mask         -> TIMED OUT,   4879 us
  command flag (b1)              09 both         -> TIMED OUT,   4879 us
  DMA flag (b20)                 09 event field  -> TIMED OUT,   4896 us
  DMA flag (b20)                 09 mask         -> TIMED OUT,   4881 us
  DMA flag (b20)                 09 both         -> TIMED OUT,   4881 us
  ECC_DONE b0                    09 event field  -> TIMED OUT,   4892 us
  ECC_DONE b0                    09 mask         -> TIMED OUT,   4878 us
  ECC_DONE b0                    09 both         -> TIMED OUT,   4895 us
  status flag (b23)              09 event field  -> PASSED,     20 us
  status flag (b23)              09 mask         -> TIMED OUT,   4870 us
  status flag (b23)              09 both         -> PASSED,     19 us
```

14.2.93 op09b (run 25)

ID: `D-op09b-25`. Device run 25. Experiment: op09b.

```
opcode 09:
  status pending, 09 ev 5 arg 0    imm 0    -> passed        20 us; NSTAT before 70 after 70
  status pending, 09 ev 5 arg 40   imm 40   -> passed        20 us; NSTAT before 70 after 70
  status pending, 09 ev 5 arg 20   imm 20   -> passed        19 us; NSTAT before 70 after 70
  status pending, 09 ev 5 arg 0    imm 40   -> passed        20 us; NSTAT before 70 after 70
  status pending, 09 ev 5 arg 0    imm 1    -> passed        19 us; NSTAT before 70 after 70
  status pending, 09 ev 5 arg 1    imm 1    -> passed        19 us; NSTAT before 70 after 70
  status pending, 09 ev 5 arg 70   imm ff   -> passed        19 us; NSTAT before 70 after 70
  status pending, 09 ev 5 arg 71   imm ff   -> passed        20 us; NSTAT before 70 after 70
  status pending, 09 ev 5 arg 40   imm 0    -> passed        19 us; NSTAT before 70 after 70
  status pending, 09 ev 5 arg ff   imm ff   -> passed        20 us; NSTAT before 70 after 70
  every flag of both sets pending, 09 event field 0..15: 0:T 1:T 2:T 3:T 4:T 5:ok 6:T 7:T 8:T 9:T 10:T 11:T 12:T 13:T
  14:T 15:T
  status latched, b23 acknowledged, 09 ev 5    -> passed, 19 us
  erase running, status latch, 09 ev 5 arg 0x40 imm 0x40  -> passed        19 us; NSTAT right after the latch 81, at t
  he end 81
  erase running, status latch, 09 ev 5 arg 0 imm 0        -> passed        20 us; NSTAT right after the latch 81, at t
  he end 81
  erase running, status latch, reference: wait 5 only     -> passed       180 us; NSTAT right after the latch 81, at t
  he end c0
```

14.2.94 op09c (run 26)

ID: `D-op09c-26`. Device run 26. Experiment: op09c.

```
nothing pending (soft reset first), event numbers that pass:
  no chip selected  09 passes for: 5
  no chip selected  wait passes for: 11
  CE0 selected      09 passes for: 5
  CE0 selected      wait passes for: 11
  CE1 selected      09 passes for: 5
  CE1 selected      wait passes for: 11
```

14.2.95 chan1pins (run 27)

ID: `D-chan1pins-27`. Device run 27. Experiment: chan1pins.

```
READ ID (90h 00h, 8 bytes), FMC channel 1 pins switched or not:
  set 0x000, CE0, pins as the test setup leaves them   -> END        18 us, 0 timeouts, ID b614d5ec 14d5ec74, status 0
  8041000
  set 0x000, CE0, channel-1 pins switched      -> END        17 us, 0 timeouts, ID b614d5ec 14d5ec74, status 08041000
  set 0x400, CE0, pins as the test setup leaves them   -> FAULT   14621 us, 3 timeouts, ID 00000000 00000000, status 0
  a041000
  set 0x400, CE0, channel-1 pins switched      -> FAULT   14641 us, 3 timeouts, ID 00000000 00000000, status 02041000
```

```
  set 0x400, CE1, channel-1 pins switched      -> FAULT   14627 us, 3 timeouts, ID ffffffff ffffffff, status 00041000
  set 0x400, CE2, channel-1 pins switched      -> FAULT   14624 us, 3 timeouts, ID ffffffff ffffffff, status 00041000
  set 0x400, CE3, channel-1 pins switched      -> FAULT   14630 us, 3 timeouts, ID ffffffff ffffffff, status 00041000
  set 0x400, CE4, channel-1 pins switched      -> FAULT   14627 us, 3 timeouts, ID ffffffff ffffffff, status 00041000
  set 0x400, CE5, channel-1 pins switched      -> FAULT   14627 us, 3 timeouts, ID ffffffff ffffffff, status 00041000
  set 0x400, CE6, channel-1 pins switched      -> FAULT   14627 us, 3 timeouts, ID ffffffff ffffffff, status 00041000
  set 0x400, CE7, channel-1 pins switched      -> FAULT   14626 us, 3 timeouts, ID ffffffff ffffffff, status 00041000
```

14.2.96 pcwrite (run 27)

ID: `D-pcwrite-27`. Device run 27. Experiment: pcwrite.

```
the sequencer and its PC:
  wr [0xc38], #lab             -> END    JUMPED to lab; r2 00000000
  st [0xc38], r1 (r1 = lab)    -> END    JUMPED to lab; r2 00000000
  ld r2, [0xc38] (read it)     -> END    fell through; r2 00000028
```

14.2.97 op12cont (run 27)

ID: `D-op12cont-27`. Device run 27. Experiment: op12cont.

```
opcode 12 at offset 0x28, CPU writes 2 ms after the start (in this order):
  none (control)                                      -> TIMEOUT still paused               20016 us, PC 28, C08 a,
  CS_IRQ 00000008
  CS_IRQ = 8, start                                   -> TIMEOUT still paused               20016 us, PC 28, C08 a,
  CS_IRQ 00000008
  start, CS_IRQ = 8                                   -> TIMEOUT still paused               20015 us, PC 28, C08 a,
  CS_IRQ 00000000
  PC = pause + 8, CS_IRQ = 8, start (the 4G/5G sequence) -> END     CONTINUED after the pause    2025 us, PC 0, C08 0,
   CS_IRQ 00000001
  PC = pause + 8, start                               -> END     CONTINUED after the pause    2024 us, PC 0, C08 0, C
  S_IRQ 00000009
  CS_IRQ = 8, start 0x1                               -> TIMEOUT still paused               20019 us, PC 28, C08 a,
  CS_IRQ 00000008
  CS_IRQ = 8, start, start                            -> TIMEOUT still paused               20016 us, PC 28, C08 a,
  CS_IRQ 00000008
```

14.2.98 ctrlfx (run 27)

ID: `D-ctrlfx-27`. Device run 27. Experiment: ctrlfx.

```
on = continued, vector = went to C4C, T<n> = n timeout faults; irq: VIC line seen while polling / at the end:
  start 0x0001 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0xfff5 | timeout: on T1 | illegal: on T1 | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0xffb5 | timeout: on T1 | illegal: on T1 | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0x0005 | timeout: on T0 | illegal: on | evjmp: vector | irq: line 0/0 CS_IRQ 15
  start 0x0011 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0x0021 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0x0041 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0x0081 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0x0101 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0x0201 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0x0401 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0x0801 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0x1001 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0x2001 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0x4001 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
  start 0x8001 | timeout: TIMEOUT | illegal: TIMEOUT | evjmp: vector | irq: line 0/0 CS_IRQ 11
```

14.2.99 op09d (run 27)

ID: `D-op09d-27`. Device run 27. Experiment: op09d.

```
second FMC set (0x400), one flag pending at a time (waits inside the set-up that time out are part of it; the sweep is
   the last wait):
  nothing pending        0x448 000c3000  wait2 passes for: 5
  nothing pending        0x448 000c3000  wait  passes for: 11
  command flag (b1)      0x448 000c3002  wait2 passes for: 5
  command flag (b1)      0x448 000c3002  wait  passes for: 6 11
  address flag (b2)      0x448 000c3004  wait2 passes for: 5
  address flag (b2)      0x448 000c3004  wait  passes for: 7 11
  transfer flag (b3)     0x448 08041008  wait2 passes for: 5
  transfer flag (b3)     0x448 08041008  wait  passes for: 9 11
  DMA flag (b20)         0x448 001c3000  wait2 passes for: 5
  DMA flag (b20)         0x448 001c3000  wait  passes for: 8 11
  status latch (b23?)    0x448 000c3000  wait2 passes for: 5
  status latch (b23?)    0x448 000c3000  wait  passes for: 11
```

14.2.100 fwidth (run 27)

ID: `D-fwidth-27`. Device run 27. Experiment: fwidth.

```
field widths:
  wr [0x1d70]          -> END    0xd70 now 11111111
  wr [0x8d70]          -> END    0xd70 now 22222222
  wr [0xfd70]          -> END    0xd70 now 33333333
```

```
  st [0x8d70], r1      -> END    0xd70 now 44444444
  ld r2, [0x1d74]      -> END    r2 5a5a5a5a (0xd74 holds 5a5a5a5a)
  ld r2, [0x8d74]      -> END    r2 5a5a5a5a (0xd74 holds 5a5a5a5a)
  bnz      target lab | 0x1000     -> END     went elsewhere (no marker)
  jr       target lab | 0x1000     -> END     fell through
  movi r8  target lab | 0x1000     -> END     went elsewhere (no marker)
  bnz      target lab | 0x10000    -> END     went elsewhere (no marker)
  jr       target lab | 0x10000    -> END     fell through
  movi r8  target lab | 0x10000    -> END     went elsewhere (no marker)
  bnz      target lab | 0x1000000  -> FAULT   went elsewhere (no marker) (timeout fault)
  jr       target lab | 0x1000000  -> END     fell through
  movi r8  target lab | 0x1000000  -> FAULT   went elsewhere (no marker) (timeout fault)
  bnz      target lab | 0x80000000 -> END     landed on lab (upper bits ignored)
  jr       target lab | 0x80000000 -> END     fell through
  movi r8  target lab | 0x80000000 -> END     landed on lab (upper bits ignored)
```

14.2.101 irqsrc (run 27)

ID: `D-irqsrc-27`. Device run 27. Experiment: irqsrc.

```
interrupt sources (VIC bits: raw lines seen during the run that were not set in the baseline):
  baseline (nothing pending, no enables): VIC0 seen 00000020, VIC1 seen 00000002
  nothing   none        CS_IRQ 00 C50 ffff1800 FMC_STATUS 000c3000 0x050 00000000  new VIC0 00000000 VIC1 00000000
  nothing   0x044=ffff  CS_IRQ 00 C50 ffff1800 FMC_STATUS 000c3000 0x050 00000000  new VIC0 00000000 VIC1 00000000
  nothing   0x844=2f    CS_IRQ 00 C50 ffff1800 FMC_STATUS 000c3000 0x050 00000000  new VIC0 00000000 VIC1 00000000
  command   none        CS_IRQ 00 C50 ffff1801 FMC_STATUS 000c3002 0x050 00000000  new VIC0 00000000 VIC1 00000000
  command   0x044=ffff  CS_IRQ 10 C50 ffff1801 FMC_STATUS 000c3002 0x050 00000000  new VIC0 00100000 VIC1 00000000
  command   0x844=2f    CS_IRQ 00 C50 ffff1801 FMC_STATUS 000c3002 0x050 00000000  new VIC0 00000000 VIC1 00000000
  address   none        CS_IRQ 00 C50 ffff1802 FMC_STATUS 000c3004 0x050 00000000  new VIC0 00000000 VIC1 00000000
  address   0x044=ffff  CS_IRQ 10 C50 ffff1802 FMC_STATUS 000c3004 0x050 00000000  new VIC0 00000000 VIC1 00000000
  address   0x844=2f    CS_IRQ 00 C50 ffff1802 FMC_STATUS 000c3004 0x050 00000000  new VIC0 00000000 VIC1 00000000
  transfer  none        CS_IRQ 00 C50 ffff1808 FMC_STATUS 08041008 0x050 00000000  new VIC0 00000000 VIC1 00000000
  transfer  0x044=ffff  CS_IRQ 10 C50 ffff1808 FMC_STATUS 08041008 0x050 00000000  new VIC0 00000000 VIC1 00000000
  transfer  0x844=2f    CS_IRQ 00 C50 ffff1808 FMC_STATUS 08041008 0x050 00000000  new VIC0 00000000 VIC1 00000000
  DMA       none        CS_IRQ 00 C50 ffff1804 FMC_STATUS 001c3000 0x050 00000001  new VIC0 00000000 VIC1 00000000
  DMA       0x044=ffff  CS_IRQ 10 C50 ffff1804 FMC_STATUS 001c3000 0x050 00000001  new VIC0 00000000 VIC1 00000000
  DMA       0x844=2f    CS_IRQ 00 C50 ffff1804 FMC_STATUS 001c3000 0x050 00000001  new VIC0 00000000 VIC1 00000000
  ECC       none        CS_IRQ 00 C50 ffff1810 FMC_STATUS 000c3000 0x050 00000000  new VIC0 00000000 VIC1 00000000
  ECC       0x044=ffff  CS_IRQ 00 C50 ffff1810 FMC_STATUS 000c3000 0x050 00000000  new VIC0 00000000 VIC1 00000000
  ECC       0x844=2f    CS_IRQ 40 C50 ffff1810 FMC_STATUS 000c3000 0x050 00000000  new VIC0 00000000 VIC1 00000000
  status    none        CS_IRQ 00 C50 ffff1820 FMC_STATUS 008c3000 0x050 00000000  new VIC0 00000000 VIC1 00000000
  status    0x044=ffff  CS_IRQ 14 C50 ffff1800 FMC_STATUS 010c3000 0x050 00000000  new VIC0 00100000 VIC1 00000000 (FA
  ULT)
  status    0x844=2f    CS_IRQ 04 C50 ffff1800 FMC_STATUS 010c3000 0x050 00000000  new VIC0 00000000 VIC1 00000000 (FA
  ULT)
  command   single 0x044 bits that reach CS_IRQ: b0->CS_IRQ 10
  address   single 0x044 bits that reach CS_IRQ: b0->CS_IRQ 10, b1->CS_IRQ 10
  transfer  single 0x044 bits that reach CS_IRQ: b0->CS_IRQ 10, b1->CS_IRQ 10, b2->CS_IRQ 10, b11->CS_IRQ 10
  DMA       single 0x044 bits that reach CS_IRQ: b11->CS_IRQ 10
  status    single 0x044 bits that reach CS_IRQ: b0->CS_IRQ 10, b13->CS_IRQ 10
```

14.2.102 eccpar (run 27)

ID: `D-eccpar-27`. Device run 27. Experiment: eccpar.

```
ECC engine with a known sector (no flash access):
  zeros  encode: END; ECC area words that changed: 810:00000011 814:00000180 840:0000003b 900:95982422 904:f3a2a203 90
  8:77a3b310 90c:0000003e; parity buffer DMA done: ffffff3e77a3b310f3a2a20395982422
  zeros  decode, no error         ECC_RESULT 00000011 ECC_DONE 3d; changed vs after the encode: 840:0000003d 900:00000
  000 904:00000000 908:00000000 90c:00000000 9a4:0000009b
  zeros  decode, 1 bit flipped    ECC_RESULT 00000011 ECC_DONE 3d; changed vs after the encode: 840:0000003d 900:00000
  000 904:00000000 908:00000000 90c:00000000 9a4:0000009c
  zeros  decode, 2 bits flipped   ECC_RESULT 00000011 ECC_DONE 3d; changed vs after the encode: 840:0000003d 900:00000
  000 904:00000000 908:00000000 90c:00000000 9a4:0000009d
  ones   encode: END; ECC area words that changed: 840:0000003b 900:b1b427c5 904:aad5f9ff 908:83f9f67b 90c:00000043; p
  arity buffer DMA done: ffffff4383f9f67baad5f9ffb1b427c5
  ones   decode, no error         ECC_RESULT 00000000 ECC_DONE 75; changed vs after the encode: 810:00000000 818:00000
  000 840:00000075 900:00000000 904:00000000 908:00000000 90c:00000000 9a0:ff0fffff 9a4:0000009e
  ones   decode, 1 bit flipped    ECC_RESULT 00000011 ECC_DONE 3d; changed vs after the encode: 840:0000003d 900:00000
  000 904:00000000 908:00000000 90c:00000000 9a0:ff0fffff 9a4:0000009f
  ones   decode, 2 bits flipped   ECC_RESULT 00000011 ECC_DONE 3d; changed vs after the encode: 840:0000003d 900:00000
  000 904:00000000 908:00000000 90c:00000000 9a0:ff0fffff 9a4:000000a0
  ramp   encode: END; ECC area words that changed: 840:0000003b 900:a816dd67 904:3112048d 908:18165999 90c:00000076; p
  arity buffer DMA done: ffffff761816599931120 48da816dd67
  ramp   decode, no error         ECC_RESULT 00000011 ECC_DONE 3d; changed vs after the encode: 840:0000003d 900:00000
  000 904:00000000 908:00000000 90c:00000000 9a4:000000a1
  ramp   decode, 1 bit flipped    ECC_RESULT 00000011 ECC_DONE 3d; changed vs after the encode: 840:0000003d 900:00000
  000 904:00000000 908:00000000 90c:00000000 9a4:000000a2
  ramp   decode, 2 bits flipped   ECC_RESULT 00000011 ECC_DONE 3d; changed vs after the encode: 840:0000003d 900:00000
  000 904:00000000 908:00000000 90c:00000000 9a4:000000a3
```

14.2.103 bufsize (run 27)

ID: `D-bufsize-27`. Device run 27. Experiment: bufsize.

```
data buffer size (RAM -> buffer 0 -> RAM, N bytes):
  N = 256   -> fill ok, drain ok; first 256 bytes intact; bytes written past N: yes; DMA_ADDR after 08055f80 (start 08
  055f80)
  N = 512   -> fill ok, drain ok; first 512 bytes intact; bytes written past N: none; DMA_ADDR after 08055f80 (start 0
  8055f80)
  N = 513   -> fill ok, drain ok; first 512 bytes intact; bytes written past N: none; DMA_ADDR after 08055f80 (start 0
  8055f80)
  N = 1024  -> fill ok, drain ok; first 512 bytes intact; bytes written past N: none; DMA_ADDR after 08055f80 (start 0
  8055f80)
  N = 2048  -> fill ok, drain ok; first 512 bytes intact; bytes written past N: none; DMA_ADDR after 08055f80 (start 0
  8055f80)
```

14.2.104 eccstr (run 27)

ID: `D-eccstr-27`. Device run 27. Experiment: eccstr.

```
ECC decode of one known sector with bits flipped (encode in the same mode first):
  ECC_MODE 000180,  0 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data restored
  ECC_MODE 000180,  1 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000180,  2 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000180,  3 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000180,  4 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000180,  5 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000180,  6 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000180,  7 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000180,  8 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000180,  9 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000180, 10 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000180, 11 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000180, 12 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE e00000,  1 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE e00000,  8 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE e00000,  9 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE e00000, 12 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000000,  1 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000000,  8 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000000,  9 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000000, 12 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000080,  1 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000080,  8 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000080,  9 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000080, 12 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000100,  1 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000100,  8 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000100,  9 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000100, 12 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 0001c0,  1 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 0001c0,  8 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 0001c0,  9 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 0001c0, 12 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000200,  1 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000200,  8 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000200,  9 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000200, 12 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000380,  1 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000380,  8 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000380,  9 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
  ECC_MODE 000380, 12 bits flipped -> ECC_RESULT 00000011 (b0 1, corrected  0), data as sent
```

14.2.105 access (run 27)

ID: `D-access-27`. Device run 27. Experiment: access.

```
bits that follow a write of all ones / zero (writer->reader; - = not tested). Not written: 0x004 0x008 0x038 0x07c 0x0
  80 0x204 0x208 0x238 0x27c 0x280 0x404 0x408 0x438 0x47c 0x480 0x604 0x608 0x638 0x67c 0x680 0x80c 0xa0c 0xc00 0xc04
   0xc4c 0xc58 0xc5c; not written by the sequencer: r0..r7, PC
  0x000..0x010  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
  0x014..0x014  cpu->cpu 0000ffff  cpu->seq 0000ffff  seq->seq 0000ffff  seq->cpu 0000ffff
  0x018..0x018  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
  0x01c..0x01c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
  0x020..0x020  cpu->cpu 00ffffff  cpu->seq 00ffffff  seq->seq 00ffffff  seq->cpu 00ffffff
  0x024..0x024  cpu->cpu 000001ff  cpu->seq 000001ff  seq->seq 000001ff  seq->cpu 000001ff
  0x028..0x028  cpu->cpu 00000003  cpu->seq 00000003  seq->seq 000001ff  seq->cpu 00000003
  0x02c..0x02c  cpu->cpu 00000007  cpu->seq 00000007  seq->seq 00000007  seq->cpu 00000007
  0x030..0x030  cpu->cpu 00001fff  cpu->seq 00001fff  seq->seq 00001fff  seq->cpu 00001fff
  0x034..0x040  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
  0x044..0x044  cpu->cpu 0000ffff  cpu->seq 0000ffff  seq->seq 0000ffff  seq->cpu 0000ffff
  0x048..0x04c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
  0x050..0x050  cpu->cpu 00002000  cpu->seq 00002000  seq->seq 00000000  seq->cpu 00000000
  0x054..0x05c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
  0x060..0x078  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
  0x084..0x1b4  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
  0x1b8..0x1b8  cpu->cpu 00000001  cpu->seq 00000001  seq->seq 00000001  seq->cpu 00000001
  0x1bc..0x1c8  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
  0x1cc..0x1d0  cpu->cpu 00ffffff  cpu->seq 00ffffff  seq->seq 00ffffff  seq->cpu 00ffffff
  0x1d4..0x210  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
```

```
0x214..0x214  cpu->cpu 0000ffff  cpu->seq 0000ffff  seq->seq 0000ffff  seq->cpu 0000ffff
0x218..0x218  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x21c..0x21c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x220..0x220  cpu->cpu 00ffffff  cpu->seq 00ffffff  seq->seq 00ffffff  seq->cpu 00ffffff
0x224..0x224  cpu->cpu 000001ff  cpu->seq 000001ff  seq->seq 000001ff  seq->cpu 000001ff
0x228..0x228  cpu->cpu 00000007  cpu->seq 00000003  seq->seq 000001ff  seq->cpu 00000007
0x22c..0x22c  cpu->cpu 00000007  cpu->seq 00000007  seq->seq 00000007  seq->cpu 00000007
0x230..0x230  cpu->cpu 00001fff  cpu->seq 00001fff  seq->seq 00001fff  seq->cpu 00001fff
0x234..0x240  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x244..0x244  cpu->cpu 0000ffff  cpu->seq 0000ffff  seq->seq 0000ffff  seq->cpu 0000ffff
0x248..0x24c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x250..0x250  cpu->cpu 00003000  cpu->seq 00003000  seq->seq 00002800  seq->cpu 00002800
0x254..0x25c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x260..0x278  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x284..0x3a4  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x3a8..0x3b4  cpu->cpu 00000000  cpu->seq ffffffff  seq->seq 00000000  seq->cpu 00000000
0x3b8..0x3b8  cpu->cpu 00000001  cpu->seq 00000001  seq->seq 00000001  seq->cpu 00000001
0x3bc..0x3c8  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x3cc..0x3d0  cpu->cpu 00ffffff  cpu->seq 00ffffff  seq->seq 00ffffff  seq->cpu 00ffffff
0x3d4..0x3e4  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x3e8..0x3fc  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x400..0x410  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x414..0x414  cpu->cpu 0000ffff  cpu->seq 0000ffff  seq->seq 0000ffff  seq->cpu 0000ffff
0x418..0x418  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x41c..0x41c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x420..0x420  cpu->cpu 00ffffff  cpu->seq 00ffffff  seq->seq 00ffffff  seq->cpu 00ffffff
0x424..0x424  cpu->cpu 000001ff  cpu->seq 000001ff  seq->seq 000001ff  seq->cpu 000001ff
0x428..0x428  cpu->cpu 000001ff  cpu->seq 00000007  seq->seq 000001ff  seq->cpu 000001ff
0x42c..0x42c  cpu->cpu 00000007  cpu->seq 00000007  seq->seq 00000007  seq->cpu 00000007
0x430..0x430  cpu->cpu 00001fff  cpu->seq 00001fff  seq->seq 00001fff  seq->cpu 00001fff
0x434..0x440  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x444..0x444  cpu->cpu 0000ffff  cpu->seq 0000ffff  seq->seq 0000ffff  seq->cpu 0000ffff
0x448..0x44c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x450..0x450  cpu->cpu 00000000  cpu->seq 00003000  seq->seq 00003000  seq->cpu 00003000
0x454..0x45c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x460..0x478  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x484..0x4b0  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x4b4..0x4b8  cpu->cpu 00000001  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x4bc..0x5b4  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x5b8..0x5b8  cpu->cpu 00000001  cpu->seq 00000001  seq->seq 00000001  seq->cpu 00000001
0x5bc..0x5c8  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x5cc..0x5d0  cpu->cpu 00ffffff  cpu->seq 00ffffff  seq->seq 00ffffff  seq->cpu 00ffffff
0x5d4..0x5f8  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x5fc..0x5fc  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x600..0x610  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x614..0x614  cpu->cpu 0000ffff  cpu->seq 0000ffff  seq->seq 0000ffff  seq->cpu 0000ffff
0x618..0x618  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x61c..0x61c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x620..0x620  cpu->cpu 00ffffff  cpu->seq 00ffffff  seq->seq 00ffffff  seq->cpu 00ffffff
0x624..0x624  cpu->cpu 000001ff  cpu->seq 000001ff  seq->seq 000001ff  seq->cpu 000001ff
0x628..0x628  cpu->cpu 00000003  cpu->seq 00000007  seq->seq 000001ff  seq->cpu 00000003
0x62c..0x62c  cpu->cpu 00000007  cpu->seq 00000007  seq->seq 00000007  seq->cpu 00000007
0x630..0x630  cpu->cpu 00001fff  cpu->seq 00001fff  seq->seq 00001fff  seq->cpu 00001fff
0x634..0x640  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x644..0x644  cpu->cpu 0000ffff  cpu->seq 0000ffff  seq->seq 0000ffff  seq->cpu 0000ffff
0x648..0x64c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x650..0x650  cpu->cpu 00001000  cpu->seq 00001000  seq->seq 00000000  seq->cpu 00000000
0x654..0x65c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x660..0x678  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x684..0x6cc  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x6d0..0x6d0  cpu->cpu 00000001  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x6d4..0x7b4  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x7b8..0x7b8  cpu->cpu 00000001  cpu->seq 00000001  seq->seq 00000001  seq->cpu 00000001
0x7bc..0x7c8  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x7cc..0x7d0  cpu->cpu 00ffffff  cpu->seq 00ffffff  seq->seq 00ffffff  seq->cpu 00ffffff
0x7d4..0x7fc  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x800..0x800  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x804..0x808  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x810..0x810  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x814..0x814  cpu->cpu 01ffffff  cpu->seq 01ffffff  seq->seq 01ffffff  seq->cpu 01ffffff
0x818..0x840  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x844..0x844  cpu->cpu 0000002f  cpu->seq 0000002f  seq->seq 0000002f  seq->cpu 0000002f
0x848..0x93c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x940..0x950  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0x954..0x9bc  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0x9c0..0x9c0  cpu->cpu 00000001  cpu->seq 00000001  seq->seq 00000001  seq->cpu 00000001
0x9c4..0xa00  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0xa04..0xa08  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0xa10..0xa10  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0xa14..0xa14  cpu->cpu 01ffffff  cpu->seq 01ffffff  seq->seq 01ffffff  seq->cpu 01ffffff
0xa18..0xa40  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0xa44..0xa44  cpu->cpu 0000002f  cpu->seq 0000002f  seq->seq 0000002f  seq->cpu 0000002f
0xa48..0xb3c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
0xb40..0xb50  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
0xb54..0xbbc  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
```

```
  0xbc0..0xbc0  cpu->cpu 00000001  cpu->seq 00000001  seq->seq 00000001  seq->cpu 00000001
  0xbc4..0xc0c  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
  0xc10..0xc10  cpu->cpu 000000ff  cpu->seq 000000ff  seq->seq 000000ff  seq->cpu 000000ff
  0xc14..0xc14  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
  0xc18..0xc34  cpu->cpu ffffffff  cpu->seq 00000000  seq->seq -  seq->cpu -
  0xc38..0xc38  cpu->cpu 00000000  cpu->seq 00000000  seq->seq -  seq->cpu -
  0xc3c..0xc3c  cpu->cpu ffffffff  cpu->seq 00000000  seq->seq ffffffff  seq->cpu 00000000
  0xc40..0xc44  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
  0xc48..0xc48  cpu->cpu ffffffff  cpu->seq 00000000  seq->seq ffffffff  seq->cpu ffffffff
  0xc50..0xc50  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
  0xc54..0xc54  cpu->cpu 00000fff  cpu->seq 00000000  seq->seq 00000fff  seq->cpu 00000fff
  0xc60..0xcfc  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
  0xd00..0xd7c  cpu->cpu ffffffff  cpu->seq ffffffff  seq->seq ffffffff  seq->cpu ffffffff
  0xd80..0xffc  cpu->cpu 00000000  cpu->seq 00000000  seq->seq 00000000  seq->cpu 00000000
```

14.2.106 fields0 (run 28)

ID: `D-fields0-28`. Device run 28. Experiment: fields0.

```
fields left 0 by the original programs (expected without an effect: r2 = 7777cafe / aaaaaaaa / 11111111, the word writ
  ten at its plain place):
  st [0xd70], r7, imm 0xff     -> END    r2 00000000  0xd70..0xd78 7777cafe 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  ldi r2, [r1], imm 0xff       -> END    r2 bbbbbbbb  0xd70..0xd78 11111111 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  mov r2, r7, imm 0xff         -> END    r2 7777cafe  0xd70..0xd78 11111111 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  sti [r1], r7, imm 0xff       -> END    r2 00000000  0xd70..0xd78 11111111 22222222 33333333  RAM +0..+12 aaaaaaaa 77
  77cafe cccccccc dddddddd
  pld r2, [r5], imm 0xff       -> END    r2 11111111  0xd70..0xd78 11111111 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  pst [r5], r7, imm 0xff       -> END    r2 00000000  0xd70..0xd78 7777cafe 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  st [0xd70], r7, imm 0x4      -> END    r2 00000000  0xd70..0xd78 7777cafe 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  ldi r2, [r1], imm 0x4        -> END    r2 bbbbbbbb  0xd70..0xd78 11111111 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  mov r2, r7, imm 0x4          -> END    r2 7777cafe  0xd70..0xd78 11111111 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  sti [r1], r7, imm 0x4        -> END    r2 00000000  0xd70..0xd78 11111111 22222222 33333333  RAM +0..+12 aaaaaaaa 77
  77cafe cccccccc dddddddd
  pld r2, [r5], imm 0x4        -> END    r2 11111111  0xd70..0xd78 11111111 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  pst [r5], r7, imm 0x4        -> END    r2 00000000  0xd70..0xd78 7777cafe 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  wr [0xd70], #x, R 3          -> END    r2 00000000  0xd70..0xd78 44444444 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  wr [0xd70], #x, R 7          -> END    r2 00000000  0xd70..0xd78 44444444 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  movi r2, #x, arg 0x1234      -> END    r2 55555555  0xd70..0xd78 11111111 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  movi r2, #x, arg 0xd70       -> END    r2 55555555  0xd70..0xd78 11111111 22222222 33333333  RAM +0..+12 aaaaaaaa bb
  bbbbbb cccccccc dddddddd
  end                          -> END    SEQ_RESULT 66666666, r6 66666666, CS_IRQ 00000001
  end R 1                      -> FAULT  SEQ_RESULT bad0bad0, r6 bad0bad0, CS_IRQ 00000005
  end arg 1                    -> FAULT  SEQ_RESULT bad0bad0, r6 bad0bad0, CS_IRQ 00000005
  end imm 0x12345678           -> END    SEQ_RESULT 66666666, r6 66666666, CS_IRQ 00000001
  end R 1, arg 7, imm 0xff     -> FAULT  SEQ_RESULT bad0bad0, r6 bad0bad0, CS_IRQ 00000005
```

14.2.107 tmofield (run 28)

ID: `D-tmofield-28`. Device run 28. Experiment: tmofield.

```
time from the start to the timeout of a wait that is never met:
  start 0x0005 -> FAULT  fault at      2 us, C14 after the fault fe
  start 0x0105 -> FAULT  fault at      0 us, C14 after the fault fe
  start 0x0205 -> FAULT  fault at      2 us, C14 after the fault fe
  start 0x0405 -> FAULT  fault at      0 us, C14 after the fault fe
  start 0x1005 -> FAULT  fault at      2 us, C14 after the fault fe
  start 0x4005 -> FAULT  fault at      0 us, C14 after the fault fe
  start 0x8005 -> FAULT  fault at      0 us, C14 after the fault fe
  start 0xff05 -> FAULT  fault at      2 us, C14 after the fault fe
  start 0x00f5 -> FAULT  fault at     16 us, C14 after the fault 00
  start 0x0ff5 -> FAULT  fault at    310 us, C14 after the fault ff
  start 0x0f05 -> FAULT  fault at      0 us, C14 after the fault fe
  start 0xfff5 -> FAULT  fault at   4868 us, C14 after the fault ff
```

14.2.108 eccstr2 (run 28)

ID: `D-eccstr2-28`. Device run 28. Experiment: eccstr2.

```
ECC, encode and decode in one program:
   0 bits flipped -> END    ECC_RESULT 00000000 (uncorrectable 0, corrected  0), ECC_DONE 35, data corrected
   1 bits flipped -> END    ECC_RESULT 00010000 (uncorrectable 0, corrected  1), ECC_DONE 35, data corrected
   2 bits flipped -> END    ECC_RESULT 00020000 (uncorrectable 0, corrected  2), ECC_DONE 35, data corrected
   3 bits flipped -> END    ECC_RESULT 00030000 (uncorrectable 0, corrected  3), ECC_DONE 35, data corrected
```

```
   4 bits flipped -> END    ECC_RESULT 00040000 (uncorrectable 0, corrected  4), ECC_DONE 35, data corrected
   5 bits flipped -> END    ECC_RESULT 00050000 (uncorrectable 0, corrected  5), ECC_DONE 35, data corrected
   6 bits flipped -> END    ECC_RESULT 00060000 (uncorrectable 0, corrected  6), ECC_DONE 35, data corrected
   7 bits flipped -> END    ECC_RESULT 00070000 (uncorrectable 0, corrected  7), ECC_DONE 35, data corrected
   8 bits flipped -> END    ECC_RESULT 00080000 (uncorrectable 0, corrected  8), ECC_DONE 35, data corrected
   9 bits flipped -> END    ECC_RESULT 00000011 (uncorrectable 1, corrected  0), ECC_DONE 3d, data as sent
  10 bits flipped -> END    ECC_RESULT 00000011 (uncorrectable 1, corrected  0), ECC_DONE 3d, data as sent
  11 bits flipped -> END    ECC_RESULT 00000011 (uncorrectable 1, corrected  0), ECC_DONE 3d, data as sent
  12 bits flipped -> END    ECC_RESULT 00000011 (uncorrectable 1, corrected  0), ECC_DONE 3d, data as sent
  parity of zeros    : ffffff3e77a3b310f3a2a20395982422
  parity of ones     : ffffff4383f9f67baad5f9ffb1b427c5
  parity of bit 0    : ffffff92b29212590fea32db09c2cda8
  parity of bit 1    : ffffff7204d411f8072004f368e90c15
  parity of bit 2    : ffffffa6914cf6c11aa7efe26f7a744c
  parity of bit 7    : ffffff56c92cce6b0a1464a31e146d94
  parity of bit 8    : fffffff725bfb0d3e57ba1f05a0ba788
  parity of bit 9    : ffffffb92a8f54edd20322a5cf7bd855
  parity of bit 4088 : ffffff691653865aeb56c346eef4b367
  parity of bit 4095 : ffffff8b9c96fade676d55c970afc0f9
```

14.2.109 ev5en (run 28)

ID: `D-ev5en-28`. Device run 28. Experiment: ev5en.

```
status latch and wait 5 (T = wait 5 timed out), FMC_STATUS right after the latch and after the wait:
  no enable              ok (before wait 008c3000, after 008c3000) | ok (before wait 008c3000, after 008c3000) | ok (b
  efore wait 008c3000, after 008c3000)
  0x044 = 1 (CPU)        ok (before wait 008c3000, after 008c3000) | ok (before wait 008c3000, after 008c3000) | ok (b
  efore wait 008c3000, after 008c3000)
  0x044 = 0x2000 (CPU)   ok (before wait 008c3000, after 008c3000) | ok (before wait 008c3000, after 008c3000) | ok (b
  efore wait 008c3000, after 008c3000)
  0x844 = 1 (CPU)        ok (before wait 008c3000, after 008c3000) | ok (before wait 008c3000, after 008c3000) | ok (b
  efore wait 008c3000, after 008c3000)
  0x044 = 1 (program)    ok (before wait 008c3000, after 008c3000) | ok (before wait 008c3000, after 008c3000) | ok (b
  efore wait 008c3000, after 008c3000)
```

14.2.110 tmobits (run 29)

ID: `D-tmobits-29`. Device run 29. Experiment: tmobits.

```
time from the start to the timeout of a wait that is never met:
  start 0xff15 -> FAULT  fault at     18 us, C14 after the fault ff
  start 0xff25 -> FAULT  fault at     72 us, C14 after the fault ff
  start 0xff45 -> FAULT  fault at   1218 us, C14 after the fault ff
  start 0xff85 -> FAULT  fault at   4868 us, C14 after the fault ff
  start 0xff35 -> FAULT  fault at    296 us, C14 after the fault ff
  start 0xffc5 -> FAULT  fault at   4868 us, C14 after the fault ff
  start 0x0f15 -> FAULT  fault at      2 us, C14 after the fault ff
  start 0x0f25 -> FAULT  fault at      0 us, C14 after the fault ff
  start 0x0f45 -> FAULT  fault at     72 us, C14 after the fault ff
  start 0x0f85 -> FAULT  fault at    308 us, C14 after the fault ff
  start 0x01f5 -> FAULT  fault at     46 us, C14 after the fault ff
  start 0x03f5 -> FAULT  fault at     72 us, C14 after the fault ff
  start 0x07f5 -> FAULT  fault at    152 us, C14 after the fault ff
  start 0x3ff5 -> FAULT  fault at   1218 us, C14 after the fault ff
  start 0x7ff5 -> FAULT  fault at   2430 us, C14 after the fault ff
```

14.2.111 eccmode (run 29)

ID: `D-eccmode-29`. Device run 29. Experiment: eccmode.

```
ECC_MODE sweep (decode: flipped bits: ECC_RESULT/ECC_DONE, data corrected / as sent / other):
  mode 0x0000180: END; encode done 3b; slot 0 ffffff7a82a426c457e3a863a1cb34c8 (the 0x180 code); slots 1..3 fffffffff
  fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff; 0x900..0x93c 000000000000000
  000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
      decode  0: 00000000/35 corr |  8: 00080000/35 corr |  9: 00000011/3d sent | 12: 00000011/3d sent | 16: 00000011/
  3d sent
  mode 0x0000000: END; encode done 33; slot 0 ffffffffffffffffffffffffffffffff (DIFFERENT); slots 1..3 ffffffffffffff
  fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff; 0x900..0x93c 233c853f32c8ce55a1c3
  21a1fc000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
      decode  0: 00000011/3d corr |  8: 00000011/3d sent |  9: 00000011/3d sent | 12: 00000011/3d sent | 16: 00000011/
  3d sent
  mode 0x0000080: END; encode done 3b; slot 0 ffffffffffffffffffffffffffffffff (DIFFERENT); slots 1..3 ffffffffffffff
  fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff; 0x900..0x93c dcfca60c49c28aa03983
  6fbe7c000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
      decode  0: 00000011/3d corr |  8: 00000011/3d sent |  9: 00000011/3d sent | 12: 00000011/3d sent | 16: 00000011/
  3d sent
  mode 0x0000100: END; encode done 3b; slot 0 ffffff7a82a426c457e3a863a1cb34c8 (the 0x180 code); slots 1..3 fffffffff
  fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff; 0x900..0x93c 000000000000000
  000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
      decode  0: 00000000/35 corr |  8: 00080000/35 corr |  9: 00000011/3d sent | 12: 00000011/3d sent | 16: 00000011/
  3d sent
  mode 0x00001c0: END; encode done 3b; slot 0 ffffff7a82a426c457e3a863a1cb34c8 (the 0x180 code); slots 1..3 fffffffff
  fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff; 0x900..0x93c 000000000000000
  000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
      decode  0: 00000000/35 corr |  8: 00080000/35 corr |  9: 00000011/3d sent | 12: 00000011/3d sent | 16: 00000011/
```

```
  3d sent
  mode 0x0000181: END; encode done 3b; slot 0 ffffff7a82a426c457e3a863a1cb34c8 (the 0x180 code); slots 1..3 fffffffff
  ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff; 0x900..0x93c 000000000000000
  000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
      decode  0: 00000000/35 corr |  8: 00080000/35 corr |  9: 00000011/3d sent | 12: 00000011/3d sent | 16: 00000011/
  3d sent
  mode 0x0000380: END; encode done 3b; slot 0 ffffff7a82a426c457e3a863a1cb34c8 (the 0x180 code); slots 1..3 fffffffff
  ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff; 0x900..0x93c 000000000000000
  000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
      decode  0: 00000000/35 corr |  8: 00080000/35 corr |  9: 00000011/3d sent | 12: 00000011/3d sent | 16: 00000011/
  3d sent
  mode 0x0000780: END; encode done 3b; slot 0 ffffff7a82a426c457e3a863a1cb34c8 (the 0x180 code); slots 1..3 fffffffff
  ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff; 0x900..0x93c 000000000000000
  000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
      decode  0: 00000000/35 corr |  8: 00080000/35 corr |  9: 00000011/3d sent | 12: 00000011/3d sent | 16: 00000011/
  3d sent
  mode 0x0e00000: END; encode done 1b; slot 0 ffffffffffffffffffffffffffffff (DIFFERENT); slots 1..3 ffffffffffffff
  ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff; 0x900..0x93c b5d2b4ab2eacd0192881
  214ff70000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
      decode  0: 00000011/1d corr |  8: 00000011/1d sent |  9: 00000011/1d sent | 12: 00000011/1d sent | 16: 00000011/
  1d sent
  mode 0x1ffffff: END; encode done 13; slot 0 509999fff94faba1ffff8124ffffffff (DIFFERENT); slots 1..3 ffffffffffffff
  ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff; 0x900..0x93c 8124f94faba1509999ff
  31982cb8c5549cad90d80000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
      decode  0: 00000000/15 corr |  8: 00010000/15 other |  9: 00010000/15 other | 12: 00020000/15 other | 16: 000200
  00/15 other
```

14.2.112 eccchunk (run 29)

ID: `D-eccchunk-29`. Device run 29. Experiment: eccchunk.

```
parity buffer DMA, meta encode, decode of host-built chunks:
  DMA memory -> parity buffer (DATA_SEL 0x10) -> memory: END, first 64 bytes back c5d2dfecf90613202d3a4754616e7b8895a2
  afbcc9d6e3f0fd0a1724313e4b5865727f8c99a6b3c0cddae7f4010e1b2835424f5c697683909daab7c4d1deebf8
  DMA memory -> parity buffer (DATA_SEL 0x20) -> memory: END, first 64 bytes back c5d2dfecf90613202d3a4754616e7b8895a2
  afbcc9d6e3f0fd0a1724313e4b5865727f8c99a6b3c0cddae7f4010e1b2835424f5c697683909daab7c4d1deebf8
  meta encode (META = a0..ab, 0x3210) + encode slots 0..3: END; slots: a0a1a2aa0cd773401a693eb53a632ccf(code ok) a3a4a
  5ee0a13318e6b8599c72dadb6c5(code ok) a6a7a86c8947e91865fd95df565803cb(code ok) a9aaab0199264d18b5f03addb57f8e6c(code
   ok)
  decode, clean                                                -> END   ECC_RESULT 00000011 ECC_DONE 3d, data clean, s
  lot ffffffffffffffffffffffffffffff
  decode, meta 1 bit                                           -> END   ECC_RESULT 00000011 ECC_DONE 3d, data clean, s
  lot ffffffffffffffffffffffffffffff
  decode, meta 3 bits                                          -> END   ECC_RESULT 00000011 ECC_DONE 3d, data clean, s
  lot ffffffffffffffffffffffffffffff
  decode, parity 1 bit                                         -> END   ECC_RESULT 00000011 ECC_DONE 3d, data clean, s
  lot ffffffffffffffffffffffffffffff
  decode, parity 8 bits                                        -> END   ECC_RESULT 00000011 ECC_DONE 3d, data clean, s
  lot ffffffffffffffffffffffffffffff
  decode, data 4 + parity 4                                    -> END   ECC_RESULT 00000011 ECC_DONE 3d, data as sent,
   slot ffffffffffffffffffffffffffffff
  decode, parity 9 bits                                        -> END   ECC_RESULT 00000011 ECC_DONE 3d, data clean, s
  lot ffffffffffffffffffffffffffffff
  decode, meta byte order: chunk built with the bytes reversed -> END   ECC_RESULT 00000011 ECC_DONE 3d, data clean, s
  lot ffffffffffffffffffffffffffffff
```

14.2.113 rawpage (run 29)

ID: `D-rawpage-29`. Device run 29. Experiment: rawpage.

```
raw page reads, CE0 (chunk as stored, checked against the identified code; res = engine decode, st = FMC_STATUS after
  the sector's data):
  row 0x000000 END   s0 erased                res 00000000 st 0003c000 chunk ffffffffffffffffffffffffffffffff
                   s1 erased                res 00000000 st 0003c000 chunk ffffffffffffffffffffffffffffffff
                   s2 erased                res 00000000 st 0003c000 chunk ffffffffffffffffffffffffffffffff
                   s3 erased                res 00000000 st 0003c000 chunk ffffffffffffffffffffffffffffffff
  row 0x000080 END   s0 code ok                res 00000000 st 0803c000 chunk fffff26426253358a227a1c8f3335a1
                   s1 code ok               res 00000000 st 0803c000 chunk ffffff3e77a3b310f3a2a20395982422
                   s2 code ok               res 00000000 st 0803c000 chunk ffff00620004b809a9fcc776ac46a657
                   s3 code ok               res 00000000 st 0803c000 chunk 80ffffaa9f4f48da3bf3aa151041f41b
  row 0x003201 END   s0 code ok                res 00000000 st 0803c000 chunk 04140258c636bf190b43b15561dbd1d4
                   s1 code ok               res 00000000 st 0803c000 chunk 00ff34dcdefb0beb4fa0ab138f638dea
                   s2 code ok               res 00000000 st 0803c000 chunk 0100ff43691540bf75e726d04dcade29
                   s3 code ok               res 00000000 st 0803c000 chunk 40ffff92bf5756e072be0ee05bb953af
  row 0x01f400 END   s0 code ok                res 00000000 st 0803c000 chunk 00d211c8cfcc6399cf5e09fb0c07f760
                   s1 code ok               res 00000000 st 0803c000 chunk 008a78afd219b4b2b6eeb1b12e15474c
                   s2 code ok               res 00000000 st 0803c000 chunk 0100ff603c9955c82679227a222659c8
                   s3 code ok               res 00000000 st 0803c000 chunk 40fffff6cb3c9bc78c119b5d5149396f
  row 0x03e805 END   s0 code ok                res 00000000 st 0803c000 chunk 14cc18d7420fb10f9e743d3f24917b80
                   s1 code ok               res 00000000 st 0803c000 chunk 00a3103cbdfd4ad044e1a8005add4875
                   s2 code ok               res 00000000 st 0803c000 chunk 0100ff8af264ab05b72499e4a88f3d50
                   s3 code ok               res 00000000 st 0803c000 chunk 40ffff47478af4bc6cb7daef940d7097
  row 0x06d600 END   s0 code ok                res 00000000 st 0803c000 chunk 00ba1a48cc88d9cec52dda6d0ab228fd
                   s1 code ok               res 00000000 st 0803c000 chunk 00f2379e30a95091c82dd4cf484a2496
                   s2 code ok               res 00000000 st 0803c000 chunk 0000ff64a6e3b23c54a70b948491c0f7
                   s3 code ok               res 00000000 st 0803c000 chunk 40ffff47478af4bc6cb7daef940d7097
```

14.2.114 rawwrite (run 29)

ID: `D-rawwrite-29`. Device run 29. Experiment: rawwrite.

```
scratch block write paths and raw read-back (res = engine decode, st = FMC_STATUS after the data):
  erase block: END, NSTAT c0
  p0 write (engine: meta encode + encode): END, NSTAT 70
  p1 write (host chunks via parity-buffer DMA): END, NSTAT 70
  p2 write (raw: chunks from a data buffer): END, NSTAT 70
  p3 write (raw, stored errors: 8 / 9 data bits, zero chunk, 2 meta bits): END, NSTAT 70
  p0 s0 code ok                res 00000000 st 0803c000 chunk 4142430648a7fd436a0dee8ab9ada46b data = sent; meta bytes
   414243 (sent 414243)
  p0 s1 code ok                res 00000000 st 0803c000 chunk 44454677ea778d147a2fcf612a367686 data = sent; meta bytes
   444546 (sent 444546)
  p0 s2 code ok                res 00000000 st 0803c000 chunk 4748492e5699c1d567692e96ccbd6e9c data = sent; meta bytes
   474849 (sent 474849)
  p0 s3 code ok                res 00000000 st 0803c000 chunk 4a4b4c0b3164549d25efb6b1c911aaee data = sent; meta bytes
   4a4b4c (sent 4a4b4c)
  p1 s0 NOT a code word        res 00000011 st 0803c000 chunk ffffffffffffffffffffffffffffffff stored data =, chunk ff
  ffffffffffffffffffffffffffffffff (sent 5152535212d40d397543e85a7e77add2)
  p1 s1 NOT a code word        res 00000011 st 0803c000 chunk ffffffffffffffffffffffffffffffff stored data =, chunk ff
  ffffffffffffffffffffffffffffffff (sent 545556dd1fc4a9255e3d81800b1abd7c)
  p1 s2 NOT a code word        res 00000011 st 0803c000 chunk ffffffffffffffffffffffffffffffff stored data =, chunk ff
  ffffffffffffffffffffffffffffffff (sent 575859de0ebed657e57ce2bf076fa9ba)
  p1 s3 NOT a code word        res 00000011 st 0803c000 chunk ffffffffffffffffffffffffffffffff stored data =, chunk ff
  ffffffffffffffffffffffffffffffff (sent 5a5b5c543088613bc2cb500867a47ec2)
  p2 s0 code ok                res 00000000 st 0803c000 chunk 6162631dd97391e8540db3213763a487 stored = sent
  p2 s1 code ok                res 00000000 st 0803c000 chunk 646566b00b3a1f2b2c63e8dfe437e293 stored = sent
  p2 s2 code ok                res 00000000 st 0803c000 chunk 676869d9a73e1362d1fc3d945b763539 stored = sent
  p2 s3 code ok                res 00000000 st 0803c000 chunk 6a6b6cdfb63a2fa44c0ffd8760a75c0b stored = sent
  p3 s0 NOT a code word        res 00080000 st 0803c000 chunk 717273bde0bf8e08c0d201d5e5f1ed1c stored = sent; after de
  code clean
  p3 s1 NOT a code word        res 00000011 st 0803c000 chunk 7475760b30bb75534fa12695523e478e stored = sent; after de
  code as stored
  p3 s2 NOT a code word        res 00000011 st 0803c000 chunk 00000000000000000000000000000000 stored = sent; after de
  code clean
  p3 s3 NOT a code word        res 00020000 st 0803c000 chunk 7e7b3ca20e7f5dc82ce0e82e8d1d9744 stored = sent; after de
  code clean
  p4 s0 erased                 res 00000000 st 0003c000 chunk ffffffffffffffffffffffffffffffff
  p4 s1 erased                 res 00000000 st 0003c000 chunk ffffffffffffffffffffffffffffffff
  p4 s2 erased                 res 00000000 st 0003c000 chunk ffffffffffffffffffffffffffffffff
  p4 s3 erased                 res 00000000 st 0003c000 chunk ffffffffffffffffffffffffffffffff
```

14.2.115 writeseq (run 29)

ID: `D-writeseq-29`. Device run 29. Experiment: writeseq.

```
write sequences (scratch blocks), raw read-back:
  erase CE0 block 3000: FAULT, NSTAT after D0h c0, ready c0, 6003 us
  erase CE0 block 3001: FAULT, NSTAT after D0h c0, ready c0, 6009 us
  erase CE1 block 3000: FAULT, NSTAT after D0h c0, ready c0, 6029 us
  (1) 4 pages, 0x2e4 overlap, DMA_CTRL 7: END, NSTAT per page 70 70 70 70, 941 us
  read (1) CE0 page 10                    all 4 sectors = expected (data + meta + parity); ECC_RESULT 0 0 0 0
  read (1) CE0 page 11                    sectors [0, 1, 2, 3] DIFFER: ffffffffffffffffffffffffffffffff ffffffffffffff
  ffffffffffffffff ffffffffffffffffffffffffffffffff ffffffffffffffffffffffffffffffff; ECC_RESULT 0 0 0 0
  read (1) CE0 page 12                    all 4 sectors = expected (data + meta + parity); ECC_RESULT 0 0 0 0
  read (1) CE0 page 13                    sectors [0, 1, 2, 3] DIFFER: ffffffffffffffffffffffffffffffff ffffffffffffff
  ffffffffffffffff ffffffffffffffffffffffffffffffff ffffffffffffffffffffffffffffffff; ECC_RESULT 0 0 0 0
  (2) CE0 + CE1 in one run: END, NSTAT 70 / 70
  read (2) CE0 page 20                    all 4 sectors = expected (data + meta + parity); ECC_RESULT 0 0 0 0
  read (2) CE1 page 20                    all 4 sectors = expected (data + meta + parity); ECC_RESULT 0 0 0 0
  (3) two-plane 80h..11h / 81h..10h: END, NSTAT 70 / 70, 473 us
  read (3) block B page 30                all 4 sectors = expected (data + meta + parity); ECC_RESULT 0 0 0 0
  read (3) block B+1 page 30              all 4 sectors = expected (data + meta + parity); ECC_RESULT 0 0 0 0
  (4) page 10 programmed again without an erase: END, NSTAT 70 (b0 = FAIL)
    s0 data = old AND new, ECC_RESULT 00000011
    s1 data = old AND new, ECC_RESULT 00000011
    s2 data = old AND new, ECC_RESULT 00000011
    s3 data = old AND new, ECC_RESULT 00000011
```

14.2.116 op09nand (run 29)

ID: `D-op09nand-29`. Device run 29. Experiment: op09nand.

```
wait2 against chip activity (scratch blocks):
  erase running     wait2 passes for: none   [N = 0: fault at 4868 us, NSTAT after c0]
  erase finished    wait2 passes for: none   [N = 0: fault at 5028 us, NSTAT after c0]
  program running   wait2 passes for: none   [N = 0: fault at 4910 us, NSTAT after 70]
  program finished  wait2 passes for: none   [N = 0: fault at 4908 us, NSTAT after 70]
  read tR running   wait2 passes for: none   [N = 0: fault at 4868 us, NSTAT after c0]
  idle              wait2 32..255 passes for: none
  erase running     wait2 32..255 passes for: none
```

14.2.117 eraseseq (run 29)

ID: D-eraseseq-29. Device run 29. Experiment: eraseseq.

```
erase (scratch blocks; per page: four sectors' state):
    before: CE0 block B page 10                NOT a code word NOT a code word NOT a code word NOT a code word
    before: CE0 block B page 20                code ok code ok code ok code ok
    before: CE0 block B page 30                code ok code ok code ok code ok
  erase CE0 block B (written pages)            FAULT, NSTAT after D0h c0, ready c0,  6002 us
    after: CE0 block B page 10                 erased erased erased erased
    after: CE0 block B page 20                 erased erased erased erased
    after: CE0 block B page 30                 erased erased erased erased
  erase CE0 block B again (already erased)     FAULT, NSTAT after D0h c0, ready c0,  5052 us
    before: CE0 block B+1 page 30              code ok code ok code ok code ok
  two-plane erase B, B+1                       FAULT, NSTAT after D0h c0, ready c0,  6016 us
    after: CE0 block B+1 page 30               erased erased erased erased
    before: CE1 block B page 20                code ok code ok code ok code ok
  erase CE1 block B                            FAULT, NSTAT after D0h c0, ready c0,  6015 us
    after: CE1 block B page 20                 erased erased erased erased
  erase B+2 with FMC_ANUM = 3                  END, ready c0,  1141 us (a 4th address byte: accepted / ignored?)
```

14.2.118 eccchunk2 (run 30)

ID: D-eccchunk2-30. Device run 30. Experiment: eccchunk2.

```
decode of host-built chunks in parity slot 0 (slot = after the decode):
  clean                                          -> END   ECC_RESULT 00000000 ECC_DONE 35, data clean, slot clean
  meta 1 bit                                     -> END   ECC_RESULT 00010000 ECC_DONE 35, data clean, slot clean
  meta 3 bits                                    -> END   ECC_RESULT 00030000 ECC_DONE 35, data clean, slot clean
  parity 1 bit                                   -> END   ECC_RESULT 00010000 ECC_DONE 35, data clean, slot clean
  parity 8 bits                                  -> END   ECC_RESULT 00080000 ECC_DONE 35, data clean, slot clean
  data 4 + parity 4                              -> END   ECC_RESULT 00080000 ECC_DONE 35, data clean, slot clean
  data 5 + meta 3                                -> END   ECC_RESULT 00080000 ECC_DONE 35, data clean, slot clean
  parity 9 bits                                  -> END   ECC_RESULT 00000011 ECC_DONE 3d, data clean, slot as sent
  meta byte order: built with the bytes reversed -> END   ECC_RESULT 00040000 ECC_DONE 35, data clean, slot 56341245
  a321cc53d4692df9359caeb7
```

14.2.119 eccmode2 (run 30)

ID: D-eccmode2-30. Device run 30. Experiment: eccmode2.

```
ECC_MODE map (encode only; bit numbering of the fit sectors: bit 0 = MSB of byte 0):
  0x0000180: END ECC_DONE 33 ECC_RESULT 00040000 slot0 ffffff7a82a426c457e3a863a1cb34c8 area c834cba163a8e357c426a4827
  a0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000003181884fc82804
  0x0000380: END ECC_DONE 33 ECC_RESULT 00040000 slot0 ffffff7a82a426c457e3a863a1cb34c8 area c834cba163a8e357c426a4827
  a0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000003181884fc82804  (= 0x180)
  0x0000580: END ECC_DONE 33 ECC_RESULT 00040000 slot0 ffffff7a82a426c457e3a863a1cb34c8 area c834cba163a8e357c426a4827
  a0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000003181884fc82804  (= 0x180)
  0x0000980: END ECC_DONE 33 ECC_RESULT 00040000 slot0 ffffff7a82a426c457e3a863a1cb34c8 area c834cba163a8e357c426a4827
  a0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000003181884fc82804  (= 0x180)
  0x0001180: END ECC_DONE 33 ECC_RESULT 00040000 slot0 ffffff7a82a426c457e3a863a1cb34c8 area c834cba163a8e357c426a4827
  a0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000003181884fc82804  (= 0x180)
  0x0002180: END ECC_DONE 33 ECC_RESULT 00040000 slot0 ffffff7a82a426c457e3a863a1cb34c8 area c834cba163a8e357c426a4827
  a0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000003181884fc82804  (= 0x180)
  0x0004180: END ECC_DONE 33 ECC_RESULT 00040000 slot0 ffffff7a82a426c457e3a863a1cb34c8 area c834cba163a8e357c426a4827
  a0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000003181884fc82804  (= 0x180)
  0x0008180: END ECC_DONE 33 ECC_RESULT 00020000 slot0 522ee44c7cc113acebcdfffffffffff area cdebac13c17c4ce42e52c4ce9
  2066e7b38caf046000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  000000000000000000000000000008bc68bc6a7b1a7b1a7b1a7b1a7b1a7b13181884fc82804
  0x0010180: END ECC_DONE 33 ECC_RESULT 00040000 slot0 ffffff5f3e6308be82eb5ae2251dc2c4 area c4c21d25e25aeb82be08633e5
  f0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000003181884fc82804
  0x0020180: END ECC_DONE 33 ECC_RESULT 00040000 slot0 fffffcb5bda54c8635dfc5aabb10a78 area 780ab1ab5afc5d63c854da5bc
  b0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000003181884fc82804
  0x0040180: END ECC_DONE 33 ECC_RESULT 00040000 slot0 ffffff33836b2964d0c95b1d4032d686 area 86d632401d5bc9d064296b833
  3000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000003181884fc82804
  0x0080180: END ECC_DONE 33 ECC_RESULT 00040000 slot0 ffffff098b982a1c282601651c2202c8 area c802221c650126281c2a988b0
  9000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
```

```
0000000000
```

```
000000000fa52fa52000000000000000000000003181884fc82804
  fit 0x0000080 bit 4095 slot0 ffffffffffffffffffffffffffffffff area b0c691ff3e6a42d353fb5a0bcb00000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000a873a8730000000000000000000003181884fc82804
  fit 0x0e00000 zeros    slot0 ffffffffffffffffffffffffffffffff area 4684475615107a7cfd75b3cebe00000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000007dcb7dcb0000000000000000000003181884fc82804
  fit 0x0e00000 bit 0    slot0 ffffffffffffffffffffffffffffffff area 5da0a18c3e1f8acedf03f2ac68000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000001be61be6000000000000000000000003181884fc82804
  fit 0x0e00000 bit 1    slot0 ffffffffffffffffffffffffffffffff area b1250ddf69ebfa90821e21278300000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000817881780000000000000000000003181884fc82804
  fit 0x0e00000 bit 8    slot0 ffffffffffffffffffffffffffffffff area a49605af7f7209f9bc12b71a3a00000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000016df16df0000000000000000000003181884fc82804
  fit 0x0e00000 bit 9    slot0 ffffffffffffffffffffffffffffffff area 5f6698cd901dfb497dc805f4f80000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000536e536e0000000000000000000003181884fc82804
  fit 0x0e00000 bit 4095 slot0 ffffffffffffffffffffffffffffffff area 56fb4de8fda80f967a303d769f00000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000008f828f8200000000000000000000003181884fc82804
  fit 0x1ffffff zeros    slot0 23113931a4371b24ffffeaeaffffffff area eaeaa4371b242311393165bfd8a29bf6357775910000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000fc3bfc3bfc3bfc3bfc3bfc3bfc3bfc3b3181884fc82804
  fit 0x1ffffff bit 0    slot0 2e4c25eee8c1cc9affff2e05ffffffff area 2e05e8c1cc9a2e4c25eed298a1d21422673a3ac00000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000feecfeecfeecfeecfeecfeecfeecfeec3181884fc82804
  fit 0x1ffffff bit 1    slot0 b77f975e8aecf1fbffff088fffffffff area 088f8aecf1fbb77f975eba3ca41a5d9c9cd1f2380000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000007d507d507d507d507d507d507d507d503181884fc82804
  fit 0x1ffffff bit 8    slot0 5eeff873720d7b17ffff76ceffffffff area 76ce720d7b175eeff873196046465acb673c40ac0000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000e3b0e3b0e3b0e3b0e3b0e3b0e3b0e3b3181884fc82804
  fit 0x1ffffff bit 9    slot0 0dae59918f0a2bb9ffffa4eaffffffff area a4ea8f0a2bb90dae5991dbc2d7507aec9ed2ef8f0000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000853b853b853b853b853b853b853b853b3181884fc82804
  fit 0x1ffffff bit 4095 slot0 23113931a4371b24ffffeaeaffffffff area eaeaa4371b242311393165bfd8a29bf6357775910000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000fc3bfc3bfc3bfc3bfc3bfc3bfc3bfc3b3181884fc82804
  fit 0x0008180 zeros    slot0 00000000000000000000ffffffffffff area 00000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000fc3bfc3bfc3bfc3bfc3b3181884fc82804
  fit 0x0008180 bit 0    slot0 86f49e603532a5c87af7ffffffffffff area f77ac8a53235609ef486282cc4f7ed491edc23b50000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000002f822f82fc3bfc3bfc3bfc3bfc3bfc3b3181884fc82804
  fit 0x0008180 bit 1    slot0 c27a07321e1952e43f7fffffffffffff area 7f3fe452191e32077ac21404eadbf7201faeb15a0000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000015811581fc3bfc3bfc3bfc3bfc3bfc3b3181884fc82804
  fit 0x0008180 bit 8    slot0 0a235adfde97782d116fffffffffffff area 6f112d7897dedf5a230a0bf17d7b9207f7482dd50000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000f200f2fc3bfc3bfc3bfc3bfc3bfc3b3181884fc82804
  fit 0x0008180 bit 9    slot0 0531ed7defca9c168ab3ffffffffffff area b38a169ccaef7ded310581faf63dc8876bac966a0000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
000000000000790079fc3bfc3bfc3bfc3bfc3b3181884fc82804
  fit 0x0008180 bit 4095 slot0 3ff0a6afee076a5a6590ffffffffffff area 90655a6a07eeafa6f03f00000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000005800580fc3bfc3bfc3bfc3bfc3bfc3b3181884fc82804
  fit 0x0010180 zeros    slot0 ffffff9bbcef60c75438704874365886 area 8658367448703854c760efbc9b0000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000003181884fc82804
  fit 0x0010180 bit 0    slot0 ffffffff302601dbcad8eb6e8ffba1130 area 3011baffe8b68eadbc1d6002f30000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000003181884fc82804
```

```
    fit 0x0010180 bit 1    slot0 ffffffafe3a8de7aa8e3131831f07c5d area 5d7cf0311813e3a87adea8e3af00000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000003181884fc82804
    fit 0x0010180 bit 8    slot0 ffffff9bbcef60c75438704874365886 area 8658367448703854c760efbc9b00000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000003181884fc82804
    fit 0x0010180 bit 9    slot0 ffffff9bbcef60c75438704874365886 area 8658367448703854c760efbc9b00000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000003181884fc82804
    fit 0x0010180 bit 4095 slot0 ffffff9bbcef60c75438704874365886 area 8658367448703854c760efbc9b00000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000003181884fc82804
    fit 0x0020180 zeros    slot0 ffffffb8ce0bfebef248feaeba8e8166 area 66818ebaaefe48f2befe0bceb800000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000003181884fc82804
    fit 0x0020180 bit 0    slot0 ffffffd0708483c50bfe380e3102c8d0 area d0c802310e38fe0bc5838470d00000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000003181884fc82804
    fit 0x0020180 bit 1    slot0 ffffff8c914c40030e939dfeff48a5bd area bda548fffe9d930e03404c918c00000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000003181884fc82804
    fit 0x0020180 bit 8    slot0 fffffffeeac932e1a0c6cb6c2f4a3f5a3 area a3f5a3f4c2b66c0c1a2e93acee00000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000003181884fc82804
    fit 0x0020180 bit 9    slot0 ffffff1903cde6d10b5319387f7a4695 area 95467a7f3819530bd1e6cd03190000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000003181884fc82804
    fit 0x0020180 bit 4095 slot0 ffffffb8ce0bfebef248feaeba8e8166 area 66818ebaaefe48f2befe0bceb800000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  00000000000000000000000000000000000000003181884fc82804
```

14.2.120 tmobits2 (run 30)

ID: `D-tmobits2-30`. Device run 30. Experiment: tmobits2.

```
timeout length, bits 7:4 = 5..7:
  start 0xff55 -> FAULT  fault at   2432 us (n = 5; 2**11 per step would be 4855 us)
  start 0xff65 -> FAULT  fault at   4868 us (n = 6; 2**11 per step would be 4855 us)
  start 0xff75 -> FAULT  fault at   4868 us (n = 7; 2**11 per step would be 4855 us)
  start 0x0f55 -> FAULT  fault at    152 us (n = 5; 2**11 per step would be 303 us)
  start 0x0f75 -> FAULT  fault at    310 us (n = 7; 2**11 per step would be 303 us)
```

14.2.121 ev5ready (run 30)

ID: `D-ev5ready-30`. Device run 30. Experiment: ev5ready.

```
status latch event (5) against chip activity (timeouts ~4.87 ms each):
  idle, after a soft reset: 1 latch                END,    17 us, 0 timeout(s)
      latch 0: STATUS 008c3000 NSTAT c0 -> after the wait 008c3000
  idle, no soft reset: 1 latch                     END,    18 us, 0 timeout(s)
      latch 0: STATUS 008c3000 NSTAT c0 -> after the wait 008c3000
  idle, no soft reset: 1 latch (again)             END,    17 us, 0 timeout(s)
      latch 0: STATUS 008c3000 NSTAT c0 -> after the wait 008c3000
  idle: 3 latches                                  FAULT,  9739 us, 2 timeout(s)
      latch 0: STATUS 008c3000 NSTAT c0 -> after the wait 008c3000
      latch 1: STATUS 000c3000 NSTAT c0 -> after the wait 000c3000
      latch 2: STATUS 000c3000 NSTAT c0 -> after the wait 000c3000
  FFh reset, then 2 latches                        FAULT,  4884 us, 1 timeout(s)
      latch 0: STATUS 008c3000 NSTAT c0 -> after the wait 008c3000
      latch 1: STATUS 000c3000 NSTAT c0 -> after the wait 000c3000
  erase, then 3 latches                            FAULT, 10883 us, 2 timeout(s)
      latch 0: STATUS 000c3000 NSTAT 81 -> after the wait 008c3000
      latch 1: STATUS 000c3000 NSTAT c0 -> after the wait 000c3000
      latch 2: STATUS 000c3000 NSTAT c0 -> after the wait 000c3000
  erase, NSTAT polled without waits, then 1 latch  END,   183 us, 0 timeout(s)
      latch 0: STATUS 008c3000 NSTAT c0 -> after the wait 008c3000
  read page (00h .. 30h), then 2 latches           FAULT,  4911 us, 1 timeout(s)
      latch 0: STATUS 000c3000 NSTAT 80 -> after the wait 008c3000
      latch 1: STATUS 000c3000 NSTAT c0 -> after the wait 000c3000
```

14.2.122 pageorder (run 30)

ID: `D-pageorder-30`. Device run 30. Experiment: pageorder.

```
page program order (scratch blocks), read back raw:
  (a) block 3008 pages [0, 1, 2, 3, 4, 5, 6, 7]: END, 1872 us; page 0 ok (NSTAT 70), page 1 ERASED (NSTAT 70), page 2
```

```
  ok (NSTAT 70), page 3 ERASED (NSTAT 70), page 4 ok (NSTAT 70), page 5 ERASED (NSTAT 70), page 6 ERASED (NSTAT 70), p
  age 7 ERASED (NSTAT 70)
  (b) block 3009 pages [5]: END, 249 us; page 5 ok (NSTAT 70)
  (c) block 3010 pages [4]: END, 250 us; page 4 ok (NSTAT 70)
  (d) block 3011 pages [4, 5]: END, 473 us; page 4 ok (NSTAT 70), page 5 ERASED (NSTAT 70)
  (e) block 3012 pages [1, 3]: END, 474 us; page 1 ok (NSTAT 70), page 3 ERASED (NSTAT 70)
  (f) erase with 4 address cycles: END, 1141 us; page 0 afterwards: erased erased erased erased
```

14.2.123 op09w33 (run 30)

ID: `D-op09w33-30`. Device run 30. Experiment: op09w33.

```
wait2, idle chip (no other waits in the program):
  wait2  33 (0b00100001): passes, fault at 0 us
  wait2   5 (0b00000101): passes, fault at 0 us
  wait2   0 (0b00000000): times out, fault at 4868 us
  wait2   1 (0b00000001): times out, fault at 4868 us
  wait2   4 (0b00000100): times out, fault at 4868 us
  wait2  32 (0b00100000): times out, fault at 4868 us
  wait2  37 (0b00100101): times out, fault at 4868 us
  wait2  65 (0b01000001): times out, fault at 4868 us
  wait2  69 (0b01000101): times out, fault at 4868 us
  wait2  97 (0b01100001): times out, fault at 4854 us
  wait2 133 (0b10000101): times out, fault at 4856 us
  wait2 161 (0b10100001): times out, fault at 4868 us
  wait2 165 (0b10100101): times out, fault at 4868 us
```

14.2.124 ev5arm (run 31)

ID: `D-ev5arm-31`. Device run 31. Experiment: ev5arm.

```
what re-arms event 5 inside a program (each line: latch, X, latch; a timeout ~4.87 ms):
  latch, nothing,                     latch: second wait timed out ( 4870 us); STATUS after latch 1 000c3000, NSTAT c0
  latch, set-up word,                 latch: second wait PASSED (   17 us); STATUS after latch 1 008c3000, NSTAT c0
  latch, status-clear word,           latch: second wait timed out ( 4896 us); STATUS after latch 1 000c3000, NSTAT c0
  latch, set-up + status-clear,       latch: second wait PASSED (   17 us); STATUS after latch 1 008c3000, NSTAT c0
  latch, CE deselect / reselect,      latch: second wait timed out ( 4896 us); STATUS after latch 1 000c3000, NSTAT c0
  latch, FMCTRL1 0x100e0,             latch: second wait PASSED (   17 us); STATUS after latch 1 008c3000, NSTAT c0
  latch, 16-byte read from the chip,  latch: second wait PASSED (   18 us); STATUS after latch 1 008c3000, NSTAT c0
  latch, ECC kick,                    latch: second wait timed out ( 4905 us); STATUS after latch 1 000c3000, NSTAT c0
  latch, DMA,                         latch: second wait PASSED (   18 us); STATUS after latch 1 008c3000, NSTAT c0
  latch with DATA_SEL 0x100, then a normal latch: 4883 us; latch 0 STATUS 008c3000 NSTAT 70, latch 1 STATUS 000c3000 N
  STAT c0
```

14.2.125 writeseq2 (run 31)

ID: `D-writeseq2-31`. Device run 31. Experiment: writeseq2.

```
multi-page program runs with the status poll on FMC_DATA_SEL = 1 (ready byte; b0 = FAIL):
  8 pages in order                END,  5827 us: page 0 ok (ready c0, just after 10h 81), page 1 ok (ready c0, just a
  fter 10h 81), page 2 ok (ready c0, just after 10h 81), page 3 ok (ready c0, just after 10h 81), page 4 ok (ready c0,
   just after 10h 81), page 5 ok (ready c0, just after 10h 81), page 6 ok (ready c0, just after 10h 81), page 7 ok (re
  ady c0, just after 10h 81)
  pages 9, 11                     END,  2019 us: page 9 ok (ready c0), page 11 ok (ready c0)
  page 0 again without an erase   END,   464 us: page 0 DIFFERENT (ready c0, just after 10h 81)
  pages 1, 3 of an empty block    END,   919 us: page 1 ok (ready c0), page 3 ok (ready c0)
```

14.2.126 eccmode3 (run 31)

ID: `D-eccmode3-31`. Device run 31. Experiment: eccmode3.

```
ECC_MODE length field and samples:
  b20:16 =  3: encodes the first 3 bytes
  b20:16 =  5: encodes the first 5 bytes
  b20:16 = 12: encodes the first 12 bytes
  b20:16 = 31: encodes the first 31 bytes
  b20:16 = 1 plus 0x0200000: encodes the first 1 bytes
  b20:16 = 1 plus 0x0400000: encodes the first 1 bytes
  b20:16 = 1 plus 0x1000000: encodes the first 1 bytes
  b20:16 = 1 plus 0x03f0000: encodes the first 31 bytes
  decode with b20:16 = 16, byte 100 changed (8 bits): ECC_RESULT 00000000 ECC_DONE 35
  decode with b20:16 = 16, byte 5 changed (8 bits): ECC_RESULT 00080000 ECC_DONE 35
  parity samples for 0x8180: zeros, bits 0..15, 4080..4095
  parity samples for 0x0: zeros, bits 0..15, 4080..4095
  parity samples for 0x80: zeros, bits 0..15, 4080..4095
```

14.2.127 ev5two (run 32)

ID: `D-ev5two-32`. Device run 32. Experiment: ev5two.

```
event 5 and a second chip (each poll waits on event 5):
  program CE0 + CE1, poll CE0, poll CE1    END,   350 us (0 timeout(s)); poll 1 NSTAT 81 -> c0, poll 2 NSTAT 81 -> c0
  program CE0 + CE1, poll CE1, poll CE0    FAULT,  5224 us (1 timeout(s)); poll 1 NSTAT 81 -> c0, poll 2 NSTAT c0 -> c
  0
  idle: latch, address cycles, latch       END,    17 us (0 timeout(s)); poll 1 NSTAT c0 -> c0, poll 2 NSTAT c0 -> c0
```

14.2.128 b15ecc (run 32)

ID: `D-b15ecc-32`. Device run 32. Experiment: b15ecc.

```
ECC_MODE b15 code, strength (encode and decode in one program) and samples:
  0 bits, one per symbol         -> END   ECC_RESULT 00000000 ECC_DONE 35, data corrected
  1 bits, one per symbol         -> END   ECC_RESULT 00010000 ECC_DONE 35, data corrected
  2 bits, one per symbol         -> END   ECC_RESULT 00020000 ECC_DONE 35, data corrected
  4 bits, one per symbol         -> END   ECC_RESULT 00040000 ECC_DONE 35, data corrected
  8 bits, one per symbol         -> END   ECC_RESULT 00000011 ECC_DONE 3d, data as sent
  9 bits, one per symbol         -> END   ECC_RESULT 00000011 ECC_DONE 3d, data as sent
  10 bits, one per symbol        -> END   ECC_RESULT 00000011 ECC_DONE 3d, data as sent
  12 bits, one per symbol        -> END   ECC_RESULT 00000011 ECC_DONE 3d, data as sent
  16 bits, one per symbol        -> END   ECC_RESULT 00000011 ECC_DONE 3d, data as sent
  17 bits, one per symbol        -> END   ECC_RESULT 00000011 ECC_DONE 3d, data as sent
  burst: 10 bits in 1 symbol     -> END   ECC_RESULT 00020000 ECC_DONE 35, data corrected
  burst: 40 bits in 4 symbols    -> END   ECC_RESULT 00000011 ECC_DONE 3d, data as sent
  burst: 80 bits in 8 symbols    -> END   ECC_RESULT 00000011 ECC_DONE 3d, data as sent
  burst: 90 bits in 9 symbols    -> END   ECC_RESULT 00000011 ECC_DONE 3d, data as sent
  parity samples: one bit at bytes 0..511 (bit 0 and 7), and two bits together
```

14.2.129 eccmode4 (run 33)

ID: `D-eccmode4-33`. Device run 33. Experiment: eccmode4.

```
ECC_MODE leftovers (area = 0x900..0x913 after the run):
  0x0 ramp, after a soft reset (1)   ECC_DONE 3b area 233c853f32c8ce55a1c321a1fc00000000000000 0x990 f0f4f0f4
  0x0 ramp, after a soft reset (2)   ECC_DONE 3b area 2e1c79573386228532df6ca2da00000000000000 0x990 5bb85bb8
  0x0 ramp, no reset (3)             ECC_DONE 3b area 7b55880c275b93c33a7204f80700000000000000 0x990 47344734
  0x0 ramp, no reset (4)             ECC_DONE 3b area 611570dd25c74b621d4b9efe4b00000000000000 0x990 f6adf6ad
  0x0 zeros                          ECC_DONE 3b area e87c56af4cfa3be376f15bb3e400000000000000 0x990 3d6d3d6d
  0x0 bit 0                          ECC_DONE 3b area dcfca60c49c28aa039836fbe7c00000000000000 0x990 73537353
  0x0 bit 1                          ECC_DONE 3b area edd42e2dda3f79ae9517f0dc3700000000000000 0x990 aa32aa32
  0x0 bits 0 + 1                     ECC_DONE 3b area a62f0baf90c8082570138c3f1200000000000000 0x990 83d683d6
  0x0 ramp after a BCH decode        ECC_DONE 3b area c47f1becb633ef38283ab3fa8400000000000000 0x990 b5b6b5b6
  0x0 ramp after a BCH encode        ECC_DONE 3b area 528950e823dd0c2ee3334b3ccf00000000000000 0x990 854a854a
  0x80 ramp                          ECC_DONE 3b area b5d2b4ab2eacd0192881214ff700000000000000 0x990 ce7fce7f
  0x80 ramp, then a decode           ECC_DONE 3b area 993f23a304711734be92d1c26000000000000000 0x990 ca68ca68
  0xe00000 ramp                      ECC_DONE 1b area 5788eb241e93af5b28f704241000000000000000 0x990 ca68ca68
  0x8180 ramp, encode only           ECC_DONE 3b area cdebac13c17c4ce42e52c4ce92066e7b38caf046 0x990 c68bc68b
  0x8180 ramp, then a decode         ECC_DONE 3b area cdebac13c17c4ce42e52c4ce92066e7b38caf046 0x990 c68bc68b
  0x8180 zeros                       ECC_DONE 33 area 000000000000000000000000000000000000000000000000 0x990 00000000
  mode 0x01f8100 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01f8100 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x01f8101 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01f8101 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x01f8102 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01f8102 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x01f8104 zeros area c44e5623898b4ca58d3c9981945cce71fcc564a0
  mode 0x01f8104 ramp  area dbed7a741a9b52038eddd591b20d29ef46914fc1
  mode 0x01f8108 zeros area 1b8814333def7d5c00aec54b302f429dcb9900e1
  mode 0x01f8108 ramp  area 7c4f08935a9d1648df6737804b2631a9f38e5f31
  mode 0x01f8110 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01f8110 ramp  area 8124f94faba1509999ff31982cb8c5549cad90d8
  mode 0x01f8120 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01f8120 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x01f8140 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01f8140 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x01f8180 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01f8180 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x01f8300 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01f8300 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x01f8500 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01f8500 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x01f8900 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01f8900 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x01f9100 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01f9100 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x01fa100 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01fa100 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x01fc100 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x01fc100 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x03f8100 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x03f8100 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x05f8100 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x05f8100 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x09f8100 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x09f8100 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
  mode 0x11f8100 zeros area eaeaa4371b242311393165bfd8a29bf635777591
  mode 0x11f8100 ramp  area 70eb4fd4a29b8a816cfb7318a03a023bd1f0a127
```

14.2.130 finale (run 34)

ID: `D-finale-34`. Device run 34. Experiment: finale.

```
final open points:
(1) ECC_MODE b1..b5:
  0x00182: zeros same, ramp same; ECC_DONE 3b; slot (ramp) ffffff7a82a426c457e3a863a1cb34c8
  0x00184: zeros same, ramp same; ECC_DONE 3b; slot (ramp) ffffff7a82a426c457e3a863a1cb34c8
  0x00188: zeros same, ramp same; ECC_DONE 3b; slot (ramp) ffffff7a82a426c457e3a863a1cb34c8
  0x00190: zeros same, ramp CHANGED; ECC_DONE 3b; slot (ramp) 9dffffff58f7b2ed3b029fe7ed47cadd
  0x001a0: zeros same, ramp same; ECC_DONE 3b; slot (ramp) ffffff7a82a426c457e3a863a1cb34c8
  0x08102: zeros same, ramp same; ECC_DONE 33; slot (ramp) 522ee44c7cc113acebcdebcdffffffff
  0x08104: zeros same, ramp CHANGED; ECC_DONE 33; slot (ramp) 186d249847a87441d927d4965ed6e4ff
  0x08108: zeros same, ramp CHANGED; ECC_DONE 33; slot (ramp) 4e2923086f4cfc4b7b77718291507950
  0x08110: zeros same, ramp CHANGED; ECC_DONE 33; slot (ramp) bc3140a699f933affff79d7ffffffff
  0x08120: zeros same, ramp same; ECC_DONE 33; slot (ramp) 522ee44c7cc113acebcdfffffffffffff
(2) END; first values: 2ca900f00bae330c7f30f0c62a b0c691ff3e6a42d353fb5a0bcb 4684475615107a7cfd75b3cebe ... (48 values
  )
(3) FMC_28 change over 32 repetitions of:
  nothing (32 x movi)            0 (before 000 after 000)
  sti to RAM                     0 (before 000 after 000)
  ldi from RAM                   0 (before 000 after 000)
  wr to the parameter RAM        0 (before 000 after 000)
  DMA RAM -> buffer              0 (before 000 after 000)
  ECC encode                     0 (before 000 after 000)
  16-byte chip read (CE0)        0 (before 000 after 000)
(4) a 2112-byte transfer from the chip with 0x1cc / 0x1d0 changed (r2 = sequencer time stamps):
  default            END,   220 us
  0x1cc = 0          END,   220 us
  0x1cc = 0xffffff   END,   221 us
  0x1d0 = 0          END,   220 us
  0x1d0 = 0xffffff   END,   220 us
(5) VIC0 line 20 / VIC1 line 22 per FMC interrupt source (0x044 enable bit, action):
  command  0x044 bit  0: CS_IRQ 11, VIC0 line 20 SET, VIC1 line 22 clear
  command  0x044 bit  1: CS_IRQ 01, VIC0 line 20 clear, VIC1 line 22 clear
  command  0x044 bit  2: CS_IRQ 01, VIC0 line 20 clear, VIC1 line 22 clear
  address  0x044 bit  0: CS_IRQ 01, VIC0 line 20 clear, VIC1 line 22 clear
  address  0x044 bit  1: CS_IRQ 11, VIC0 line 20 SET, VIC1 line 22 clear
  status   0x044 bit  0: CS_IRQ 11, VIC0 line 20 clear, VIC1 line 22 clear
  status   0x044 bit 13: CS_IRQ 01, VIC0 line 20 clear, VIC1 line 22 clear
  DMA      0x044 bit 11: CS_IRQ 11, VIC0 line 20 clear, VIC1 line 22 clear
(6) wait2 during ECC encode in flight: passes for 5 33
(6) wait2 during DMA in flight: passes for 5 33
```

14.2.131 startdiff (run 35)

ID: `D-startdiff-35`. Device run 35. Experiment: startdiff.

```
the same programs under start words differing only in bit 6 (differences only; drifting words FMC_28, 0x050, 0x9a4 lef
  t out):
  end only                                        0xfff5 vs 0xffb5: window after (a run changed it) 428 000001f8/0000
  00e8, 450 00000000/00001000, 628 00000074/00000164, 650 00000800/00001000
  end only                                        0xffd5 vs 0xff95: window after (a run changed it) 428 000001d8/0000
  00c8, 450 00001800/00002800, 628 00000054/00000144, 650 00002000/00002800
  sig, end                                        0xfff5 vs 0xffb5: window after (a run changed it) 428 000001b8/0000
  00a8, 450 00003000/00000000, 628 00000034/00000124, 650 00003800/00000000
  sig, end                                        0xffd5 vs 0xff95: window after (a run changed it) 428 00000198/0000
  0088, 450 00000800/00001800, 628 00000014/00000104, 650 00001000/00001800
  loop 2000                                       0xfff5 vs 0xffb5: window after (a run changed it) 428 00000178/0000
  0068, 450 00002000/00003000, 628 000001f4/000000e4, 650 00002000/00003000
  loop 2000                                       0xffd5 vs 0xff95: window after (a run changed it) 428 00000158/0000
  0048, 450 00003800/00000800, 628 000001d4/000000c4, 650 00003800/00000800
  wait 10 (never), C58 0                          0xfff5 vs 0xffb5: window after (a run changed it) 428 00000138/0000
  0028, 450 00001000/00002000, 628 000001b4/000000a4, 650 00001000/00002000
  wait 10 (never), C58 0                          0xffd5 vs 0xff95: window after (a run changed it) 428 00000118/0000
  0008, 450 00002800/00003800, 628 00000194/00000084, 650 00002800/00003800
  wait 10 (never), C58 4                          0xfff5 vs 0xffb5: window after (a run changed it) 428 000000f8/0000
  01e8, 628 00000174/00000064, 650 00000000/00001000
  wait 10 (never), C58 4                          0xffd5 vs 0xff95: window after (a run changed it) 428 000000d8/0000
  01c8, 450 00001800/00002000, 628 00000154/00000044, 650 00001800/00002800
  illegal 0x30, C58 4                             0xfff5 vs 0xffb5: window after (a run changed it) 428 000000b8/0000
  01a8, 450 00003000/00003800, 628 00000134/00000024, 650 00003000/00000000
  illegal 0x30, C58 4                             0xffd5 vs 0xff95: window after (a run changed it) 428 00000098/0000
  0188, 450 00000800/00001000, 628 00000114/00000004, 650 00000800/00001800
  evjmp, nothing pending                          0xfff5 vs 0xffb5: window after (a run changed it) 428 00000078/0000
  0168, 450 00002000/00002800, 628 000000f4/000001e4, 650 00002000/00002800
  evjmp, nothing pending                          0xffd5 vs 0xff95: window after (a run changed it) 428 00000058/0000
  0148, 450 00003800/00000000, 628 000000d4/000001c4, 650 00003800/00000000
  tmwait                                          0xfff5 vs 0xffb5: time 4857 / 18 us; window after (a run changed it
  ) 428 00000038/00000128, 450 00001000/00001800, 628 000000b4/000001a4, 650 00001000/00001800, c14 00000000/000000ff
  tmwait                                          0xffd5 vs 0xff95: time 4856 / 17 us; window after (a run changed it
  ) 428 00000018/00000108, 450 00002800/00003000, 628 00000094/00000184, 650 00002800/00003000, c14 00000000/000000ff
  DMA in + ECC encode                             0xfff5 vs 0xffb5: window after (a run changed it) 428 000001f8/0000
  00e8, 450 00003800/00000800, 628 00000074/00000164, 650 00000000/00000800
  DMA in + ECC encode                             0xffd5 vs 0xff95: window after (a run changed it) 428 000001d8/0000
  00c8, 450 00001000/00002000, 628 00000054/00000144, 650 00001800/00002000
  status latch, wait 5                            0xfff5 vs 0xffb5: window after (a run changed it) 428 000001b8/0000
  00a8, 450 00002800/00003800, 628 00000034/00000124, 650 00003000/00003800
  status latch, wait 5                            0xffd5 vs 0xff95: window after (a run changed it) 428 00000198/0000
  0088, 450 00000000/00001000, 628 00000014/00000104, 650 00000800/00001000
```

```
  reserved: sequencer reads                       non-zero reserved words read by the sequencer (0xfff5): 3a8=ffffffff, 3ac=ffffffff, 3b0=ffffffff, 3b4=ffffffff, 100=ffffffff, 104=ffffffff, 108=ffffffff, 10c=ffffffff, 110=ffffffff, 114=ffffffff, 118=ffffffff, 11c=ffffffff, 120=ffffffff, 124=ffffffff, 128=ffffffff, 12c=ffffffff
  reserved: sequencer reads                       0xfff5 vs 0xffb5: window after (a run changed it) 428 00000178/00000068, 450 00001800/00002800, 628 000001f4/000000e4, 650 00001800/00002800
  reserved: sequencer reads                       0xffd5 vs 0xff95: window after (a run changed it) 428 00000158/00000048, 450 00003000/00000000, 628 000001d4/000000c4, 650 00003000/00000000
  reserved: sequencer writes all ones, reads back  non-zero reserved words read by the sequencer (0xfff5): 3a8=ffffffff, 3ac=ffffffff, 3b0=ffffffff, 3b4=ffffffff, 100=ffffffff, 104=ffffffff, 108=ffffffff, 10c=ffffffff, 110=ffffffff, 114=ffffffff, 118=ffffffff, 11c=ffffffff, 120=ffffffff, 124=ffffffff, 128=ffffffff, 12c=ffffffff
  reserved: sequencer writes all ones, reads back  0xfff5 vs 0xffb5: window after (a run changed it) 428 00000138/00000028, 450 00000800/00001800, 628 000001b4/000000a4, 650 00000800/00001800
  reserved: sequencer writes all ones, reads back  0xffd5 vs 0xff95: window after (a run changed it) 428 00000118/00000008, 450 00002000/00003000, 628 00000194/00000084, 650 00002000/00003000
```

14.2.132 startbits (run 35)

ID: `D-startbits-35`. Device run 35. Experiment: startbits.

```
start-word bits flipped one at a time from 0xfff5 (differences from the baseline only):
  baseline 0xfff5 x3, run time spread (us): loop, SDRAM 811..828, loop, SRAM 497..498, 2000 ldi 1636..1641, 2000 sti 1511..1514, 2000 pld 1305..1317, sig, end 17..17, DMA + ECC encode 18..19, fault, C58 4 (vector) 4870..4883
  bit  4 (0x0000ffe5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000078, 450 00003800/00001800, 628 00000074/000000f4, 650 00000000/00001800 | loop, SRAM: window after (a run changed it) 428 000000e8/00000168, 450 00000800/00002000, 628 00000164/000001e4 | 2000 ldi: window after (a run changed it) 428 000001d8/00000058, 450 00001000/00003000, 628 00000054/000000d4, 650 00001800/00003000 | 2000 sti: window after (a run changed it) 428 000000c8/00000148, 450 00002000/00003800, 628 00000144/000001c4 | 2000 pld: window after (a run changed it) 428 000001b8/00000038, 450 00002800/00000800, 628 00000034/000000b4, 650 00003000/00000800 | sig, end: window after (a run changed it) 428 000000a8/00000128, 450 00003800/00001000, 628 00000124/000001a4 | DMA + ECC encode: window after (a run changed it) 428 00000198/00000018, 450 00000000/00002000, 628 00000014/00000094, 650 00000800/00002000
  bit  5 (0x0000ffd5): loop, SDRAM: window after (a run changed it) 650 00000000/00003800 | loop, SRAM: window after (a run changed it) 450 00000800/00000000 | 2000 ldi: window after (a run changed it) 650 00001800/00001000 | 2000 sti: window after (a run changed it) 450 00002000/00001800 | 2000 pld: window after (a run changed it) 650 00003000/00002800 | sig, end: window after (a run changed it) 450 00003800/00003000 | DMA + ECC encode: window after (a run changed it) 650 00000800/00000000
  bit  6 (0x0000ffb5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000178, 628 00000074/000001f4, 650 00000000/00001000 | loop, SRAM: window after (a run changed it) 428 000000e8/00000068, 450 00000800/00002000, 628 00000164/000000e4, 650 00000800/00002000 | 2000 ldi: window after (a run changed it) 428 000001d8/00000158, 628 00000054/000001d4, 650 00001800/00002800 | 2000 sti: window after (a run changed it) 428 000000c8/00000048, 450 00002000/00003800, 628 00000144/000000c4, 650 00002000/00003800 | 2000 pld: time 1478 us (base 1305..1317); window after (a run changed it) 428 000001b8/00000138, 628 00000034/000001b4, 650 00003000/00000000 | sig, end: window after (a run changed it) 428 000000a8/00000028, 450 00003800/00001000, 628 00000124/000000a4, 650 00003800/00001000 | DMA + ECC encode: window after (a run changed it) 428 00000198/00000118, 628 00000014/00000194, 650 00000800/00001800
  bit  7 (0x0000ff75): loop, SDRAM: window after (a run changed it) 428 000001f8/000000f8, 628 00000074/00000174, 650 00000000/00003000 | loop, SRAM: window after (a run changed it) 428 000000e8/000001e8, 450 00000800/00003800, 628 00000164/00000064, 650 00000800/00000000 | 2000 ldi: window after (a run changed it) 428 000001d8/000000d8, 450 00001000/00000800, 628 00000054/00000154, 650 00001800/00000800 | 2000 sti: window after (a run changed it) 428 000000c8/000001c8, 450 00002000/00001000, 628 00000144/00000044, 650 00002000/00001800 | 2000 pld: window after (a run changed it) 428 000001b8/000000b8, 450 00002800/00002000, 628 00000034/00000134, 650 00003000/00002000 | sig, end: window after (a run changed it) 428 000000a8/000001a8, 450 00003800/00002800, 628 00000124/00000024, 650 00003800/00003000 | DMA + ECC encode: window after (a run changed it) 428 00000198/00000098, 450 00000000/00003800, 628 00000014/00000114, 650 00000800/00003800
  bit  8 (0x0000fef5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000078, 450 00003800/00001000, 628 00000074/000000f4, 650 00000000/00001000 | loop, SRAM: window after (a run changed it) 428 000000e8/00000168, 450 00000800/00001800, 628 00000164/000001e4 | 2000 ldi: window after (a run changed it) 428 000001d8/00000058, 450 00001000/00002800, 628 00000054/000000d4, 650 00001800/00002800 | 2000 sti: window after (a run changed it) 428 000000c8/00000148, 450 00002000/00003000, 628 00000144/000001c4 | 2000 pld: window after (a run changed it) 428 000001b8/00000038, 450 00002800/00000000, 628 00000034/000000b4, 650 00003000/00000000 | sig, end: window after (a run changed it) 428 000000a8/00000128, 450 00003800/00000800, 628 00000124/000001a4 | DMA + ECC encode: window after (a run changed it) 428 00000198/00000018, 450 00000000/00001800, 628 00000014/00000094, 650 00000800/00001800, c14 000000ff/000000fe
  bit  9 (0x0000fdf5): loop, SDRAM: window after (a run changed it) 650 00000000/00003000 | loop, SRAM: window after (a run changed it) 450 00000800/00003800 | 2000 ldi: window after (a run changed it) 650 00001800/00000800 | 2000 sti: window after (a run changed it) 450 00002000/00001000 | 2000 pld: window after (a run changed it) 650 00003000/00002000 | sig, end: window after (a run changed it) 450 00003800/00002800 | DMA + ECC encode: window after (a run changed it) 650 00000800/00003800, c14 000000ff/000000fd
  bit 10 (0x0000fbf5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000178, 628 00000074/000001f4, 650 00000000/00000800 | loop, SRAM: window after (a run changed it) 428 000000e8/00000068, 450 00000800/00001800, 628 00000164/000000e4, 650 00000800/00001800 | 2000 ldi: window after (a run changed it) 428 000001d8/00000158, 628 00000054/000001d4, 650 00001800/00002000 | 2000 sti: window after (a run changed it) 428 000000c8/00000048, 450 00002000/00003000, 628 00000144/000000c4, 650 00002000/00003000 | 2000 pld: window after (a run changed it) 428 000001b8/00000138, 628 00000034/000001b4, 650 00003000/00003800 | sig, end: window after (a run changed it) 428 000000a8/00000028, 450 00003800/00000800, 628 00000124/000000a4, 650 00003800/00000800 | DMA + ECC encode: window after (a run changed it) 428 00000198/00000118, 628 00000014/00000194, 650 00000800/00001000, c14 000000ff/000000fb | fault, C58 4 (vector): fault at 4776 us (base 4854)
  bit 11 (0x0000f7f5): loop, SDRAM: window after (a run changed it) 428 000001f8/000000f8, 628 00000074/00000174, 650 00000000/00002800 | loop, SRAM: window after (a run changed it) 428 000000e8/000001e8, 450 00000800/00003000, 628 00000164/00000064, 650 00000800/00003800 | 2000 ldi: time 1969 us (base 1636..1641); window after (a run changed it) 428 000001d8/000000d8, 450 00001000/00000000, 628 00000054/00000154, 650 00001800/00000000 | 2000 sti: window after (a run changed it) 428 000000c8/000001c8, 450 00002000/00000800, 628 00000144/00000044, 650 00002000/00001000 | 2000 pld: window after (a run changed it) 428 000001b8/000000b8, 450 00002800/00001800, 628 00000034/00000134, 650 00003000/00001800 | sig, end: window after (a run changed it) 428 000000a8/000001a8, 450 00003800/00002000, 628 00000124/00000024, 650 00003800/00002800 | DMA + ECC encode: window after (a run changed it) 428 00000198/00000098, 450 00000000/00003000, 628 00000014/00000114, 650 00000800/00003000, c14 000000ff/000000f7 | fault, C58 4 (vector): fault at 4710 us (base 4854)
  bit 12 (0x0000eff5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000078, 450 00003800/00000800, 628
```

```
00000074/000000f4, 650 00000000/00000800 | loop, SRAM: window after (a run changed it) 428 000000e8/00000168, 450 00
000800/00001000, 628 00000164/000001e4 | 2000 ldi: window after (a run changed it) 428 000001d8/00000058, 450 000010
00/00002000, 628 00000054/000000d4, 650 00001800/00002000 | 2000 sti: window after (a run changed it) 428 000000c8/0
0000148, 450 00002000/00002800, 628 00000144/000001c4 | 2000 pld: window after (a run changed it) 428 000001b8/00000
038, 450 00002800/00003800, 628 00000034/000000b4, 650 00003000/00003800 | sig, end: window after (a run changed it)
 428 000000a8/00000128, 450 00003800/00000000, 628 00000124/000001a4 | DMA + ECC encode: window after (a run changed
 it) 428 00000198/00000018, 450 00000000/00001000, 628 00000014/00000094, 650 00000800/00001000, c14 000000ff/000000
ef | fault, C58 4 (vector): fault at 4552 us (base 4854)
bit 13 (0x0000dff5): loop, SDRAM: window after (a run changed it) 650 00000000/00002800 | loop, SRAM: window after (
a run changed it) 450 00000800/00003000 | 2000 ldi: window after (a run changed it) 650 00001800/00000000 | 2000 sti
: window after (a run changed it) 450 00002000/00000800 | 2000 pld: window after (a run changed it) 650 00003000/000
01800 | sig, end: window after (a run changed it) 450 00003800/00002000 | DMA + ECC encode: window after (a run chan
ged it) 650 00000800/00003000, c14 000000ff/000000df | fault, C58 4 (vector): fault at 4248 us (base 4854)
bit 14 (0x0000bff5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000178, 628 00000074/000001f4 | loo
p, SRAM: window after (a run changed it) 428 000000e8/00000068, 450 00000800/00001000, 628 00000164/000000e4, 650 00
000800/00001000 | 2000 ldi: window after (a run changed it) 428 000001d8/00000158, 628 00000054/000001d4 | 2000 sti:
 window after (a run changed it) 428 000000c8/00000048, 450 00002000/00002800, 628 00000144/000000c4, 650 00002000/0
0002800 | 2000 pld: window after (a run changed it) 428 000001b8/00000138, 628 00000034/000001b4 | sig, end: window
after (a run changed it) 428 000000a8/00000028, 450 00003800/00000000, 628 00000124/000000a4, 650 00003800/00000000
| DMA + ECC encode: window after (a run changed it) 428 00000198/00000118, 628 00000014/00000194, c14 000000ff/00000
0bf | fault, C58 4 (vector): fault at 3638 us (base 4854)
bit 15 (0x00007ff5): loop, SDRAM: window after (a run changed it) 428 000001f8/000000f8, 628 00000074/00000174, 650
00000000/00002000 | loop, SRAM: window after (a run changed it) 428 000000e8/000001e8, 450 00000800/00002800, 628 00
000164/00000064, 650 00000800/00003000 | 2000 ldi: window after (a run changed it) 428 000001d8/000000d8, 450 000010
00/00003800, 628 00000054/00000154, 650 00001800/00003800 | 2000 sti: window after (a run changed it) 428 000000c8/0
00001c8, 450 00002000/00000000, 628 00000144/00000044, 650 00002000/00000800 | 2000 pld: window after (a run changed
 it) 428 000001b8/000000b8, 450 00002800/00001000, 628 00000034/00000134, 650 00003000/00001000 | sig, end: window a
fter (a run changed it) 428 000000a8/000001a8, 450 00003800/00001800, 628 00000124/00000024, 650 00003800/00002000 |
 DMA + ECC encode: window after (a run changed it) 428 00000198/00000098, 450 00000000/00002800, 628 00000014/000001
14, 650 00000800/00002800, c14 000000ff/0000007f | fault, C58 4 (vector): fault at 2430 us (base 4854)
bit 17 (0x0002fff5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000078, 450 00003800/00000000, 628
00000074/000000f4 | loop, SRAM: window after (a run changed it) 428 000000e8/00000168, 628 00000164/000001e4 | 2000
ldi: window after (a run changed it) 428 000001d8/00000058, 450 00001000/00001800, 628 00000054/000000d4 | 2000 sti:
 window after (a run changed it) 428 000000c8/00000148, 628 00000144/000001c4 | 2000 pld: window after (a run change
d it) 428 000001b8/00000038, 450 00002800/00003000, 628 00000034/000000b4 | sig, end: window after (a run changed it
) 428 000000a8/00000128, 628 00000124/000001a4 | DMA + ECC encode: window after (a run changed it) 428 00000198/0000
0018, 450 00000000/00000800, 628 00000014/00000094
bit 18 (0x0004fff5): loop, SDRAM: window after (a run changed it) 650 00000000/00002000 | loop, SRAM: window after (
a run changed it) 450 00000800/00002800 | 2000 ldi: window after (a run changed it) 650 00001800/00003800 | 2000 sti
: window after (a run changed it) 450 00002000/00000000 | 2000 pld: window after (a run changed it) 650 00003000/000
01000 | sig, end: window after (a run changed it) 450 00003800/00001800 | DMA + ECC encode: window after (a run chan
ged it) 650 00000800/00002800
bit 19 (0x0008fff5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000178, 628 00000074/000001f4, 650
00000000/00003800 | loop, SRAM: window after (a run changed it) 428 000000e8/00000068, 628 00000164/000000e4 | 2000
ldi: window after (a run changed it) 428 000001d8/00000158, 628 00000054/000001d4, 650 00001800/00001000 | 2000 sti:
 window after (a run changed it) 428 000000c8/00000048, 628 00000144/000000c4 | 2000 pld: window after (a run change
d it) 428 000001b8/00000138, 628 00000034/000001b4, 650 00003000/00002800 | sig, end: window after (a run changed it
) 428 000000a8/00000028, 628 00000124/000000a4 | DMA + ECC encode: window after (a run changed it) 428 00000198/0000
0118, 628 00000014/00000194, 650 00000800/00000000
bit 20 (0x0010fff5): loop, SDRAM: window after (a run changed it) 428 000001f8/000000f8, 628 00000074/00000174, 650
00000000/00001800 | loop, SRAM: window after (a run changed it) 428 000000e8/000001e8, 450 00000800/00002000, 628 00
000164/00000064, 650 00000800/00002800 | 2000 ldi: window after (a run changed it) 428 000001d8/000000d8, 450 000010
00/00003000, 628 00000054/00000154, 650 00001800/00003000 | 2000 sti: window after (a run changed it) 428 000000c8/0
00001c8, 450 00002000/00003800, 628 00000144/00000044, 650 00002000/00000000 | 2000 pld: window after (a run changed
 it) 428 000001b8/000000b8, 450 00002800/00000800, 628 00000034/00000134, 650 00003000/00000800 | sig, end: window a
fter (a run changed it) 428 000000a8/000001a8, 450 00003800/00001000, 628 00000124/00000024, 650 00003800/00001800 |
 DMA + ECC encode: window after (a run changed it) 428 00000198/00000098, 450 00000000/00002000, 628 00000014/000001
14, 650 00000800/00002000
bit 21 (0x0020fff5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000078, 628 00000074/000000f4, 650
00000000/00003800 | loop, SRAM: window after (a run changed it) 428 000000e8/00000168, 450 00000800/00000000, 628 00
000164/000001e4 | 2000 ldi: window after (a run changed it) 428 000001d8/00000058, 628 00000054/000000d4, 650 000018
00/00001000 | 2000 sti: window after (a run changed it) 428 000000c8/00000148, 450 00002000/00001800, 628 00000144/0
00001c4 | 2000 pld: window after (a run changed it) 428 000001b8/00000038, 628 00000034/000000b4, 650 00003000/00002
800 | sig, end: window after (a run changed it) 428 000000a8/00000128, 450 00003800/00003000, 628 00000124/000001a4
| DMA + ECC encode: window after (a run changed it) 428 00000198/00000018, 628 00000014/00000094, 650 00000800/00000
000
bit 22 (0x0040fff5): loop, SDRAM: window after (a run changed it) 650 00000000/00001800 | loop, SRAM: window after (
a run changed it) 450 00000800/00002000 | 2000 ldi: window after (a run changed it) 650 00001800/00003000 | 2000 sti
: window after (a run changed it) 450 00002000/00003800 | 2000 pld: window after (a run changed it) 650 00003000/000
00800 | sig, end: window after (a run changed it) 450 00003800/00001000 | DMA + ECC encode: window after (a run chan
ged it) 650 00000800/00002000
bit 23 (0x0080fff5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000178, 628 00000074/000001f4, 650
00000000/00003000 | loop, SRAM: window after (a run changed it) 428 000000e8/00000068, 450 00000800/00000000, 628 00
000164/000000e4, 650 00000800/00000000 | 2000 ldi: window after (a run changed it) 428 000001d8/00000158, 628 000000
54/000001d4, 650 00001800/00000800 | 2000 sti: window after (a run changed it) 428 000000c8/00000048, 450 00002000/0
0001800, 628 00000144/000000c4, 650 00002000/00001800 | 2000 pld: window after (a run changed it) 428 000001b8/00000
138, 628 00000034/000001b4, 650 00003000/00002000 | sig, end: window after (a run changed it) 428 000000a8/00000028,
 450 00003800/00003000, 628 00000124/000000a4, 650 00003800/00003000 | DMA + ECC encode: window after (a run changed
 it) 428 00000198/00000118, 628 00000014/00000194, 650 00000800/00003800
bit 24 (0x0100fff5): loop, SDRAM: window after (a run changed it) 428 000001f8/000000f8, 628 00000074/00000174, 650
00000000/00001000 | loop, SRAM: window after (a run changed it) 428 000000e8/000001e8, 450 00000800/00001800, 628 00
000164/00000064, 650 00000800/00002000 | 2000 ldi: window after (a run changed it) 428 000001d8/000000d8, 450 000010
00/00002800, 628 00000054/00000154, 650 00001800/00002800 | 2000 sti: window after (a run changed it) 428 000000c8/0
00001c8, 450 00002000/00003000, 628 00000144/00000044, 650 00002000/00003800 | 2000 pld: window after (a run changed
 it) 428 000001b8/000000b8, 450 00002800/00000000, 628 00000034/00000134, 650 00003000/00000000 | sig, end: window a
```

```
  fter (a run changed it) 428 000000a8/000001a8, 450 00003800/00000800, 628 00000124/00000024, 650 00003800/00001000 |
   DMA + ECC encode: window after (a run changed it) 428 00000198/00000098, 450 00000000/00001800, 628 00000014/000001
  14, 650 00000800/00001800
  bit 25 (0x0200fff5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000078, 450 00003800/00003000, 628
  00000074/000000f4, 650 00000000/00003000 | loop, SRAM: window after (a run changed it) 428 000000e8/00000168, 450 00
  000800/00003800, 628 00000164/000001e4 | 2000 ldi: window after (a run changed it) 428 000001d8/00000058, 450 000010
  00/00000800, 628 00000054/000000d4, 650 00001800/00000800 | 2000 sti: window after (a run changed it) 428 000000c8/0
  0000148, 450 00002000/00001000, 628 00000144/000001c4 | 2000 pld: window after (a run changed it) 428 000001b8/00000
  038, 450 00002800/00002000, 628 00000034/000000b4, 650 00003000/00002000 | sig, end: window after (a run changed it)
   428 000000a8/00000128, 450 00003800/00002800, 628 00000124/000001a4 | DMA + ECC encode: window after (a run changed
   it) 428 00000198/00000018, 450 00000000/00003800, 628 00000014/00000094, 650 00000800/00003800
  bit 26 (0x0400fff5): loop, SDRAM: window after (a run changed it) 650 00000000/00001000 | loop, SRAM: window after (
  a run changed it) 450 00000800/00001800 | 2000 ldi: window after (a run changed it) 650 00001800/00002800 | 2000 sti
  : window after (a run changed it) 450 00002000/00003000 | 2000 pld: window after (a run changed it) 650 00003000/000
  00000 | sig, end: window after (a run changed it) 450 00003800/00000800 | DMA + ECC encode: window after (a run chan
  ged it) 650 00000800/00001800
  bit 27 (0x0800fff5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000178, 628 00000074/000001f4, 650
  00000000/00002800 | loop, SRAM: window after (a run changed it) 428 000000e8/00000068, 450 00000800/00003800, 628 00
  000164/000000e4, 650 00000800/00003800 | 2000 ldi: window after (a run changed it) 428 000001d8/00000158, 628 000000
  54/000001d4, 650 00001800/00000000 | 2000 sti: window after (a run changed it) 428 000000c8/00000048, 450 00002000/0
  0001000, 628 00000144/000000c4, 650 00002000/00001000 | 2000 pld: window after (a run changed it) 428 000001b8/00000
  138, 628 00000034/000001b4, 650 00003000/00001800 | sig, end: window after (a run changed it) 428 000000a8/00000028,
   450 00003800/00002800, 628 00000124/000000a4, 650 00003800/00002800 | DMA + ECC encode: window after (a run changed
   it) 428 00000198/00000118, 628 00000014/00000194, 650 00000800/00003000
  bit 28 (0x1000fff5): loop, SDRAM: window after (a run changed it) 428 000001f8/000000f8, 628 00000074/00000174, 650
  00000000/00000800 | loop, SRAM: window after (a run changed it) 428 000000e8/000001e8, 450 00000800/00001000, 628 00
  000164/00000064, 650 00000800/00001800 | 2000 ldi: window after (a run changed it) 428 000001d8/000000d8, 450 000010
  00/00002000, 628 00000054/00000154, 650 00001800/00002000 | 2000 sti: window after (a run changed it) 428 000000c8/0
  00001c8, 450 00002000/00002800, 628 00000144/00000044, 650 00002000/00003000 | 2000 pld: window after (a run changed
   it) 428 000001b8/000000b8, 450 00002800/00003800, 628 00000034/00000134, 650 00003000/00003800 | sig, end: window a
  fter (a run changed it) 428 000000a8/000001a8, 450 00003800/00000000, 628 00000124/00000024, 650 00003800/00000800 |
   DMA + ECC encode: window after (a run changed it) 428 00000198/00000098, 450 00000000/00001000, 628 00000014/000001
  14, 650 00000800/00001000
  bit 29 (0x2000fff5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000078, 450 00003800/00002800, 628
  00000074/000000f4, 650 00000000/00002800 | loop, SRAM: window after (a run changed it) 428 000000e8/00000168, 450 00
  000800/00003000, 628 00000164/000001e4 | 2000 ldi: window after (a run changed it) 428 000001d8/00000058, 450 000010
  00/00000000, 628 00000054/000000d4, 650 00001800/00000000 | 2000 sti: window after (a run changed it) 428 000000c8/0
  0000148, 450 00002000/00000800, 628 00000144/000001c4 | 2000 pld: window after (a run changed it) 428 000001b8/00000
  038, 450 00002800/00001800, 628 00000034/000000b4, 650 00003000/00001800 | sig, end: window after (a run changed it)
   428 000000a8/00000128, 450 00003800/00002000, 628 00000124/000001a4 | DMA + ECC encode: window after (a run changed
   it) 428 00000198/00000018, 450 00000000/00003000, 628 00000014/00000094, 650 00000800/00003000
  bit 30 (0x4000fff5): loop, SDRAM: window after (a run changed it) 650 00000000/00000800 | loop, SRAM: window after (
  a run changed it) 450 00000800/00001000 | 2000 ldi: window after (a run changed it) 650 00001800/00002000 | 2000 sti
  : window after (a run changed it) 450 00002000/00002800 | 2000 pld: window after (a run changed it) 650 00003000/000
  03800 | sig, end: window after (a run changed it) 450 00003800/00000000 | DMA + ECC encode: window after (a run chan
  ged it) 650 00000800/00001000
  bit 31 (0x8000fff5): loop, SDRAM: window after (a run changed it) 428 000001f8/00000178, 628 00000074/000001f4, 650
  00000000/00002000 | loop, SRAM: window after (a run changed it) 428 000000e8/00000068, 450 00000800/00003000, 628 00
  000164/000000e4, 650 00000800/00003000 | 2000 ldi: window after (a run changed it) 428 000001d8/00000158, 628 000000
  54/000001d4, 650 00001800/00003800 | 2000 sti: window after (a run changed it) 428 000000c8/00000048, 450 00002000/0
  0000800, 628 00000144/000000c4, 650 00002000/00000800 | 2000 pld: window after (a run changed it) 428 000001b8/00000
  138, 628 00000034/000001b4, 650 00003000/00001000 | sig, end: window after (a run changed it) 428 000000a8/00000028,
   450 00003800/00002000, 628 00000124/000000a4, 650 00003800/00002000 | DMA + ECC encode: window after (a run changed
   it) 428 00000198/00000118, 628 00000014/00000194, 650 00000800/00002800
```

14.2.133 rsdecode (run 35)

ID: `D-rsdecode-35`. Device run 35. Experiment: rsdecode.

```
Reed-Solomon 12 / 16 parity symbols, decode strength and the parity buffer:
  0x08104,  0 bytes changed: ECC_RESULT 00000000 ECC_DONE 35, data corrected
      parity buffer after the encode (slots 0..3): 186d249847a87441d927d4965ed6e4ffffffffffffffffffffffffffffffffff
  ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
      ECC area 0x900..0x914: e4d65e96d427d94174a84798246d18a8012da80100000000
  0x08104,  4 bytes changed: ECC_RESULT 00040000 ECC_DONE 35, data corrected
  0x08104,  6 bytes changed: ECC_RESULT 00060000 ECC_DONE 35, data corrected
  0x08104,  7 bytes changed: ECC_RESULT 00000011 ECC_DONE 3d, data as sent
  0x08108,  0 bytes changed: ECC_RESULT 00000000 ECC_DONE 35, data corrected
      parity buffer after the encode (slots 0..3): 4e2923086f4cfc4b7b77718291507950ffa89d05ffffffffffffffffffffffffff
  ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
      ECC area 0x900..0x914: 059da8ff507950918271777b4bfc4c6f0823294e00000000
  0x08108,  4 bytes changed: ECC_RESULT 00040000 ECC_DONE 35, data corrected
  0x08108,  8 bytes changed: ECC_RESULT 00080000 ECC_DONE 35, data corrected
  0x08108,  9 bytes changed: ECC_RESULT 00000011 ECC_DONE 3d, data as sent
  0x08108, 12 bytes changed: ECC_RESULT 00000011 ECC_DONE 3d, data as sent
```

14.2.134 vic20 (run 35)

ID: `D-vic20-35`. Device run 35. Experiment: vic20.

```
interrupt lines per FMC source, three runs each:
  command  0x044 bit  0: CS_IRQ 11 VIC0.20 1 VIC1.22 0 | CS_IRQ 11 VIC0.20 1 VIC1.22 0 | CS_IRQ 11 VIC0.20 1 VIC1.22 0
  command  0x044 bit  1: CS_IRQ 01 VIC0.20 0 VIC1.22 0 | CS_IRQ 01 VIC0.20 0 VIC1.22 0 | CS_IRQ 01 VIC0.20 0 VIC1.22 0
  command  0x044 bit  2: CS_IRQ 01 VIC0.20 0 VIC1.22 0 | CS_IRQ 01 VIC0.20 0 VIC1.22 0 | CS_IRQ 01 VIC0.20 0 VIC1.22 0
  address  0x044 bit  0: CS_IRQ 01 VIC0.20 0 VIC1.22 0 | CS_IRQ 01 VIC0.20 0 VIC1.22 0 | CS_IRQ 01 VIC0.20 0 VIC1.22 0
  address  0x044 bit  1: CS_IRQ 11 VIC0.20 1 VIC1.22 0 | CS_IRQ 11 VIC0.20 1 VIC1.22 0 | CS_IRQ 11 VIC0.20 1 VIC1.22 0
```

```
  status   0x044 bit  0: CS_IRQ 11 VIC0.20 0 VIC1.22 0 | CS_IRQ 11 VIC0.20 0 VIC1.22 0 | CS_IRQ 11 VIC0.20 0 VIC1.22 0
  status   0x044 bit 13: CS_IRQ 01 VIC0.20 0 VIC1.22 0 | CS_IRQ 01 VIC0.20 0 VIC1.22 0 | CS_IRQ 01 VIC0.20 0 VIC1.22 0
  DMA      0x044 bit 11: CS_IRQ 11 VIC0.20 0 VIC1.22 0 | CS_IRQ 11 VIC0.20 0 VIC1.22 0 | CS_IRQ 11 VIC0.20 0 VIC1.22 0
  none     0x044 bit  0: CS_IRQ 01 VIC0.20 0 VIC1.22 0 | CS_IRQ 01 VIC0.20 0 VIC1.22 0 | CS_IRQ 01 VIC0.20 0 VIC1.22 0
```

14.2.135 fmc28b (run 35)

ID: `D-fmc28b-35`. Device run 35. Experiment: fmc28b.

```
FMC_28 (9 bits) around runs:
  no set-up nothing            CPU before 232 | program start 480, end 480 | CPU after  92
  no set-up loop 2000          CPU before 464 | program start 200, end 200 | CPU after 324
  no set-up DMA                CPU before 184 | program start 432, end   0 | CPU after 124
  no set-up ECC encode         CPU before 496 | program start 232, end 232 | CPU after 356
  no set-up 16-byte chip read  CPU before 216 | program start 464, end 464 | CPU after  76
  set-up    nothing            CPU before 448 | program start   0, end   0 | CPU after 124
  set-up    loop 2000          CPU before 496 | program start   0, end   0 | CPU after 124
  set-up    DMA                CPU before 496 | program start   0, end   0 | CPU after 124
  set-up    ECC encode         CPU before 496 | program start   0, end   0 | CPU after 124
  set-up    16-byte chip read  CPU before 496 | program start   0, end   0 | CPU after 124
```

14.2.136 c58b1 (run 35)

ID: `D-c58b1-35`. Device run 35. Experiment: c58b1.

```
C58 bit 1 (run 1 | run 2 without new writes):
  C58=2, C4C                                    END (C5C 0) | then: END (C5C 0)
  C4C, C58=2                                    END (C5C 0) | then: END (C5C 0)
  C58=2 only (C4C left from the previous case)  END (C5C 0)
  C4C only (C58=2 left)                         END (C5C 0) | then: END (C5C 0)
  C58=2, C4C, C4C                               END (C5C 0) | then: END (C5C 0)
  C58=2, C4C, C5C=0                             END (C5C 0) | then: END (C5C 0)
  C58=2, C4C, C58=2                             END (C5C 0) | then: END (C5C 0)
  C4C=0x40 by CPU, C58=2, program writes C4C    END (C5C 0) | then: END (C5C 0)
  C58=3, C4C                                    END (C5C 1) | then: END (C5C 0)
  C58=6, C4C                                    END (C5C 0) | then: END (C5C 0)
```

14.2.137 evsweep2 (run 35)

ID: `D-evsweep2-35`. Device run 35. Experiment: evsweep2.

```
wait / wait2 sweep per state (short timeout; pass = no fault):
  idle, CE0 selected                          wait passes [11]; wait2 passes [5]; END/END 1645/1645 us
  erase busy                                  wait passes [11]; wait2 passes [5]; END/END 1659/1660 us
  program busy (80h addr 10h)                 wait passes [11]; wait2 passes [5]; END/END 1660/1658 us
  read busy (00h addr 30h)                    wait passes [11]; wait2 passes [5]; END/END 1645/1658 us
  ECC encode in flight                        wait passes [4, 11]; wait2 passes [5]; END/END 1645/1644 us
      beyond idle: [('wait', 4)]
  DMA in flight                               wait passes [2, 11]; wait2 passes [5]; END/END 1645/1646 us
      beyond idle: [('wait', 2)]
  0x044 / 0x844 all on, command flag pending  wait passes [0, 11]; wait2 passes [5, 20]; END/END 1634/1643 us
      beyond idle: [('wait', 0), ('wait2', 20)]
  second set: command sent                    wait passes [6, 11]; wait2 passes [5]; END/END 1643/1643 us
      beyond idle: [('wait', 6)]
```

14.2.138 c50fault (run 35)

ID: `D-c50fault-35`. Device run 35. Experiment: c50fault.

```
C50 around faults (bit 0) and bit 8:
  C58 0 timeout (wait 10)  nothing pending  before ffff1800 | after ffff1800 | handler 00000000, after w1 00000000, C3
  C 00000000 | end C50 ffff1800, FAULT r6 600d600d
  C58 0 timeout (wait 10)  command pending  before ffff1801 | after ffff1801 | handler 00000000, after w1 00000000, C3
  C 00000000 | end C50 ffff1801, FAULT r6 600d600d
  C58 0 illegal 0x30       nothing pending  before ffff1800 | after ffff1800 | handler 00000000, after w1 00000000, C3
  C 00000000 | end C50 ffff1800, FAULT r6 600d600d
  C58 0 illegal 0x30       command pending  before ffff1801 | after ffff1801 | handler 00000000, after w1 00000000, C3
  C 00000000 | end C50 ffff1801, FAULT r6 600d600d
  C58 4 timeout (wait 10)  nothing pending  before ffff1800 | after 00000000 | handler ffff1800, after w1 ffff1800, C3
  C 00000058 | end C50 ffff1800, FAULT r6 4a4d4a4d
  C58 4 timeout (wait 10)  command pending  before ffff1801 | after 00000000 | handler ffff1801, after w1 ffff1801, C3
  C 00000068 | end C50 ffff1801, FAULT r6 4a4d4a4d
  C58 4 illegal 0x30       nothing pending  before ffff1800 | after 00000000 | handler ffff1800, after w1 ffff1800, C3
  C 00000058 | end C50 ffff1800, FAULT r6 4a4d4a4d
  C58 4 illegal 0x30       command pending  before ffff1801 | after 00000000 | handler ffff1801, after w1 ffff1801, C3
  C 00000068 | end C50 ffff1801, FAULT r6 4a4d4a4d
  C50 bit 8: after a soft reset, END only   before ffff1800 after ffff1800
  C50 bit 8: again                          before ffff1800 after ffff1800
  C50 bit 8: status latch                   before ffff1800 after ffff1820
```

14.2.139 ctrlwords (run 35)

ID: `D-ctrlwords-35`. Device run 35. Experiment: ctrlwords.

```
unused control words (value written: C10 / C48 / C54) and C3C in handlers:
  timeout, C58 4     00000000 FAULT 4883us fault 4868us CS_IRQ 05 r6 fa17fa17 0x900 00000000 | ffffffff FAULT 4883us f
```

```
  ault 4868us CS_IRQ 05 r6 fa17fa17 0x900 00000000 | 55555555 FAULT 4870us fault 4854us CS_IRQ 05 r6 fa17fa17 0x900 00
  000000
  illegal, C58 4     00000000 FAULT 4883us fault 4868us CS_IRQ 05 r6 fa17fa17 0x900 00000000 | ffffffff FAULT 4868us f
  ault 4854us CS_IRQ 05 r6 fa17fa17 0x900 00000000 | 55555555 FAULT 4883us fault 4854us CS_IRQ 05 r6 fa17fa17 0x900 00
  000000
  wait2 33           00000000 FAULT 4883us fault 4868us CS_IRQ 05 r6 600d600d 0x900 00000000 | ffffffff FAULT 4881us f
  ault 4866us CS_IRQ 05 r6 600d600d 0x900 00000000 | 55555555 FAULT 4865us fault 4850us CS_IRQ 05 r6 600d600d 0x900 00
  000000
  DMA + ECC encode   00000000 END 18us fault 0us CS_IRQ 01 r6 600d600d 0x900 95982422 | ffffffff END 18us fault 0us CS
  _IRQ 01 r6 600d600d 0x900 95982422 | 55555555 END 18us fault 0us CS_IRQ 01 r6 600d600d 0x900 95982422
  tmwait twice       00000000 END 4841us fault 0us CS_IRQ 01 r6 600d600d 0x900 95982422 | ffffffff END 18us fault 0us
  CS_IRQ 01 r6 600d600d 0x900 95982422 | 55555555 END 3250us fault 0us CS_IRQ 01 r6 600d600d 0x900 95982422
  C3C: fault at offset 0x28: first handler entry C3C 00000030 (PC read there 00000058), second entry C3C 00000068; FAU
  LT r6 4a4d4a4d
  C3C: fault at offset 0x40: first handler entry C3C 00000048 (PC read there 00000070), second entry C3C 00000080; FAU
  LT r6 4a4d4a4d
  C3C: fault at offset 0x70: first handler entry C3C 00000078 (PC read there 000000a0), second entry C3C 000000b0; FAU
  LT r6 4a4d4a4d
```

14.2.140 c08self (run 35)

ID: `D-c08self-35`. Device run 35. Experiment: c08self.

```
CS_STATUS read by the program (r1, then r2 after the wait; eeee = not read):
  plain                C08 5, then eeee; END
  after DMA start      C08 5, then 5; END
  after ECC kick       C08 5, then 5; END
  after a chip command C08 5, then 5; END
  after sig            C08 5, then eeee; END
  after wait2 33       C08 5, then eeee; FAULT
  pld 0xc08            C08 5, then eeee; END
  in a fault handler   C08 5, then 4; FAULT
```

14.2.141 latchrep (run 35)

ID: `D-latchrep-35`. Device run 35. Experiment: latchrep.

```
status latch and wait 3, 40 runs each:
  fresh arm                          pass b23 1 b24 0 x40
  no fresh arm                       pass b23 1 b24 0 x40
  fresh arm, 0x044 / 0x844 written   pass b23 1 b24 0 x40
  no fresh arm, 0x044 / 0x844 written pass b23 1 b24 0 x40
  two latches, no arm between        timeout b23 0 b24 0 x40
  wait 3 on CE0 (16-byte read)       pass b23 0 b24 0 x40
```

14.2.142 irq044 (run 35)

ID: `D-irq044-35`. Device run 35. Experiment: irq044.

```
0x044 one bit at a time (three runs):
  none   CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 0  CS_IRQ 11 V0.20 0 V1.22 0 END | CS_IRQ 11 V0.20 0 V1.22 0 END | CS_IRQ 11 V0.20 0 V1.22 0 END
  bit 1  CS_IRQ 11 V0.20 0 V1.22 0 END | CS_IRQ 11 V0.20 0 V1.22 0 END | CS_IRQ 11 V0.20 0 V1.22 0 END
  bit 2  CS_IRQ 11 V0.20 0 V1.22 0 END | CS_IRQ 11 V0.20 0 V1.22 0 END | CS_IRQ 11 V0.20 0 V1.22 0 END
  bit 3  CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 4  CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 5  CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 6  CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 7  CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 8  CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 9  CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 10 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 11 CS_IRQ 11 V0.20 0 V1.22 0 END | CS_IRQ 11 V0.20 0 V1.22 0 END | CS_IRQ 11 V0.20 0 V1.22 0 END
  bit 12 CS_IRQ 11 V0.20 0 V1.22 0 END | CS_IRQ 11 V0.20 0 V1.22 0 END | CS_IRQ 11 V0.20 0 V1.22 0 END
  bit 13 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 14 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 15 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 16 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 17 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 18 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 19 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 20 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 21 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 22 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 23 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 24 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 25 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 26 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 27 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 28 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 29 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 30 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
  bit 31 CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END | CS_IRQ 01 V0.20 0 V1.22 0 END
```

14.2.143 fmcbits (run 35)

ID: D-fmcbits-35. Device run 35. Experiment: fmcbits.

```
FMC_CTRL0 bits, FMC_DMA_CTRL and drain selects on a page read:
  erase + program: END, NSTAT c0
  base 0x43803                 END   317us data ==== results 00000000 00000000 00000000 00000000 FMC_DMA_CTRL after 00
  000007
  bit 9 set                    END   302us data ==== results 00000000 00000000 00000000 00000000 FMC_DMA_CTRL after 00
  000007
  bit 11 clear                 END   317us data ==== results 00000000 00000000 00000000 00000000 FMC_DMA_CTRL after 00
  000007
  bit 24 set throughout        END   317us data ==== results 00000000 00000000 00000000 00000000 FMC_DMA_CTRL after 00
  000007
  FMC_DMA_CTRL 0               END   332us data ==== results 00000000 00000000 00000000 00000000 FMC_DMA_CTRL after 00
  000000
  FMC_DMA_CTRL 7               END   436us data ==== results 00000000 00000000 00000000 00000000 FMC_DMA_CTRL after 00
  000007
  FMC_DMA_CTRL 0xff            FAULT  5177us data ===x results 00000000 00000000 00000000 00000000 FMC_DMA_CTRL after
  000000ff
  drain 0x102                  END   361us data xxxx results 00000000 00000000 00000000 00000000 FMC_DMA_CTRL after 00
  00000f
  drain 0x1e0                  END   361us data xxxx results 00000000 00000000 00000000 00000000 FMC_DMA_CTRL after 00
  00000f
  drain 0x1e0 + DMA_CTRL 7     END   359us data xxx= results 00000000 00000000 00000000 00000000 FMC_DMA_CTRL after 00
  000007
  READID base                  END ID b614d5ec 14d5ec74 status 08041000
  READID bit 9                 END ID b614d5ec 14d5ec74 status 08041000
  READID no bit 11             END ID b614d5ec 14d5ec74 status 08041000
  READID bit 9, no bit 11      END ID b614d5ec 14d5ec74 status 08041000
```

14.2.144 fmcstat (run 35)

ID: D-fmcstat-35. Device run 35. Experiment: fmcstat.

```
FMC_STATUS power-on bits:
  after a soft reset: FMC_STATUS 000c3000, 0x448 00000000
  bit 12 (start > after w1c > after the operation): nothing 1>1>1, command 1>1>1, address 1>1>1, data read 1>1>1, stat
  us latch 1>1>1, DMA 1>1>1, ECC encode 1>1>1, meta encode 0>1>1, second set command 1>1>1
  bit 13 (start > after w1c > after the operation): nothing 1>1>1, command 1>1>1, address 1>1>1, data read 1>1>0, stat
  us latch 1>1>1, DMA 1>1>1, ECC encode 1>1>1, meta encode 0>1>1, second set command 1>1>1
  bit 18 (start > after w1c > after the operation): nothing 1>1>1, command 1>1>1, address 1>1>1, data read 1>1>1, stat
  us latch 1>1>1, DMA 1>1>1, ECC encode 1>1>1, meta encode 0>1>1, second set command 1>1>1
  bit 19 (start > after w1c > after the operation): nothing 1>1>1, command 1>1>1, address 1>1>1, data read 1>1>0, stat
  us latch 1>1>1, DMA 1>1>1, ECC encode 1>1>1, meta encode 0>1>1, second set command 1>1>1
```

14.2.145 eccregs (run 35)

ID: D-eccregs-35. Device run 35. Experiment: eccregs.

```
ECC words 0x940..0x950 and 0x980..0x99c:
  0x940..0x950 = 00000000: parity ffffff7a82a426c457e3a863a1cb34c8, decode 00080000 / 35, data corrected, 0x940.. afte
  r 00000000 00000000 00000000 00000000 00000000
  0x940..0x950 = ffffffff: parity ffffff7a82a426c457e3a863a1cb34c8, decode 00080000 / 35, data corrected, 0x940.. afte
  r ffffffff ffffffff ffffffff ffffffff ffffffff
  0x940..0x950 = 55555555: parity ffffff7a82a426c457e3a863a1cb34c8, decode 00080000 / 35, data corrected, 0x940.. afte
  r 55555555 55555555 55555555 55555555 55555555
  mode 0x0180 zeros 0x980..0x99c 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
  mode 0x0180 ones  0x980..0x99c 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
  mode 0x0180 ramp  0x980..0x99c 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
  mode 0x8100 zeros 0x980..0x99c 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
  mode 0x8100 ones  0x980..0x99c 00000000 00000000 00000000 00000000 fc36fc36 00000000 00000000 00000000
  mode 0x8100 ramp  0x980..0x99c 00000000 00000000 00000000 00000000 c68bc68b 00000000 00000000 00000000
  mode 0x8108 zeros 0x980..0x99c 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
  mode 0x8108 ones  0x980..0x99c 00000000 00000000 00000000 00000000 fc36fc36 00000000 00000000 00000000
  mode 0x8108 ramp  0x980..0x99c 00000000 00000000 00000000 00000000 c68bc68b 00000000 00000000 00000000
  mode 0x0000 zeros 0x980..0x99c 00000000 00000000 00000000 00000000 f0f4f0f4 00000000 00000000 00000000
  mode 0x0000 ones  0x980..0x99c 00000000 00000000 00000000 00000000 5bb85bb8 00000000 00000000 00000000
  mode 0x0000 ramp  0x980..0x99c 00000000 00000000 00000000 00000000 47344734 00000000 00000000 00000000
  mode 0x0080 zeros 0x980..0x99c 00000000 00000000 00000000 00000000 f6adf6ad 00000000 00000000 00000000
  mode 0x0080 ones  0x980..0x99c 00000000 00000000 00000000 00000000 3d6d3d6d 00000000 00000000 00000000
  mode 0x0080 ramp  0x980..0x99c 00000000 00000000 00000000 00000000 73537353 00000000 00000000 00000000
```

14.2.146 seqrestart (run 35)

ID: D-seqrestart-35. Device run 35. Experiment: seqrestart.

```
mode 0 encodes, area 0x900..0x913 (three per run):
  between: nothing             run 1 END edd42e2dda3f79ae9517f0dc3700000000000000 a62f0baf90c8082570138c3f12000000000
  00000 c47f1becb633ef38283ab3fa8400000000000000 | run 2 END 528950e823dd0c2ee3334b3ccf00000000000000 b5d2b4ab2eacd019
  2881214ff7000000000000000 993f23a304711734be92d1c26000000000000000
  between: soft reset          run 1 END 5788eb241e93af5b28f7042410000000000000000 2ca900f00bae330c7f30f0c62a000000000
  00000 b0c691ff3e6a42d353fb5a0bcb00000000000000 | run 2 END 4684475615107a7cfd75b3cebe00000000000000 5da0a18c3e1f8ace
  df03f2ac68000000000000000 b1250ddf69ebfa90821e21278300000000000000
  between: ECC_MODE write      run 1 END a49605af7f7209f9bc12b71a3a00000000000000 5f6698cd901dfb497dc805f4f8000000000
  00000 56fb4de8fda80f967a303d769f00000000000000 | run 2 END 83e1b2e862f0f8fb82654c520f00000000000000 91db19a3b89a1144
  8d953d56c000000000000000 18152367c7acec9306decbe7f500000000000000
  between: ECC_KICK 0          run 1 END 1f9ab13532fe2de062de3c167e00000000000000 0d07c4bdcd92d74f7549d07515000000000
```

```
  00000 20e225dd66b046a4c6a92a6f1700000000000000 | run 2 END 04d8cecd9969b2fbe987f3a8c100000000000000 5e120d0ccf2c902c
  8e46069dc500000000000000 359d1fe870186a9fabfaa0eb7d00000000000000
  between: BCH encode           run 1 END 8109986bdd922e3164784b807500000000000000 74ec7966e37cc95a54e0129410000000000
  00000 1cc57661b8694006cbe5c5430000000000000000 | run 2 END f60eb57ac4b58fd1abd5766bca00000000000000 265cab74729f9d68
  95ded8c4eb00000000000000 d130e886cba011cb2f5eaa6c6a00000000000000
  between: BCH decode           run 1 END 7195d49aa7f535b95248f6332900000000000000 bcb796bb950d3ef25ca9079a3f000000000
  00000 fcfcef834da77616a685bf24b900000000000000 | run 2 END 66b96bc129576180ba475c7794000000000000000 c5d45d74d885e344
  34fe38f38c00000000000000 f15f55f11065cc680d7aff54d600000000000000
  between: ECC_DONE write       run 1 END b784399bf3c0c9e11009365ce700000000000000 df922891620196b96201b91352000000000
  00000 5324f145a44a9dab59e72b023000000000000000 | run 2 END a0f31794c74e3117c61721644f00000000000000 b89ea43d4ad92733
  b0db04478b00000000000000 5e31699e8dd17f4a8f3a118b7500000000000000
  between: 0x940..0x950 write   run 1 END 4d10cb08d779410e18424e34e800000000000000 a2b4948a19efe2f01d60715500000000000
  00000 a70d1462ed388c744871dbd22e00000000000000 | run 2 END 5abf6fa33192d88538d5b1e9eb00000000000000 50cd6e72d83d058d
  93f7e5e6b600000000000000 89535d3520efbf63095f24692900000000000000
  between: FMC_CTRL0 write      run 1 END 9db75f97f3b004725f1a8c779300000000000000 0c71fcdc429262a311ab1b91b9000000000
  00000 07423d5da4aa078cc6c15f55d800000000000000 | run 2 END 25cf7acac6efcb2e5ba370988d00000000000000 33a9f8c90b9e0170
  f576f8104e00000000000000 77b377e7ce149935ceb3a68ae500000000000000
  between: sequencer reset      run 1 END 7884b72515701e84e9f8a3627700000000000000 d34b6dbddee23c03e4920aaf350000000000
  00000 eede29fd28ac216cd0e414860500000000000000 | run 2 END b8419cccbf896462cb30461d8000000000000000 c26b154c53145ebc
  a3dc7a4fe000000000000000 4daebaea3cd8c7acee94cb80fe00000000000000
```

14.2.147 done5 (run 35)

ID: `D-done5-35`. Device run 35. Experiment: done5.

```
ECC_DONE after encode / decode (ee = the wait timed out before it):
  base 0x0180: - 3b/35, b9 33/35, b10 33/35, b11 33/35, b12 33/35, b13 33/35, b14 33/35, b16 33/35, b17 33/35, b18 33/
  35, b19 33/35, b20 33/35, b21 13/15, b22 13/15, b23 13/15, b24 13/15
  base 0x8100: - 3b/35, b9 33/35, b10 33/35, b11 33/35, b12 33/35, b13 33/35, b14 33/35, b16 33/35, b17 33/35, b18 33/
  35, b19 33/35, b20 33/35, b21 13/15, b22 13/15, b23 13/15, b24 13/15
  base 0x0000: - 33/3d, b9 3b/3d, b10 3b/3d, b11 3b/3d, b12 3b/3d, b13 3b/3d, b14 3b/3d, b16 3b/3d, b17 3b/3d, b18 3b/
  3d, b19 3b/3d, b20 3b/3d, b21 1b/1d, b22 1b/1d, b23 1b/1d, b24 1b/1d
```

14.2.148 progtime (run 35)

ID: `D-progtime-35`. Device run 35. Experiment: progtime.

```
page program time per page (one run per page, the same pattern):
  CE0 block +60 (us per page, negative = failed):
      0:   478   478   478   477   728   781   478   478   728   821   478   478   781   821   478   478
     16:   782   822   478   478   729   820   478   478   728   781   478   478   781   781   478   478
     32:   781   821   478   478   781   782   478   478   728   820   478   478   781   821   478   478
     48:   781   820   478   478   780   781   478   478   781   820   478   478   781   782   478   478
     64:   781   821   477   478   781   820   479   478   780   820   478   478   781   781   478   479
     80:   781   821   478   478   781   821   478   478   781   820   479   478   781   820   479   478
     96:   781   781   479   478   729   821   478   478   728   781   478   471   781   781   478   478
    112:   729   781   478   478   781   781   477   478   781   821   479   478   728   781   729   782
  CE0 block +61 (us per page, negative = failed):
      0:   478   477   478   478   728   781   478   478   781   821   478   478   781   820   479   477
     16:   728   821   478   479   781   781   478   478   729   820   478   477   729   781   478   478
     32:   728   821   478   478   781   781   478   477   728   782   478   478   781   781   478   479
     48:   729   820   478   478   781   821   478   478   729   820   478   479   781   781   478   478
     64:   781   820   478   478   782   781   478   479   782   782   478   477   781   821   478   478
     80:   780   821   479   478   781   821   478   478   781   821   479   478   782   782   478   478
     96:   728   817   478   479   782   781   479   478   728   780   478   479   728   777   479   478
    112:   729   782   479   477   729   781   478   478   728   821   478   478   729   781   729   781
  CE1 block +60 (us per page, negative = failed):
      0:   477   478   478   478   835   834   530   532   834   834   477   478   833   834   531   478
     16:   833   833   478   478   834   833   532   478   833   834   472   478   834   834   531   478
     32:   834   835   478   477   833   833   531   478   834   873   531   478   873   834   530   478
     48:   873   833   478   531   834   833   531   478   835   833   478   526   873   834   531   478
     64:   834   873   479   478   873   834   530   478   834   873   478   532   874   835   531   478
     80:   834   835   478   531   833   834   478   478   833   873   478   530   833   834   478   478
     96:   834   873   478   531   835   833   478   478   834   833   478   478   874   834   477   478
    112:   835   833   478   478   834   834   530   478   833   833   478   478   873   834   782   781
```

14.2.149 regsweep (run 35)

ID: `D-regsweep-35`. Device run 35. Experiment: regsweep.

```
regsweep: A/B done to 31
regsweep: A/B done to 63
regsweep: A/B done to 95
regsweep: A/B done to 127
regsweep: A/B done to 159
regsweep: A/B done to 191
regsweep: A/B done to 223
regsweep: A/B done to 255
register numbers 8..255, every role:
 destinations (movi / mov r1 / add r1 + 1 into N; registers that changed, named by source):
  9 (1 numbers): END; movi: no register; mov: no register; add: no register
  10 (1 numbers): END; movi: r0=imm r1=imm r2=imm r3=imm r4=imm r5=imm r6=imm r7=imm; mov: r0=r1 r2=r1 r3=r1 r4=r1 r5=
  r1 r6=r1 r7=r1; add: r0=r1+1 r1=r1+1 r2=r1+1 r3=r1+1 r4=r1+1 r5=r1+1 r6=r1+1 r7=r1+1; window 428=000001d8 450=000020
  00 628=00000054 650=00002800
  11 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001c8 450=00002800 628=000
  00044 650=00003000
```

```
12 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001b8 450=00003000 628=000
00034 650=00003800
13 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001a8 450=00003800 628=000
00024 650=00000000
14 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000198 450=00000000 628=000
00014 650=00000800
15 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000188 450=00000800 628=000
00004 650=00001000
16 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000178 450=00001000 628=000
001f4 650=00001000
17 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000168 628=000001e4 650=000
01800
18 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000158 450=00002000 628=000
001d4
19 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000148 450=00002800 628=000
001c4 650=00002800
20 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000138 450=00003000 628=000
001b4 650=00003000
21 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000128 450=00003800 628=000
001a4 650=00003800
22 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000118 450=00000000 628=000
00194 650=00000000
23 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000108 450=00000800 628=000
00184 650=00000800
24 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000f8 450=00001000 628=000
00174 650=00001000
25 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000e8 628=00000164 650=000
01800
26 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000d8 450=00002000 628=000
00154
27 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000c8 450=00002800 628=000
00144 650=00002800
28 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000b8 450=00003000 628=000
00134 650=00003000
29 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000a8 450=00003800 628=000
00124 650=00003800
30 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000098 450=00000000 628=000
00114 650=00000000
31 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000088 450=00000800 628=000
00104 650=00000800
32 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000078 450=00001000 628=000
000f4 650=00001000
33 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000068 628=000000e4 650=000
01800
34 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000058 450=00002000 628=000
000d4
35 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000048 450=00002800 628=000
000c4 650=00002800
36 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000038 450=00003000 628=000
000b4 650=00003000
37 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000028 450=00003800 628=000
000a4 650=00003800
38 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000018 450=00000000 628=000
00094 650=00000000
39 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000008 450=00000800 628=000
00084 650=00000800
40 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001f8 450=00000800 628=000
00074 650=00001000
41 (1 numbers): END; movi: no register; mov: no register; add: no register; window 450=00001000 650=00001800
42 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001d8 628=00000054
43 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001c8 450=00002000 628=000
00044 650=00002800
44 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001b8 450=00002800 628=000
00034 650=00003000
45 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001a8 450=00003000 628=000
00024 650=00003800
46 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000198 450=00003800 628=000
00014 650=00000000
47 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000188 450=00000000 628=000
00004 650=00000800
48 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000178 450=00000800 628=000
001f4 650=00000800
49 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000168 450=00001000 628=000
001e4 650=00001000
50 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000158 628=000001d4 650=000
01800
51 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000148 450=00002000 628=000
001c4
52 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000138 450=00002800 628=000
001b4 650=00002800
53 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000128 450=00003000 628=000
001a4 650=00003000
54 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000118 450=00003800 628=000
00194 650=00003800
55 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000108 450=00000000 628=000
```

```
00184 650=00000000
56 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000f8 450=00000800 628=000
00174 650=00000800
57 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000e8 450=00001000 628=000
00164 650=00001000
58 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000d8 628=00000154 650=000
01800
59 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000c8 450=00002000 628=000
00144
60 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000b8 450=00002800 628=000
00134 650=00002800
61 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000a8 450=00003000 628=000
00124 650=00003000
62 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000098 450=00003800 628=000
00114 650=00003800
63 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000088 450=00000000 628=000
00104 650=00000000
64 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000078 450=00000800 628=000
000f4 650=00000800
65 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000068 450=00001000 628=000
000e4 650=00001000
66 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000058 628=000000d4 650=000
01800
67 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000048 450=00002000 628=000
000c4
68 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000038 450=00002800 628=000
000b4 650=00002800
69 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000028 450=00003000 628=000
000a4 650=00003000
70 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000018 450=00003800 628=000
00094 650=00003800
71 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000008 450=00000000 628=000
00084 650=00000000
72 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001f8 450=00000000 628=000
00074 650=00000800
73 (1 numbers): END; movi: no register; mov: no register; add: no register; window 450=00000800 650=00001000
74 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001d8 450=00001000 628=000
00054 650=00001800
75 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001c8 628=00000044
76 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001b8 450=00002000 628=000
00034 650=00002800
77 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001a8 450=00002800 628=000
00024 650=00003000
78 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000198 450=00003000 628=000
00014 650=00003800
79 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000188 450=00003800 628=000
00004 650=00000000
80 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000178 450=00000000 628=000
001f4 650=00000000
81 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000168 450=00000800 628=000
001e4 650=00000800
82 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000158 450=00001000 628=000
001d4 650=00001000
83 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000148 628=000001c4 650=000
01800
84 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000138 450=00002000 628=000
001b4
85 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000128 450=00002800 628=000
001a4 650=00002800
86 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000118 450=00003000 628=000
00194 650=00003000
87 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000108 450=00003800 628=000
00184 650=00003800
88 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000f8 450=00000000 628=000
00174 650=00000000
89 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000e8 450=00000800 628=000
00164 650=00000800
90 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000d8 450=00001000 628=000
00154 650=00001000
91 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000c8 628=00000144 650=000
01800
92 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000b8 450=00002000 628=000
00134
93 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000a8 450=00002800 628=000
00124 650=00002800
94 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000098 450=00003000 628=000
00114 650=00003000
95 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000088 450=00003800 628=000
00104 650=00003800
96 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000078 450=00000000 628=000
000f4 650=00000000
97 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000068 450=00000800 628=000
000e4 650=00000800
98 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000058 450=00001000 628=000
000d4 650=00001000
```

```
99 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000048 628=000000c4 650=000
01800
100 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000038 450=00002000 628=00
0000b4
101 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000028 450=00002800 628=00
0000a4 650=00002800
102 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000018 450=00003000 628=00
000094 650=00003000
103 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000008 450=00003800 628=00
000084 650=00003800
104 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001f8 450=00003800 628=00
000074 650=00000000
105 (1 numbers): END; movi: no register; mov: no register; add: no register; window 450=00000000 650=00000800
106 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001d8 450=00000800 628=00
000054 650=00001000
107 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001c8 450=00001000 628=00
000044 650=00001800
108 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001b8 628=00000034
109 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001a8 450=00002000 628=00
000024 650=00002800
110 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000198 450=00002800 628=00
000014 650=00003000
111 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000188 450=00003000 628=00
000004 650=00003800
112 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000178 450=00003800 628=00
0001f4 650=00003800
113 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000168 450=00000000 628=00
0001e4 650=00000000
114 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000158 450=00000800 628=00
0001d4 650=00000800
115 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000148 450=00001000 628=00
0001c4 650=00001000
116 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000138 628=000001b4 650=00
001800
117 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000128 450=00002000 628=00
0001a4
118 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000118 450=00002800 628=00
000194 650=00002800
119 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000108 450=00003000 628=00
000184 650=00003000
120 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000f8 450=00003800 628=00
000174 650=00003800
121 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000e8 450=00000000 628=00
000164 650=00000000
122 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000d8 450=00000800 628=00
000154 650=00000800
123 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000c8 450=00001000 628=00
000144 650=00001000
124 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000b8 628=00000134 650=00
001800
125 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000a8 450=00002000 628=00
000124
126 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000098 450=00002800 628=00
000114 650=00002800
127 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000088 450=00003000 628=00
000104 650=00003000
128 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000078 450=00003800 628=00
0000f4 650=00003800
129 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000068 450=00000000 628=00
0000e4 650=00000000
130 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000058 450=00000800 628=00
0000d4 650=00000800
131 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000048 450=00001000 628=00
0000c4 650=00001000
132 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000038 628=000000b4 650=00
001800
133 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000028 450=00002000 628=00
0000a4
134 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000018 450=00002800 628=00
000094 650=00002800
135 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000008 450=00003000 628=00
000084 650=00003000
136 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001f8 450=00003000 628=00
000074 650=00003800
137 (1 numbers): END; movi: no register; mov: no register; add: no register; window 450=00003800 650=00000000
138 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001d8 450=00000000 628=00
000054 650=00000800
139 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001c8 450=00000800 628=00
000044 650=00001000
140 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001b8 450=00001000 628=00
000034 650=00001800
141 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001a8 628=00000024
142 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000198 450=00002000 628=00
000014 650=00002800
143 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000188 450=00002800 628=00
```

```
000004 650=00003000
144 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000178 450=00003000 628=00
0001f4 650=00003000
145 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000168 450=00003800 628=00
0001e4 650=00003800
146 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000158 450=00000000 628=00
0001d4 650=00000000
147 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000148 450=00000800 628=00
0001c4 650=00000800
148 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000138 450=00001000 628=00
0001b4 650=00001000
149 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000128 628=000001a4 650=00
001800
150 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000118 450=00002000 628=00
000194
151 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000108 450=00002800 628=00
000184 650=00002800
152 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000f8 450=00003000 628=00
000174 650=00003000
153 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000e8 450=00003800 628=00
000164 650=00003800
154 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000d8 450=00000000 628=00
000154 650=00000000
155 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000c8 450=00000800 628=00
000144 650=00000800
156 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000b8 450=00001000 628=00
000134 650=00001000
157 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000a8 628=00000124 650=00
001800
158 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000098 450=00002000 628=00
000114
159 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000088 450=00002800 628=00
000104 650=00002800
160 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000078 450=00003000 628=00
0000f4 650=00003000
161 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000068 450=00003800 628=00
0000e4 650=00003800
162 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000058 450=00000000 628=00
0000d4 650=00000000
163 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000048 450=00000800 628=00
0000c4 650=00000800
164 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000038 450=00001000 628=00
0000b4 650=00001000
165 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000028 628=000000a4 650=00
001800
166 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000018 450=00002000 628=00
000094
167 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000008 450=00002800 628=00
000084 650=00002800
168 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001f8 450=00002800 628=00
000074 650=00003000
169 (1 numbers): END; movi: no register; mov: no register; add: no register; window 450=00003000 650=00003800
170 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001d8 450=00003800 628=00
000054 650=00000000
171 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001c8 450=00000000 628=00
000044 650=00000800
172 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001b8 450=00000800 628=00
000034 650=00001000
173 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001a8 450=00001000 628=00
000024 650=00001800
174 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000198 628=00000014
175 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000188 450=00002000 628=00
000004 650=00002800
176 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000178 450=00002800 628=00
0001f4 650=00002800
177 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000168 450=00003000 628=00
0001e4 650=00003000
178 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000158 450=00003800 628=00
0001d4 650=00003800
179 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000148 450=00000000 628=00
0001c4 650=00000000
180 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000138 450=00000800 628=00
0001b4 650=00000800
181 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000128 450=00001000 628=00
0001a4 650=00001000
182 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000118 628=00000194 650=00
001800
183 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000108 450=00002000 628=00
000184
184 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000f8 450=00002800 628=00
000174 650=00002800
185 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000e8 450=00003000 628=00
000164 650=00003000
186 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000d8 450=00003800 628=00
000154 650=00003800
```

```
187 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000c8 450=00000000 628=00
000144 650=00000000
188 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000b8 450=00000800 628=00
000134 650=00000800
189 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000a8 450=00001000 628=00
000124 650=00001000
190 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000098 628=00000114 650=00
001800
191 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000088 450=00002000 628=00
000104
192 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000078 450=00002800 628=00
0000f4 650=00002800
193 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000068 450=00003000 628=00
0000e4 650=00003000
194 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000058 450=00003800 628=00
0000d4 650=00003800
195 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000048 450=00000000 628=00
0000c4 650=00000000
196 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000038 450=00000800 628=00
0000b4 650=00000800
197 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000028 450=00001000 628=00
0000a4 650=00001000
198 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000018 628=00000094 650=00
001800
199 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000008 450=00002000 628=00
000084
200 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001f8 450=00002000 628=00
000074 650=00002800
201 (1 numbers): END; movi: no register; mov: no register; add: no register; window 450=00002800 650=00003000
202 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001d8 450=00003000 628=00
000054 650=00003800
203 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001c8 450=00003800 628=00
000044 650=00000000
204 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001b8 450=00000000 628=00
000034 650=00000800
205 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001a8 450=00000800 628=00
000024 650=00001000
206 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000198 450=00001000 628=00
000014 650=00001800
207 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000188 628=00000004
208 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000178 450=00002000 628=00
0001f4
209 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000168 450=00002800 628=00
0001e4 650=00002800
210 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000158 450=00003000 628=00
0001d4 650=00003000
211 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000148 450=00003800 628=00
0001c4 650=00003800
212 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000138 450=00000000 628=00
0001b4 650=00000000
213 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000128 450=00000800 628=00
0001a4 650=00000800
214 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000118 450=00001000 628=00
000194 650=00001000
215 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000108 628=00000184 650=00
001800
216 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000f8 450=00002000 628=00
000174
217 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000e8 450=00002800 628=00
000164 650=00002800
218 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000d8 450=00003000 628=00
000154 650=00003000
219 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000c8 450=00003800 628=00
000144 650=00003800
220 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000b8 450=00000000 628=00
000134 650=00000000
221 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000a8 450=00000800 628=00
000124 650=00000800
222 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000098 450=00001000 628=00
000114 650=00001000
223 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000088 628=00000104 650=00
001800
224 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000078 450=00002000 628=00
0000f4
225 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000068 450=00002800 628=00
0000e4 650=00002800
226 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000058 450=00003000 628=00
0000d4 650=00003000
227 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000048 450=00003800 628=00
0000c4 650=00003800
228 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000038 450=00000000 628=00
0000b4 650=00000000
229 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000028 450=00000800 628=00
0000a4 650=00000800
230 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000018 450=00001000 628=00
```

```
  000094 650=00001000
  231 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000008 628=00000084 650=00
  001800
  232 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001f8 628=00000074
  233 (1 numbers): END; movi: no register; mov: no register; add: no register; window 450=00002000 650=00002800
  234 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001d8 450=00002800 628=00
  000054 650=00003000
  235 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001c8 450=00003000 628=00
  000044 650=00003800
  236 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001b8 450=00003800 628=00
  000034 650=00000000
  237 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000001a8 450=00000000 628=00
  000024 650=00000800
  238 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000198 450=00000800 628=00
  000014 650=00001000
  239 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000188 450=00001000 628=00
  000004 650=00001800
  240 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000178 628=000001f4 650=00
  001800
  241 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000168 450=00002000 628=00
  0001e4
  242 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000158 450=00002800 628=00
  0001d4 650=00002800
  243 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000148 450=00003000 628=00
  0001c4 650=00003000
  244 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000138 450=00003800 628=00
  0001b4 650=00003800
  245 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000128 450=00000000 628=00
  0001a4 650=00000000
  246 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000118 450=00000800 628=00
  000194 650=00000800
  247 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000108 450=00001000 628=00
  000184 650=00001000
  248 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000f8 628=00000174 650=00
  001800
  249 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000e8 450=00002000 628=00
  000164
  250 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000d8 450=00002800 628=00
  000154 650=00002800
  251 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000c8 450=00003000 628=00
  000144 650=00003000
  252 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000b8 450=00003800 628=00
  000134 650=00003800
  253 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=000000a8 450=00000000 628=00
  000124 650=00000000
  254 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000098 450=00000800 628=00
  000114 650=00000800
  255 (1 numbers): END; movi: no register; mov: no register; add: no register; window 428=00000088 450=00001000 628=00
  000104 650=00001000
 sources (named: rK = seed of rK, CXX = preloaded control word, 0):
  9, 11..255 (246 numbers): END; mov r0; add+1 r0+1; or r1|r0; st r7; bnz not taken/not taken; bz not taken/not taken
  (r0 != 0 / r0 = 0); jr jumped to r0
  8 (1 numbers): TIMEOUT; mov 00000080; add+1 000000d1; or 5e015f7e; st r7; bnz not taken/not taken; bz not taken/not
  taken (r0 != 0 / r0 = 0); jr no landing
  10 (1 numbers): END; mov r1; add+1 5e015e5f; or r1|r0; st r7; bnz not taken/not taken; bz not taken/not taken (r0 !=
   0 / r0 = 0); jr jumped to r0
 addresses (ldi: which scratch marker was read; pld: which parameter-RAM marker; else the word read):
  9..255 (247 numbers): END; ldi addr r0; pld offset r0
  8 (1 numbers): END; ldi e90d1c00; pld 00000000
```

14.2.150 seqfirst (run 36)

ID: `D-seqfirst-36`. Device run 36. Experiment: seqfirst.

```
first mode-0 encode of the session: area 2e1c79573386228532df6ca2da 0x990 5bb85bb8 (END)
```

14.2.151 w2state (run 36)

ID: `D-w2state-36`. Device run 36. Experiment: w2state.

```
wait / wait2 passes per controller state (short timeout):
  first program of the session                        wait [4, 11]; wait2 [5]
  again, no reset                                     wait [4, 11]; wait2 [5]
  after a soft reset                                  wait [11]; wait2 [5]
  0x044 = 0xa5a5, 0x844 = 0x25 (CPU, before)          wait [11]; wait2 [5]
  command flag pending, 0x044 bit 0                   wait [0, 11]; wait2 [5, 20]
  command flag pending, 0x044 bit 1                   wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 2                   wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 3                   wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 4                   wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 5                   wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 6                   wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 7                   wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 8                   wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 9                   wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 10                  wait [0, 11]; wait2 [5]
```

```
  command flag pending, 0x044 bit 11                  wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 12                  wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 13                  wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 14                  wait [0, 11]; wait2 [5]
  command flag pending, 0x044 bit 15                  wait [0, 11]; wait2 [5]
  command flag pending, 0x844 = 0x2f                  wait [0, 11]; wait2 [5]
  address flag pending                                wait [1, 11]; wait2 [5]
  DMA flag pending                                    wait [2, 11]; wait2 [5]
  ECC_DONE pending                                    wait [4, 11]; wait2 [5]
  status flag pending                                 wait [5, 11]; wait2 [5]
  after a second-set READ ID                          wait [11]; wait2 [5]
  channel-1 pins switched                             wait [11]; wait2 [5]
  after a page read of CE0                            wait [4, 8, 11]; wait2 [5]
  C10 = 0x0                                           wait [11]; wait2 [5]
  C10 = 0xff                                          wait [11]; wait2 [5]
  no chip selected                                    wait [11]; wait2 [5]
  CE0 selected                                        wait [11]; wait2 [5]
  CE1 selected                                        wait [11]; wait2 [5]
  start word 0x3ff5 (step n = 15)                     wait [11]; wait2 [5]
```

14.2.152 c50b8 (run 36)

ID: D-c50b8-36. Device run 36. Experiment: c50b8.

```
C50 around runs (bit 8 = 0x100):
  END only (1)                   C50 before ffff1800 after ffff1800
  END only (2)                   C50 before ffff1800 after ffff1800
  soft reset, then END           C50 before ffff1800 after ffff1800
  sequencer reset, then END      C50 before ffff1800 after ffff1800
  fault (wait 10, C58 0)         C50 before ffff1800 after ffff1800
  second-set command             C50 before ffff1800 after ffff1840
  ECC decode                     C50 before ffff1840 after ffff1850
  END only, repeat 0             C50 before ffff1850 after ffff1850
  END only, repeat 1             C50 before ffff1850 after ffff1850
  END only, repeat 2             C50 before ffff1850 after ffff1850
  END only, repeat 3             C50 before ffff1850 after ffff1850
  END only, repeat 4             C50 before ffff1850 after ffff1850
  END only, repeat 5             C50 before ffff1850 after ffff1850
  END only, repeat 6             C50 before ffff1850 after ffff1850
  END only, repeat 7             C50 before ffff1850 after ffff1850
  END only, repeat 8             C50 before ffff1850 after ffff1850
  END only, repeat 9             C50 before ffff1850 after ffff1850
```

14.2.153 startregion (run 36)

ID: D-startregion-36. Device run 36. Experiment: startregion.

```
start words and NAND regions (page 0 of each block):
  CE0 block    0 (user): same under all four: END 335us data erased res 0 0 0 0 st 3c000 3c000 3c000 3c000
  CE0 block    1 (user): same under all four: END 309us data 1b4a2c73d4 res 0 0 0 0 st 803c000 803c000 803c000 803c000
  CE0 block    2 (user): same under all four: END 310us data 1fdd59bb55 res 0 0 0 0 st 803c000 803c000 803c000 803c000
  CE0 block  100 (user): same under all four: END 320us data fcbc08c0c1 res 0 0 0 0 st 803c000 803c000 803c000 803c000
  CE0 block 2000 (user): same under all four: END 309us data 11fd621527 res 0 0 0 0 st 803c000 803c000 803c000 803c000
  CE0 block 3000 (user): same under all four: END 324us data erased res 0 0 0 0 st 3c000 3c000 3c000 3c000
  CE0 block 3871 (user): same under all four: END 319us data 4da47e8d24 res 0 0 0 0 st 803c000 803c000 803c000 803c000
  CE0 block 3872 (above user): same under all four: END 310us data 22689fd562 res 0 0 0 0 st 803c000 803c000 803c000 8
  03c000
  CE0 block 3873 (above user): same under all four: END 310us data 106b5b49f3 res 0 0 0 0 st 803c000 803c000 803c000 8
  03c000
  CE0 block 3900 (above user): same under all four: END 310us data 39d0495b05 res 0 0 0 0 st 803c000 803c000 803c000 8
  03c000
  CE0 block 4000 (above user): same under all four: END 335us data erased res 0 0 0 0 st 3c000 3c000 3c000 3c000
  CE0 block 4064 (above user): same under all four: END 324us data erased res 0 0 0 0 st 3c000 3c000 3c000 3c000
  CE0 block 4094 (above user): same under all four: END 309us data b468b9c9a4 res 0 0 0 0 st 803c000 803c000 803c000 8
  03c000
  CE0 block 4095 (above user): same under all four: END 310us data b468b9c9a4 res 0 0 0 0 st 803c000 803c000 803c000 8
  03c000
  CE1 block    0 (user): same under all four: END 335us data erased res 0 0 0 0 st 3c000 3c000 3c000 3c000
  CE1 block    1 (user): same under all four: END 320us data 29f1115a1f res 0 0 0 0 st 803c000 803c000 803c000 803c000
  CE1 block    2 (user): same under all four: END 310us data 794dcfd051 res 0 0 0 0 st 803c000 803c000 803c000 803c000
  CE1 block  100 (user): same under all four: END 309us data e68af4508a res 0 0 0 0 st 803c000 803c000 803c000 803c000
  CE1 block 2000 (user): same under all four: END 319us data d8fb4f2f78 res 0 0 0 0 st 803c000 803c000 803c000 803c000
  CE1 block 3000 (user): same under all four: END 324us data erased res 0 0 0 0 st 3c000 3c000 3c000 3c000
  CE1 block 3871 (user): same under all four: END 320us data fafefb49e7 res 0 0 0 0 st 803c000 803c000 803c000 803c000
  CE1 block 3872 (above user): same under all four: END 320us data dcba5b945d res 0 0 0 0 st 803c000 803c000 803c000 8
  03c000
  CE1 block 3873 (above user): same under all four: END 319us data 34748cf023 res 0 0 0 0 st 803c000 803c000 803c000 8
  03c000
  CE1 block 3900 (above user): same under all four: END 335us data erased res 0 0 0 0 st 3c000 3c000 3c000 3c000
  CE1 block 4000 (above user): same under all four: END 324us data erased res 0 0 0 0 st 3c000 3c000 3c000 3c000
  CE1 block 4064 (above user): same under all four: END 325us data erased res 0 0 0 0 st 3c000 3c000 3c000 3c000
  CE1 block 4094 (above user): same under all four: END 319us data 529cdc3313 res 0 0 0 0 st 803c000 803c000 803c000 8
  03c000
  CE1 block 4095 (above user): same under all four: END 319us data 529cdc3313 res 0 0 0 0 st 803c000 803c000 803c000 8
  03c000
  CE0 user block 3002, written under fff5: erase END 179us NSTAT c0, program END 478us NSTAT c0; read fff5 END data ==
```

```
  == res 0 0 0 0 | ffb5 END data ==== res 0 0 0 0
  CE0 user block 3002, written under ffb5: erase END 1141us NSTAT c0, program END 478us NSTAT c0; read fff5 END data =
  === res 0 0 0 0 | ffb5 END data ==== res 0 0 0 0
  CE0 block 4000 (above user, read erased), written under fff5: erase END 1142us NSTAT c0, program END 478us NSTAT c0;
   read fff5 END data ==== res 0 0 0 0 | ffb5 END data ==== res 0 0 0 0
  CE0 block 4000 (above user, read erased), written under ffb5: erase END 1142us NSTAT c0, program END 479us NSTAT c0;
   read fff5 END data ==== res 0 0 0 0 | ffb5 END data ==== res 0 0 0 0
  CE1 block 3900 (above user, read erased), written under fff5: erase END 1155us NSTAT c0, program END 478us NSTAT c0;
   read fff5 END data ==== res 0 0 0 0 | ffb5 END data ==== res 0 0 0 0
  CE1 block 3900 (above user, read erased), written under ffb5: erase END 1155us NSTAT c0, program END 478us NSTAT c0;
   read fff5 END data ==== res 0 0 0 0 | ffb5 END data ==== res 0 0 0 0
```

14.2.154 bufport (run 36)

ID: `D-bufport-36`. Device run 36. Experiment: bufport.

```
FMC_FIFO (0x080) as a port onto the buffers (pattern i*29+7):
  data buffer 0, 0x340       once  END: first words 00000000000000000000000000000000; matches the buffer from byte Non
  e for 0 words; FMC_28 [4, 132, 260, 388, 4]
  data buffer 0, 0x340       twice END: first words 00000000000000000000000000000000; matches the buffer from byte Non
  e for 0 words; FMC_28 [7, 123, 239, 99, 215]; second read first words 00000000000000000000000000000000 (restarts)
  data buffer 1, 0x340       once  END: first words 00000000000000000000000000000000; matches the buffer from byte Non
  e for 0 words; FMC_28 [4, 132, 260, 388, 4]
  data buffer 1, 0x340       twice END: first words 00000000000000000000000000000000; matches the buffer from byte Non
  e for 0 words; FMC_28 [7, 123, 239, 99, 215]; second read first words 00000000000000000000000000000000 (restarts)
  data buffer 0, no 0x340    once  END: first words 00000000000000000000000000000000; matches the buffer from byte Non
  e for 0 words; FMC_28 [4, 132, 260, 388, 4]
  data buffer 0, no 0x340    twice END: first words 00000000000000000000000000000000; matches the buffer from byte Non
  e for 0 words; FMC_28 [4, 132, 260, 388, 4]; second read first words 00000000000000000000000000000000 (restarts)
  parity buffer 0, 0x340     once  END: first words 00000000000000000000000000000000; matches the buffer from byte Non
  e for 0 words; FMC_28 [4]
  parity buffer 0, 0x340     twice END: first words 00000000000000000000000000000000; matches the buffer from byte Non
  e for 0 words; FMC_28 [199]; second read first words 00000000000000000000000000000000 (restarts)
  write 0x080, FMC_DATA_SEL 1               END: buffer after: first words 0724415e7b98b5d2ef0c294663809dba; 2 of 128
   words carry the written marker
  write 0x080, FMC_DATA_SEL 1, FMCTRL1 0x340 END: buffer after: first words eeeeeeeeeeeeeeeeeeeeeeeeeeeeeeee; 26 of 12
  8 words carry the written marker
```

14.2.155 c58b2 (run 36)

ID: `D-c58b2-36`. Device run 36. Experiment: c58b2.

```
C58 bit 1, three runs per case (C58 / C4C / C5C as read before each start):
  CPU C58=2, C4C                                      END (C58 2 C4C 40 C5C 0) | END (C58 2 C4C 40 C5C 0) | END (C58
  2 C4C 40 C5C 0)
  CPU C4C, C58=2                                      END (C58 2 C4C 40 C5C 0) | END (C58 2 C4C 40 C5C 0) | END (C58
  2 C4C 40 C5C 0)
  CPU C58=2, program C4C (run-8 control, no clean-up)  END (C58 2 C4C 0 C5C 0) | END (C58 2 C4C 18 C5C 0) | END (C58 2
   C4C 18 C5C 0)
  CPU C58=2, program C4C (run-8 control, job steps)    END (C58 2 C4C 0 C5C 0) | END (C58 0 C4C 0 C5C 0) | END (C58 0
  C4C 0 C5C 0)
  program C58=2 and C4C                               END (C58 0 C4C 0 C5C 0) | END (C58 2 C4C 20 C5C 0) | END (C58 2
   C4C 20 C5C 0)
  CPU C58=3, C4C                                      END (C58 3 C4C 40 C5C 0) | END, then the vector (C58 3 C4C 40 C
  5C 1) | END, then the vector (C58 3 C4C 40 C5C 1)
  CPU C58=6, C4C                                      END, then the vector (C58 6 C4C 40 C5C 0) | END (C58 6 C4C 40 C
  5C 0) | END (C58 6 C4C 40 C5C 0)
```

14.2.156 stprev (run 36)

ID: `D-stprev-36`. Device run 36. Experiment: stprev.

```
st [0xd70], rN with N above 7, by the instruction before it:
  N=9   after seed only (last write movi r7)               st stored 88008888 (r7 seed)
  N=14  after seed only (last write movi r7)               st stored 88008888 (r7 seed)
  N=200 after seed only (last write movi r7)               st stored 88008888 (r7 seed)
  N=9   after movi r0, #0xa0a0a0a0                         st stored a0a0a0a0 ()
  N=14  after movi r0, #0xa0a0a0a0                         st stored a0a0a0a0 ()
  N=200 after movi r0, #0xa0a0a0a0                         st stored a0a0a0a0 ()
  N=9   after movi r3, #0x33333333                         st stored 33333333 ()
  N=14  after movi r3, #0x33333333                         st stored 33333333 ()
  N=200 after movi r3, #0x33333333                         st stored 33333333 ()
  N=9   after movi r7, #0x77777777                         st stored 77777777 ()
  N=14  after movi r7, #0x77777777                         st stored 77777777 ()
  N=200 after movi r7, #0x77777777                         st stored 77777777 ()
  N=9   after add r2, r2, #1                               st stored 33003334 ()
  N=14  after add r2, r2, #1                               st stored 33003334 ()
  N=200 after add r2, r2, #1                               st stored 33003334 ()
  N=9   after mov r4, r1                                   st stored 22002222 (r1 seed)
  N=14  after mov r4, r1                                   st stored 22002222 (r1 seed)
  N=200 after mov r4, r1                                   st stored 22002222 (r1 seed)
  N=9   after ld r5, [0xd74] (holds 0x5d5d5d5d)            st stored 5d5d5d5d ()
  N=14  after ld r5, [0xd74] (holds 0x5d5d5d5d)            st stored 5d5d5d5d ()
  N=200 after ld r5, [0xd74] (holds 0x5d5d5d5d)            st stored 5d5d5d5d ()
  N=9   after pld r6, [r0] (r0 = 0xd78, holds 0x6e6e6e6e)  st stored 6e6e6e6e ()
  N=14  after pld r6, [r0] (r0 = 0xd78, holds 0x6e6e6e6e)  st stored 6e6e6e6e ()
```

```
  N=200 after pld r6, [r0] (r0 = 0xd78, holds 0x6e6e6e6e)  st stored 6e6e6e6e ()
  N=9   after ldi r1, [r0] (scratch 0xcafe0001)            st stored cafe0001 ()
  N=14  after ldi r1, [r0] (scratch 0xcafe0001)            st stored cafe0001 ()
  N=200 after ldi r1, [r0] (scratch 0xcafe0001)            st stored cafe0001 ()
  N=9   after wr [0xd7c], #0x7c7c7c7c                      st stored 11001111 (r0 seed)
  N=14  after wr [0xd7c], #0x7c7c7c7c                      st stored 11001111 (r0 seed)
  N=200 after wr [0xd7c], #0x7c7c7c7c                      st stored 11001111 (r0 seed)
  N=9   after st [0xd7c], r5                               st stored 66006666 (r5 seed)
  N=14  after st [0xd7c], r5                               st stored 66006666 (r5 seed)
  N=200 after st [0xd7c], r5                               st stored 66006666 (r5 seed)
  N=9   after wait 11 (passes)                             st stored 88008888 (r7 seed)
  N=14  after wait 11 (passes)                             st stored 88008888 (r7 seed)
  N=200 after wait 11 (passes)                             st stored 88008888 (r7 seed)
  N=9   after nop                                          st stored 11001111 (r0 seed)
  N=14  after nop                                          st stored 11001111 (r0 seed)
  N=200 after nop                                          st stored 11001111 (r0 seed)
```

14.2.157 tmwait2 (run 36)

ID: `D-tmwait2-36`. Device run 36. Experiment: tmwait2.

```
tmwait and C14:
  run 1                     C14 before (CPU) ff | start ff, after each tmwait 00 ff 00 ff | 9702 us
  run 2                     C14 before (CPU) ff | start ff, after each tmwait 00 ff 00 ff | 9704 us
  run 3                     C14 before (CPU) ff | start ff, after each tmwait 00 ff 00 ff | 9704 us
  run after a timeout run   C14 before (CPU) ff | start ff, after each tmwait 00 ff 00 ff | 9713 us
  run after a soft reset    C14 before (CPU) ff | start ff, after each tmwait 00 ff 00 ff | 9721 us
```

14.2.158 irqop (run 36)

ID: `D-irqop-36`. Device run 36. Experiment: irqop.

```
ERROR (tool bug, please report): USBTimeoutError(110, 'Operation timed out')
```

14.2.159 done5b (run 37)

ID: `D-done5b-37`. Device run 37. Experiment: done5b.

```
ECC_DONE (encode / decode) per mode, each case after a soft reset:
  0x0000180: enc 33, enc+dec 33/35, dec(0x180)+enc 33
  0x0200180: enc 13, enc+dec 13/15, dec(0x180)+enc 13
  0x0400180: enc 13, enc+dec 13/15, dec(0x180)+enc 13
  0x0800180: enc 13, enc+dec 13/15, dec(0x180)+enc 13
  0x1000180: enc 13, enc+dec 13/15, dec(0x180)+enc 13
```

14.2.160 rsresult (run 37)

ID: `D-rsresult-37`. Device run 37. Experiment: rsresult.

```
ECC_RESULT before -> after an encode:
  before: nothing              0x8100: 00000000 -> 00000000 | 0x8104: 00000000 -> 00000000 | 0x8108: 00000000 -> 00000
  000 | 0x180: 00000000 -> 00000000
  before: BCH decode, 0 bits   0x8100: 00000000 -> 00000000 | 0x8104: 00000000 -> 00000000 | 0x8108: 00000000 -> 00000
  000 | 0x180: 00000000 -> 00000000
  before: BCH decode, 3 bits   0x8100: 00030000 -> 00000000 | 0x8104: 00030000 -> 00000000 | 0x8108: 00030000 -> 00000
  000 | 0x180: 00030000 -> 00030000
  before: RS decode, 0 bytes   0x8100: 00000000 -> 00000000 | 0x8104: 00000000 -> 00000000 | 0x8108: 00000000 -> 00000
  000 | 0x180: 00000000 -> 00030000
  before: RS decode, 2 bytes   0x8100: 00020000 -> 00020000 | 0x8104: 00020000 -> 00020000 | 0x8108: 00020000 -> 00020
  000 | 0x180: 00020000 -> 00030000
```

14.2.161 rschip (run 37)

ID: `D-rschip-37`. Device run 37. Experiment: rschip.

```
Reed-Solomon on the chip (block 3070):
  erase: END NSTAT c0
  page 0 mode 0x8100: program END NSTAT c0; read END data ==== results 0 11 11 11; chunk 0 as stored 2a5c0a397c8a9f585
  350ffffffffffff
  page 1 mode 0x8104: program END NSTAT c0; read END data ==== results 0 11 11 11; chunk 0 as stored 905dc73be102afc7c
  cbe2db808e706ff
  page 2 mode 0x180: program END NSTAT c0; read END data ==== results 0 0 0 0; chunk 0 as stored ffffff1fce51281e06ae8
  c16922b553a
  page 3 mode 0x8100: program END NSTAT c0; second program (4 zero bytes per sector) END NSTAT c0; read END data =xxx
  results 40000 11 11 11; chunk 0 as stored 2a5c0a397c8a9f585350ffffffffffff
  page 4 mode 0x8100: program END NSTAT c0; second program (5 zero bytes per sector) END NSTAT c0; read END data xxxx
  results 11 11 11 11; chunk 0 as stored 2a5c0a397c8a9f585350ffffffffffff
  page 5 mode 0x8104: program END NSTAT c0; second program (6 zero bytes per sector) END NSTAT c0; read END data =xxx
  results 60000 11 11 11; chunk 0 as stored 905dc73be102afc7ccbe2db808e706ff
  page 6 mode 0x8104: program END NSTAT c0; second program (7 zero bytes per sector) END NSTAT c0; read END data xxxx
  results 11 11 11 11; chunk 0 as stored 905dc73be102afc7ccbe2db808e706ff
  RS-16 into slot 3: parity buffer 0 4e2923086f4cfc4b7b77718291507950ffa89d05ffffffffffffffffffffffffffffffffffffffff
  ffffffffffffffffffffffffffffffffffffffffffffffff | parity buffer 1 ffffffffffffffffffffffffffffffffffffffffffffffffffffffff
  ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff | decode of slot 3 with 2 bad bytes:
  ECC_RESULT 00020000, data corrected
```

14.2.162 miscorr (run 37)

ID: `D-miscorr-37`. Device run 37. Experiment: miscorr.

```
decodes far beyond the code's strength:
  BCH   uncorrectable 240, corrected, right data 0, MISCORRECTED 0, UNDETECTED (reported no error) 0
  RS-8  uncorrectable 240, corrected, right data 0, MISCORRECTED 0, UNDETECTED (reported no error) 0
```

14.2.163 fullpage (run 37)

ID: `D-fullpage-37`. Device run 37. Experiment: fullpage.

```
whole 4 KiB pages, metadata bytes of the two units:
  CE0 block    1 page  0: END metadata unit 0 ffffffffffffffff0080ffff unit 1 ffffffffffffffff0080ffff (SAME)
  CE0 block    1 page  1: END metadata unit 0 ffffffffffffffff0080ffff unit 1 ffffffffffffffff0080ffff (SAME)
  CE0 block    1 page 64: END metadata unit 0 f7ffffffffffffff0080ffff unit 1 f7ffffffffffffff0080ffff (SAME)
  CE0 block    2 page  0: END metadata unit 0 efffffffffffffff0080ffff unit 1 efffffffffffffff0080ffff (SAME)
  CE0 block    2 page  1: END metadata unit 0 efffffffffffffff0080ffff unit 1 efffffffffffffff0080ffff (SAME)
  CE0 block    2 page 64: END metadata unit 0 e7ffffffffffffff0080ffff unit 1 e7ffffffffffffff0080ffff (SAME)
  CE0 block   10 page  0: END metadata unit 0 00d01600ef4e0100ff40ffff unit 1 00d01600ef4e0100ff40ffff (SAME)
  CE0 block   10 page  1: END metadata unit 0 04d01600ef4e0100ff40ffff unit 1 04d01600ef4e0100ff40ffff (SAME)
  CE0 block   10 page 64: END metadata unit 0 00d11600ef4e0100ff40ffff unit 1 00d11600ef4e0100ff40ffff (SAME)
  CE0 block  100 page  0: END metadata unit 0 00140200ff340100ff40ffff unit 1 00140200ff340100ff40ffff (SAME)
  CE0 block  100 page  1: END metadata unit 0 04140200ff340100ff40ffff unit 1 04140200ff340100ff40ffff (SAME)
  CE0 block  100 page 64: END metadata unit 0 00150200ff340100ff40ffff unit 1 00150200ff340100ff40ffff (SAME)
  CE0 block  500 page  0: END metadata unit 0 004e0b0019e90100ff40ffff unit 1 004e0b0019e90100ff40ffff (SAME)
  CE0 block  500 page  1: END metadata unit 0 044e0b0019e90100ff40ffff unit 1 044e0b0019e90100ff40ffff (SAME)
  CE0 block  500 page 64: END metadata unit 0 004f0b0019e90100ff40ffff unit 1 004f0b0019e90100ff40ffff (SAME)
  CE0 block 1000 page  0: END metadata unit 0 00d211008a780100ff40ffff unit 1 00d211008a780100ff40ffff (SAME)
  CE0 block 1000 page  1: END metadata unit 0 04d211008a780100ff40ffff unit 1 04d211008a780100ff40ffff (SAME)
  CE0 block 1000 page 64: END metadata unit 0 00d311008c780100ff40ffff unit 1 00d311008c780100ff40ffff (SAME)
  CE0 block 2000 page  0: END metadata unit 0 00cc1800a3100100ff40ffff unit 1 00cc1800a3100100ff40ffff (SAME)
  CE0 block 2000 page  1: END metadata unit 0 04cc1800a3100100ff40ffff unit 1 04cc1800a3100100ff40ffff (SAME)
  CE0 block 2000 page 64: END metadata unit 0 00cd1800a3100100ff40ffff unit 1 00cd1800a3100100ff40ffff (SAME)
  CE1 block    1 page  0: END metadata unit 0 ffffffffffffffff0080ffff unit 1 ffffffffffffffff0080ffff (SAME)
  CE1 block    1 page  1: END metadata unit 0 ffffffffffffffff0080ffff unit 1 ffffffffffffffff0080ffff (SAME)
  CE1 block    1 page 64: END metadata unit 0 f7ffffffffffffff0080ffff unit 1 f7ffffffffffffff0080ffff (SAME)
  CE1 block    2 page  0: END metadata unit 0 efffffffffffffff0080ffff unit 1 efffffffffffffff0080ffff (SAME)
  CE1 block    2 page  1: END metadata unit 0 efffffffffffffff0080ffff unit 1 efffffffffffffff0080ffff (SAME)
  CE1 block    2 page 64: END metadata unit 0 e7ffffffffffffff0080ffff unit 1 e7ffffffffffffff0080ffff (SAME)
  CE1 block   10 page  0: END metadata unit 0 01d01600ef4e0100ff40ffff unit 1 01d01600ef4e0100ff40ffff (SAME)
  CE1 block   10 page  1: END metadata unit 0 05d01600ef4e0100ff40ffff unit 1 05d01600ef4e0100ff40ffff (SAME)
  CE1 block   10 page 64: END metadata unit 0 01d11600ef4e0100ff40ffff unit 1 01d11600ef4e0100ff40ffff (SAME)
  CE1 block  100 page  0: END metadata unit 0 01140200ff340100ff40ffff unit 1 01140200ff340100ff40ffff (SAME)
  CE1 block  100 page  1: END metadata unit 0 05140200ff340100ff40ffff unit 1 05140200ff340100ff40ffff (SAME)
  CE1 block  100 page 64: END metadata unit 0 01150200ff340100ff40ffff unit 1 01150200ff340100ff40ffff (SAME)
  CE1 block  500 page  0: END metadata unit 0 014e0b0019e90100ff40ffff unit 1 014e0b0019e90100ff40ffff (SAME)
  CE1 block  500 page  1: END metadata unit 0 054e0b0019e90100ff40ffff unit 1 054e0b0019e90100ff40ffff (SAME)
  CE1 block  500 page 64: END metadata unit 0 014f0b0019e90100ff40ffff unit 1 014f0b0019e90100ff40ffff (SAME)
  CE1 block 1000 page  0: END metadata unit 0 01d211008a780100ff40ffff unit 1 01d211008a780100ff40ffff (SAME)
  CE1 block 1000 page  1: END metadata unit 0 05d211008a780100ff40ffff unit 1 05d211008a780100ff40ffff (SAME)
  CE1 block 1000 page 64: END metadata unit 0 01d311008c780100ff40ffff unit 1 01d311008c780100ff40ffff (SAME)
  CE1 block 2000 page  0: END metadata unit 0 01cc1800a3100100ff40ffff unit 1 01cc1800a3100100ff40ffff (SAME)
  CE1 block 2000 page  1: END metadata unit 0 05cc1800a3100100ff40ffff unit 1 05cc1800a3100100ff40ffff (SAME)
  CE1 block 2000 page 64: END metadata unit 0 01cd1800a3100100ff40ffff unit 1 01cd1800a3100100ff40ffff (SAME)
```

14.2.164 dmactl (run 37)

ID: `D-dmactl-37`. Device run 37. Experiment: dmactl.

```
FMC_DMA_CTRL and data selects on a page read (block 3051):
  program: END NSTAT c0
  drain 0x100 FMC_DMA_CTRL -     END      317us data ====; FMC_DMA_CTRL after 00000007
  drain 0x100 FMC_DMA_CTRL 0x1   END      373us data ====; FMC_DMA_CTRL after 00000001
  drain 0x100 FMC_DMA_CTRL 0x2   FAULT  29495us data =xxx; FMC_DMA_CTRL after 00000042
  drain 0x100 FMC_DMA_CTRL 0x4   FAULT  29477us data =xxx; FMC_DMA_CTRL after 00000084
  drain 0x100 FMC_DMA_CTRL 0x8   END      438us data ====; FMC_DMA_CTRL after 00000008
  drain 0x100 FMC_DMA_CTRL 0x10  FAULT  48965us data xxxx; FMC_DMA_CTRL after 00000010
  drain 0x100 FMC_DMA_CTRL 0x20  FAULT  48974us data xxxx; FMC_DMA_CTRL after 00000020
  drain 0x100 FMC_DMA_CTRL 0x40  FAULT  48971us data xxxx; FMC_DMA_CTRL after 00000040
  drain 0x100 FMC_DMA_CTRL 0x80  FAULT  48972us data xxxx; FMC_DMA_CTRL after 00000080
  drain 0x1e0 FMC_DMA_CTRL -     END      451us data ====; FMC_DMA_CTRL after 00000000
  drain 0x1e0 FMC_DMA_CTRL 0x1   END      383us data ====; FMC_DMA_CTRL after 00000001
  drain 0x1e0 FMC_DMA_CTRL 0x2   FAULT  29486us data =xxx; FMC_DMA_CTRL after 00000042
  drain 0x1e0 FMC_DMA_CTRL 0x4   FAULT  29486us data =xxx; FMC_DMA_CTRL after 00000084
  drain 0x1e0 FMC_DMA_CTRL 0x8   END      437us data ====; FMC_DMA_CTRL after 00000008
  drain 0x1e0 FMC_DMA_CTRL 0x10  FAULT  48966us data xxxx; FMC_DMA_CTRL after 00000010
  drain 0x1e0 FMC_DMA_CTRL 0x20  FAULT  48966us data xxxx; FMC_DMA_CTRL after 00000020
  drain 0x1e0 FMC_DMA_CTRL 0x40  FAULT  48967us data xxxx; FMC_DMA_CTRL after 00000040
  drain 0x1e0 FMC_DMA_CTRL 0x80  FAULT  48970us data xxxx; FMC_DMA_CTRL after 00000080
  combined selects, data buffers alternating (read 0x201 / 0x102, DMA 0x102 / 0x201): END data ==== results 0 0 0 0
```

14.2.165 fmcstat2 (run 37)

ID: `D-fmcstat2-37`. Device run 37. Experiment: fmcstat2.

```
FMC_STATUS bits 12, 13, 18, 19:
  nothing                 bits 12/13/18/19 after the 16-byte read 1010, after the operation 1010 (END)
  DMA into data buffer 0  bits 12/13/18/19 after the 16-byte read 1010, after the operation 1111 (END)
  DMA out of data buffer 0 bits 12/13/18/19 after the 16-byte read 1010, after the operation 1010 (END)
  DMA into parity buffer 0 bits 12/13/18/19 after the 16-byte read 1010, after the operation 1111 (END)
  chip read 512 bytes     bits 12/13/18/19 after the 16-byte read 1010, after the operation 0000 (END)
  ECC encode              bits 12/13/18/19 after the 16-byte read 1010, after the operation 1010 (END)
  status latch            bits 12/13/18/19 after the 16-byte read 1010, after the operation 1111 (END)
  command                 bits 12/13/18/19 after the 16-byte read 1010, after the operation 1010 (END)
  second-set data read    bits 12/13/18/19 after the 16-byte read 1010, after the operation 1010 (END)
  set-up word             bits 12/13/18/19 after the 16-byte read 1010, after the operation 1111 (END)
```

14.2.166 ch1buf (run 37)

ID: `D-ch1buf-37`. Device run 37. Experiment: ch1buf.

```
buffers of the second register set (DMA via 0x400):
  second set, parity buffer 0 END: back = pattern; first set's parity buffer 0 CHANGED (c5d2dfecf90613202d3a4754616e7b
  88)
  second set, parity buffer 1 END: back = pattern; first set's parity buffer 0 CHANGED (ffffffffffffffffffffffffffff
  ff)
  second set, data buffer 1   END: back = pattern; first set's parity buffer 0 CHANGED (ffffffffffffffffffffffffffff
  ff)
  second set, data buffer 0   END: back = pattern; first set's parity buffer 0 CHANGED (ffffffffffffffffffffffffffff
  ff)
```

14.2.167 evsweep3 (run 37)

ID: `D-evsweep3-37`. Device run 37. Experiment: evsweep3.

```
wait / wait2 passes during further operations (short timeout):
  second-set DMA in flight       wait [8, 11]; wait2 [5]
  meta encode in flight          wait [11]; wait2 [5]
  meta decode in flight          wait [11]; wait2 [5]
  ECC decode in flight           wait [4, 11]; wait2 [5]
  chip data write in flight      wait [11]; wait2 [5]
  parity-buffer DMA in flight    wait [11]; wait2 [5]
```

14.2.168 blk100b (run 37)

ID: `D-blk100b-37`. Device run 37. Experiment: blk100b.

```
0x100 block words and 0x3a8..0x3b4:
  default            END  10153us; wait 10 timeout, wait2 33 timeout, wait2 5 pass
  0x1bc = 0          END  10169us; wait 10 timeout, wait2 33 timeout, wait2 5 pass
  0x1c0 = 0          END  10154us; wait 10 timeout, wait2 33 timeout, wait2 5 pass
  0x1c4 = 0          END  10157us; wait 10 timeout, wait2 33 timeout, wait2 5 pass
  0x1c8 = 0          END  10158us; wait 10 timeout, wait2 33 timeout, wait2 5 pass
  0x1cc = 0          END  10145us; wait 10 timeout, wait2 33 timeout, wait2 5 pass
  0x1d0 = 0          END  10154us; wait 10 timeout, wait2 33 timeout, wait2 5 pass
  0x1d4 = 0          END  10143us; wait 10 timeout, wait2 33 timeout, wait2 5 pass
  0x1d8 = 0          END  10156us; wait 10 timeout, wait2 33 timeout, wait2 5 pass
  0x1b8 = 1          END  10158us; wait 10 timeout, wait2 33 timeout, wait2 5 pass
  0x1cc = 0xffffff   END  10157us; wait 10 timeout, wait2 33 timeout, wait2 5 pass
  0x1d0 = 0xffffff   END  10143us; wait 10 timeout, wait2 33 timeout, wait2 5 pass
  0x3a8..0x3b4: CPU wrote c0de000x; sequencer read ffffffff ffffffff ffffffff ffffffff; sequencer wrote 3a83a80x, read
   back ffffffff ffffffff ffffffff ffffffff; CPU reads after: 00000000 00000000 00000000 00000000
```

14.2.169 straps (run 37)

ID: `D-straps-37`. Device run 37. Experiment: straps.

```
GPIO / PHY words: PCON1 11111100 PDAT1 00000003 PCON3 33331222 PDAT3 00000017 PHY+0x28 0000000f; P1.0 1 P1.1 1; P3.6 0
   P3.7 0 -> boot device SPI flash, SPI port 0; PHY+0x28 bit 0 = 1
```

14.2.170 irqop (run 37)

ID: `D-irqop-37`. Device run 37. Experiment: irqop.

```
0x044 bit x operation (flag left pending): CS_IRQ bit 4, VIC0 line 20, wait event 15:
  none      command: b4 0 V0.20 0 ev15 timeout | address: b4 0 V0.20 0 ev15 timeout | data read: b4 0 V0.20 0 ev15 tim
  eout | status latch: b4 0 V0.20 0 ev15 timeout | DMA: b4 0 V0.20 0 ev15 timeout
  bit 0     command: b4 1 V0.20 1 ev15 timeout | address: b4 0 V0.20 0 ev15 timeout | data read: b4 0 V0.20 0 ev15 tim
  eout | status latch: b4 1 V0.20 0 ev15 timeout | DMA: b4 0 V0.20 0 ev15 timeout
  bit 1     command: b4 0 V0.20 0 ev15 timeout | address: b4 1 V0.20 1 ev15 timeout | data read: b4 0 V0.20 0 ev15 tim
  eout | status latch: b4 0 V0.20 0 ev15 timeout | DMA: b4 0 V0.20 0 ev15 timeout
  bit 2     command: b4 0 V0.20 0 ev15 timeout | address: b4 0 V0.20 0 ev15 timeout | data read: b4 1 V0.20 1 ev15 tim
  eout | status latch: b4 0 V0.20 0 ev15 timeout | DMA: b4 0 V0.20 0 ev15 timeout
  bit 11    command: b4 0 V0.20 0 ev15 timeout | address: b4 0 V0.20 0 ev15 timeout | data read: b4 0 V0.20 0 ev15 tim
  eout | status latch: b4 0 V0.20 0 ev15 timeout | DMA: b4 1 V0.20 1 ev15 timeout
  bit 12    command: b4 0 V0.20 0 ev15 timeout | address: b4 0 V0.20 0 ev15 timeout | data read: b4 0 V0.20 0 ev15 tim
  eout | status latch: b4 1 V0.20 1 ev15 timeout | DMA: b4 0 V0.20 0 ev15 timeout
```

14.2.171 w2vic (run 38)

ID: `D-w2vic-38`. Device run 38. Experiment: w2vic.

```
wait2 conditions against the raw interrupt lines (VIC0 0..31, VIC1 32..63):
  idle                                FAULT: wait2 passed [5]; raw lines set [5]; agree 64 of 64
  command flag pending, 0x044 bit 0   FAULT: wait2 passed [5, 20]; raw lines set [5, 20]; agree 64 of 64
  DMA flag pending, 0x044 bit 11      FAULT: wait2 passed [5, 20]; raw lines set [5, 20]; agree 64 of 64
  ECC_DONE pending, 0x844 bit 0       FAULT: wait2 passed [5, 43]; raw lines set [5, 43]; agree 64 of 64
  wait2 64..255, idle: passed []; VIC0 / VIC1 raw seen during the run 00000020 / 00000000
```

14.3 Reverse-engineering outcomes

Observations of the original firmware (osos = iPod OS image, aupd = restore OS image, bootrom = mask ROM). osos and aupd addresses: IMG1 body at file+0x800, mem = 0x08000000 + file - 0x800 for osos and for the first aupd copy of the programs. Each entry states only what the bytes or the code at the cited addresses contain: the address read or written, the constants, masks, shifts and compares applied, the branch that follows a test, the instruction found at a program offset, and counts of bounded searches. Register and parameter names are labels for window offsets (chapter 4, chapter 9, chapter 10); a label says nothing about use. Sequencer instructions are written with the mnemonics of chapter 5, section 5.3. +0xNNN is a byte offset from a program's first instruction; "the C4C offset" is the immediate of the program's opcode-01 write to FMSS_C4C (0xc4c). Program starts are listed in RE-3. What the observations are taken to show is in chapter 15.

Notation in the entries. A window offset is written 0xNNN. "0xNNN = V" is an opcode-01 instruction (wr) that writes the immediate V to window offset 0xNNN; "FMC_CMD = V" is the same for offset 0x008. "wait event N" is an opcode-07 instruction whose R field is N. "ld rR from 0xNNN mask M" is an opcode-04 instruction that loads window offset 0xNNN ANDed with M into register rR; "st rR to 0xNNN" is opcode 02, ldi is opcode 03, sti opcode 11, bnz opcode 0e and bz opcode 17. movi, mov, and, or, add, sub, shl and shr are opcodes 05, 06, 0a, 0b, 0c, 0d, 13 and 14. Registers are r0 to r7. Offsets of the form +0xNNN are byte offsets within the named program. Addresses written [x + 0xNN] are the word at address x plus 0xNN. ARM code is cited by address and written with ARM mnemonics.

14.3.1 RE-1: Nine sequencer programs in the iPod OS image

Citation: osos file+0x9430,0x9830,0xa488,0xa698,0xaff0,0xb438,0xbdf8,0xc7b8,0xd138; aupd 18 hits.

The 24 bytes 0c0c0001 ff000000 100c0001 01000000 580c0001 04000000 (three instructions: opcode 01 with arg 0x0c0c imm 0xff; opcode 01 with arg 0x0c10 imm 1; opcode 01 with arg 0x0c58 imm 4) occur 9 times in the osos body, at file+0x9430, 0x9830, 0xa488, 0xa698, 0xaff0, 0xb438, 0xbdf8, 0xc7b8 and 0xd138, and 18 times in the aupd body: at file+0x2ee5c, 0x2f25c, 0x2feb4, 0x300c4, 0x30a1c, 0x30e64, 0x31824, 0x321e4, 0x32b64 and at file+0xe76f0, 0xe7af0, 0xe8748, 0xe8958, 0xe92b0, 0xe96f8, 0xea0b8, 0xeaa78, 0xeb3f8. The byte ranges starting at the nine osos hits (lengths in RE-2) are equal to the ranges at the nine hits of each aupd set (equal SHA-1 values, position by position). In every one of the nine programs the fourth instruction is opcode 01 with arg 0x0c4c and an immediate that is a multiple of 8 below the program length; the instruction at that offset is opcode 04 with r=2, arg 0x0c3c and imm 0xffffffff. Each program contains two opcode-00 instructions: one at the C4C offset minus 8 and the last instruction of the program, which is also the first opcode 00 at or after the C4C offset. From the C4C offset to the last instruction there are 13 instructions in ERASE and in the five WRITE programs and 9 in READ, READID and RAWREAD. The start of each program is the byte after the last instruction of the previous program (no gap), in osos and in each aupd set.

14.3.2 RE-2: Program table

Citation: osos file+0x9430..0xd138 (table of the nine programs).

program	instructions	bytes	C4C offset	osos file	aupd file (first set / second set)	FMC_CMD values
ERASE	128	0x400	0x398	+0x9430	+0x2ee5c / +0xe76f0	70 60 D0 70
READ	395	0xc58	0xc10	+0x9830	+0x2f25c / +0xe7af0	00 30 70 00 70 00
READID	66	0x210	0x1c8	+0xa488	+0x2feb4 / +0xe8748	FF 90
WRITE	299	0x958	0x8f0	+0xa698	+0x300c4 / +0xe8958	80 10 70 80 10 70
RAWREAD	137	0x448	0x400	+0xaff0	+0x30a1c / +0xe92b0	00 30 70 00
WRITE2P	312	0x9c0	0x958	+0xb438	+0x30e64 / +0xe96f8	80 11 10 70 80 81 10 70
WRITE2P_78	312	0x9c0	0x958	+0xbdf8	+0x31824 / +0xea0b8	80 11 10 78 80 10 70
WRITE_78	304	0x980	0x918	+0xc7b8	+0x321e4 / +0xeaa78	80 10 78 80 10 70
WRITECACHE	319	0x9f8	0x990	+0xd138	+0x32b64 / +0xeb3f8	80 11 10 15 70 80 10 71

The last column lists the immediates of the opcode-01 writes to window offset 0x008 (FMC_CMD), in the order they appear in the code. The program names are working names.

14.3.3 RE-3: Program locations as memory addresses

Citation: `osos mem:0x8008c30` (ERASE), `mem:0x8009030` (READ), `mem:0x8009c88` (READID), `mem:0x8009e98` (WRITE), `mem:0x800a7f0` (RAWREAD), `mem:0x800ac38` (WRITE2P), `mem:0x800b5f8` (WRITE2P_78), `mem:0x800bfb8` (WRITE_78), `mem:0x800c938` (WRITECACHE); `aupd mem:0x0802e65c..0x08032d5c`.

The nine programs start at the `osos` addresses above (IMG1 body at file+0x800, mem = 0x08000000 + file - 0x800). The first `aupd` set starts at `aupd mem:` 0x0802e65c (ERASE), 0x0802ea5c (READ), 0x0802f6b4 (READID), 0x0802f8c4 (WRITE), 0x0803021c (RAWREAD), 0x08030664 (WRITE2P), 0x08031024 (WRITE2P_78), 0x080319e4 (WRITE_78), 0x08032364 (WRITECACHE) and ends at 0x08032d5c; the second set starts at 0x080e6ef0, 0x080e72f0, 0x080e7f48, 0x080e8158, 0x080e8ab0, 0x080e8ef8, 0x080e98b8, 0x080ea278, 0x080eabf8. The bytes of all nine programs in both `aupd` sets are equal to those of the `osos` programs. Bounded search of the 16 bytes `0c0c0001 ff000000 100c0001 01000000` (first two prologue instructions): 9 matches in the `osos` body, 18 in the `aupd` body, 0 in the `wtf` body, 0 in `bootrom`; a search for the 4 bytes `4c0c0001` (opcode 01, arg 0x0c4c) gives the same counts 9, 18, 0, 0.

14.3.4 RE-4: Instruction encoding and opcode set used by the programs

Citation: decoded streams of the nine programs.

Each instruction is 8 bytes: bytes 0..1 `arg`, byte 2 `r`, byte 3 `op`, bytes 4..7 `imm`, little endian (the first instruction of ERASE is `0c 0c 00 01 ff 00 00 00`). The op bytes that occur are `00 01 02 03 04 05 06 07 0a 0b 0c 0d 0e 11 13 14 17 18 19`. Op counts over all nine programs: 00:18 01:852 02:223 03:85 04:255 05:68 06:25 07:236 0a:35 0b:88 0c:97 0d:27 0e:92 11:9 13:74 14:34 17:51 18:2 19:1; total 2272 instructions. No program contains op `08`, `09`, `0f`, `10`, `12`, `15`, `16`, `1a..fe` or `ff`. The `imm` of every op `0e` and `17` instruction, and the `imm` written to `0xc4c`, is a multiple of 8 smaller than the length of its program.

14.3.5 RE-5: Prologue values

Citation: first 0x30 bytes of every program.

In ERASE, READ, WRITE, RAWREAD, WRITE2P, WRITE2P_78, WRITE_78 and WRITECACHE the first six instructions are opcode 01 writes: `0xc0c` = 0xff (+0x00), `0xc10` = 1 (+0x08), `0xc58` = 4 (+0x10), `0xc4c` = the C4C offset (+0x18), `0x004` = 0x0ff3f8e0 (+0x20), `0x048` = 0x0ff00ffe (+0x28). In READID the first four instructions are the same writes of `0xc0c`, `0xc10`, `0xc58`, `0xc4c`; instructions 5..7 are opcode 04 loads of `0xd08` (r7), `0xd0c` (r6), `0xd10` (r5) with mask 0xffffffff; `0x004` = 0x0ff3f8e0 is written at +0x38 and `0x048` = 0x0ff00ffe at +0x40.

14.3.6 RE-6: Opcodes 18 and 19

Citation: `osos mem:0x8009110` (READ+0xe0, op `19`), `mem:0x8009190` (READ+0x160, op `18`), `mem:0x8009710` (READ+0x6e0, op `18`); `osos file+0x9910` (op `19`, r=1, arg=3), `osos file+0x9990` (op `18`, r=7, arg=5), `osos file+0x9f10` (op `18`, r=2, arg=0); `movi r3,#0xd60` at `osos file+0x9890`, `movi r5,#0xd60` at `osos file+0x9988`, `0xd50` initialised to `0xd64` at `osos file+0x9978`.

Op `19` occurs once, at READ+0xe0 (r=1, arg=3, imm 0); op `18` occurs twice, at READ+0x160 (r=7, arg=5, imm 0) and READ+0x6e0 (r=2, arg=0, imm 0); no other program contains either. READ+0x60 is `movi r3,#0xd60`; READ+0xe8 is `add r3,r3,#4`; READ+0x158 is `movi r5,#0xd60`; READ+0x168 is `add r5,r5,#4`; READ+0x148 and READ+0x150 are `movi r0,#0xd64` and a `st` of r0 to `0xd50`; at READ+0x6d8 `ld r0` reads `0xd50`, READ+0x6e8 adds 4 to r0 and READ+0x6f0 stores r0 back to `0xd50`. READ+0x238 is `ld r5` from `0xd60`. The value in r1 at READ+0xe0 is the one-hot word built at READ+0xb0..0xc0 (`movi r0,#1; shl r0,r0,r7; mov r1,r0`).

14.3.7 RE-7: Wait event 5 shape

Citation: 19 uses, e.g. `osos file+0x9ac8`; `osos mem:0x80092c8` (READ+0x298), `mem:0x800a578` (WRITE+0x6e0).

`wait` with `R` = 5 occurs 19 times, at: ERASE +0x120, +0x148, +0x178, +0x2f0, +0x318, +0x348; READ +0x298, +0x750; WRITE +0x6e0, +0x8a8; RAWREAD +0x1a8; WRITE2P +0x720, +0x910; WRITE2P_78 +0x748, +0x910; WRITE_78 +0x708, +0x8d0; WRITECACHE +0x780, +0x948. The 13 outside ERASE have the same neighbours: three instructions before the wait are opcode 01 `0x004` = 0xca, opcode 04 `r0` from `0x04c` with mask 0x40, and opcode 0e `bnz r0` with a target equal to the address of the instruction after the wait; the instruction after the wait is opcode 01 `0x048` = 0x800000. The 6 in ERASE are described in RE-8. The `R` values of the `wait` instructions in the nine programs are 0, 1, 2, 3, 4 and 5 only (no `wait` with `R` = 12).

14.3.8 RE-8: ERASE ready waits

Citation: `osos mem:0x8008d30` (ERASE+0x100); `osos file+0x9548..0x95a8` and `0x9718..0x9778`.

First group, ERASE+0x100..+0x1a0: +0x100 `0x004` = 0xca; +0x108 `ld r0` from `0x04c` mask 0x40; +0x110 `bnz r0` to +0x180; +0x118 `0xc58` = 0; +0x120 `wait` event 5; +0x128 `0xc14` = 0xff; +0x130 `0xc0c` = 4; +0x138 `ld r0` from `0x04c` mask 0x40; +0x140 `bnz r0` to +0x180; +0x148 `wait` event 5; +0x150 `0xc14` = 0xff; +0x158 `0xc0c` = 4; +0x160 `ld r0` from `0x04c` mask 0x40; +0x168 `bnz r0` to +0x180; +0x170 `0xc58` = 4; +0x178 `wait` event 5; +0x180 `0xc58` = 4; +0x188 `0x048` = 0x800000; +0x190 `ld r2` from `0x04c` mask 1; +0x198 `0x004` = 0x20; +0x1a0 `bnz r2` to the C4C offset 0x398. Second group, ERASE+0x2e8..+0x370: the same instruction sequence at +0x2e8 (`0xc58` = 0) to +0x370, with `bnz` targets +0x350, the `ld` of `0x04c` mask 1 into r4 at +0x360 and `bnz r4` to 0x398 at +0x370.

14.3.9 RE-9: Control word and chip-enable changes

Citation: `osos mem:0x8009060` (READ+0x30), `mem:0x8009370` (READ+0x340), `mem:0x8009f28` (WRITE+0x90).

Offset `0x000` (`FMC_CTRL0`) is written at +0x30..+0x48 of ERASE, READ, WRITE, RAWREAD and the other four WRITE programs by `movi r0,#0x801`, `ld r1` from `0xd4c` (from `0xd48` in the five WRITE programs), `or r0,r0,r1`, `st` r0 to `0x000`. Chip changes (ERASE +0xb0, READ +0x178 and +0x248 and +0x700, WRITE +0xb0, RAWREAD +0xc0, and the corresponding positions in the other programs): `ld r1` from `0x000` with mask 0xfffffe01, `or r1,r1,r0`, `st` r1 to `0x000`, where r0 is a value shifted left by 1 just before (READ +0x170 `shl r0,r7,#1`; ERASE +0xa8 `shl r0,r3,#1`, with r3 = `1 << r2` built at +0x98..+0xa0). Mask 0xfffffe01 is also loaded and stored back without an `or` at the end of each program (for example READ +0xbe8). Bit 24: `ld r0` from `0x000`, `or r0,r0,#0x1000000`, `st`, at READ +0x340..+0x350, WRITE +0xf8..+0x108, RAWREAD +0x208..+0x218 (WRITE2P and WRITE2P_78 +0x100, WRITE_78 +0xf8, WRITECACHE +0x120); `ld` with mask 0xfeffffff and `st` back at READ +0xbd8, WRITE +0x780, RAWREAD +0x3d8, WRITE2P and WRITE2P_78 +0x7e8, WRITE_78 +0x7a8, WRITECACHE +0x820. ERASE and READID contain neither.

14.3.10 RE-10: READID control word

Citation: `osos mem:0x8009cd0` (READID+0x48).

READID+0x48 is opcode 01 `0x000` = 0x00033201 (binary bits set: 0, 9, 12, 13, 16, 17); bit 9 is set and bit 11 is clear in this value, whereas the other programs build `0x000` from 0x801 (bits 0 and 11) ORed with a parameter word. READID+0x50 writes `0x014` = 0; READID+0x58 is `movi r3,#0xff`, READID+0x60 `shl r0,r3,#1`, followed by the mask-0xfffffe01 read-modify-write of `0x000` (+0x68..+0x78).

14.3.11 RE-11: FMCTRL1 codes and FMC_DATA_SEL fields

Citation: counts over all nine programs.

Opcode-01 writes to `0x004` (`FMCTRL1`) over the nine programs, value:count: 0x0ff3f8e0:9, 0x1:16, 0xca:15, 0x20:15, 0x100e0:3, 0x1c2:1, 0x340:1, 0x32:6, 0xe2:11, 0x34:25, 0xe4:5, 0x2e0:5, 0x2e4:20, 0x100:9, 0x1e0:2. Opcode-01 writes to `0x014` (`FMC_DATA_SEL`): 0x0:2, 0x1:23, 0x10:6, 0x100:1, 0x102:16, 0x200:7, 0x201:13, 0x1000:25. Opcode-01 writes to `0x030` (`FMC_DNUM`): 0x7:1, 0xf:31, 0x1ff:43. No program writes `0x004`, `0x014` or `0x030` with opcode 02.

14.3.12 RE-12: ECC protocol in READ

Citation: `osos mem:0x80093b0` (READ+0x380..); `osos file+0x9bf0..0x9c30`; 36 writes of `0x80c` (6 with bit 0 set, 30 with bit 1 set).

READ writes `0x80c` six times, with values and offsets 0x1101 (+0x3d8), 0x11201 (+0x4c8), 0x21101 (+0x5b8), 0x31201 (+0x6d0), 0x31201 (+0x998), 0x31201 (+0xa98). Each of the six is preceded by `0x840` = 0x1ff and `0x814` = 0x180 (for the first: +0x3c8 and +0x3d0), followed by `wait` event 4 (at +0x3e0, +0x4d0, +0x5c0, +0x768, +0x9a0, +0xaa0), then `0x840` = 1, then `ld r1` from `0x810` with mask 0xffffffff (at +0x3f0, +0x4e0, +0x5d0, +0x778, +0x9b0, +0xab0). WRITE, WRITE2P, WRITE2P_78, WRITE_78 and WRITECACHE write `0x80c` six times each, 30 in total, with values 0x1102 (three times), 0x11202, 0x21102 and 0x31202 (WRITE: +0x1c0, +0x2d0, +0x360, +0x3f0, +0x500, +0x5c8), each preceded by `0x840` = 0x1ff and `0x814` = 0x180; each of these programs contains six `wait` event 4, each followed by `0x840` = 1. In total 36 writes of `0x80c`. The only window offsets of 0x800 and above that any program accesses are `0x80c`, `0x810` (READ only), `0x814` and `0x840`. ERASE, READID and RAWREAD contain no access to offsets of 0x800 and above.

14.3.13 RE-13: Page verdict in READ

Citation: `osos mem:0x8009408` (READ+0x3d8..0x440); `osos file+0x9c20..0x9c60`.

After the `ld r1` from `0x810` at +0x3f0: +0x3f8 `and r0,r1,#1`; +0x400 `bz r0` to +0x418; +0x408 `or r5,r5,#0x40000000`; +0x410 `bnz r0` to +0x448. At +0x418: `shr r1,r1,#0x10`; `and r1,r1,#0xf`; `movi r0,#1`; `shl r0,r0,r1`; `shr r1,r0,#1`; +0x440 `or r5,r5,r1`. The same instruction sequence follows each of the six `ld` of `0x810` (offsets in RE-12), with the branch targets shifted accordingly.

14.3.14 RE-14: Decode order

Citation: READ+0x3d8..+0x468; READ+0x4c8..+0xb30.

READ+0x3d8 writes `0x80c` = 0x1101; +0x3e0 `wait` event 4; +0x3e8 `0x840` = 1; +0x3f0 `ld r1` from `0x810`; the sequence of RE-13 follows to +0x440; +0x448 writes `0x004` = 0x100; +0x450 `wait` event 2; +0x458 `0x048` = 0x100000; +0x460 `wait` event 3; +0x468 `0x048` = 8. The order `0x80c` write, `wait` event 4, `0x004` = 0x100, `wait` event 2 also holds for the `0x80c` writes at +0x4c8 (`0x004` = 0x100 at +0x538, `wait` event 2 at +0x540), +0x5b8 (+0x628, +0x630) and +0x998 (+0xa08, +0xa10). For the `0x80c` write at +0x6d0, `wait` event 4 is at +0x768, the `0x07c` sequence of RE-16 follows, and `0x004` = 0x100 is at +0x8e0 with `wait` event 2 at +0x8e8. For the `0x80c` write at +0xa98, `wait` event 4 is at +0xaa0, followed by `0x038` = 7 (+0xb08), `0x014` = 0x200, `0x030` = 0x1ff, `0x004` = 0x1e0 (+0xb20) and `wait` event 2 (+0xb28).

14.3.15 RE-15: Page stream

Citation: `FMC_DNUM = 0xf` then `0x1ff` in READ+0x2e8..; `osos file+0x9b30..0x9c00`; `0x34` then `0x2e4` in WRITE+0x268; `osos file+0xa918`.

READ+0x2e8..+0x338: `0x030` = 0xf; `0x014` = 0x10; `0x024` = 0; `0x004` = 0x32; `wait` event 3; `0x048` = 8; `0x030` = 0x1ff; `0x014` = 1; `0x004` = 0xe2; `wait` event 3; `0x048` = 8. The `0x024` writes in READ are 0 (+0x2f8), 0x10 (+0x390), 0x20 (+0x480), 0x30 (+0x570), 0 (+0x878), 0 (+0x950), each followed by `0x004` = 0x32 after `0x030` = 0xf and `0x014` = 0x10, and then by `0x030` = 0x1ff, `0x014` = 0x102 or 0x201 (0x1 in the first block) and `0x004` = 0xe2. WRITE+0x268..: `0x030` = 0xf; `0x014` = 0x1000; `0x028` = 0; `0x004` = 0x34; `wait` event 3; `0x048` = 8; `0x014` = 0x102; `0x030` = 0x1ff; `0x004` = 0x2e4; `wait` event 2; `0x048` = 0x100000. The `0x028` writes in WRITE are 0 (+0x278), 0x10 (+0x308), 0x20 (+0x398), 0x30 (+0x478), 0x30 (+0x7a0).

14.3.16 RE-16: Meta engine use

Citation: `osos mem:0x8009ef0..0x800a018` (WRITE+0x58..0x180), `mem:0x8009800` (READ+0x7d0); `osos file+0xa000, 0xa818`.

WRITE+0x128..+0x178: `ld r1` from `0xd1c`; three times (`ldi r0,[r1]`, `add r1,r1,#4`, `st` r0 to `0x060`, `0x064`, `0x068` in turn); `st` r1 to `0xd1c`. +0x180 `0x078` = 0x3210; +0x188 `0x07c` = 1; +0x190 `ld r0` from `0x07c` mask 1; +0x198 `bnz r0` to +0x190. This block occurs again at +0x530..+0x5a0 (WRITE), after the `bnz r7` at +0x528. The `0x07c` = 1 writes are at WRITE +0x188 and +0x590, WRITE2P and WRITE2P_78 +0x190 and +0x598, WRITE_78 +0x188 and +0x590, WRITECACHE +0x1b0 and +0x5b8. In WRITE the first block is before the first `0x80c` write (+0x1c0). READ+0x7d0: `0x078` = 0x3210; +0x7d8 `0x07c` = 2; +0x7e0 `ld r0` from `0x07c` mask 2; +0x7e8 `bnz r0` to +0x7e0; then three times (`ld r0` from `0x060`, `0x064`, `0x068` in turn, `sti [r1],r0`, `add r1,r1,#4`) with r1 loaded from `0xd1c` at +0x7f0 and stored back at +0x840. The block occurs a second time at READ +0xb38..+0xba8. Every `0x078` write in the nine programs has the value 0x3210. ERASE, READID and RAWREAD contain no access to `0x060..0x07c`.

14.3.17 RE-17: READ batches

Citation: `osos mem:0x8009088` (READ+0x58..0x158), `mem:0x8009190` (READ+0x160); `osos file+0x9860..0x9b30`.

READ+0x58..+0x100 is a loop: `movi r2,#0` (+0x58) and `movi r3,#0xd60` (+0x60); +0x68 `ld r4` from `0xd18`; +0x70..+0x88 `ld r1` from `0xd10`, `ldi r7,[r1]`, `add r1,r1,#4`, `st` r1 to `0xd10`; +0x90 `shl r7,r7,#2`; +0x98..+0xa8 `ld r0` from `0xd14`, `add r0,r0,r7`, `ldi r7,[r0]`; +0xb0..+0xc0 `movi r0,#1`, `shl r0,r0,r7`, `mov r1,r0`; +0xc8 `and r0,r0,r2`; +0xd0 `bnz r0` to +0x108; +0xd8 `or r2,r2,r1`; +0xe0 op 19; +0xe8 `add r3,r3,#4`; +0xf0 `sub r4,r4,#1`; +0xf8 `bz r4` to +0x108; +0x100 `bnz r3` to +0x70. +0x108..+0x150: `ld r0` from `0xd10`, `sub r0,r0,#4`, `st` to `0xd10`; `sub r3,r3,#0xd60`; `shr r3,r3,#2`; `st` r3 to `0xd5c`; `st` r2 to `0xd58`; `st` r4 to `0xd18`; `movi r0,#0xd64`, `st` to `0xd50`. +0x158..+0x230: `movi r5,#0xd60`, op 18 (+0x160), `add r5,r5,#4`, the command and address cycles with `FMC_CMD` = 0x00 (+0x190), `0x02c` = 4, `FMC_CMD` = 0x30 (+0x210), `sub r3,r3,#1` (+0x228), `bnz r3` to +0x160 (+0x230). Results: at +0x8b0..+0x8d0, `ld r0` from `0xd24`, `sti [r0],r5`, `add r0,r0,#4`, `st` to `0xd24`, `or r6,r6,r5`.

14.3.18 RE-18: Page word and FMC_STATUS b27

Citation: `osos mem:0x80096d0` (READ+0x6a0), `mem:0x8009a98` (READ+0xa68); `osos file+0x9ed0`, `0xa298`.

READ+0x6a0: `ld r0` from `0x048` with mask 0x08000000; +0x6a8 `0x048` = 0x08000000; +0x6b0 `bnz r0` to +0x6c0; +0x6b8 `movi r5,#0x20000000`. The same four instructions are at +0xa68..+0xa80 (targets +0xa88). At +0x8b0..+0x8d8 and +0xbb0..+0xbd0 the word in r5 is stored with `sti` to the address read from `0xd24` (which is then increased by 4 and stored back), r5 is ORed into r6, and (at +0x8d8) r5 is set to 0.

14.3.19 RE-19: DMA address and DMA control

Citation: `osos mem:0x80093b0` (READ+0x380), `mem:0x8009b38` (READ+0xb08), `mem:0x800a658` (WRITE+0x7c0).

`st` to `0x034` (FMC_DMA_ADDR) occurs at READ +0x378, +0x918, +0xa40; WRITE +0xf0, +0x4a8; RAWREAD +0x240, +0x3a0; WRITE2P and WRITE2P_78 +0xf8, +0x4b0; WRITE_78 +0xf0, +0x4a8; WRITECACHE +0x118, +0x4d0; in each case the stored register was loaded by `ldi` from the address in `0xd20` three instructions earlier (`ld r1` from `0xd20`, `ldi r2,[r1]`, `add r1,r1,#4`, `st` r1 to `0xd20`, `st` r2 to `0x034`). `0x038` (FMC_DMA_CTRL) = 7 is written once in each of READ (+0xb08), WRITE (+0x7c0), RAWREAD (+0x3a8), WRITE2P and WRITE2P_78 (+0x828), WRITE_78 (+0x7e8) and WRITECACHE (+0x860); ERASE and READID never write `0x034` or `0x038`.

14.3.20 RE-20: WRITE DMA list

Citation: WRITE+0x468...

In WRITE the five-instruction group `ld r1` from `0xd20`, `ldi r2,[r1]`, `add r1,r1,#4`, `st` r1 to `0xd20`, `st` r2 to `0x034` occurs at +0xd0..+0xf0 (before the first `0x80c` write) and at +0x488..+0x4a8 (after `0x028` = 0x30, `0x004` = 0x34 at +0x478..+0x480). The loop word `0xd54` is set from `0xd28` at +0x248..+0x250 and again at +0x438..+0x440, decremented at +0x418..+0x428 (`ld r7`, `sub r7,r7,#1`, `st`), with `bnz r7` to +0x468 at +0x430. `0xd18` is decremented at +0x448..+0x458 (`ld r2`, `sub r2,r2,#1`, `st`), with `bz r2` to +0x780 at +0x460. The same structure is present in the other four WRITE programs at offsets shifted by the differences in RE-2.

14.3.21 RE-21: Two-plane status polls

Citation: `osos mem:0x800b220` (WRITE2P+0x5e8), `mem:0x800b3a8` (WRITE2P+0x770).

WRITE2P+0x5e8: `bnz r5` to +0x618; +0x5f0 FMC_CMD = 0x11; +0x5f8 `wait` event 0; +0x608 `movi r5,#1`; +0x610 `bnz r5` to +0x638; +0x618 FMC_CMD = 0x10; +0x620 `wait` event 0; +0x630 `movi r5,#0`. +0x638..+0x6b8: the list and map reads of RE-29 for the next entry, the read-modify-write of `0x000` with mask 0xfffffe01, `and r3,r3,r6` (+0x6b0), `or r6,r6,r2` (+0x6b8); +0x6e0 `bz r3` to +0x748. When r3 is non-zero: +0x6e8 `0x014` = 1; +0x6f0 FMC_CMD = 0x70; +0x708 `0x004` = 0xca; +0x710 `ld r0` from `0x04c` mask 0x40; +0x718 `bnz r0` to +0x728; +0x720 `wait` event 5; +0x728 `0x048` = 0x800000; +0x730 `ld r4` from `0x04c` mask 1; +0x738 `0x004` = 0x20; +0x740 `bnz r4` to 0x958. +0x748 `bnz r5` to +0x770; +0x750 FMC_CMD = 0x80; +0x768 `bz r5` to +0x788; +0x770 FMC_CMD = 0x81. In WRITE2P_78 FMC_CMD = 0x11 is at +0x5f0 and 0x10 at +0x618; in WRITECACHE 0x11 is at +0x610, 0x10 at +0x658 and 0x15 at +0x678.

14.3.22 RE-22: Program FAIL path

Citation: `osos mem:0x800a788` (WRITE+0x8f0).

In WRITE, `ld r4` from `0x04c` with mask 1 at +0x6f0 is followed by `0x004` = 0x20 (+0x6f8) and `bnz r4` to +0x8f0 (+0x700); the same triple is at +0x8b8..+0x8c8 (`bnz` target +0x8f0). The instruction at +0x8f0 is the first of the C4C tail described in RE-25; in WRITE its `ld r0` from `0x000` mask 0x1fe is at +0x8f8, `shr r0,r0,#1` at +0x900, `and r6,r6,#0xffffff00` at +0x908, `or r6,r6,r0` at +0x910, and `or r6,r6,#0x10000000` at +0x948.

14.3.23 RE-23: ERASE sequence

Citation: `osos mem:0x8008c88`, `mem:0x8008e78` (ERASE+0x58, +0x248).

ERASE+0x50 `movi r6,#0`. +0x58..+0xc0: the list and map reads of RE-29 with the read-modify-write of `0x000` with mask 0xfffffe01. +0xc8 `mov r2,r3`; +0xd0 `and r2,r2,r6`; +0xd8 `bz r2` to +0x1a8. When r2 is non-zero: +0xe0 `0x014` = 1; +0xe8 FMC_CMD = 0x70; +0xf0 `wait` event 0; +0xf8 `0x048` = 2; then the first group of RE-8 (+0x100..+0x1a0). +0x1a8 `or r6,r6,r3`; +0x1b0 FMC_CMD = 0x60; +0x1b8 `wait` event 0; +0x1c8..+0x1e0 `ld r1` from `0xd0c`, `ldi r2,[r1]`, `add r1,r1,#4`, `st` r1 to `0xd0c`; +0x1e8 `0x02c` = 2; +0x1f0 `st` r2 to `0x00c`; +0x1f8 `0x004` = 1; +0x200 `wait` event 1; +0x208 `0x048` = 4; +0x210 FMC_CMD = 0xd0; +0x218 `wait` event 0; +0x228..+0x238 `ld r2` from `0xd18`, `sub r2,r2,#1`, `st` r2 to `0xd18`; +0x240 `bnz r2` to +0x58. +0x248 `movi r2,#1`; +0x250 `movi r3,#8`; +0x260 `sub r3,r3,#1` and +0x268 `bz r3` to +0x380; +0x270 `shl r2,r2,#1`; +0x278..+0x288 `mov r0,r2`, `and r0,r0,r6`, `bz r0` to +0x260; when non-zero +0x290..+0x2a8 `shl r0,r2,#1` and the mask-0xfffffe01 read-modify-write of `0x000`, +0x2b0 `0x014` = 1, +0x2b8 FMC_CMD = 0x70, +0x2c0 `wait` event 0, then the second group of RE-8 (+0x2d0..+0x370); +0x378 `bnz r3` to +0x260; +0x380..+0x388 the mask-0xfffffe01 load and store of `0x000`; +0x390 `end` (r6 is the value at that point).

14.3.24 RE-24: READID sequence

Citation: `osos mem:0x8009cd0`, `mem:0x8009d20`, `mem:0x8009d48`; `osos mem:0x8009e00` (READID+0x178); `osos file+0xa508, 0xa5d8, 0xa600`; delay loop `osos file+0xa520..0xa538`.

READID+0x58 `movi r3,#0xff`; +0x60 `shl r0,r3,#1`; +0x68..+0x78 the mask-0xfffffe01 read-modify-write of `0x000`; +0x80 `FMC_CMD` = 0xff; +0x88 `wait` event 0; +0x90 `0x048` = 2. +0x98 `ld r1` from `0xd14`; +0xa0 `sub r1,r1,#1`; +0xa8 `bnz r1` to +0xa0. +0xb0 `ld r1` from `0x000` mask 0xfffffe01 and +0xb8 `st` back. Loop head +0xc0 (`0x014` = 0): +0xc8 `ldi r2,[r5]`; +0xd0 `movi r3,#1`; +0xd8 `shl r3,r3,r2`; +0xe0 `shl r0,r3,#1`; +0xe8..+0xf8 the read-modify-write of `0x000`; +0x100 `FMC_CMD` = 0x90; +0x108 `wait` event 0; +0x118 `0x02c` = 0; +0x120 `0x00c` = 0; +0x128 `0x004` = 1; +0x130 `wait` event 1; +0x140 `0x030` = 7; +0x148 `0x014` = 1; +0x150 `0x004` = 0x1c2; +0x158 `wait` event 3; +0x168 `0x014` = 0x100; +0x170 `0x004` = 0x340; +0x178 `ld r0` from `0x080`; +0x180 `sti [r7],r0`; +0x188 `ld r0` from `0x080` (not stored); +0x190..+0x198 the read-modify-write of `0x000` with mask 0xfffffe01 and no `or`; +0x1a0 `add r5,r5,#4`; +0x1a8 `add r7,r7,#4`; +0x1b0 `sub r6,r6,#1`; +0x1b8 `bnz r6` to +0xc0; +0x1c0 `end`. r7, r6 and r5 hold the words loaded from `0xd08`, `0xd0c` and `0xd10` at +0x20..+0x30.

14.3.25 RE-25: Fault handlers

Citation: `osos mem:0x8009c40` (READ+0xc10), `mem:0x800a788` (WRITE+0x8f0), `mem:0x8008fc8` (ERASE+0x398).

Every program's C4C tail begins `ld r2` from `0xc3c`; `ld r0` from `0x000` with mask 0x1fe; `shr r0,r0,#1`; `and r6,r6,#0xffffff00`; `or r6,r6,r0`. After that: in READ (+0xc38..+0xc50), READID (+0x1f0..+0x208) and RAWREAD (+0x428..+0x440): `ld r0` from `0xc50` mask 1; `0xc50` = 1; `or r6,r6,#0x8000000`; `end`. In ERASE (+0x3c0..+0x3f8), WRITE (+0x918..+0x950), WRITE2P, WRITE2P_78, WRITE_78 and WRITECACHE: `ld r0` from `0xc50` mask 1; `bz r0` to the instruction after the `or`; `0xc50` = 1; `or r6,r6,#0x8000000`; then the mask-0xfffffe01 read-modify-write of `0x000` without `or`; `or r6,r6,#0x10000000`; `end`. The `bz` is absent in READ, READID and RAWREAD; `or r6,r6,#0x10000000` is absent in those three.

14.3.26 RE-26: RAWREAD

Citation: `osos mem:0x800aa50`; `osos file+0xaff0..0xb438`; `osos file+0xb250, 0xb290, 0xb2d0, 0xb310`.

RAWREAD +0x1e0..+0x1f8: `0x030` = 0x1ff, `0x014` = 1, `0x004` = 0xe2, `wait` event 3. +0x250..+0x258 stores `0xd28` into `0xd54`. Loop +0x260..+0x378: four groups of `0x014` = 0x102; `0x030` = 0x1ff; `0x004` = 0xe2; `0x004` = 0x100; `wait` event 3; `0x048` = 8; `wait` event 2; `0x048` = 0x100000 (at +0x260, +0x2a0, +0x2e0, +0x320); +0x360..+0x378 `ld r7` from `0xd54`, `sub r7,r7,#1`, `st`, `bnz r7` to +0x260. +0x3a0 `st` to `0x034`; +0x3a8 `0x038` = 7; +0x3b0 `0x014` = 0x200; +0x3b8 `0x030` = 0x1ff; +0x3c0 `0x004` = 0x1e0; +0x3c8 `wait` event 2. RAWREAD contains no write of `0x030` = 0xf or `0x004` = 0x32, and no access to offsets `0x800` and above. The `0x014` values written in RAWREAD are 0x102 (four times), 0x1 (twice) and 0x200; READ also writes 0x201 (three times) and 0x102 (twice), WRITE writes 0x201 (twice) and 0x102 (twice).

14.3.27 RE-27: Write variants

Citation: WRITE_78+0x6b0, WRITECACHE+0x638..+0x678; `osos file+0xba28, 0xbba8, 0xc4e8, 0xce68`; `osos file+0xd198, 0xd1b0, 0xd7b0, 0xda50`.

`FMC_CMD` writes with their offsets: WRITE2P 0x80 (+0x1d0), 0x11 (+0x5f0), 0x10 (+0x618), 0x70 (+0x6f0), 0x80 (+0x750), 0x81 (+0x770), 0x10 (+0x858), 0x70 (+0x8e0). WRITE2P_78: 0x80 (+0x1d0), 0x11 (+0x5f0), 0x10 (+0x618), 0x78 (+0x6f0), 0x80 (+0x770), 0x10 (+0x858), 0x70 (+0x8e0). WRITE_78: 0x80 (+0x1c8), 0x10 (+0x5e0), 0x78 (+0x6b0), 0x80 (+0x730), 0x10 (+0x818), 0x70 (+0x8a0). After each `0x78` write the instructions are `wait` event 0, `0x048` = 2, `0x02c` = 2, `st` r2 to `0x00c`, `0x004` = 1, `wait` event 1, `0x048` = 4, `0x004` = 0xca, `ld r0` from `0x04c` mask 0x40, `bnz`, `wait` event 5, `0x048` = 0x800000, `ld r4` from `0x04c` with mask 3 (WRITE_78 +0x718, WRITE2P_78 +0x758), `0x004` = 0x20, `bnz r4` to the C4C offset; the corresponding `ld r4` after the `0x70` writes (for example WRITE2P +0x730) has mask 1. WRITECACHE: `ld r1` from `0xd18` (+0x638); `ld r0` from `0xd44` (+0x640); `and r0,r0,r1` (+0x648); `bnz r0` to +0x678 (+0x650); `FMC_CMD` = 0x10 (+0x658); `bz r0` to +0x690 (+0x670); `FMC_CMD` = 0x15 (+0x678). `0xd44` is stored at +0x78 from r0, where r0 = 0 (+0x68) minus the word loaded from `0xd04` (+0x60, +0x70). WRITECACHE's last group writes `FMC_CMD` = 0x71 (+0x918) and has `ld r4` from `0x04c` with mask 1 (+0x958).

14.3.28 RE-28: Releases 1.0.1 to 1.1.3

Citation: every decrypted release searched; `osos 1.1.3 file+0xd1c8..0xd470` (WRITE_78, final group); WRITE_78 final group at `osos file+0xcfe8..0xd0b0`; WRITE2P_78 at `osos file+0xc668..0xc730` and `osos 1.1.3 file+0xc668..0xc910`; WRITE_78 at `osos 1.1.3 file+0xc998`.

The prologue pattern of RE-1 occurs 9 times in the `osos` image and 18 times in the `aupd` image of each of the releases 1.0.1, 1.0.2, 1.0.3, 1.1, 1.1.2 and 1.1.3; the image `FIRMWARE x1242` contains 18 matches. The nine programs of 1.0.1, 1.0.2, 1.0.3, 1.1 and 1.1.2 have the same lengths as in RE-2 and the same SHA-1 values (at different file offsets). In 1.1.3 (`osos` and both `aupd` sets) and in `FIRMWARE x1242` (both sets) seven programs are equal to those of 1.1.2; WRITE2P_78 has 372 instructions (C4C offset 0xb38) and WRITE_78

has 364 (C4C offset 0xaf8), against 312 (0x958) and 304 (0x918). The final group of instructions in the 1.1.3 WRITE_78 (+0x830..+0xad8; `osos` 1.1.3 file+0xd1c8..0xd470) per set bit of r6 does: `ld r1` from `0xd00`; `0x014` = 1; `FMC_CMD` = 0x78; `wait` event 0; `0x02c` = 2; `st` r1 to `0x00c`; `0x004` = 1; `wait` event 1; `0x048` = 4; `0x004` = 0xca; `ld r0` from `0x04c` mask 0x40; `bnz`; `wait` event 5; `0x048` = 0x800000; `ld r4` from `0x04c` mask 3; `0x004` = 0x20; `bnz r4` to the C4C offset. This is repeated with `add r1,r1,#0x80` (+0x928), with `movi r1,#0` (+0x9b8) and with `movi r1,#0x80` (+0xa48): four groups. In 1.1.2 the corresponding group (`osos` file+0xcfe8..0xd0b0, +0x830..+0x8f8) has one `FMC_CMD` = 0x70 write and one `ld r4` from `0x04c` with mask 1; the same difference exists between WRITE2P_78 of 1.1.2 (`osos` file+0xc668..0xc730) and of 1.1.3 (file+0xc668..0xc910).

14.3.29 RE-29: Chip addressing in the programs

Citation: `osos file+0x9488..0x94f0`.

The sequence `ld r1` from `0xd10`; `ldi r2,[r1]`; `add r1,r1,#4`; `st` r1 to `0xd10`; `shl r2,r2,#2`; `ld r0` from `0xd14`; `add r0,r0,r2`; `ldi r2,[r0]` starts at ERASE +0x58, READ +0x70 (the last `ldi` targets r7), WRITE +0x58, RAWREAD +0x68, WRITE2P and WRITE2P_78 +0x60, WRITE_78 +0x58, WRITECACHE +0x80; in each it is followed by `movi rX,#1`, `shl rX,rX,r2` (READ: `shl r0,r0,r7`) and a shift left by 1 into the mask-0xfffffe01 read-modify-write of `0x000`. The WRITE programs repeat it a second time (WRITE and WRITE_78 +0x5f8, WRITE2P and WRITE2P_78 +0x638, WRITECACHE +0x698). READID reads `0xd10` into r5 at +0x30 and executes `ldi r2,[r5]` at +0xc8; it has no `shl r2,r2,#2` / `add` / second `ldi`; its read of `0xd14` is at +0x98 (RE-24).

14.3.30 RE-30: Absent programs

Citation: bounded prologue scan of `osos` and `aupd`.

The scan is the search of RE-3 for the 16 bytes `0c0c0001 ff000000 100c0001 01000000` over the `osos`, `aupd`, `wtf` and `bootrom` bodies: 9, 18, 0 and 0 matches, all inside the programs of RE-2. Every program found has one of the nine command lists of RE-2; none has a command list other than those.

14.3.31 RE-31: CPU driver: start

Citation: `aupd mem:0x08010c40..0x08010c50`, literal `0x08010c5c`.

The function at `aupd mem:0x08010c40` loads the word at `0x08010c58` (value 0x38a00000) into r0; stores 0xff to `[r0 + 0xc0c]`; loads the word at `0x08010c5c` (value 0x0000fff5) and stores it to `[r0 + 0xc00]`; returns.

14.3.32 RE-32: CPU driver: wait and classify

Citation: `aupd mem:0x08010c60..0x08010d0c`.

The function at `aupd mem:0x08010c60` calls `0x08002cc0` (it returns the word at `0x3c7000b4`) and keeps the result as a start stamp; r5 = 0x38e01000 (word at `0x08010d14`); r6 = 0x38a00000 (word at `0x08010d18`). Loop: it reads `[r5 + 8]` and tests bit 22 (mask 0x400000). When the bit is clear it reads `[r5 + 8]` again and calls `0x08002ce8` with the stamp and 0x7d0; that function returns 1 when `now - stamp >= 0x7d0 * 1000` (`now` = the word at `0x3c7000b4`), 0 otherwise, and 0xffffffff when its second argument exceeds the word at `0x080031b8`. A zero result repeats the loop. A non-zero result prints the string at `0x0801bef0` with `[r6 + 0xc0c]` and the value 0x227, calls `0x08010c08` and returns 0. When bit 22 is set it reads `[r6 + 0xc0c]` and tests bit 2 (mask 4). If bit 2 is set: it prints the string at `0x08010d24` with the value read; prints the string at `0x0801bf50` with `[r6 + 0xc18]`, `[r6 + 0xc20]`, `[r6 + 0xc30]` and the value 0x237; calls `0x08010c08`; sets the result to 0; calls `0x080107f0`; returns 0. If bit 2 is clear and bit 0 (mask 1) is set: result 1, calls `0x080107f0`, returns 1. If bits 2 and 0 are both clear: it prints the string at `0x0801bfb8` with the value read, sets the result to 0, calls `0x080107f0`, returns 0. Apart from the register save at entry and the argument store to the stack at `0x08010cd0`, the function contains no store instruction.

14.3.33 RE-33: CPU driver: reset

Citation: `aupd mem:0x08010c08`; zero bytes at `aupd mem:0x0802e654`.

The function at `aupd mem:0x08010c08` loads r4 = 0x38a00000 (word at `0x08010c38`); stores 0x30000 to `[r4 + 0xc00]`; calls `0x08002d1c` with 1 (a loop that runs until `now - start >= 1 * 1000`, `now` being the word at `0x3c7000b4`); stores the word at `0x08010c3c` (value 0x0802e654) to `[r4 + 0xc04]`; calls the function at `0x08010c40` (RE-31); and returns through a jump to `0x08002d1c` with 1. The 16 bytes at `aupd mem:0x0802e64c..0x0802e65b` are zero; the first program starts at `0x0802e65c`.

14.3.34 RE-34: CPU driver: clear

Citation: `aupd mem:0x080107f0`.

The function at `aupd mem:0x080107f0` stores 5 to `[0x38a00000 + 0xc0c]` (base from the word at `0x0801084c`), calls `0x08002cc0` for a start stamp, and loops: reads `[base + 0xc0c]`, tests mask 5, returns when the result is 0; otherwise adds 1 to a counter and calls `0x08002ccc` with the stamp and 0x64 (returns 1 when `now - stamp >= 0x64`); a non-zero return prints the string at `0x0801be98` with the counter and the value read, then branches to itself without end.

14.3.35 RE-35: CPU driver: timing word

Citation: `aupd mem:0x08010734`; callers `aupd mem:0x08014b30`, `mem:0x08014b44`.

The function at `aupd mem:0x08010734` takes three arguments (r0, r1, r2) and does the following in this order:

1. calls `0x08016900` with 2
2. calls `0x08002370` (it returns r0 = quotient and r1 = remainder of r0 / r1) with r0 = 1000 and r1 = the result of `0x08016900`, keeping the quotient as T
3. computes for each of r0, r1 and r2 `q + (remainder != 0)` with q the quotient of the argument by T (c1, c2, c3)
4. when `c3 + c2 < c1` sets c3 = c1 - c2
5. computes hold = c3 - 1 (1 when that is 0), setup = c2 - 1 (1 when that is 0), edo = hold + 1 limited to 3
6. returns `(setup & 7) << 16 | (hold & 7) << 12 | (edo & 3) << 28`

The function at `0x08014978` calls it at `0x08014b30` with the halfwords at offsets 0x10, 0x12, 0x14 of the record at `r5` and stores the result to `[0x0802e624 + 0x1c]`, and at `0x08014b44` with the halfwords at 0x16, 0x18, 0x1a and stores the result to `[0x0802e624 + 0x20]` (`r5 = 0x0801bb70 + 44 * index`, a table of 18 records).

At `aupd mem:0x08014f00`, `0x08014ff4`, `0x080150ec`, `0x08015274`, `0x08015424`, `0x08015758` and `0x08015ad4` a word loaded from `[struct + 0x1c]` (struct = `0x0802e624`) is stored to window offset `0xd4c` and a word loaded from `[struct + 0x20]` is stored to `0xd48` (the store to `0xd48` is two instructions after the store to `0xd4c`); the instruction after each store to `0xd48` is a call to `0x08010c40`.

The programs load `0xd4c` in ERASE, READ and RAWREAD and `0xd48` in the five WRITE programs, each at +0x38, followed by `or` with 0x801 and a store to `0x000` (RE-9).

14.3.36 RE-36: CPU driver: chip reset

Citation: `aupd mem:0x0802f6b4`, `aupd mem:0x080155f4`.

The function at `aupd mem:0x080155f4`, in this order:

1. stores the word at `0x08015648` (value 0x0802f6b4) to `[0x38a00000 + 0xc04]`
2. stores `[0x0802e624 + 0x14]` to `0xd08`, the halfword at `[0x08036920 + 0x10]` to `0xd0c`, `[0x0802e624 + 0]` to `0xd10`, and 0x8000 to `0xd14`
3. calls `0x080107f0`, `0x08010c40` and `0x08010c60` in this order
4. returns the word at `0x08015658` (0x80010000) when the call to `0x08010c60` returned 0, and 0 otherwise.

`aupd mem:0x0802f6b4` is the start of the READID program (RE-3), whose first `FMC_CMD` write is 0xff (+0x80, RE-24). Stores with immediate offset 8 in `0x08014978..0x08014d4c` are at `0x08014c1c` (base r8 = `0x0802e624`) and `0x08014ce0` (base r1 = `0x08036b38`); stores with immediate offset 0x400 there are at `0x08014990` (base r0 = `0x38a00000`); `0x080155f4..0x08015648` contains no store with offset 8 or 0x400.

14.3.37 RE-37: CPU driver: initialisation

Citation: `aupd mem:0x08014978`.

The function at `aupd mem:0x08014978`, in this order:

1. calls `0x08010c08` (RE-33)
2. stores 0 to `[0x38a00000 + 0x000]` and to `[0x38a00000 + 0x400]` (base from the word at `0x08014d4c`)
3. stores the word at `0x08014d50` (0x0802f8c4) to `[0x0802e624 + 0x24]`
4. when `[0x0802e624 + 0x0]` is 0 it calls `0x080060c8` with 0x20 and stores the result there, and does the same with 0x20 for `[0x0802e624 + 0x14]`
5. stores to `[[0x0802e624 + 0x0] + 4 * i]` the value i, for i = 0 to 7
6. stores 8 as a halfword at `[0x08036920 + 0x10]`
7. calls `0x080155f4` (RE-36)
8. when its result is non-zero branches to `0x08014cc8`
9. stores 0 to the halfword `[0x08036920 + 0x10]`

10. then for r4 = 0..7 loads `[[0x0802e624 + 0x14] + 4 * r4]`, skips it when it is 0 or 0xffffffff, otherwise prints a string with r4, compares it with the first word of each of the 18 records at `0x0801bb70` (stride 44 bytes) and adds 1 to the halfword `[0x08036920 + 0x10]` when it matches.

14.3.38 RE-38: CPU driver: modes

Citation: `aupd mem:0x08010854`, jump table 0x08010890.

The function at `aupd mem:0x08010854` takes a value r4. If r4 > 13 it returns 0; otherwise `add pc,pc,r4,lsl #2` at `0x08010888` selects the branch at `0x08010890 + 4 * r4`. The branch targets are `0x08010988` (return 0) for r4 = 0, 5, 6, 7, 10, 11; and for the others the following code, each storing a halfword to `[0x08036920 + 0x3a]` and words to `[0x0802e624 + 0x24]`, `+0x28`, `+0x2c`, then calling `0x08003f40` and returning r4:

- r4 = 1 (`0x080108c8`): halfword 1; +0x24 = 0x0802f8c4; +0x28 = +0x2c = 0x08010bfc.
- r4 = 2 (`0x080108f4`): 2; +0x24 = 0x080319e4; +0x28 = 0x08010b70; +0x2c = 0x08010a70.
- r4 = 3 (`0x0801090c`): 2; +0x24 = 0x08031024; +0x28 = 0x08010ba0; +0x2c = 0x08010aa0.
- r4 = 4 (`0x0801091c`): 4; +0x24 = 0x08031024; +0x28 = 0x08010bc4; +0x2c = 0x08010ac4.
- r4 = 8 (`0x080108e0`): 2; +0x24 = 0x08030664; +0x28 = 0x08010ba0; +0x2c = 0x08010aa0.
- r4 = 9 (`0x0801093c`): 2; +0x24 = 0x0802f8c4; +0x28 = 0x08010b70; +0x2c = 0x08010a70.
- r4 = 12 (`0x08010950`): 2; +0x24 = 0x08032364; +0x28 = 0x08010afc; +0x2c = 0x080109fc.
- r4 = 13 (`0x08010964`): 2; +0x24 = 0x08032364; +0x28 = 0x08010b2c; +0x2c = 0x08010a2c.

The values 0x0802f8c4, 0x080319e4, 0x08031024, 0x08030664 and 0x08032364 are the start addresses of WRITE, WRITE_78, WRITE2P_78, WRITE2P and WRITECACHE in the first `aupd` set (RE-3).

14.3.39 RE-39: CPU driver: result classification

Citation: `aupd mem:0x0802ea5c` offsets 0x06a0 and 0x0a68; evaluator `aupd mem:0x0800ff00`.

Program side: READ+0x6a0 and +0xa68 are described in RE-18; the instruction sequences at the six `ld` of `0x810` in READ are described in RE-13.

Function side: the function at `aupd mem:0x0800ff00` takes r0 (a word), r1 (a pointer or 0) and r2. When r1 is non-zero it stores a zero byte to `[r1]`.

It then proceeds in this order:

1. if bit 30 (mask 0x40000000) of r0 is set it adds 1 to the 64-bit counter at `[0x08036a38 + 0x80]` and returns the word at `0x080100d0` (value 0x80020000)
2. if r2 equals 1 and bit 29 (mask 0x20000000) of r0 is set and the low byte `r0 & 0xff` is 0, it adds 1 to the 64-bit counter at `[0x08036a38 + 0x78]` and returns 1
3. otherwise, if the low byte is 0, it adds 1 to the counter at `[0x08036a38 + 0xb0]` and returns 0
4. otherwise it tests the bits of the low byte from bit 7 down to bit 0: for the highest set bit it adds 1 to the counter at `[0x08036a38 + 0xf0]` (bit 7), 0xe8 (bit 6), 0xe0 (bit 5), 0xd8 (bit 4), 0xd0 (bit 3), 0xc8 (bit 2), 0xc0 (bit 1), 0xb8 (bit 0), stores 8, 7, 6, 5, 4, 3, 2 or 1 respectively to `[r1]` when r1 is non-zero, and returns 0.

It is called from `0x080152bc` (r0 = `[r10 + 0xc30]`, r2 = 1 when r4 equals 1 and 0 otherwise, r1 = `[sp + 0x3c]`) and branched to from `0x08015450` when the result of `0x08010c60` is non-zero (r0 = `[r4 + 0xc30]`, r1 = `[sp + 0x38]`, r2 = 0).

Four places load `[rN + 0xc30]` and test bit 28 (mask 0x10000000): at `0x08014f30` (r9) and `0x08015024` (r4) the result is the word 0x80030000 (words at `0x08014f5c`, `0x08015050`) when the bit is set and 0 when it is clear; at `0x0801578c` (r10) and `0x08015b0c` (r4) the result is 0 when it is clear and, when it is set, a string is printed (with the values 0x526 and 0x4dd, words at `0x080157cc`, `0x08015b50`) and the result is 0x80030000 (words at `0x08015808`, `0x08015b8c`).

14.3.40 RE-40: CPU driver: other sequencer registers

Citation: functions of the original CPU driver.

A scan of `aupd mem:0x08000000..0x0802e653` and `0x08032d70..0x080e6edf` for ARM word load or store instructions (condition always, immediate offset added) with an immediate offset of 0xc08, 0xc10, 0xc14, 0xc1c, 0xc24, 0xc28, 0xc2c, 0xc34, 0xc38, 0xc3c, 0xc4c, 0xc50, 0xc5c, 0xc60, 0xc64, 0xc6c, 0x814 or 0x840 finds no match. The same scan finds, for offset 0xc00, stores at `0x08010c14` and `0x08010c50`; for 0xc04, stores at `0x08010c24`, `0x08014e7c`, `0x08014f98`, `0x08015094`, `0x080151bc`, `0x0801531c`, `0x08015600`, `0x080156a0` and `0x08015890`; for 0xc0c, stores at `0x08010800` and `0x08010c48` and loads at `0x08010828`, `0x0801083c`, `0x08010c94` and `0x08010cb8`; for 0xc18 a load at `0x08010cdc`; for 0xc20 a load at `0x08010cd8`; for

0xc30 loads at 0x08010cd4, 0x08014f30, 0x08015024, 0x080152ac, 0x0801578c and 0x08015b0c (and the conditional ldrne at 0x0801543c). The scan does not cover accesses with a register offset or a computed address.

14.3.41 RE-41: Second microcode copy

Citation: aupd mem:0x080e6ef0..0x080eb3f8+0x9f8, tag at aupd mem:0x08032d6c; aupd mem:0x08000020.

The second set of nine programs starts at aupd mem:0x080e6ef0 (RE-3) and its bytes are equal to those of the first set and of the osos programs. In the aupd body the 32-bit words equal to the start address of a program of the first set are at: ERASE 0x08014f48, 0x08015038; READ 0x080152c8, 0x0801546c; READID 0x08015648; WRITE 0x080109a8, 0x08014d50, 0x080157b4, 0x08015b40; RAWREAD 0x08015124; WRITE2P 0x080109b8; WRITE2P_78 0x08010994; WRITE_78 0x080109bc; WRITECACHE 0x08010990. No 32-bit word of the aupd body equals the start address of a program of the second set, or has a value in 0x080e6ee0..0x080ebfff. The words at aupd mem:0x08032d64 and 0x08032d68 are 0x000000d8 and 0x00000002; the 4 bytes at 0x08032d6c are 64 6c 79 50 (ASCII dlyP), at 0x08032d7c 70 55 77 46 (pUwF) and at 0x08032d84 68 73 6c 66 (hslf); the 16 bytes before 0x080e6ef0 are zero. The word at aupd mem:0x08000020 is 0xeb000d11 (an ARM bl); no program bytes are there.

14.3.42 RE-42: Driver generation

Citation: strings from aupd mem:0x0801c000.

A string at aupd mem:0x0801c000 ends with Whimory2_1\FIL\M1A\M1Aflash.c. Strings that follow it begin [FIL:ERR] m1fmssResetAndVerifyIds, [FIL:ERR] m1fmssWriteScatteredPages, [FIL:ERR] m1fmssReadScatteredPages, [FIL:ERR] m1fmssEraseSequentialBlocks, [FIL:ERR] m1fmssEraseSingleBlock, [FIL:ERR] m1fmssWriteSequentialPages and [FIL:ERR] m1fmssReadSequentialPages. The string at aupd mem:0x08010d24 reads [FIL:ERR] FMSS sequence timed out (FC_CS_IRQ := 0x%x)!; the string at 0x0801bef0 begins [FIL:ERR] _waitForCSInterrupt() timed out after 2 seconds.

14.3.43 RE-43: Boot ROM NAND access

Citation: bootrom mem:0x20009910 (page reader), bootrom mem:0x20002e0c (channel < 2), bootrom mem:0x20001830 (pin set-up).

Page reader, bootrom mem:0x20009910 with arguments (c, ce, row, dest, out); b = 0x38a00000 + c * 0x400 (word at 0x20009cdc); wait(a, m) below is a call of 0x20000e50(a, m, 0x186a0), a loop that returns when all bits of m are set in [a] or after 0x186a0 iterations.

Sequence, in order:

1. [b + 4] = 0x0001f0e0
2. [b + 0] = 0x43801 | 1 << (ce + 1)
3. [b + 8] = 0
4. wait(b + 0x48, 2)
5. [b + 0x48] = 2
6. [b + 0x2c] = 4
7. [b + 0xc] = row << 16
8. [b + 0x10] = row >> 16
9. [b + 4] = 1
10. wait(b + 0x48, 4)
11. [b + 0x48] = 4
12. [b + 8] = 0x30
13. wait(b + 0x48, 2)
14. [b + 0x48] = 2
15. call 0x20009614(c, ce)
16. [b + 8] = 0
17. wait(b + 0x48, 2)
18. [b + 0x48] = 2
19. [0x38a00840] = 0x7f

Then, for i = 0..3, in order:

1. [b + 0x48] = 0x80000000
2. [b + 0x30] = 0xf

3. `[b + 0x14] = 1 << ((i & 1) + 4)`
4. `[b + 0x24]` = 0 (i < 2) or 0x10
5. `[b + 4]` = 0x32
6. `wait(b + 0x48, 8)`
7. `[b + 0x48]` = 8
8. `[b + 0x30]` = 0x1ff
9. `[b + 0x14] = 1 << (i & 1)`
10. `[b + 0x24]` = 0
11. `[b + 4]` = 0x22
12. `wait(b + 0x48, 8)`
13. `[b + 0x48]` = 8
14. the word `[b + 0x48]` is read and, when its bit 27 (0x8000000) is set, `0x2000ac1c(i & 1, (i & 1) + 4, (i & 3) >> 1)` is called and `[0x38a00810]`, that bit and `[0x38a00840]` are ORed into three accumulators
15. `[b + 0x34] = dest + i * 0x200`
16. `[b + 0x38]` = 7
17. `[b + 0] = ([b + 0] & 0xfffffbff) | 0x1000001`
18. `[b + 0x14] = 1 << ((i & 1) + 8)`
19. `[b + 4]` = 0x1a0
20. `wait(b + 0x48, 0x100000)`
21. `[b + 0x48]` = 0x100000

After the loop, in order:

1. `[b + 0x78]` = 0x5140
2. `[b + 0x7c]` = 2
3. the loop runs while `[b + 0x7c] & 2`
4. `out[0..2] = [b + 0x60], [b + 0x64], [b + 0x68]`
5. `[b + 0]` = 0x43802
6. the result is 0 when no iteration saw bit 27, else 3 when bit 0 of the ORed `0x810` words is set, else 6 when bit 5 of the ORed `0x840` words is set, else 1
7. `[b + 0x48]` = 0xffffffff
8. `[0x38a00840]` = 0x7f

Channel routine, `bootrom mem:0x20002e0c(ch)`: does nothing when ch >= 2; otherwise calls `0x2000147c(8)`, `0x2000147c(0xc)`, `0x20001830(ch)`, `0x200096e4(ch, 0)`, `0x20009910(ch, 0, 0, 0x22020000, buf)`, and `0x20009910(ch, 0, k + 1, k * 0x800 + 0x22000000, buf)` for k from 0 below min(ceil(w / 0x800), 0x3f), where w is the word at offset 0xc (or 0x10) of the buffer at `0x22020000`.

Pin routine, `bootrom mem:0x20001830(ch)`, base 0x3cf00000 (word at `0x200019ec`): ch = 0 stores `([base + 0x140] & 0xffff0000) | 0x2222` to `[base + 0x140]`, `([base + 0x120] & 0xfff0fff0) | 0x20002` to `[base + 0x120]` and 0x22222222 to `[base + 0x100]`; ch = 1 stores `([base + 0x180] & 0xffff0000) | 0x2222` to `[base + 0x180]`, `([base + 0xe0] & 0xfff0ffff) | 0x20000` to `[base + 0xe0]`, `([base + 0x140] & 0xffff) | 0x22220000` to `[base + 0x140]` and `([base + 0x160] & 0xfff00000) | 0x22222` to `[base + 0x160]`.

14.3.44 RE-44: No cache maintenance in the CPU-side driver

Citation: `aupd`, the CPU-side NAND driver functions (function table at `aupd mem:0x08036b38`).

The function at `aupd mem:0x08014978` stores these twelve function addresses to the table at `0x08036b38` (base from the word at `0x08014de8`), at the offsets given, with stores between `0x08014cd0` and `0x08014d30`: 0x00 `0x08015560`, 0x04 `0x08015484`, 0x08 `0x08015054`, 0x0c `0x080152e0`, 0x10 `0x08015178`, 0x14 `0x080154f0`, 0x18 `0x0801565c`, 0x1c `0x0801580c`, 0x20 `0x08015b90`, 0x24 `0x08014f60`, 0x28 `0x08014e40`, 0x2c `0x08015bf0`. The ARM words in `aupd mem:0x08000000..0x0802e653` that are coprocessor-15 register transfers (MCR or MRC, coprocessor number 15, condition always) number 176; they lie in `0x080033ec..0x080036f0` and at `0x080213a8` and above; none lies in `0x08004000..0x080213a7`, which contains the twelve functions above, `0x08010734`, `0x08010854`, `0x08010c08`, `0x08010c40`, `0x08010c60` and `0x080107f0`. The transfers with CRn = 7 are at `0x080034b0`, `0x080034b8`, `0x080034c4`, `0x080034cc`, `0x08021f3c` and `0x080228fc`. No branch word (B or BL, any condition) in `0x08010000..0x08015fff` has a target in `0x080033ec..0x080036f0` or `0x080213a8..0x080229ff`.

14.3.45 RE-45: Chip table row of this unit's chip

Citation: `aupd mem:0x0801bb70` (`file+0x1c370`), `aupd mem:0x0801bb9c` (`file+0x1c39c`).

Two consecutive 0x2c-byte rows begin with the bytes `ec d5 14 b6`. The halfwords at row offsets +4, +6, +8, +0xa, +0xc and +0xe are 2, 4096, 128, 8, 128, 1 in the first row and 4, 4096, 128, 8, 128, 1 in the second; the halfword at +0x20 is 3872 in both; the word at +0x24 is 8 in the first row and 1 in the second.

14.3.46 RE-46: Bank count in WRITECACHE

Citation: `osos file+0xd138` (WRITECACHE) +0x58..+0x78 and +0x630..+0x690.

- +0x58 `movi r5,#0`; +0x60 `ld r1` from `0xd04` mask `0xffffffff`; +0x68 `movi r0,#0`; +0x70 `sub` with `R` = 0, `arg` = 1, `imm` = 0 (register form: r0 = r0 - r1); +0x78 `st` r0 to `0xd44`.
- +0x630 `bnz r5` to +0x698; +0x638 `ld r1` from `0xd18`; +0x640 `ld r0` from `0xd44`; +0x648 `and` with `R` = 0, `arg` = 1, `imm` = 0 (register form: r0 = r0 AND r1); +0x650 `bnz r0` to +0x678.
- +0x658 `FMC_CMD` (`0x008`) = `0x10`; +0x660 `wait` event 0; +0x668 `0x048` = 2; +0x670 `bz r0` to +0x690.
- +0x678 `FMC_CMD` = `0x15`; +0x680 `wait` event 0; +0x688 `0x048` = 2; +0x690 `movi r5,#0`.
- No other instruction of the nine programs reads or writes `0xd04` or `0xd44` (all opcode `01`, `02` and `04` instructions with these offsets searched).

14.3.47 RE-47: Boot ROM boot-device selection

Citation: `bootrom mem:0x20003230`, `mem:0x200019d4`, `mem:0x200018fc`.

- `0x200019d4(p, n)` returns bit n of the word at `0x3cf00004 + 0x20 * p` (the word address is the constant at `bootrom mem:0x200019f8`, `0x3cf00004`).
- The function at `0x20003230` calls `0x200018fc(1, 0, 0)` and `0x200018fc(1, 1, 0)`, then a = `0x200019d4(1, 0)` and b = `0x200019d4(1, 1)`.
- If bit 0 of the word at `0x3c400028` is 0, or a != 1 and b != 0, it calls `0x200018fc(3, 6, 0)` and `0x200018fc(3, 7, 0)`, then c = `0x200019d4(3, 6)` and d = `0x200019d4(3, 7)`, and calls: `0x20002fa8(0)` for c = 0, d = 0; `0x20002fa8(2)` for c = 0, d = 1; `0x20002e0c(0)` for c = 1, d = 0; `0x20002e0c(1)` for c = 1, d = 1.
- Otherwise, and after any of these calls returns, execution continues at `0x200033b8`.
- `0x20002e0c` is the NAND boot routine of RE-43 (channel 0 or 1); `0x20002fa8(x)` reads from an SPI controller x (calls `0x20008f90(x, ...)` and `0x2000906c(x, 0x800, ...)`).

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 13, chapter 15, chapter 18.

Chapter 15 — Evidence interpretation

Summary: What the outcomes of chapter 14 are taken to show, by topic. Every statement cites its evidence and carries an evidence class (device, observed, inferred or open).

Every statement in this chapter cites evidence entries (`D-<experiment>-<device run>` for device outcomes, device runs as listed in chapter 13, section 13.2; `RE-<n>` for reverse-engineering outcomes) and carries one of the evidence classes of chapter 1, section 1.1:

- **Device**: a fact measured on a unit.
- **Observed**: seen in the original programs or in the original CPU driver.
- **Inferred**: an interpretation of observed use or of measurements.
- **Open**: not settled.

The experiments themselves are described in chapter 13; the finding numbers refer to chapter 18.

15.1 Instruction set and registers

- **Device.** The register file is r0..r7 at `0xc18 + 4n`, with `SEQ_RESULT` (`0xc30`) being r6; a CPU write before a start sets the register's starting value; a start keeps the register contents; and a sequencer reset clears the registers (finding 38). Evidence: `regmap`, device run 1, `regmap`, device run 6, `regmap`, device run 7, `control`, device run 1, `control`, device run 2, `control`, device run 3.
- **Device.** A two-instruction loop takes about 216 ns per instruction (873 us for 2 000 iterations and 8 657 us for 20 000 iterations). Evidence: `clock`.
- **Device.** Instruction time is set by the fetch of the 8-byte instruction: 217 ns from SDRAM and 138 ns from internal SRAM; a register-window access adds 35..37 ns; `ldi` adds 170 ns (SDRAM) or 218 ns (SRAM); and `sti` adds 186 ns (SDRAM) or 120 ns (SRAM). Evidence: `opcost`, `optime`.
- **Device.** ALU edge cases: `shr` is a logical shift; a shift count of 32 or more gives 0 (whether the count is an immediate or a register); `add` and `sub` wrap at 32 bits; an immediate of 0 selects the two-operand (register) form; and `ld` applies its mask by AND. Evidence: `alu`, device run 1, `alu`, device run 2, `alu`, device run 3, `holdout`, device run 9, `holdout`, device run 10.
- **Inferred.** The ALU operand rule (when the immediate is non-zero, the source register is in the `arg` field; when the immediate is zero, the instruction uses the register form) is the rule that holds in every use in the original programs; an immediate of 0 therefore has no encoding (finding 2). Evidence: RE-4. The device test confirms the behaviour of both forms (`alu`, device run 1, `alu`, device run 2, `alu`, device run 3).
- **Device.** `ldi`/`sti` are little-endian 32-bit accesses and an unaligned address is rounded down; `pld` and `pst` wrap the offset at 12 bits and reach the registers r0..r7. Evidence: `memory`, `indirect`, device run 1, `indirect`, device run 2, `indirect`, device run 3.
- **Inferred.** Opcodes `18` and `19` load and store the register window at a register-held offset; the original programs use them only on `0xd60..0xd7c` (finding 3). Evidence: RE-6; the device confirms the reach to the whole window (`indirect`, device run 1, `indirect`, device run 2, `indirect`, device run 3, `fmcmap`).
- **Device.** Window offsets are 12 bits wide: `wr`, `st` and `ld` ignore bits 15:12 of `arg`. Branch targets and writes to r8 use bits 24:12 of the target and ignore bit 31. `jr` does not jump when the register value has bit 12, 16, 24 or 31 set. Evidence: `fwidth`.
- **Device.** Register numbers of 8 and above behave as follows. As a source in the argument field, numbers 9, 15 and 255 read r0. As a source in the register field and as the address register of `ldi`, numbers 9 and 14 read 0. As a destination, numbers 9, 11..16, 24 and 255 change nothing. Number 8 is the PC, and a write to number 10 goes to r0..r7. Numbers 24 and 26 do nothing, so the whole register field is decoded. Evidence: `regnum`, `r8`, device run 16, `r8`, device run 17. **Open:** the behaviour of register 8 as a `st` source, as an `ldi` address register and in `bnz` was not tested or is inferred (finding 69).
- **Device.** `bz` and `bnz` ignore `arg`, test all 32 bits of the register and take unaligned targets as given. `wait` ignores `arg` and `imm`, and a `wait` on an event number above 15 never passes. Evidence: `branch`, `waitf`.
- **Device.** The fields the original programs leave 0 (`imm` of `st`, `ldi`, `mov`, `sti`, `pld`, `pst`, R of `wr`, `arg` of `movi`) are ignored; `end` faults when `R` or `arg` is non-zero and ignores `imm`. Evidence: `fields0`.
- **Device.** The program can write the PC (`0xc38`, which is r8); such a write is a jump. An `ld` of `0xc38` reads the offset of that instruction. The CPU can write the PC only while the program is stopped at a pause (opcode `12`). Evidence: `pcwrite`, `r8`, device run 16, `r8`, device run 17, `op12cont`.
- **Device.** Register numbers 8..255 in every role (chapter 4, section 4.3). **Device.** `st` with a register number from 8 to 255 stores the last value that passed through the register file: the value the previous instruction wrote into a register, or the register value it read (r5 after `st [..], r5`, r0 after `nop` and `wr`, whose register field is 0); `wait` leaves it unchanged. **Inferred:** a number above 7 in the register field selects no register,

and the register file's output keeps the last value it carried. `bnz` and `bz` with these numbers never branched, so they do not test that value. Evidence: `regnum`, `regsweep`, `stprev`.

15.2 Opcodes

- **Device.** Of all 256 opcode values only the 19 used by the original programs and `08`, `09`, `0f`, `10`, `12`, `15`, `16`, `ff` do not fault. The opcode byte is decoded in all 8 bits: the values `20`, `40` and `80`, each being `00` with one high bit set, fault like the other illegal values instead of acting as `end`. The other values stall for about 4.87 ms and then fault, with no other effect. Evidence: `opsweep`, `unknown`, device run 1, `unknown`, device run 2, `unknown`, device run 3.
- **Device.** `08` and `ff` have no visible effect in any operand form, cost the time of `movi` and do not react to pending flags. Evidence: `optime`, `opstate`. **Open:** why there are two nop instructions (`08` nop, `ff` nop2) that behave the same way.
- **Device.** `0f` jumps to the byte offset held in the register named by `arg`, unrounded, with no return address kept. Evidence: `op0f`.
- **Device.** `10` sets `CS_IRQ` bit 1 and continues, whatever flags are pending; it did not raise the CPU interrupt line. Evidence: `op10`, `opstate`, `irqline`, device run 15, `irqline`, device run 16, `irqline`, device run 17.
- **Device.** `12` stops the program, with no timeout, in every operand form and state. While it is stopped, `CS_IRQ` bit 3 is set, `C08` reads `0xa`, `FMSS_CTRL` bit 18 is set and the instruction is held in `0xc40` and `0xc44`. Of eighteen kinds of CPU write tried, only one released it: writing the PC (normally to the offset of the pause instruction + 8) and then a start word, which continues the program. Evidence: `op12`, `op12wait`, `op12ack`, `op12rel`, `op12cont`, `opstate`. A start word without a PC write continues at the pause itself (`op12ack`, `op12cont`). **Reading (inferred):** `12` is a pause (breakpoint) instruction for debugging programs: it stops the program where it is, raises `CS_IRQ` bit 3, ignores its fields, and the CPU continues it by writing the PC (`0xc38`) and a start word. The behaviour is a device result; "for debugging" is the reading of that behaviour.
- **Device.** `15` waits until the timer `C14`, counting down from a non-zero value, reaches 0 (about 4.85 ms); if `C14` is already 0, the instruction loads `0xff` into it at once. Evidence: `op15`, `c14`.
- **Device.** `16` waits for any event flag of either register set or of the ECC engine and then continues at the fault vector `C4C`, with no fault raised; with nothing pending it times out and faults. It waits for a flag that arrives during the wait; `C58` makes no difference. Evidence: `op16`, `op16b`, `opstate`.
- **Device.** `09` (`wait2`) continues at once while the interrupt line it names is raised and otherwise times out like an unmet `wait`; `arg` and `imm` are ignored. Evidence: `op09`, `op09b`, `op09c`, `op09d`, `w2vic`.

15.3 Events, waits and the timeout

- **Inferred.** `wait` events 0..5 pair with these flags: `wait` event 0 with `FMC_STATUS` bit 1, event 1 with bit 2, event 2 with bit 20, event 3 with bit 3, event 4 with `ECC_DONE` bit 0, and event 5 with `FMC_STATUS` bit 23 (finding 4); the pairing is taken from the flag that each wait in the original programs acknowledges. Evidence: RE-7, RE-12; the page-read experiment confirms the pairing of `wait` events 1, 0, 3, 4 and 2 with the address, command, transfer, ECC and DMA steps respectively (`readev`, device run 15, `readev`, device run 16, `readev`, device run 17, `readev`, device run 21).
- **Device.** With nothing pending, a `wait` on events 0..10 and 12..15 times out after 4 865..4 882 us, while `wait` event 11 passes at once. Event 11 also passes during a block erase and with any chip enable selected, so it is not the chips' ready line. Evidence: `wait`, device run 1, `wait`, device run 4, `ev11`, `evsweep`, device run 11, `evsweep`, device run 14.
- **Device.** Events are levels and are not consumed; a sequencer read of `FMC_STATUS` or `FMC_NSTAT` does not consume an event. Evidence: `evcons`, `evclear`.
- **Device.** `wait` event 4 passes when any `ECC_DONE` bit except bit 4 is set. `wait` events 6, 7, 8 and 9 are the command, address, DMA and transfer flags of the second register set. `wait` event 15 passes while an enabled NAND controller (FMC) or ECC interrupt source is pending (inferred from its passing only when `0x044` was non-zero). Evidence: `evcons`, `ev8`, `dmaloop`, device run 20, `dmaloop`, device run 21, `evsweep`, device run 11, `evsweep`, device run 14, `op09d`. **Device.** In a clean state with the interrupt enables clear, `wait` event 15 never passed (`state`, device run 17, `state`, device run 18).
- **Device.** A `wait` times out after about 4.87 ms with start word `0xfff5`, whatever `C58` holds; `C58` bit 2 decides whether the program then continues at the fault vector or at the next instruction; and start-word bit 2 enables the timeout. Evidence: `wait`, device run 1, `wait`, device run 4, `c58`, device run 2, `c58`, device run 3, `c58ill`, `ctrlfx`, `ctrlbits`.

- **Device.** The timeout length is (start-word bits 15:8 + 1) x 2^s bus cycles. Here s = 3, 5, 7, 9 and 10 for start-word bits 7:4 = 1..5, and s = 11 for bits 7:4 of 6 and above; bits 7:4 = 0 give an immediate timeout. The bus clock of about 108 MHz is inferred. Evidence: `tmofield`, `tmobits`, `tmobits2`.
- **Device.** `C58` bit 0 at an END sets `C5C` and makes every later END run the fault vector once; `C58` bit 1 diverts one END when the CPU first wrote `C58 = 2` and afterwards wrote `C4C`; `C10`, `C3C`, `C48` and `C54` do not change the timeout. Evidence: `resume`, `pending`, device run 5, `pending`, device run 6, `oneshot`, `timeregs`. **Open:** the mechanism of `C58` bit 1 (inferred: armed by the CPU write of `C4C`).
- **Device.** `C14` is the timeout timer. A `wait` that starts waiting loads it with `0xff`, and it then counts down. It reads `0xff` after a timeout, is frozen at the end of the program, and ignores writes. Evidence: `c14`, `op15`, `c08`.
- **Device.** `wait` event 5 passes about 155 us into a block erase when a status poll is taken during it (the status byte changes from `0x81` to `0xc0`), and passes at once on an idle chip. Evidence: `ev5`.
- **Device.** `wait` event 5 is `FMC_STATUS` bit 23 as a level; a status latch sets it only while `wait` event 5 is armed, and uses the arm (the arming rule is in the next statement); after a latch a new command is refused until `FMCTRL1 = 0x20`; a latch after `90h` sets no bit 23. Evidence: `ev5rule`.
- **Device.** A status latch sets bit 23 only while `wait` event 5 is armed and uses the arm; `FMCTRL1` operations (the set-up word, `0x100e0`, a transfer from the chip, a DMA) and address cycles re-arm it, and so does the polled chip becoming ready; `FMCTRL1 = 0x20`, the status-clear word, a CE deselect and reselect, and an ECC operation do not. Evidence: `ev5ready`, `ev5arm`, `ev5two`.
- **Device.** With `FMC_DATA_SEL = 0x100` a status latch returns the last command byte (`0x70`), not the status. Evidence: `writeseq`, `pageorder`.
- **Open.** A `wait` event 5 straight after "`70h`, latch" timed out although `FMC_STATUS` bit 23 read set afterwards. The same latch passed after `70h`, `wait` event 0 and clearing `FMC_STATUS` bit 1. One status-latch `wait` event 5 timed out in one of two identical runs. One `wait` event 3 with CE0 selected passed in one program and timed out in three programs run after it. Evidence: `evsweep`, device run 11, `evsweep`, device run 14, `evclear`, `ev5t`, device run 16, `ev5t`, device run 17, `op16`.
- **Open.** `FMC_STATUS` bit 24 appeared instead of bit 23 in both status-latch programs of `irqsrc` that wrote an interrupt enable and in one program of `op16` with none, and in none of the fifteen programs of `ev5en`; the enables are not shown to cause it. Evidence: `irqsrc`, `op16`, `ev5en`. Findings 104, 110.
- **Device.** `C14` reloads with start-word bits 15:8, so the timeout length field is the counter's start value. Evidence: `startbits`.
- **Device.** `wait2` condition 33 passed while the chip was idle in `op09w33` and during an ECC operation and a DMA in `finale`; it did not pass in `op09nand` (idle and during an erase), `evsweep2`, `ctrlwords`, `w2state`, `blk100b` or `w2vic`.
- **Device.** `wait2` condition N is interrupt source N of the CPU's interrupt controllers, read from their raw status: VIC0 line N for N = 0..31, VIC1 line N - 32 for N = 32..63; 64 of 64 conditions matched the raw lines in four states. Condition 20 is the NAND controller's line, condition 43 the ECC engine's line; conditions 5 and 33 are lines of other devices. **Inferred:** `wait2` lets a program wait for any interrupt source of the SoC, not only for the FMSS engine's own flags. Evidence: `w2vic`, `w2state`.

15.4 Control registers

- **Device.** `CS_IRQ` bit 0 is END, bit 1 is set by opcode `10`, bit 2 is a fault, and bit 3 is set while opcode `12` blocks. Bits 6:4 follow the NAND controller and ECC interrupt sources once these are enabled in `0x044` and `0x844`. Bit 6 reflects the pending `ECC_DONE` bits. Evidence: `op10`, `op12`, `irqen`, device run 16, `irqen`, device run 17, `irqsrc`.
- **Device.** `C08` is a state code: 1 means running, 4 waiting or stalled on an illegal opcode, 3 paused by start-word bit 1, `0xa` blocked on `12`, and 0 idle or held; 5 is the value seen when the program itself reads `C08`. Evidence: `c08`, `ctrlhold`. **Open:** the full code set.
- **Device.** `C50` bits 5:0 each follow the pending flag of events 0..5; bit 0 stayed clear inside the handlers of a timeout and of an illegal opcode, with and without an acknowledge of the pending flag. Evidence: `c50`, `c50x`, `op16`, `irqsrc`. **Inferred:** the original handlers' test of bit 0 reads as "a command flag is still pending". **Device.** No fault tried set `C50` bit 0 (`holdout`, device run 9, `holdout`, device run 10). **Open:** which fault, if any, sets it, and bit 8 after a soft reset.
- **Device.** Start-word bit 0 starts the sequencer. Bit 1 pauses it before the first instruction, or exactly where the program is during a run, until a start word without bit 1 is written. Bits 3 and 16 hold the sequencer in the post-reset state or abort a running program, which restarts from instruction 0 on release. Bit 2 enables the timeout, and bits 15:4 set the timeout. Bit 17 does not clear registers. There is no single-step. Evidence: `ctrlbits`, `ctrlhold`, `pause`, `ctrlfx`, `tmofield`.

- **Open.** Start-word bit 6 (compared between the start words `0xfff5` and `0xffb5`): no effect on the timeout length, the CPU interrupt line, loops, waits, illegal opcodes, `evjmp`, program fetch, `ldi` / `sti` / `pld`, DMA, ECC or reserved-word access was found, and the reading "bit 6 = interrupt enable" is not confirmed. Accesses to different parts of the NAND under the two start words were not compared. Evidence: `irqline`, device run 15, `ctrlfx`, `ctrlbits`, `startbits`, `startdiff`. Findings 54, 153.
- **Device.** After `FMSS_CTRL = 0x30000` the first start fetches instruction 0 into `0xc40`/`0xc44` and executes it only if it is `end`; any other instruction is left waiting, with `FMSS_CTRL` bit 18 set, until a second start. Evidence: `startseq`, `startseq2`, `startseq3`.
- **Device.** The CPU write masks, in writable bits, are 32 for `0xc04`, r0..r7, `0xc3c`, `0xc48` and `0xc4c`; 8 for `0xc10`; 12 for `0xc54`; 8 for `0xc58`; and 1 for `0xc5c`. The words `0xc08`, `0xc14`, `0xc38`, `0xc40`, `0xc44` and `0xc50` are read-only. `C48` and `C54` are cleared at a start and kept after the end; `C3C` reads 0 after a run and held the vector address in a handler (inferred: the address of the last vector taken). Evidence: `rwmask`, `access`, `timeregs`.
- **Device.** `C3C` inside a handler is the offset of the faulting instruction + 8. **Inferred:** it is the return address of the fault, the instruction after the one that faulted. Evidence: `ctrlwords`, `c50fault`.
- **Device.** `C50` bit 0 follows the command flag; a fault neither sets nor clears it. Evidence: `c50fault`.
- **Device.** Start-word bit 6 and bits 17..31 changed nothing in any program tried. Evidence: `startbits`, `startdiff`.
- **Inferred.** In `c58b1` the second run of each case started after the park of the first run, which writes 0 to `C58`, `C4C` and `C5C` (chapter 13, section 13.1.4). The second runs therefore had `C58` = 0 and show nothing about whether the one-shot is used up; only the first runs test `C58` bit 1. Evidence: `c58b1`.

15.5 Memory maps

- **Device.** Offset bit 9 is not decoded below `0xc00`: `0x200..0x3fc` repeats `0x000..0x1fc`, `0x600` repeats `0x400` and `0xa00` repeats `0x800`. The range `0x400..0x4fc` is a second, independent register set. The range `0x100..0x1fc` is a separate block, repeated at `0x300`, `0x500` and `0x700`. The words `0xc60..0xcfc` and `0xd80..0xffc` read 0 and drop writes. Parameter RAM is `0xd00..0xd7c`. Evidence: `fmcmap`, `rwmask`, `access`, `pram`, device run 2, `pram`, device run 3.
- **Device.** The read and write behaviour of every word of the window was measured from both the CPU and the sequencer (the read/write matrix), with the exceptions listed in chapter 3. For the words `0x3a8..0x3b4`, the sequencer read back the values that the CPU had written, while the CPU itself read 0. Evidence: `access`. **Open:** the `0x3a8..0x3b4` difference.
- **Device.** The `0x100` block has writable words at `0x1b8..0x1d8`; setting `0x1cc`/`0x1d0` to 0 or `0xffffff` did not change the time of a 2112-byte page transfer, so they are not NAND bus timing. Evidence: `blk100`, `finale`. **Open:** the meaning of the block.
- **Device.** `FMC_28` rose by 124 from one probe run to the next whatever the program did, and advanced by exactly 128 over 128 iterations of a `pld`/`sti` loop. In programs that write the set-up word first, it stayed 0 through instructions, `sti`, `ldi`, DMAs, ECC encodes and chip reads. Evidence: `fmc28`, `finale`. **Open:** what it counts.
- **Device.** Power-on values (as the CPU reads them; only non-zero values are listed): `0x020 = 0x7f167f`, `0x038 = 7`, `0x048 = 0xc3000`, `0x04c = 0xff`, the same at `0x420..0x44c`, `0x800 = 0x20060808`, `0x814 = 0xe00000`, `0x840 = 0x10`, `0xc50 = 0xffff1800`. After a soft reset of the writable words, seven words and the timer `C14` still differ from the power-on state, but every experiment that was repeated both after a power cycle and after a soft reset behaved as at power-on. Evidence: `state`, device run 17, `state`, device run 18, `restore`, device run 16, `restore`, device run 19, `restore`, device run 21. Findings 78, 80. **Open:** the remaining differences.
- **Inferred.** The original CPU driver does no cache maintenance around a run, while the sequencer reads its program and lists from system memory and writes results there; so those buffers are uncached or the data cache is off when the original firmware runs the engine (the sequencer's accesses are not coherent with the data cache). Evidence: RE-44.

15.6 NAND controller

- **Device.** `FMC_CTRL0` bit 24 enables DMA: with it clear no DMA runs in either direction and either register set; with it set, a loopback from RAM into the data buffer and back to RAM returns 512 bytes intact. A DMA always moves 512 bytes whatever `FMC_DNUM` says. Evidence: `dmaloop`, device run 20, `dmaloop`, device run 21, `readev`, device run 15, `readev`, device run 16, `readev`, device run 17, `readev`, device run 21, `bufsize`.

- **Device.** For one sector of a page read, the NAND transfer of 512 bytes takes 49 us, the ECC decode event passes at once, the DMA to RAM takes 14 us (about 36 MB/s), and the whole time from the command to the data in RAM is 123 us. Evidence: `readev`, device run 15, `readev`, device run 16, `readev`, device run 17, `readev`, device run 21.
- **Device.** The second register set runs a READ ID (command, address, 8-byte transfer counted in `0x424`), has its own data buffer and DMA engine, and its DMA flag is `wait` event 8. No chip answered on it, with or without its pins switched; chip enables CE0..CE3 on the first set answered. Evidence: `ev8`, `dmaloop`, device run 20, `dmaloop`, device run 21, `chan1`, `chan1pins`. **Inferred:** only channel 0 is used on this board, from the boot ROM's two-channel pin set-up (RE-43) and the single board tested. Findings 73, 79, 82, 93, 100.
- **Device.** `FMC_STATUS` bit 27 is set after every written sector read and clear after every erased one: it means that the sector that was read had been written. Evidence: `rawpage`, `rawwrite`. Bit 27 is set for sectors with and without stored errors; the third-party driver classifies it as "programmed page" (3p).
- **Device.** `wait2` does not wait on the second register set's flags: with each second-set flag pending alone, `wait2` conditions 0 to 15 were tried and only condition 5 passed (condition 33 was outside that range); `wait` events 6, 7, 8 and 9 passed in the same states. Evidence: `op09d`.
- **Device.** `FMC_DMA_CTRL` = 7 is needed for the drain with `FMCTRL1` = `0x1e0` to deliver correct data; `FMC_CTRL0` bits 9, 11 and 24 made no difference to a page read. Evidence: `fmcbits`.
- **Device.** As seen from the CPU, `FMC_28` changes only between runs; a DMA and the set-up word reset it. **Inferred:** CPU reads of some window word advance it (the dumps read every word). Evidence: `fmc28b`.
- **Device.** The FIFO port `FMC_FIFO` advances `FMC_28` by 4 per read and returned 0 from data buffers filled by DMA. **Inferred:** `FMC_28` is the FIFO port's byte position; the READ ID bytes are held in storage the DMA does not fill, or are read from a data buffer only after a chip transfer (open). Evidence: `bufport`.
- **Device.** `FMC_DMA_CTRL` bits 1, 2 and 4..7 each make DMAs fail, and the value persists between runs. **Inferred:** the wrong data of the `0x102` / `0x1e0` drains in `fmcbits` came from `FMC_DMA_CTRL` = `0xff` left by the case before them. Evidence: `dmactl`, `fmcbits`.
- **Device.** Patterns DMA-ed through the second register set into its parity buffer 0 appeared in the first set's parity buffer 0. **Inferred:** the two register sets share parity buffer 0 (and perhaps all buffers). Evidence: `ch1buf`.

15.7 ECC codes and modes

- **Device.** The engine corrects up to 8 data bits per sector with `ECC_MODE = 0x180`; 9..12 flipped bits give `ECC_RESULT = 0x11` and leave the data as it was sent; `ECC_DONE` is `0x35` for a decode and `0x3d` after an uncorrectable one. A decode in a separate program from its encode reports every sector uncorrectable (inferred: the set-up word clears the buffers). Evidence: `eccstr`, `eccstr2`, `eccpar`.
- **Device.** After an encode the 13 parity bytes are in `0x900..0x90c` and the parity slot can be copied out by DMA. Zero data gives non-zero parity because the three metadata bytes (`0xff` there) are part of the code word. Evidence: `eccpar`, `eccstr2`. The metadata bytes enter the parity.
- **Device.** Every measured parity matches binary BCH, t = 8, over GF(2^13) (`0x201b`), generator of degree 104, message = 512 data bytes + 3 slot metadata bytes, checked against a computed code word for all twelve encodes. Evidence: `eccstr2`, `eccpar`.
- **Device.** Pages written by the original firmware are code words of this code with their real metadata in slot bytes 0..2 (20 sectors); raw writes of chunks built by the CPU read back as written; errors stored on the flash decode as the identified code predicts (8 corrected, 9 not; errors in metadata and parity bits are corrected and counted). Evidence: `rawpage`, `rawwrite`, `eccchunk`, `eccchunk2`.
- **Device.** A parity buffer is 64 bytes; a 512-byte DMA into it wraps and leaves the source's last 64 bytes; the metadata path fills slot bytes 0..2 in order. Evidence: `eccchunk`.
- **Device.** `ECC_MODE` bits: bit 8 selects the slot path. Bit 15 selects a Reed-Solomon code over GF(2^10) with byte symbols. Bits 3:2 together with bit 15 select 8, 12 or 16 parity symbols, which correct 4, 6 and 8 bad bytes (measured; beyond that the sector is uncorrectable). A Reed-Solomon encode and decode ignore the slot field and use slot 0; the 16-symbol parity (20 bytes) fills slot 0 and the first 4 bytes of slot 1. Bit 4 swaps the bytes of each 32-bit word. Bits 20:16 are the data length L. `ECC_DONE` bit 3 is the sticky uncorrectable flag. Evidence: `finale`, `b15ecc`, `rsdecode`, `rschip`. **Inferred:** of the three Reed-Solomon strengths, the 16-symbol form does not fit a 4 KiB page with 128 spare bytes in the 16-byte chunk layout (8 x 532 > 4224), and none of them protects the metadata bytes (chapter 10, section 10.9).
- **Device.** Without bit 8 (`ECC_MODE` 0, `0x80`, `0xe00000`) no code is computed: the n-th such operation since start-up writes the n-th value of a fixed sequence of 61 distinct values into `0x900..0x90c` and `0x990`, whatever the data or the mode; a decode does not advance it and every decode is uncorrectable. Evidence: `eccmode2`, `eccmode4`, `finale`. **Open:** what generates the sequence and which event restarts it

(between two device runs (chapter 13, section 13.2) it restarted once and continued once). Findings 118, 137, 144. In `seqrestart` no action tried restarts the sequence (`seqrestart`); the first such operation can give value 1 (`eccregs`, `eccmode4`) or continue the sequence (`finale`: value 14) (**inferred:** a power cycle restarts it, not tested).

- **Inferred.** Byte order (chapter 10, section 10.14): the FMSS engine is little-endian in instruction fields, words and the ECC area, while the parity in the slot and on the flash is stored most significant byte first. The reading is that the parity order is the bit-serial convention of the code (the division runs over a most-significant-bit-first stream, so the highest-order parity byte comes first), and that `ECC_MODE` bit 4 is an optional byte swap of each 32-bit word for data held in the other order. Evidence: `eccpar`, `eccmode2`, `finale`. The parity byte orders and the effect of bit 4 are device results; the explanation is not tested.
- **Device.** No mode corrects more than 8 scattered bits per 512-byte sector. Evidence: `eccstr2`, `b15ecc`, `finale`.
- **Device.** A Reed-Solomon operation ignores the slot field and uses slot 0. **Inferred:** a Reed-Solomon page needs each sector's chunk moved to and from slot 0 around its encode and decode, so the engine's four-slot layout serves only the BCH code. Evidence: `rschip`.

15.8 NAND operations

- **Device.** `FMCTRL1` takes one operation at a time: writing `0xe2` straight after `0x100` loses the DMA that `0x100` started (finding 36). Evidence: none among the experiments of chapter 13 (a device measurement recorded in the finding list).
- **Device.** A single-page program on CE0 and on CE1, a program on two chip enables in one run, a two-plane program (`80h`, `11h`, `81h`, `10h`), a two-plane erase and an erase on CE1 each give correct pages when the pages are read back raw; multi-page runs with the status poll on `FMC_DATA_SEL = 1` write every page. Evidence: `writeseq`, `eraseseq`, `writeseq2`. **Device.** With the status poll on `FMC_DATA_SEL = 0x100` the latch returns `0x70`, the poll ends while the chip is still programming, and the next `80h` is ignored, so that page stays erased (`writeseq`, `pageorder`).
- **Device.** Reprogramming a page without an erase stores the bitwise AND of the old and new data and reports no FAIL (the status byte is `0xc0`); an erase with four address cycles erases the block; the status byte is `0xc0` when ready, `0x81` right after `10h` and during an erase, `0x80` during a page read's tR (the time the chip needs to load the page). Evidence: `writeseq`, `writeseq2`, `pageorder`, `eraseseq`.
- **Device.** Program run times follow a fixed per-page pattern: they are about 478 us for pages 0..3, 6-7, 10-11, ..., 122-123, and 728 to 873 us for pages 4-5, 8-9, ..., 124-127. The pattern is the same in three blocks on two chips (**inferred:** lower and upper pages of the chip's cell pairs); erase about 1.1 ms with data and about 0.18 ms blank. Evidence: `progtime`, `eraseseq`.
- **Observed.** The original programs follow the sequences described in chapter 11: prologue, chip selection, command and address cycles, the ready test before `wait` event 5, the page stream of `[16-byte chunk][512 bytes]` per sector, one meta encode or decode per page, and the page word built from `FMC_STATUS` bit 27 and the per-sector verdicts. Evidence: RE-5, RE-7, RE-9, RE-12, RE-13, RE-15, RE-16, RE-17, RE-18, RE-19, RE-21, RE-22, RE-23, RE-24, RE-25.
- **Observed.** The `78h` write forms of release 1.1.3 sweep four rows per chip enable with mask 3, where the releases before 1.1.3 used one `70h` with mask 1; of the nine programs, seven are unchanged from 1.0.1 to 1.1.3 (RE-28). **Inferred:** the reason is the status of the die or plane addressed last. **Open:** which row the driver writes to `0xd00`.

15.9 Interrupts

- **Device.** VIC1 raw bit 22 stayed clear for an END, a fault, opcode `10` and a block on opcode `12`, with and without start-word bit 6, after a power cycle and after a soft reset alike. Evidence: `irqline`, device run 15, `irqline`, device run 16, `irqline`, device run 17, `ctrlfx`, `state`, device run 17, `state`, device run 18.
- **Device.** NAND controller enable `0x044` bit 0 routes `FMC_STATUS` bit 1 into `CS_IRQ` bit 4; ECC enable `0x844` bits 0, 2, 5 route `ECC_DONE` into `CS_IRQ` bit 6; single `0x044` bits that put a pending flag into `CS_IRQ` bit 4 follow no simple pattern. Evidence: `irqen`, device run 16, `irqen`, device run 17, `irqsrc`.
- **Device.** VIC0 line 20 was seen in 4 of 102 runs of the `irqsrc` experiment. With one source at a time after a soft reset, it was seen with a command pending and `0x044` bit 0 set, and with address cycles and `0x044` bit 1 set. It was not seen with the other sources, although `CS_IRQ` bit 4 was set for some of them. Evidence: `irqsrc`, `finale`. **Open:** the exact condition of line 20 and the enable outside the sequencer block that the original driver relies on (RE-32: it polls the raw status bit and writes no enable). Findings 76, 102, 140, 146.

- **Device.** 0x044 bits 0, 1, 2, 11 and 12 feed CS_IRQ bit 4; VIC0 line 20 appeared only while the command or address flag was still pending at the end of the run. Evidence: irq044, vic20.

15.10 Original firmware (reverse engineering)

- **Observed.** The nine programs, their locations, sizes and command orders are in RE-1, RE-2, RE-3; all use the same prologue (RE-5) and the opcode set of RE-4.
- **Observed.** The original CPU driver starts a program with CS_IRQ = 0xff and FMSS_CTRL = 0xfff5, polls VIC1 raw bit 22 for up to 2000 ms, classifies by CS_IRQ bits 2 and 0, resets the sequencer by writing FMSS_CTRL = 0x30000 after a failure, and never reads CS_STATUS or the PC (RE-31, RE-32, RE-33, RE-34, RE-40).
- **Observed.** The timing word has the form setup << 16 | hold << 12 | edo << 28 (RE-35). **Inferred:** the bus clock behind the timing is about 108 MHz; this is not established in any other way.
- **Inferred.** The 16-byte chunk is 13 bytes of parity plus a 3-byte metadata fragment (finding 14): the BCH parity length of 13 bytes and the 4-bit count field are consistent with it, and the chunk bytes read from pages written by the original firmware contain the real metadata in slot bytes 0..2 (RE-15, RE-16, rawpage).
- **Observed.** wait event 5 is used 19 times. 13 uses (all outside ERASE) have one five-instruction shape with the ready test in microcode, and the 6 uses in ERASE stage their waits with C58 = 0 (RE-7, RE-8). **Device:** a timeout with C58 bit 2 clear continues at the next instruction (c58, device run 2, c58, device run 3).
- **Inferred.** A program is the span from its prologue to its last opcode-00 instruction; the instructions from the fault vector offset (C4C) to that last instruction are the program's fault tail (run when a fault takes the vector), and the opcode-00 instruction just before that offset ends the normal path. Evidence: RE-1, RE-5.
- **Inferred.** Opcode 19 stores the one-hot chip mask of the chip just accepted into a READ batch, at an address that starts at 0xd60 and advances by 4; opcode 18 loads it back, once per transfer; The parameter word 0xd50 holds the parameter-RAM offset that the second pld (at READ+0x6e0) reads next: it starts at 0xd64 and READ adds 4 after each load. The offsets used are 0xd60..0xd7c. Evidence: RE-6, RE-17.
- **Inferred.** The five-instruction group of wait event 5 is a ready test in the sequencer program: FMCTRL1 = 0xca starts the NAND status poll, FMC_NSTAT bit 6 is the ready flag, a set flag skips the wait event 5, and the flag is not tested again after the wait. ERASE does the same test in its six uses, but stages C58 around them (C58 = 0 before the first wait, C58 = 4 before the third) and writes C14 = 0xff and CS_IRQ = 4 after the first two; why it does so is open. Evidence: RE-7, RE-8, RE-23.
- **Inferred.** FMC_CTRL0 bits 8:1 select the chip enables (the value written is 1 << (CE + 1)) and every chip change is a read-modify-write that keeps the other bits (mask 0xfffffe01); bit 24 is set around the DMA phases of READ, RAWREAD and the WRITE programs; the base word 0x801 ORed with the timing word is the setting for the data-transfer programs, while READID uses 0x33201 (bit 9 set instead of bit 11; the reason is open). Evidence: RE-9, RE-10, RE-35.
- **Inferred.** Writes to 0x80c with bit 0 set start decodes, and with bit 1 set start encodes (chapter 10). For each sector READ clears ECC_DONE (0x1ff), writes the mode 0x180, starts the decode, waits for wait event 4, acknowledges ECC_DONE and reads ECC_RESULT; the data buffer is moved to memory by DMA after the decode, so the decode works on the buffer before it leaves the controller. Evidence: RE-12, RE-14.
- **Inferred.** The per-sector verdict in READ is formed as follows. If ECC_RESULT bit 0 is set (uncorrectable), bit 30 is put into the page word. Otherwise the corrected-bit count n (bits 19:16, mask 0xf) sets bit n - 1 of the page word. The page word therefore collects one bit per count seen, the highest set bit marks the largest count, and an all-zero low byte means that no sector reported a non-zero count. Evidence: RE-13.
- **Inferred.** READ transfers, per sector, the 16-byte chunk into a slot of the parity buffer (FMC_24 = 0, 0x10, 0x20, 0x30) and then 512 bytes into a data buffer, sector after sector. WRITE sends the chunk first (from the slot selected by FMC_28) and then the data. WRITE runs the meta encode once per page before the first sector's encode and READ runs the meta decode once per page after the last sector's decode; FMC_METACFG is always 0x3210, and the three meta words are 0x060..0x068. Evidence: RE-15, RE-16.
- **Inferred.** READ issues 30h for pages on distinct chips before it transfers any of them; a batch ends at the first repeated chip (the new mask overlaps the accumulated mask) or when the page count is exhausted; at most 8 masks are queued because there are 8 chip-enable bits; the per-page results are stored in request order at the address in 0xd24. Evidence: RE-17, RE-18.
- **Inferred.** The page word starts as 0x20000000 when FMC_STATUS bit 27 stayed clear for the page (an erased page: no ECC data was carried, chapter 10). The last sector's verdict is added to it, bit 27 is acknowledged at the end of the page, one summary word per page is stored, and all the summary words are ORed into r6. Evidence: RE-18.

- **Inferred.** FMC_DMA_ADDR is written once per DMA unit and FMC_DMA_CTRL = 7 before the last DMA of a run; WRITE reads the DMA list one entry ahead, and the number of entries read is the number of pages times the number of units per page. Evidence: RE-19, RE-20.
- **Inferred.** After 11h the two-plane programs poll the chip (70h, ready test, and a fail test on FMC_NSTAT bit 0, written NSTAT & 1) before sending 81h. A failed test jumps at once to the fault tail at the C4C offset, where r6 holds the failing chip-enable mask in bits 7:0 and bit 28 set. Bit 27 is set in READ, READID and RAWREAD unconditionally, and in the other programs only when C50 bit 0 is set. Evidence: RE-21, RE-22, RE-25.
- **Inferred.** ERASE sends 60h, three row cycles and D0h for each list entry without waiting, then runs a status sweep per chip enable with the staged ready wait; on a normal end r6 is the mask of the chip enables used. READID sends FFh with all eight chip enables selected, runs a software delay (count from 0xd14), then for each listed chip sends 90h and one address cycle and reads 8 ID bytes through FMC_FIFO, reading the FIFO twice and storing the first word. Evidence: RE-23, RE-24.
- **Inferred.** RAWREAD reads 4 x P_CHUNKS + 1 blocks of 512 bytes with no parity-chunk handling and no ECC access, and always uses the data buffer selected by 0x102 where READ and WRITE alternate 0x102 and 0x201. Evidence: RE-26.
- **Inferred.** WRITE2P sends 80h .. 11h then 81h .. 10h; WRITE2P_78 repeats 80h; WRITE_78 and WRITE2P_78 poll with 78h and a three-byte row and test NSTAT & 3; WRITECACHE sends 15h while the remaining page count (0xd18) AND the word 0xd44 (0 minus the bank count from 0xd04) is non-zero, and 10h otherwise, and ends with a 71h status sweep. Evidence: RE-27, RE-46.
- **Inferred.** The programs take a list of logical chip numbers (0xd10) and a map table (0xd14) and turn each entry into a chip-enable bit through the map; READID takes chip-enable indices directly. The 1.1.3 change replaces one 70h poll (mask 1) per chip enable by four 78h polls (mask 3) in WRITE_78 and WRITE2P_78. Evidence: RE-28, RE-29.
- **Inferred.** The original CPU driver's wait function classifies by CS_IRQ: CS_IRQ bit 2 means a fault (the function prints R0, R2 and R6, resets the sequencer and fails), bit 0 means done, and if neither is set the function reports an unknown interrupt and fails; the timeouts of 2000 ms (wait) and 100 us (clear) hold if the word at 0x3c7000b4 counts microseconds. Resetting is FMSS_CTRL = 0x30000, a 1 ms wait, an idle program of one all-zero (opcode 00) instruction, a start and a 1 ms wait. Evidence: RE-32, RE-33, RE-34.
- **Inferred.** The timing word is built from three nanosecond figures per record (c1, c2, c3 = ceil(figure / T)), with T = 1000 divided (integer division) by the value the driver's clock function returns for argument 2, read as the bus clock in MHz; the first triple feeds READ, RAWREAD and ERASE through 0xd4c and the second feeds the WRITE programs through 0xd48. Evidence: RE-35.
- **Inferred.** The CPU does not reset the chips; the READID program does (its first command is FFh), and the initialisation function resets the sequencer, zeroes FMC_CTRL0 and the register at +0x400, builds an identity chip-enable map of 8 words and runs the READID program. Evidence: RE-36, RE-37.
- **Inferred.** The chip modes 1, 2, 3, 4, 8, 9, 12 and 13 select the write program (WRITE, WRITE_78, WRITE2P, WRITE2P_78 or WRITECACHE), the number of planes (the halfword 1, 2 or 4) and the two callbacks. Evidence: RE-38.
- **Inferred.** The driver's result classification: for a run result word, bits 7:0 are the per-page correction bits, bit 29 an erased page and bit 30 an uncorrectable page; the evaluator returns 0x80020000 for an uncorrectable page, 1 for a single erased page (bit 29, low byte 0), 0 otherwise and sets the level (1..8) to the highest correction bit; for a program or erase, bits 7:0 are the chip-enable mask, bit 27 is set by the fault tail when C50 bit 0 is set, and bit 28 marks a status fail or a sequencer fault (the driver returns 0x80030000 when it is set). Evidence: RE-13, RE-18, RE-25, RE-39.
- **Inferred.** The driver never sets an ECC mode (the programs do) and runs no buffer-reset handshake (as the 4G and 5G drivers do), because it never accesses 0x814, 0xc60 or 0xc64; it never reads CS_STATUS or the PC. Evidence: RE-40.
- **Inferred.** The second set of nine programs is a duplicate that the driver does not use, because no word of the image points to it; it sits in the restore image's update payload. Evidence: RE-3, RE-41.
- **Inferred.** The strings place the driver in the Whimory 2.1 FIL layer for the M1A flash interface; the same text shows that the original firmware reports a sequence timeout. Evidence: RE-42.
- **Inferred.** The boot ROM reads a page without the sequencer, one controller per channel (stride 0x400) and one sector at a time through the data buffers and the ECC engine, and its channel set-up switches the channel's pins to function 2; only channels 0 and 1 are handled. Evidence: RE-43.
- **Inferred.** The original CPU driver does no cache maintenance in any function of the NAND driver, including those that load and start programs (the coprocessor-15 transfers in the image lie outside the driver). Evidence: RE-44.

15.11 Open questions

The points that remain open are collected in chapter 17: the other conditions of `wait2`, why there are two nop instructions, register numbers 8 and 10, `C58` bit 1, start-word bit 6, `wait` events 10 and 12 to 14, the `C08` code set, the cause of `C50` bit 0, the unused control words, intermittent status latches, the interrupt line, the `0x100` block, `FMC_28`, channel 1, the generator of the modes without bit 8, the stronger Reed-Solomon forms, the row in `0xd00`, and the soft-reset differences.

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 13, chapter 14, chapter 16, chapter 17, chapter 18.

Chapter 16 — Device findings: risks and what only a device can settle

Summary: Risks that were identified for the engine and their status (section 16.1), and the results that only a device could settle, grouped by topic (section 16.2). The finding list with the evidence per finding is in chapter 18.

16.1 Risks and their status

#	item	what was at stake	status
1	Meaning of `wait` event 5	an erase or program that polls on `wait` event 5 can end in a timeout fault too early, or wait without end	settled: the timeout (about 4.87 ms), `FMSS_C58` and the arming rule of `wait` event 5 (chapter 16, section 16.2, chapter 7, section 7.1); the intermittent cases are open (chapter 17, section 17.2)
2	Sequencer instruction rate	delay loops in a program are sized from the time per instruction	settled: about 216 ns per instruction (chapter 16, section 16.2)
3	`FMC_STATUS` bit 27 semantics	the READ program's page word (its page result) depends on it	settled (device): set for every written sector, clear for erased ones (RD7, RD8)
4	Blank chunk decode result	an erased sector might decode as uncorrectable	settled (device): decoding an erased sector gives `ECC_RESULT = 0`
5	`FMC_CTRL0` bit 24, `FMC_DMA_CTRL = 7` placement	DMA stalls or truncates	the original programs set bit 24 around DMA and write `FMC_DMA_CTRL = 7` before the last DMA (R7); `FMC_CTRL0` bit 24 enables DMA (device); the role of `FMC_DMA_CTRL` is open
6	Parity-buffer reuse between pages	corrupted metadata or parity	the original programs finish the metadata encode / decode of a page (the copy between the metadata registers and the slots) before the parity slot is reused (RD6, WR3)
7	`pld` / `pst` (register-window load / store at an offset held in a register) outside `0xd60..0xd7c`	which addresses the indirect accesses can reach	settled: the offset wraps at 12 bits; they reach the whole register window including the sequencer registers r0..r7 (chapter 16, section 16.2)
8	Whether a run that contains only an `end` instruction, started after a reset, leaves flags set	waits in the next run pass early because of stale flags	the original prologue clears all flags at every start (C2); the run of only an `end` instruction after a reset is explained in chapter 2, section 2.3

16.2 Settled on a device

Small test programs were run on the sequencer of a unit. The CPU preset registers and parameter words, started the program, and read the register file, the PC and the control, NAND controller (FMC), ECC and parameter windows before and after, and sampled them while it ran. The chip enables were deselected throughout, except where a row says otherwise. Each result below was repeated in at least two runs, from a clean state (`C58`, `C4C` and `C5C` written 0 before each start).

16.2.1 Registers, instructions and fields

topic	result
register file	r0..r7 are `0xc18 + 4n`; `SEQ_RESULT` (`0xc30`) is r6. A CPU write before the start is the register's starting value; a program start keeps the register values, a sequencer reset clears them
speed	about 216 ns per instruction (measured with a two-instruction loop: 873 us for 2 000 iterations, 8 657 us for 20 000)
instruction times	Each instruction was timed by executing it 100 x 10 times in a loop. Program in SDRAM: 217 ns per instruction for `movi`, `mov`, `add`, `0x08` (nop), a `bnz` not taken and a `wait` that passes; +37 ns for every access to the register window (`wr`/`st`/`ld`/`pld`/`pst`, NAND controller or parameter RAM alike); `ldi` +170 ns, `sti` +186 ns (RAM in SDRAM). The same programs from internal SRAM: 138 ns per instruction, window access +35 ns, `ldi` +218 ns, `sti` +120 ns. Instruction time is set by the fetch of the 8-byte instruction, not by the engine
ALU	`shr` is logical; a shift count of 32 or more gives 0 (immediate or register); `add`/`sub` wrap at 32 bits; an immediate of 0 selects the two-operand form (`r = r op rA`); `ld` ANDs the value read with its mask
memory	`ldi`/`sti`: little-endian 32-bit words; an unaligned address is rounded down to the word
`pld` / `pst`	the offset wraps at 12 bits (`0x1d64` reads `0xd64`); `pld 0xc18` returns r0
parameter RAM	32 words `0xd00..0xd7c` (chapter 4, section 4.4)

topic	result
field widths	wr/st/ld offsets 0x1d70, 0x8d70, 0xfd70 reached 0xd70; branch targets and values written to r8 (the PC) with bit 12, 16 or 24 set went elsewhere, with bit 31 set landed on the target; jr (opcode 0x0f) with any of those bits set did not jump
end fields	the R or arg field non-zero: the instruction faults instead of ending; the imm field is ignored; an END leaves its instruction word and imm in 0xc40 / 0xc44
Fields left 0	imm of st, ldi, mov, sti, pld, pst, R of wr, arg of movi: setting them to 0xff or 4 had no effect
branches	bz/bnz ignore the argument field, test all 32 bits (0x80000000 is non-zero), and take a target that is not a multiple of 8 as it is (a target 4 bytes after a label lands in the middle of an instruction, 8 bytes before it runs the instruction before the label)
wait fields	the argument and the immediate are ignored (register offsets with masks of set or clear bits made no difference); event numbers 16, 27 (event 11 plus 16), 0x8b (event 11 plus 128) and 255 never passed: all 8 bits of the event number count, and a number above 15 does not act as its low bits
register numbers 8..255	every number in every role (regsweep; the table is in chapter 4, section 4.3): as a destination only 8 (jump) and 10 (all of r0..r7) act; in the argument (arg) field 9 and 11..255 read r0; bnz / bz with 8..255 never branch; jr, ldi, pld with 9..255 use r0 as the register or address; st with 8..255 stored the value the instruction just before it had written (0 after movi r6, #0 in regnum, 0x5e075e5e after movi r7, #0x5e075e5e in regsweep); no number reaches a control word. With the instruction before the st varied (stprev), st with 8..255 stored the last value that passed through the register file: the value the previous instruction wrote into a register, or the register value it read (r5 after st [..], r5, r0 after nop or wr); wait left it unchanged.
r8, r10	r8 is the PC: movi r8, #label and mov r8, r1 jump to the label, mov r1, r8 returns the offset of that instruction, and ldi r1, [r8] reads the bus address equal to that offset. A write to r10 goes to all eight registers (movi, mov, add). Register numbers 24 and 26 do nothing, so the whole field is decoded
writing the PC	the program's wr [0xc38], #label and st [0xc38], r1 jumped to the label; ld r2, [0xc38] read the offset of that instruction
0x0f	**jump through a register**: PC = r[arg], a byte offset from the program start (marker programs: arg 1 with r1 = label 1 landed on label 1; register field 1 with arg 0 landed on r0's label, so the target comes from the argument field; arg 9 and arg 0xd70 used r0; the immediate is ignored). The offset is not rounded: +4 lands in the middle of an instruction and the stream is decoded from there; -8 runs the instruction before the label. No return address is kept (r0..r7, C3C, C48 unchanged), so a subroutine return is "load the return offset into a register, 0x0f through it". Further cases fit: with RAM addresses in the registers the PC ended at that address + 8 and the fetched words were not program code; with r1 = 0xd70 it ran into zeroed memory and ended

16.2.2 Opcodes

topic	result
opcode map	all 256 opcodes, each in three operand forms (registers seeded with RAM pointers, C58 = 0). The 19 opcodes used by the original programs (0x00..0x07, 0x0a..0x0e, 0x11, 0x13, 0x14, 0x17..0x19) plus 0x08, 0x0f, 0x10, 0x12, 0x15 and 0xff did not fault in the sweep. 0x09 (wait2) and 0x16 (evjmp) timed out there, because the sweep used no wait2 condition 5 or 33 and no event was pending. No other opcode changed a register, RAM, parameter RAM or any window word. The decoder uses the whole opcode byte (0x20, 0x40 and 0x80, each 0x00 with one high bit set, fault like the other illegal values instead of acting as end)
opcodes without a meaning	0x08 and 0xff: no visible effect, and they take no longer than movi (1 000 executions each, three operand forms). 0x0f: an indirect jump (row 0x0f). 0x10: sets CS_IRQ bit 1 and continues. 0x12: stops the program with no timeout (CS_IRQ bit 3, CS_STATUS = 0xa, the instruction held in 0xc40 and 0xc44, FMSS_CTRL bit 18); it is the pause instruction (row "pause (opcode 12)"). 0x15: waits for the timer C14 (row "timer C14"). 0x09 and 0x16: in the opcode sweep they stalled about 4.87 ms and then raised the timeout fault; they are wait2 and evjmp (rows wait2 and "opcodes and pending flags"). Every opcode 0x1a..0xfe: stalls about 4.87 ms, then the timeout fault
0x12 and its fields	stops the program in every form tried: register field 0, 3 or 11 (with the matching event set or not), argument 5, immediate 0x40; so it is not a wait without a timeout.
0x12 and CPU writes	none of the following CPU writes released it: C48, C54, C3C, C5C, C58, C4C, C10, FMSS_CTRL (0x1, 0x3, 0x9, 0x4, 0x20001), a CPU-sent 70h, FMC_STATUS all ones, parameter RAM, r0, C14; FMSS_CTRL = 0x3 changed C08 to 3 and FMSS_CTRL = 0x9 changed it to 0, without releasing the pause; C58 = 0xff set C5C. The CPU continues it by writing the PC (0xc38) and then a start word (chapter 6); otherwise the reset ends it
pause (opcode 12)	the program stayed blocked for 2 ms; then CPU writes of the PC (0xc38) = the offset of the pause instruction + 8 and of a start word continued it (END 25 us later), with or without acknowledging CS_IRQ bit 3; acknowledge then start, start then acknowledge, and start twice left it paused
opcodes and pending flags	from a clean state, with nothing, one flag (command, DMA, ECC, status), all four, or one of the second register set's flags pending: 08, 10, 15, ff behaved the same in every state; 12 blocked in every state; 16 went to the fault vector for every flag; 09 timed out in every state with event field 0
wait2	condition 5 passed in every state of every run; condition 33 passed on an idle chip in op09w33 and during an ECC operation and a DMA in finale, and in no state of evsweep2; in evsweep2 condition 20 passed with every interrupt enable on and a command flag pending; every other N (0..255) never, also during a chip's erase, program or read, an ECC operation, a DMA and after a command to the second register set. With each flag of the second register set pending alone, wait2 conditions 0 to 15 were tried and only condition 5 passed, while wait events 6, 7, 8 and 9 passed for the second set's command, address, DMA and transfer flags respectively. In w2state, which tried many device states, condition 20 passed only with a command flag pending and 0x044 bit 0 set; condition 33 passed in none of 34 states, the first program after a power cycle included. With both raw interrupt status words read just before each wait2 (w2vic), wait2 condition N followed raw interrupt line N (VIC0 0..31, VIC1 32..63) in 64 of 64 conditions in four states; the NAND controller raised line 20 and the ECC engine line 43.

topic	result
check of the rules	seven combinations of the rules above (three timeouts in a row with C58 = 0: about 14.6 ms, CS_IRQ = 5, the program runs on to its END; a shift by 31; a pld of r6 through the window; four 0x15 instructions; a 100-iteration loop; a wait event 11 followed by a timeout and a 0x10 instruction, giving CS_IRQ = 7; a store and an unaligned load) gave the results the rules above give for them; FMSS_C50 bit 0 was not set by a fault, although the original handlers suggest it would be

16.2.3 Events, waits and the timeout

topic	result
wait	events 0..10 and 12..15 were not set with nothing pending and timed out after 4 865..4 882 us; event 11 was set at idle (the wait passed at once; it is not the chips' ready line, see the row "event 11")
events, full sweep	wait events 0..15 in five states (CE0 selected and idle; after a command byte; after 70h + status latch; right after an erase confirm; 2 ms into it), with no status read before the wait: only events 0, 11 and 15 ever passed. Event 0 and **event 15** pass after a command byte and fail once FMC_STATUS bit 1 is cleared (event 15 passes while an enabled interrupt source is pending, see the row "page read events"); event 11 passes in every state; events 1..4 and 6..14 never passed in these states (events 1..4 belong to steps that these states did not include: address, ECC, DMA, transfer). The idle instruction time inside a delay loop was 203 ns here (216 ns in the row "speed")
event 11	passes at once also during a block erase (status 0x81 read at that moment), with the other chip enable selected and with none: it is not the chips' ready line
events and FMC_STATUS reads	a sequencer read of FMC_STATUS or FMC_NSTAT (once or twice) before the wait does **not** consume an event (status latch then wait event 5: passed in all four variants; wait event 0 after a read passed; wait event 3 timed out with and without a read). The cases that appeared to contradict this come from the latch: 70h immediately followed by FMCTRL1 = 0xca (the command-done flag, FMC_STATUS bit 1, still set) never passes wait event 5, although FMC_STATUS bit 23 reads set; if a wait event 0 and a clearing of bit 1 follow the 70h, the same latch passes. So wait event 5 is not simply bit 23 (the arming rule in the row "wait event 5" explains this in part; the straight-after case stays open). One wait event 3 with CE0 selected passed in one program and timed out in three other programs: unresolved
wait event 5	a status read (70h, then FMCTRL1 = 0xca) during an erase: the wait passed about 155 us later and the status byte went from 0x81 to 0xc0; idle: at once. Re-arming with FMCTRL1 = 0xca alone (no new 70h) eight times gave eight timeouts. The arming rule: a latch sets bit 23 only when armed and uses the arm; FMCTRL1 operations (set-up word, 0x100e0, a transfer, a DMA) and address cycles re-arm it, and FMCTRL1 = 0x20, a status-flag clear, a chip-enable toggle and an ECC kick do not; a chip's busy-to-ready transition after the last latch re-arms it too (two-chip polls in both orders). In latchrep, 160 single latches passed (with and without a fresh arm, with and without 0x044 / 0x844 written; bit 24 never set), a second latch preceded only by a new 70h (no FMCTRL1 write) timed out 40 of 40 times, and wait event 3 on CE0 passed 40 of 40 (latchrep)
Status latch select	with FMC_DATA_SEL = 0x100 the latch returns the last command byte (0x70), not the status
events after the power cycle	wait events are levels, not consumed flags (see chapter 7, section 7.1); wait events 6, 7 and 9 belong to the second register set; wait event 15 never passed with the interrupt enables clear
FMSS_C58	per bit as in chapter 8, section 8.1. Every value 0..31 and 0x40, 0x80, 0xff, 0x100 gave the same timeout time; bits 7:3 had no visible effect; it holds eight bits (0xffff reads 0xff, 0x100 reads 0)
C58 bit 1	an END ran the fault vector once when the CPU had written C58 = 2 and then C4C; with C4C written by the program, no END was diverted, with or without a fault before it (oneshot). In c58b1 none of ten CPU write orders (C58, C4C, C5C in different orders and repeats, C58 = 3 or 6) diverted an END (c58b1); C58 = 3 set C5C at the END (bit 0). With the job's clean-up off (c58b2), C58 = 2 diverted no end in three program runs in a row, whatever the order of the C4C write; C58 = 6 diverted the first end once; C58 = 3 diverted every end after the first.
start word and the timeout	with start word 0x1: an unmet wait and an illegal opcode waited for ever (no fault, timer frozen at 0xff); 0x5: fault after about 2 us; 0xfff5 / 0xffb5: fault after 4.85 ms; start word 0x1 with any one of bits 15:4 also set (bit 2 still clear): no timeout either, so only bit 2 enables it; no start word changed evjmp or raised a CPU interrupt line
Timeout length	(start-word bits 15:8 + 1) x 2^s bus cycles, s = 3, 5, 7, 9, 10, 11 for bits 7:4 = 1, 2, 3, 4, 5 and 6-or-above respectively (0: immediate); measured with 24 start words
C10, C3C, C48, C54	none changes the 4.87 ms timeout or the behaviour of a wait2, a DMA with an ECC encode or tmwait (values tried: C54 0..0xfff, C10 0..0xff, C48 / C3C 0 and all ones; ctrlwords). C3C reads 0 after every run whatever the CPU wrote; inside a fault handler it holds the offset of the faulting instruction + 8 (faults at 0x28, 0x40, 0x70, and a second fault inside the handler)
timer C14	a wait that has to wait loads the start word's bits 15:8 (0xff with 0xfff5; 0xfe .. 0x7f when one of start-word bits 8..15 is cleared, startbits) and counts down (0xf6 read mid-wait), reads the reload value after the timeout; opcode 0x15 from a non-zero value (also a frozen 0xf0) waits a full period and leaves 0, from 0 reloads at once; two tmwait in one program made the whole program run 4841, 18 and 3250 us in three runs (ctrlwords); writes are ignored Four tmwait in one program alternated regularly in five runs (tmwait2).

16.2.4 Start word, states and the window

topic	result
start word	each bit alone with bit 0, on a 1 000-iteration loop: bits 2 and 4..15 and 17..31 change nothing (same time, same samples); bits 1, 3 or 16 keep the program from running (FMSS_CTRL bit 18 set, PC 0, no END within 50 ms), as do 0xfff7, 0xfffd, 0xffff. Bits 0..15 read back (bit 0 cleared), bits 31:16 read 0 apart from bit 18. Without bit 0 nothing starts. The original CPU driver's start word 0xfff5 has every bit of 15:0 set except bits 1 and 3. In startbits and startdiff (startbits, startdiff): bits 4..31 except 16, flipped one at a time from 0xfff5, changed nothing in program fetch from SDRAM and SRAM, 2000 ldi / sti / pld, sig, a DMA with an ECC encode or a vectored fault, except for the timeout length set by bits 8..15; 0xfff5 / 0xffb5 and 0xffd5 / 0xff95 gave the same results in twelve programs. The same loop ran in 497 us from SRAM and 811..828 us from SDRAM
start word bits 1, 3, 16	**bit 1 = pause before the first instruction**: the program waits (C08 = 3, instruction 0 fetched into 0xc40/0xc44, bit 18 set) and runs in full once the CPU writes the start word without bit 1 (0x1 or 0xfff5, 2 ms later: END 411 us after that write); writes of 0x2 keep it paused. **Bit 3** and **bit 16** put it in the post-reset state (instruction 0 fetched, not run; C08 = 0) until a start without them; repeating 0x9 or 0x8 does not step it, so there is no single-step. Bit 17 alone does not clear r0..r7. Bit 3 reads back 0, bit 1 reads back 1
pause and abort while running	a start word with bit 1 set, written by the CPU 1 ms into a 20 000-iteration loop: the program froze at once (no sample after the write; C08 = 3) and, with 0xfff5 written 4 ms later, continued from the same point (END 4 ms late). Bit 3 or bit 16 written while running: the program stopped and, released 4 ms later, ran again from instruction 0 (END 5 ms after the release, the full run). A start word without these bits written while a program runs does nothing (no restart)
reset and start	after FMSS_CTRL = 0x30000 the first START fetches instruction 0 into 0xc40/0xc44 and executes it only if it is end (an END-only program completes from any address); any other instruction waits with FMSS_CTRL bit 18 set, and a second START (any CPU write of FMSS_CTRL with bit 0 set) runs the program from instruction 0. Register accesses between the reset and the START make no difference; writes without bit 0 do not release it
C08	a state code (chapter 8, section 8.1): read by the CPU 1 ms into each state: 1 running, 4 waiting (wait event 2) and 4 stalled (illegal opcode; C14 counted down to 0xc0 in both), 0xa blocked, 3 paused, 0 held by start-word bit 3 and after an END; 5 when the program read it itself (after starting a DMA, an ECC operation or a chip command, after sig and in a fault handler: always 5, c08self)
C50	bit 0 is set while a command flag is pending and clear otherwise; a timeout or an illegal opcode does not change it, and a handler's write of 1 does not clear it (c50fault); bit 8 stayed clear after soft resets (c50fault). Bit 6 was set after a second-set command (wait event 6 pending); bit 8 did not appear in 17 reads (c50b8).
CPU register masks	the value read back after a CPU write of all ones, of 0 and of 0xa5a5a5a5, for every block of the window; the masks are listed in chapter 3, section 3.3
address decode (sequencer view)	pld of every word 0x000..0xffc; the decode (offset bit 9 not decoded, the mirrors, the second register set, the 0x100 block) is described in chapter 4, section 4.2
access matrix	every word 0x000..0xffc (27 words that trigger an action when accessed were left out), CPU and sequencer writing all ones / zero, read by both: see chapter 3, section 3.2
block 0x100	writable words (sequencer pst, read back at the 0x300 copy too): 0x1b8 1 bit wide; 0x1bc..0x1c8, 0x1d4, 0x1d8 32 bits (power-on value all ones); 0x1cc, 0x1d0 24 bits (power-on values 0x008060 and 0x3010d0); the rest reads 0 and keeps nothing. Meaning open. 0x1cc and 0x1d0 set to 0 or 0xffffff did not change the time of a 2112-byte page transfer (220 us), so they are not NAND bus timing (finale)
FMC_28	a 9-bit value, writable. It advanced by exactly 128 over 128 iterations of a pld/sti loop, and rose by 0x7c (124) per probe run whatever the program did (64 instructions of eleven kinds, sequencer reads of FMC_28 included, made no difference). In programs that write the set-up word first it read 0 before and after 32 repetitions each of plain instructions, sti, ldi, parameter-RAM writes, DMAs (0x2e0), ECC encodes and 16-byte chip reads (finale): none of these advance it, and the set-up word (or something in it) clears it. It is the read position of the FIFO port (next sentence). In fmc28b it did not change while a program ran, except that a DMA and the set-up word set it to 0; it advanced by 248 between the CPU's before-dump and the start and by 124 between the end and the after-dump. It is the read position of the FIFO port: each read of 0x080 advanced it by 4, wrapping at 512 (bufport).
interrupt line	VIC1 raw bit 22 stayed clear for an END, a fault, opcode 0x10, a 0x12 stop and every start word, after a power cycle and after a soft reset alike. The NAND controller and ECC interrupt enables route flags into CS_IRQ bits 4 and 6 but not into that interrupt line; VIC0 line 20 was seen in some cases. The details are in chapter 3, section 3.4; the enable outside the sequencer block that would raise the line is open. 0x044 bits 0, 1, 2, 11 and 12 each raise CS_IRQ bit 4, the other 27 bits do not (three of three runs); VIC0 line 20 per source was the same in three of three runs (irq044, vic20) One operation per run (irqop, device run 37): 0x044 bit 0 routes the command flag, bit 1 the address flag, bit 2 the transfer flag, bit 11 the DMA flag and bit 12 the status flag into CS_IRQ bit 4 and VIC0 line 20; wait event 15 never passed. Programs that ended with such a flag pending and enabled stopped the device from answering USB in irqop, device run 36; with the flags acknowledged and the enables cleared before the end they ran.
bus error	a DMA to 0x50000000 never finished: its wait event 2 timed out (about 4.88 ms); the handler read C3C = 0x70 (the vector) and C50 = 0xffff1800. An ldi from 0x50000000 completed without a fault. C50 bit 0 was still never seen set. (The CS_IRQ bit 4 seen in that run and later came from the interrupt enables the register-mask experiment had left in the NAND controller word 0x044, not from the DMA; see chapter 8, section 8.1)

16.2.5 NAND controller and DMA

topic	result
DMA	FMC_CTRL0 **bit 24 enables DMA.** RAM -> buffer -> RAM loopback (0x2e0, then 0x100, or FMC_DMA_CTRL = 7 + 0x1e0), 512 bytes: with bit 24 clear nothing moved in either register set (waits timed out, FMC_STATUS bit 20 never set); with it set the data came back intact through both sets. Through the second set wait event 8 passed and wait event 2 timed out, so the second set has its own data buffer and DMA engine and event 8 is its DMA flag. Draining with 0x100 left FMC_STATUS bit 25 set, FMC_DMA_CTRL = 7 + 0x1e0 did not. The DMA address register read back the written value after the transfer. On a page read (fmcbits): FMC_CTRL0 bit 24 left set throughout, bit 9 set or bit 11 clear changed nothing; FMC_DMA_CTRL = 7 before the last DMA added about 100 us, 0xff hung it; drain selects 0x102 and 0x1e0 delivered wrong data and left FMC_DMA_CTRL = 0xf, except 0x1e0 after FMC_DMA_CTRL = 7. FMC_STATUS bits 12, 13, 18, 19 are set after a soft reset, not cleared by writing 1, and a data read clears 13 and 19 (fmcstat) FMC_DMA_CTRL bit by bit (dmactl): bits 0 and 3 give correct data, bits 1 and 2 make every DMA after the first time out, bits 4..7 make every DMA time out; with FMC_DMA_CTRL = 0, 1 or 8 the drain select 0x1e0 and the alternating combined selects work.
DMA length	RAM -> buffer -> RAM with FMC_DNUM 255, 511, 512, 1023, 2047: always 512 bytes moved
second register set	READ ID through the 0x400 set on CE0..CE7: no answer (the FIFO port returned 0x4a01faa1, 0x679a704e, then 0x01010101), every wait timed out; through the 0x000 set CE0..CE3 all answered ec d5 14 b6 74 .... With only the channel-0 pins configured the 0x400 set drives no chip and raises no sequencer event. With its pins switched as the boot ROM does, the READ ID still timed out and returned ID bytes 0x00 on CE0 and 0xff on CE1..CE7 (chan1pins), and channel 0 unaffected by the pin switch. READ ID through 0x400 step by step: 0x408 = 0x90, 0x448 bit 1; address: bit 2; ID transfer: 0x430 = 7, 0x424 counted to 8 (bytes moved), 0x404 read 0x100, 0x448 bits 3 and 27. The set's state machine runs; nothing answers on its bus It has its own data buffers 0 and 1 and parity buffers 0 and 1; parity buffer 0 is shared with the first set (ch1buf).
page read events	one sector of a page on CE0, each event tested after each step: address cycles -> event 1; 30h -> event 0; no event after the status poll and after 00h; parity and data transfers -> event 3 (512 data bytes: 49 us after the start of the transfer, about 10 MB/s); ECC decode -> event 4 at once; the DMA to RAM did not finish (event 2 never came; the ECC and NAND controller interrupt enables and ECC configuration words were still at the register-mask experiment's patterns, so this step was repeated in other runs). Event 11 passed in every step. (This run had the NAND controller word 0x044 left at 0xa5a5; event 15 passed in it after steps with a pending flag. On clean state event 15 never passed and the other results were the same, see chapter 7, section 7.1)
page read, complete	with bit 24 set around the DMA: address -> event 1, command -> 0, parity and data transfers -> 3 (512 bytes in 49 us), ECC decode -> 4, DMA of the sector to RAM -> 2 after 14 us (about 36 MB/s); one sector from the command to RAM in 123 us, including the chip's page read time tR and the status poll
FMC_STATUS bit 27	set after reading each sector that had been written, clear after reading an erased sector

16.2.6 ECC engine

topic	result
ECC parity	encode of zeros / ones / a ramp: parity in 0x900..0x90c and, by DMA of the parity slot, as 16 bytes (three 0xff metadata bytes + 13 parity bytes); zeros give non-zero parity (the 0xff metadata bytes are part of the code word); a decode clears 0x900..0x90c, 0x9a4 counts decodes. Decodes in a separate program from the encode were all "uncorrectable" (buffers cleared by the set-up word, inferred)
ECC strength	encode and decode in one program: BCH 1..8 flipped data bits corrected (ECC_RESULT = n << 16, ECC_DONE 0x35), 9..12 uncorrectable (ECC_RESULT 0x11, ECC_DONE 0x3d, data left as sent); Reed-Solomon with 8 parity symbols corrects 4 bad bytes, with 12 symbols 6 (7 uncorrectable), with 16 symbols 8 (9 uncorrectable) (rsdecode); the methods are in chapter 10, section 10.2 On the chip (rschip): RS-8 corrected 4 stored bad bytes and not 5, RS-12 6 and not 7. Beyond the limits, 480 decodes (BCH 9..64 bits, RS-8 5..40 bytes) were all reported uncorrectable (miscorr).
ECC code	every measured parity matches binary BCH t = 8 over GF(2^13), p = 0x201b, message = data + 3 slot metadata bytes (chapter 10, section 10.3)
ECC on the flash	five pages written by the original firmware are code words of the identified code with their real metadata; raw writes of chunks built by the CPU read back as written; stored errors decode as the identified code gives (8 corrected, 9 not, metadata bits corrected and counted)
Parity buffer	64 bytes; a DMA into it wraps and keeps the source's last 64 bytes; the metadata path fills slot bytes 0..2 in order
ECC_MODE	bit 8 selects the slot path; bit 15 selects the Reed-Solomon code over GF(2^10) (byte symbols, data only) with 8, 12 or 16 parity symbols by bits 3:2; bits 20:16 = the number of data bytes encoded and checked (the rest counted as 0xff); bit 4 swaps the byte order of 32-bit words; without bits 8 and 15 a fixed sequence is written instead of a code, and no action tried within a run restarts it (seqrestart); ECC_DONE bit 5 is cleared by bits 21..24 (done5) (chapter 10, section 10.10) A Reed-Solomon encode and decode ignore the slot field of ECC_KICK and use slot 0 (rschip).

16.2.7 NAND operations

topic	result
Write / erase sequences	single page, two CEs in one run, two-plane program and erase, erase with an extra address cycle, reprogram without erase (the stored result is old AND new): as in chapter 11, section 11.5.10 (device check there). Program time per page over three whole blocks (progtime): pages 0..3, 6-7, 10-11, ..., 122-123 about 478 us per run, 4-5, 8-9, ..., 124-127 728 to 873 us, the same pattern in every block
Multi-page writes	with the status poll on FMC_DATA_SEL = 1: eight pages in order with 0x2e4 and FMC_DMA_CTRL = 7, and page pairs, all correct; the status byte reads 0xc0 when ready and 0x81 when busy

16.2.8 Soft reset and power-on values

topic	result
soft reset	writing the writable words to their power-on values, acknowledging the status words, rewriting the 0x100 block and zeroing parameter RAM brought the number of window words that differ from the power-on state from 58 to 7, each of them either an action register still holding its last written value (0x008 / 0x408 command bytes, 0x80c ECC kick, 0x404) or a status word that ignores writes (0x050, 0x450, 0x9a4); idle events (only 11), CS_IRQ and the CPU line were as after power-on. Behaviour matched a power cycle in every experiment repeated both ways, so a soft reset of the writable words is sufficient; the soft state still differed from a power cycle in status words (FMC_STATUS bits 17:14, 0x050, 0x448/0x450, ECC-area words 0x980/0x984/0x9a4, C50 bit 8) and in the timer C14, while the behaviour of the interrupt line, interrupt enables, wait event 5 timing and r8/r10 was the same and the page read differed only in event 8 (the second set's flag left set). Power-on values (CPU view, non-zero): 0x020 = 0x7f167f, 0x038 = 7, 0x048 = 0xc3000, 0x04c = 0xff, the same four at 0x420..0x44c (0x44c = 1), 0x800 = 0x20060808, 0x814 = 0xe00000, 0x840 = 0x10, 0xc50 = 0xffff1800; the sequencer view adds the 0x100 block and 0x9c0 = 1

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 17, chapter 18, chapter 15.

Chapter 17 — Open questions

Summary: Every point that these documents marks as open, not measured, unexplained, unknown or only inferred where the text says it is not settled. Each entry gives the status as recorded, the chapter with the details and the finding numbers of chapter 18. Nothing here is a new finding; the entries only collect what the other chapters state, and each open point is listed once.

17.1 Instruction set and sequencer behaviour

#	open point	status as recorded	details	findings
1	Which devices raise interrupt lines 5 and 33, the two `wait2` conditions seen true without FMSS activity (`wait2` condition N is interrupt source N, chapter 7, section 7.2)	not determined	(chapter 7, section 7.2)	120, 174
2	Why there are two nop instructions (`00` nop, `ff` nop2) that behave the same way	device: both have no visible effect in every operand form and take the same time	chapter 6	47, 89
3	`C58` bit 1: with bit 1 alone no `end` was diverted in `c58b1` and in `c58b2`, which also ran three programs in a row with the device job's clean-up switched off; with bits 1 and 2 (`C58` = 6) the first `end` was diverted once; in `oneshot`, `C58` = 2 with `C4C` written by the program diverted one `end`	condition open	chapter 8, section 8.1.15	46, 162
4	`wait` events 10 and 12 to 15 never passed in any state tried (`wait` event 15 also not with an enabled interrupt source pending); `wait` event 11 passes in every state (what it is remains open; it is not the chips' ready line, finding 57); `wait` event 8 passed after a page read through the first register set	open	chapter 7, chapter 16	43, 57, 62, 75, 152, 163
5	`C08`: the full set of state codes	"full code set open"	chapter 8, section 8.1.3	67, 157
6	`FMSS_C50` bit 8, seen set once, after a soft reset (`C50` read `0xffff1900`); not seen after a soft reset in `c50fault` and `c50b8`	meaning open	chapter 8, section 8.1.13	85
7	`FMSS_C10`, `C48`, `C54` can be written by the CPU, but no effect of the written value was found	no effect found	chapter 8	55, 64
8	What start-word bit 6 does: `0xfff5` (the original operating systems) and `0xffb5` (the 4G/5G bootloader) differ only in bit 6; no difference was found in any program tried (timeout, interrupt line, loops, waits, illegal opcodes, `evjmp`, program fetch, `ldi` / `sti` / `pld`, DMA, ECC, reserved-word access); access to other NAND regions under the two words is not measured; reads, erases and programs in the user area and in the blocks above it, on both chips, showed no difference either	open	chapter 8, section 8.1.1	54, 76, 153
9	Timeout length rule: the steps were measured, not derived (n, the value of start-word bits 7:4: n = 5 breaks the pattern of n = 1 to 4); the bus clock (about 108 MHz), which the timeout length and the timing word of the original CPU driver depend on, is inferred and otherwise not established	device (steps); clock inferred	chapter 7, section 7.5, chapter 12, section 12.3	31, 107, 111
10	Two `tmwait` in one program made the whole program run 4841, 18 and 3250 us with three different values written to `C10`, `C48` and `C54` before it (`ctrlwords`), while four `tmwait` in one program alternated regularly between a full wait and no wait in five repetitions (`tmwait2`)	not explained	chapter 7, section 7.3	47, 151

17.2 Events and the status latch

#	open point	status as recorded	details	findings
11	A `wait` event 5 placed straight after a `70h` command and a status latch (`FMCTRL1` = `0xca`) timed out once although `FMC_STATUS` bit 23 read set afterwards; the status latch's `wait` event 5 timed out in one of two identical runs; in `latchrep`, 160 of 160 latches passed	intermittent, cause open	chapter 7, chapter 16	63, 66, 87, 156

#	open point	status as recorded	details	findings
12	FMC_STATUS bit 24 set instead of bit 23 after a status latch (so wait event 5 never came): seen in irqsrc in both status-latch programs with 0x044 or 0x844 written and in one program of op16 with no enable, not in the fifteen programs of ev5en and not in the 160 latches of latchrep	cause open	(chapter 9, section 9.4), chapter 16	104, 110, 121, 156
13	One wait event 3 with CE0 selected passed in one program and timed out in three other programs; it passed 40 of 40 times in latchrep	unresolved	chapter 16	63, 156

17.3 Interrupts

#	open point	status as recorded	details	findings
14	The enable outside the sequencer block that the original firmware sets, so that VIC1 raw bit 22 (a CPU interrupt line) follows END (it never set in the device tests)	open	(chapter 2, section 2.3), chapter 3	76, 102, 140, 146
15	The source of CS_IRQ bit 5	not identified	chapter 8, chapter 16	65, 76, 102

17.4 NAND controller registers

#	open point	status as recorded	details	findings
16	FMC_CTRL0 bit 9 (set in the original READID's control word) and bit 11 (set in every program except READID): no effect found on a page read or a READ ID	meaning unknown	(chapter 9, section 9.1)	11, 158
17	FMC_DMA_CTRL (written 7 before the last DMA of a run): bits 0 and 3 alone deliver correct data, bits 1 and 2 break every DMA after the first and read back with bit 6 or 7 added, bits 4..7 break every DMA; what the bits mean	meaning open	(chapter 9, section 9.3), chapter 11 (R7)	24, 158
18	FMC_STATUS bits 12, 13, 18, 19: set at power-on and after a soft reset, not cleared by writing 1; a 16-byte data read clears 13 and 19, a 512-byte data read clears all four, a DMA into a buffer, a status latch and the set-up word set 13 and 19 again	meaning open	(chapter 9, section 9.4)	78, 159
19	What the FIFO port FMC_FIFO (0x080) reads: it returns the bytes of a READ ID, returned 0 for data buffers filled by DMA, and 128 writes to it changed 26 words of data buffer 0; FMC_28 is its read position	open	chapter 1, section 1.5, chapter 9, section 9.3	60, 145, 155, 164
20	The block at 0x100..0x1fc (writable words 0x1b8..0x1d8): setting any writable word changed neither NAND bus timing nor the time of DMAs, ECC operations, a chip read or a timeout	meaning open	chapter 3, chapter 9, chapter 16	74, 145
21	The words 0x3a8..0x3b4: in access the sequencer's reads returned the values the CPU had written; in blk100b the sequencer read 0xffffffff (also after its own writes) and the CPU read 0	unexplained	(chapter 3, section 3.2)	103
22	ECC engine words 0x990..0x99c (after an encode: a 16-bit value that depends on the data with the Reed-Solomon code, 0 with BCH), which is no CRC-16 of the 512 data bytes (every polynomial, both bit orders and both byte orders of the result checked against the measured values) and 0x800 (an identification word, read 0x20060808, "perhaps a date")	meaning open / perhaps	chapter 10, section 10.13	56, 103, 149
23	No chip answers on channel 1 of this board; "channel 0 only" is inferred from the one board	inferred from one board	(chapter 9, section 9.5)	73, 93, 100

17.5 ECC engine

#	open point	status as recorded	details	findings
24	What generates the fixed sequence of the modes without a code (61 distinct values, no short linear-feedback shift register; not a multiple of the BCH generator, not a 16-bit Galois LFSR), and which event restarts it (none of the ten actions tried between encodes in seqrestart restarts it; the first such encode of a device run gave value 1 in eccmode4 and in eccregs, value 14 in finale, and at value 2 in seqfirst after a power cycle	open (bounded negative searches)	(chapter 10, section 10.12)	137, 144
25	The 2 extra slot bytes with ECC_MODE bit 1 and RS-8 (one sample)	one sample	(chapter 10, section 10.5)	136
26	ECC_DONE bit 5 with ECC_MODE bits 21, 22, 23 alone: clear in done5 and done5b, set in eccmode2	why eccmode2 differs is open	(chapter 10, section 10.11)	139

#	open point	status as recorded	details	findings
27	What an encode writes to `ECC_RESULT` when its code differs from the last decode's: a Reed-Solomon encode after a BCH decode set 0 in `rsresult` or `0x20000` in `eccmode2`; a BCH encode after a Reed-Solomon decode set `0x30000`	open	(chapter 10, section 10.11)	139, 166
28	The reading of the 8-symbol Reed-Solomon form as an older controller's "4-bit ECC per 512 bytes"	"Interpretation (not measured)"	chapter 15	134

17.6 Original programs and the CPU interface

#	open point	status as recorded	details	findings
29	Why the page word discards the verdicts of earlier sectors when `FMC_STATUS` bit 27 stayed clear (RD8); a plausible reason is that a blank sector may decode as uncorrectable (settled for erased sectors by item 4 of chapter 16, section 16.1)	"is not known"	chapter 11 (RD8), chapter 16	22, 117
30	Why `FMC_CTRL0` bit 24 is set and cleared where it is (R7); set for a whole page read it changed nothing	reason unknown	chapter 11	24, 81, 158
31	RAWREAD loads `FMC_DATA_SEL` = `0x102` in all steps; used as the drain select of a page read it delivered wrong data	observed; meaning open	chapter 18	34, 158
32	The reading of the two program times (about 478 and 728..873 us per run) as the chip's lower and upper pages of a cell pair	inferred	chapter 11	131
33	Which row address the original CPU driver writes to the parameter word `0xd00`, which the `78h` programs of release 1.1.3 read	open	chapter 4, section 4.4, chapter 11 (WR9)	37

17.7 Soft reset versus power cycle

#	open point	status as recorded	details	findings
34	After a soft reset of every writable word, status words (`FMC_STATUS` bits 17:14, `0x050`, `0x448` / `0x450`, ECC-area words `0x980`, `0x984`, `0x9a4`, `C50` bit 8) and the timer `C14` still differ from a power cycle	recorded as differing; behaviour matched	chapter 16	78, 80
35	Why the device stopped answering during the first version of `irqop` (`irqop`, device run 36; programs that left a NAND controller flag pending with its `0x044` enable set when they ended); the version that acknowledged every flag and cleared the enables ran without fault (`irqop`, device run 37)	not explained	chapter 13	167

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 16, chapter 18, chapter 15.

Chapter 18 — Finding list

Summary: The list of findings, entries 1 to 175, each with its evidence and its class (device, observed, inferred, 3p).

Program locations are in RE-3; "READ+0xNNN" below means that program's base plus the offset. The entries are numbered within this chapter.

18.1 Findings

#	claim	evidence	class
1	8-byte encoding, 17 base opcodes plus 18/19, byte-offset branch targets, r0..r7, r6 = result	image bytes (RE-4)	observed
2	ALU operand rule: with an immediate (`imm`) other than 0 the source operand is in the `arg` field; with `imm` = 0 the register form is used, so an immediate of 0 cannot be encoded	dataflow check over all uses	inferred
3	Opcode 18 (`pld`) loads and 19 (`pst`) stores a register-window word at an offset held in a register; used only on `0xd60..0xd7c`	`osos mem:0x8009110` (READ+0xe0, store), `mem:0x8009190` (READ+0x160, load), `mem:0x8009710` (READ+0x6e0, load) (RE-6)	inferred
4	`wait` event numbers 0..5 pair with `FMC_STATUS` bits 1, 2, 20, 3, `ECC_DONE` bits 0, 23	flag acknowledged after each wait in every program	inferred
5	Flags are sticky (they stay set until acknowledged); stale flags let waits pass early	acknowledge after every wait in all nine programs; third-party driver comment and fix (RE-7, RE-12)	observed / 3p
6	`FMC_STATUS` bit 23 is set after every status poll, whether or not the chip is ready	third-party driver measurement (128 of 128)	3p
7	Of the 19 `wait` event 5 uses in the original programs, 13 test `FMC_NSTAT & 0x40` first and enter `wait` event 5 only when that bit is clear (a five-instruction shape); the 6 uses in ERASE are arranged differently; READ and PROGRAM do not re-test after the wait	`osos mem:0x8009268` (READ+0x238), `mem:0x800a3c8` (WRITE+0x530) (RE-7)	observed
8	ERASE waits for the chip in up to three tries. Before the first two `wait` event 5 it sets `C58` = 0, so that a timeout continues at the next instruction instead of the fault vector; after each of them it reloads the timer (`C14` = `0xff`), acknowledges the fault flag (`CS_IRQ` = 4) and tests `FMC_NSTAT` bit 6 (ready) again. The third `wait` event 5 runs with `C58` = 4, so a third timeout goes to the fault handler	`osos mem:0x8008d30` (ERASE+0x100) (RE-8)	observed (code) / inferred (timeout survivable)
9	The prologue (the first instructions) of every program writes `CS_IRQ` = `0xff`, `C10` = 1, `C58` = 4, `C4C` = the fault vector address, `FMCTRL1` = `0x0ff3f8e0` and `FMC_STATUS` = `0x0ff00ffe` (the third-party driver writes the same)	first 0x30 bytes of every program (RE-5)	observed / 3p
10	`FMC_CTRL0` is written as `0x801` ORed with the timing word; the chip-enable bit is `1 << (CE+1)`; a chip-enable change is a read-modify-write that masks with `0xfffffe01`; bit 24 is set around the DMA phases	`osos mem:0x8009060` (READ+0x30), `mem:0x8009370` (READ+0x340), `mem:0x8009f28` (WRITE+0x90) (RE-9)	observed
11	The original READID writes the `FMC_CTRL0` control word `0x33201` (bit 9 set, bit 11 clear), the third-party driver `0x43803`; both read the ID	`osos mem:0x8009cd0` (READID+0x48); third-party driver comment (RE-10)	observed / 3p
12	`FMCTRL1` codes and `FMC_DATA_SEL` field layout of chapter 9, section 9.2 and section 9.3	counts over all nine programs; third-party driver (RE-11)	observed / 3p
13	ECC protocol in READ: write `ECC_DONE` = `0x1ff` and `ECC_MODE` = `0x180`, kick, `wait` event 4, write `ECC_DONE` = 1, read `ECC_RESULT`	`osos mem:0x80093b0` (READ+0x380..) ; 36 kicks (RE-12)	observed
14	Kick fields: bit 0 decode, bit 1 encode, bits 9:8 data buffer, bits 13:12 parity buffer, bits 17:16 slot	8 distinct kick words; third-party driver builds the word from buffer and slot numbers (RE-12)	inferred / 3p

#	claim	evidence	class
15	Page verdict: ECC_RESULT bit 0 sets bit 30 of the page word; the count (ECC_RESULT >> 16) & 0xf = n sets bit n - 1	osos mem:0x8009408 (READ+0x3d8..0x440) (RE-13)	observed
16	Decode corrects the data buffer in place; DMA follows the decode	READ order kick, wait event 4, DMA; third-party driver kicks then DMAs (RE-14)	observed / 3p
17	The stream on the chip is [16-byte chunk][512 data bytes] four times per 2 KiB unit; a 4 KiB page is two units in one stream; WRITE sends the parity slot (the chunk) first	FMC_DNUM = 0xf then 0x1ff in READ+0x2e8..; 0x34 then 0x2e4 in WRITE+0x268 (RE-15)	observed
18	The chunk is 13 B parity plus a 3-byte meta fragment; FMC_METACFG = a list of 4-bit slot numbers (nibbles)	0x3210 in the original programs, 0x5140 for the third-party single-chunk layout; 12-byte window +0x60..0x68; 4-bit count (RE-15, RE-16)	inferred
19	The metadata encode runs once per page before the first sector's encode; the metadata decode once per page after the last sector's decode	osos mem:0x8009ef0..0x800a018 (WRITE+0x58..0x180), mem:0x8009800 (READ+0x7d0) (RE-16)	observed
20	Both 2 KiB units of a 4 KiB page carry the same metadata bytes (3 per chunk)	write path encodes metadata once per page; fullpage: 42 of 42 pages written by the original firmware	device
21	READ batches: all 30h of a batch (distinct CEs) are issued before the transfers; the batch ends at the first repeated CE; results in request order	osos mem:0x8009088 (READ+0x58..0x158), mem:0x8009190 (READ+0x160) (RE-17)	observed
22	The page word is replaced by 0x20000000 when FMC_STATUS bit 27 stayed clear; the last sector's verdict is added afterwards; bit 27 is acknowledged at page end	osos mem:0x80096d0 (READ+0x6a0), mem:0x8009a98 (READ+0xa68) (RE-18)	observed
23	FMC_STATUS bit 27 = "non-blank chunk read": every programmed page sets it, every erased page does not (measured on a 4 GB unit)	third-party driver's page-read classification and its measurement note	3p
24	DMA address is written once per unit and advances 512 per sector; FMC_DMA_CTRL = 7 before the last DMA	osos mem:0x80093b0 (READ+0x380), mem:0x8009b38 (READ+0xb08), mem:0x800a658 (WRITE+0x7c0); third-party driver (RE-19)	observed / 3p
25	WRITE reads the DMA list one entry ahead; total entries read = pages x units	WRITE+0x468.. (RE-20)	observed
26	After 11h the chip is polled (70h status read, ready check, FAIL check) before 81h	osos mem:0x800b220 (WRITE2P+0x5e8), mem:0x800b3a8 (WRITE2P+0x770) (RE-21)	observed
27	A program failure (FMC_NSTAT bit 0 set after the status poll) ends the run at once: r6 bits 7:0 = the mask of the chip enables selected at that moment, and r6 bit 28 (the failure bit of chapter 12, section 12.2) is set	osos mem:0x800a788 (WRITE+0x8f0) (RE-22)	observed
28	ERASE issues 60 / A3 / D0 per entry without waiting, then a status sweep with the staged ready wait; r6 on normal end = mask of CEs used	osos mem:0x8008c88, mem:0x8008e78 (ERASE+0x58, +0x248) (RE-23)	observed
29	READID resets all eight CEs together, loops P_CEMAP times, reads 8 ID bytes through the FIFO, stores the first word	osos mem:0x8009cd0, mem:0x8009d20, mem:0x8009d48 (RE-24)	observed
30	CPU driver protocol: FMSS_PROG, CS_IRQ = 0xff, start word FMSS_CTRL = 0xfff5, a 2 s bound on the wait, sequencer reset by writing 0x30000	aupd mem:0x08010c40..0x08010d0c (RE-31, RE-32, RE-33)	observed
31	Timing word: setup at bit 16, hold at bit 12, EDO at bit 28, with the formula of chapter 12, section 12.3	aupd mem:0x08010734 (RE-35)	observed (formula) / inferred (clock)
32	READID reads FMC_FIFO twice; the third-party driver reads once or twice and does not need the second	osos mem:0x8009e00 (READID+0x178); third-party driver (RE-24)	observed / 3p
33	The fault handlers write the result word r6: bits 7:0 = the mask of the selected chip enables. READ, READID and RAWREAD always set bit 27. PROGRAM and ERASE set bit 27 only if FMSS_C50 bit 0 is set, and always set bit 28	osos mem:0x8009c40 (READ+0xc10), mem:0x800a788 (WRITE+0x8f0), mem:0x8008fc8 (ERASE+0x398) (RE-25)	observed

#	claim	evidence	class
34	RAWREAD (no ECC) loads FMC_DATA_SEL = 0x102 in all steps; not required by the original CPU driver	osos mem:0x800aa50 (RE-26)	observed; meaning open
35	78h variants test FMC_NSTAT & 3 and use three row cycles; WRITECACHE confirms 15h while pages remain beyond a group and finishes with 71h	WRITE_78+0x6b0, WRITECACHE+0x5e8 (RE-27)	observed
36	FMCTRL1 takes one operation at a time: 0x100 written and immediately replaced by 0xe2 loses the DMA (the DMA flag that wait event 2 waits for never comes, and the sequencer times out at that wait)	device: PC sampled at the hanging wait event 2 for about 10 ms, one 512-byte sector in memory, CS_IRQ = 5, r6 = 0x08000001	device
37	The 78h write forms of release 1.1.3 sweep four rows per CE with 78h + row and mask 3 (releases before 1.1.3: one 70h, mask 1); every other program is unchanged from 1.0.1 to 1.1.3	osos 1.1.3 file+0xd1c8..0xd470 (WRITE_78 sweep) (RE-28)	observed
38	r0..r7 are 0xc18 + 4n (r6 = SEQ_RESULT); a CPU write before the start is the starting value; a start keeps them, the reset clears them	test programs with marker values, register windows read before and after	device
39	About 216 ns per instruction	two-instruction loop at 2 000 and 20 000 iterations	device
40	ALU: shr logical; shift counts >= 32 give 0; 32-bit wrap; immediate 0 selects the two-operand form; ld ANDs its mask	edge-case programs, two runs	device
41	ldi/sti little-endian, unaligned address rounded down; pld/pst offset wraps at 12 bits and reaches r0..r7	scratch memory and window tests	device
42	Parameter RAM is 0xd00..0xd7c; 0xd80..0xdfc read 0 and drop CPU and program writes	64-word write/read pattern from both sides	device
43	The wait timeout is about 4.87 ms, independent of C58; wait events 0..10 and 12..15 are not set at idle, wait event 11 is set	wait events 0..15 with nothing pending; C58 sweep 0..31 and larger values	device
44	C58 bit 2 = continue at the fault vector on a timeout or illegal-opcode fault (else the next instruction); bits 0 and 2 set C5C; C58 holds 8 bits	C58 sweep with a timed-out wait event 1 and with opcode 0x09 (wait2 condition 0, which times out)	device
45	C58 bit 0 at an END sets C5C and makes every later END run the fault vector once until the CPU rewrites C58/C4C/C5C; CPU writes alone do not create it	back-to-back runs without the clean state (C58, C4C, C5C written 0), then with it	device
46	C58 bit 1: in oneshot one end was diverted to the fault vector when the CPU wrote C58 = 2 and the program wrote C4C; with bit 1 alone no end was diverted in c58b1 (ten CPU write orders) nor in c58b2 (three programs in a row with the device job's clean-up switched off); with bits 1 and 2 (C58 = 6) the first end was diverted once in both c58b1 and c58b2	oneshot (four variants), c58b1, c58b2	device; condition open
47	Opcodes: 0x0f is an indirect jump (finding 53); 0x10 sets CS_IRQ bit 1; 0x12 stops the program (CS_IRQ bit 3, 0xc08 = 0xa, instruction held in 0xc40/0xc44) until the CPU writes the PC and a start word (finding 94); 0x15 toggles C14 (0xff -> 0x00 after about 4.87 ms, 0x00 -> 0xff at once); 0x08, 0xff no visible effect; 0x1a..0x1f, 0x20, 0x40, 0x80 stall about 4.87 ms then fault	each opcode with seeded registers and a control row (or), two immediates, two runs	device
48	C14 reads 0xff after power-on and ignores CPU and program writes	CPU preset and program wr of several values	device
49	After a reset the first START fetches instruction 0 (0xc40/0xc44) and executes it only if it is end; else FMSS_CTRL bit 18 stays set and nothing runs until a second START; accesses in between make no difference	reset/start variants with and without an END-only start, register accesses, late CPU writes of FMSS_CTRL	device

#	claim	evidence	class
50	FMSS_C50 bit 0 is set while a command flag is pending and clear otherwise; a timeout or an illegal opcode does not change it, and a write of 1 by the handler does not clear it	handler reads C50 before and after its acknowledge; c50fault	device
51	FMSS_C14 is the timeout timer: loaded 0xff by a waiting wait, device run 1, counts down (0xf6 mid-wait), 0xff after a timeout, frozen at the end, writes ignored; 0x15 waits for it from a non-zero value (full period, leaves 0) and reloads it from 0	program reads and snapshots of C14 around waits and 0x15	device
52	Opcode map: 0x1a..0xfe fault after about 4.87 ms with no other effect; 0x09 (wait2) and 0x16 (evjmp) time out only when their condition or event does not come (findings 84 to 88, 90 to 92, 120); the full opcode byte is decoded; 0x08/0xff cost the same time as movi	all 256 opcodes x three operand forms with seeded registers, parameter RAM and RAM; 1 000-execution timing loops	device
53	0x0f = jump to the byte offset in r[arg] (the offset is not rounded; an arg of 8 or more gave r0, tested with 9 and 0xd70); immediate ignored; no return address kept	marker programs (six cases) and the pointer runs	device; "arg >= 8 is r0" inferred from two values
54	Start word: bits 1, 3 and 16 each keep the program from running (bit 18 set); bits 2, 4..15, 17..31 have no visible effect; bits 15:0 read back	each bit alone with bit 0 on a timed loop	device
55	CPU write masks of the sequencer, NAND controller (FMC), ECC and 0x400 words as in chapter 16, section 16.2; 0xc38 (PC) is read-only; 0xc3c, 0xc48 (32 bits) and 0xc54 (12 bits) are writable words without a known use	ones / zeros / 0xa5a5a5a5 written by the CPU, read back before the start	device
56	Offset bit 9 is not decoded in the register window; 0x400 is a second register set with the same layout; 0x100..0x1fc is a separate block mirrored four times; 0x800 reads 0x20060808	sequencer pld dump of 0x000..0xffc; CPU write masks	device
57	Event 11 is not the ready line (passes during an erase)	wait event 11 right after the erase confirm, CE0/CE1/none selected	device
58	wait event 5 passes when the polled chip becomes ready (about 155 us into an erase); the rule for when a latch can set it is in findings 121 and 135	status poll during an erase and idle; eight re-arms without 70h	device
59	A DMA to 0x50000000 never finishes (its wait times out); an ldi from the same address does not fault (CS_IRQ bit 4 was seen then: see finding 65)	c50x cases	device
60	FMC_28 advances by one per pld+sti loop iteration (128 over 128)	sequencer dump reading 0x028 and 0x228 128 iterations apart	device; what advances it: finding 155
61	Start-word bit 1 pauses before instruction 0 until a start without it; bits 3 and 16 give the post-reset hold; no single-step; bit 17 does not clear registers	ctrlhold: ten start/late-write combinations, register preload	device
62	Only wait events 0, 11 and 15 passed in the five NAND states (wait event 15: finding 75)	wait events 0..15 x five states, no status read before the wait	device
63	A sequencer read of FMC_STATUS/FMC_NSTAT does not consume an event; a wait event 5 placed straight after "70h, latch" timed out once although bit 23 read set afterwards (open)	evclear (eight programs) and evsweep, device run 11	device
64	C10, C3C, C48, C54 do not change the timeout, a wait, a DMA or an ECC encode; C3C reads 0 after a run; inside a handler it holds the offset of the faulting instruction + 8 (faults at 0x28, 0x40, 0x70, and a second fault inside the handler)	timeregs (fifteen settings), ctrlwords, c50fault	device
65	CS_IRQ bits 6:4 are NAND controller and ECC interrupt sources gated by 0x044 and ECC 0x844; bit 6 = ECC_DONE pending (406 of 406 runs); they appeared only after an experiment left 0x044 = 0xa5a5, 0x844 = 0x25	every run before and after that experiment	device; bits 4 and 5 sources open

#	claim	evidence	class
66	`wait` event 5 is `FMC_STATUS` bit 23 as a level; after a latch a new command is refused until `FMCTRL1 = 0x20` (after the command write `FMC_STATUS` bit 1 stays clear and bit 26 is set); a latch after `90h` sets no bit 23 and leaves `0x90` in `FMC_NSTAT`; when a latch sets bit 23: findings 121 and 135	`ev5rule` (eight orders); the bit 23-level rule fits all 8 orders	device
67	`C08` = 1 running, 4 waiting or stalled (illegal opcode = a wait that never passes), 3 paused, `0xa` blocked, 0 held/idle	CPU reads 1 ms into each state	device
68	Start-word bit 1 pauses a running program exactly and a later start resumes it; bit 3/16 while running aborts to the post-reset hold, the release restarts from 0; a plain start while running is ignored	`pause`, seven cases with two timed CPU writes	device
69	Register numbers 8..255 in every role: as a destination only 10 acts (all of r0..r7); 9 and 11..255 change no register and no window word; in the argument field 9 and 11..255 read r0, 10 reads the destination register in the immediate form and r0 in the register form; `st` with any of 8..255 stores the last value that passed through the register file (the register the previous instruction wrote, or the one it read: r5 after `st [..], r5`, r0 after `nop` or `wr`; `wait` leaves it); `bnz` and `bz` with any of 8..255 never branch; `jr` with 9..255 jumps to r0; `ldi` and `pld` with 9..255 use r0 as the address	`regnum` (19 programs), `regsweep` (every number, every role), `stprev`	device
70	`bz`/`bnz` ignore arg, test 32 bits, take unaligned targets as given; `wait`, device run 1 ignores arg/imm; event numbers > 15 never pass	`branch`, `waitf`	device
71	Instruction time is fetch-bound: 217 ns (SDRAM) / 138 ns (SRAM); window access +35..37 ns; `ldi`/`sti` +120..218 ns	`opcost`	device
72	Page read: `wait` events address 1, command 0, transfers 3 (512 B in 49 us), ECC 4 (`wait` event 15: finding 75)	`readev`, device run 15	device; the DMA step never finished
73	The `0x400` set answers no READ ID and raises no event with only channel 0's pins configured; four chip enables on the `0x000` set answer `ec d5 14 b6 74`	`chan1`	device
74	Block `0x100` has writable words at `0x1b8..0x1d8` (values `0xffffffff`, `0x008060`, `0x3010d0`)	`blk100`	device; meaning open
75	Events are levels, not consumed; event 4 = any `ECC_DONE` bit but bit 4; events 6, 7, 9 = the second set's bits 1, 2, 3 and event 8 its bit 20; `wait` event 15 did not pass with every `0x044` / `0x844` enable on and a command flag pending, nor with each of `0x044` bits 0, 1, 2, 11, 12 and its own flag pending	`evcons`, `ev8`, `state`, device run 17 idle events, `evsweep2`, `irqop`, device run 37	device; event 15 open
76	Only enabled NAND controller and ECC sources reach `CS_IRQ` bits 4 and 6: `0x044` bit 0 routes `FMC_STATUS` bit 1 (command sent) into `CS_IRQ` bit 4, and `0x844` bits 0, 2 and 5 each route `ECC_DONE` into `CS_IRQ` bit 6. END, faults, `sig` (`10`) and `pause` (`12`) never raised the CPU line, and start-word bit 6 made no difference	`irqline`, device run 15, `irqen`, device run 16 on clean state	device
77	r8 is the PC (read = current offset, write = jump); a write to r10 sets r0..r7	`r8`, device run 16	device
78	Power-on values of every window (see chapter 16, section 16.2); a soft reset of the writable words leaves status words and `C14` different	`state`, device run 17 after a power cycle, `restore`, device run 16 before it	device
79	The second set's controller runs a READ ID (command, address, 8-byte transfer counted in `0x424`) with no chip answering	`ev8` window snapshots	device

#	claim	evidence	class
80	A soft reset of the writable words (to the power-on values) plus status acknowledges gives power-on behaviour; 7 words keep last-written action values or ignore writes	`restore`, device run 16 after a polluted state, compared with `state`, device run 17 after a power cycle	device
81	`FMC_CTRL0` bit 24 enables DMA: without it nothing moves (both sets, both directions), with it the loopback returns the data intact	`dmaloop`, device run 20 with and without bit 24, `readev`, device run 15	device
82	The second register set has its own data buffer and DMA engine; event 8 is its DMA flag	`dmaloop`, device run 20 through `0x400` with `wait` event 8 / `wait` event 2	device
83	Page read per sector: NAND transfer 512 B in 49 us, DMA to RAM 512 B in 14 us, command to RAM 123 us	`readev`, device run 15 with bit 24	device
84	Opcode `16` is conditional: timed out in the sweep (near power-on state), went on at once in every operand form after DMA / ECC / page-read runs (see 86); `09` timed out in all sixteen forms, including register-and-mask conditions that held	`op0916` against the `opsweep` records	device
85	`C50` bits 5:0 = events 0..5 pending (all six seen, finding 102); bit 0 = a command flag pending (finding 50)	`op16`, `irqsrc`, `c50fault`	device
86	Opcode `16` (`evjmp`) goes to the fault vector at once while any event is pending, else times out and faults	`op16`: one pending flag at a time from a clean state, the vector marker reached in about 20 us	device
87	The status latch's `wait` event 5 timed out in one of two identical runs	`op16` status cases	device; intermittent, cause open
88	Opcode `16` (`evjmp`) waits for an event that arrives during the wait; its jump target is `C4C`; `C58` is irrelevant; the flag stays set; flags of the second register set count	`op16b`, `opstate`	device
89	`08`, `10`, `15`, `ff` do not react to pending flags; `12` blocks in every state	`opstate` (seven opcodes x eight states)	device
90	`09` (`wait2`) continued with condition 5 (event field 5) while the status flag was pending; timed out with other flags / event numbers / masks alone; otherwise identical to `1a`, `20` and an unmet wait	`op09`, repeated with identical outcomes	device; the list of conditions: findings 91, 92, 120 and 147
91	`09` ignores `arg` / `imm`; only event field 5 passes (0..15 tried with every flag pending); it passes with the status flag acknowledged and while the chip is busy (no wait for ready)	`op09b`	device
92	From a clean state `wait2` passes only for condition 5 of the conditions 0..31 tried (no chip / CE0 / CE1 selected), as `wait`, device run 1 only for event 11; condition 33 is outside that range (120)	`op09c`	device
93	The `0x400` set is channel 1 of the NAND controller with its own GPIO pins (ports 10 upper, 11, 12, 7 pin 4); the boot ROM can boot from channel 0 or 1; in the device tests only channel 0's pins were configured	`bootrom mem:0x20002e0c`, `0x20001830`, its constants (RE-43)	observed (ROM)
94	Opcode `12` is `pause`: the CPU continues it by writing the PC (normally the offset of the pause instruction + 8) and a start word; the CPU can write the PC only then	`op12cont` (seven write sequences)	device
95	The PC (`0xc38`, = r8) is writable by the program (a jump)	`pcwrite`	device
96	Start-word bit 2 enables the wait timeout; without it waits and illegal opcodes wait for ever; the length is set by bits 15:8 and 7:4 (finding 111)	`ctrlfx`	device
97	Window offsets are 12 bits; branch / r8 targets use bits 24:12 and ignore bit 31; `jr` with a target above `0xfff` does not jump	`fwidth`	device

#	claim	evidence	class
98	The 13 parity bytes are visible in `0x900..0x90c` after an encode and the parity slot can be DMA-ed out; zero data gives non-zero parity; `0x9a4` counts decodes	`eccpar`	device; explained by finding 109: the three `0xff` metadata bytes are part of the code word
99	A DMA always moves 512 bytes; `FMC_DNUM` sets NAND transfer lengths only	`bufsize`	device
100	No chip answers on channel 1 of the NAND controller of this board with its pins switched	`chan1pins`	device; "channel 0 only" inferred from one board
101	`wait2`'s conditions are not the second register set's flags	`op09d`	device
102	`C50` bits 5:0 each follow their event's flag (all six seen); `0x044` bits 0, 1, 2, 11 and 12 route flags into `CS_IRQ` bit 4 (finding 150); VIC0 line 20 seen with NAND controller enables, VIC1 line 22 never	`irqsrc`, `irq044`	device
103	Read / write matrix of the whole window from both sides (chapter 3, section 3.2); mirrors confirmed by writes; `0x940..0x950` read/write; `C3C`/`C48`/`C54` cleared at a start	`access`	device
104	A status latch raised `FMC_STATUS` bit 24 instead of bit 23 (`wait` event 5 never came) in every run with `0x044` or `0x844` written (eighteen runs), in none without	`irqsrc` status cases	device; cause open
105	`end` faults when `R` or `arg` is non-zero; its `imm` is ignored; the END's word and `imm` are latched in `0xc40` / `0xc44`	`fields0`	device
106	The fields the original programs leave 0 (`imm` of `st`/`ldi`/`mov`/`sti`/`pld`/`pst`, `R` of `wr`, `arg` of `movi`) are ignored	`fields0`	device
107	Timeout: start-word bits 7:4 = 0 make it immediate; with bits 7:4 = `0xf` the length grows with bits 15:8 (16 / 310 / 4868 us for `0x00` / `0x0f` / `0xff`); the step per bits 7:4 is in finding 111	`tmofield`	device
108	The ECC corrects up to 8 data bits per sector; 9..12 are reported uncorrectable (`ECC_RESULT 0x11`) and the data is left as sent	`eccstr2`	device
109	The ECC code is binary BCH t = 8 over GF(2^13) (`p = 0x201b`, generator chapter 10, section 10.3), message = 512 data bytes + 3 slot metadata bytes, MSB first; parity in slot bytes 3..15	`eccstr2`, `eccpar` parities	device data, matched offline (twelve of twelve)
110	The `FMC_STATUS` bit 24-instead-of-bit 23 failure (finding 104) did not recur in fifteen runs with and without interrupt enables: the enables are not shown to cause it	`ev5en`	device; cause open
111	Timeout length = (start-word bits 15:8 + 1) x 2^s bus cycles, s = 3, 5, 7, 9, 10 for bits 7:4 = 1..5 and 11 from 6 up (0: immediate); the steps were measured, n = 5 does not follow the pattern of n = 1..4	`tmofield`, `tmobits`, `tmobits2`	device
112	Pages written by the original firmware are code words of the identified BCH code with their metadata in slot bytes 0..2	`rawpage`	device (20 sectors)
113	The metadata path writes `FMC_META0..2` into slots 0..3, bytes in order; the encode includes them	`eccchunk`	device
114	A parity buffer is 64 bytes; a 512-byte DMA into it wraps and leaves the source's last 64 bytes	`eccchunk`	device
115	`FMC_DNUM` limits a write from a data buffer: a raw page with host-built chunks is written without the ECC engine	`rawwrite`	device
116	Errors stored on the flash: 8 data bits corrected, 9 uncorrectable, 2 metadata bits corrected and counted, an all-zero chunk uncorrectable	`rawwrite`	device

#	claim	evidence	class
117	FMC_STATUS bit 27 = the sector read was written (set for every written sector, clean or not; clear for erased ones)	rawpage, rawwrite	device
118	ECC_MODE bit 8 enables the slot path; bits 0, 6, 7, 9, 10 make no difference with it; the modes without bit 8 are 144	eccmode	device
119	ECC_DONE reads 0x33 after an encode, 0x35 after a decode, 0x3d after an uncorrectable one; bit 3 stays set after an uncorrectable decode (finding 139)	eccmode, eccstr2	device
120	No wait2 condition N in 0..255 follows a chip's erase, program or read; wait2 condition 33 passed while the chip was idle in op09w33 and during an ECC operation and a DMA in finale; it did not pass in op09nand (idle and during an erase), evsweep2, ctrlwords, w2state, blk100b or w2vic	op09nand, op09w33, finale, evsweep2, ctrlwords, w2state, blk100b, w2vic	device; condition 33 is VIC1 line 1: finding 174
121	wait event 5 (FMC_STATUS bit 23) is set by a status latch only while armed, and the latch uses the arm; FMCTRL1 operations re-arm it (set-up word, 0x100e0, chip transfer, DMA), 0x20 / status-clear / CE toggle / ECC kick do not	ev5ready, ev5arm	device; re-arm by the chip becoming ready: 135
122	Single-page program on CE0 / CE1, two CEs in one run, two-plane program (80h/11h/81h/10h) and erase (60h/60h/D0h), erase on CE1: correct by raw read-back	writeseq, eraseseq	device
123	With the status poll on FMC_DATA_SEL = 0x100 the latch returns 0x70 (the last command byte, whose bit 6 reads as "ready"), so the poll ends while the chip is still programming and the next page's 80h is ignored: that page stays erased. Pages that stayed erased: 11 and 13 in a run writing pages 10..13; 1, 3, 5, 6 and 7 in a run writing pages 0..7; 5 in a run writing pages 4 and 5; 3 in a run writing pages 1 and 3. A run writing a single page wrote it correctly	writeseq, pageorder	device
124	Reprogramming a page without an erase stores old AND new (its status byte was not read correctly, finding 123)	writeseq	device
125	ECC_MODE bits 20:16 = number of data bytes encoded (1, 2, 4, 8, 16 matched exactly; the rest counted as 0xff, same code, metadata last)	eccmode2	device data, matched offline
126	Metadata and parity-byte errors are corrected and counted with data errors, up to 8 in the whole code word	eccchunk2	device
127	An erase sent with 4 address cycles erases the block	pageorder	device
128	With FMC_DATA_SEL = 0x100 a status latch returns the last command byte (0x70)	writeseq, pageorder	device
129	Multi-page program runs with the status poll on FMC_DATA_SEL = 1 write every page (eight in order with 0x2e4 / FMC_DMA_CTRL = 7, pairs 9/11 and 1/3)	writeseq2	device
130	Status byte 0xc0 when ready, 0x81 right after 10h; reprogramming without an erase reports no FAIL	writeseq2	device
131	Program time follows the page pairing of the chip: pages 0..3, 6, 7, 10, 11, ..., 122, 123 take about 478 us per run (ready poll included), pages 4, 5, 8, 9, ..., 124, 125 and 126, 127 take 728 to 873 us; the same pattern in three blocks on CE0 and CE1; erase about 1.1 ms with data, about 0.18 ms blank	progtime, eraseseq	device; from run times
132	The ECC_MODE length field is bits 20:16 only (1..31; 0 = 512); a decode with it checks only the first L bytes	eccmode3	device

#	claim	evidence	class
133	ECC_MODE bit 15 code: Reed-Solomon over GF(2^10) (x^10 + x^3 + 1), byte symbols, generator with roots a^0..a^7, 10 parity bytes, no metadata (43 of 43 sectors)	eccmode2, eccmode3, b15ecc	device data, matched offline
134	The 8-symbol Reed-Solomon code corrects up to 4 bad bytes and counts bytes; 5 or more are uncorrectable	b15ecc	device
135	A chip's busy-to-ready transition re-arms wait event 5 (two-chip polls in both orders); address cycles re-arm it	ev5two	device
136	ECC_MODE bit 15 with the length field (bits 20:16) L = 31: Reed-Solomon over the first 31 bytes, rest 0xff; ECC_MODE bits 2, 3, 4 change the result (bit 4 only for non-zero data), not by any simple data transform tried; ECC_MODE = 0x1ffffff gives the bit 4 variant	eccmode2, eccmode4	device data, checked offline; bits 4:2: 141 and 142
137	Modes without a code (bits 8 and 15 clear): the n-th encode writes the n-th value of a fixed sequence (chapter 10, section 10.12) into 0x900..0x90c and 0x990, whatever the data or mode; a decode does not advance it; slot untouched, every decode uncorrectable	eccmode2, eccmode4	device; generator open; all 61 values: 144
138	With ECC_MODE bit 15 set, the ECC area bytes 0x90a..0x913 are the encoder's trace: the register's top symbol before each of the last 8 input symbols (quotient XOR message), 38 of 38 sectors; unchanged by a decode	b15ecc, eccmode2, eccmode4	device data, matched offline
139	ECC_DONE bit 3 = the last decode was uncorrectable, sticky across encodes, programs and the 0x1ff clear until the next good decode; bit 5 is cleared by any one of ECC_MODE bits 21..24 in every case of done5 and done5b, while in eccmode2 (encode only) bits 21, 22, 23 alone on 0x180 left it set	eccmode..eccmode4, b15ecc, done5, done5b	device; the encode-only series open
140	VIC0 line 20 was seen in exactly 4 of 102 irqsrc runs: a command with 0x044 = 0xffff or with 0x044 bit 0 set, a status latch with 0x044 = 0xffff or with 0x044 bit 13 set; never for address, transfer or DMA sources (which also reach CS_IRQ bit 4); VIC0 bit 5 and VIC1 bit 1 were set in every run	irqsrc	device; per-source test in finale
141	ECC_MODE bit 4 = 32-bit byte swap of the data read and the slot write (BCH and RS; 10 of 10 samples each)	finale	device data, matched offline
142	ECC_MODE bits 3:2 with bit 15 = Reed-Solomon strength: 8 / 12 / 16 parity symbols (10 / 15 / 20 bytes; 3 = as 0), generators with roots a^0.. (10 of 10 samples each); 12 symbols correct 6 bad bytes (7 uncorrectable), 16 symbols correct 8 (9 uncorrectable)	finale, rsdecode, rschip	device; codes matched offline
143	No mode corrects more than 8 scattered bits per 512-byte sector; Reed-Solomon with 16 parity symbols (RS-16) corrects 8 bad bytes, any number of bits inside each	eccstr2, b15ecc, rsdecode	device
144	The sequence of the modes without a code: 61 distinct values listed in chapter 10, section 10.12, no short LFSR (linear complexity about half the length); within a device run nothing tried restarts it (soft reset, sequencer reset, writes of ECC_MODE / ECC_KICK / ECC_DONE / 0x940..0x950 / FMC_CTRL0, BCH encodes and decodes); the first such encode of a device run gave value 1 in eccmode4 and in eccregs, value 14 in finale, and value 2 in seqfirst, run right after a power cycle	eccmode4, finale, seqrestart, eccregs, seqfirst	device; restart event open

#	claim	evidence	class
145	`FMC_28` stays 0 after the set-up word through instructions, `sti`, `ldi`, DMAs, ECC operations and chip reads; `0x1cc` / `0x1d0` are not NAND timing	`finale`	device
146	VIC0 line 20 per source (one at a time): a command with `0x044` bit 0 and an address with `0x044` bit 1 only, not the same set as `CS_IRQ` bit 4 and not the same as in `irqsrc`; VIC1 line 22 never	`finale`, `irqsrc`	device; condition open
147	`wait2`: only conditions 5 and 33 passed while an ECC operation or a DMA ran (N = 0..63)	`finale`	device
148	Reed-Solomon ignores the slot field of `ECC_KICK`: the encode writes its parity at byte 0 of the parity buffer (RS-16: bytes 0..19, slot 0 and bytes 0..3 of slot 1; RS-12: bytes 0..14), also when the kick names slot 3, and the decode reads it from there; the ECC area holds the parity reversed (20 / 15 bytes)	`rsdecode`, `finale`, `rschip`	device
149	`0x990` after an encode: 0 with BCH; a data-dependent 16-bit value repeated in both halves with RS (zeros 0, ones `fc36`, the ramp `c68b`, the same for 8 and 16 symbols); the sequence value without a code; `0x940..0x950` change neither parity nor decode	`eccregs`	device; meaning of `0x990` open
150	`0x044` routes one `FMC_STATUS` flag per bit into `CS_IRQ` bit 4 and VIC0 line 20: bit 0 command sent, bit 1 address sent, bit 2 data transfer done, bit 11 DMA done, bit 12 status latched (tested with one operation per program, its flag left pending); `CS_IRQ` bit 4 stays set after the flag is acknowledged, VIC0 line 20 only while the flag is pending	`irq044`, `vic20`, `irqop`, device run 37	device
151	`C14` reloads with start-word bits 15:8, not `0xff`: after a run with waits it read `0xfe`, `0xfd`, `0xfb`, `0xf7`, `0xef`, `0xdf`, `0xbf`, `0x7f` with start-word bit 8, 9, ..., 15 cleared respectively	`startbits`	device
152	In `evsweep2`, over the listed states, `wait2` passed only for condition 5 in every state (idle, erase, program and read busy, ECC and DMA in flight, every enable on with a command flag pending, second-set command), and also for condition 20 with every enable on and a command flag pending; `wait` events passed: event 11 always, event 4 with an ECC operation, event 2 with a DMA, event 0 with a command flag, event 6 with a command to the second register set	`evsweep2`, `w2state`	device
153	Start-word bit 6 and bits 17..31 change nothing in program fetch from SDRAM or SRAM, 2000 `ldi` / `sti` / `pld`, `sig`, a DMA with an ECC encode or a vectored fault (0xfff5 against 0xffb5 and 0xffd5 against 0xff95 also in a reserved-word read / write program); bit 6 also changes nothing in reads, erases and programs of user blocks and of blocks above the user area on CE0 and CE1; clearing bit 4, 5 or 7 leaves the timeout unchanged, because the step field n (bits 7:4) stays at 6 or more, where the step length no longer changes	`startbits`, `startdiff`, `startregion`	device
154	Program fetch from internal SRAM is faster than from SDRAM: the same loop takes 497 us against 811..828 us	`startbits`	device

#	claim	evidence	class
155	FMC_28 is the read position of the FIFO port: each read of FMC_FIFO (0x080) advances it by 4, wrapping at 512; it does not change while a program runs without reading 0x080, except that a DMA and the set-up word set it to 0; it advanced by 248 between the CPU's dump before a run and the start and by 124 between the end and the CPU's dump after the run	fmc28b, bufport	device
156	A status latch passed 160 of 160 times (with and without a fresh arm, with and without 0x044 / 0x844 written), FMC_STATUS bit 24 never set; a second latch without an FMCTRL1 write between timed out 40 of 40 although its own 70h was sent; wait event 3 on CE0 passed 40 of 40	latchrep	device
157	Read by the program itself, CS_STATUS is 5 in every state tried (after starting a DMA, an ECC operation or a chip command, after sig, in a fault handler)	c08self	device
158	On a real page read FMC_CTRL0 bits 9, 11 and 24 change nothing (data, decode results, READ ID); FMC_DMA_CTRL bit 0 or bit 3 alone gives correct data, bit 1 or 2 makes every DMA after the first time out (read back with bit 6 / 7 added), bits 4..7 make every DMA time out, and its value persists between runs; with FMC_DMA_CTRL = 0, 1 or 8 the drain select 0x1e0 and the alternating combined selects 0x201 / 0x102 deliver correct data	fmcbits, dmactl	device
159	FMC_STATUS bits 12, 13, 18, 19 are set after a soft reset and are not cleared by writing 1; a 16-byte data read clears bits 13 and 19 and a 512-byte data read clears all four; a DMA into a data or parity buffer, a status latch and the set-up word set 13 and 19 again	fmcstat, fmcstat2	device
160	Of the nine programs only WRITECACHE reads the bank count (0xd04): it stores 0 minus it in 0xd44 and confirms a page with 15h instead of 10h while the remaining page count (0xd18) AND that value is non-zero	RE-46	observed
161	wait2 condition 20 is true while a command flag (FMC_STATUS bit 1) is pending and 0x044 bit 0 is set; any other 0x044 bit or 0x844 = 0x2f with the command flag pending leaves it false; wait2 condition 33 was false in all 34 states tried, the first program after a power cycle included	w2state	device; condition N = interrupt source N: finding 174
162	C58 = 6 (bits 1 and 2) with C4C written by the CPU diverts the first end to the fault vector once; C58 = 3 diverts every end after the first (bit 0); C58 = 2 diverted none in three runs	c58b2	device
163	wait event 8 passed during a second-set DMA and after a page read of CE0 through the first register set; the first program after a power cycle passed wait event 4	evsweep3, w2state	device; the page-read case open
164	The FIFO port FMC_FIFO returned 0 in 130 reads from data buffer 0 or 1 or parity buffer 0 filled by DMA (with and without FMCTRL1 = 0x340); 128 writes to it after FMCTRL1 = 0x340 changed 26 of 128 words of data buffer 0	bufport	device

#	claim	evidence	class
165	Reed-Solomon on the chip: pages written by the engine store the slot as it was (RS-8: 10 parity bytes and six `0xff`; RS-12: 15 and one `0xff`); because the slot field is ignored, only sector 0 of a page written with slots 0..3 carried its parity; stored bad bytes in sector 0 decoded as in the buffers (RS-8: 4 corrected, 5 not; RS-12: 6 corrected, 7 not)	`rschip`	device
166	An encode in the same code as the last decode leaves `ECC_RESULT` unchanged; a Reed-Solomon encode after a BCH decode set it to 0, a BCH encode after a Reed-Solomon decode set it to `0x30000`	`rsresult`	device
167	In the first version of `irqop`, programs that ended with a NAND controller flag pending and its `0x044` enable set stopped the device from answering USB; in the second version, with every flag acknowledged and `0x044` / `0x844` cleared before the `end`, the same operations ran without fault	`irqop`, device run 36, `irqop`, device run 37	device; cause open
168	No decode reported success with wrong data: 240 BCH sectors with 9..64 random bad bits and 240 RS-8 sectors with 5..40 random bad bytes were all reported uncorrectable	`miscorr`	device
169	The second register set has its own data buffers 0 and 1 and parity buffers 0 and 1 (patterns DMA-ed through it came back); parity buffer 0 is shared with the first set (after the second set's DMA into it, the first set's parity buffer 0 read the second set's pattern); the set-up word written to `0x404` left the first set's parity buffer 0 all `0xff`	`ch1buf`	device; sharing inferred
170	`C50` bit 6 = `wait` event 6 pending (set after a second-set command); `C50` bit 8 did not appear in 17 reads around soft resets, a sequencer reset, a fault, a second-set command, an ECC decode and `end`-only runs	`c50b8`	device
171	Four `tmwait` in one program alternate: from `C14` = `0xff` a full wait (about 4.85 ms) that leaves 0, from 0 a reload to `0xff` with no wait; the same in five runs (about 9.7 ms each)	`tmwait2`	device
172	Setting any writable word of the `0x100` block (to 0, 1 or `0xffffff`) changed neither the time of DMAs, ECC operations, a chip read and a timeout nor `wait2` 5 / 33; `0x3a8..0x3b4` read `0xffffffff` from the sequencer, also after the CPU and the sequencer wrote them, and 0 from the CPU	`blk100b`	device
173	On this unit the boot ROM's strap pins P3.6 and P3.7 read 0 and 0, the code for a boot from the SPI flash on SPI port 0 (RE-47)	`straps`	device / observed
174	`wait2` condition N is true while interrupt source N is raised in the CPU's interrupt controllers (VIC0 line N for N = 0..31, VIC1 line N - 32 for N = 32..63); numbers 64..255 never pass; 64 of 64 conditions matched the raw status in four states	`w2vic`	device
175	All NAND controller sources share interrupt source 20 (VIC0 line 20); the ECC engine's sources, enabled in `0x844`, raise interrupt source 43 (VIC1 line 11); sources 5 and 33 belong to devices outside the FMSS engine	`w2vic`	device

References: chapter 14 (outcomes), chapter 13 (what was done and why), chapter 15 (what the outcomes mean).

Related chapters: chapter 14, chapter 15, chapter 17.